
DATA SCIENCE

Série 1 — Volume I

Fondations

Mathématiques · Probabilités · Statistique
Programmation · Bases de données · Outils

Un livre pour apprendre le métier pas à pas,
avec une boutique d'artisanat tunisienne pour fil rouge.

Table des matières

Avant-propos	1
Bienvenue	1
À qui s'adresse ce livre ?	1
Comment ce livre est construit	2
Les encadrés	2
Le principe « essentiel / pour aller plus loin »	2
Trois parcours de lecture	2
Le fil rouge : la boutique Dar Jasmin	3
Le code de ce livre	4
La carte du volume	4
Faites le point : un petit test de départ	4
Rappel express : les bases à avoir en tête	6
R.1 Fractions et pourcentages	6
R.2 Puissances et racines	6
R.3 Équations et fonctions affines	7
R.4 Le symbole $\sum$ (somme)	7
R.5 Exponentielle et logarithme	8
R.6 Fonctions, graphes et équations de droites	8
R.7 Un peu de vocabulaire de programmation	8
Faites le point	9
Chapitre 1 : Mathématiques pour la data science	10
Pourquoi des mathématiques ?	10
Le chemin de ce chapitre	10
1.1 Algèbre linéaire	11
1.1.1 Vecteurs : décrire un client par une liste de nombres	11
1.1.2 Matrices : le tableau de données devient un objet mathématique	14
1.1.3 Valeurs propres et vecteurs propres : les directions privilégiées	19
1.1.4 La décomposition en valeurs singulières (SVD)	23
1.2 Analyse : mesurer le changement	26
1.2.1 La dérivée : la vitesse à laquelle une fonction change	26
1.2.2 Dérivées partielles et gradient : plusieurs variables à la fois	31
1.2.3 Intégrales : accumuler les petits changements	34
1.3 Optimisation : trouver le meilleur choix	36
1.3.1 Le problème d'optimisation	36
1.3.2 La convexité : quand il n'y a qu'une vallée	37
1.3.3 La descente de gradient	39
1.3.4 ■ Application : le prix qui maximise les recettes	42
1.3.5 Optimisation sous contraintes : les multiplicateurs de Lagrange	44
1.4 La fiche de notations	46

1.4.1 Conventions typographiques	46
1.4.2 Ensembles et logique	46
1.4.3 Sommes, produits, fonctions	47
1.4.4 Algèbre linéaire et analyse	47
1.4.5 L'alphabet grec en data science	48
1.4.6 Exemple : lire une formule du début à la fin	48
1.5 ⊕ Pour aller plus loin : analyse numérique	49
1.5.1 Pourquoi $0,1 + 0,2$ n'est pas égal à $0,3$	49
1.5.2 Les grands nombres avalent les petits	50
1.5.3 L'annulation catastrophique	50
1.5.4 Le conditionnement : quand un problème est « fragile »	51
1.5.5 Deux algorithmes classiques	52
1.6 ⊕ Pour aller plus loin : mathématiques discrètes et graphes	53
1.6.1 Ensembles : le langage de base	53
1.6.2 Compter : le principe multiplicatif	54
1.6.3 Les graphes : modéliser des relations	55
1.6.4 Parcourir un graphe : la recherche en largeur	56
1.7 Exercices du chapitre 1	57
Énoncés	57
Corrigés	58
Bilan du chapitre 1	60
Chapitre 2 : Probabilités	61
Le chemin de ce chapitre	61
2.1 Probabilités et formule de Bayes	62
2.1.1 Le vocabulaire : univers, événements, probabilité	62
2.1.2 Voir une probabilité : la simulation	62
2.1.3 Probabilité conditionnelle : changer d'information	63
2.1.4 Indépendance	64
2.1.5 La formule des probabilités totales	65
2.1.6 La formule de Bayes	65
2.1.7 Application : un mini-classifieur (le filtre naïf de Bayes)	67
2.1.8 Trois paradoxes pour s'entraîner à se méfier de l'intuition	67
2.2 Variables aléatoires et lois usuelles	69
2.2.1 Qu'est-ce qu'une variable aléatoire ?	69
2.2.2 Loi de Bernoulli : le oui/non	70
2.2.3 Loi binomiale : compter les succès	70
2.2.4 Loi de Poisson : compter des événements rares	72
2.2.5 Les variables continues : la densité	73
2.2.6 Quelle loi choisir ?	75
2.3 Espérance, variance, covariance	76
2.3.1 L'espérance : la moyenne « à long terme »	76
2.3.2 La variance : mesurer l'étalement	78
2.3.3 Covariance et corrélation : bouger ensemble	79
2.4 Loi des grands nombres et théorème central limite	82
2.4.1 La moyenne d'échantillon est elle-même une variable aléatoire	82
2.4.2 La loi des grands nombres	83
2.4.3 Le théorème central limite	84
2.4.4 Applications	85
2.4.5 Un mot de prudence	87
2.5 ⊕ Pour aller plus loin : la théorie de la mesure (ce que « probabilité » veut vraiment dire)	87

2.5.1 Le problème : on ne peut pas mesurer tous les ensembles	87
2.5.2 Le cadre : un espace probabilisé $(\Omega, \mathcal{F}, P)$	87
2.5.3 Une conséquence surprenante : événements de probabilité 0 qui arrivent . . .	88
2.5.4 L'espérance comme intégrale	88
2.5.5 Les modes de convergence	89
2.5.6 La densité comme dérivée d'une mesure	89
2.6 $\oplus$ Pour aller plus loin : les processus stochastiques	90
2.6.1 La marche aléatoire	90
2.6.2 Les chaînes de Markov : le futur ne dépend que du présent	91
2.6.3 Le processus de Poisson : des arrivées au hasard	93
2.7 Exercices du chapitre 2	94
Énoncés	94
Corrigés	95
Bilan du chapitre 2	97
Chapitre 3 : Statistique	98
Le chemin de ce chapitre	98
3.1 Statistique descriptive	99
3.1.1 Population, échantillon, variables	99
3.1.2 Charger et regarder le jeu de données	99
3.1.3 Mesures de position : où est le centre ?	100
3.1.4 Mesures de dispersion : de combien ça varie ?	101
3.1.5 La forme : histogrammes, asymétrie, boîtes à moustaches	102
3.1.6 Variables qualitatives et comparaison de groupes	103
3.1.7 Relier deux variables : corrélation, et pourquoi il faut toujours dessiner	104
3.2 Estimation	105
3.2.1 Un estimateur est une variable aléatoire	105
3.2.2 Qu'est-ce qu'un bon estimateur ?	106
3.2.3 Pourquoi divise-t-on par $n - 1$ pour la variance ?	106
3.2.4 La méthode des moments	107
3.2.5 Le maximum de vraisemblance	108
3.2.6 Pourquoi le maximum de vraisemblance est la référence	111
3.3 Intervalles de confiance	111
3.3.1 Construire un intervalle pour une moyenne	111
3.3.2 Que veut dire « 95 % de confiance » ?	112
3.3.3 Quand l'écart-type est inconnu : la loi de Student	114
3.3.4 Intervalle pour une proportion	115
3.3.5 Quand il n'y a pas de formule : le bootstrap	116
3.3.6 Dimensionner un échantillon	117
3.4 Tests d'hypothèses	117
3.4.1 Le vocabulaire et la méthode	118
3.4.2 Test de Student sur une moyenne	118
3.4.3 Comparer deux groupes : le test de Welch	119
3.4.4 Test sur une proportion	121
3.4.5 Le test A/B : comparer deux proportions	122
3.4.6 Le test du khi-deux : deux variables qualitatives sont-elles liées ?	122
3.4.7 Comment choisir son test ?	123
3.5 p-valeurs, puissance et tests multiples	124
3.5.1 Ce que la p-valeur est vraiment	124
3.5.2 Ce que la p-valeur n'est pas	125
3.5.3 Signification statistique $\neq$ importance pratique	126
3.5.4 La puissance : savoir si l'on peut voir ce qu'on cherche	126

3.5.5 Les tests multiples : le piège du « fouillis de comparaisons »	128
3.5.6 Les bonnes pratiques, en dix lignes	130
3.6 ⊕ Pour aller plus loin : les sondages et l'échantillonnage	130
3.6.1 Le biais de sélection : le pire ennemi	130
3.6.2 L'échantillonnage aléatoire simple	131
3.6.3 L'échantillonnage stratifié	132
3.6.4 Autres plans de sondage	133
3.6.5 Redresser un échantillon biaisé : la pondération	133
3.7 ⊕ Pour aller plus loin : les méthodes non paramétriques	134
3.7.1 Remplacer les valeurs par leurs rangs	134
3.7.2 Le test de Mann-Whitney (deux groupes indépendants)	135
3.7.3 Autres tests de rangs	135
3.7.4 Les tests de permutation : l'idée la plus simple de la statistique	136
3.7.5 Tester la normalité	137
3.8 Exercices du chapitre 3	138
Énoncés	138
Corrigés	139
Bilan du chapitre 3	142
Chapitre 4 : Programmation	143
Le chemin de ce chapitre	143
4.1 Les fondamentaux de Python	144
4.1.1 Pourquoi Python, et comment l'exécuter	144
4.1.2 Variables et types	144
4.1.3 Le texte et les f-strings	146
4.1.4 Les collections : listes, tuples, dictionnaires, ensembles	146
4.1.5 Décider : conditions	148
4.1.6 Répéter : boucles	148
4.1.7 Fonctions : écrire ses propres outils	149
4.1.8 Les erreurs : vos meilleures amies	150
4.1.9 Modules, fichiers et données réelles	151
4.1.10 Un programme complet : la caisse de la boutique	152
4.2 Les fondamentaux de R	154
4.2.1 Lancer R, et premières différences	155
4.2.2 Vecteurs : la brique de base	155
4.2.3 Fonctions, conditions et boucles	156
4.2.4 Le <code>data.frame</code> : le tableau de données	157
4.2.5 Les statistiques « sortent de la boîte »	159
4.2.6 Le tidyverse : manipuler les tableaux avec des « phrases »	160
4.2.7 Python ou R ? Un tableau de correspondance	161
4.3 Algorithmes et structures de données	161
4.3.1 Qu'est-ce qu'un algorithme ?	162
4.3.2 Choisir sa structure de données	162
4.3.3 Piles et files	164
4.3.4 La récursion : une fonction qui s'appelle elle-même	165
4.3.5 Rechercher : de la liste à la dichotomie	167
4.3.6 Trier	169
4.3.7 Une dernière application : les « top k »	171
4.4 NumPy et pandas : manipuler des données	172
4.4.1 NumPy : pourquoi un tableau n'est pas une liste	172
4.4.2 Créer, indexer, trancher	174
4.4.3 Le broadcasting : calculer entre tableaux de formes différentes	175

4.4.4 Agréger, trier, tirer au hasard	177
4.4.5 pandas : Series et DataFrame	178
4.4.6 Sélectionner : colonnes, lignes, conditions	181
4.4.7 Créer et transformer des colonnes	182
4.4.8 Copie, vue et pièges de pandas 3.0	184
4.4.9 Regrouper : le « split-apply-combine » (groupby)	184
4.4.10 Combiner plusieurs tables : merge et concat	186
4.4.11 Valeurs manquantes	188
4.4.12 Travailler avec des dates	189
4.4.13 ■ Petite application : le rapport hebdomadaire de Yasmine	191
4.5 Premières visualisations	192
4.5.1 Quel graphique pour quelle question ?	193
4.5.2 Anatomie d'un graphique matplotlib	194
4.5.3 Les quatre graphiques essentiels, avec pandas	195
4.5.4 seaborn : des graphiques statistiques en une ligne	198
4.5.5 ggplot2 : la grammaire des graphiques en R	199
4.5.6 Bien faire, mal faire : les pièges du graphique trompeur	202
4.5.7 Enregistrer, réutiliser : de la figure au rapport	203
4.6 ⊕ Pour aller plus loin : programmation orientée objet, code propre et tests	206
4.6.1 Pourquoi des objets ? Le problème des dictionnaires	206
4.6.2 Votre première classe	206
4.6.3 Le cahier des charges du module	207
4.6.4 Code propre : lisible avant tout	208
4.6.5 Tester son code : le filet de sécurité	209
4.6.6 Une application : tester une fonction d'analyse	213
4.7 ⊕ Pour aller plus loin : d'autres langages (SAS, MATLAB, Julia)	214
4.7.1 La tâche : un résumé par canal	214
4.7.2 SAS : le langage des grandes organisations	215
4.7.4 Julia : la promesse « rapide comme C, simple comme Python »	216
4.7.6 Alors, lequel choisir ?	217
4.8 ⊕ Pour aller plus loin : complexité algorithmique et optimisation du code	218
4.8.1 L'idée : compter les opérations, pas les secondes	218
4.8.2 La notation $O(\cdot)$: une définition rigoureuse	219
4.8.3 Mesurer : le temps, avec méthode	220
4.8.4 Boucles Python contre calcul vectorisé	222
4.8.5 La mémoïsation : ne jamais calculer deux fois la même chose	223
4.8.6 Profiler : mesurer avant d'optimiser	224
4.8.7 Application : les paires de produits achetés ensemble	225
4.9 Exercices corrigés	226
Énoncés	227
Corrigés	228
Bilan du chapitre 4	238
Chapitre 5 : Bases de données et SQL	239
Le chemin de ce chapitre	239
5.1 Modèle relationnel et conception de bases de données	240
5.1.1 Pourquoi pas simplement un fichier CSV ?	240
5.1.2 Le modèle relationnel	241
5.1.3 Construire la base de Dar Jasmin	243
5.1.4 Types, contraintes : la base se défend	246
5.1.5 SQL sur un fichier CSV : l'import en cinq lignes	247
5.2 Requêtes SQL : sélection, jointures, agrégation	248

5.2.1 Anatomie d'une requête : SELECT ... FROM ...	248
5.2.2 Filtrer avec WHERE	249
5.2.3 Trier, limiter, dédoubler	250
5.2.4 Agréger : COUNT, SUM, AVG, GROUP BY	251
5.2.5 Les jointures : relier les tables	253
5.2.6 Les sous-requêtes : une requête dans une requête	256
5.2.7 Les pièges : NULL et dates	256
5.2.8 Mises en pratique	259
5.3 Fonctions fenêtres et CTE	261
5.3.1 Le principe : fonction(...) OVER (...)	261
5.3.2 Les classements : ROW_NUMBER, RANK, DENSE_RANK	262
5.3.3 Cumuls et moyennes mobiles : le cadre de la fenêtre	263
5.3.4 Comparer avec la ligne précédente : LAG et LEAD	265
5.3.5 Les CTE : donner un nom à une étape	266
5.3.6 Les CTE récursives : des requêtes qui se rappellent elles-mêmes	267
5.4 Normalisation et conception de schémas	270
5.4.1 Le problème : la grande feuille unique	270
5.4.2 Dépendances fonctionnelles : la théorie derrière la normalisation	272
5.4.3 Les trois premières formes normales	274
5.4.4 Aller plus loin, ou s'arrêter plus tôt ?	276
5.4.5 Les index : retrouver une ligne sans tout lire	277
5.4.6 Les transactions : tout ou rien	278
5.5 ⊕ Pour aller plus loin : les bases NoSQL (MongoDB, Redis)	280
5.5.1 Pourquoi sortir du relationnel ?	280
5.5.2 Les bases de documents : l'exemple de MongoDB	281
5.5.3 Les bases clé-valeur : l'exemple de Redis	283
5.5.4 Alors, que choisir ?	284
5.6 Exercices du chapitre 5	285
Énoncés	285
Corrigés	286
Bilan du chapitre 5	292
Chapitre 6 : Outils de travail	294
Le chemin de ce chapitre	294
L'atelier de Yasmine	295
6.1 Git et gestion de versions	296
6.1.1 Pourquoi pas simplement « analyse_FINAL_vrai.py » ?	296
6.1.2 Premiers réglages	296
6.1.3 Le premier dépôt : init, add, commit	297
6.1.4 Ce qu'il y a vraiment dans un commit	298
6.1.5 Modifier, comparer, enregistrer : le cycle de travail	299
6.1.6 Défaire une erreur	301
6.1.7 Ce qu'il ne faut pas suivre : .gitignore	302
6.1.8 Les branches : tester une idée sans risque	303
6.1.9 Étiqueter une version : les tags	307
6.1.10 Dépôts distants : partager et sauvegarder	307
6.1.11 Bonnes pratiques pour la data science	308
6.2 Notebooks Jupyter	309
6.2.1 Les ingrédients : cellules et noyau	309
6.2.2 Un notebook est un fichier texte (JSON)	310
6.2.3 Le piège : l'état caché	312
6.2.4 Un détecteur d'ordre suspect	314

6.2.5 Bonnes pratiques	314
6.2.6 Notebooks et Git : le problème des sorties	315
6.2.7 Exporter : du notebook au rapport, au script	315
6.2.8 Pour finir : choisir son outil	317
6.3 Ligne de commande et environnements	317
6.3.1 S'orienter dans les dossiers	318
6.3.2 Regarder le contenu d'un fichier	319
6.3.3 Interroger des données avec des outils de texte	320
6.3.4 Pipes et redirections : assembler de petits outils	321
6.3.5 Lancer des programmes Python depuis le terminal	322
6.3.6 Le problème des bibliothèques : les environnements virtuels	323
6.3.7 Variables d'environnement et PATH	324
6.3.8 Et sous Windows ?	324
6.4 ⊕ Pour aller plus loin : scripts shell, environnements avancés et Docker	325
6.4.1 Écrire un script shell	325
6.4.2 Enchaîner des étapes : un petit pipeline	327
6.4.3 Faire tourner un script tout seul : cron	328
6.4.4 Isoler plus finement : plusieurs versions d'une même bibliothèque	328
uv : un installateur très rapide, compatible avec pip	330
6.5 ⊕ Pour aller plus loin : recherche reproductible	331
6.5.1 Reproductible, répliquable : de quoi parle-t-on ?	331
6.5.2 L'aléa maîtrisé : les graines	332
6.5.3 Relever l'environnement	333
6.5.4 Un rapport qui se fabrique tout seul	334
6.5.5 make : ne refaire que ce qui a changé	336
6.5.6 Les outils « tout-en-un » : R Markdown et Quarto	337
6.5.7 LaTeX : composer proprement les formules et les PDF	338
6.5.8 Garder une empreinte des données	339
6.6 Exercices du chapitre 6	340
Énoncés	340
Corrigés	341
Bilan du chapitre 6	347
Projet du volume : l'étude Dar Jasmin 2025	349
P.1 Le cahier des charges	349
P.2 Étape 1 : charger et contrôler les données	350
P.3 Étape 2 : photographier l'année (question 1)	351
P.4 Étape 3 : Instagram dépense-t-il vraiment moins ? (question 2)	354
P.5 Étape 4 : les retards font-ils baisser la satisfaction ? (question 3)	356
P.6 Étape 5 : qui relancer en janvier ? (question 4)	358
P.7 Étape 6 : rédiger le rapport	359
P.8 Rendre le projet reproductible	360
P.9 Ce que cette étude ne dit pas, et la suite	361
Points clés et auto-évaluation	363
Les points clés, chapitre par chapitre	363
Trente questions pour s'auto-évaluer	364
Mathématiques (chapitre 1)	364
Probabilités (chapitre 2)	364
Statistique (chapitre 3)	364
Programmation (chapitre 4)	364

SQL (chapitre 5)	365
Outils (chapitre 6)	365
Vérifier les réponses chiffrées	365
Corrigé	366
Votre grille d'auto-évaluation	367
Et maintenant ?	368

Avant-propos

Bienvenue

Ce livre est le premier d'une série de six volumes qui racontent, pas à pas, comment on devient data scientist : on part de ce qu'il y a de plus simple (additionner des nombres, ranger des données dans un tableau) et on arrive à ce qu'il y a de plus récent (modèles de langage, mise en production, modélisation du risque en banque et en assurance).

Ce premier volume est le **socle**. Il ne contient aucun modèle « à la mode » : ni réseau de neurones, ni intelligence artificielle spectaculaire. Il contient ce qui rend tout le reste possible :

- les **mathématiques** dont on a vraiment besoin (et seulement celles-là) ;
- les **probabilités**, qui donnent un langage précis à l'incertitude ;
- la **statistique**, qui apprend à tirer des conclusions honnêtes à partir de données imparfaites ;
- la **programmation**, pour que la machine fasse les calculs à notre place ;
- les **bases de données et le SQL**, pour aller chercher l'information là où elle vit ;
- les **outils de travail** (Git, notebooks, ligne de commande), pour travailler proprement et reproduire ses résultats.

Si vous ne deviez retenir qu'une idée de ce livre, ce serait celle-ci :

La data science n'est pas de la magie. C'est de l'artisanat : des mathématiques simples, du code propre et beaucoup de bon sens, appliqués avec méthode.

Un artisan, ça se forme avec des gestes répétés. C'est pourquoi ce livre est rempli d'exemples chiffrés que vous pouvez refaire à la main, puis refaire avec l'ordinateur.

À qui s'adresse ce livre ?

À toute personne qui veut apprendre le métier depuis le début :

- l'étudiant ou l'étudiante qui entre dans une filière de statistique, d'informatique ou d'ingénierie ;
- la personne en reconversion qui veut comprendre ce qu'il y a sous le capot des outils ;
- l'analyste qui sait faire des tableaux croisés et veut passer à la modélisation ;
- l'ingénieur ou l'ingénieure déjà formé(e) qui veut un ouvrage de référence clair pour réviser ses bases.

Prérequis : le niveau de mathématiques du lycée, de la curiosité, et un ordinateur. Aucune expérience de programmation n'est supposée. Si certaines notions de base sont rouillées, le « Rappel express » qui suit cet avant-propos les remet en route en quelques pages.

Comment ce livre est construit

Les livres de mathématiques ont souvent deux défauts opposés. Certains sont **rigoureux mais froids** : des définitions, des théorèmes, des démonstrations, et on se demande à quoi tout cela sert. D'autres sont **sympathiques mais flous** : on a l'impression d'avoir compris, puis on ne sait plus rien refaire seul.

Ce volume essaie d'éviter les deux, en **alternant systématiquement** :

1. une **intuition** simple, avec une image ou une histoire ;
2. un **exemple chiffré** assez petit pour être refait à la main ;
3. quand c'est utile, une **démonstration rigoureuse**, pour que la certitude remplace la confiance aveugle ;
4. une **petite application réelle** que l'on exécute avec du code ;
5. des **exercices corrigés** pour vérifier que l'idée est bien entrée.

Chaque idée importante reçoit au moins un exemple « sans issue de secours » : un exemple si concret qu'on ne puisse plus se dire « je n'ai pas compris ».

Les encadrés

Pour que la lecture reste fluide, chaque type de contenu a son pictogramme :

Pictogramme	Rôle
■ Intuition	L'idée en langage courant, sans formule. À lire en premier.
■ Exemple	Un calcul concret, souvent refait à la main.
■ Démonstration	Le raisonnement rigoureux. On peut la sauter à la première lecture, mais elle est ce qui rend la compréhension durable.
■ Application	Une petite application réelle, avec du code.
△ Piège	Une erreur classique, que presque tout le monde fait une fois.
✓ À retenir	Le résumé à garder en mémoire.
Exercices	À faire avec un crayon avant de regarder les corrigés.
⊕ Pour aller plus loin	Section facultative : approfondissement, outil ou sujet connexe.

Le principe « essentiel / pour aller plus loin »

Dans chaque chapitre, tout ce qui n'est **pas** marqué ⊕ constitue le **parcours essentiel**. Il est complet : un lecteur qui suit uniquement ce parcours acquiert la vue d'ensemble et les compétences utiles pour la suite de la série.

Les sections marquées ⊕ **Pour aller plus loin** sont des cadeaux, pas des obligations : elles approfondissent un point, présentent un outil de plus ou ouvrent une porte. Certains lecteurs préfèrent se concentrer sur la grande idée ; d'autres aiment creuser. Les deux manières de lire ce livre sont bonnes.

Trois parcours de lecture

Parcours	Pour qui	Que lire
Essentiel	Vous voulez la vue d'ensemble, vite	Les encadrés ■ ■ ■ ✓, sans les ■ ni les ⊕
Complet	Vous voulez comprendre en profondeur	Tout le parcours essentiel, démonstrations comprises, plus les exercices
Praticien	Vous aimez coder avant de théoriser	Le chapitre 6 (outils), puis le 4 (programmation), puis le 5 (SQL), puis les chapitres 1 à 3

Vous n'avez jamais programmé ? Pas de panique : chaque bloc de code est suivi de **sa sortie réelle**. On peut donc lire le livre sans rien exécuter et comprendre ce qui se passe. Quand vous serez prêt(e), le chapitre 4 (section 4.1) vous apprend Python depuis zéro.

Le fil rouge : la boutique Dar Jasmin

Apprendre sur des données abstraites est ennuyeux. Tout au long de ce volume, nous travaillons donc sur une même histoire :

Yasmine vient de lancer **Dar Jasmin**, une boutique en ligne d'artisanat tunisien : poteries, huile d'olive, tapis, bijoux. Elle vend, elle livre, elle reçoit des retours. Elle a des questions ; vous allez l'aider à y répondre avec des données.

Voici quelques-unes de ses questions, et le chapitre où nous y répondrons :

Question de Yasmine	Outil	Chapitre
« Quels clients se ressemblent ? »	vecteurs, similarité	1
« Comment ajuster mes prix pour maximiser le chiffre d'affaires ? »	optimisation	1
« Cette commande est-elle une fraude ? »	théorème de Bayes	2
« Combien de commandes dois-je prévoir à 14 h ? »	loi de Poisson	2
« Mon nouveau transporteur est-il vraiment plus rapide ? »	tests d'hypothèses	3
« Combien vaut mon panier moyen, à ± 2 dinars près ? »	intervalle de confiance	3
« Comment automatiser mes rapports de ventes ? »	Python, pandas	4
« Quels sont mes dix meilleurs clients ? »	SQL	5
« Comment retrouver la version de mon analyse d'il y a un mois ? »	Git	6

Une précision importante : Dar Jasmin et toutes ses données sont **fictifs**. Les chiffres sont générés par ordinateur, avec des règles simples que nous connaissons, ce qui permet de vérifier que nos méthodes retrouvent bien la réalité. Toute ressemblance avec une vraie boutique serait une coïncidence.

Le code de ce livre

Les exemples de code sont écrits en **Python** (langage principal du livre), avec quelques exemples en **R** et en **SQL**. Chaque exemple Python ou SQL est suivi de sa sortie, introduite par un bloc grisé comme celui-ci :

```
prix = [45.0, 12.5, 80.0]
print(f"Total du panier : {sum(prix):.2f} DT")
```

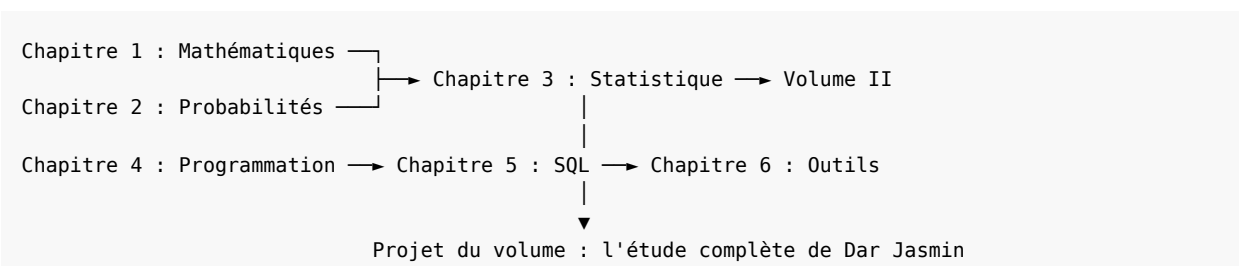
```
Total du panier : 137.50 DT
```

Les données aléatoires sont générées avec une **graine** (*seed*) fixée, ce qui fait qu'en rejouant le code, vous obtiendrez les mêmes nombres que dans le livre, à de très légères variations près selon les versions des bibliothèques.

Pour installer ce qu'il faut, un seul jeu de commandes suffit (détaillé au chapitre 6) :

```
bash noexec python -m venv .venv source .venv/bin/activate          # sous Windows :
.venv\Scripts\activate pip install numpy pandas scipy matplotlib seaborn
```

La carte du volume



Chapitre	Il répond à la question...	À la fin, vous saurez...
1. Mathématiques	De quels outils mathématiques a-t-on besoin ?	manipuler vecteurs et matrices, dériver, minimiser une fonction
2. Probabilités	Comment parler d'incertitude avec précision ?	calculer des probabilités, reconnaître les lois usuelles, comprendre la loi des grands nombres
3. Statistique	Que peut-on conclure à partir d'un échantillon ?	estimer, construire un intervalle de confiance, tester une hypothèse
4. Programmation	Comment faire calculer l'ordinateur ?	écrire du Python, manipuler des données avec pandas, tracer des graphiques
5. SQL	Comment interroger une base de données ?	écrire des requêtes avec jointures, agrégations et fonctions fenêtres
6. Outils	Comment travailler proprement ?	utiliser Git, des notebooks, la ligne de commande

Faites le point : un petit test de départ

Prenez dix minutes, un crayon, et répondez **sans calculatrice**. Les réponses sont juste après.

1. Calculez $\frac{3}{4} + \frac{5}{6}$.

2. Développez $(x + 2)^2$.
3. Résolvez $2x - 7 = 11$.
4. Quelle est la moyenne des nombres 4, 8, 15, 16, 23, 42 ?
5. On lance deux dés équilibrés. Quelle est la probabilité que la somme fasse 7 ?
6. Quelle est la dérivée de x^2 ?
7. Calculez $2^3 \times 2^4$.
8. Que vaut $\log_{10}(1000)$?
9. Un article coûte 120 DT. On applique 25 % de remise. Quel est le nouveau prix ?
10. Dans une phrase : à quoi sert une « variable » en programmation ?

Réponses.

1. $\frac{3}{4} + \frac{5}{6} = \frac{9}{12} + \frac{10}{12} = \frac{19}{12}$.
2. $(x + 2)^2 = x^2 + 4x + 4$.
3. $2x = 18$, donc $x = 9$.
4. $(4 + 8 + 15 + 16 + 23 + 42)/6 = 108/6 = 18$.
5. Il y a $6 \times 6 = 36$ résultats possibles, dont 6 donnent 7 : (1,6), (2,5), (3,4), (4,3), (5,2), (6,1).
Probabilité : $6/36 = 1/6$.
6. $2x$.
7. $2^{3+4} = 2^7 = 128$.
8. 3, car $10^3 = 1000$.
9. $120 \times 0,75 = 90$ DT.
10. Une variable est un nom qui désigne une valeur gardée en mémoire (par exemple prix = 45.0), que l'on peut relire et modifier.

Comment interpréter votre score ?

- **8 à 10 bonnes réponses** : vous êtes prêt(e). Commencez par le chapitre 1.
- **5 à 7** : lisez d'abord le « Rappel express », puis commencez.
- **Moins de 5** : lisez le « Rappel express » attentivement, en refaisant les calculs à la main. Ce n'est pas une affaire de talent, seulement de pratique.

Maintenant, au travail. Yasmine nous attend, et sa boutique reçoit une commande à l'instant.

Rappel express : les bases à avoir en tête

Quelques pages pour remettre en route ce qu'on a vu au collège et au lycée. Rien ici n'est nouveau : tout sert dans la suite du livre. Si tout vous semble évident, passez directement au chapitre 1.

R.1 Fractions et pourcentages

Une **fraction** est une division : $\frac{3}{4}$ veut dire « 3 divisé par 4 », soit 0,75. Pour additionner deux fractions, on les met sur le même dénominateur :

$$\frac{3}{4} + \frac{5}{6} = \frac{3 \times 3}{4 \times 3} + \frac{5 \times 2}{6 \times 2} = \frac{9}{12} + \frac{10}{12} = \frac{19}{12}.$$

Un **pourcentage** est une fraction de dénominateur 100. « Prendre 25 % de 120 DT » revient à calculer $120 \times \frac{25}{100} = 30$ DT.

Exemple (Dar Jasmin). Un tapis coûte 240 DT. Yasmine propose une remise de 15 %.

- Montant de la remise : $240 \times 0,15 = 36$ DT.
- Nouveau prix : $240 - 36 = 204$ DT, ou directement $240 \times 0,85 = 204$ DT.

Astuce : « baisser de 15 % » revient à **multiplier par 0,85**. « Augmenter de 19 % » (la TVA) revient à multiplier par 1,19.

Piège. Une hausse de 20 % suivie d'une baisse de 20 % **ne ramène pas** au prix de départ : $100 \rightarrow 120 \rightarrow 96$. Les pourcentages se multiplient, ils ne s'additionnent pas.

```
prix = 100
prix = prix * 1.20 # +20 %
prix = prix * 0.80 # -20 %
print(prix)
```

96.0

R.2 Puissances et racines

a^n signifie « a multiplié n fois par lui-même » : $2^3 = 2 \times 2 \times 2 = 8$. Les règles à connaître :

$$a^m \times a^n = a^{m+n}, \quad \frac{a^m}{a^n} = a^{m-n}, \quad (a^m)^n = a^{mn}, \quad a^0 = 1, \quad a^{-n} = \frac{1}{a^n}.$$

La **racine carrée** $\sqrt{a}$ est le nombre positif dont le carré vaut a : $\sqrt{49} = 7$. On a aussi $\sqrt{a} = a^{1/2}$.

Exemple. $2^{10} = 1024$; $10^{-2} = 0,01$; $\sqrt{2} \approx 1,414$. Et $2^3 \times 2^4 = 2^7 = 128$, qu'on vérifie : $8 \times 16 = 128$.

R.3 Équations et fonctions affines

Résoudre une équation du premier degré, c'est isoler l'inconnue en faisant la **même opération des deux côtés** :

$$2x - 7 = 11 \implies 2x = 18 \implies x = 9.$$

Une **fonction affine** s'écrit $y = ax + b$. Le nombre a est la **pente** (de combien y augmente quand x augmente de 1), et b est l'**ordonnée à l'origine** (la valeur de y quand $x = 0$).

Exemple. Les frais de livraison de Dar Jasmin valent 5 DT de forfait plus 0,80 DT par kilo. Si x est le poids en kilos :

$$\text{frais}(x) = 0,8x + 5.$$

Pour un colis de 6 kg : $0,8 \times 6 + 5 = 9,8$ DT. La pente (0,8) est le coût d'un kilo de plus ; l'ordonnée à l'origine (5) est le forfait.

Cette idée, « une pente et une ordonnée à l'origine », reviendra souvent : c'est la base de la régression linéaire.

R.4 Le symbole $\sum$ (somme)

En statistique, on additionne sans arrêt. Pour ne pas écrire $x_1 + x_2 + \dots + x_n$ à chaque fois, on utilise le grand sigma :

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n.$$

Cela se lit : « somme, pour i allant de 1 à n , des x_i ». C'est exactement une boucle for qui additionne.

Exemple. Les ventes de Dar Jasmin sur cinq jours (en dinars) : $x_1 = 120$, $x_2 = 80$, $x_3 = 200$, $x_4 = 100$, $x_5 = 150$.

$$\sum_{i=1}^5 x_i = 120 + 80 + 200 + 100 + 150 = 650 \text{ DT.}$$

La **moyenne** est $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{650}{5} = 130$ DT.

```
ventes = [120, 80, 200, 100, 150]
total = sum(ventes)
moyenne = total / len(ventes)
print(total, moyenne)
```

650 130.0

Deux propriétés utiles (elles se vérifient en écrivant les sommes) :

$$\sum (x_i + y_i) = \sum x_i + \sum y_i, \quad \sum c x_i = c \sum x_i \quad (c \text{ constante}).$$

R.5 Exponentielle et logarithme

Le **logarithme** répond à la question : « à quelle puissance faut-il élever la base pour obtenir ce nombre ? ». Ainsi $\log_{10}(1000) = 3$, car $10^3 = 1000$.

En data science, on utilise surtout le **logarithme népérien** $\ln$ (base $e \approx 2,718$), et son opposé, la **fonction exponentielle** e^x . Ce qu'il faut savoir :

$$\ln(ab) = \ln a + \ln b, \quad \ln(a^n) = n \ln a, \quad \ln(e^x) = x, \quad e^{a+b} = e^a e^b.$$

Intuition. Le logarithme transforme les **produits en sommes**. C'est précieux : une somme est bien plus facile à manipuler (et à dériver) qu'un produit. C'est la raison pour laquelle, plus tard, nous prendrons le log de produits de probabilités pour estimer des paramètres.

Exemple. Un placement double tous les 10 ans. Après 30 ans, il a été multiplié par $2^3 = 8$. Combien de « doublements » faut-il pour multiplier par 1000 ? $\log_2(1000) \approx 9,97$, soit environ 10 doublements.

```
import math
print(math.log10(1000))      # logarithme en base 10
print(math.log2(1000))      # logarithme en base 2
print(math.log(math.e**5))  # ln(e^5)
```

```
3.0
9.965784284662087
5.0
```

R.6 Fonctions, graphes et équations de droites

Une **fonction** associe à chaque entrée x une sortie $f(x)$. Son **graphe** est la courbe des points $(x, f(x))$. Quelques fonctions que nous croiserons :

Fonction	Forme	Allure
affine	$ax + b$	droite
carré	x^2	parabole (« bol », avec un minimum en 0)
exponentielle	e^x	croît de plus en plus vite
logarithme	$\ln x$	croît de plus en plus lentement
racine	$\sqrt{x}$	croît lentement

R.7 Un peu de vocabulaire de programmation

Avant de lire du code, reprenez trois mots :

- une **variable** : un nom pour une valeur (`prix = 45.0`) ;
- une **fonction** : une recette réutilisable (`sum([1, 2, 3])` renvoie 6) ;
- une **liste** : une suite ordonnée de valeurs (`[120, 80, 200]`).

Le reste sera expliqué quand il apparaîtra, et le chapitre 4 reprend tout depuis le début.

Faites le point

Si vous savez répondre à ces cinq questions, vous avez tout ce qu'il faut pour commencer :

1. Quel est le prix final d'un article à 80 DT avec 19 % de TVA ?
2. Que vaut $\sum_{i=1}^4 i^2$?
3. Quelle est la pente de $y = -3x + 10$?
4. Simplifiez $\ln(e^2 \times e^3)$.
5. Combien font $\sqrt{81} + 2^{-1}$?

Réponses. (1) $80 \times 1,19 = 95,2$ DT. (2) $1 + 4 + 9 + 16 = 30$. (3) -3 (la droite descend). (4) $\ln(e^5) = 5$. (5) $9 + 0,5 = 9,5$.

Chapitre 1 : Mathématiques pour la data science

« Un tableau de données, c'est une **matrice**. Un modèle, c'est une **fonction**. Apprendre, c'est **minimiser une erreur**. »

Ces trois phrases résument ce chapitre. Si elles vous paraissent mystérieuses, c'est normal : à la fin du chapitre, elles vous sembleront évidentes.

Pourquoi des mathématiques ?

On entend parfois que « la data science, ce sont des bibliothèques Python : on appelle `fit()` et c'est fini ». C'est vrai... jusqu'au jour où le modèle donne un résultat absurde et où l'on doit comprendre pourquoi. Les mathématiques sont ce qui permet alors de **raisonner au lieu de deviner**.

Rassurez-vous : il ne s'agit pas de refaire un cursus de mathématiques. Il suffit de **quatre familles d'idées**, que nous construirons de zéro :

Idée	Ce que c'est	À quoi ça sert en data science
Vecteurs et matrices	des listes et des tableaux de nombres, avec des règles de calcul	représenter les données, comparer des individus, réduire la dimension
Dérivées et gradients	la mesure de « à quelle vitesse ça change »	savoir dans quel sens améliorer un modèle
Intégrales	la mesure d'une « quantité accumulée »	calculer des probabilités (chapitre 2)
Optimisation	trouver le meilleur choix selon un critère	entraîner (presque) tous les modèles

Le chemin de ce chapitre

Nous allons suivre la boutique **Dar Jasmin** et ses questions concrètes :

- **1.1 Algèbre linéaire** : Yasmine veut savoir quels clients se ressemblent, calculer son chiffre d'affaires par mois sans boucle interminable, et résumer un tableau de ventes en quelques tendances.
- **1.2 Analyse** : comment son bénéfice change-t-il quand elle modifie un prix un tout petit peu ? Et quelle est la probabilité qu'une commande arrive dans les trois prochaines minutes ?
- **1.3 Optimisation** : quel prix maximise ses recettes ? Comment répartir son budget publicitaire ?
- **1.4 Fiche de notations** : toutes les notations du livre, au même endroit.

- **⊕ Pour aller plus loin** : pourquoi l'ordinateur se trompe parfois (analyse numérique), et comment compter et relier des objets (mathématiques discrètes et graphes).

Comment lire ce chapitre. Chaque notion suit le même rythme : une intuition, un exemple chiffré à la main, puis (si nécessaire) une démonstration, puis le code. Si une démonstration vous décourage, sautez-la : l'exemple et le code suffisent pour avancer. Revenez-y plus tard.

1.1 Algèbre linéaire

L'algèbre linéaire est le langage des tableaux de nombres. Tout ce que fait un modèle de data science, de la régression la plus simple au réseau de neurones le plus profond, s'écrit avec elle. Nous allons en construire les quatre briques : **vecteurs**, **matrices**, **valeurs propres** et **décomposition en valeurs singulières**.

1.1.1 Vecteurs : décrire un client par une liste de nombres

Intuition. Un **vecteur** est une liste ordonnée de nombres. On peut le voir de deux façons, qui sont les deux faces d'une même pièce :

- comme une **flèche** partant de l'origine et pointant vers un point ;
- comme la **fiche d'identité** d'un individu : une ligne de tableau où chaque case décrit une caractéristique.

En data science, la seconde vision est la plus utile : *un individu = un vecteur*.

Un exemple concret

Dar Jasmin compte, pour chaque client, le nombre d'achats dans trois catégories : **poteries**, **huile d'olive**, **bijoux**. Chaque client devient un vecteur à trois composantes :

Client	Poteries	Huile	Bijoux	Vecteur
Amel	3	1	0	a = (3, 1, 0)
Bilel	6	2	0	b = (6, 2, 0)
Chaima	0	1	4	c = (0, 1, 4)

On note $\mathbf{a} = (a_1, a_2, \dots, a_p)$ un vecteur à p composantes, et $\mathbb{R}^p$ l'ensemble de tous les vecteurs à p composantes réelles. Ici $p = 3$: nos clients vivent dans $\mathbb{R}^3$.

Convention du livre. Un nombre seul (un *scalaire*) s'écrit en italique : a , λ . Un vecteur s'écrit en gras minuscule : **a**. Une matrice s'écrit en gras majuscule : **A**.

Les opérations de base

Addition. On additionne composante par composante. Si Amel et Chaima regroupent leurs achats :

$$\mathbf{a} + \mathbf{c} = (3 + 0, 1 + 1, 0 + 4) = (3, 2, 4).$$

Multiplication par un scalaire. On multiplie chaque composante :

$$2\mathbf{a} = (2 \times 3, 2 \times 1, 2 \times 0) = (6, 2, 0) = \mathbf{b}.$$

Regardez bien : **Bilel est exactement « deux fois Amel »**. Il achète les mêmes choses dans les mêmes proportions, simplement en plus grande quantité. Nous allons voir comment la mathématique capture cette idée.

Combinaison linéaire. Combiner les deux opérations donne une *combinaison linéaire* : $\lambda_1 \mathbf{u}_1 + \lambda_2 \mathbf{u}_2$. Par exemple, « la moitié d'Amel plus un tiers de Chaima » est $\frac{1}{2}\mathbf{a} + \frac{1}{3}\mathbf{c}$. Presque tout ce que nous ferons en modélisation est une combinaison linéaire de quelque chose.

En Python, la bibliothèque **NumPy** manipule les vecteurs comme des objets à part entière :

```
import numpy as np

amel = np.array([3, 1, 0])
bilel = np.array([6, 2, 0])
chaima = np.array([0, 1, 4])

print("Amel + Chaima :", amel + chaima)
print("2 x Amel      :", 2 * amel)
print("Bilel == 2 x Amel ?", np.array_equal(bilel, 2 * amel))
```

```
Amel + Chaima : [3 2 4]
2 x Amel      : [6 2 0]
Bilel == 2 x Amel ? True
```

Le produit scalaire : mesurer l'accord entre deux vecteurs

Le **produit scalaire** de deux vecteurs de même taille est la somme des produits des composantes :

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^p a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_p b_p.$$

Exemple à la main.

$$\mathbf{a} \cdot \mathbf{b} = 3 \times 6 + 1 \times 2 + 0 \times 0 = 18 + 2 + 0 = 20.$$

$$\mathbf{a} \cdot \mathbf{c} = 3 \times 0 + 1 \times 1 + 0 \times 4 = 0 + 1 + 0 = 1.$$

Intuition. Le produit scalaire est grand quand les deux vecteurs « vont dans le même sens » (ils ont des composantes fortes aux mêmes endroits), proche de 0 quand ils n'ont rien en commun, et négatif quand ils s'opposent. Amel et Bilel s'entendent bien (20) ; Amel et Chaima presque pas (1) : le seul point commun est une unité d'huile.

La **norme** d'un vecteur est sa longueur, donnée par le théorème de Pythagore généralisé :

$$\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{a_1^2 + a_2^2 + \dots + a_p^2}.$$

Pour nos clients : $\|\mathbf{a}\| = \sqrt{9 + 1 + 0} = \sqrt{10} \approx 3,162$, $\|\mathbf{b}\| = \sqrt{36 + 4} = \sqrt{40} \approx 6,325$ et $\|\mathbf{c}\| = \sqrt{0 + 1 + 16} = \sqrt{17} \approx 4,123$. On remarque que $\|\mathbf{b}\| = 2\|\mathbf{a}\|$: doubler un vecteur double sa longueur.

Le produit scalaire a aussi une **interprétation géométrique** qui est fondamentale :

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta,$$

où θ est l'angle entre les deux flèches. En isolant le cosinus, on obtient la **similarité cosinus** :

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

Exemple à la main.

- Amel et Bilel : $\cos \theta = \frac{20}{\sqrt{10} \sqrt{40}} = \frac{20}{\sqrt{400}} = \frac{20}{20} = 1$. Même direction : **mêmes goûts**.
- Amel et Chaima : $\cos \theta = \frac{1}{\sqrt{10} \sqrt{17}} = \frac{1}{\sqrt{170}} \approx 0,077$. Presque perpendiculaires : **goûts très différents**.

La similarité cosinus ignore la *taille* des vecteurs et ne regarde que leur *direction*. C'est exactement ce que l'on veut pour comparer des profils d'achat : un gros acheteur et un petit acheteur qui ont les mêmes goûts doivent apparaître comme semblables.

```
def cosinus(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("cos(Amel, Bilel) =", round(cosinus(amel, bilel), 4))
print("cos(Amel, Chaima) =", round(cosinus(amel, chaima), 4))
```

```
cos(Amel, Bilel) = 1.0
cos(Amel, Chaima) = 0.0767
```

Démonstration : pourquoi le cosinus reste toujours entre -1 et 1.

Il faut montrer que $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|$. C'est l'**inégalité de Cauchy-Schwarz**. Supposons $\mathbf{b} \neq \mathbf{0}$ et considérons, pour un réel t quelconque, la quantité

$$q(t) = \|\mathbf{a} - t\mathbf{b}\|^2 = \|\mathbf{a}\|^2 - 2t(\mathbf{a} \cdot \mathbf{b}) + t^2\|\mathbf{b}\|^2.$$

Une norme au carré est toujours positive ou nulle, donc $q(t) \geq 0$ **pour tout** t . Or q est un trinôme du second degré en t ; pour qu'il ne prenne jamais de valeur négative, son discriminant doit être négatif ou nul :

$$\Delta = 4(\mathbf{a} \cdot \mathbf{b})^2 - 4\|\mathbf{a}\|^2\|\mathbf{b}\|^2 \leq 0 \iff (\mathbf{a} \cdot \mathbf{b})^2 \leq \|\mathbf{a}\|^2\|\mathbf{b}\|^2.$$

En prenant la racine carrée, $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|$, donc $-1 \leq \cos \theta \leq 1$. ■

L'égalité (cosinus égal à ± 1) a lieu exactement quand le discriminant est nul, c'est-à-dire quand il existe un t tel que $\mathbf{a} = t\mathbf{b}$: les deux vecteurs sont alignés. C'est le cas d'Amel et Bilel.

La distance entre deux vecteurs

La **distance euclidienne** entre deux individus est la norme de leur différence :

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\| = \sqrt{\sum_{i=1}^p (a_i - b_i)^2}.$$

Exemple.

- $d(\mathbf{a}, \mathbf{b}) = \|(3 - 6, 1 - 2, 0 - 0)\| = \|(-3, -1, 0)\| = \sqrt{10} \approx 3,162$.
- $d(\mathbf{a}, \mathbf{c}) = \|(3 - 0, 1 - 1, 0 - 4)\| = \|(3, 0, -4)\| = \sqrt{9 + 0 + 16} = 5$.

```
print("distance(Amel, Bilel) =", round(np.linalg.norm(amel - bilel), 3))
print("distance(Amel, Chaima) =", round(np.linalg.norm(amel - chaima), 3))
```

```
distance(Amel, Bilel) = 3.162
distance(Amel, Chaima) = 5.0
```

Remarquez la nuance. Selon le **cosinus**, Amel et Bilel sont *identiques* (1) ; selon la **distance**, ils sont à 3,16 unités l'un de l'autre, parce que Bilel achète plus. Ni l'une ni l'autre mesure n'est « la bonne » : elles répondent à des questions différentes.

Mesure	Elle répond à...	À utiliser quand...
Cosinus	« Ont-ils les mêmes goûts ? »	seule la <i>proportion</i> compte (profils, textes, recommandations)
Distance	« Sont-ils proches en valeur absolue ? »	la <i>quantité</i> compte (regroupement par niveau de dépense, par exemple)

Piège : les unités. La distance additionne des carrés de différences, donc une variable exprimée en grands nombres écrase les autres. Soit trois clients décrits par (âge, panier moyen en DT) : $P = (30, 100)$, $Q = (60, 102)$ et $R = (31, 160)$.

- $d(P, Q) = \sqrt{30^2 + 2^2} = \sqrt{904} \approx 30,1$
- $d(P, R) = \sqrt{1^2 + 60^2} = \sqrt{3601} \approx 60,0$

Selon la distance, P (30 ans) ressemble davantage à Q (60 ans) qu'à R (31 ans), simplement parce que 60 dinars « pèsent » plus que 30 ans dans le calcul ! La cause : les deux variables n'ont pas la même échelle. Le remède est de **standardiser** les variables avant de les comparer, ce que nous ferons au chapitre 3.

À retenir (vecteurs).

- Un individu est un vecteur ; ses composantes sont ses caractéristiques.
- Le produit scalaire $\sum a_i b_i$ mesure l'accord entre deux vecteurs ; la norme mesure leur longueur.
- Le cosinus compare les **directions**, la distance compare les **positions**. Toujours se demander laquelle des deux correspond à la question posée.
- Avant de comparer des variables d'échelles différentes, on les standardise.

1.1.2 Matrices : le tableau de données devient un objet mathématique

Intuition. Une **matrice** est un tableau rectangulaire de nombres, rangés en lignes et en colonnes. Si un vecteur est « un individu », une matrice est « **tous les individus à la fois** » : chaque ligne est un individu, chaque colonne est une variable. Un fichier de données, c'est une matrice.

On note $\mathbf{A} \in \mathbb{R}^{n \times p}$ une matrice à n lignes et p colonnes (on dit « n par p »), et a_{ij} le nombre situé à la ligne i et à la colonne j . Une matrice carrée a autant de lignes que de colonnes ($n = p$).

Un exemple concret

Yasmine a noté ses ventes des trois premiers mois pour trois produits (poteries, huile, bijoux). Chaque **ligne** est un mois, chaque **colonne** un produit :

$$\mathbf{V} = \begin{pmatrix} 10 & 40 & 20 \\ 12 & 35 & 25 \\ 15 & 50 & 18 \end{pmatrix} \begin{array}{l} \leftarrow \text{janvier} \\ \leftarrow \text{février} \\ \leftarrow \text{mars} \end{array}$$

Les prix unitaires sont $\mathbf{p} = (45, 12, 30)$ dinars. Question : quel est le chiffre d'affaires de chaque mois ?

Le produit matrice-vecteur

Pour janvier, c'est un produit scalaire : $10 \times 45 + 40 \times 12 + 20 \times 30 = 450 + 480 + 600 = 1\,530$ DT. On fait de même pour février et mars. Calculer ces trois produits scalaires d'un coup, c'est ce qu'on appelle le **produit matrice-vecteur** $\mathbf{Vp}$:

$$\mathbf{Vp} = \begin{pmatrix} 10 \cdot 45 + 40 \cdot 12 + 20 \cdot 30 \\ 12 \cdot 45 + 35 \cdot 12 + 25 \cdot 30 \\ 15 \cdot 45 + 50 \cdot 12 + 18 \cdot 30 \end{pmatrix} = \begin{pmatrix} 1530 \\ 1710 \\ 1815 \end{pmatrix}.$$

Formellement, $(\mathbf{Vp})_i = \sum_j v_{ij} p_j$: la composante i du résultat est le produit scalaire de la **ligne** i de $\mathbf{V}$ avec $\mathbf{p}$.

Une seconde lecture, tout aussi utile. On peut aussi voir $\mathbf{Vp}$ comme une **combinaison linéaire** des colonnes de $\mathbf{V}$:

$$\mathbf{Vp} = 45 \begin{pmatrix} 10 \\ 12 \\ 15 \end{pmatrix} + 12 \begin{pmatrix} 40 \\ 35 \\ 50 \end{pmatrix} + 30 \begin{pmatrix} 20 \\ 25 \\ 18 \end{pmatrix}.$$

Le chiffre d'affaires est « 45 fois les ventes de poteries, plus 12 fois les ventes d'huile, plus 30 fois les ventes de bijoux ». C'est exactement ce que fait une régression linéaire : un mélange pondéré de colonnes.

```
import numpy as np

V = np.array([[10, 40, 20], # janvier : poteries, huile, bijoux
              [12, 35, 25], # février
              [15, 50, 18]]) # mars
prix = np.array([45, 12, 30])

print("Forme de V :", V.shape)
print("CA par mois :", V @ prix)
```

```
Forme de V : (3, 3)
CA par mois : [1530 1710 1815]
```

L'opérateur `@` est le produit matriciel de Python. Il évite d'écrire une boucle.

Le produit matrice-matrice

Et si Yasmine veut comparer deux grilles de prix : les prix actuels et les prix augmentés de 10 % ? On met les deux grilles côte à côte dans une matrice $\mathbf{P}$ (une colonne par scénario) et on calcule $\mathbf{VP}$.

Règle de calcul. Le produit de $\mathbf{A} \in \mathbb{R}^{n \times p}$ par $\mathbf{B} \in \mathbb{R}^{p \times q}$ est la matrice $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{n \times q}$ dont l'élément (i, j) est le produit scalaire de la **ligne** i de $\mathbf{A}$ avec la **colonne** j de $\mathbf{B}$:

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}.$$

Règle d'or des dimensions. Pour multiplier, le **nombre de colonnes de A doit éga**ler le **nombre de lignes de B** : $(n \times p)(p \times q) \rightarrow (n \times q)$. Les deux p « du milieu » doivent être identiques et disparaissent ; il reste $n \times q$. Quand un code plante avec une erreur de « forme » (*shape*), c'est presque toujours cette règle qui est violée.

```
P = np.array([[45, 49.5],      # poteries : prix actuel, prix +10 %
              [12, 13.2],      # huile
              [30, 33.0]])      # bijoux
print(V @ P)
```

```
[[1530.  1683. ]
 [1710.  1881. ]
 [1815. 1996.5]]
```

Chaque ligne est un mois, chaque colonne un scénario. La deuxième colonne vaut 1,1 fois la première, comme attendu.

Piège : le produit matriciel n'est pas commutatif. En général, $\mathbf{AB} \neq \mathbf{BA}$. Un petit exemple suffit à s'en convaincre :

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Calcul de $\mathbf{AB}$: première ligne $(1 \cdot 0 + 2 \cdot 1, 1 \cdot 1 + 2 \cdot 0) = (2, 1)$; seconde ligne $(3 \cdot 0 + 4 \cdot 1, 3 \cdot 1 + 4 \cdot 0) = (4, 3)$. Donc $\mathbf{AB} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}$: multiplier à droite par $\mathbf{B}$ **échange les colonnes** de $\mathbf{A}$.

Calcul de $\mathbf{BA}$: on trouve $\begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}$: multiplier à gauche par $\mathbf{B}$ **échange les lignes** de $\mathbf{A}$. Les deux résultats sont différents.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])
print("A @ B =\n", A @ B)
print("B @ A =\n", B @ A)
```

```
A @ B =
[[2 1]
 [4 3]]
B @ A =
[[3 4]
 [1 2]]
```

Ce que l'on garde, en revanche : le produit est **associatif** ($(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$) et **distributif** ($\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$).

La transposée

La **transposée** $\mathbf{A}^\top$ d'une matrice échange ses lignes et ses colonnes : $(\mathbf{A}^\top)_{ij} = a_{ji}$. Une matrice 3×2 devient une matrice 2×3 . On a la règle $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ (l'ordre s'inverse).

Pour un vecteur colonne $\mathbf{x}$, le nombre $\mathbf{x}^\top \mathbf{x} = \sum x_i^2 = \|\mathbf{x}\|^2$ est son carré de norme : le produit scalaire s'écrit donc $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b}$. Vous croiserez cette écriture partout.

La matrice identité et l'inverse

La **matrice identité** $\mathbf{I}$ (des 1 sur la diagonale, des 0 ailleurs) est le « 1 » des matrices : $\mathbf{IA} = \mathbf{AI} = \mathbf{A}$.

L'**inverse** d'une matrice carrée $\mathbf{A}$, quand il existe, est la matrice $\mathbf{A}^{-1}$ telle que

$$\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}.$$

C'est l'analogue de $a \times \frac{1}{a} = 1$. Pour une matrice 2×2 , il existe une formule simple. Si $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, son **déterminant** est $\det \mathbf{A} = ad - bc$, et lorsque $\det \mathbf{A} \neq 0$:

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Exemple. Soit $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$. Son déterminant vaut $2 \times 3 - 1 \times 5 = 1$, donc

$$\mathbf{A}^{-1} = \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix}.$$

Vérification : $\mathbf{AA}^{-1} = \begin{pmatrix} 2 \cdot 3 + 1 \cdot (-5) & 2 \cdot (-1) + 1 \cdot 2 \\ 5 \cdot 3 + 3 \cdot (-5) & 5 \cdot (-1) + 3 \cdot 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. ✓

```
A = np.array([[2, 1], [5, 3]])
A_inv = np.linalg.inv(A)
print(A_inv.round(6) + 0)          # le "+ 0" évite l'affichage de -0.
print((A @ A_inv).round(6) + 0)
```

```
[[ 3. -1.]
 [-5.  2.]]
[[1.  0.]
 [0.  1.]]
```

Résoudre un système d'équations linéaires

Voici l'utilité majeure de l'inverse. Yasmine a perdu sa liste de prix, mais retrouve deux factures :

- commande 1 : 2 poteries + 1 huile = 102 DT ;
- commande 2 : 1 poterie + 3 huiles = 81 DT.

Notons x_1 le prix d'une poterie et x_2 celui d'une huile. Le système s'écrit $\mathbf{Mx} = \mathbf{b}$ avec

$$\mathbf{M} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 102 \\ 81 \end{pmatrix}.$$

Résolution à la main. $\det \mathbf{M} = 2 \times 3 - 1 \times 1 = 5$ et $\mathbf{M}^{-1} = \frac{1}{5} \begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}$. Donc

$$\mathbf{x} = \mathbf{M}^{-1}\mathbf{b} = \frac{1}{5} \begin{pmatrix} 3 \cdot 102 - 81 \\ -102 + 2 \cdot 81 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 225 \\ 60 \end{pmatrix} = \begin{pmatrix} 45 \\ 12 \end{pmatrix}.$$

Une poterie coûte 45 DT, une huile 12 DT. Vérifions : $2 \times 45 + 12 = 102$ ✓ et $45 + 3 \times 12 = 81$ ✓.

```
M = np.array([[2, 1], [1, 3]])
b = np.array([102, 81])
print("Prix retrouvés :", np.linalg.solve(M, b))
```

```
Prix retrouvés : [45. 12.]
```

Piège : en pratique, on évite d'inverser. Pour résoudre $\mathbf{Mx} = \mathbf{b}$, on utilise `np.linalg.solve`, jamais `np.linalg.inv(M) @ b` : c'est plus rapide et plus précis (voir la section ④ sur l'analyse numérique). L'inverse est un outil de **raisonnement** ; la machine, elle, résout directement.

Quand le système n'a pas de solution unique : déterminant et rang

Imaginons que la commande 2 soit en réalité « 4 poteries + 2 huiles = 204 DT » : c'est exactement le **double** de la commande 1. Elle n'apporte **aucune information nouvelle**. Il y a alors une infinité de couples de prix qui conviennent. Le déterminant le détecte : $\det \begin{pmatrix} 2 & 1 \\ 4 & 2 \end{pmatrix} = 2 \times 2 - 1 \times 4 = 0$.

On dit que la matrice est **singulière** (non inversible).

Une notion plus générale est le **rang** : le nombre de colonnes (ou de lignes) *linéairement indépendantes*, c'est-à-dire qui ne s'obtiennent pas comme combinaison des autres. Ici le rang vaut 1 au lieu de 2.

```
S = np.array([[2, 1], [4, 2]])
print("déterminant :", np.linalg.det(S))
print("rang       :", np.linalg.matrix_rank(S))
```

```
déterminant : 0.0
rang        : 1
```

Application : des colonnes redondantes. Cette situation est fréquente dans les vraies données. Supposons qu'un fichier contienne le prix hors taxes (HT) *et* le prix toutes taxes comprises (TTC = 1,19 × HT). La seconde colonne est un multiple de la première : elle ne dit rien de plus.

```
ht = np.array([40.0, 25.0, 60.0, 15.0])
table = np.column_stack([ht, 1.19 * ht]) # deux colonnes : HT et TTC
print("rang de la table :", np.linalg.matrix_rank(table), "(alors qu'il y a 2 colonnes)")
```

```
rang de la table : 1 (alors qu'il y a 2 colonnes)
```

Un modèle de régression qui utiliserait ces deux colonnes serait incapable de départager leurs rôles. C'est le problème de la **multicolinéarité**, que nous retrouverons au volume II.

Application : la matrice des similarités entre clients

Revenons à nos clients et à la similarité cosinus. Ajoutons deux clients, Dorra et Elyes, et mettons les cinq profils dans une matrice $\mathbf{X}$ (une ligne par client, une colonne par catégorie).

L'astuce : si l'on **normalise** chaque ligne pour que sa norme vaille 1, on obtient une matrice $\mathbf{U}$ dont chaque ligne est un vecteur de longueur 1. Alors le produit scalaire de deux lignes de $\mathbf{U}$ est directement leur cosinus. Et **tous** les produits scalaires entre lignes sont rassemblés dans le produit $\mathbf{UU}^T$:

$$(\mathbf{UU}^T)_{ij} = \mathbf{u}_i \cdot \mathbf{u}_j = \cos \theta_{ij}.$$

```
clients = {"Amel": [3, 1, 0], "Bilel": [6, 2, 0], "Chaima": [0, 1, 4],
           "Dorra": [1, 0, 5], "Elyes": [2, 2, 2]}
noms = list(clients)
X = np.array(list(clients.values()), dtype=float)      # une ligne par client

U = X / np.linalg.norm(X, axis=1, keepdims=True)      # chaque ligne ramenée à la norme 1
S = U @ U.T                                           # toutes les similarités d'un coup

print(" " * 8 + "".join(f"{n:>8}" for n in noms))
for i, n in enumerate(noms):
    print(f"{n:<8}" + "".join(f"S[{i}, j]:{8.2f}" for j in range(len(noms))))
```

	Amel	Bilel	Chaima	Dorra	Elyes
Amel	1.00	1.00	0.08	0.19	0.73
Bilel	1.00	1.00	0.08	0.19	0.73
Chaima	0.08	0.08	1.00	0.95	0.70
Dorra	0.19	0.19	0.95	1.00	0.68
Elyes	0.73	0.73	0.70	0.68	1.00

Lecture : la diagonale vaut 1 (chacun ressemble parfaitement à lui-même), la matrice est symétrique (la ressemblance d'Amel à Bilel est celle de Bilel à Amel), Amel et Bilel valent 1,00, et Chaima et Dorra, tous deux fans de bijoux, sont très proches. Elyes, qui achète un peu de tout, est moyennement proche de chacun (entre 0,68 et 0,73) sans avoir de « jumeau » parmi eux.

Pour recommander un produit à Chaima, Yasmine pourrait regarder ce qu'achète son voisin le plus proche, ici Dorra. Avec une seule multiplication de matrices, nous avons comparé les cinq clients entre eux ; avec cinq mille clients, ce serait la même ligne de code.

À retenir (matrices).

- Une matrice est un tableau $n \times p$: lignes = individus, colonnes = variables.
- $(\mathbf{AB})_{ij}$ = ligne i de $\mathbf{A}$ · colonne j de $\mathbf{B}$; les dimensions doivent s'emboîter : $(n \times p)(p \times q)$.
- Le produit matriciel n'est **pas commutatif**.
- Résoudre $\mathbf{Mx} = \mathbf{b}$: on utilise `np.linalg.solve`. Si $\det \mathbf{M} = 0$ (rang insuffisant), il n'y a pas de solution unique.
- Des colonnes redondantes font chuter le rang : c'est la source de la multicolinéarité.

1.1.3 Valeurs propres et vecteurs propres : les directions privilégiées

Intuition. Une matrice carrée agit sur le plan comme une machine à déformer : elle étire, elle fait pivoter, elle cisaille. Si vous lui donnez un vecteur, elle en renvoie un autre, en général dans une **autre direction**.

Mais il existe parfois des vecteurs « têtus » : la machine les **étire (ou les écrase) sans les faire tourner**. Ce sont les **vecteurs propres**. Le facteur d'étirement est la **valeur propre**.

Formellement, un vecteur $\mathbf{v} \neq \mathbf{0}$ est un **vecteur propre** de la matrice carrée $\mathbf{A}$ s'il existe un nombre λ (la **valeur propre**) tel que

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}.$$

Un exemple que l'on vérifie à la main

Prenons $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ et essayons trois vecteurs.

Test 1. $\mathbf{v} = (1, 1)$: $\mathbf{A}\mathbf{v} = (2 \cdot 1 + 1 \cdot 1, 1 \cdot 1 + 2 \cdot 1) = (3, 3) = 3 \mathbf{v}$. ✓ La direction est conservée, le vecteur est triplé : **vecteur propre, valeur propre 3**.

Test 2. $\mathbf{w} = (1, -1)$: $\mathbf{A}\mathbf{w} = (2 - 1, 1 - 2) = (1, -1) = 1 \cdot \mathbf{w}$. ✓ **Vecteur propre, valeur propre 1**.

Test 3. $\mathbf{u} = (1, 0)$: $\mathbf{A}\mathbf{u} = (2, 1)$. Ce n'est pas un multiple de $(1, 0)$: la direction a changé. ✗ **Pas un vecteur propre**.

La figure ci-dessous montre ce que fait $\mathbf{A}$ sur tous les vecteurs de longueur 1 (le cercle unité) : elle le transforme en une **ellipse**, allongée d'un facteur 3 dans la direction $(1, 1)$ et conservée (facteur 1) dans la direction $(1, -1)$. Les axes de l'ellipse sont exactement les vecteurs propres.

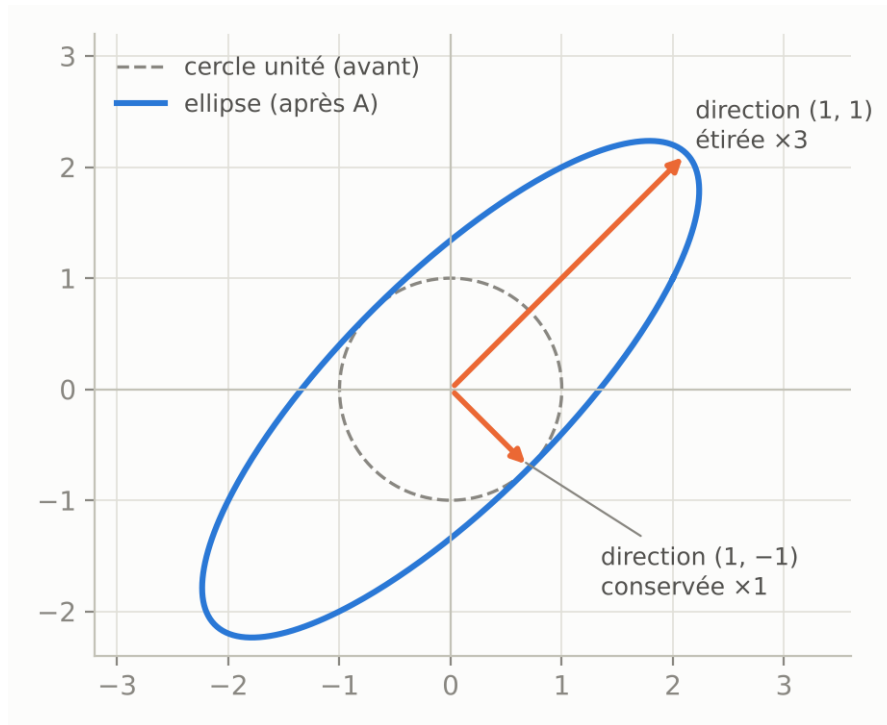


Figure 1: Action de la matrice $\mathbf{A}$ sur le cercle unité : l'ellipse est étirée d'un facteur 3 le long de $(1, 1)$ et inchangée le long de $(1, -1)$.

Comment trouver les valeurs propres ?

On cherche les λ pour lesquels l'équation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ admet une solution non nulle. On la réécrit $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$. Un système homogène a une solution non nulle exactement lorsque sa matrice est **singulière**, c'est-à-dire lorsque son déterminant est nul :

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0.$$

C'est l'**équation caractéristique**. Pour notre matrice :

$$\det \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix} = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3).$$

Les valeurs propres sont donc $\lambda_1 = 3$ et $\lambda_2 = 1$. On trouve ensuite les vecteurs propres en résolvant $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$:

- pour $\lambda = 3$: $\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $v_1 = v_2$, soit $\mathbf{v} = (1, 1)$ (à un multiple près) ;
- pour $\lambda = 1$: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $v_2 = -v_1$, soit $\mathbf{w} = (1, -1)$.

Deux contrôles rapides. La **somme** des valeurs propres vaut la **trace** de la matrice (somme de la diagonale) : $3 + 1 = 4 = 2 + 2$. Le **produit** des valeurs propres vaut le **déterminant** : $3 \times 1 = 3 = 2 \cdot 2 - 1 \cdot 1$. Utile pour détecter une erreur de calcul.

```
import numpy as np

A = np.array([[2, 1], [1, 2]])
valeurs, vecteurs = np.linalg.eigh(A)      # eigh : pour les matrices symétriques
print("valeurs propres :", valeurs)
print("vecteurs propres (en colonnes) :\n", vecteurs.round(4))
print("trace =", np.trace(A), " somme des valeurs propres =", valeurs.sum())
print("det  =", round(np.linalg.det(A), 4), " produit des valeurs propres =", valeurs.prod())
```

```
valeurs propres : [1. 3.]
vecteurs propres (en colonnes) :
[[-0.7071  0.7071]
 [ 0.7071  0.7071]]
trace = 4  somme des valeurs propres = 4.0
det  = 3.0  produit des valeurs propres = 3.0
```

NumPy range les valeurs propres par ordre croissant et renvoie des vecteurs **de longueur 1** (le signe est arbitraire : si $\mathbf{v}$ est propre, $-\mathbf{v}$ l'est aussi).

Le cas des matrices symétriques

Une matrice est **symétrique** si $\mathbf{A}^\top = \mathbf{A}$ (la matrice de similarité de la section précédente en est une, et la matrice de covariance, que nous rencontrerons dans un instant, aussi). Ces matrices ont une propriété exceptionnelle :

Théorème spectral. Une matrice symétrique réelle de taille $p \times p$ a p valeurs propres **réelles** et on peut choisir pour elle p vecteurs propres **orthogonaux** deux à deux (et de longueur 1). Si $\mathbf{V}$ est la matrice qui les contient en colonnes et Λ la matrice diagonale des valeurs propres, alors

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^\top.$$

Nous admettons l'existence (la preuve complète se trouve dans tout cours d'algèbre linéaire), mais voici pourquoi les vecteurs propres associés à des valeurs propres différentes sont orthogonaux :

Démonstration : orthogonalité. Soient $\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ et $\mathbf{A}\mathbf{v}_2 = \lambda_2\mathbf{v}_2$ avec $\lambda_1 \neq \lambda_2$ et $\mathbf{A}$ symétrique. Calculons de deux façons le nombre $(\mathbf{A}\mathbf{v}_1) \cdot \mathbf{v}_2$:

- directement : $(\mathbf{A}\mathbf{v}_1) \cdot \mathbf{v}_2 = \lambda_1 (\mathbf{v}_1 \cdot \mathbf{v}_2)$;
- en déplaçant $\mathbf{A}$: comme $\mathbf{A}$ est symétrique, $(\mathbf{A}\mathbf{v}_1) \cdot \mathbf{v}_2 = \mathbf{v}_1^\top \mathbf{A}^\top \mathbf{v}_2 = \mathbf{v}_1^\top \mathbf{A} \mathbf{v}_2 = \mathbf{v}_1 \cdot (\mathbf{A}\mathbf{v}_2) = \lambda_2 (\mathbf{v}_1 \cdot \mathbf{v}_2)$.

En soustrayant : $(\lambda_1 - \lambda_2) (\mathbf{v}_1 \cdot \mathbf{v}_2) = 0$. Comme $\lambda_1 \neq \lambda_2$, on a $\mathbf{v}_1 \cdot \mathbf{v}_2 = 0$: les vecteurs sont orthogonaux. ■

Dans notre exemple : $(1, 1) \cdot (1, -1) = 1 - 1 = 0$. ✓

Une conséquence pratique de $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$ est que les **puissances** deviennent triviales : $\mathbf{A}^k = \mathbf{V}\mathbf{\Lambda}^k\mathbf{V}^\top$, et élever une matrice diagonale à la puissance k , c'est simplement élever ses éléments. (Nous nous en servons pour les chaînes de Markov dans la section $\oplus$ du chapitre 2.)

```
k = 5
via_diagonalisation = vecteurs @ np.diag(valeurs**k) @ vecteurs.T
directement = np.linalg.matrix_power(A, k)
print("A^5 par diagonalisation :\n", via_diagonalisation.round(6))
print("A^5 directement :\n", directement)
```

```
A^5 par diagonalisation :
[[122. 121.]
 [121. 122.]]
A^5 directement :
[[122 121]
 [121 122]]
```

■ Application : la direction principale d'un nuage de clients

Yasmine mesure, pour 200 clients, le nombre de visites mensuelles sur son site et leur dépense mensuelle. Elle soupçonne que les deux sont liées. On **standardise** chaque variable (on retranche la moyenne et on divise par l'écart-type, pour qu'elles aient la même échelle, comme promis dans la section sur les vecteurs), puis on calcule la **matrice de covariance** des deux variables standardisées :

$$\mathbf{C} = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix},$$

où r est la corrélation entre les deux variables. C'est une matrice symétrique : le théorème spectral s'applique.

```
rng = np.random.default_rng(42)
n = 200
visites = rng.normal(6, 2, n) # visites par mois
depense = 15 * visites + rng.normal(0, 12, n) # dépense en DT, liée aux visites

def standardise(x):
    return (x - x.mean()) / x.std(ddof=1)

Z = np.column_stack([standardise(visites), standardise(depense)])
C = np.cov(Z.T) # matrice de covariance 2 x 2
print("Matrice de covariance :\n", C.round(3))

valeurs, vecteurs = np.linalg.eigh(C)
print("Valeurs propres :", valeurs.round(3))
print("Part de la variance expliquée :", (valeurs / valeurs.sum()).round(3))
print("Direction principale :", np.abs(vecteurs[:, -1]).round(3))
```

```
Matrice de covariance :
[[1. 0.903]
 [0.903 1. ]]
Valeurs propres : [0.097 1.903]
Part de la variance expliquée : [0.049 0.951]
Direction principale : [0.707 0.707]
```

Lecture. La plus grande valeur propre (environ 1,90) correspond à la direction (0,71; 0,71), la diagonale : c'est l'axe le long duquel le nuage de clients est le plus étiré, l'axe « client actif et dépensier ». Elle capte environ 95 % de toute la variabilité. La seconde (0,10) correspond à la

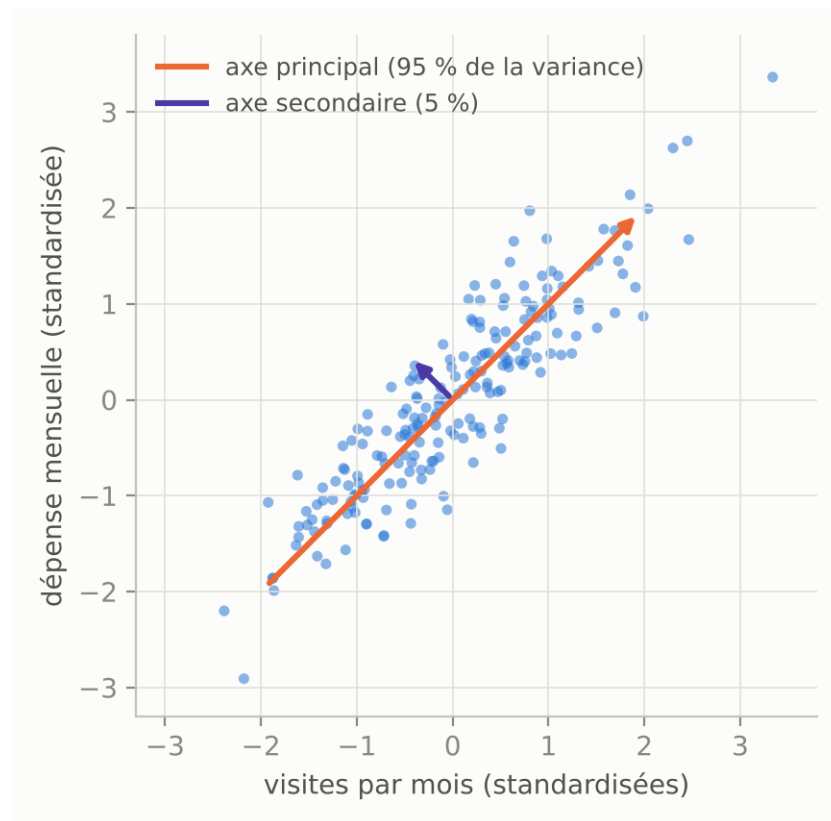


Figure 2: Nuage des 200 clients (variables standardisées) et son axe principal : le long de la diagonale, le nuage est étiré ; perpendiculairement, il est mince.

direction perpendiculaire, qui ne représente que les écarts « dépense inhabituelle pour ce nombre de visites ».

Autrement dit, on peut résumer ces deux variables par **une seule** (la position le long de l'axe principal), en ne perdant que 5 % de l'information. C'est le principe de l'**analyse en composantes principales** (ACP), que nous étudierons en détail au volume II : *trouver les directions de plus grande variance, ce sont les vecteurs propres de la matrice de covariance*.

À retenir (valeurs propres).

- $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$: un vecteur propre garde sa direction, la valeur propre dit de combien il est étiré.
- On les trouve avec $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. Somme des valeurs propres = trace ; produit = déterminant.
- Une matrice symétrique a des valeurs propres réelles et des vecteurs propres orthogonaux ; $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$.
- Les vecteurs propres de la matrice de covariance sont les axes principaux d'un nuage de données.

1.1.4 La décomposition en valeurs singulières (SVD)

Les valeurs propres ne sont définies que pour des matrices **carrées**. Or un jeu de données est rarement carré : n individus, p variables, avec $n \neq p$. La **décomposition en valeurs singulières**, ou **SVD** (*Singular Value Decomposition*), étend la même idée à **n'importe quelle matrice**. C'est l'un des outils les plus puissants de toute la data science.

Intuition. Toute matrice, même rectangulaire, agit en trois temps : **une rotation, un étirement le long des axes, puis une autre rotation.** On écrit :

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top.$$

Les matrices $\mathbf{U}$ et $\mathbf{V}$ sont des rotations (leurs colonnes sont des vecteurs de longueur 1, orthogonaux entre eux) ; $\mathbf{\Sigma}$ est « diagonale » et contient les facteurs d'étirement $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$, appelés **valeurs singulières**.

On peut aussi lire la SVD comme une **somme de couches simples** (« de rang 1 »), classées de la plus importante à la moins importante :

$$\mathbf{A} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top + \dots + \sigma_r \mathbf{u}_r \mathbf{v}_r^\top.$$

Chaque couche $\mathbf{u}_i \mathbf{v}_i^\top$ est un tableau « produit » : (profil des lignes) $\times$ (profil des colonnes). La première couche, celle de plus grande valeur singulière, capture la structure dominante des données ; les suivantes en sont les raffinements.

Un exemple minuscule

Exemple à la main. Prenons $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ (rang 1 : les deux lignes sont identiques). Calculons $\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$. Ses valeurs propres sont 4 et 0 (trace 4, déterminant 0). La plus grande valeur singulière est donc $\sigma_1 = \sqrt{4} = 2$ et la seconde est $\sigma_2 = 0$. Le vecteur propre associé à 4 est $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1)$, et $\mathbf{u}_1 = \mathbf{A}\mathbf{v}_1/\sigma_1 = \frac{1}{\sqrt{2}}(1, 1)$. On vérifie :

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 2 \cdot \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \mathbf{A}. \checkmark$$

Une seule valeur singulière non nulle : la matrice n'a qu'**une** couche. Plus généralement, le **nombre de valeurs singulières non nulles est le rang** de la matrice.

Démonstration : d'où viennent les valeurs singulières ? (l'idée de la construction)

Soit $\mathbf{A} \in \mathbb{R}^{n \times p}$. La matrice $\mathbf{A}^\top \mathbf{A}$ est carrée ($p \times p$), **symétrique**, et **positive** : pour tout vecteur $\mathbf{x}$, $\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} = \|\mathbf{A}\mathbf{x}\|^2 \geq 0$. Le théorème spectral nous donne donc des vecteurs propres orthonormés $\mathbf{v}_1, \dots, \mathbf{v}_p$ associés à des valeurs propres $\lambda_i \geq 0$ (elles sont positives car $\lambda_i = \mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_i = \|\mathbf{A}\mathbf{v}_i\|^2$).

On pose $\sigma_i = \sqrt{\lambda_i}$ et, pour $\sigma_i > 0$, $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$. Ces vecteurs $\mathbf{u}_i$ sont **orthonormés** :

$$\mathbf{u}_i \cdot \mathbf{u}_j = \frac{(\mathbf{A}\mathbf{v}_i)^\top (\mathbf{A}\mathbf{v}_j)}{\sigma_i \sigma_j} = \frac{\mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_j}{\sigma_i \sigma_j} = \frac{\lambda_j \mathbf{v}_i \cdot \mathbf{v}_j}{\sigma_i \sigma_j} = \begin{cases} 1 & i = j \\ 0 & i \neq j. \end{cases}$$

Enfin, par construction $\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$: c'est exactement l'égalité $\mathbf{A}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$, qui donne $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. ■

Autrement dit : **les valeurs singulières de $\mathbf{A}$ sont les racines carrées des valeurs propres de $\mathbf{A}^\top \mathbf{A}$.** Le lien avec la section précédente est direct.

Approximer une matrice par un nombre réduit de couches

Voici la propriété qui rend la SVD si utile en pratique.

Théorème (Eckart-Young). Parmi toutes les matrices de rang k , la meilleure approximation de $\mathbf{A}$ (au sens de la somme des carrés des erreurs) est obtenue en **ne gardant que les k premières couches** de la SVD :

$$\mathbf{A}_k = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \dots + \sigma_k \mathbf{u}_k \mathbf{v}_k^\top.$$

L'erreur vaut alors $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$, où $\|\cdot\|_F$ est la racine de la somme des carrés de tous les éléments.

Autrement dit : si les premières valeurs singulières sont grandes et les suivantes minuscules, on peut **jeter** les dernières couches sans presque rien perdre. C'est de la **compression**.

■ Application : résumer un tableau de ventes

Yasmine a les ventes hebdomadaires de 6 produits sur 8 semaines. Les données sont simulées selon une règle simple : *ventes = popularité du produit × effet de la semaine + un peu de bruit*. Une telle structure « produit × saison » doit se retrouver dans la première couche de la SVD.

```
rng = np.random.default_rng(7)
popularite = np.array([50, 30, 20, 12, 8, 5.0])          # un niveau par produit
saison = np.array([1.0, 1.1, 0.9, 1.2, 1.5, 1.4, 1.0, 0.8]) # un effet par semaine
M = (np.outer(popularite, saison) + rng.normal(0, 1.0, (6, 8))).round(0)

print("Tableau des ventes (6 produits x 8 semaines) :")
print(M.astype(int))
```

Tableau des ventes (6 produits x 8 semaines) :

```
[[50 55 45 59 75 69 50 41]
 [30 32 27 36 45 41 30 25]
 [19 22 16 23 28 28 19 16]
 [12 13  8 14 18 17 10  9]
 [ 7  8  8  9 12 12  7  6]
 [ 5  6  3  6  9  5  6  4]]
```

```
U, s, Vt = np.linalg.svd(M)
print("valeurs singulières :", s.round(2))
part = 100 * s**2 / (s**2).sum()
print("part de l'énergie par couche (%) :", " ".join(f"{x:.2f}" for x in part))

M1 = s[0] * np.outer(U[:, 0], Vt[0])          # approximation avec UNE seule couche
erreur_relative = np.linalg.norm(M - M1) / np.linalg.norm(M)
print("erreur relative de l'approximation de rang 1 :", round(erreur_relative, 4))
print("nombres à stocker : 48 (tableau) contre", 6 + 8 + 1, "(couche 1)")
```

```
valeurs singulières : [202.05  3.53  3.51  1.81  1.18  0.75]
part de l'énergie par couche (%) : 99.93 0.03 0.03 0.01 0.00 0.00
erreur relative de l'approximation de rang 1 : 0.0271
nombres à stocker : 48 (tableau) contre 15 (couche 1)
```

Lecture. La première valeur singulière (environ 202) est près de soixante fois plus grande que la deuxième (environ 3,5) : **99,9 %** de l'« énergie » du tableau est dans une seule couche. Une approximation de rang 1, qui ne stocke que 15 nombres au lieu de 48, reproduit le tableau avec une erreur relative d'environ 2,7 %. Le reste (les couches 2 à 6) est du bruit.

La SVD a donc **retrouvé toute seule** la structure « popularité × saison » que nous avions mise dans les données, sans qu'on lui dise de la chercher. Vous venez de voir le principe de la **réduction de**

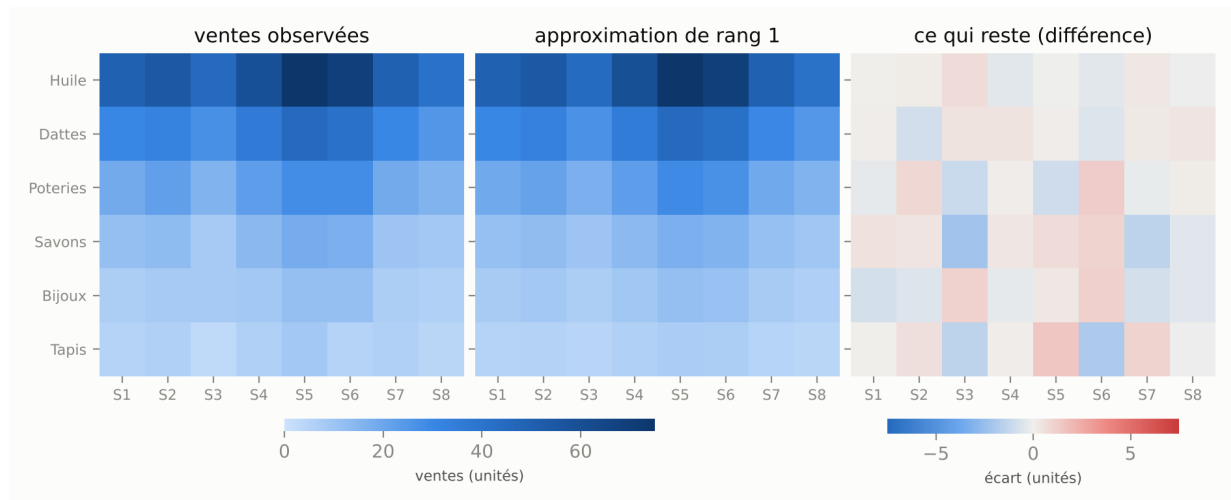


Figure 3: Ventes observées (à gauche), approximation par une seule couche (au centre) et ce qui reste (à droite). L'échelle de couleur est la même pour les deux premiers panneaux.

dimension et celui des **systèmes de recommandation** par factorisation de matrices, deux sujets que nous développerons aux volumes suivants.

À retenir (SVD).

- Toute matrice se décompose en $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$: rotation, étirement, rotation.
- Les valeurs singulières mesurent l'importance de chaque couche ; leur nombre non nul est le rang.
- Garder les k premières couches donne la meilleure approximation de rang k (Eckart-Young) : c'est la base de la compression, du débruitage et de l'ACP.
- L'ACP n'est rien d'autre que la SVD des données centrées.

1.2 Analyse : mesurer le changement

L'algèbre linéaire nous a appris à décrire des données. L'**analyse** nous apprend à décrire comment les choses **changent**. C'est ce qui permet de répondre à des questions comme : « si j'augmente mon prix d'un dinar, mes recettes montent-elles ou descendent-elles, et de combien ? ». Elle repose sur deux notions : la **dérivée** (le changement instantané) et l'**intégrale** (le changement accumulé).

1.2.1 La dérivée : la vitesse à laquelle une fonction change

Intuition. Si une fonction décrit **où vous en êtes**, sa dérivée décrit **à quelle vitesse vous avancez**. Dans une voiture, la distance parcourue est la fonction ; le compteur de vitesse affiche sa dérivée. Graphiquement, la dérivée en un point est la **pente de la tangente** à la courbe en ce point.

Une fonction pour commencer

Une **fonction** associe à chaque valeur d'entrée x une valeur de sortie $f(x)$. Voici celle que nous utiliserons dans cette section : le bénéfice hebdomadaire de Yasmine pour une poterie, en fonction du nombre q de pièces vendues. Plus elle en vend, plus elle doit baisser le prix, et il y a 300 DT de frais fixes :

$$P(q) = -2q^2 + 80q - 300.$$

Par exemple, $P(20) = -2 \cdot 400 + 1600 - 300 = 500$ DT.

D'abord, une idée de limite

Pour définir la dérivée, il faut la notion de **limite** : la valeur *vers laquelle tend* une quantité quand on s'approche d'un point, même si l'on ne peut pas l'atteindre.

Exemple. Considérons $f(x) = \frac{x^2 - 1}{x - 1}$. Elle n'est pas définie en $x = 1$ (division par zéro). Mais que se passe-t-il quand x s'en approche ?

```
import numpy as np
```

```
def f(x):
    return (x**2 - 1) / (x - 1)
```

```
for x in [0.9, 0.99, 0.999, 1.001, 1.01, 1.1]:
    print(f"x = {x:<6} f(x) = {f(x):.4f}")
```

```
x = 0.9      f(x) = 1.9000
x = 0.99     f(x) = 1.9900
x = 0.999    f(x) = 1.9990
x = 1.001    f(x) = 2.0010
x = 1.01     f(x) = 2.0100
x = 1.1      f(x) = 2.1000
```

On voit que $f(x)$ se rapproche de 2. On écrit $\lim_{x \rightarrow 1} f(x) = 2$. (Algébriquement, $x^2 - 1 = (x - 1)(x + 1)$, donc $f(x) = x + 1$ dès que $x \neq 1$.) Une limite décrit un comportement **au voisinage** d'un point, pas au point lui-même.

Définition de la dérivée

Comment mesurer la pente d'une courbe en un seul point ? Une pente se mesure entre deux points. L'idée est donc de prendre deux points très proches, x et $x + h$, et de regarder la pente de la droite qui les relie : le **taux d'accroissement**

$$\frac{f(x + h) - f(x)}{h}.$$

Puis de faire tendre h vers zéro. La **dérivée** est la limite, quand elle existe :

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}.$$

Exemple chiffré. Prenons $f(x) = x^2$ au point $x = 3$ et rapprochons h de zéro :

```
def carre(x):
    return x**2
```

```
for h in [1, 0.1, 0.01, 0.001, 0.0001]:
    taux = (carre(3 + h) - carre(3)) / h
    print(f"h = {h:<7} taux d'accroissement = {taux:.4f}")
```

```

h = 1      taux d'accroissement = 7.0000
h = 0.1    taux d'accroissement = 6.1000
h = 0.01   taux d'accroissement = 6.0100
h = 0.001  taux d'accroissement = 6.0010
h = 0.0001 taux d'accroissement = 6.0001

```

Les valeurs se rapprochent de 6. Et $2 \times 3 = 6$: la dérivée de x^2 semble être $2x$. Prouvons-le.

Démonstration : la dérivée de x^2 est $2x$.

$$\frac{(x+h)^2 - x^2}{h} = \frac{x^2 + 2xh + h^2 - x^2}{h} = \frac{2xh + h^2}{h} = 2x + h.$$

Quand $h \rightarrow 0$, cette quantité tend vers $2x$. Donc $(x^2)' = 2x$. ■

Notez que les calculs numériques ci-dessus suivent exactement ce résultat : le taux vaut $2 \cdot 3 + h = 6 + h$, soit 7, 6,1, 6,01, 6,001, 6,0001.

Les règles de calcul

On ne recalcule pas une limite à chaque fois. On utilise des règles, qu'il faut connaître par cœur :

Fonction	Dérivée	Exemple
constante c	0	$(7)' = 0$
x^n	$n x^{n-1}$	$(x^3)' = 3x^2$; $(\sqrt{x})' = \frac{1}{2\sqrt{x}}$
e^x	e^x	
$\ln x$	$\frac{1}{x}$	
somme $f + g$	$f' + g'$	$(x^2 + 5x)' = 2x + 5$
multiple cf	cf'	$(4x^3)' = 12x^2$
produit fg	$f'g + fg'$	$(x e^x)' = e^x + x e^x$
composée $f(g(x))$	$f'(g(x)) \cdot g'(x)$	voir ci-dessous

La dernière règle, celle de la **dérivée d'une composée** (ou « règle de la chaîne »), est la plus importante pour la suite : c'est elle qui fait fonctionner l'entraînement de tous les réseaux de neurones. Elle dit : *dérivez l'extérieur sans toucher à l'intérieur, puis multipliez par la dérivée de l'intérieur*.

Exemple. $h(x) = (3x + 1)^2$. L'extérieur est « carré », l'intérieur est $3x + 1$.

$$h'(x) = \underbrace{2(3x+1)}_{\text{dérivée de l'extérieur}} \times \underbrace{3}_{\text{dérivée de l'intérieur}} = 6(3x+1) = 18x + 6.$$

Contrôle : en développant, $h(x) = 9x^2 + 6x + 1$, dont la dérivée est $18x + 6$. ✓

Autre exemple, qui servira bientôt : $\frac{d}{dx} e^{-0,5x} = -0,5 e^{-0,5x}$ (l'intérieur est $-0,5x$, de dérivée $-0,5$).

Pour **vérifier** une dérivée calculée à la main (ou dénicher une erreur), on peut la comparer à une dérivée numérique : on calcule la pente entre $x - h$ et $x + h$ pour un très petit h (la « différence centrée »).

```

def derivee_numerique(f, x, h=1e-6):
    return (f(x + h) - f(x - h)) / (2 * h)

tests = [

```

```

("x^3 en x = 2", lambda x: x**3, 2.0, 3 * 2.0**2),
("(3x+1)^2 en x = 1", lambda x: (3 * x + 1)**2, 1.0, 6 * (3 * 1.0 + 1)),
("exp(-0.5x) en x = 2", lambda x: np.exp(-0.5 * x), 2.0, -0.5 * np.exp(-1.0)),
("ln(2x) en x = 4", lambda x: np.log(2 * x), 4.0, 1 / 4.0),
]
for nom, fonction, x0, exacte in tests:
    print(f"{nom:<20} numérique = {derivee_numerique(fonction, x0):9.6f} formule = {exacte:9.6f}")

```

x^3 en x = 2	numérique = 12.000000	formule = 12.000000
(3x+1)^2 en x = 1	numérique = 24.000000	formule = 24.000000
exp(-0.5x) en x = 2	numérique = -0.183940	formule = -0.183940
ln(2x) en x = 4	numérique = 0.250000	formule = 0.250000

Bonne habitude. Chaque fois que vous dérivez une expression compliquée, comparez-la à une dérivée numérique. Les erreurs de signe et les oublis de « fois la dérivée de l'intérieur » sont les fautes les plus courantes.

■ Application : à quel niveau de ventes le bénéfice est-il maximal ?

Reprenons $P(q) = -2q^2 + 80q - 300$. Observons-la d'abord :

```

def benefice(q):
    return -2 * q**2 + 80 * q - 300

for q in [10, 15, 20, 25, 30]:
    print(f"q = {q:>2}    bénéfice = {benefice(q):>4} DT")

```

q = 10	bénéfice = 300 DT
q = 15	bénéfice = 450 DT
q = 20	bénéfice = 500 DT
q = 25	bénéfice = 450 DT
q = 30	bénéfice = 300 DT

Le bénéfice monte, atteint un sommet, puis redescend : c'est une parabole « en cloche ». **Au sommet, la tangente est horizontale, donc la dérivée est nulle.** C'est la clé de l'optimisation. On dérive :

$$P'(q) = -4q + 80.$$

On résout $P'(q) = 0$: $-4q + 80 = 0$, donc $q = 20$ pièces, pour un bénéfice $P(20) = 500$ DT.

La dérivée a aussi une lecture économique directe : c'est le **bénéfice marginal**, le gain approximatif que rapporte la pièce supplémentaire. En $q = 10$, $P'(10) = -40 + 80 = 40$ DT. Vérifions avec la vraie différence :

```

print("gain réel de la 11e pièce : P(11) - P(10) =", benefice(11) - benefice(10), "DT")
print("pente en q = 10          : P'(10) = -4*10 + 80 =", -4 * 10 + 80, "DT")

```

gain réel de la 11e pièce : P(11) - P(10) = 38 DT
pente en q = 10 : P'(10) = -4*10 + 80 = 40 DT

La pente (40) est une bonne approximation du gain réel (38) : elle est exacte pour une variation infiniment petite, et approchée pour une variation d'une pièce. Enfin, confirmons le sommet par une recherche numérique, sans utiliser la dérivée :

```

from scipy.optimize import minimize_scalar

res = minimize_scalar(lambda q: -benefice(q), bounds=(0, 40), method="bounded")

```

```
print(f"optimum numérique : q = {res.x:.3f}, bénéfice = {-res.fun:.2f} DT")
```

```
optimum numérique : q = 20.000, bénéfice = 500.00 DT
```

(On minimise $-P$ puisque la fonction cherche un minimum : maximiser P revient à minimiser $-P$.)

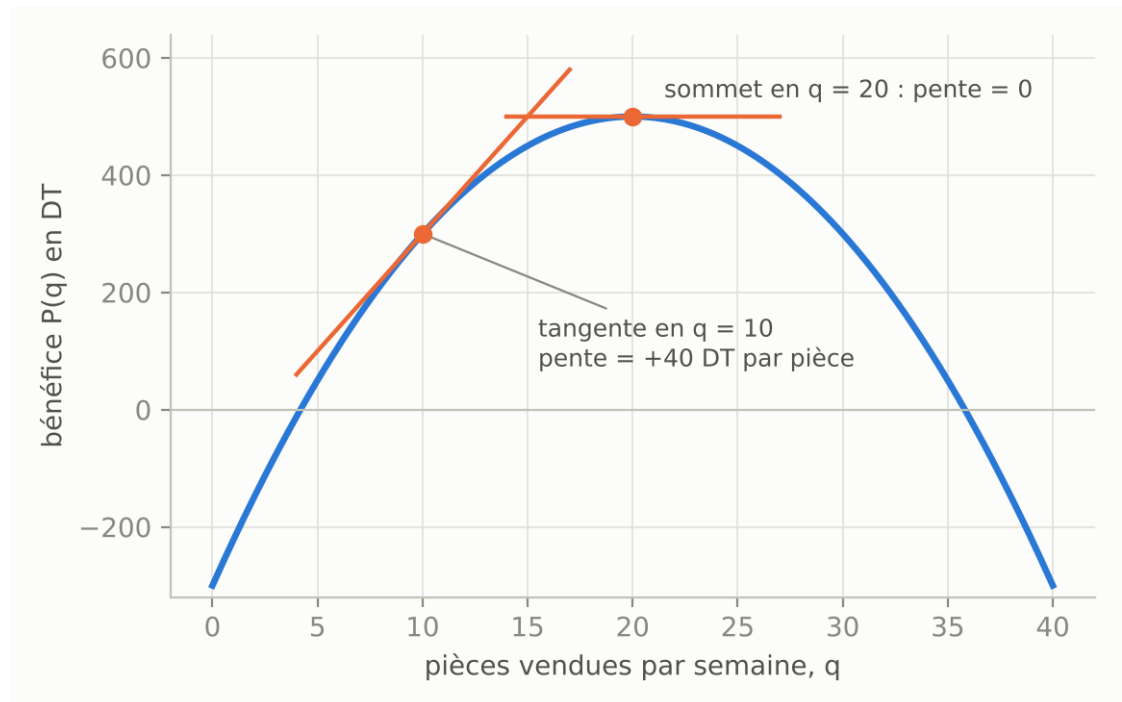


Figure 4: Bénéfice hebdomadaire en fonction du nombre de pièces vendues. En $q = 10$ la tangente monte avec une pente de 40 ; en $q = 20$ elle est horizontale : c'est le sommet.

Comment savoir si c'est un maximum ou un minimum ?

Une dérivée nulle signale un point **candidat**, pas forcément un maximum. On regarde alors la **dérivée seconde** f'' (la dérivée de la dérivée, qui mesure comment la pente elle-même change) :

- $f''(x) < 0$: la pente décroît, la courbe est une « bosse », c'est un **maximum local** ;
- $f''(x) > 0$: la pente croît, la courbe est un « creux », c'est un **minimum local** ;
- $f''(x) = 0$: on ne peut pas conclure.

Ici $P''(q) = -4 < 0$: c'est bien un maximum.

Piège. « Dérivée nulle » ne veut pas dire « extremum ». Pour $f(x) = x^3$, on a $f'(0) = 3 \cdot 0^2 = 0$, et pourtant la fonction ne fait que croître : en 0, la tangente est horizontale mais la courbe continue de monter (c'est un *point d'inflexion*). Vérifiez toujours avec la dérivée seconde ou en comparant les valeurs de part et d'autre.

À retenir (dérivée).

- $f'(x)$ est la pente de la tangente en x : c'est le taux de variation instantané.
- Les cinq règles à maîtriser : puissances, somme, multiple, produit, composée.
- Pour optimiser : on cherche $f'(x) = 0$, puis on contrôle avec f'' .
- Comparer une dérivée à la main à une dérivée numérique permet de détecter les er-

reurs.

1.2.2 Dérivées partielles et gradient : plusieurs variables à la fois

Un modèle de data science a rarement un seul paramètre : il en a deux, dix, ou un milliard. Il nous faut donc dériver des fonctions de **plusieurs variables**.

Intuition. Imaginez que vous êtes debout sur le flanc d'une colline brumeuse et que vous voulez monter le plus vite possible. À vos pieds, le sol penche plus dans une direction que dans les autres. Le **gradient** est la flèche qui indique **la direction de la plus forte montée**, et sa longueur dit à quel point c'est raide. Pour descendre, on marche dans la direction opposée.

Dérivée partielle

Pour une fonction $f(x, y)$, la **dérivée partielle par rapport à x** , notée $\frac{\partial f}{\partial x}$, s'obtient en dérivant par rapport à x **en traitant y comme une constante**. De même pour y .

Exemple. Soit $f(x, y) = x^2 + 3y^2$ (un « bol » plus raide dans la direction y).

$$\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 6y.$$

(Pour $\partial f / \partial x$, le terme $3y^2$ est une constante : sa dérivée est 0.)

Le **gradient** est le vecteur qui rassemble toutes les dérivées partielles :

$$\nabla f(x, y) = \begin{pmatrix} \partial f / \partial x \\ \partial f / \partial y \end{pmatrix} = \begin{pmatrix} 2x \\ 6y \end{pmatrix}.$$

En $(1, 1)$, on obtient $\nabla f(1, 1) = (2, 6)$: la pente est 3 fois plus forte dans la direction y que dans la direction x .

Pourquoi le gradient indique-t-il la plus forte montée ?

Dans une direction donnée par un vecteur $\mathbf{u}$ de longueur 1, la pente de f (la **dérivée directionnelle**) vaut

$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u}.$$

Et ce produit scalaire, nous savons le majorer.

Démonstration. Par la formule du produit scalaire vue en 1.1.1,

$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \|\mathbf{u}\| \cos \theta = \|\nabla f\| \cos \theta,$$

où θ est l'angle entre $\mathbf{u}$ et le gradient. Cette quantité est maximale quand $\cos \theta = 1$, c'est-à-dire quand $\mathbf{u}$ pointe **dans la même direction que le gradient**, et alors la pente vaut $\|\nabla f\|$. Elle est minimale (la descente la plus forte, de pente $-\|\nabla f\|$) quand $\theta = 180^\circ$, c'est-à-dire dans la direction $-\nabla f$. ■

(Au passage : on retrouve la même inégalité de Cauchy-Schwarz que dans la section 1.1.1.)

Vérifions numériquement en $(1, 1)$, en mesurant la pente de f dans quatre directions :

```
def f2(x, y):
    return x**2 + 3 * y**2

def gradient_f2(x, y):
    return np.array([2 * x, 6 * y])

g = gradient_f2(1, 1)
print("gradient en (1, 1) :", g, "   norme =", round(np.linalg.norm(g), 4))

h = 1e-6
directions = [
    ("direction (1, 0)", np.array([1.0, 0.0])),
    ("direction (0, 1)", np.array([0.0, 1.0])),
    ("direction du gradient", g / np.linalg.norm(g)),
    ("opposée au gradient", -g / np.linalg.norm(g))
]
for nom, u in directions:
    pente = (f2(1 + h * u[0], 1 + h * u[1]) - f2(1, 1)) / h
    print(f"{nom:<24} pente mesurée = {pente:8.3f}")
```

```
gradient en (1, 1) : [2 6]   norme = 6.3246
direction (1, 0)      pente mesurée = 2.000
direction (0, 1)      pente mesurée = 6.000
direction du gradient  pente mesurée = 6.325
opposée au gradient   pente mesurée = -6.325
```

La pente mesurée est maximale (6,325, soit $\|\nabla f\| = \sqrt{40}$) dans la direction du gradient, et minimale ($-6,325$) dans la direction opposée. Aucune autre direction ne fait mieux.

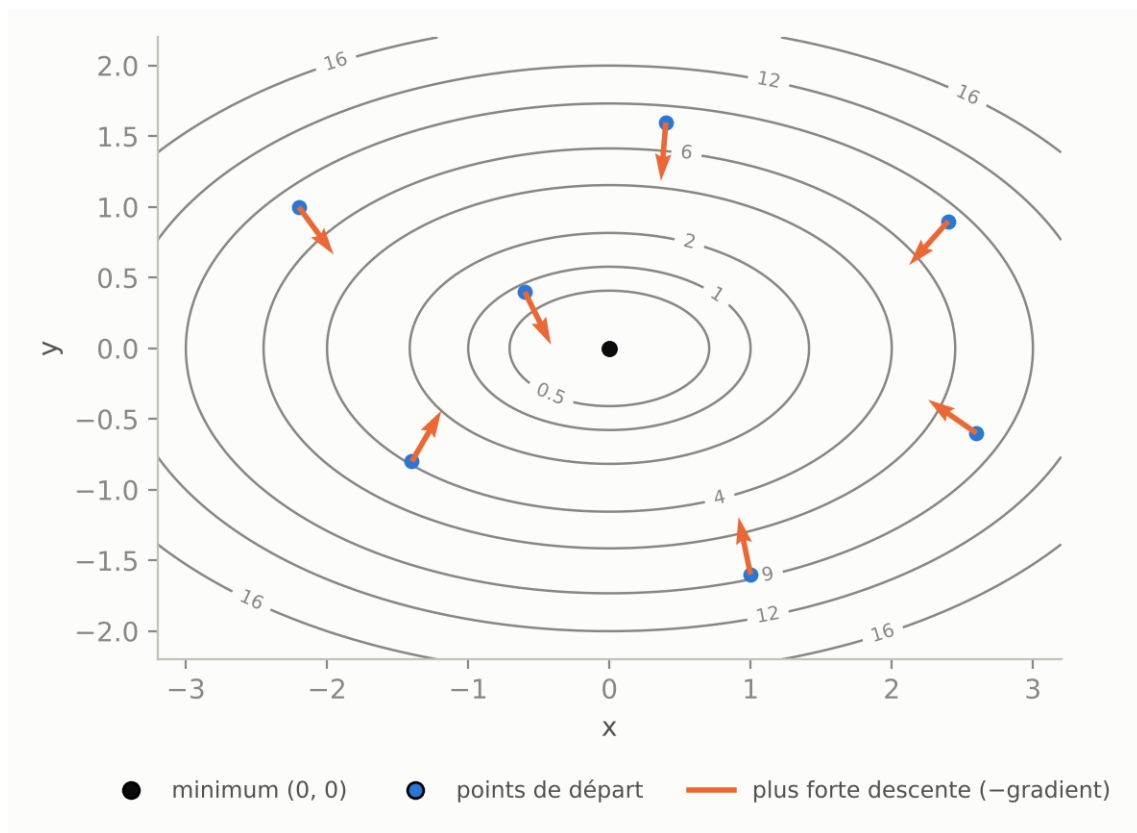


Figure 5: Courbes de niveau de $f(x,y) = x^2 + 3y^2$ (chaque ellipse est une « altitude » constante) et, en chaque point, la direction de la plus forte descente ($-\text{gradient}$). Les flèches sont perpendiculaires aux courbes de niveau et pointent vers le fond du bol.

Deux faits à retenir sur cette figure : le gradient est **perpendiculaire aux courbes de niveau**, et **−gradient pointe vers le bas**. C'est exactement ce que nous utiliserons en 1.3 pour descendre vers le minimum.

■ Application : la pente de l'erreur d'une droite

Yasmine veut ajuster une droite $y = ax + b$ à trois mesures (x_i, y_i) : (1,2), (2,3) et (3,5). Pour mesurer la qualité d'une droite candidate, on utilise la **somme des carrés des erreurs** :

$$L(a, b) = \sum_{i=1}^3 (y_i - (ax_i + b))^2.$$

C'est une fonction de **deux** variables, a et b . Notons $r_i = y_i - (ax_i + b)$ l'erreur du point i . Par la règle de la composée :

$$\frac{\partial L}{\partial a} = -2 \sum_i x_i r_i, \quad \frac{\partial L}{\partial b} = -2 \sum_i r_i.$$

Calcul à la main en $(a, b) = (0, 0)$ (la droite $y = 0$). Les erreurs sont $r = (2, 3, 5)$.

- $L(0, 0) = 4 + 9 + 25 = 38$.
- $\partial L / \partial a = -2 (1 \cdot 2 + 2 \cdot 3 + 3 \cdot 5) = -2 \times 23 = -46$.
- $\partial L / \partial b = -2 (2 + 3 + 5) = -20$.

Le gradient $(-46, -20)$ est très négatif : en augmentant a et b , l'erreur diminue fortement.

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 3.0, 5.0])

def perte(a, b):
    return np.sum((y - (a * x + b))**2)

def grad_perte(a, b):
    r = y - (a * x + b)
    return np.array([-2 * np.sum(x * r), -2 * np.sum(r)])

print("perte en (0, 0)      :", perte(0, 0))
print("gradient en (0, 0)  :", grad_perte(0, 0))

# la droite des moindres carrés (calculée en 1.3) : a = 1,5 et b = 1/3
a_opt, b_opt = 1.5, 1 / 3
print("perte en (1,5 ; 1/3) :", round(perte(a_opt, b_opt), 4))
print("gradient en (1,5 ; 1/3) :", (grad_perte(a_opt, b_opt).round(10) + 0))
```

```
perte en (0, 0)      : 38.0
gradient en (0, 0)   : [-46. -20.]
perte en (1,5 ; 1/3) : 0.1667
gradient en (1,5 ; 1/3) : [0. 0.]
```

Au point (1,5 ; 1/3), le gradient est **nul** : on est au fond du bol. Nous montrerons en 1.3 comment y arriver automatiquement.

À retenir (gradient).

- Le gradient ∇f rassemble les dérivées partielles ; il pointe vers la **plus forte montée** et sa norme mesure la pente.
- $-\nabla f$ pointe vers la plus forte **descente**.
- Au minimum (ou au maximum, ou au col), le gradient est nul.

- La règle de la composée rend le calcul du gradient d'une « somme de carrés d'erreurs » mécanique : c'est le cœur de l'entraînement des modèles.

1.2.3 Intégrales : accumuler les petits changements

Intuition. L'intégrale est l'opération inverse de la dérivée. Si la dérivée donne « la vitesse à chaque instant », l'**intégrale** donne « la distance parcourue sur un intervalle » : elle **additionne une infinité de petits morceaux**. Graphiquement, c'est l'**aire sous la courbe**.

En data science, l'intégrale sert surtout à une chose : calculer des **probabilités**. Quand une quantité est décrite par une courbe de densité, la probabilité d'un événement est l'aire sous la courbe sur la zone concernée.

Calculer une aire avec des rectangles

L'aire sous une courbe f entre a et b se note $\int_a^b f(x) dx$. L'idée de **Riemann** : découper l'intervalle en n bandes étroites, approcher chaque bande par un rectangle de hauteur $f(x)$, et additionner. Plus n est grand, meilleure est l'approximation.

Exemple à la main. Aire sous $f(x) = x^2$ entre 0 et 3, avec $n = 3$ rectangles de largeur 1 (hauteur mesurée au bord droit) : $1 \cdot 1^2 + 1 \cdot 2^2 + 1 \cdot 3^2 = 1 + 4 + 9 = 14$. C'est une approximation grossière (les rectangles dépassent la courbe). Avec $n = 6$ rectangles de largeur 0,5 : $0,5(0,5^2 + 1^2 + 1,5^2 + 2^2 + 2,5^2 + 3^2) = 0,5 \times 22,75 = 11,375$. Mieux !

```
def riemann(f, a, b, n):
    largeur = (b - a) / n
    xs = a + largeur * np.arange(1, n + 1)      # extrémités droites des bandes
    return np.sum(f(xs)) * largeur

for n in [3, 6, 30, 300, 3000]:
    print(f"n = {n:>4} rectangles : aire ≈ {riemann(lambda t: t**2, 0, 3, n):.4f}")
print("valeur exacte : 3^3 / 3 =", 3**3 / 3)
```

```
n =    3 rectangles : aire ≈ 14.0000
n =    6 rectangles : aire ≈ 11.3750
n =   30 rectangles : aire ≈  9.4550
n =  300 rectangles : aire ≈  9.0451
n = 3000 rectangles : aire ≈  9.0045
valeur exacte : 3^3 / 3 = 9.0
```

Les approximations convergent vers 9. L'intégrale est cette limite.

Le théorème fondamental : la dérivée à l'envers

Faut-il toujours découper en rectangles ? Heureusement non. Le **théorème fondamental de l'analyse** relie intégrale et dérivée :

Si F est une **primitive** de f (c'est-à-dire $F' = f$), alors

$$\int_a^b f(x) dx = F(b) - F(a).$$

Exemple. Pour $f(x) = x^2$, une primitive est $F(x) = x^3/3$ (car $(x^3/3)' = x^2$). Donc $\int_0^3 x^2 dx = F(3) - F(0) = 9 - 0 = 9$. ✓ C'est la valeur vers laquelle convergeaient nos rectangles.

Quelques primitives à connaître :

$f(x)$	une primitive $F(x)$
x^n ($n \neq -1$)	$\frac{x^{n+1}}{n+1}$
e^{kx}	$\frac{e^{kx}}{k}$
$\frac{1}{x}$	$\ln x $

Pourquoi ça marche : l'idée de la preuve. Notons $A(x) = \int_a^x f(t) dt$ l'aire accumulée jusqu'en x . Quand x avance de h , l'aire gagne une fine bande de largeur h et de hauteur à peu près $f(x)$: $A(x+h) - A(x) \approx f(x)h$. En divisant par h et en faisant tendre h vers 0, on obtient $A'(x) = f(x)$. L'aire accumulée est donc **une** primitive de f ; deux primitives ne diffèrent que d'une constante, qui disparaît dans la différence $F(b) - F(a)$. ■ (Preuve simplifiée, suffisante pour l'intuition.)

■ Application : une commande dans les trois prochaines minutes ?

Sur le site de Dar Jasmin, en heure de pointe, une commande arrive en moyenne toutes les 2 minutes. On montrera au chapitre 2 que le temps d'attente T (en minutes) avant la prochaine commande suit une **loi exponentielle** de taux $\lambda = 0,5$ par minute, dont la densité est

$$f(t) = \lambda e^{-\lambda t} = 0,5 e^{-0,5t}, \quad t \geq 0.$$

La probabilité qu'une commande arrive dans les 3 prochaines minutes est l'aire sous cette courbe entre 0 et 3.

À la main. Une primitive de $0,5 e^{-0,5t}$ est $-e^{-0,5t}$ (on dérive : $-(-0,5)e^{-0,5t} = 0,5 e^{-0,5t}$ ✓). Donc

$$P(T \leq 3) = [-e^{-0,5t}]_0^3 = -e^{-1,5} - (-e^0) = 1 - e^{-1,5} \approx 0,777.$$

```
from scipy.integrate import quad

lam = 0.5
densite = lambda t: lam * np.exp(-lam * t)

p_riemann = riemann(densite, 0, 3, 3000)
p_quad, _ = quad(densite, 0, 3)
print("rectangles de Riemann (3000) :", round(p_riemann, 5))
print("intégration numérique (quad) :", round(p_quad, 5))
print("formule 1 - exp(-1,5)          :", round(1 - np.exp(-1.5), 5))

total, _ = quad(densite, 0, np.inf)
print("aire totale sous la courbe    :", round(total, 5))
```

```
rectangles de Riemann (3000) : 0.77668
intégration numérique (quad) : 0.77687
formule 1 - exp(-1,5)       : 0.77687
aire totale sous la courbe   : 1.0
```

Lecture. Les trois méthodes concordent : environ 77,7 % de chances. Et l'aire **totale** sous la courbe vaut 1 : c'est une exigence de toute densité de probabilité (la probabilité que *quelque chose* arrive est de 100 %). Nous reverrons ces idées en détail au chapitre 2.

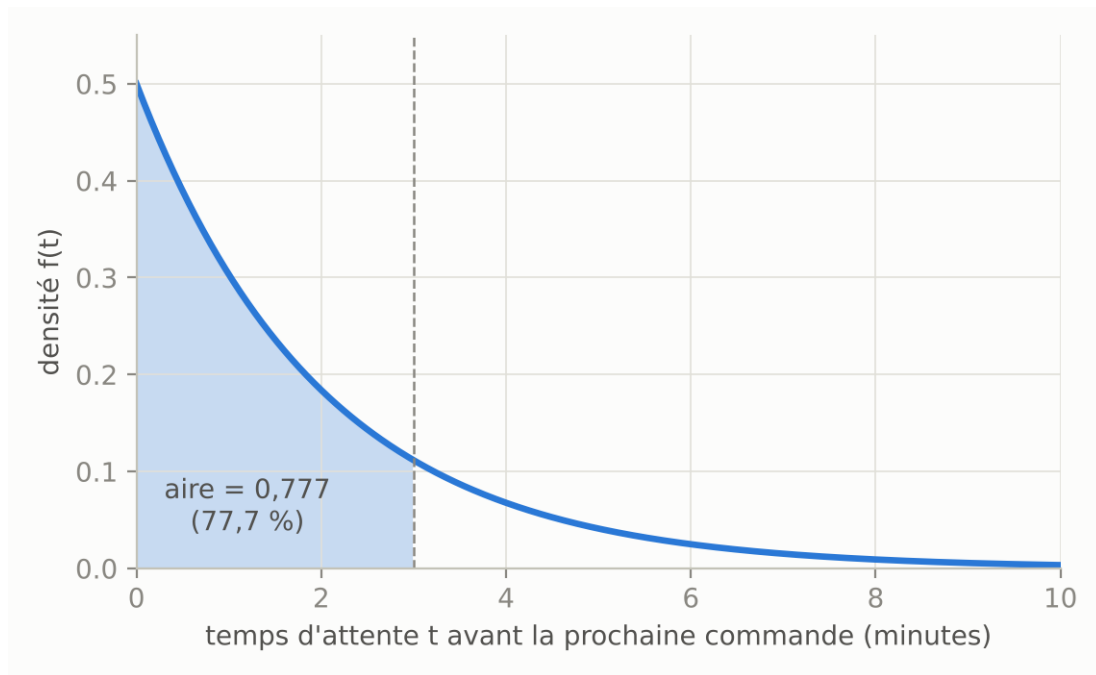


Figure 6: Densité du temps d'attente avant la prochaine commande. L'aire grisée entre 0 et 3 minutes vaut 0,777 : il y a environ 78 % de chances qu'une commande arrive dans les trois prochaines minutes.

Piège : les aires sous l'axe comptent négativement. Si f est négative sur une partie de l'intervalle, l'intégrale en tient compte avec un signe moins. L'intégrale mesure une quantité **algébrique** accumulée, pas toujours une surface géométrique.

À retenir (intégrale).

- $\int_a^b f(x) dx$ est l'aire (algébrique) sous la courbe entre a et b : une limite de sommes de rectangles.
- Théorème fondamental : $\int_a^b f = F(b) - F(a)$ où $F' = f$.
- Pour une densité de probabilité, l'aire sous la courbe sur une zone **est** la probabilité de cette zone ; l'aire totale vaut 1.

1.3 Optimisation : trouver le meilleur choix

Presque tout problème de data science se ramène à la même question : **parmi toutes les valeurs possibles des paramètres de mon modèle, laquelle est la meilleure ?** Et « meilleure » veut dire : celle qui **minimise une erreur** (ou maximise un gain). C'est la troisième phrase de notre épigraphe : *apprendre, c'est minimiser une erreur*. Tout ce que nous avons vu dans ce chapitre (vecteurs, matrices, dérivées, gradients) converge ici.

1.3.1 Le problème d'optimisation

Un **problème d'optimisation** consiste à trouver les paramètres θ qui minimisent une **fonction objectif** $f(\theta)$ (on dit aussi *fonction de coût* ou *fonction de perte*) :

$$\theta^* = \arg \min_{\theta} f(\theta).$$

La notation $\arg \min$ se lit « l'argument qui minimise » : on cherche où le minimum est atteint, pas seulement sa valeur. Maximiser f revient à minimiser $-f$: on peut donc toujours se ramener à une minimisation.

Nous avons déjà rencontré deux problèmes de ce type :

- le bénéfice $P(q)$ de la section 1.2 (une variable, maximisation) ;
- la perte $L(a, b)$ d'une droite ajustée à des points (deux variables, minimisation).

Rappel utile. Aux points qui annulent le gradient ($\nabla f = 0$), la fonction est « à plat ». Ces points sont des **candidats** : minimum local, maximum local, ou col. La question est de savoir lequel, et surtout de savoir si c'est le **meilleur de tous** (minimum *global*) ou seulement le meilleur dans son voisinage (minimum *local*).

1.3.2 La convexité : quand il n'y a qu'une vallée

Intuition. Imaginez un bol : où que vous posiez une bille, elle roule vers l'unique point le plus bas. Maintenant imaginez un paysage de montagnes avec plusieurs vallées : la bille peut se retrouver coincée dans une petite vallée alors qu'une plus profonde existe ailleurs. Une fonction **convexe** est un bol : **pas de piège possible**.

Formellement, f est **convexe** si, pour deux points quelconques x et y et pour tout $t \in [0, 1]$,

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y).$$

Graphiquement : **la corde qui relie deux points de la courbe est toujours au-dessus de la courbe**. Pour une fonction d'une variable deux fois dérivable, c'est équivalent à $f''(x) \geq 0$ partout.

Fonction	f''	Convexe ?
x^2	2	oui
$ x $	(pas dérivable en 0, mais la corde est au-dessus)	oui
e^x	$e^x > 0$	oui
x^3	$6x$ (change de signe)	non
$\sin x$	$-\sin x$ (change de signe)	non

La propriété qui rend la convexité précieuse :

Théorème. Si f est convexe, **tout minimum local est un minimum global**.

Démonstration. Raisonnons par l'absurde. Soit x^* un minimum local de f , et supposons qu'il existe un point y avec $f(y) < f(x^*)$. Pour $t \in (0, 1)$, considérons le point $z_t = (1-t)x^* + ty$, sur le segment qui joint x^* à y . Par convexité,

$$f(z_t) \leq (1-t)f(x^*) + tf(y) < (1-t)f(x^*) + tf(x^*) = f(x^*).$$

Or, quand $t \rightarrow 0$, le point z_t se rapproche de x^* tout en gardant $f(z_t) < f(x^*)$: il existe donc des points arbitrairement proches de x^* où f est plus petite. Cela contredit le fait que x^* est un minimum local. ■

La perte des moindres carrés est convexe

Voici le résultat qui justifie pourquoi la régression linéaire est « facile » à optimiser. Pour un modèle linéaire, la perte s'écrit avec une matrice de données $\mathbf{X}$ (une ligne par individu), un vecteur de paramètres θ et un vecteur de cibles $\mathbf{y}$:

$$L(\theta) = \|\mathbf{y} - \mathbf{X}\theta\|^2.$$

Son gradient est $\nabla L(\theta) = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\theta)$ (c'est la forme matricielle de ce que nous avons calculé à la main en 1.2.2), et sa dérivée seconde, la **matrice hessienne**, est $\mathbf{H} = 2\mathbf{X}^\top\mathbf{X}$.

Démonstration : la hessienne est « positive », donc L est convexe. Pour tout vecteur $\mathbf{v}$,

$$\mathbf{v}^\top \mathbf{H} \mathbf{v} = 2\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = 2\|\mathbf{X}\mathbf{v}\|^2 \geq 0.$$

Une hessienne dont toutes les valeurs propres sont positives ou nulles correspond à une fonction convexe (c'est l'analogue en plusieurs variables de $f'' \geq 0$). ■

De plus, si les colonnes de $\mathbf{X}$ sont **indépendantes** (rang plein), alors $\mathbf{X}\mathbf{v} = \mathbf{0}$ n'est possible que pour $\mathbf{v} = \mathbf{0}$: la hessienne est strictement positive, le bol a un **unique** fond.

On retrouve ici le rang de la section 1.1. Reprenons nos colonnes redondantes (prix HT et prix TTC) : si $\mathbf{X}$ contient les deux, alors $\theta = (1, 0)$ (« on utilise le HT ») et $\theta = (0, 1/1,19)$ (« on utilise le TTC divisé par 1,19 ») produisent **exactement les mêmes prédictions** et donc la même perte : le fond du bol est une *rigole*, pas un point. Le minimum existe mais n'est pas unique.

```
import numpy as np

ht = np.array([40.0, 25.0, 60.0, 15.0])
X_redondant = np.column_stack([ht, 1.19 * ht])
for theta in [(1.0, 0.0), (0.0, 1 / 1.19)]:
    print("theta =", np.round(theta, 4), "-> prédictions :", np.round(X_redondant @ np.array(theta),
        ↪ 4))

theta = [1. 0.] -> prédictions : [40. 25. 60. 15.]
theta = [0.      0.8403] -> prédictions : [40. 25. 60. 15.]
```

Pour notre petit exemple de trois points, la hessienne est $2\mathbf{X}^\top\mathbf{X}$ avec $\mathbf{X}$ constituée de la colonne $x = (1, 2, 3)$ et d'une colonne de 1 :

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 3.0, 5.0])
X = np.column_stack([x, np.ones_like(x)])          # colonnes : x et 1 -> theta = (a, b)

H = 2 * X.T @ X
print("Hessienne :\n", H)
print("valeurs propres :", np.linalg.eigvalsh(H).round(3))

Hessienne :
[[28. 12.]
 [12.  6.]]
valeurs propres : [ 0.721 33.279]
```

Les deux valeurs propres (0,72 et 33,28) sont positives : L est convexe, avec un unique minimum. Remarquez toutefois qu'elles sont **très inégales** (un rapport de 46). Cela signifie que le bol est très allongé : raide dans une direction, presque plat dans l'autre. Nous allons voir que cela a des conséquences concrètes.

1.3.3 La descente de gradient

Intuition. Vous êtes un randonneur dans le brouillard ; vous ne voyez pas la vallée, mais vous sentez la pente sous vos pieds. La stratégie : **faire un pas dans la direction où le sol descend le plus, puis recommencer**. Nous savons déjà que cette direction est $-\nabla f$.

L'algorithme

On part d'un point initial θ_0 et on répète :

$$\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k)$$

Le nombre $\eta > 0$ est le **pas d'apprentissage** (*learning rate*) : la taille de nos enjambées. On s'arrête quand le gradient est presque nul, ou après un nombre fixé d'itérations.

Un exemple à une variable, calculé à la main

Minimisons $f(x) = (x - 3)^2$. Sa dérivée est $f'(x) = 2(x - 3)$ et son minimum est évidemment en $x = 3$. Partons de $x_0 = 0$ avec $\eta = 0,1$.

Pas à pas.

- $x_1 = 0 - 0,1 \times 2(0 - 3) = 0 + 0,6 = 0,6$
- $x_2 = 0,6 - 0,1 \times 2(0,6 - 3) = 0,6 + 0,48 = 1,08$
- $x_3 = 1,08 - 0,1 \times 2(1,08 - 3) = 1,08 + 0,384 = 1,464$

On avance vers 3, avec des pas qui raccourcissent (la pente s'adoucit à mesure qu'on approche du fond).

Peut-on prévoir ce comportement ? Oui, et c'est instructif :

Démonstration : quand la descente converge-t-elle ? Écrivons l'itération : $x_{k+1} = x_k - 2\eta(x_k - 3)$. Soustrayons 3 des deux côtés :

$$x_{k+1} - 3 = (x_k - 3) - 2\eta(x_k - 3) = (1 - 2\eta)(x_k - 3).$$

L'écart au minimum est donc **multiplié à chaque pas par le même facteur** $(1 - 2\eta)$. Par récurrence, $x_k - 3 = (1 - 2\eta)^k (x_0 - 3)$. Cet écart tend vers zéro si et seulement si $|1 - 2\eta| < 1$, c'est-à-dire

$$0 < \eta < 1.$$

Avec $\eta = 0,1$, le facteur vaut 0,8 : on réduit l'erreur de 20 % à chaque pas ($x_1 - 3 = 0,8 \times (-3) = -2,4$, soit $x_1 = 0,6$ ✓). Le cas $\eta = 0,5$ donne un facteur 0 : convergence **en un seul pas** (c'est le pas idéal, égal à l'inverse de la courbure $f'' = 2$). Pour $\eta > 1$, le facteur dépasse 1 en valeur absolue : l'erreur **grandit** à chaque pas. ■

Observons les cinq comportements possibles :

```
grad_ld = lambda x: 2 * (x - 3)

for eta in [0.1, 0.5, 0.9, 1.0, 1.1]:
    x_k = 0.0
    suite = [x_k]
    for _ in range(6):
        x_k = x_k - eta * grad_ld(x_k)
```

```
suite.append(round(x_k, 3))
print(f"eta = {eta:<4}", suite)
```

```
eta = 0.1  [0.0, 0.6, 1.08, 1.464, 1.771, 2.017, 2.214]
eta = 0.5  [0.0, 3.0, 3.0, 3.0, 3.0, 3.0, 3.0]
eta = 0.9  [0.0, 5.4, 1.08, 4.536, 1.771, 3.983, 2.214]
eta = 1.0  [0.0, 6.0, 0.0, 6.0, 0.0, 6.0, 0.0]
eta = 1.1  [0.0, 6.6, -1.32, 8.184, -3.221, 10.465, -5.958]
```

On y voit : $\eta = 0,1$ converge doucement ; $\eta = 0,5$ tombe sur 3 dès le premier pas ; $\eta = 0,9$ converge **en oscillant** de part et d'autre du minimum ; $\eta = 1$ oscille éternellement entre 0 et 6 sans jamais converger ; $\eta = 1,1$ **diverge** (les valeurs s'éloignent de plus en plus).

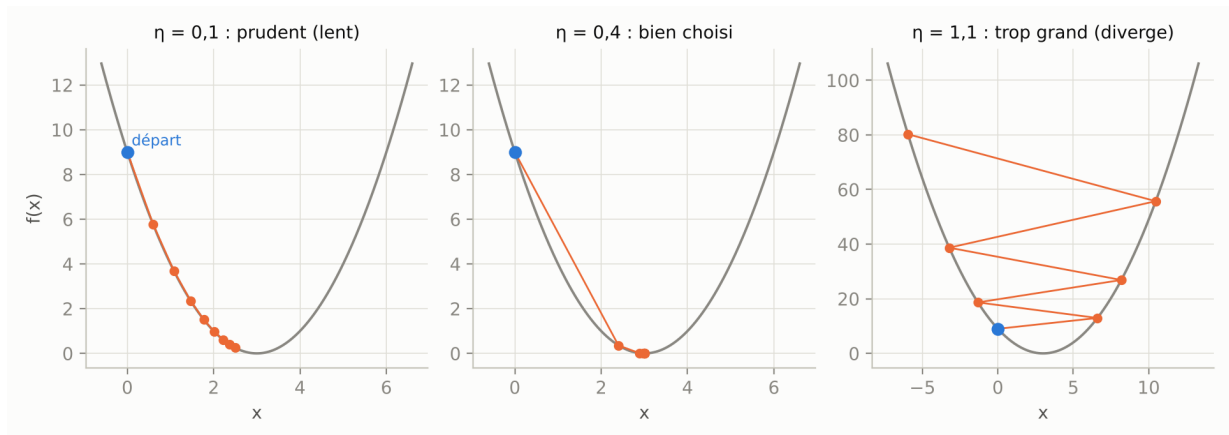


Figure 7: Descente de gradient sur $f(x) = (x-3)^2$ pour trois pas d'apprentissage : prudent (lent), bien choisi (rapide), trop grand (divergence).

Piège : le choix du pas. Un pas trop petit donne un calcul lent ; un pas trop grand fait diverger l'algorithme. En pratique, le pas se règle à l'essai, ou avec des méthodes adaptatives (que nous verrons au volume III).

Descente de gradient en deux dimensions : ajuster une droite

Passons au problème de la droite ajustée aux points $(1,2)$, $(2,3)$, $(3,5)$. Écrivons l'algorithme une fois pour toutes sous une forme réutilisable :

```
def descente_de_gradient(grad, theta0, eta, n_iter=1000, tol=1e-8):
    """Descente de gradient : renvoie le point final et le chemin parcouru."""
    theta = np.array(theta0, dtype=float)
    chemin = [theta.copy()]
    for _ in range(n_iter):
        g = grad(theta)
        if np.linalg.norm(g) < tol:           # presque à plat : on s'arrête
            break
        theta = theta - eta * g
        chemin.append(theta.copy())
    return theta, np.array(chemin)

def grad_L(theta, X, y):
    return -2 * X.T @ (y - X @ theta)

def perte_L(theta, X, y):
    return np.sum((y - X @ theta)**2)
```

On a vu que la hessienne a pour valeurs propres 0,72 et 33,28. Pour que la descente converge,

le pas doit être inférieur à $2/\lambda_{\max} = 2/33,28 \approx 0,060$ (même raisonnement que $0 < \eta < 1$ en une dimension, où la courbure était 2). Essayons un pas de 0,05 (acceptable), puis de 0,07 (trop grand) :

```
theta, chemin = descente_de_gradient(lambda t: grad_L(t, X, y), [0, 0], eta=0.05, n_iter=5000,
    tol=1e-6)
print("eta = 0,05 : itérations =", len(chemin) - 1, "   theta =", theta.round(4))

_, chemin = descente_de_gradient(lambda t: grad_L(t, X, y), [0, 0], eta=0.07, n_iter=15, tol=1e-6)
print("eta = 0,07 : pertes aux étapes 0, 5, 10, 15 :", [float(round(perte_L(chemin[k], X, y), 1)) for
    k in (0, 5, 10, 15)])

eta = 0,05 : itérations = 335   theta = [1.5   0.3333]
eta = 0,07 : pertes aux étapes 0, 5, 10, 15 : [38.0, 652.5, 11256.2, 194234.4]
```

Avec $\eta = 0,05$, l'algorithme converge vers (1,5 ; 0,3333), la droite $y = 1,5x + 1/3$, mais il lui faut **335 itérations** pour un problème à trois points. Avec $\eta = 0,07$ (juste au-dessus de la limite 0,060), la perte **explose**.

Pourquoi tant d'itérations ? À cause du bol allongé : le pas est limité par la direction raide (valeur propre 33), mais la progression dans la direction presque plate (valeur propre 0,72) est minuscule. Il existe un remède classique et très simple : **centrer** la variable. Au lieu de $x = (1, 2, 3)$, on utilise $x - \bar{x} = (-1, 0, 1)$.

Pourquoi ça aide ? Avec la colonne centrée, le produit $X^T X$ devient **diagonal** : $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$. Les deux directions deviennent indépendantes et de courbures comparables (4 et 6 pour la hessienne). Le bol est presque rond.

```
xc = x - x.mean()
Xc = np.column_stack([xc, np.ones_like(xc)])
print("valeurs propres (variable centrée) :", np.linalg.eigvalsh(2 * Xc.T @ Xc).round(3))

theta_c, chemin_c = descente_de_gradient(lambda t: grad_L(t, Xc, y), [0, 0], eta=0.15, n_iter=5000,
    tol=1e-6)
a, b_centre = theta_c
print("variable centrée : itérations =", len(chemin_c) - 1, "   (a, b') =", theta_c.round(4))
print("retour à la droite d'origine : a =", round(a, 4), "   b =", round(b_centre - a * x.mean(), 4))

valeurs propres (variable centrée) : [4. 6.]
variable centrée : itérations = 18   (a, b') = [1.5   3.3333]
retour à la droite d'origine : a = 1.5   b = 0.3333
```

18 itérations au lieu de 335, pour exactement le même résultat. (On retrouve la droite d'origine par $b = b' - a\bar{x}$.) C'est la raison pour laquelle on **centre et standardise** presque toujours les variables avant d'entraîner un modèle par descente de gradient.

Enfin, pour ce problème précis, une solution **exacte** existe, sans itérer. Le gradient s'annule quand $-2X^T(y - X\theta) = 0$, c'est-à-dire pour

$$X^T X \theta = X^T y,$$

un système linéaire (les « équations normales ») que l'on sait résoudre avec ce que nous avons vu en 1.1.2 :

```
theta_exact = np.linalg.solve(X.T @ X, X.T @ y)
print("équations normales :", theta_exact.round(4))
```

équations normales : [1.5 0.3333]

Pourquoi alors se servir de la descente de gradient ? Parce que **la plupart des modèles n'ont pas de solution exacte** : réseaux de neurones, régression logistique, etc. La descente de gradient fonctionne partout où l'on sait calculer un gradient. La régression linéaire est notre terrain d'entraînement, car on connaît la bonne réponse.

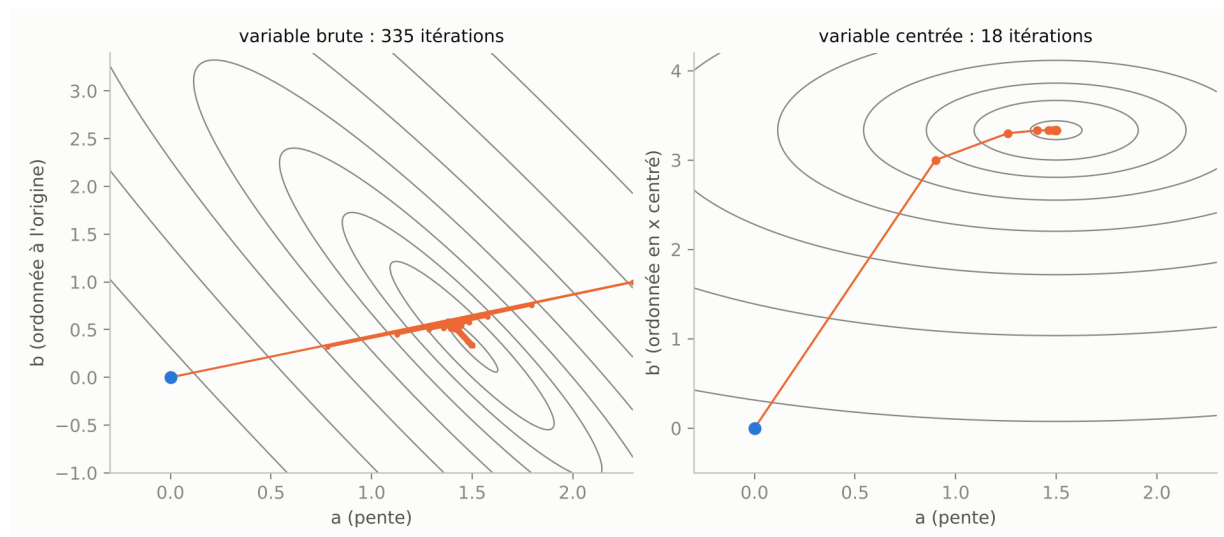


Figure 8: Chemin de la descente de gradient sur les courbes de niveau de la perte, avec la variable brute (à gauche) et la variable centrée (à droite). Le bol allongé force à zigzaguer ; le bol presque rond permet d'aller droit au but.

1.3.4 ■ Application : le prix qui maximise les recettes

Voici un vrai petit problème de décision, qui combine tout le chapitre. Yasmine a testé huit prix pour un même bol en céramique, chacun pendant une semaine :

Prix p (DT)	25	28	31	34	37	40	43	46
Ventes q (pièces)	93	82	82	69	67	56	56	47

Étape 1 : modéliser la demande. On suppose une relation linéaire $q = \alpha + \beta p$ et on cherche α et β par descente de gradient. Ici les prix valent environ 35 et les ventes environ 70 : les variables n'ont pas la même échelle. Nous appliquons donc ce que nous venons d'apprendre : **standardiser le prix** avant de descendre.

```
prix = np.array([25, 28, 31, 34, 37, 40, 43, 46], dtype=float)
ventes = np.array([93, 82, 82, 69, 67, 56, 56, 47], dtype=float)

z = (prix - prix.mean()) / prix.std() # prix standardisé
Z = np.column_stack([z, np.ones_like(z)])

theta, chemin = descente_de_gradient(lambda t: grad_L(t, Z, ventes), [0, 0], eta=0.05, n_iter=1000,
    tol=1e-6)
print("itérations :", len(chemin) - 1)

# retour aux unités d'origine : q = alpha + beta * p
beta = theta[0] / prix.std()
alpha = theta[1] - theta[0] * prix.mean() / prix.std()
```

```
print(f"alpha = {alpha:.3f}    beta = {beta:.3f}")
print("contrôle avec np.polyfit :", np.polyfit(prix, ventes, 1).round(3)[:,-1])
```

```
itérations : 13
alpha = 143.944    beta = -2.111
contrôle avec np.polyfit : [143.944 -2.111]
```

Treize itérations seulement. Le modèle trouvé est $q \approx 143,9 - 2,11 p$: **chaque dinar de hausse fait perdre environ 2,1 ventes par semaine**. (La ligne polyfit vérifie notre résultat avec la fonction toute faite de NumPy.)

Étape 2 : exprimer les recettes. Les recettes sont le prix multiplié par les quantités vendues :

$$R(p) = p \cdot q(p) = p(\alpha + \beta p) = \alpha p + \beta p^2.$$

Étape 3 : maximiser. On dérive et on annule : $R'(p) = \alpha + 2\beta p = 0$, d'où

$$p^* = -\frac{\alpha}{2\beta}.$$

Comme $\beta < 0$, on a $R'' = 2\beta < 0$: c'est bien un maximum.

```
p_opt = -alpha / (2 * beta)
R = lambda p: p * (alpha + beta * p)
print(f"prix optimal p* = {p_opt:.2f} DT")
print(f"recettes prévues à p* : {R(p_opt):8.1f} DT par semaine")
print(f"recettes prévues à 40 DT : {R(40):8.1f} DT par semaine")
print(f"recettes prévues à 46 DT : {R(46):8.1f} DT par semaine")
```

```
prix optimal p* = 34.09 DT
recettes prévues à p* : 2453.7 DT par semaine
recettes prévues à 40 DT : 2380.0 DT par semaine
recettes prévues à 46 DT : 2154.3 DT par semaine
```

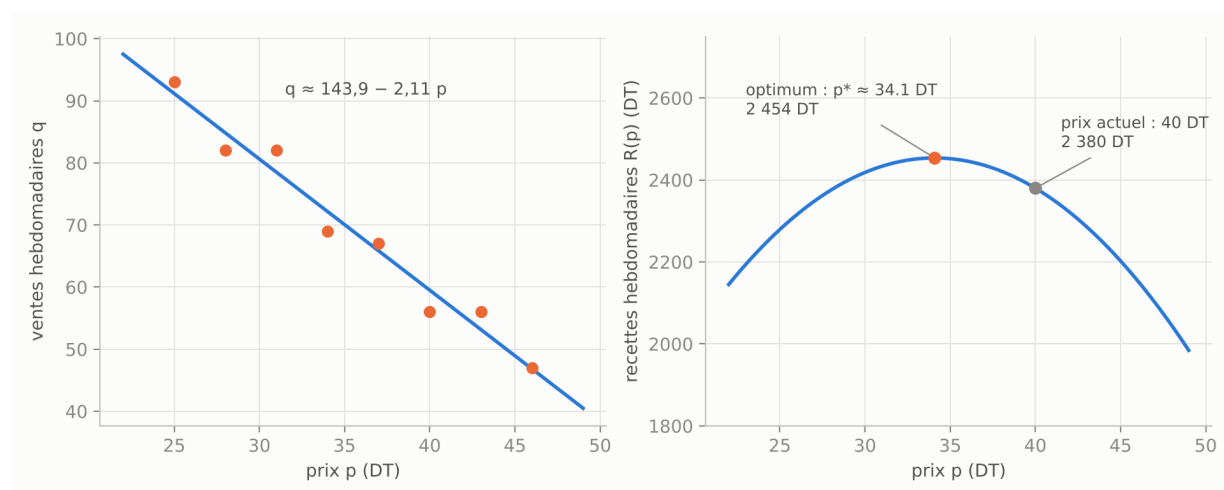


Figure 9: À gauche : les huit mesures et la droite de demande ajustée. À droite : les recettes prévues selon le prix, avec leur maximum vers 34 DT.

Résultat. Le prix qui maximise les recettes est d'environ **34 DT**, avec 2 454 DT de recettes hebdomadaires prévues. Au prix actuel de 40 DT, elles seraient de 2 380 DT : baisser le prix de 6 dinars rapporterait un peu plus de **70 DT par semaine**, soit environ 3 % de mieux.

Prudence. Ce résultat est obtenu avec **huit** points et un modèle très simple. Il ne dit rien de l'incertitude (de combien p^* pourrait-il se tromper ?), ni du bénéfice (ici nous avons maximisé les *recettes*, sans tenir compte des coûts), ni de l'extrapolation hors de la plage de prix testée. Ces questions sont l'objet du chapitre 3 (statistique) et du volume II (régression). Retenez la démarche : **modéliser, écrire la fonction objectif, la dériver, l'annuler.**

1.3.5 Optimisation sous contraintes : les multiplicateurs de Lagrange

Dans la vraie vie, on n'optimise presque jamais librement : on a un **budget**, une capacité, un poids maximal. On cherche alors le meilleur choix **parmi ceux qui respectent une contrainte**.

Intuition. Vous voulez atteindre le point le plus haut d'une colline, mais vous devez rester sur un sentier. Au meilleur point du sentier, vous ne pouvez plus monter en suivant le sentier : celui-ci est **tangent à une courbe de niveau** de la colline. À cet endroit, la direction de plus forte montée de la colline (le gradient de f) est **perpendiculaire au sentier**, donc **parallèle au gradient de la contrainte** (qui, lui aussi, est perpendiculaire au sentier).

Un exemple concret

Yasmine dispose de 1 000 DT de budget publicitaire à répartir entre Facebook (x dinars) et Instagram (y dinars). Elle estime les recettes générées par :

$$R(x, y) = 80\sqrt{x} + 120\sqrt{y}.$$

(La racine carrée traduit des **rendements décroissants** : les premiers dinars investis rapportent plus que les derniers.) Elle veut maximiser R sous la contrainte $x + y = 1000$.

La méthode de Lagrange. On introduit un nombre λ (le *multiplicateur*) et on forme le **lagrangien** :

$$\mathcal{L}(x, y, \lambda) = 80\sqrt{x} + 120\sqrt{y} - \lambda(x + y - 1000).$$

On annule toutes ses dérivées partielles :

- $\frac{\partial \mathcal{L}}{\partial x} = \frac{40}{\sqrt{x}} - \lambda = 0$
- $\frac{\partial \mathcal{L}}{\partial y} = \frac{60}{\sqrt{y}} - \lambda = 0$
- $\frac{\partial \mathcal{L}}{\partial \lambda} = -(x + y - 1000) = 0$ (qui redonne la contrainte).

Résolution à la main. Les deux premières équations donnent $\frac{40}{\sqrt{x}} = \frac{60}{\sqrt{y}}$, donc $\sqrt{y} = 1,5\sqrt{x}$, soit $y = 2,25x$. En reportant dans la contrainte : $x + 2,25x = 1000$, donc

$$x = \frac{1000}{3,25} \approx 307,7 \text{ DT}, \quad y = 2,25x \approx 692,3 \text{ DT}.$$

Recettes : $80\sqrt{307,7} + 120\sqrt{692,3} \approx 1\,403,3 + 3\,157,4 = 4\,560,7 \text{ DT}$.

Pourquoi cette méthode marche. Le long de la contrainte, on peut paramétrer $y = 1000 - x$ et regarder R comme fonction d'une seule variable. Au maximum, sa dérivée s'annule, ce qui s'écrit $\nabla R \cdot \mathbf{t} = 0$ où $\mathbf{t}$ est la direction du sentier : ∇R est perpendiculaire au sentier. Or le gradient de $g(x, y) = x + y - 1000$ l'est aussi. Deux vecteurs perpendiculaires à la même direction (en dimension 2) sont parallèles : $\nabla R = \lambda \nabla g$. C'est exactement ce que disent les équations $\partial \mathcal{L} / \partial x = \partial \mathcal{L} / \partial y = 0$. ■

Vérifions par trois chemins différents : un solveur numérique, une recherche exhaustive le long de la contrainte, et la formule.

```
from scipy.optimize import minimize

R2 = lambda v: 80 * np.sqrt(v[0]) + 120 * np.sqrt(v[1])

res = minimize(lambda v: -R2(v), x0=[500, 500], method="SLSQP",
               bounds=[(1e-6, None), (1e-6, None)],
               constraints=[{"type": "eq", "fun": lambda v: v[0] + v[1] - 1000}])
print("solveur SLSQP      : x = %.1f, y = %.1f, recettes = %.2f" % (res.x[0], res.x[1], -res.fun))

grille = np.linspace(1, 999, 998)
valeurs = [R2([g, 1000 - g]) for g in grille]
i = int(np.argmax(valeurs))
print("recherche exhaustive : x = %.1f, y = %.1f, recettes = %.2f" % (grille[i], 1000 - grille[i],
                               valeurs[i]))

x_th = 1000 / 3.25
print("formule           : x = %.1f, y = %.1f, recettes = %.2f" % (x_th, 1000 - x_th, R2([x_th,
                               1000 - x_th])))
```

```
solveur SLSQP      : x = 307.7, y = 692.3, recettes = 4560.70
recherche exhaustive : x = 307.7, y = 692.3, recettes = 4560.70
formule           : x = 307.7, y = 692.3, recettes = 4560.70
```

Le multiplicateur λ a une signification concrète. Au point optimal, $\lambda = 40 / \sqrt{x} \approx 40 / 17,54 \approx 2,28$. C'est le **prix de l'ombre** (*shadow price*) de la contrainte : **un dinar de budget supplémentaire rapporterait environ 2,28 DT de recettes en plus**, si l'on réoptimise la répartition. Vérifions :

```
meilleur = lambda B: R2([B / 3.25, B - B / 3.25]) # répartition optimale pour un budget B
print("gain réel avec 1 DT de plus :", round(meilleur(1001) - meilleur(1000), 4))
print("multiplicateur lambda      :", round(40 / np.sqrt(x_th), 4))
```

```
gain réel avec 1 DT de plus : 2.2798
multiplicateur lambda      : 2.2804
```

C'est une information précieuse pour fixer un budget : 1 DT de publicité en plus rapporte environ 2,28 DT de recettes. Il n'est donc rentable d'augmenter le budget que si la marge brute de Yasmine dépasse $1/2,28 \approx 44\%$ (sinon la dépense supplémentaire coûte plus qu'elle ne rapporte).

À retenir (optimisation).

- Apprendre = minimiser une fonction de perte. $\arg \min$ désigne le point où le minimum est atteint.
- Si la fonction est **convexe**, tout minimum local est global. La perte des moindres carrés est convexe (hessienne $2\mathbf{X}^T\mathbf{X}$ positive) ; elle a un fond unique si les colonnes de $\mathbf{X}$ sont indépendantes.
- Descente de gradient : $\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k)$. Le pas η ne doit être ni trop petit (lent) ni trop grand (divergence : $\eta < 2/\lambda_{\max}$ pour une fonction quadratique).
- **Centrer et standardiser** les variables arrondit le bol et accélère considérablement la

descente.

- Sous contrainte, on annule les dérivées du lagrangien ; λ mesure la valeur marginale de la contrainte.

1.4 La fiche de notations

À quoi sert cette fiche ? Les livres de data science sont pleins de symboles. Le but n'est pas de les apprendre par cœur, mais de **savoir où regarder** quand l'un d'eux vous bloque. Gardez cette page ouverte pendant la lecture de tous les volumes.

1.4.1 Conventions typographiques

Ce que vous voyez	Ce que cela désigne	Exemple
Lettre minuscule italique : x, a, η	un nombre (scalaire)	$x = 3,5$
Minuscule grasse : $\mathbf{x}, \mathbf{y}, \boldsymbol{\theta}$	un vecteur (colonne par défaut)	$\mathbf{x} = (1, 2, 3)^\top$
Majuscule grasse : $\mathbf{A}, \mathbf{X}$	une matrice	$\mathbf{X} \in \mathbb{R}^{n \times p}$
Majuscule italique : X, Y	(au chapitre 2) une variable aléatoire	$X = \text{montant d'un panier}$
Chapeau : $\hat{y}, \hat{\theta}$	une valeur estimée ou prédite	$\hat{y}_i = \text{prédiction pour } i$
Barre : $\bar{x}$	la moyenne	$\bar{x} = \frac{1}{n} \sum x_i$
Étoile : p^*, θ^*	la valeur optimale	$p^* = 34,09$
Exposant $\top$: $\mathbf{A}^\top$	la transposée (lignes $\leftrightarrow$ colonnes)	
Exposant -1 : $\mathbf{A}^{-1}$	l' inverse d'une matrice	$\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$

Attention aux indices. x_i est le i -ème **élément** d'un vecteur, ou la i -ème **observation**. x_{ij} est l'élément de la ligne i , colonne j d'une matrice. Mathématiquement on compte à partir de **1** ; en Python, à partir de **0** : x_1 s'écrit $\mathbf{x}[0]$. C'est la source d'erreur n°1 quand on traduit une formule en code.

1.4.2 Ensembles et logique

Symbole	Se lit	Exemple et sens
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$	entiers naturels, relatifs, rationnels, réels	$\mathbb{R}$ = tous les nombres de la droite
$\mathbb{R}^n$	vecteurs de n nombres réels	$\mathbb{R}^3$: l'espace usuel
$\mathbb{R}^{n \times p}$	matrices à n lignes et p colonnes	un tableau de données
$\in$	« appartient à »	$3 \in \mathbb{N}$
$\subset$	« est inclus dans »	$\mathbb{N} \subset \mathbb{R}$
$\cup, \cap$	union, intersection	$A \cup B$: dans A ou B
$\emptyset$	ensemble vide	
$\forall$	« pour tout »	$\forall x \in \mathbb{R}, x^2 \geq 0$
$\exists$	« il existe »	$\exists x, x^2 = 2$
$\Rightarrow$	« implique »	$x > 2 \Rightarrow x > 1$
$\Leftrightarrow$	« si et seulement si »	
$:=$	« est défini comme »	$f(x) := x^2$
$\approx, \propto$	environ égal ; proportionnel à	

1.4.3 Sommes, produits, fonctions

Symbole	Sens	Équivalent NumPy
$\sum_{i=1}^n x_i$	$x_1 + x_2 + \dots + x_n$	<code>x.sum()</code>
$\prod_{i=1}^n x_i$	$x_1 \times x_2 \times \dots \times x_n$	<code>x.prod()</code>
$\bar{x} = \frac{1}{n} \sum x_i$	moyenne	<code>x.mean()</code>
max, min	plus grande, plus petite valeur	<code>x.max()</code> , <code>x.min()</code>
$\arg \min_{\theta} f(\theta)$	l'argument θ qui rend f minimale	<code>np.argmin(f_values)</code>
arg max	idem pour le maximum	<code>np.argmax</code>
$ x $	valeur absolue	<code>np.abs(x)</code>
$\lfloor x \rfloor$, $\lceil x \rceil$	partie entière inférieure, supérieure	<code>np.floor</code> , <code>np.ceil</code>
$\exp(x) = e^x$, $\ln x$	exponentielle, log népérien	<code>np.exp</code> , <code>np.log</code>
$\mathbb{1}[\text{cond}]$	vaut 1 si la condition est vraie, 0 sinon	<code>(cond).astype(int)</code>
$f : A \rightarrow B$	f prend ses valeurs de A vers B	
$f \circ g$	composition : $(f \circ g)(x) = f(g(x))$	
$\lim_{x \rightarrow a} f(x)$	limite	
$\binom{n}{k}$	« k parmi n »	<code>math.comb(n, k)</code>
$n!$	factorielle	<code>math.factorial(n)</code>

1.4.4 Algèbre linéaire et analyse

Symbole	Sens	Équivalent NumPy
$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^\top \mathbf{v}$	produit scalaire	<code>u @ v</code>
$\ \mathbf{v}\ $ ou $\ \mathbf{v}\ _2$	norme euclidienne $\sqrt{\sum v_i^2}$	<code>np.linalg.norm(v)</code>
$\ \mathbf{v}\ _1$	norme 1 : $\sum v_i $	<code>np.linalg.norm(v, 1)</code>
$\mathbf{AB}$	produit matriciel	<code>A @ B</code>
$\mathbf{A}^\top$	transposée	<code>A.T</code>
$\mathbf{A}^{-1}$	inverse	<code>np.linalg.inv(A)</code> (ou mieux : <code>solve</code>)
$\mathbf{I}$	matrice identité	<code>np.eye(n)</code>
$\det \mathbf{A}$	déterminant	<code>np.linalg.det(A)</code>
rang $\mathbf{A}$	nombre de colonnes indépendantes	<code>np.linalg.matrix_rank(A)</code>
tr $\mathbf{A}$	trace (somme de la diagonale)	<code>np.trace(A)</code>
$\lambda, \mathbf{v}$	valeur propre, vecteur propre : $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$	<code>np.linalg.eig(A)</code>
$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$	décomposition en valeurs singulières	<code>np.linalg.svd(A)</code>
$f'(x)$ ou $\frac{df}{dx}$	dérivée	
$\frac{\partial f}{\partial x_j}$	dérivée partielle	
∇f	gradient (vecteur des dérivées partielles)	
$\nabla^2 f$ ou $\mathbf{H}$	hessienne (matrice des dérivées secondes)	
$\int_a^b f(x) dx$	intégrale (aire sous la courbe)	<code>scipy.integrate.quad</code>

Symbole	Sens	Équivalent NumPy
---------	------	------------------

1.4.5 L'alphabet grec en data science

Les lettres grecques reviennent partout. Voici celles que vous rencontrerez, avec leur usage **habituel** (pas une règle absolue).

Lettre	Nom	Usage courant
α	alpha	niveau de risque d'un test ; ordonnée à l'origine ; paramètre de régularisation
β	bêta	coefficients d'une régression
γ	gamma	pas d'apprentissage (parfois) ; fonction Gamma
δ, Δ	delta	une petite variation ; Δx = écart
ε	epsilon	une très petite quantité ; l' erreur (bruit) d'un modèle
η	êta	pas d'apprentissage (learning rate)
θ	thêta	paramètres d'un modèle (ce que l'on apprend)
λ	lambda	valeur propre ; multiplicateur de Lagrange ; taux d'une loi exponentielle ; force de la régularisation
μ	mu	moyenne d'une loi (théorique)
ν	nu	degrés de liberté
π	pi	3,14159... ; ou une probabilité
ρ	rhô	coefficient de corrélation
σ, Σ	sigma	écart-type (σ) ; matrice de covariance (Σ) ; $\Delta \Sigma$ est la somme, pas la même lettre
τ	tau	un seuil
ϕ, φ	phi	densité de la loi normale (φ) ; une fonction de transformation
Φ	Phi majuscule	fonction de répartition de la loi normale
χ^2	khi-deux	loi et test du khi-deux
ω, Ω	oméga	l'ensemble des issues possibles (Ω)

Astuce pour lire une formule inconnue. (1) Repérez d'abord ce qui est un nombre, un vecteur, une matrice (grâce à la typographie). (2) Cherchez ce qui est *sommé* ou *minimisé*. (3) Remplacez les symboles par un exemple chiffré minuscule ($n = 3$). Une formule devient presque toujours claire sur un exemple à trois éléments.

1.4.6 Exemple : lire une formule du début à la fin

Voici la fonction de perte des moindres carrés, que vous connaissez maintenant :

$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \theta)^2.$$

Lecture mot à mot : « le vecteur de paramètres estimé $\hat{\theta}$ est l'**argument** θ qui **minimise** la somme, sur les n observations, du **carré de l'écart** entre la valeur observée y_i et la prédiction $\mathbf{x}_i^T \theta$ ». Et en Python :

```
import numpy as np
x = np.array([1.0, 2.0, 3.0]); y = np.array([2.0, 3.0, 5.0])
X = np.column_stack([x, np.ones_like(x)])
theta_chapeau = np.linalg.solve(X.T @ X, X.T @ y)
print("theta chapeau =", theta_chapeau.round(4))
print("somme des carrés =", round((y - X @ theta_chapeau) ** 2).sum(), 4))
```

```
theta chapeau = [1.5    0.3333]
somme des carrés = 0.1667
```

Une formule, une phrase, trois lignes de code : c'est cette triple lecture qui vous rendra autonome.

1.5 ⊕ Pour aller plus loin : analyse numérique

Section optionnelle. Vous pouvez la sauter sans perdre le fil du livre. Mais si un jour votre code donne un résultat « presque juste » et que vous ne comprenez pas pourquoi, c'est ici qu'est la réponse.

Les mathématiques des sections précédentes travaillent avec des nombres **exacts**. Un ordinateur, lui, n'en a pas : il stocke des nombres avec un nombre **fini** de chiffres. L'**analyse numérique** étudie ce que cela change. Bonne nouvelle : on peut tout comprendre avec quelques expériences.

1.5.1 Pourquoi $0,1 + 0,2$ n'est pas égal à $0,3$

Intuition. En base 10, le nombre $1/3 = 0,3333 \dots$ ne s'écrit pas avec un nombre fini de chiffres. Il en va de même en base 2 pour $0,1$: l'ordinateur en garde une **approximation**, très bonne mais pas exacte.

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(f"{0.1:.25f}")
print(f"{0.3:.25f}")
```

```
0.30000000000000004
False
0.1000000000000000055511151
0.2999999999999999888977698
```

Les nombres décimaux de Python (type `float`) suivent la norme IEEE 754 en **double précision** : 64 bits, soit environ **16 chiffres significatifs**. L'erreur relative maximale d'un arrondi est appelée **epsilon machine** :

```
import numpy as np
print("epsilon machine :", np.finfo(float).eps)
```

```
epsilon machine : 2.220446049250313e-16
```

Règle d'or. On ne compare **jamais** deux flottants avec `==`. On teste s'ils sont *proches* :

```
import math
print(math.isclose(0.1 + 0.2, 0.3))
print(np.isclose(0.1 + 0.2, 0.3))
```

```
True
True
```

Autre piège du même type, que nous avons rencontré en préparant ce livre : demander le logarithme de 1000 en base 10.

```
print(math.log(1000, 10))      # calcule log(1000)/log(10) : deux arrondis
print(math.log10(1000))       # fonction dédiée : exacte ici
```

```
2.9999999999999996
3.0
```

Morale : quand une fonction **dédiée** existe (`log10`, `expm1`, `hypot`...), elle est plus précise que la formule naïve.

1.5.2 Les grands nombres avalent les petits

Additionner un très petit nombre à un très grand peut ne **rien changer** :

```
print(1e16 + 1 == 1e16)
print(1e15 + 1 == 1e15)
```

```
True
False
```

Conséquence pratique : l'**ordre** des additions compte. Additionner un million de fois 0,1 ne donne pas exactement 100 000.

```
s = 0.0
for _ in range(1_000_000):
    s += 0.1
print(s)
print(math.fsum([0.1] * 1_000_000))      # somme compensée : exacte à l'arrondi final
```

```
100000.00000133288
100000.0
```

1.5.3 L'annulation catastrophique

C'est le piège le plus important pour un data scientist. Quand on **soustrait deux nombres presque égaux**, les premiers chiffres s'annulent et il ne reste que... du bruit d'arrondi.

Intuition. Vous mesurez deux immeubles de 300,00 m et 300,01 m, avec une règle précise au mètre près. Leur différence (0,01 m) est noyée dans l'imprécision.

Exemple réel : le calcul de la **variance**. Il existe deux formules équivalentes en mathématiques :

- formule stable : $\text{Var} = \frac{1}{n} \sum (x_i - \bar{x})^2$;
- formule « du calcul à la main » : $\text{Var} = \overline{x^2} - \bar{x}^2$ (moyenne des carrés moins carré de la moyenne).

Appliquons les deux à des chiffres d'affaires de l'ordre du milliard, avec une petite dispersion :

```
rng = np.random.default_rng(0)
x = 1e9 + rng.normal(0, 1, size=1000)      # moyenne ~ 1 milliard, écart-type ~ 1

var_stable = np.mean((x - x.mean()) ** 2)
var_naive = np.mean(x ** 2) - x.mean() ** 2

print("formule stable :", round(var_stable, 6))
print("formule naïve :", var_naive)
```

```
formule stable : 0.954046
formule naïve : -128.0
```

La vraie variance vaut environ 1. La formule stable la retrouve (0,95) ; la naïve donne **−128**, une variance *négative*, ce qui est impossible ! Elle est fautive, car $\bar{x}^2 \approx 10^{18}$ et $\bar{x}^2 \approx 10^{18}$ sont deux nombres presque égaux dont la différence est de l'ordre de 1 : on soustrait des nombres de 18 chiffres avec seulement 16 chiffres de précision.

Conséquence. NumPy et pandas utilisent des algorithmes stables, pas la formule naïve. Moralité : **faites confiance aux fonctions des bibliothèques** plutôt que de recoder des formules vues en cours.

Pourquoi mathématiquement ? Si a et b sont connus avec une erreur relative ε , leur différence $a - b$ a une erreur **absolue** d'environ $\varepsilon(|a| + |b|)$, donc une erreur **relative** de $\varepsilon \frac{|a| + |b|}{|a - b|}$. Quand $a \approx b$, le quotient est gigantesque : l'erreur relative explose.

1.5.4 Le conditionnement : quand un problème est « fragile »

Certains problèmes sont intrinsèquement sensibles : un minuscule changement des données change beaucoup la réponse. On mesure cette sensibilité par le **nombre de conditionnement** d'une matrice,

$$\kappa(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}},$$

le rapport entre sa plus grande et sa plus petite valeur singulière (section 1.1.4). Si κ est grand, la matrice est « presque singulière » : c'est exactement le cas de la **multicollinéarité** vue au 1.1.

Voici un système $\mathbf{Ax} = \mathbf{b}$ dont les deux équations sont presque identiques :

```
A = np.array([[1.0, 1.0],
              [1.0, 1.0001]])
b = np.array([2.0, 2.0001])
print("solution :", np.linalg.solve(A, b))
print("conditionnement :", round(np.linalg.cond(A)))
```

```
solution : [1. 1.]
conditionnement : 40002
```

Perturbons légèrement le second membre (changement de 10^{-4}) :

```
b2 = np.array([2.0, 2.0002])
print("solution perturbée :", np.linalg.solve(A, b2))
```

solution perturbée : [0. 2.]

Un changement de 0,0001 sur les données a **complètement changé la solution** : de (1,1) à (0,2). Le conditionnement est de l'ordre de 40 000 : on peut perdre jusqu'à 4 à 5 chiffres de précision.

Règle empirique. Avec $\kappa \approx 10^k$, on perd environ k chiffres significatifs sur les 16 disponibles.

Deux conséquences pratiques :

1. Pour résoudre $Ax = b$, on utilise `np.linalg.solve(A, b)`, **pas** `np.linalg.inv(A) @ b` : c'est plus rapide **et** plus précis.
2. Centrer et standardiser les variables (1.3.3) diminue le conditionnement, ce qui explique en partie le gain de 335 à 18 itérations.

```
x = np.array([1.0, 2.0, 3.0])
X_brut = np.column_stack([x, np.ones(3)])
X_centre = np.column_stack([x - x.mean(), np.ones(3)])
print("kappa(X^T X) brut : ", round(np.linalg.cond(X_brut.T @ X_brut), 1))
print("kappa(X^T X) centré : ", round(np.linalg.cond(X_centre.T @ X_centre), 1))
```

```
kappa(X^T X) brut : 46.1
kappa(X^T X) centré : 1.5
```

1.5.5 Deux algorithmes classiques

La méthode de Newton pour résoudre $f(x) = 0$. Idée : on remplace la courbe par sa **tangente** (1.2.1) et on prend l'endroit où la tangente coupe l'axe des abscisses :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Pour calculer $\sqrt{2}$, on cherche le zéro de $f(x) = x^2 - 2$, avec $f'(x) = 2x$:

```
x = 1.0
for k in range(6):
    print(f"étape {k} : x = {x:.15f}   erreur = {abs(x - 2**0.5):.2e}")
    x = x - (x**2 - 2) / (2 * x)
```

```
étape 0 : x = 1.000000000000000   erreur = 4.14e-01
étape 1 : x = 1.500000000000000   erreur = 8.58e-02
étape 2 : x = 1.416666666666667   erreur = 2.45e-03
étape 3 : x = 1.414215686274510   erreur = 2.12e-06
étape 4 : x = 1.414213562374690   erreur = 1.59e-12
étape 5 : x = 1.414213562373095   erreur = 0.00e+00
```

Le nombre de chiffres justes **double** à chaque étape (on parle de convergence quadratique). C'est ainsi que votre calculatrice calcule vraiment les racines carrées.

La dichotomie. Plus lente mais beaucoup plus robuste : si f est continue et change de signe entre a et b , il existe un zéro entre les deux. On coupe l'intervalle en deux, on garde la moitié qui change de signe, et on recommence. L'erreur est divisée par 2 à chaque tour.

```
def dichotomie(f, a, b, tol=1e-10):
    n = 0
    while b - a > tol:
        m = (a + b) / 2
        if f(a) * f(m) <= 0:
            b = m
```

```

else:
    a = m
    n += 1
    return (a + b) / 2, n

racine, n = dichotomie(lambda x: x**2 - 2, 0, 2)
print(f"racine = {racine:.10f} en {n} étapes")

```

```
racine = 1.4142135624 en 35 étapes
```

Il faut 35 étapes à la dichotomie, contre 5 à Newton, pour une précision comparable. Newton est rapide mais peut diverger si on part mal ; la dichotomie est lente mais ne rate jamais. Les bibliothèques (`scipy.optimize.brentq`) combinent les deux.

À retenir (analyse numérique).

- Un flottant a ~16 chiffres significatifs ; on compare avec `isclose`, jamais avec `==`.
- Soustraire deux nombres proches détruit la précision (annulation) : préférez les fonctions de bibliothèque.
- Le conditionnement κ mesure la fragilité d'un problème ; on perd environ $\log_{10} \kappa$ chiffres.
- solve plutôt que inv. Centrer et standardiser améliore le conditionnement.
- Newton : rapide mais capricieux ; dichotomie : lente mais sûre.

1.6 ⊕ Pour aller plus loin : mathématiques discrètes et graphes

Section optionnelle. Tout ce qui précède traitait de quantités qui varient de façon *continue* (prix, temps, pentes). Ici, on compte et on relie : des objets distincts, des choix, des liens. Ces outils servent pour les probabilités (chapitre 2), les algorithmes (chapitre 4), les bases de données (chapitre 5) et les systèmes de recommandation.

1.6.1 Ensembles : le langage de base

Un **ensemble** est une collection d'objets distincts, sans ordre. Yasmine propose quatre produits : $P = \{\text{bol}, \text{tapis}, \text{lampe}, \text{plateau}\}$. Ses clients de la semaine ont acheté des sous-ensembles de P .

Les opérations de la fiche de notations (1.4.2) se testent directement en Python avec le type `set` :

```

panier_amel = {"bol", "tapis", "lampe"}
panier_karim = {"bol", "plateau"}

print("union          :", sorted(panier_amel | panier_karim))
print("intersection   :", sorted(panier_amel & panier_karim))
print("amel sans karim:", sorted(panier_amel - panier_karim))
print("taille         :", len(panier_amel))

```

```

union          : ['bol', 'lampe', 'plateau', 'tapis']
intersection   : ['bol']
amel sans karim: ['lampe', 'tapis']
taille         : 3

```

Application : similarité de Jaccard. Comment mesurer à quel point deux paniers se ressemblent ? On divise la taille de ce qu'ils ont **en commun** par la taille de ce qu'ils ont **au total** :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Elle vaut 1 si les paniers sont identiques, 0 s'ils n'ont rien en commun. C'est l'équivalent « ensembliste » de la similarité cosinus du 1.1.1.

```
def jaccard(a, b):
    return len(a & b) / len(a | b)
```

```
print("Jaccard(Amel, Karim) =", round(jaccard(panier_amel, panier_karim), 3))
```

```
Jaccard(Amel, Karim) = 0.25
```

Un seul produit en commun (le bol), sur quatre produits au total : $1/4 = 0,25$.

1.6.2 Compter : le principe multiplicatif

Intuition. Si vous avez 3 pulls et 4 pantalons, vous avez $3 \times 4 = 12$ tenues. Quand on enchaîne des choix indépendants, on **multiplie** le nombre d'options.

Yasmine veut proposer un **coffret cadeau** : un produit principal (4 choix), un emballage (3 choix), une carte message (2 choix). Le nombre de coffrets différents est $4 \times 3 \times 2 = 24$.

Permutations. De combien de façons peut-on **ranger** n objets distincts dans l'ordre ? n choix pour la première place, $n - 1$ pour la suivante, etc. :

$$n! = n \times (n - 1) \times \dots \times 2 \times 1.$$

Les 4 produits peuvent être alignés en vitrine de $4! = 24$ façons.

Arrangements et combinaisons. Choisir k objets parmi n :

- quand l'ordre compte (podium : 1er, 2e, 3e) : $\frac{n!}{(n-k)!}$;
- quand l'ordre ne compte pas (un panier de 3 produits) :

$$\binom{n}{k} = \frac{n!}{k! (n-k)!}.$$

Pourquoi cette formule ? Il y a $\frac{n!}{(n-k)!}$ façons de choisir k objets *dans l'ordre*. Mais chaque groupe de k objets a été compté $k!$ fois (une fois par ordre possible). On divise donc par $k!$.

```
import math
print("tenues          :", 3 * 4)
print("coffrets        :", 4 * 3 * 2)
print("ordres de vitrine :", math.factorial(4))
print("paniers de 2 parmi 4:", math.comb(4, 2))
print("podium 3 parmi 10  :", math.perm(10, 3))
print("paniers de 3 parmi 40:", math.comb(40, 3))
```

```
tenues          : 12
coffrets        : 24
ordres de vitrine : 24
paniers de 2 parmi 4: 6
podium 3 parmi 10 : 720
paniers de 3 parmi 40: 9880
```


Attention à l'explosion. Si le catalogue contient 40 produits, il y a déjà 9 880 paniers de trois produits. Pour des paniers de dix produits parmi 40, on dépasse les 847 millions :

```
print(math.comb(40, 10))
```

```
847660528
```

C'est pourquoi les algorithmes de recommandation ne peuvent pas **énumérer** tous les cas : on y reviendra avec la complexité (section ⊕ du chapitre 4).

Le triangle de Pascal et le binôme. Les nombres $\binom{n}{k}$ se calculent par la règle $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ et vérifient $(a+b)^n = \sum_k \binom{n}{k} a^k b^{n-k}$. Nous les retrouverons au chapitre 2 dans la **loi binomiale**.

1.6.3 Les graphes : modéliser des relations

Un **graphe** est un ensemble de **sommets** (des objets) et d'**arêtes** (des liens entre eux). Tout ce qui est « réseau » est un graphe : amis sur un réseau social, routes entre villes, produits souvent achetés ensemble, pages web reliées par des liens.

Voici le graphe des produits **achetés ensemble** dans la boutique : deux produits sont reliés si au moins un client les a pris dans le même panier.

```
produits = ["bol", "tapis", "lampe", "plateau", "coussin"]
aretes = [("bol", "plateau"), ("bol", "tapis"), ("tapis", "lampe"),
          ("tapis", "coussin"), ("lampe", "coussin")]

# liste d'adjacence : pour chaque sommet, la liste de ses voisins
voisins = {p: [] for p in produits}
for a, b in aretes:
    voisins[a].append(b)
    voisins[b].append(a)

for p in produits:
    print(f"{p:8s} -> {voisins[p]}")
print("degrés :", {p: len(v) for p, v in voisins.items()})
```

```
bol      -> ['plateau', 'tapis']
tapis    -> ['bol', 'lampe', 'coussin']
lampe    -> ['tapis', 'coussin']
plateau  -> ['bol']
coussin  -> ['tapis', 'lampe']
degrés : {'bol': 2, 'tapis': 3, 'lampe': 2, 'plateau': 1, 'coussin': 2}
```

Le **degré** d'un sommet est son nombre de voisins : le tapis (degré 3) est le produit le plus « connecté », donc un bon candidat pour une promotion croisée.

La matrice d'adjacence. On peut ranger le graphe dans une matrice A : $A_{ij} = 1$ si i et j sont reliés, 0 sinon.

```
import numpy as np
idx = {p: i for i, p in enumerate(produits)}
A = np.zeros((5, 5), dtype=int)
for a, b in aretes:
    A[idx[a], idx[b]] = A[idx[b], idx[a]] = 1
print(A)
print("symétrique :", (A == A.T).all())
```

```
[[0 1 0 1 0]
 [1 0 1 0 1]
 [0 1 0 0 1]
 [1 0 0 0 0]]
```

```
[0 1 1 0 0]]
symétrique : True
```

Propriété remarquable : les puissances de A comptent les chemins. L'élément $(A^k)_{ij}$ est le nombre de chemins de longueur k entre i et j .

Preuve pour $k = 2$. Par définition du produit matriciel, $(A^2)_{ij} = \sum_m A_{im}A_{mj}$. Le terme $A_{im}A_{mj}$ vaut 1 exactement quand $i \rightarrow m$ et $m \rightarrow j$ sont deux arêtes, c'est-à-dire quand m est un intermédiaire possible. La somme compte donc les intermédiaires, c'est-à-dire les chemins en deux pas. Le cas général se démontre par récurrence sur k avec le même argument. ■

```
A2 = A @ A
print("chemins de longueur 2 entre bol et lampe :", A2[idx["bol"], idx["lampe"]])
print("chemins de longueur 2 entre tapis et tapis :", A2[idx["tapis"], idx["tapis"]])
A3 = np.linalg.matrix_power(A, 3)
print("trace de A^3 / 6 = nombre de triangles :", np.trace(A3) // 6)
```

```
chemins de longueur 2 entre bol et lampe : 1
chemins de longueur 2 entre tapis et tapis : 3
trace de A^3 / 6 = nombre de triangles : 1
```

Entre le bol et la lampe, il y a un seul chemin en deux pas (bol $\rightarrow$ tapis $\rightarrow$ lampe). Le tapis a trois chemins de longueur 2 vers lui-même : c'est son degré (il part vers un voisin et revient). Et la trace de A^3 compte les triangles (chaque triangle est compté 6 fois : 3 sommets de départ $\times$ 2 sens) ; ici on trouve 1 triangle : tapis-lampe-coussin. Un triangle dans un graphe de co-achat signale un trio de produits qui se vendent bien ensemble.

1.6.4 Parcourir un graphe : la recherche en largeur

Intuition. Vous lancez une pierre dans l'eau : les ondes atteignent d'abord les voisins directs, puis les voisins des voisins, etc. La **recherche en largeur** (*BFS*, *breadth-first search*) explore un graphe de la même manière, « cercle par cercle ». Elle trouve le **plus court chemin** (en nombre d'arêtes) entre deux sommets.

L'algorithme utilise une **file** (premier arrivé, premier servi) :

1. Mettre le sommet de départ dans la file, avec la distance 0.
2. Tant que la file n'est pas vide : retirer le premier sommet ; pour chacun de ses voisins non encore vus, noter sa distance (celle du sommet + 1) et le mettre en fin de file.

```
from collections import deque

def distances_depuis(depart, voisins):
    dist = {depart: 0}
    file = deque([depart])
    while file:
        courant = file.popleft()
        for v in voisins[courant]:
            if v not in dist:
                dist[v] = dist[courant] + 1
                file.append(v)
    return dist

print(distances_depuis("plateau", voisins))
```

```
{'plateau': 0, 'bol': 1, 'tapis': 2, 'lampe': 3, 'coussin': 3}
```

Depuis le plateau : le bol est à distance 1, le tapis à 2, la lampe et le coussin à 3. Si un client achète

un plateau, la « chaîne de recommandations » la plus courte vers une lampe passe par le bol puis le tapis.

À retenir (discret et graphes).

- Ensembles : $\cup, \cap, \setminus$; Jaccard = $|A \cap B|/|A \cup B|$.
- Principe multiplicatif ; $n!$ ordres ; $\binom{n}{k}$ choix sans ordre ; ça explose très vite.
- Un graphe = sommets + arêtes ; le stocker en **liste d'adjacence** ou en **matrice d'adjacence**.
- $(A^k)_{ij}$ = nombre de chemins de longueur k de i à j .
- BFS : exploration par cercles, plus courts chemins sur graphe non pondéré.

1.7 Exercices du chapitre 1

Mode d'emploi. Essayez chaque exercice **sur papier ou dans un notebook avant** de lire le corrigé. Les exercices sont rangés par difficulté croissante : ★ (application directe), ★★ (demande un raisonnement), ★★★ (synthèse). Les corrigés sont juste après l'énoncé de la série, avec le calcul à la main **et** la vérification par code.

Énoncés

Exercice 1 ★ (cosinus). Deux clientes sont décrites par leurs achats (bols, tapis, lampes) : $\mathbf{u} = (1, 2, 2)$ et $\mathbf{v} = (2, 0, 1)$. Calculez leur similarité cosinus. Sont-elles plutôt semblables ?

Exercice 2 ★ (produit matriciel et système). Soit $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ et $\mathbf{b} = \begin{pmatrix} 5 \\ 10 \end{pmatrix}$. (a) Calculez $\mathbf{A}^2$. (b) Résolvez $\mathbf{Ax} = \mathbf{b}$ à la main (par substitution), puis vérifiez avec NumPy.

Exercice 3 ★★ (valeurs propres). Trouvez les valeurs propres et des vecteurs propres de $\mathbf{M} = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$. Vérifiez que la trace est la somme et le déterminant le produit des valeurs propres.

Exercice 4 ★ (dérivées). Pour $f(x) = x^3 - 6x^2 + 9x$, calculez f' , trouvez les points où la pente est nulle, et déterminez s'il s'agit de minima ou de maxima grâce à f'' .

Exercice 5 ★★ (gradient). Pour $f(x, y) = x^2 + 3y^2$, calculez le gradient en $(1, 1)$ puis effectuez **un pas** de descente de gradient avec $\eta = 0,1$. La valeur de f a-t-elle diminué ?

Exercice 6 ★★ (pas d'apprentissage). Soit $f(x) = 2(x - 1)^2$. (a) Écrivez la récurrence de la descente de gradient. (b) Pour quelles valeurs de η converge-t-elle ? (c) Quelle valeur de η converge en un seul pas ?

Exercice 7 ★★ (Lagrange). Yasmine dispose de 10 mètres de ruban pour entourer un rectangle de tissu. Quel rectangle d'aire maximale peut-on former ? Utilisez un multiplicateur de Lagrange, c'est-à-dire maximisez xy sous la contrainte $x + y = 5$ (demi-périmètre), et interprétez λ .

Exercice 8 ★★★ (Python : régression à la main). Les ventes de bougies en fonction de la température sont : $x = (10, 15, 20, 25)$ et $y = (40, 35, 28, 22)$. (a) Ajustez la droite $y = ax + b$ avec les équations normales. (b) Retrouvez le résultat par descente de gradient sur la variable **centrée**. (c) Prédisez les ventes à 18 °C.

Exercice 9 ★★ (analyse numérique). Expliquez pourquoi $0.1 * 3 == 0.3$ est faux en Python et écrivez le test correct. Puis donnez la formule du nombre de chiffres perdus pour un conditionnement $\kappa = 10^6$.

Exercice 10 ★★ (graphes). Dans le graphe de co-achat du 1.6, calculez avec $\mathbf{A}^2$ le nombre de chemins de longueur 2 entre « plateau » et « tapis », puis entre « bol » et « coussin ». Vérifiez en les énumérant à la main.

Corrigés

Corrigé 1. $\mathbf{u} \cdot \mathbf{v} = 1 \cdot 2 + 2 \cdot 0 + 2 \cdot 1 = 4$. $\|\mathbf{u}\| = \sqrt{1 + 4 + 4} = 3$, $\|\mathbf{v}\| = \sqrt{4 + 0 + 1} = \sqrt{5}$. Donc $\cos \theta = \frac{4}{3\sqrt{5}} \approx 0,596$.

```
import numpy as np
u = np.array([1, 2, 2]); v = np.array([2, 0, 1])
cos = u @ v / (np.linalg.norm(u) * np.linalg.norm(v))
print("cosinus =", round(cos, 3), " angle =", round(np.degrees(np.arccos(cos)), 1), "degrés")
```

```
cosinus = 0.596   angle = 53.4 degrés
```

Un cosinus de 0,6 correspond à un angle d'environ 53° : **moyennement semblables** (1 = identiques, 0 = rien en commun). Elles aiment toutes deux les bols mais diffèrent sur les tapis.

Corrigé 2. (a) $\mathbf{A}^2 = \begin{pmatrix} 2 \cdot 2 + 1 \cdot 1 & 2 \cdot 1 + 1 \cdot 3 \\ 1 \cdot 2 + 3 \cdot 1 & 1 \cdot 1 + 3 \cdot 3 \end{pmatrix} = \begin{pmatrix} 5 & 5 \\ 5 & 10 \end{pmatrix}$. (b) Les équations sont $2x + y = 5$ et $x + 3y = 10$. De la première : $y = 5 - 2x$. En remplaçant : $x + 15 - 6x = 10$, donc $-5x = -5$, $x = 1$ et $y = 3$.

```
A = np.array([[2, 1], [1, 3]]); b = np.array([5, 10])
print(A @ A)
print("solution :", np.linalg.solve(A, b))
```

```
[[ 5  5]
 [ 5 10]]
solution : [1. 3.]
```

Corrigé 3. Le polynôme caractéristique est $\det(\mathbf{M} - \lambda \mathbf{I}) = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2)$. Les valeurs propres sont 5 et 2. Pour $\lambda = 5$: $(\mathbf{M} - 5\mathbf{I})\mathbf{v} = \begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $\mathbf{v} = (1, 1)$. Pour $\lambda = 2$: $\begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $\mathbf{v} = (1, -2)$. Trace : $4 + 3 = 7 = 5 + 2$ ✓. Déterminant : $4 \cdot 3 - 1 \cdot 2 = 10 = 5 \times 2$ ✓.

```
M = np.array([[4, 1], [2, 3]])
valeurs, vecteurs = np.linalg.eig(M)
print("valeurs propres :", np.sort(valeurs.real))
print("trace =", np.trace(M), " det =", round(np.linalg.det(M), 6))
print("M @ (1,1) =", M @ np.array([1, 1]), " = 5 * (1,1)")
```

```
valeurs propres : [2. 5.]
trace = 7   det = 10.0
M @ (1,1) = [5 5] = 5 * (1,1)
```

Corrigé 4. $f'(x) = 3x^2 - 12x + 9 = 3(x - 1)(x - 3)$, nulle en $x = 1$ et $x = 3$. $f''(x) = 6x - 12$. En $x = 1$: $f'' = -6 < 0$, c'est un **maximum local** ($f(1) = 4$). En $x = 3$: $f'' = 6 > 0$, c'est un **minimum local** ($f(3) = 0$).

```
f = lambda x: x**3 - 6*x**2 + 9*x
print("f(1) =", f(1), " f(3) =", f(3))
```

```
f(1) = 4   f(3) = 0
```

Corrigé 5. $\nabla f = (2x, 6y)$, donc $\nabla f(1,1) = (2,6)$. Le pas : $(1,1) - 0,1(2,6) = (0,8; 0,4)$. Avant : $f(1,1) = 1 + 3 = 4$. Après : $f(0,8;0,4) = 0,64 + 3 \cdot 0,16 = 1,12$. La valeur a bien diminué (de 4 à 1,12).

```
grad = lambda p: np.array([2 * p[0], 6 * p[1]])
f2 = lambda p: p[0]**2 + 3 * p[1]**2
p = np.array([1.0, 1.0])
p_nouveau = p - 0.1 * grad(p)
print(p_nouveau, f2(p), round(f2(p_nouveau), 2))
```

```
[0.8 0.4] 4.0 1.12
```

Corrigé 6. (a) $f'(x) = 4(x - 1)$, donc $x_{k+1} = x_k - 4\eta(x_k - 1)$, soit $x_{k+1} - 1 = (1 - 4\eta)(x_k - 1)$. (b) L'écart à 1 est multiplié par $(1 - 4\eta)$ à chaque pas ; il tend vers 0 si $|1 - 4\eta| < 1$, c'est-à-dire $0 < \eta < \frac{1}{2}$. (On retrouve $\eta < 2/\lambda_{\max}$ avec $f'' = 4$.) (c) Le facteur est nul si $\eta = \frac{1}{4}$: on tombe sur le minimum en un seul pas.

```
for eta in (0.1, 0.25, 0.45, 0.5, 0.55):
    x = 5.0
    for _ in range(20):
        x = x - eta * 4 * (x - 1)
    print(f"eta = {eta:<5} -> x après 20 pas = {x:.6g}")
```

```
eta = 0.1   -> x après 20 pas = 1.00015
eta = 0.25  -> x après 20 pas = 1
eta = 0.45  -> x après 20 pas = 1.04612
eta = 0.5   -> x après 20 pas = 5
eta = 0.55  -> x après 20 pas = 154.35
```

Corrigé 7. On maximise $g(x,y) = xy$ sous $x + y = 5$. Lagrangien : $\mathcal{L} = xy - \lambda(x + y - 5)$. Les conditions $\partial_x \mathcal{L} = y - \lambda = 0$ et $\partial_y \mathcal{L} = x - \lambda = 0$ donnent $x = y = \lambda$. La contrainte donne $2\lambda = 5$, donc $x = y = 2,5$ m : le carré de 2,5 m de côté, d'aire 6,25 m². Le multiplicateur $\lambda = 2,5$ s'interprète comme le prix de l'ombre : un mètre de demi-périmètre en plus rapporte environ 2,5 m² d'aire en plus (l'aire optimale est $(c/2)^2$ et sa dérivée par rapport à c est $c/2 = 2,5$).

```
from scipy.optimize import minimize
res = minimize(lambda v: -v[0] * v[1], x0=[1, 1],
               constraints={"type": "eq", "fun": lambda v: v[0] + v[1] - 5})
print(res.x.round(3), "aire =", round(-res.fun, 3))
```

```
[2.5 2.5] aire = 6.25
```

Corrigé 8. (a) On construit $\mathbf{X}$ avec une colonne x et une colonne de 1, puis on résout $\mathbf{X}^\top \mathbf{X} \theta = \mathbf{X}^\top \mathbf{y}$. (b) On centre x (moyenne 17,5), on descend le gradient, puis on revient à l'ordonnée d'origine par $b = b' - a\bar{x}$. (c) On évalue la droite en 18. Attention au pas : la hessienne vaut ici $2\mathbf{X}_c^\top \mathbf{X}_c = \text{diag}(250, 8)$, donc il faut $\eta < 2/250 = 0,008$; nous prendrons $\eta = 0,003$.

```
x = np.array([10, 15, 20, 25.0]); y = np.array([40, 35, 28, 22.0])
X = np.column_stack([x, np.ones_like(x)])
a, b = np.linalg.solve(X.T @ X, X.T @ y)
print(f"(a) a = {a:.3f}   b = {b:.3f}")

xc = x - x.mean()
Xc = np.column_stack([xc, np.ones_like(xc)])
theta = np.zeros(2)
for _ in range(500):
    theta = theta - 0.003 * (-2 * Xc.T @ (y - Xc @ theta))
a2, b2 = theta[0], theta[1] - theta[0] * x.mean()
```

```
print(f"(b) a = {a2:.3f} b = {b2:.3f}")
print("(c) ventes à 18 °C :", round(a * 18 + b, 1))
```

```
(a) a = -1.220 b = 52.600
(b) a = -1.220 b = 52.600
(c) ventes à 18 °C : 30.6
```

Lecture : chaque degré de plus fait perdre environ 1,2 ventes de bougies ; à 18 °C on prévoit environ 31 bougies.

Corrigé 9. Le nombre 0,1 n'a pas d'écriture finie en base 2 ; $0.1 * 3$ vaut 0.30000000000000004 , un flottant différent de celui de 0.3. Le bon test est `math.isclose(0.1 * 3, 0.3)`. Pour $\kappa = 10^6$ on perd environ $\log_{10} \kappa = 6$ chiffres significatifs sur les 16 disponibles : il en reste environ 10.

```
import math
print(0.1 * 3, 0.1 * 3 == 0.3, math.isclose(0.1 * 3, 0.3))
```

```
0.30000000000000004 False True
```

Corrigé 10. Avec l'ordre des produits ["bol", "tapis", "lampe", "plateau", "coussin"] (indices 0 à 4), on lit $(A^2)_{\text{plateau}, \text{tapis}}$ et $(A^2)_{\text{bol}, \text{coussin}}$. À la main : plateau—bol—tapis est le seul chemin en deux pas du plateau au tapis (le plateau n'a qu'un voisin, le bol), donc 1. Bol—tapis—coussin est le seul chemin en deux pas du bol au coussin (les voisins du bol sont plateau et tapis ; seul le tapis touche le coussin), donc 1.

```
produits = ["bol", "tapis", "lampe", "plateau", "coussin"]
aretes = [("bol", "plateau"), ("bol", "tapis"), ("tapis", "lampe"),
          ("tapis", "coussin"), ("lampe", "coussin")]
idx = {p: i for i, p in enumerate(produits)}
A = np.zeros((5, 5), dtype=int)
for a_, b_ in aretes:
    A[idx[a_], idx[b_]] = A[idx[b_], idx[a_]] = 1
A2 = A @ A
print("plateau-tapis :", A2[idx["plateau"], idx["tapis"]])
print("bol-coussin   :", A2[idx["bol"], idx["coussin"]])
```

```
plateau-tapis : 1
bol-coussin   : 1
```

Bilan du chapitre 1

Vous savez maintenant :

- **représenter** des données comme des vecteurs et des matrices, mesurer des ressemblances (produit scalaire, cosinus), résoudre des systèmes et comprendre le rang ;
- **décomposer** une matrice (valeurs propres, SVD) pour en extraire la structure — l'idée derrière l'ACP ;
- **dériver** et calculer un gradient pour savoir comment une quantité réagit à ses paramètres ;
- **optimiser** : écrire une fonction de perte, la minimiser par descente de gradient, gérer les contraintes avec Lagrange ;
- **lire** une formule grâce à la fiche de notations, et **se méfier** des arrondis de l'ordinateur.

Le chapitre 2 change de point de vue : jusqu'ici, nos nombres étaient certains. Désormais ils seront **aléatoires**, et il faudra apprendre à raisonner dans l'incertitude.

Chapitre 2 : Probabilités

« Le hasard n'est pas le désordre : c'est un ordre qui apparaît quand on regarde **assez de fois**. »

Dans le chapitre 1, tous nos nombres étaient connus avec certitude. Mais les données réelles ne le sont jamais : demain, Yasmine vendra peut-être 12 bols, peut-être 25. Un client cliquera ou non sur la publicité. Un paiement sera légitime ou frauduleux. Les **probabilités** sont le langage mathématique de cette incertitude, et la **statistique** (chapitre 3) en est la réciproque : à partir de ce qu'on observe, remonter à ce qui se passe « derrière ».

Le chemin de ce chapitre

- **2.1 Probabilités et formule de Bayes** : calculer des chances, mettre à jour ses croyances quand on apprend quelque chose. C'est le cœur du raisonnement en incertitude, et il est plus subtil qu'il n'y paraît.
- **2.2 Variables aléatoires et lois usuelles** : donner un « visage » au hasard avec les lois de Bernoulli, binomiale, de Poisson, uniforme, exponentielle et normale, et savoir laquelle choisir.
- **2.3 Espérance, variance, covariance** : résumer une loi par quelques nombres, et mesurer comment deux variables bougent ensemble.
- **2.4 Loi des grands nombres et théorème central limite** : les deux résultats qui rendent la statistique possible. Pourquoi une moyenne sur beaucoup d'observations devient fiable, et pourquoi la courbe en cloche est partout.
- **⊕ Pour aller plus loin** : la théorie de la mesure (ce que « probabilité » veut vraiment dire) et les processus stochastiques (le hasard qui évolue dans le temps).
- **2.7 Exercices corrigés**.

Deux manières de voir une probabilité.

- **Fréquentiste** : $P(\text{pile}) = 0,5$ signifie que, sur un très grand nombre de lancers, environ la moitié donne pile.
- **Bayésienne** : $P(\text{la livraison arrivera demain}) = 0,8$ exprime un **degré de confiance**, même pour un événement qui n'arrivera qu'une fois.

Les règles de calcul sont les mêmes dans les deux cas. Nous utiliserons les deux interprétations selon les situations.

Notre outil : la simulation. Beaucoup de résultats de ce chapitre se démontrent. Mais on peut aussi **les voir** en faisant « jouer » l'ordinateur des milliers de fois. Nous utiliserons systématiquement les deux : la démonstration pour *comprendre pourquoi*, la simulation pour *y croire*. Avec NumPy, le générateur aléatoire s'obtient par `rng = np.random.default_rng(graine)`. La **graine** (*seed*) fixe la suite de nombres tirés : avec la même graine, vous obtiendrez exactement les mêmes résultats que dans ce livre.

2.1 Probabilités et formule de Bayes

2.1.1 Le vocabulaire : univers, événements, probabilité

Intuition. Une **expérience aléatoire** est une situation dont l'issue est incertaine : lancer un dé, observer si un visiteur achète, mesurer le temps avant la prochaine commande. On liste d'abord **tout ce qui peut arriver**, puis on attribue à chaque possibilité un nombre entre 0 et 1 qui mesure sa chance.

- L'**univers** Ω (oméga) est l'ensemble de toutes les issues possibles. Pour un dé : $\Omega = \{1, 2, 3, 4, 5, 6\}$.
- Un **événement** est une partie de Ω : « obtenir un nombre pair » est $A = \{2, 4, 6\}$.
- La **probabilité** $P(A)$ est un nombre entre 0 (impossible) et 1 (certain).

Quand toutes les issues sont **équiprobables** (un dé non truqué), on compte :

$$P(A) = \frac{\text{nombre d'issues favorables}}{\text{nombre d'issues possibles}} = \frac{|A|}{|\Omega|}.$$

Ici, la section 1.6 sur le dénombrement nous sert directement : $P(\text{pair}) = 3/6 = 0,5$.

Les trois règles du jeu. Toute la théorie des probabilités découle de trois règles (les *axiomes de Kolmogorov*) :

1. $P(A) \geq 0$ pour tout événement A .
2. $P(\Omega) = 1$: il arrive forcément *quelque chose*.
3. Si A et B sont **incompatibles** (jamais ensemble), alors $P(A \cup B) = P(A) + P(B)$.

Trois conséquences immédiates.

- **Complémentaire** : $P(\bar{A}) = 1 - P(A)$. *Preuve* : A et $\bar{A}$ sont incompatibles et leur union est Ω , donc $P(A) + P(\bar{A}) = P(\Omega) = 1$. ■
- **Union quelconque** : $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. *Preuve* : en additionnant $P(A)$ et $P(B)$ on compte deux fois $A \cap B$; on retire donc une fois ce qui est en trop. (Formellement, on écrit $A \cup B$ comme union des trois morceaux incompatibles $A \setminus B$, $A \cap B$, $B \setminus A$.) ■
- **Monotonie** : si $A \subset B$ alors $P(A) \leq P(B)$.

Exemple chiffré. Yasmine envoie une promotion. La chance qu'un client ouvre l'e-mail est $P(O) = 0,4$, celle qu'il visite le site est $P(V) = 0,3$, et celle qu'il fasse les deux est $P(O \cap V) = 0,2$. Quelle est la probabilité qu'il fasse **au moins une** des deux choses ?

$$P(O \cup V) = 0,4 + 0,3 - 0,2 = 0,5.$$

Sans retrancher 0,2, on aurait trouvé 0,7 : on aurait compté deux fois les gens qui font les deux.

2.1.2 Voir une probabilité : la simulation

Une probabilité est la valeur vers laquelle tend la **fréquence** quand on répète l'expérience. Vérifions-le sur un dé, en lançant 10, 100, 1 000, 100 000 fois et en notant la fréquence de « 6 » (qui devrait être proche de $1/6 \approx 0,1667$) :

```
import numpy as np
rng = np.random.default_rng(1)
```



```
for n in (10, 100, 1_000, 100_000):
    lancers = rng.integers(1, 7, size=n)          # entiers de 1 à 6 inclus
    freq = (lancers == 6).mean()
    print(f"{n:>7} lancers : fréquence de 6 = {freq:.4f}    (écart à 1/6 : {abs(freq - 1/6):.4f})")
```

```
10 lancers : fréquence de 6 = 0.2000    (écart à 1/6 : 0.0333)
100 lancers : fréquence de 6 = 0.1500    (écart à 1/6 : 0.0167)
1000 lancers : fréquence de 6 = 0.1650    (écart à 1/6 : 0.0017)
100000 lancers : fréquence de 6 = 0.1665    (écart à 1/6 : 0.0002)
```

Vous voyez l'écart se réduire à mesure que n grandit. Ce phénomène porte un nom, la **loi des grands nombres** ; nous le démontrerons en 2.4.

Astuce de programmation. `(lancers == 6)` crée un tableau de booléens (True/False) ; sa moyenne vaut la **proportion** de True, car True compte pour 1 et False pour 0. C'est l'idiome de base pour estimer une probabilité par simulation.

2.1.3 Probabilité conditionnelle : changer d'information

Intuition. Les probabilités dépendent de ce que l'on sait. La probabilité qu'une personne prise au hasard achète est de 20 %. Mais si l'on **sait** qu'elle vient d'Instagram, ce n'est plus la même question : on se restreint à la population Instagram.

Voici les 1 000 dernières visites de Dar Jasmin, classées par canal d'arrivée et par résultat (achat ou non) :

	Achat	Pas d'achat	Total
Instagram	60	340	400
Site (recherche)	70	280	350
Boutique (passage)	75	175	250
Total	205	795	1 000

- $P(\text{achat}) = 205/1000 = 0,205$.
- $P(\text{achat et Instagram}) = 60/1000 = 0,06$.
- Parmi les 400 visiteurs Instagram, 60 achètent : $P(\text{achat} | \text{Instagram}) = 60/400 = 0,15$.

Cette dernière quantité se note $P(A | B)$, « probabilité de A sachant B ». Remarquez qu'on peut la retrouver à partir des probabilités globales :

$$P(A | B) = \frac{P(A \cap B)}{P(B)} = \frac{0,06}{0,40} = 0,15.$$

C'est la **définition** de la probabilité conditionnelle (valable si $P(B) > 0$) : on divise par $P(B)$ parce qu'on a **réduit l'univers** à B .

```
import pandas as pd          # bibliothèque de tableaux, étudiée en détail à la section 4.4
tableau = pd.DataFrame({"achat": [60, 70, 75], "pas_achat": [340, 280, 175]},
                        index=["Instagram", "Site", "Boutique"])
tableau["total"] = tableau.sum(axis=1)
tableau["P(achat | canal)"] = (tableau["achat"] / tableau["total"]).round(3)
print(tableau)
print("P(achat) global =", tableau["achat"].sum() / tableau["total"].sum())
```

	achat	pas_achat	total	P(achat canal)
Instagram	60	340	400	0.15
Site	70	280	350	0.20
Boutique	75	175	250	0.30
P(achat) global = 0.205				

Le canal « Boutique » convertit deux fois mieux qu'Instagram (30 % contre 15 %). Mais attention à ne pas confondre :

$P(A | B) \neq P(B | A)$. Ici, $P(\text{achat} | \text{Instagram}) = 0,15$; mais $P(\text{Instagram} | \text{achat}) = 60/205 \approx 0,29$. Parmi les acheteurs, 29 % viennent d'Instagram, alors que parmi les visiteurs Instagram, seuls 15 % achètent. Ce sont deux questions différentes. Cette confusion est l'erreur de raisonnement la plus répandue, y compris chez les professionnels (nous en verrons un cas spectaculaire plus bas).

La règle du produit. En réarrangeant la définition :

$$P(A \cap B) = P(A | B) P(B).$$

Pour qu'un visiteur soit à la fois Instagram et acheteur, il faut d'abord qu'il vienne d'Instagram (0,40), puis qu'il achète sachant cela (0,15) : $0,40 \times 0,15 = 0,06$ ✓.

2.1.4 Indépendance

Deux événements sont **indépendants** si en connaître un ne change rien à la probabilité de l'autre : $P(A | B) = P(A)$, ce qui équivaut à

$$P(A \cap B) = P(A) P(B).$$

Dans le tableau, achat et canal sont-ils indépendants ? Si oui, on aurait $P(\text{achat} | \text{Instagram}) = P(\text{achat})$. Or $0,15 \neq 0,205$: **ils ne sont pas indépendants**. Le canal d'arrivée *informe* sur la probabilité d'achat, et c'est précisément ce qu'on cherche en analyse de données : repérer les variables qui informent sur d'autres.

Indépendance $\neq$ incompatibilité. Deux événements incompatibles ($A \cap B = \emptyset$) de probabilités non nulles sont au contraire **très dépendants** : si A arrive, on est *certain* que B n'arrive pas.

Exemple d'événements indépendants. Deux lancers de pièce successifs : $P(\text{pile puis pile}) = 0,5 \times 0,5 = 0,25$. Plus généralement, pour n événements indépendants, on multiplie.

Application : « au moins un ». Chaque colis a 2 % de chances d'être endommagé, indépendamment des autres. Sur 30 colis, quelle est la probabilité qu'**au moins un** soit endommagé ? Le complémentaire est « aucun n'est endommagé », de probabilité $0,98^{30}$. Donc

$$P(\text{au moins un}) = 1 - 0,98^{30}.$$

```
print("P(au moins un colis endommagé sur 30) =", round(1 - 0.98 ** 30, 4))
```

$P(\text{au moins un colis endommagé sur 30}) = 0.4545$

Environ 45 % : avec 30 colis, il est presque *aussi probable qu'improbable* d'avoir au moins un problème, bien que chaque colis soit sûr à 98 %. Passer par le complémentaire est le réflexe à avoir pour tout « au moins un ».

2.1.5 La formule des probabilités totales

Intuition. Pour trouver la probabilité d'un événement, on peut **découper** la population en groupes qui ne se chevauchent pas et dont l'union est tout l'univers (une **partition**), calculer la probabilité dans chaque groupe, puis faire la moyenne **pondérée** par la taille des groupes.

Si $B_1, \dots, B_k$ forment une partition de Ω :

$$P(A) = \sum_{j=1}^k P(A | B_j) P(B_j).$$

Preuve. Les événements $A \cap B_j$ sont deux à deux incompatibles et leur union est A . Par l'axiome 3, $P(A) = \sum_j P(A \cap B_j)$. On applique ensuite la règle du produit à chaque terme : $P(A \cap B_j) = P(A | B_j)P(B_j)$. ■

Exemple. Reprenons le tableau : $P(\text{achat}) = 0,15 \times 0,40 + 0,20 \times 0,35 + 0,30 \times 0,25 = 0,06 + 0,07 + 0,075 = 0,205$. On retrouve bien 205/1000.

2.1.6 La formule de Bayes

Voici le résultat le plus célèbre de ce chapitre. On connaît $P(A | B)$ et on veut **renverser** la condition pour obtenir $P(B | A)$.

Formule de Bayes. Pour $P(A) > 0$ et $P(B) > 0$,

$$P(B | A) = \frac{P(A | B) P(B)}{P(A)}.$$

Preuve. La règle du produit s'écrit de deux façons : $P(A \cap B) = P(A | B)P(B) = P(B | A)P(A)$. En divisant la seconde égalité par $P(A)$, on obtient la formule. ■

Avec la partition $B_1, \dots, B_k$ et la formule des probabilités totales au dénominateur :

$$P(B_i | A) = \frac{P(A | B_i) P(B_i)}{\sum_j P(A | B_j) P(B_j)}.$$

Les trois ingrédients ont des noms qu'on retrouvera tout au long de la data science :

Terme	Nom	Signification
$P(B)$	a priori (<i>prior</i>)	ce que l'on croyait <i>avant</i> d'observer
$P(A B)$	vraisemblance (<i>likelihood</i>)	à quel point l'observation est probable <i>si B</i> est vrai
$P(B A)$	a posteriori (<i>posterior</i>)	ce que l'on croit <i>après</i> avoir observé

Exemple fondateur : l'alerte antifraude. Dar Jasmin reçoit des paiements en ligne. Parmi eux, 1 % sont frauduleux. Le système de détection se déclenche pour 95 % des paiements frauduleux

(c'est sa *sensibilité*), mais aussi pour 5 % des paiements légitimes (ses *fausses alertes*). Un paiement déclenche l'alerte. **Quelle est la probabilité qu'il soit réellement frauduleux ?**

Réfléchissez avant de lire : beaucoup de gens répondent « environ 95 % ». Calculons.

Méthode des « fréquences naturelles » (la plus intuitive) : imaginons **10 000 paiements**.

- 1 % de fraudes : **100** paiements frauduleux. Le système en repère 95 % : **95** alertes justifiées.
- 99 % de légitimes : **9 900** paiements. Le système se trompe pour 5 % d'entre eux : **495** fausses alertes.
- Au total : $95 + 495 = 590$ alertes, dont seulement 95 sont justifiées.

$$P(\text{fraude} \mid \text{alerte}) = \frac{95}{590} \approx 0,161.$$

Moins de 16 %. Une alerte sur six seulement est une vraie fraude. La formule de Bayes donne exactement la même chose :

$$P(F \mid A) = \frac{P(A \mid F) P(F)}{P(A \mid F) P(F) + P(A \mid \bar{F}) P(\bar{F})} = \frac{0,95 \times 0,01}{0,95 \times 0,01 + 0,05 \times 0,99} = \frac{0,0095}{0,0590}.$$

```
p_f = 0.01          # a priori : proportion de fraudes
sens = 0.95         # P(alerte | fraude)
fausse_alerte = 0.05 # P(alerte | légitime)

p_alerte = sens * p_f + fausse_alerte * (1 - p_f) # probabilités totales
p_f_sachant_alerte = sens * p_f / p_alerte        # Bayes
print("P(alerte) =", round(p_alerte, 4))
print("P(fraude | alerte) =", round(p_f_sachant_alerte, 4))
```

```
P(alerte) = 0.059
P(fraude | alerte) = 0.161
```

Pourquoi est-ce si contre-intuitif ? Parce que la fraude est **rare** (1 %). Même un détecteur très bon produit, sur l'immense masse de paiements honnêtes, beaucoup plus de fausses alertes que de vraies. On appelle cela l'**erreur du taux de base** (*base rate fallacy*). Elle explique pourquoi un test médical « fiable à 95 % » pour une maladie rare donne surtout de faux positifs, et pourquoi il est si difficile de repérer des événements rares (fraude, panne, défaut).

Mettre à jour en continu. Supposons que le **même paiement** soit examiné par un second système indépendant, qui se déclenche aussi. La probabilité *a posteriori* d'hier devient l'*a priori* d'aujourd'hui :

```
def mise_a_jour(prior, sens, fausse_alerte):
    """Probabilité de fraude après une nouvelle alerte."""
    return sens * prior / (sens * prior + fausse_alerte * (1 - prior))

p = 0.01
for i in range(1, 4):
    p = mise_a_jour(p, 0.95, 0.05)
    print(f"après {i} alerte(s) : P(fraude) = {p:.3f}")
```

```
après 1 alerte(s) : P(fraude) = 0.161
après 2 alerte(s) : P(fraude) = 0.785
après 3 alerte(s) : P(fraude) = 0.986
```

Après une alerte : 16 %. Après deux : 78,5 %. Après trois : 98,6 %. Chaque nouvelle information **déplace** la croyance, et c'est exactement ce que fait un modèle bayésien. Ce schéma « prior →

vraisemblance → posterior » est le fondement de l'inférence bayésienne et du filtre antispam que nous esquissons plus bas.

2.1.7 Application : un mini-classifieur (le filtre naïf de Bayes)

Yasmine reçoit trop d'avis clients à lire un par un. Elle veut les classer automatiquement en « positif » ou « négatif ». Voici un échantillon de 20 avis déjà étiquetés, où l'on note si le mot « cassé » y apparaît :

	Avis positif	Avis négatif
Nombre d'avis	14	6
... dont contenant « cassé »	1	5

Un nouvel avis contient « cassé ». Est-il positif ou négatif ?

- A priori : $P(\text{pos}) = 14/20 = 0,7$ et $P(\text{nég}) = 0,3$.
- Vraisemblances : $P(\text{cassé} | \text{pos}) = 1/14$, $P(\text{cassé} | \text{nég}) = 5/6$.

$$P(\text{nég} | \text{cassé}) = \frac{\frac{5}{6} \times 0,3}{\frac{5}{6} \times 0,3 + \frac{1}{14} \times 0,7} = \frac{0,25}{0,25 + 0,05} \approx 0,833.$$

```
prior = {"pos": 14 / 20, "neg": 6 / 20}
vrais = {"pos": 1 / 14, "neg": 5 / 6}
non_normalise = {c: prior[c] * vrais[c] for c in prior}
total = sum(non_normalise.values())
print({c: round(v / total, 3) for c, v in non_normalise.items()})
```

```
{'pos': 0.167, 'neg': 0.833}
```

L'avis est négatif avec 83 % de confiance. Le **classifieur naïf de Bayes** étend cette idée à des dizaines de mots à la fois, en supposant (naïvement) que les mots sont indépendants entre eux *sachant la classe*, ce qui permet de **multiplier** leurs vraisemblances. Malgré cette hypothèse fautive, il marche étonnamment bien pour le texte ; nous le retrouverons au volume II.

2.1.8 Trois paradoxes pour s'entraîner à se méfier de l'intuition

Le problème de Monty Hall. Dans un jeu télévisé, trois portes : derrière l'une, une voiture ; derrière les deux autres, une chèvre. Vous choisissez la porte 1. L'animateur, qui **sait** où est la voiture, ouvre une porte où il y a une chèvre (disons la 3) et vous propose de **changer** pour la porte 2. Faut-il changer ?

Beaucoup pensent « c'est du 50/50, donc peu importe ». Simulons plutôt :

```
rng = np.random.default_rng(3)
n = 100_000
voiture = rng.integers(0, 3, size=n) # porte gagnante
choix = np.zeros(n, dtype=int) # on choisit toujours la porte 0

# si on ne change pas, on gagne quand notre porte est la bonne
gain_rester = (voiture == choix).mean()
# si on change : l'animateur élimine une chèvre, on prend l'autre porte ;
# on gagne donc exactement quand notre premier choix était MAUVAIS
gain_changer = (voiture != choix).mean()
print("gain en restant :", round(gain_rester, 3))
print("gain en changeant:", round(gain_changer, 3))
```

```
gain en restant : 0.333
gain en changeant: 0.667
```

Changer fait gagner dans environ **2 cas sur 3**. Explication : votre premier choix a 1/3 de chances d'être le bon, et l'animateur ne change pas cela. Les 2/3 restants se « concentrent » sur l'unique autre porte fermée. (L'animateur apporte de l'information *parce qu'il ne choisit pas au hasard* : il évite la voiture.)

Le paradoxe des anniversaires. Dans une salle de 23 personnes, quelle est la probabilité que deux d'entre elles au moins aient le même anniversaire ? Dites un chiffre avant de continuer. Par le complémentaire (et le principe multiplicatif de 1.6) :

$$P(\text{coïncidence}) = 1 - \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - n + 1}{365}.$$

```
def p_anniversaire(n):
    p_aucune = 1.0
    for i in range(n):
        p_aucune *= (365 - i) / 365
    return 1 - p_aucune

for n in (10, 23, 40, 70):
    print(f"{n:>3} personnes : P(au moins une coïncidence) = {p_anniversaire(n):.3f}")
```

```
10 personnes : P(au moins une coïncidence) = 0.117
23 personnes : P(au moins une coïncidence) = 0.507
40 personnes : P(au moins une coïncidence) = 0.891
70 personnes : P(au moins une coïncidence) = 0.999
```

Dès 23 personnes, la probabilité dépasse 50 % ; à 70, elle est quasi certaine. Contre-intuitif parce qu'on pense à *notre* anniversaire alors que ce sont les $\binom{23}{2} = 253$ **paires** qui comptent. Pour la data science, c'est la clé pour comprendre les **collisions** (deux clients avec le même identifiant haché, deux enregistrements en double) : elles arrivent bien plus tôt qu'on ne l'imagine.

Le paradoxe de Simpson. Une tendance observée dans plusieurs groupes peut s'**inverser** quand on les fusionne. Exemple : Yasmine teste deux versions d'une page de paiement, A et B, auprès de clients sur mobile et sur ordinateur.

	Mobile	Ordinateur	Total
Page A	20 / 100 = 20 %	210 / 700 = 30 %	230 / 800 = 28,75 %
Page B	150 / 700 = 21,4 %	40 / 100 = 40 %	190 / 800 = 23,75 %

B est meilleure sur mobile (21,4 % contre 20 %) **et** sur ordinateur (40 % contre 30 %), pourtant A est meilleure au total (28,75 % contre 23,75 %).

```
import pandas as pd
d = pd.DataFrame({
    "page": ["A", "A", "B", "B"],
    "appareil": ["mobile", "ordi", "mobile", "ordi"],
    "achats": [20, 210, 150, 40],
    "visites": [100, 700, 700, 100]})
d["taux"] = (d["achats"] / d["visites"]).round(3)
print(d)
print(d.groupby("page")[["achats", "visites"]].sum().assign(taux=lambda t: (t.achats /
↪ t.visites).round(4)))
```

	page	appareil	achats	visites	taux
0	A	mobile	20	100	0.200
1	A	ordi	210	700	0.300
2	B	mobile	150	700	0.214
3	B	ordi	40	100	0.400
			achats	visites	taux
	page				
	A		230	800	0.2875
	B		190	800	0.2375

L'explication est que la page A a reçu surtout des clients sur ordinateur (qui achètent beaucoup, quelle que soit la page), alors que B a reçu surtout des clients sur mobile (qui achètent moins). Le total mélange l'effet de la page avec l'effet de l'appareil. La leçon est cruciale : **une moyenne globale peut cacher une variable qui explique tout**. Avant de conclure, on se demande toujours : « quels sous-groupes cachés se mélangent ici ? ». Nous creuserons cette idée de *confusion* au volume II (chapitre 7, facultatif, consacré à l'inférence causale).

À retenir (probabilités et Bayes).

- Une probabilité est un nombre entre 0 et 1 respectant 3 axiomes ; on en déduit $P(\bar{A}) = 1 - P(A)$ et $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.
- Pour un « au moins un », passez par le **complémentaire**.
- $P(A | B) = P(A \cap B) / P(B)$; règle du produit $P(A \cap B) = P(A | B)P(B)$; indépendance $\Leftrightarrow P(A \cap B) = P(A)P(B)$.
- **Probabilités totales** : $P(A) = \sum_j P(A | B_j)P(B_j)$.
- **Bayes** : $P(B | A) = \frac{P(A | B)P(B)}{P(A)}$; *posterior* $\propto$ *vraisemblance* $\times$ *prior*.
- $P(A | B) \neq P(B | A)$, et méfiez-vous de l'**erreur du taux de base** et du **paradoxe de Simpson**.

2.2 Variables aléatoires et lois usuelles

2.2.1 Qu'est-ce qu'une variable aléatoire ?

Intuition. Jusqu'ici nous parlions d'**événements** (« le client achète »). Mais les données sont des **nombres** : le montant du panier, le nombre de commandes, le temps d'attente. Une **variable aléatoire** (v.a.) est simplement **un nombre dont la valeur dépend du hasard**. On la note par une majuscule, X , et ses valeurs possibles par des minuscules, x .

Exemples chez Dar Jasmin :

- X = nombre de commandes reçues entre 14 h et 15 h (0, 1, 2, 3, ...) ;
- Y = montant en dinars du prochain panier (n'importe quel nombre positif) ;
- $B = 1$ si le prochain visiteur achète, 0 sinon.

On distingue deux familles :

	Variable discrète	Variable continue
Valeurs	une liste dénombrable : 0, 1, 2, ...	tout un intervalle de réels
Exemple	nombre de commandes	temps d'attente, montant exact
Description	fonction de masse $P(X = k)$	densité $f(x)$
Probabilité d'un intervalle	somme des $P(X = k)$	aire sous la densité (intégrale du chapitre 1)

Piège à connaître dès maintenant. Pour une variable continue, la probabilité d'une valeur **exacte** est **zéro** : $P(X = 2,5 \text{ min}) = 0$. Avez-vous déjà mesuré un temps d'attente de *exactement* 2,500000... minutes ? Seuls des **intervalles** ont une probabilité : $P(2 \leq X \leq 3)$. La densité $f(x)$ n'est donc *pas* une probabilité : c'est une probabilité *par unité de longueur* (comme une densité de population se mesure en habitants par km²).

La fonction de répartition. Pour les deux familles, on peut définir

$$F(x) = P(X \leq x),$$

la probabilité de rester **sous** le seuil x . Elle croît de 0 à 1. Elle sert à tout : $P(a < X \leq b) = F(b) - F(a)$, et la **probabilité d'excéder** un seuil est $P(X > x) = 1 - F(x)$. Pour une variable continue, $F(x) = \int_{-\infty}^x f(t) dt$, et inversement $f = F'$: c'est le théorème fondamental du 1.2.4.

Dans tout le chapitre, nous utiliserons la bibliothèque **SciPy** (`scipy.stats`), qui fournit pour chaque loi les mêmes méthodes :

Méthode	Rôle
<code>.pmf(k)</code> / <code>.pdf(x)</code>	masse (discret) ou densité (continu)
<code>.cdf(x)</code>	fonction de répartition $F(x) = P(X \leq x)$
<code>.sf(x)</code>	« survie » $P(X > x) = 1 - F(x)$ (plus précis que <code>1 - cdf</code>)
<code>.ppf(q)</code>	quantile : la valeur x telle que $F(x) = q$ (inverse de <code>cdf</code>)
<code>.rvs(size, random_state)</code>	simuler des tirages
<code>.mean()</code> , <code>.var()</code> , <code>.std()</code>	espérance, variance, écart-type (section 2.3)

2.2.2 Loi de Bernoulli : le oui/non

C'est la plus simple : une expérience à **deux issues**, « succès » (1) avec probabilité p , « échec » (0) avec probabilité $1 - p$.

$$P(B = 1) = p, \quad P(B = 0) = 1 - p.$$

Le visiteur d'Instagram qui achète avec probabilité $p = 0,15$ est une Bernoulli(0,15). On la note $B \sim \text{Bern}(p)$ (le symbole $\sim$ se lit « suit la loi »). C'est la brique de base de toutes les prédictions « oui/non » (clic, achat, fraude, désabonnement).

2.2.3 Loi binomiale : compter les succès

Intuition. Vous répétez n fois la même expérience de Bernoulli, **indépendamment** ; la loi binomiale décrit le **nombre de succès**.

Yasmine reçoit 20 visiteurs venant de la publicité ; chacun achète avec la probabilité $p = 0,2$, indépendamment des autres. Soit X le nombre d'acheteurs. Quelle est la probabilité d'avoir **exactement 4 acheteurs** ?

Construction de la formule. Prenons une configuration précise, par exemple : les 4 premiers visiteurs achètent, les 16 suivants non. Par indépendance, sa probabilité est $p^4(1-p)^{16}$. Mais il y a d'autres configurations avec 4 acheteurs : on choisit **lesquels** des 20 visiteurs achètent,

soit $\binom{20}{4}$ façons (section 1.6). Chacune a la même probabilité $p^4(1-p)^{16}$, et les configurations sont incompatibles : on additionne.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, \dots, n.$$

Pour $k = 4$: $\binom{20}{4} = 4845$, donc $P(X = 4) = 4845 \times 0,2^4 \times 0,8^{16} \approx 0,218$.

```
from scipy import stats
import numpy as np
import math

n, p = 20, 0.2
# à la main, avec la formule
print("à la main :", round(math.comb(n, 4) * p**4 * (1 - p)**(n - 4), 4))
# avec scipy
X = stats.binom(n, p)
print("scipy      :", round(X.pmf(4), 4))
print("P(X >= 6) :", round(X.sf(5), 4), " (sf(5) = P(X > 5))")
print("P(X <= 2) :", round(X.cdf(2), 4))
```

```
à la main : 0.2182
scipy      : 0.2182
P(X >= 6) : 0.1958 (sf(5) = P(X > 5))
P(X <= 2) : 0.2061
```

Quatre acheteurs est le résultat le plus fréquent (21,8 %), ce qui est logique car $20 \times 0,2 = 4$. Mais **six acheteurs ou plus** arrive avec une probabilité de presque 20 % : un jour « exceptionnel » n'est pas si exceptionnel. Voyez la forme complète :

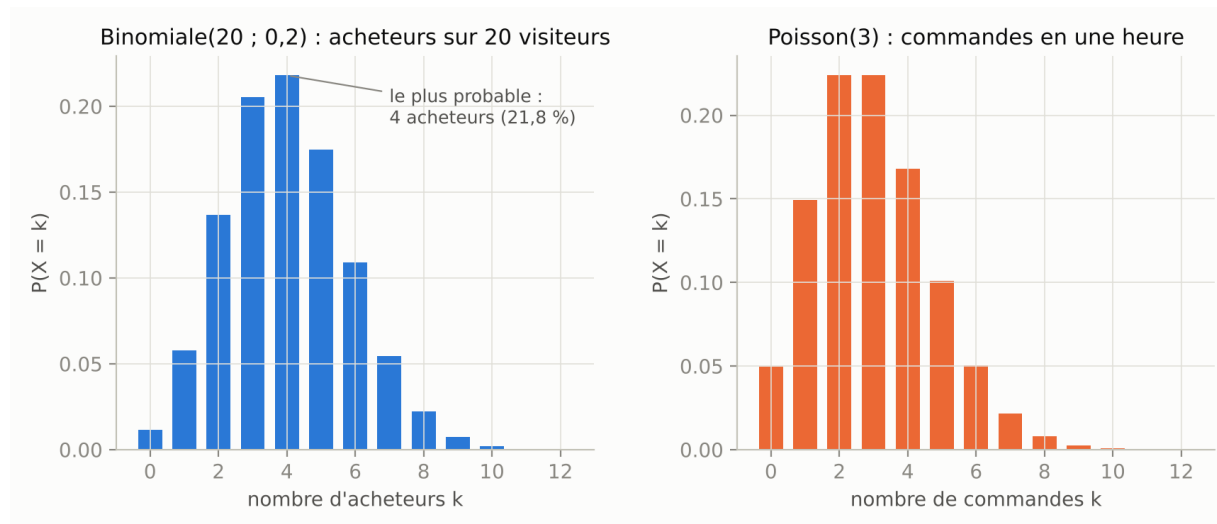


Figure 10: Loi binomiale et loi de Poisson : les probabilités de chaque valeur.

Vérification par simulation. Simulons 100 000 journées de 20 visiteurs :

```
rng = np.random.default_rng(10)
jours = rng.binomial(n=20, p=0.2, size=100_000) # un nombre d'acheteurs par jour
print("fréquence de X = 4 :", round((jours == 4).mean(), 4))
print("fréquence de X >= 6:", round((jours >= 6).mean(), 4))
```

```
fréquence de X = 4 : 0.2195
fréquence de X >= 6: 0.1956
```

Les fréquences simulées retombent sur les valeurs théoriques, à quelques millièmes près.

2.2.4 Loi de Poisson : compter des événements rares

Intuition. On compte les **événements qui surviennent au hasard dans le temps ou l'espace** : appels à un standard, commandes par heure, fautes de frappe par page, pannes par mois. On connaît seulement la **cadence moyenne** λ (« en moyenne 3 commandes par heure »).

$$P(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

Si Yasmine reçoit en moyenne **3 commandes par heure**, la probabilité de ne **recevoir aucune** commande pendant une heure est $e^{-3} \approx 0,0498$, soit 5 % : environ une heure sur vingt est complètement vide. La probabilité d'en recevoir **exactement 3** est $e^{-3} \cdot 3^3/3! \approx 0,224$.

```
Y = stats.poisson(mu=3) # scipy appelle λ « mu »
print("P(0 commande) =", round(Y.pmf(0), 4))
print("P(3 commandes) =", round(Y.pmf(3), 4))
print("P(>= 6) =", round(Y.sf(5), 4))
print("P(>= 10) =", round(Y.sf(9), 5))
```

```
P(0 commande) = 0.0498
P(3 commandes) = 0.224
P(>= 6) = 0.0839
P(>= 10) = 0.0011
```

Six commandes ou plus en une heure arrive environ 8 % du temps : utile pour dimensionner le personnel d'emballage.

D'où vient cette formule : Poisson comme limite de la binomiale. Découpons l'heure en n très petits intervalles (disons $n = 3600$ secondes). Dans chacun, une commande arrive avec une probabilité minuscule $p = \lambda/n$ (pour que la moyenne np reste égale à λ). Le nombre de commandes est alors binomial($n, \lambda/n$) et

$$P(X = k) = \binom{n}{k} \left(\frac{\lambda}{n}\right)^k \left(1 - \frac{\lambda}{n}\right)^{n-k} = \frac{\lambda^k}{k!} \cdot \frac{n(n-1) \cdots (n-k+1)}{n^k} \cdot \left(1 - \frac{\lambda}{n}\right)^n \left(1 - \frac{\lambda}{n}\right)^{-k}.$$

Quand $n \rightarrow \infty$: la fraction $\frac{n(n-1) \cdots (n-k+1)}{n^k} \rightarrow 1$ (il y a k facteurs, chacun tend vers 1) ; $(1 - \lambda/n)^n \rightarrow e^{-\lambda}$ (la définition de l'exponentielle) ; $(1 - \lambda/n)^{-k} \rightarrow 1$. Il reste $e^{-\lambda} \lambda^k / k!$. ■

Vérifions-le numériquement : binomiale(1000; 0,003) contre Poisson(3) :

```
for k in range(0, 6):
    b = stats.binom(1000, 0.003).pmf(k)
    q = stats.poisson(3).pmf(k)
    print(f"k = {k} : binomiale = {b:.5f} Poisson = {q:.5f}")
```

```
k = 0 : binomiale = 0.04956 Poisson = 0.04979
k = 1 : binomiale = 0.14914 Poisson = 0.14936
k = 2 : binomiale = 0.22415 Poisson = 0.22404
k = 3 : binomiale = 0.22438 Poisson = 0.22404
k = 4 : binomiale = 0.16828 Poisson = 0.16803
k = 5 : binomiale = 0.10087 Poisson = 0.10082
```

Les deux colonnes sont presque identiques. **Règle pratique** : quand n est grand et p petit, on peut remplacer Binomiale(n, p) par Poisson(np).

2.2.5 Les variables continues : la densité

Pour une variable continue, la loi est donnée par une **densité** f : une fonction positive dont l'aire totale vaut 1, avec

$$P(a \leq X \leq b) = \int_a^b f(x) dx.$$

C'est ici que l'intégrale du chapitre 1 trouve son usage : une probabilité est une **aire**. Trois lois continues essentielles :

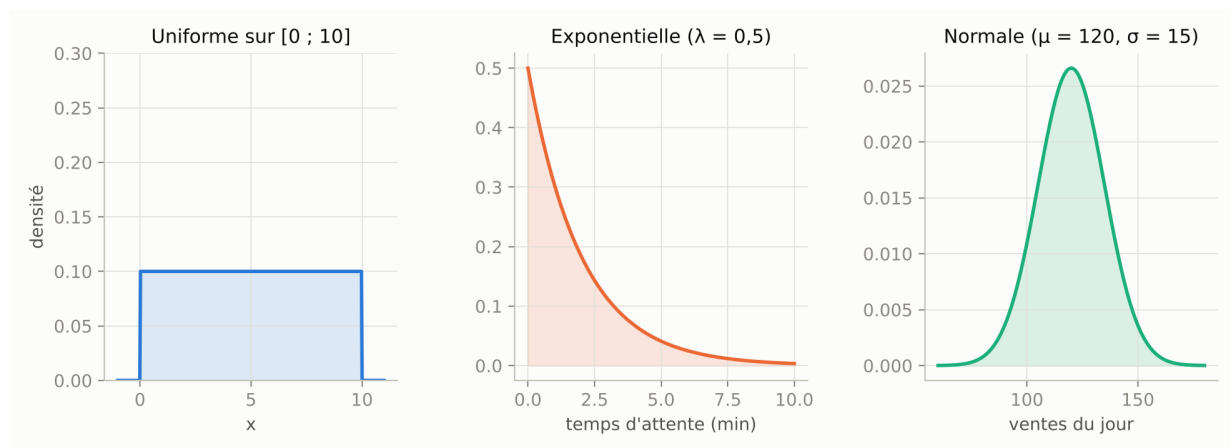


Figure 11: Trois lois continues : uniforme, exponentielle, normale.

Loi uniforme : « aucune préférence »

$X \sim \mathcal{U}(a, b)$: toutes les valeurs de $[a, b]$ sont également probables, de densité $\frac{1}{b-a}$. Un client appelle à une heure uniformément répartie entre 0 et 10 minutes après le début de l'heure : $P(X \leq 3) = 3/10$. Le générateur aléatoire de l'ordinateur produit des $\mathcal{U}(0, 1)$; toutes les autres lois en sont fabriquées.

Loi exponentielle : le temps d'attente

$X \sim \text{Exp}(\lambda)$ avec densité $f(x) = \lambda e^{-\lambda x}$ pour $x \geq 0$. C'est le **temps d'attente** entre deux événements d'un processus de Poisson (nous l'avions rencontrée en 1.2.4). Sa fonction de répartition est $F(x) = 1 - e^{-\lambda x}$, donc

$$P(X > x) = e^{-\lambda x}.$$

Si les clients arrivent à la cadence de $\lambda = 0,5$ par minute (un toutes les 2 minutes en moyenne), la probabilité d'attendre **plus de 3 minutes** le prochain client est $e^{-0,5 \times 3} = e^{-1,5} \approx 0,223$.

```
T = stats.expon(scale=1 / 0.5)          # scipy utilise scale = 1/λ
print("P(attendre > 3 min) =", round(T.sf(3), 4), " (formule : exp(-1.5) =", round(math.exp(-1.5), 4),
      "\n")
print("médiane du temps d'attente :", round(T.ppf(0.5), 3), "min")
print("90 % des attentes sont < ", round(T.ppf(0.90), 2), "min")
```

$P(\text{attendre} > 3 \text{ min}) = 0.2231$ (formule : $\exp(-1.5) = 0.2231$)
 médiane du temps d'attente : 1.386 min
 90 % des attentes sont < 4.61 min

La **médiane** (1,39 min) est inférieure à la moyenne (2 min) : la loi exponentielle est **asymétrique**, avec quelques attentes très longues qui tirent la moyenne vers le haut.

La propriété d'absence de mémoire. Si vous avez déjà attendu 2 minutes sans voir de client, la probabilité d'attendre encore 3 minutes est **la même** que si vous veniez d'arriver. Preuve :

$$P(X > s + t \mid X > s) = \frac{P(X > s + t)}{P(X > s)} = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = e^{-\lambda t} = P(X > t). \blacksquare$$

La première égalité est la définition du conditionnel (car $\{X > s+t\} \subset \{X > s\}$). Le processus « ne se souvient pas » de ce qui s'est passé : il n'y a pas de « retard » à rattraper. (Contre-intuitif pour un bus supposé passer toutes les 10 minutes, mais exact pour des arrivées vraiment aléatoires.)

vérification numérique : $P(X > 5 \mid X > 2) == P(X > 3)$
`print(round(T.sf(5) / T.sf(2), 6), round(T.sf(3), 6))`

0.22313 0.22313

Loi normale : la courbe en cloche

$X \sim \mathcal{N}(\mu, \sigma^2)$, de densité

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Deux paramètres : μ **centre** la cloche, σ (l'écart-type) mesure son **étalement**. On la rencontre partout : tailles, erreurs de mesure, moyennes d'échantillons (on verra pourquoi en 2.4).

Les ventes quotidiennes de Dar Jasmin suivent à peu près $\mathcal{N}(\mu = 120, \sigma = 15)$. Aucune formule fermée n'existe pour la fonction de répartition : on utilise l'ordinateur.

```
V = stats.norm(loc=120, scale=15)
print("P(X > 150)          =", round(V.sf(150), 4))
print("P(105 < X < 135)     =", round(V.cdf(135) - V.cdf(105), 4))
print("jour 'exceptionnel' : 95 % des jours sont sous", round(V.ppf(0.95), 1), "ventes")
```

$P(X > 150)$ = 0.0228
 $P(105 < X < 135)$ = 0.6827
 jour 'exceptionnel' : 95 % des jours sont sous 144.7 ventes

Centrer et réduire : le score z. Comment comparer des valeurs de lois différentes ? On mesure **combien d'écarts-types** on est du centre :

$$z = \frac{x - \mu}{\sigma}.$$

Si $X \sim \mathcal{N}(\mu, \sigma^2)$, alors $Z = (X - \mu)/\sigma \sim \mathcal{N}(0, 1)$, la **loi normale centrée réduite**. Un jour à 150 ventes correspond à $z = (150 - 120)/15 = 2$: deux écarts-types au-dessus de la moyenne. Toutes les probabilités normales se ramènent à celles de $\mathcal{N}(0, 1)$, dont on retient trois repères :

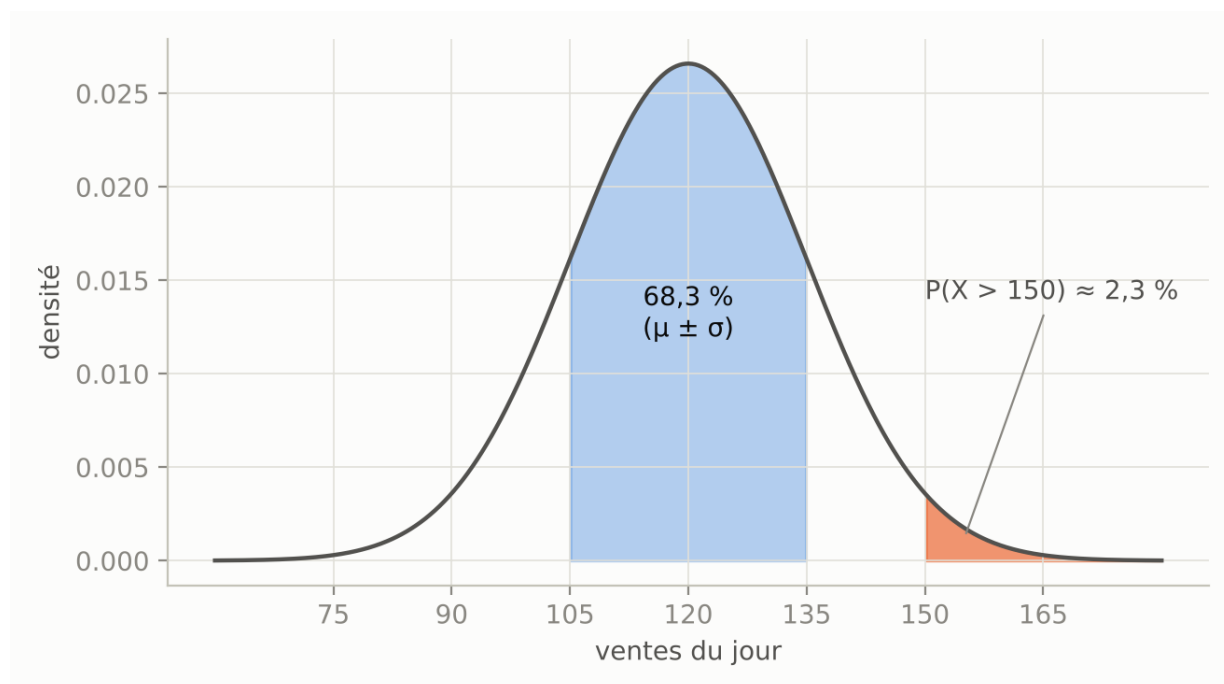


Figure 12: Les ventes quotidiennes suivent $N(120 ; 15^2)$. Un jour à plus de 150 ventes survient environ 2 fois sur 100.

Intervalle	Probabilité
$\mu \pm 1\sigma$	environ 68 %
$\mu \pm 2\sigma$	environ 95 %
$\mu \pm 3\sigma$	environ 99,7 %

```
for k in (1, 2, 3):
    print(f"P(|Z| < {k}) = {stats.norm.cdf(k) - stats.norm.cdf(-k):.4f}")
print("z tel que P(Z < z) = 0.975 :", round(stats.norm.ppf(0.975), 3))
```

```
P(|Z| < 1) = 0.6827
P(|Z| < 2) = 0.9545
P(|Z| < 3) = 0.9973
z tel que P(Z < z) = 0.975 : 1.96
```

Ce dernier nombre, **1,96**, apparaîtra sans cesse dans les intervalles de confiance du chapitre 3.

Test express « est-ce normal ? ». Un jour à 190 ventes ($z = 4,67$) serait extraordinairement rare si la loi était vraiment normale (environ 1 chance sur 700 000 d'être aussi haut). Si cela arrive, la loi n'est probablement pas la bonne ou un événement particulier s'est produit (promotion, fête). Détecter les valeurs avec $|z|$ très grand est la méthode de base de détection d'anomalies.

2.2.6 Quelle loi choisir ?

Situation	Loi	Paramètres
un oui/non	Bernoulli	p
nombre de succès sur n essais indépendants	Binomiale	n, p

Situation	Loi	Paramètres
nombre d'événements dans un intervalle, cadence λ	Poisson	λ
temps d'attente entre deux événements	Exponentielle	λ
aucune préférence sur un intervalle	Uniforme	a, b
somme de nombreux petits effets, mesures	Normale	μ, σ

À retenir (variables aléatoires et lois).

- Une variable aléatoire est un nombre issu du hasard. Discrète : on décrit $P(X = k)$. Continue : on décrit une densité, et les probabilités sont des **aires**.
- La fonction de répartition $F(x) = P(X \leq x)$; avec `scipy.stats` : `pmf/pdf`, `cdf`, `sf`, `ppf`, `rvs`.
- Bernoulli (oui/non), binomiale (compter les succès, $\binom{n}{k} p^k (1-p)^{n-k}$), Poisson (événements rares, $e^{-\lambda} \lambda^k / k!$).
- Exponentielle : temps d'attente, sans mémoire. Normale : cloche, score z , règle 68-95-99,7.
- Simulez toujours pour vérifier vos formules.

2.3 Espérance, variance, covariance

Une loi complète (une courbe, un tableau) est riche mais encombrante. Dans la pratique, on la résume par **quelques nombres** : où est son centre ? De combien s'étale-t-elle ? Comment deux variables évoluent-elles ensemble ?

2.3.1 L'espérance : la moyenne « à long terme »

Intuition. L'**espérance** $E[X]$ est la valeur moyenne que prendrait X si on répétait l'expérience un très grand nombre de fois. C'est une **moyenne pondérée** : chaque valeur est comptée proportionnellement à sa probabilité.

Pour une variable discrète et pour une variable continue (de densité f) :

$$E[X] = \sum_k k P(X = k) \quad \text{et} \quad E[X] = \int_{-\infty}^{+\infty} x f(x) dx.$$

Exemple 1 : le dé. $E[X] = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \dots + 6 \cdot \frac{1}{6} = \frac{21}{6} = 3,5$. Remarquez que l'espérance n'est **pas** une valeur possible : on n'obtiendra jamais 3,5. C'est un centre de gravité, pas un résultat.

Exemple 2 : une décision. Yasmine hésite à lancer une nouvelle lampe. Elle envisage trois scénarios pour le bénéfice du premier trimestre :

Scénario	Probabilité	Bénéfice (DT)
Grand succès	0,3	+5 000
Succès moyen	0,5	+1 000
Échec	0,2	-3 000

$$E[X] = 0,3 \times 5000 + 0,5 \times 1000 + 0,2 \times (-3000) = 1500 + 500 - 600 = 1400.$$

Le lancement rapporte **en moyenne** 1 400 DT. Une option alternative sûre rapporterait 1 200 DT. Faut-il lancer ? L'espérance seule dit oui, mais elle ne dit rien du **risque** : il y a 20 % de chances de perdre de l'argent. C'est exactement le rôle de la variance, ci-dessous.

```
import numpy as np
benefices = np.array([5000, 1000, -3000])
probas = np.array([0.3, 0.5, 0.2])

esperance = (benefices * probas).sum()
print("E[X] =", esperance)
```

E[X] = 1400.0

Propriétés de l'espérance

Linéarité. Pour toutes variables X, Y et constantes a, b :

$$E[aX + b] = aE[X] + b, \quad E[X + Y] = E[X] + E[Y].$$

Preuve de la première (cas discret). $E[aX + b] = \sum_k (ak + b)P(X = k) = a \sum_k kP(X = k) + b \sum_k P(X = k) = aE[X] + b \cdot 1$. ■

La seconde se démontre de la même façon avec une somme double. Elle est **toujours vraie, même si X et Y sont dépendantes** : c'est ce qui la rend si puissante.

Exemple d'usage. Si les ventes du jour ont une espérance de 120 bols à 25 DT l'unité, les recettes $25X$ ont pour espérance $25 \times 120 = 3000$ DT : on multiplie simplement.

Application élégante : l'espérance d'une binomiale. Une binomiale $X \sim \text{Bin}(n, p)$ est la somme de n Bernoulli : $X = B_1 + \dots + B_n$, avec $E[B_i] = 1 \cdot p + 0 \cdot (1 - p) = p$. Par linéarité,

$$E[X] = E[B_1] + \dots + E[B_n] = np.$$

Sans aucun calcul avec $\binom{n}{k}$! Pour nos 20 visiteurs à 20 % : $E[X] = 4$, ce qu'on avait deviné.

Loi	Espérance
Bernoulli(p)	p
Binomiale(n, p)	np
Poisson(λ)	λ
Exponentielle(λ)	$1/\lambda$
Uniforme(a, b)	$(a + b)/2$
Normale(μ, σ^2)	μ

Attention : $E[XY] \neq E[X]E[Y]$ **en général**, et $E[f(X)] \neq f(E[X])$ en général. Exemple : pour le dé, $E[X^2] = \frac{91}{6} \approx 15,17$, alors que $(E[X])^2 = 12,25$. L'écart entre ces deux nombres est justement la **variance**.

2.3.2 La variance : mesurer l'étalement

Deux commerçants ont chacun un bénéfice moyen de 1 000 DT par mois. Chez l'un, c'est toujours entre 950 et 1 050. Chez l'autre, ça varie de −2 000 à +4 000. Même espérance, risques très différents. La **variance** mesure l'écart typique au centre :

$$\text{Var}(X) = E[(X - \mu)^2], \quad \mu = E[X].$$

On prend le **carré** de l'écart pour que les écarts positifs et négatifs ne se compensent pas. L'**écart-type** $\sigma = \sqrt{\text{Var}(X)}$ ramène le résultat à l'unité d'origine (des dinars, pas des dinars²).

Formule de calcul (« moyenne des carrés moins carré de la moyenne »).

$$\text{Var}(X) = E[X^2] - (E[X])^2.$$

Preuve. On développe le carré : $(X - \mu)^2 = X^2 - 2\mu X + \mu^2$. Par linéarité, $E[(X - \mu)^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - 2\mu^2 + \mu^2 = E[X^2] - \mu^2$. ■

(Rappel de la section ⊕ analyse numérique : cette formule est parfaite sur le papier, mais dangereuse sur ordinateur si μ est grand devant σ .)

Retour à la lampe. Calculons $E[X^2]$ puis la variance :

$$E[X^2] = 0,3 \times 5000^2 + 0,5 \times 1000^2 + 0,2 \times 3000^2 = 7,5 \cdot 10^6 + 0,5 \cdot 10^6 + 1,8 \cdot 10^6 = 9,8 \cdot 10^6,$$

$$\text{Var}(X) = 9,8 \cdot 10^6 - 1400^2 = 9,8 \cdot 10^6 - 1,96 \cdot 10^6 = 7,84 \cdot 10^6, \quad \sigma = 2800 \text{ DT}.$$

```
e_carre = (benefices**2 * probas).sum()
variance = e_carre - esperance**2
print("E[X^2]   =", e_carre)
print("Var(X)   =", variance)
print("écart-type =", np.sqrt(variance))
print("P(perte)  =", probas[benefices < 0].sum())
```

```
E[X^2]   = 9800000.0
Var(X)   = 7840000.0
écart-type = 2800.0
P(perte)  = 0.2
```

L'écart-type (2 800 DT) est **deux fois plus grand** que l'espérance (1 400 DT) : l'option est très risquée. Face à l'option sûre à 1 200 DT, le gain moyen n'est supérieur que de 200 DT alors que le risque est considérable. Beaucoup de gens (et de gérants) préféreraient l'option sûre. Il n'y a pas de « bonne » réponse mathématique : la variance **quantifie** le risque pour que la décision soit éclairée.

Propriétés de la variance

Effet d'un changement d'échelle. $\text{Var}(aX + b) = a^2 \text{Var}(X)$.

Preuve. $E[aX + b] = a\mu + b$, donc $(aX + b) - E[aX + b] = a(X - \mu)$ et $\text{Var}(aX + b) = E[a^2(X - \mu)^2] = a^2 \text{Var}(X)$. ■

Conséquences : ajouter une constante ($+b$) ne change pas l'étalement ; multiplier par a multiplie l'écart-type par $|a|$. Convertir des dinars en euros multiplie l'écart-type par le taux de

change, et c'est tout.

Somme de variables indépendantes. Si X et Y sont **indépendantes**, $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$. (Nous démontrons le cas général au 2.3.3.)

Variance d'une binomiale. B_i de Bernoulli : $E[B_i^2] = p$ (car $B_i^2 = B_i$), donc $\text{Var}(B_i) = p - p^2 = p(1 - p)$. Pour $X = \sum B_i$ avec des B_i indépendantes : $\text{Var}(X) = np(1 - p)$.

Loi	Variance
Bernoulli(p)	$p(1 - p)$
Binomiale(n, p)	$np(1 - p)$
Poisson(λ)	λ (variance = espérance !)
Exponentielle(λ)	$1/\lambda^2$
Uniforme(a, b)	$(b - a)^2/12$
Normale(μ, σ^2)	σ^2

Vérifions numériquement avec `scipy` et par simulation :

```
from scipy import stats
rng = np.random.default_rng(21)

lois = {"Binomiale(20; 0,2)": (stats.binom(20, 0.2), rng.binomial(20, 0.2, 200_000)),
        "Poisson(3)": (stats.poisson(3), rng.poisson(3, 200_000)),
        "Exponentielle(0,5)": (stats.expon(scale=2), rng.exponential(2, 200_000)),
        "Normale(120; 15)": (stats.norm(120, 15), rng.normal(120, 15, 200_000))}

print(f"{'loi':<20}{'E théo':>9}{'E simulé':>10}{'Var théo':>10}{'Var simulée':>12}")
for nom, (loi, ech) in lois.items():
    print(f"{'nom':<20}{'loi.mean()':>9.3f}{'ech.mean()':>10.3f}{'loi.var()':>10.3f}{'ech.var()':>12.3f}")
```

loi	E théo	E simulé	Var théo	Var simulée
Binomiale(20; 0,2)	4.000	4.004	3.200	3.209
Poisson(3)	3.000	3.005	3.000	3.006
Exponentielle(0,5)	2.000	2.001	4.000	3.968
Normale(120; 15)	120.000	120.001	225.000	225.467

La **loi de Poisson** a la propriété particulière que sa variance est égale à son espérance. Si les commandes de Yasmine varient *beaucoup plus* que leur moyenne (on parle de **sur-dispersion**), c'est un signe que le modèle de Poisson est trop simple.

2.3.3 Covariance et corrélation : bouger ensemble

Intuition. Les jours où Yasmine dépense plus en publicité, vend-elle plus ? On cherche à mesurer si deux variables **varient dans le même sens**. Chaque jour, on regarde si X est au-dessus de sa moyenne et si Y l'est aussi : si les deux écarts ont **le même signe** la plupart du temps, la covariance est positive.

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] = E[XY] - E[X]E[Y].$$

Exemple à la main. Quatre semaines : dépenses publicitaires $x = (10, 20, 30, 40)$ DT et ventes $y = (12, 18, 26, 32)$.

- Moyennes : $\bar{x} = 25, \bar{y} = 22$.

- Écarts à la moyenne : $x - \bar{x} = (-15, -5, 5, 15)$ et $y - \bar{y} = (-10, -4, 4, 10)$.
- Produits : $(150, 20, 20, 150)$, de somme 340, donc covariance = $340/4 = 85$.

```
x = np.array([10, 20, 30, 40.0]); y = np.array([12, 18, 26, 32.0])
cov_xy = ((x - x.mean()) * (y - y.mean())).mean()
print("covariance (à la main) =", cov_xy)
print("np.cov (n-1)          =", np.cov(x, y)[0, 1].round(2), " <- divise par n-1, voir chapitre 3")
```

```
covariance (à la main) = 85.0
np.cov (n-1)          = 113.33 <- divise par n-1, voir chapitre 3
```

n ou $n - 1$? `np.cov` divise par $n - 1$ (estimateur sans biais, section 3.2) et donne 113,33 ; la formule de la loi divise par n . Pour de grands échantillons la différence disparaît ; ici avec $n = 4$ elle est visible. On y reviendra.

Problème : la covariance dépend des unités. Si on mesure les dépenses en centimes, la covariance est multipliée par 100 sans que la relation change ! On **normalise** en divisant par les écarts-types, ce qui donne la **corrélation** de Pearson :

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1].$$

Pourquoi ρ est toujours entre -1 et 1 . C'est exactement l'inégalité de Cauchy-Schwarz du chapitre 1 ! Rangez les écarts à la moyenne $(x_i - \bar{x})$ dans un vecteur $\mathbf{u}$ et $(y_i - \bar{y})$ dans $\mathbf{v}$. Alors $\text{Cov} \propto \mathbf{u} \cdot \mathbf{v}$, $\sigma_X \propto \|\mathbf{u}\|$, $\sigma_Y \propto \|\mathbf{v}\|$ (avec le même facteur $1/n$), et

$$\rho = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \cos \theta \in [-1, 1].$$

La corrélation est le cosinus de l'angle entre les deux vecteurs d'écarts. $\rho = 1$: même direction ; $\rho = -1$: directions opposées ; $\rho = 0$: vecteurs perpendiculaires (orthogonaux). ■

Ici : $\rho = \frac{85}{\sqrt{125 \times 58}} \approx 0,998$, une relation presque parfaitement linéaire.

```
print("corrélation :", np.corrcoef(x, y)[0, 1].round(4))
```

```
corrélation : 0.9983
```

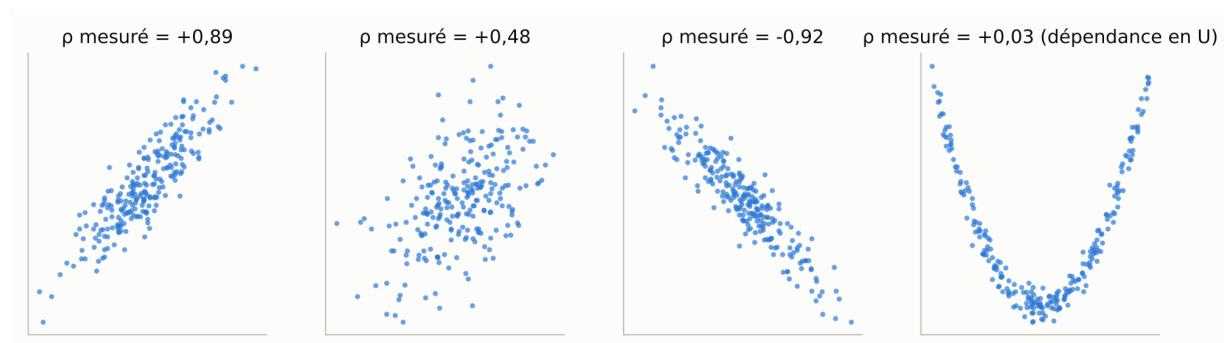


Figure 13: Quatre nuages de points et leur corrélation. Le dernier montre qu'une corrélation nulle n'implique pas l'indépendance.

Trois mises en garde essentielles.

1. **Corrélation n'est pas causalité.** Les glaces et les coups de soleil sont corrélés ; ni l'un ne cause l'autre : c'est la chaleur qui explique les deux. (Le volume II y consacre un chapitre facultatif, l'inférence causale.)
2. **Indépendantes $\Rightarrow$ non corrélées, mais pas l'inverse.** Le dernier nuage de la figure est une parabole : Y est une **fonction exacte** de X (à un peu de bruit près), donc très dépendante, pourtant $\rho \approx 0$. Cela arrive parce que ρ ne détecte que les relations **linéaires**.
3. **Regardez toujours le nuage de points.** Un chiffre unique peut cacher une structure très différente (nous le montrerons avec les « quartets » de la section 3.1).

Variance d'une somme (cas général). $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$.

Preuve. Posons $\tilde{X} = X - \mu_X$ et $\tilde{Y} = Y - \mu_Y$. Alors $\text{Var}(X + Y) = E[(\tilde{X} + \tilde{Y})^2] = E[\tilde{X}^2] + 2E[\tilde{X}\tilde{Y}] + E[\tilde{Y}^2]$, ce qui est bien $\text{Var} X + 2\text{Cov}(X, Y) + \text{Var} Y$. ■

Si X et Y sont indépendantes, $\text{Cov} = 0$ et on retrouve l'additivité des variances.

Application : la diversification. Les ventes quotidiennes de bols (X) et de tapis (Y) ont chacune une moyenne de 50 et un écart-type de 10. Quelle est la variabilité des ventes **totales** $X + Y$?

- Si les deux produits sont **corrélés positivement** ($\rho = +0,5$) : $\text{Var} = 100 + 100 + 2 \times 0,5 \times 100 = 300$, soit $\sigma \approx 17,3$.
- Si les deux produits sont **corrélés négativement** ($\rho = -0,5$: quand l'un se vend mal, l'autre se vend bien) : $\text{Var} = 100 + 100 - 100 = 100$, soit $\sigma = 10$.

```
rng = np.random.default_rng(8)
for rho in (0.5, -0.5):
    cov = [[100, rho * 100], [rho * 100, 100]] # matrice de covariance
    ventes = rng.multivariate_normal([50, 50], cov, size=100_000)
    total = ventes.sum(axis=1)
    print(f"rho = {rho:+.1f} : écart-type du total = {total.std():.2f} (théorie : {np.sqrt(200 + 2 *
        rho * 100):.2f})")
```

```
rho = +0.5 : écart-type du total = 17.29 (théorie : 17.32)
rho = -0.5 : écart-type du total = 9.97 (théorie : 10.00)
```

Mêmes moyennes, mêmes écarts-types individuels, mais un total **bien moins variable** (écart-type de 10 au lieu de 17) quand les produits se compensent. C'est le principe de la **diversification** : on réunit des produits (ou des placements) dont les hauts et les bas ne coïncident pas, pour stabiliser le tout.

La matrice de covariance

Avec plusieurs variables, on range toutes les variances et covariances dans une **matrice de covariance** Σ : variances sur la diagonale, covariances ailleurs. Elle est **symétrique** et ses valeurs propres sont **positives** ; c'est celle dont nous avons calculé les vecteurs propres au 1.1.3 (aperçu de l'ACP).

```
rng = np.random.default_rng(2)
n = 365
temperature = rng.normal(25, 6, n)
visites = 80 + 3 * temperature + rng.normal(0, 15, n)
ventes = 0.2 * visites + rng.normal(0, 3, n)

donnees = np.column_stack([temperature, visites, ventes])
print("matrice de covariance :")
print(np.cov(donnees.T).round(1))
print("matrice de corrélation :")
```

```
print(np.corrcoef(donnees.T).round(2))
```

```
matrice de covariance :
[[ 36.7 106.2  22.6]
 [106.2 529.4 107.1]
 [ 22.6 107.1  31.1]]
matrice de corrélation :
[[1.   0.76 0.67]
 [0.76 1.   0.84]
 [0.67 0.84 1.   ]]
```

La matrice de **corrélation** est la covariance « normalisée » : diagonale de 1 et tout entre -1 et 1 . Ici, on lit que la température est fortement corrélée aux visites, et que les visites sont corrélées aux ventes ; la corrélation température-ventes, un peu plus faible, est un effet **indirect** (la température agit sur les visites, qui agissent sur les ventes).

À retenir (espérance, variance, covariance).

- $E[X]$ = moyenne pondérée à long terme ; **linéaire** : $E[aX + b] = aE[X] + b$, $E[X + Y] = E[X] + E[Y]$ toujours.
- $\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2$; $\sigma = \sqrt{\text{Var}}$; $\text{Var}(aX + b) = a^2 \text{Var}(X)$.
- $\text{Var}(X + Y) = \text{Var } X + \text{Var } Y + 2 \text{Cov}(X, Y)$: la covariance mesure comment les variables se combinent.
- $\rho = \text{Cov} / (\sigma_X \sigma_Y)$ est un **cosinus** : entre -1 et 1 , sans unité. Il ne mesure que le lien **linéaire**, et ne prouve jamais une causalité.
- La matrice de covariance range toutes les covariances ; c'est l'objet central de l'ACP.

2.4 Loi des grands nombres et théorème central limite

Ces deux résultats sont la **raison d'être de la statistique**. Le premier dit : *avec assez de données, la moyenne observée se rapproche de la vraie valeur*. Le second dit : *et l'erreur qui reste suit presque toujours la même courbe en cloche*. Ensemble, ils permettent de transformer un échantillon en conclusion chiffrée avec une marge d'erreur.

2.4.1 La moyenne d'échantillon est elle-même une variable aléatoire

Observons n clients et notons $X_1, \dots, X_n$ leurs paniers. Ces variables sont **indépendantes et identiquement distribuées** (on écrit **i.i.d.**) : indépendantes entre elles, et issues de la même loi, d'espérance μ et de variance σ^2 . Leur **moyenne d'échantillon** est

$$\bar{X}_n = \frac{X_1 + \dots + X_n}{n}.$$

Un point capital, que beaucoup de débutants manquent : $\bar{X}_n$ **est elle-même aléatoire**. Si on reprend un autre échantillon de n clients, on obtient une autre moyenne. Quelle est sa loi ? Par les propriétés de la section 2.3 :

$$E[\bar{X}_n] = \frac{1}{n} \sum E[X_i] = \mu, \quad \text{Var}(\bar{X}_n) = \frac{1}{n^2} \sum \text{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}.$$

- La moyenne d'échantillon est **centrée sur la vraie moyenne** μ (on dit qu'elle est **sans biais**).
- Sa dispersion vaut $\sigma / \sqrt{n}$, appelée **erreur-type** (*standard error*). Elle **diminue** quand n augmente, mais seulement comme $1/\sqrt{n}$: pour diviser l'erreur par 2, il faut **4 fois** plus de données ; par 10, il en faut 100 fois plus.

Pourquoi la variance se divise par n ? Quand on moyenne, les écarts positifs d'un client compensent partiellement les écarts négatifs d'un autre. L'aléa « se dilue » : c'est le même phénomène que la diversification vue en 2.3.3.

2.4.2 La loi des grands nombres

Énoncé (loi faible des grands nombres). Si $X_1, X_2, \dots$ sont i.i.d. d'espérance μ et de variance finie, alors pour tout $\varepsilon > 0$,

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \xrightarrow{n \rightarrow \infty} 0.$$

En mots : la probabilité que la moyenne observée s'écarte de μ de plus de ε **tend vers 0**.

Voyons-le en action sur le taux de conversion de Dar Jasmin (vraie valeur $p = 0,205$). Quatre « expériences » indépendantes observent les visiteurs un à un et notent la proportion d'acheteurs au fil du temps :

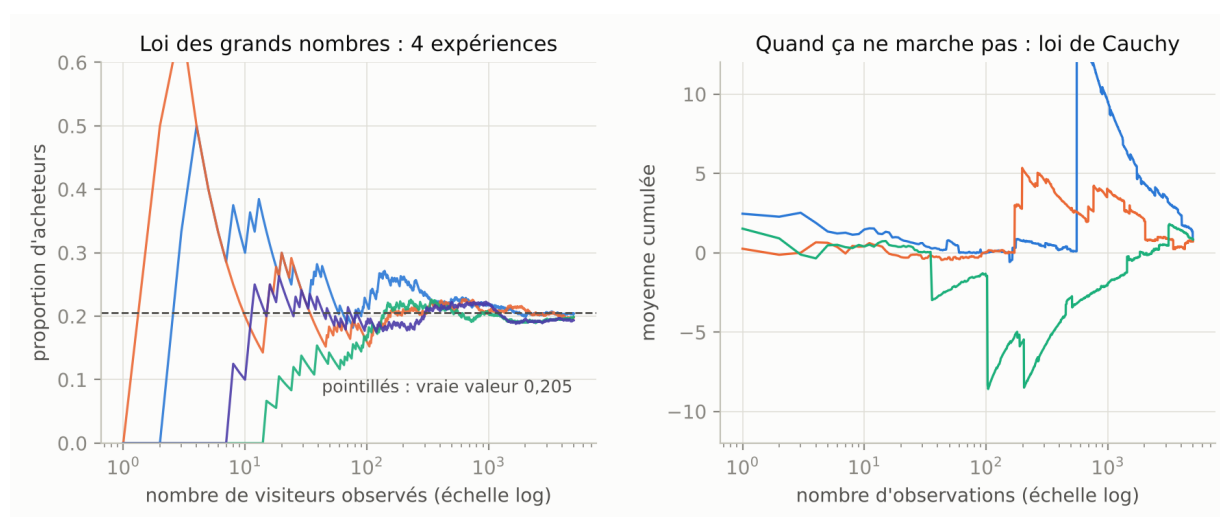


Figure 14: À gauche : la proportion observée se stabilise sur la vraie valeur, quel que soit le départ. À droite : avec une loi de Cauchy, la moyenne ne converge jamais.

Au début, les courbes sont chaotiques (avec 3 visiteurs, la proportion vaut 0, 33 % ou 67 %...) ; puis elles s'**écrasent** autour de 0,205. C'est la loi des grands nombres.

Démonstration. Elle repose sur une inégalité très utile.

Inégalité de Markov. Pour une variable $Y \geq 0$ et $a > 0$: $P(Y \geq a) \leq E[Y]/a$. *Preuve.* $E[Y] \geq E[Y \cdot \mathbb{1}_{Y \geq a}] \geq a P(Y \geq a)$. ■

Inégalité de Tchebychev. On applique Markov à $Y = (X - \mu)^2$ avec $a = \varepsilon^2$:

$$P(|X - \mu| \geq \varepsilon) \leq \frac{\text{Var}(X)}{\varepsilon^2}.$$

Conclusion. Appliquée à $\bar{X}_n$, dont la variance vaut σ^2/n :

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n \varepsilon^2} \xrightarrow{n \rightarrow \infty} 0. \blacksquare$$

La preuve donne même une information **quantitative** : la borne décroît comme $1/n$. Appliquons-

la. Combien de visiteurs faut-il observer pour que la proportion mesurée soit à ± 2 points de la vérité avec une probabilité d'au moins 95 % ? Ici $\sigma^2 = p(1-p) = 0,163$ et $\varepsilon = 0,02$; on veut $\frac{0,163}{n \times 0,0004} \leq 0,05$.

```
import numpy as np
from scipy import stats

p = 0.205
sigma2 = p * (1 - p)
eps = 0.02
n_tchebychev = sigma2 / (0.05 * eps**2)
print("variance d'un acheteur/non-acheteur :", round(sigma2, 4))
print("n garanti par Tchebychev          :", round(n_tchebychev))
```

```
variance d'un acheteur/non-acheteur : 0.163
n garanti par Tchebychev           : 8149
```

Tchebychev garantit le résultat à partir d'environ **8 150 visiteurs**. C'est une borne **sûre mais très pessimiste** (elle marche pour *n'importe quelle* loi). Le théorème central limite, ci-dessous, donnera beaucoup mieux.

L'erreur du joueur. « La roulette est tombée 5 fois sur rouge, le noir est *dû*. » Faux : la loi des grands nombres ne dit **pas** que le hasard « compense » le passé. Chaque tirage est indépendant. Elle dit que la **proportion** se stabilise parce que les premiers tirages sont **dilués** dans une masse de tirages futurs, pas parce que le futur corrige le passé.

Quand elle échoue : la loi de Cauchy. Le graphique de droite montre la moyenne cumulée de tirages d'une loi de Cauchy, une loi aux queues si lourdes qu'elle **n'a pas d'espérance** : des valeurs gigantesques surviennent régulièrement et ruinent la moyenne. La moyenne ne se stabilise *jamais*. Moralité : les hypothèses d'un théorème comptent. Dans les données réelles (revenus, tailles de fichiers, populations de villes), les **valeurs extrêmes** peuvent rendre la moyenne instable ; on utilise alors la **médiane**, plus robuste.

2.4.3 Le théorème central limite

La loi des grands nombres dit *où* va la moyenne ; le **théorème central limite** (TCL) dit *comment* elle fluctue autour.

Énoncé. Soient $X_1, \dots, X_n$ i.i.d. d'espérance μ et de variance σ^2 finie. Alors, quand n est grand,

$$\frac{\bar{X}_n - \mu}{\sigma / \sqrt{n}} \approx \mathcal{N}(0, 1), \quad \text{c'est-à-dire} \quad \bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

La portée est stupéfiante : **quelle que soit la loi d'origine** (asymétrique, discrète, bizarre), la moyenne d'un grand nombre d'observations est **approximativement normale**. C'est la raison pour laquelle la courbe en cloche est partout : *beaucoup de phénomènes sont la somme de nombreux petits effets indépendants*.

Voyons-le. Partons de la loi exponentielle (très asymétrique, voir 2.2.5) et regardons la distribution de la moyenne de $n = 1, 2, 10, 50$ observations, sur 20 000 échantillons. La courbe orange est la loi normale prédite par le TCL.

Le même résultat se mesure avec un seul chiffre, l'**asymétrie** (*skewness*) de la distribution, qui vaut 0 pour une cloche parfaite :

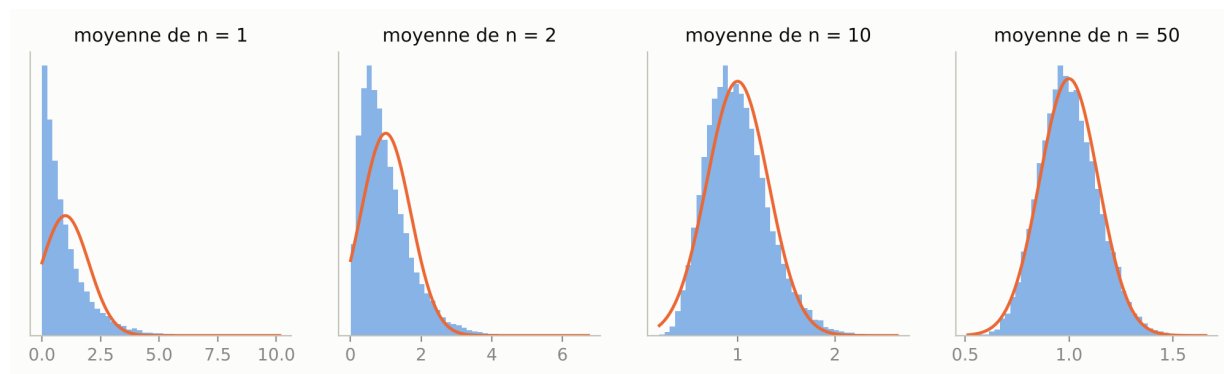


Figure 15: Distribution de la moyenne d'échantillon pour des tirages exponentiels. À $n = 1$ on voit la loi d'origine ; dès $n = 10$ la cloche apparaît ; à $n = 50$ elle est quasi parfaite. La courbe orange est la loi normale prédite par le TCL.

```
rng = np.random.default_rng(4)
for n in (1, 2, 10, 50, 500):
    moyennes = rng.exponential(1.0, size=(20_000, n)).mean(axis=1)
    print(f"n = {n:>3} : moyenne = {moyennes.mean():.3f}   écart-type = {moyennes.std():.3f}"
          f"      (théorie 1/√n = {1 / np.sqrt(n):.3f})   asymétrie = {stats.skew(moyennes):+.2f}")
```

n = 1	: moyenne = 0.997	écart-type = 1.004	(théorie 1/√n = 1.000)	asymétrie = +2.02
n = 2	: moyenne = 1.005	écart-type = 0.706	(théorie 1/√n = 0.707)	asymétrie = +1.41
n = 10	: moyenne = 0.999	écart-type = 0.316	(théorie 1/√n = 0.316)	asymétrie = +0.59
n = 50	: moyenne = 1.000	écart-type = 0.141	(théorie 1/√n = 0.141)	asymétrie = +0.29
n = 500	: moyenne = 0.999	écart-type = 0.045	(théorie 1/√n = 0.045)	asymétrie = +0.12

On observe trois choses : la moyenne reste à 1 (sans biais) ; l'écart-type suit la loi $1/\sqrt{n}$; l'asymétrie s'efface (de 2 pour $n = 1$ vers 0).

Idée de la preuve (esquisse). On étudie la **fonction génératrice des moments** $M(t) = E[e^{tZ}]$ de la variable centrée réduite $Z_n = \sqrt{n}(\bar{X}_n - \mu)/\sigma$. Par indépendance, $M_{Z_n}(t) = [M(t/\sqrt{n})]^n$ où M est celle d'une variable centrée réduite. Un développement de Taylor donne $M(s) = 1 + \frac{s^2}{2} + o(s^2)$ (le terme en s disparaît car l'espérance est nulle, et le coefficient de s^2 est $\text{Var}/2 = \frac{1}{2}$). Donc $M_{Z_n}(t) = (1 + \frac{t^2}{2n} + o(1/n))^n \rightarrow e^{t^2/2}$, qui est précisément la fonction génératrice de $\mathcal{N}(0, 1)$. Une démonstration complète, avec fonctions caractéristiques, relève de la $\oplus$ théorie de la mesure (section 2.5).

2.4.4 Applications

Application 1 : la probabilité sur une moyenne. Les paniers de Dar Jasmin sont très asymétriques (beaucoup de petits achats, quelques gros) ; supposons-les exponentiels de moyenne 60 DT, donc d'écart-type 60 DT. Yasmine regarde les 40 prochains paniers. Quelle est la probabilité que leur **moyenne dépasse 70 DT** ?

Par le TCL, $\bar{X}_{40} \approx \mathcal{N}(60, 60^2/40)$, d'erreur-type $60/\sqrt{40} \approx 9,49$. Le score z est $(70-60)/9,49 \approx 1,05$, d'où $P \approx 0,146$.

```
mu, sigma, n = 60, 60, 40
se = sigma / np.sqrt(n)
approx_tcl = stats.norm(mu, se).sf(70)
```

```
rng = np.random.default_rng(5)
paniers = rng.exponential(60, size=(200_000, n)) # 200 000 échantillons de 40 paniers
```



```
simule = (paniers.mean(axis=1) > 70).mean()
print("erreur-type : ", round(se, 2))
print("P(moyenne > 70) par le TCL : ", round(approx_tcl, 4))
print("P(moyenne > 70) simulée : ", round(simule, 4))
```

```
erreur-type : 9.49
P(moyenne > 70) par le TCL : 0.1459
P(moyenne > 70) simulée : 0.1477
```

Environ 15 % ; l'approximation normale est assez proche de la simulation (elle sous-estime très légèrement la queue de droite, à cause de l'asymétrie de la loi d'origine). Observez ce que le TCL a fait : **sans connaître la loi des paniers**, seulement leur moyenne et leur écart-type, on a répondu à une question de probabilité.

Application 2 : de combien de visiteurs a-t-on besoin ? Reprenons la question du 2.4.2 avec le TCL. La proportion observée $\hat{p} \approx \mathcal{N}(p, p(1-p)/n)$. Avec probabilité 95 %, $\hat{p}$ est à moins de 1,96 erreurs-types de p (le fameux 1,96 du 2.2.5). On veut donc

$$1,96\sqrt{\frac{p(1-p)}{n}} \leq \varepsilon \iff n \geq \left(\frac{1,96}{\varepsilon}\right)^2 p(1-p).$$

```
n_tcl = (1.96 / eps) ** 2 * p * (1 - p)
print("n nécessaire par le TCL : ", round(n_tcl))
print("n garanti par Tchebychev : ", round(n_tchebychev))
```

```
n nécessaire par le TCL : 1565
n garanti par Tchebychev : 8149
```

Le TCL demande environ **1 565 visiteurs** au lieu de 8 150 : **5 fois moins**. Cette formule est celle des **tailles d'échantillon** des sondages et des tests A/B (chapitre 3).

Vérifions par simulation que 1 570 visiteurs (arrondi) suffisent bien à tenir la marge de ± 2 points, dans environ 95 % des cas :

```
rng = np.random.default_rng(6)
n_obs = 1570
taux = rng.binomial(n_obs, p, size=100_000) / n_obs
print("part des échantillons à moins de 2 points de la vérité : ", round((np.abs(taux - p) <=
    eps).mean(), 4))
```

```
part des échantillons à moins de 2 points de la vérité : 0.9514
```

Application 3 : la normale approche la binomiale. Une binomiale est une somme de n Bernoulli ; le TCL dit donc que pour n grand, $\text{Bin}(n, p) \approx \mathcal{N}(np, np(1-p))$. Sur 100 visiteurs à 20 % de conversion, quelle est la probabilité d'avoir **au moins 30 acheteurs** ?

```
b = stats.binom(100, 0.2)
exact = b.sf(29) # P(X >= 30) = P(X > 29)
approx = stats.norm(20, 4).sf(29.5) # correction de continuité : on coupe à 29,5
print("exact (binomiale) : ", round(exact, 4))
print("approx. normale : ", round(approx, 4))
print("approx. sans correction: ", round(stats.norm(20, 4).sf(30), 4))
```

```
exact (binomiale) : 0.0112
approx. normale : 0.0088
approx. sans correction: 0.0062
```

La **correction de continuité** (couper à 29,5 plutôt qu'à 30) améliore sensiblement l'approximation

: on remplace des barres discrètes par une courbe continue, et la barre « 30 » occupe l'intervalle $[29,5 ; 30,5]$.

2.4.5 Un mot de prudence

Le TCL est un résultat **asymptotique** : « $\approx$ » devient exact quand $n \rightarrow \infty$. À partir de quelle taille est-ce valable ? Cela dépend de la loi d'origine :

Loi d'origine	n suffisant (règle empirique)
symétrique (uniforme, normale)	quelques unités à 10
modérément asymétrique (exponentielle)	une trentaine
très asymétrique, queues lourdes	des centaines, voire jamais (si la variance est infinie)

À retenir (LGN et TCL).

- $\bar{X}_n$ est une variable aléatoire : $E[\bar{X}_n] = \mu$, $\text{Var}(\bar{X}_n) = \sigma^2/n$, **erreur-type** = $\sigma/\sqrt{n}$. L'erreur diminue en $1/\sqrt{n}$: quatre fois plus de données pour deux fois moins d'erreur.
- **LGN** : $\bar{X}_n \rightarrow \mu$ (preuve par Tchebychev). Elle ne dit rien d'un « rattrapage » du hasard.
- **TCL** : $\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \approx \mathcal{N}(0, 1)$ pour *toute* loi de variance finie. C'est le pont entre les probabilités et la statistique.
- Utilisations : probabilités sur des moyennes, taille d'échantillon $n \geq (1,96/\varepsilon)^2 p(1-p)$, approximation normale de la binomiale.
- Les hypothèses comptent : avec des queues très lourdes (Cauchy), rien de tout cela ne marche.

2.5 ⊕ Pour aller plus loin : la théorie de la mesure (ce que « probabilité » veut vraiment dire)

Section optionnelle, plus abstraite. Elle n'est pas nécessaire pour la suite du livre. Elle répond à une question que se posent les lecteurs curieux : « *jusqu'ici, on a dit « une probabilité est un nombre entre 0 et 1 qui vérifie des règles », mais pour quels événements est-elle définie, et pourquoi ces règles ?* »

2.5.1 Le problème : on ne peut pas mesurer tous les ensembles

Pour un dé, tout est simple : Ω a six éléments et on peut attribuer une probabilité à **n'importe quelle** partie. Mais prenons une variable **uniforme** sur $[0, 1]$: on voudrait que la probabilité d'un intervalle soit sa **longueur** ($P([a, b]) = b - a$). Peut-on étendre cette idée à **tous** les sous-ensembles de $[0, 1]$ en gardant des règles raisonnables (invariance par translation, additivité) ?

Non. En 1905, Giuseppe Vitali a construit des ensembles « pathologiques » auxquels aucune longueur cohérente ne peut être attribuée (la construction utilise l'axiome du choix). Ce n'est pas une curiosité : cela oblige à **limiter** les événements auxquels on attribue une probabilité.

2.5.2 Le cadre : un espace probabilisé $(\Omega, \mathcal{F}, P)$

La théorie moderne (Kolmogorov, 1933) est fondée sur trois objets.

- Ω : l'**univers** (toutes les issues).

- $\mathcal{F}$: une **tribu** (ou σ -algèbre), c'est-à-dire la famille des événements **dont on sait parler**. Elle doit contenir Ω , être stable par complémentaire et par **union dénombrable**. (Pour $[0, 1]$, on prend la tribu **borélienne**, engendrée par les intervalles.)
- P : une **mesure de probabilité** sur $\mathcal{F}$, avec $P(\Omega) = 1$ et la σ -**additivité** : pour des événements $A_1, A_2, \dots$ deux à deux incompatibles,

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

Notez la différence avec l'axiome 3 du 2.1.1 : on demande l'additivité pour des unions **infinies dénombrables**, pas seulement finies. C'est elle qui rend possibles les passages à la limite (comme la loi des grands nombres).

Intuition. La tribu est la liste des « questions autorisées » : « la variable tombe-t-elle entre 0,3 et 0,4 ? », « est-elle rationnelle ? », « est-elle dans l'union d'une suite d'intervalles ? ». La mesure P répond à chacune par un nombre.

2.5.3 Une conséquence surprenante : événements de probabilité 0 qui arrivent

Soit X uniforme sur $[0, 1]$. Pour tout réel x , $P(X = x) = 0$ (déjà vu en 2.2.1). Et pourtant X prend bien *une* valeur. Un événement de probabilité nulle n'est donc **pas impossible**.

Les rationnels ont une probabilité nulle. Les nombres rationnels de $[0, 1]$ forment un ensemble **dénombrable** : $q_1, q_2, q_3, \dots$. Par σ -additivité,

$$P(X \in \mathbb{Q}) = \sum_{i=1}^{\infty} P(X = q_i) = \sum_{i=1}^{\infty} 0 = 0.$$

Presque sûrement, un nombre tiré au hasard dans $[0, 1]$ est irrationnel ! (On dit qu'un événement est vrai **presque sûrement** (p.s.) quand sa probabilité vaut 1.)

2.5.4 L'espérance comme intégrale

En théorie de la mesure, une **variable aléatoire** est une fonction $X : \Omega \rightarrow \mathbb{R}$ **mesurable** (pour toute question « $X \leq x$? », la réponse est un événement de la tribu). Son espérance est l'**intégrale de Lebesgue** :

$$E[X] = \int_{\Omega} X dP.$$

Cette seule définition recouvre les cas discret ($\sum$) et continu ($\int f$) du 2.3.1, mais aussi les cas **mixtes** que ni l'un ni l'autre ne gère. Voici un exemple pratique.

Un cas réel : les dépenses « à zéros ». Un client visitant la boutique dépense **0 DT** avec une probabilité de 70 % (il regarde sans acheter) ; sinon sa dépense suit une loi exponentielle de moyenne 80 DT. Cette variable n'a **ni** fonction de masse (car elle prend un continuum de valeurs) **ni** densité (car elle a un « atome » en 0 : $P(X = 0) = 0,7 > 0$). C'est une loi **mixte**. Mais son espérance se calcule sans difficulté : on décompose selon le cas.

$$E[X] = 0,7 \times 0 + 0,3 \times 80 = 24 \text{ DT}.$$

```
import numpy as np
rng = np.random.default_rng(31)
n = 500_000
achete = rng.random(n) < 0.3
depense = np.where(achete, rng.exponential(80, size=n), 0.0)

print("P(X = 0) simulée :", round((depense == 0).mean(), 4))
print("E[X] simulée      :", round(depense.mean(), 2), "(théorie : 24)")
print("médiane          :", round(np.median(depense), 2), "(70 % des clients ne dépensent rien)")
```

```
P(X = 0) simulée : 0.6998
E[X] simulée      : 24.03 (théorie : 24)
médiane          : 0.0 (70 % des clients ne dépensent rien)
```

Les données réelles de commerce sont très souvent de ce type (« zero-inflated »), et c'est la raison pour laquelle on ne peut pas toujours plaquer une loi normale ou exponentielle sans réfléchir.

Un conseil pratique. Pour une loi mixte, la moyenne (24) et la médiane (0) racontent des histoires totalement différentes. Résumer par un seul nombre est trompeur ; il faut présenter la part de zéros et la moyenne conditionnelle aux achats.

2.5.5 Les modes de convergence

Quand on dit « $\bar{X}_n$ converge vers μ », encore faut-il dire **en quel sens**. Pour des variables aléatoires, il existe plusieurs façons, de la plus forte à la plus faible :

Mode	Notation	Signification	Exemple
Presque sûre	$X_n \xrightarrow{p.s.} X$	pour (presque) chaque « histoire » ω , la suite de nombres $X_n(\omega)$ converge au sens usuel	loi forte des grands nombres
En probabilité	$X_n \xrightarrow{P} X$	$P(X_n - X > \varepsilon) \rightarrow 0$	loi faible (démontrée en 2.4.2)
En loi	$X_n \xrightarrow{\mathcal{L}} X$	les fonctions de répartition convergent	TCL

On a : p.s. $\Rightarrow$ en probabilité $\Rightarrow$ en loi (et jamais l'inverse en général). La **loi forte** des grands nombres, plus difficile à démontrer, affirme que, pour *presque* chaque suite d'observations, la moyenne cumulée converge. C'est ce que montre chacune des courbes de la figure de la section 2.4.2 : chaque trajectoire se stabilise.

Pourquoi se soucier de cela ? Pour un praticien, la différence compte surtout dans la formulation des garanties : « avec une probabilité 95 %, mon estimation est à ± 2 points » est un énoncé **en probabilité/en loi** sur *un* échantillon ; « mon estimateur finira par donner la bonne valeur » est un énoncé **presque sûr** sur une suite infinie de données.

2.5.6 La densité comme dérivée d'une mesure

Quand dit-on qu'une variable « a une densité » ? Réponse : quand sa loi P_X est **absolument continue** par rapport à la mesure de Lebesgue (la « longueur »), c'est-à-dire que tout ensemble de longueur nulle a une probabilité nulle. Le **théorème de Radon–Nikodym** garantit alors l'existence

d'une densité f telle que $P_X(A) = \int_A f(x) dx$. Dans l'exemple des dépenses à zéros, $P_X(\{0\}) = 0,7$ alors que $\{0\}$ est de longueur nulle : pas de densité, ce que nous avons remarqué.

À retenir (théorie de la mesure).

- On ne peut pas assigner de probabilité à *tous* les sous-ensembles : on se limite à une **tribu** $\mathcal{F}$.
- Un espace probabilisé est $(\Omega, \mathcal{F}, P)$ avec P **σ -additive**.
- Un événement de probabilité 0 n'est pas impossible ; « presque sûrement » = avec probabilité 1.
- Espérance = intégrale de Lebesgue ; elle gère les lois mixtes (atome + densité).
- Trois convergences : presque sûre $\implies$ en probabilité $\implies$ en loi (LGN forte, LGN faible, TCL).

2.6 ☉ Pour aller plus loin : les processus stochastiques

Section optionnelle. Jusqu'ici, une variable aléatoire était un nombre tiré **une fois**. Un **processus stochastique** est une variable aléatoire qui **évolue dans le temps** : $X_0, X_1, X_2, \dots$ Les stocks, les clients actifs, le cours d'une action, le nombre d'appels : tout cela est une suite de variables aléatoires dépendantes les unes des autres.

2.6.1 La marche aléatoire

Intuition. Chaque jour, le stock d'un produit varie de $+1$ ou -1 de façon aléatoire. Où sera-t-il après 100 jours ?

On part de $S_0 = 0$ et on pose $S_n = S_{n-1} + \zeta_n$ où chaque pas ζ_n vaut $+1$ ou -1 avec probabilité $\frac{1}{2}$. Comme $E[\zeta_n] = 0$ et $\text{Var}(\zeta_n) = 1$, les propriétés du 2.3 (somme de variables indépendantes) donnent

$$E[S_n] = 0, \quad \text{Var}(S_n) = n, \quad \sigma(S_n) = \sqrt{n}.$$

Une marche aléatoire n'a donc **pas de tendance**, mais elle s'écarte de 0 de l'ordre de $\sqrt{n}$. Après 100 pas, on s'attend à être à environ 10 de l'origine, pas à 100 ! C'est la même loi en $\sqrt{n}$ que celle de l'erreur-type.

```
import numpy as np
rng = np.random.default_rng(14)

pas = rng.choice([-1, 1], size=(20_000, 100))          # 20 000 marches de 100 pas
position_finale = pas.sum(axis=1)
print("moyenne des positions finales :", round(position_finale.mean(), 3))
print("écart-type des positions      :", round(position_finale.std(), 3), "(théorie :  $\sqrt{100} = 10$ )")
print("P(|S_100| <= 10)              :", round((np.abs(position_finale) <= 10).mean(), 3))
```

```
moyenne des positions finales : 0.082
écart-type des positions      : 10.076 (théorie :  $\sqrt{100} = 10$ )
P(|S_100| <= 10)              : 0.723
```

Par le TCL, $S_n / \sqrt{n} \approx \mathcal{N}(0, 1)$: environ 68 % des marches finissent à moins de $\sqrt{n}$ de l'origine (ici, 72 % : un peu plus que 68 %, car la borne ± 10 est incluse et la marche ne prend que des valeurs paires).

La marche aléatoire est partout. Le cours d'une action est souvent modélisé comme une marche aléatoire (le **mouvement brownien**, limite continue de la marche quand les pas deviennent infiniment petits). Elle explique aussi pourquoi les prévisions à long terme sont si incertaines : l'incertitude grandit en $\sqrt{\text{temps}}$.

2.6.2 Les chaînes de Markov : le futur ne dépend que du présent

Intuition. Dans une **chaîne de Markov**, la probabilité de passer à l'état suivant ne dépend que de l'**état actuel**, pas de la manière dont on y est arrivé :

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P(X_{n+1} = j \mid X_n = i) = P_{ij}.$$

Yasmine classe chaque mois ses clients en trois états : **Actif** (A : au moins 2 achats ce mois), **Occasionnel** (O : 1 achat) et **Inactif** (I : aucun achat). Elle a estimé les transitions d'un mois au suivant :

de ↓ / vers →	Actif	Occasionnel	Inactif
Actif	0,80	0,15	0,05
Occasionnel	0,30	0,50	0,20
Inactif	0,10	0,20	0,70

On range ces nombres dans la **matrice de transition P** : chaque **ligne** est une loi de probabilité (somme égale à 1).

```
P = np.array([[0.80, 0.15, 0.05],
              [0.30, 0.50, 0.20],
              [0.10, 0.20, 0.70]])
etats = ["Actif", "Occasionnel", "Inactif"]
print("somme des lignes :", P.sum(axis=1))
```

```
somme des lignes : [1. 1. 1.]
```

Où sera un client dans 2 mois ? Un client Actif aujourd'hui peut être Actif dans 2 mois de plusieurs façons : $A \rightarrow A \rightarrow A$, $A \rightarrow O \rightarrow A$, $A \rightarrow I \rightarrow A$. La probabilité totale est $0,8 \times 0,8 + 0,15 \times 0,3 + 0,05 \times 0,1 = 0,64 + 0,045 + 0,005 = 0,69$. Mais c'est **exactement** le produit matriciel de la ligne A par la colonne A de **P** ! En général :

Les probabilités de transition en n pas sont les éléments de P^n . (Même mécanisme que pour compter les chemins d'un graphe au 1.6.3 : la formule des probabilités totales fait apparaître le produit matriciel.)

```
P2 = np.linalg.matrix_power(P, 2)
print("P^2 :\n", P2.round(3))
print("P(Actif -> Actif en 2 mois) =", P2[0, 0].round(3))
```

```
P^2 :
[[0.69  0.205 0.105]
 [0.41  0.335 0.255]
 [0.21  0.255 0.535]]
P(Actif -> Actif en 2 mois) = 0.69
```

Et dans 12 mois, ou 2 ans ? On calcule des puissances plus élevées :

```

for n in (1, 3, 6, 12, 24):
    Pn = np.linalg.matrix_power(P, n)
    print(f"n = {n:>2} mois : ligne 'Actif' = {Pn[0].round(4)}   ligne 'Inactif' = {Pn[2].round(4)}")

n = 1 mois : ligne 'Actif' = [0.8  0.15 0.05]   ligne 'Inactif' = [0.1 0.2 0.7]
n = 3 mois : ligne 'Actif' = [0.624 0.227 0.149]   ligne 'Inactif' = [0.298 0.266 0.436]
n = 6 mois : ligne 'Actif' = [0.5368 0.2448 0.2183]   ligne 'Inactif' = [0.4366 0.2581 0.3053]
n = 12 mois : ligne 'Actif' = [0.5034 0.2495 0.247 ]   ligne 'Inactif' = [0.4941 0.2508 0.2551]
n = 24 mois : ligne 'Actif' = [0.5  0.25 0.25]   ligne 'Inactif' = [0.4999 0.25  0.25 ]

```

Observez : à mesure que n grandit, **toutes les lignes deviennent identiques**. Le système « oublie » son point de départ : qu'un client ait commencé Actif ou Inactif, sa probabilité d'être dans chaque état dans 2 ans est la même. Cette loi limite π s'appelle la **distribution stationnaire**.

La calculer exactement : valeurs propres, encore ! La loi stationnaire vérifie $\pi \mathbf{P} = \pi$: elle ne change plus après une transition. En transposant, $\mathbf{P}^\top \pi^\top = \pi^\top$: c'est un **vecteur propre de $\mathbf{P}^\top$ pour la valeur propre 1** (section 1.1.3).

```

valeurs, vecteurs = np.linalg.eig(P.T)
k = np.argmin(np.abs(valeurs - 1))           # la valeur propre égale à 1
pi = np.real(vecteurs[:, k])
pi = pi / pi.sum()                          # normaliser pour que la somme soit 1
for e, v in zip(etats, pi):
    print(f"{e:<12} {v:.4f}")
print("valeurs propres de P :", np.round(np.real(valeurs), 3))

```

```

Actif      0.5000
Occasionnel 0.2500
Inactif     0.2500
valeurs propres de P : [1.    0.673 0.327]

```

À long terme, exactement **50 %** des clients sont Actifs, **25 %** Occasionnels et **25 %** Inactifs. Les deux autres valeurs propres (0,673 et 0,327, de module < 1) pilotent la **vitesse** de convergence : plus elles sont petites, plus vite le système oublie son passé.

Une application économique : la valeur à long terme d'un client. Supposons qu'un client Actif rapporte en moyenne 30 DT par mois, un Occasionnel 10 DT, un Inactif 0 DT. À long terme, le revenu moyen mensuel par client est $\pi \cdot \mathbf{v}$:

```

v = np.array([30, 10, 0])
print("revenu mensuel moyen par client (long terme) :", round(pi @ v, 2), "DT")

```

```
revenu mensuel moyen par client (long terme) : 17.5 DT
```

Soit 17,5 DT par client et par mois. Cette quantité permet à Yasmine de **chiffrer** l'effet d'une campagne : si une relance fait passer la probabilité Inactif→Actif de 0,10 à 0,20, quel est le gain ? Il suffit de modifier $\mathbf{P}$ et de recalculer π .

```

def revenu_long_terme(P, v):
    valeurs, vecteurs = np.linalg.eig(P.T)
    pi = np.real(vecteurs[:, np.argmin(np.abs(valeurs - 1))])
    pi = pi / pi.sum()
    return pi @ v

P_relance = np.array([[0.80, 0.15, 0.05],
                      [0.30, 0.50, 0.20],
                      [0.20, 0.20, 0.60]])           # Inactif -> Actif passe de 0,10 à 0,20
print("avant relance :", round(revenu_long_terme(P, v), 2), "DT")
print("après relance :", round(revenu_long_terme(P_relance, v), 2), "DT")

```

```
avant relance : 17.5 DT
après relance : 19.3 DT
```

La relance fait gagner environ 1,8 DT par client et par mois, soit +10 %. Avec 2 000 clients, cela représente environ 3 600 DT par mois, de quoi décider si la relance vaut son coût. C'est un modèle simple, mais l'idée (états, transitions, régime permanent) est utilisée en analyse de la fidélité, en marketing, et à la base de l'algorithme PageRank de Google (le web est une chaîne de Markov dont les états sont les pages).

2.6.3 Le processus de Poisson : des arrivées au hasard

Un **processus de Poisson** de cadence λ modélise des événements arrivant au hasard et indépendamment : appels, commandes, pannes. Il a deux visages **équivalents**, que nous avons déjà croisés :

- le **nombre** d'événements dans une durée t suit une loi de **Poisson**(λt) (2.2.4) ;
- les **temps d'attente** entre événements successifs sont **exponentiels**(λ), indépendants (2.2.5).

Vérifions que ces deux visages coïncident, en construisant le processus par ses temps d'attente puis en comptant :

```
from scipy import stats
rng = np.random.default_rng(15)
lam = 3                                # 3 commandes par heure
n_heures = 50_000

comptes = np.empty(n_heures, dtype=int)
for h in range(n_heures):
    t, k = 0.0, 0
    while True:
        t += rng.exponential(1 / lam)    # temps d'attente jusqu'à la prochaine commande
        if t > 1.0:                      # on sort de l'heure
            break
        k += 1
    comptes[h] = k

print("moyenne des comptes  :", round(comptes.mean(), 3), "(théorie 3)")
print("variance des comptes :", round(comptes.var(), 3), "(théorie 3)")
for k in range(0, 7):
    print(f"P(N = {k}) : simulée = {(comptes == k).mean():.4f}   Poisson(3) =
    ↳ {stats.poisson(3).pmf(k):.4f}")
```

```
moyenne des comptes  : 3.005 (théorie 3)
variance des comptes : 2.98 (théorie 3)
P(N = 0) : simulée = 0.0492   Poisson(3) = 0.0498
P(N = 1) : simulée = 0.1491   Poisson(3) = 0.1494
P(N = 2) : simulée = 0.2228   Poisson(3) = 0.2240
P(N = 3) : simulée = 0.2241   Poisson(3) = 0.2240
P(N = 4) : simulée = 0.1687   Poisson(3) = 0.1680
P(N = 5) : simulée = 0.1023   Poisson(3) = 0.1008
P(N = 6) : simulée = 0.0511   Poisson(3) = 0.0504
```

Les fréquences simulées (obtenues **uniquement** avec des temps d'attente exponentiels) correspondent à la loi de Poisson : les deux descriptions sont bien le même objet. C'est pourquoi les files d'attente (guichets, serveurs, centres d'appels) se modélisent presque toujours avec ce processus.

À retenir (processus stochastiques).

- Un processus stochastique est une famille (X_t) de variables aléatoires indexée par le temps.
- **Marche aléatoire** : $E[S_n] = 0$, $\sigma(S_n) = \sqrt{n}$; l'incertitude croît en $\sqrt{\text{temps}}$.
- **Chaîne de Markov** : le futur ne dépend que du présent ; probabilités en n pas = $\mathbf{P}^n$; **loi stationnaire** = vecteur propre de $\mathbf{P}^\top$ pour la valeur propre 1.
- **Processus de Poisson** : nombres de Poisson $\Leftrightarrow$ temps d'attente exponentiels.

2.7 Exercices du chapitre 2

Même mode d'emploi qu'au chapitre 1 : cherchez d'abord, comparez ensuite. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse.

Énoncés

Exercice 1 ★ (union). Pour une newsletter : $P(\text{ouvre}) = 0,35$, $P(\text{clique}) = 0,25$, $P(\text{ouvre et clique}) = 0,10$. Quelle est la probabilité qu'un destinataire fasse **au moins une** des deux actions ? Qu'il ne fasse **aucune** ?

Exercice 2 ★★ (Bayes). Dans l'atelier d'un fournisseur, 2 % des bols sont défectueux. Un contrôle automatique signale 90 % des bols défectueux, mais aussi 4 % des bols corrects. Un bol est signalé : quelle est la probabilité qu'il soit réellement défectueux ? Interprétez.

Exercice 3 ★ (« au moins un »). Chaque colis a 2 % de chances d'être endommagé, indépendamment des autres. (a) Probabilité qu'au moins un colis soit endommagé parmi 50 ? (b) Combien de colis faut-il pour que cette probabilité dépasse 90 % ?

Exercice 4 ★ (binomiale). Sur 15 visiteurs d'une publicité, chacun achète avec la probabilité 0,3. Calculez $P(X = 5)$, l'espérance et l'écart-type de X , et $P(X \geq 8)$.

Exercice 5 ★★ (Poisson et exponentielle). Le service client reçoit en moyenne 4 appels par heure (processus de Poisson). (a) Probabilité de ne recevoir **aucun** appel pendant une demi-heure ? (b) Le temps d'attente entre deux appels est exponentiel : quelle est sa moyenne, et quelle est la probabilité d'attendre plus de 20 minutes ? (c) On a déjà attendu 10 minutes sans appel : quelle est la probabilité d'attendre **encore** 20 minutes ?

Exercice 6 ★★ (normale). Le poids des colis expédiés suit $\mathcal{N}(500 \text{ g}, 40^2)$. (a) Probabilité qu'un colis pèse moins de 450 g ? (b) Entre 460 et 540 g ? (c) Quel poids n'est dépassé que par 1 % des colis ?

Exercice 7 ★★ (espérance et variance). Un jeu de fidélité distribue : 0 DT avec la probabilité 0,80 ; 10 DT avec 0,15 ; 50 DT avec 0,05. Calculez l'espérance, la variance et l'écart-type du gain. Si participer coûte 5 DT, le jeu est-il favorable au client ? Et si le client joue 100 fois, quelle est l'espérance et l'écart-type de son gain total ?

Exercice 8 ★★ (covariance). Cinq clients ont noté le délai de livraison ($x = 1, 2, 3, 4, 5$ jours) et leur satisfaction ($y = 2, 4, 5, 4, 5$ sur 5). Calculez à la main la covariance (divisée par n), la variance de chaque variable et la corrélation. Vérifiez avec NumPy.

Exercice 9 ★★★ (TCL). Le panier d'un client a une moyenne de 45 DT et un écart-type de 30 DT (loi très asymétrique). On observe 100 clients. (a) Quelle est la loi approchée de la moyenne ? (b) Probabilité que la moyenne dépasse 50 DT ? (c) Combien de clients faut-il observer pour que l'erreur-type de la moyenne soit inférieure à 1 DT ? (d) Vérifiez (b) par simulation. Attention

: une loi exponentielle de moyenne 45 a un écart-type de 45, pas 30 ; quelle loi asymétrique de moyenne 45 et d'écart-type 30 peut-on utiliser à la place ?

Exercice 10 ★★★ (Markov). Chaque jour, une machine est **en marche** (M) ou **en panne** (P). Si elle est en marche, elle tombe en panne le lendemain avec probabilité 0,1. Si elle est en panne, elle est réparée le lendemain avec probabilité 0,4. (a) Écrivez la matrice de transition. (b) Si elle est en marche aujourd'hui, quelle est la probabilité qu'elle le soit après-demain ? (c) Quelle est la proportion de temps passée en panne à long terme ?

Corrigés

Corrigé 1. $P(\text{au moins une}) = 0,35 + 0,25 - 0,10 = 0,50$. Aucune : $1 - 0,50 = 0,50$. (On retranche l'intersection pour ne pas la compter deux fois ; le complémentaire de « au moins une » est « aucune ».)

Corrigé 2. Sur 10 000 bols : 200 défectueux, dont 180 signalés ; 9 800 corrects, dont 392 signalés à tort. Total des signalements : 572, dont 180 justifiés : $180/572 \approx 0,315$.

```
p_def, sens, fausse = 0.02, 0.90, 0.04
p_signal = sens * p_def + fausse * (1 - p_def)
print("P(défectueux | signalé) =", round(sens * p_def / p_signal, 4))
```

```
P(défectueux | signalé) = 0.3147
```

Seulement **31,5 %** : même avec un bon contrôle, **deux bols signalés sur trois sont corrects**, car les défauts sont rares (erreur du taux de base). Ils justifient une seconde vérification plutôt qu'un rejet automatique.

Corrigé 3. (a) $1 - 0,98^{50} \approx 0,636$. (b) On veut $1 - 0,98^n > 0,9 \Leftrightarrow 0,98^n < 0,1 \Leftrightarrow n > \frac{\ln 0,1}{\ln 0,98} \approx 113,97$, donc **114 colis**.

```
import numpy as np
from scipy import stats
print("(a)", round(1 - 0.98**50, 4))
print("(b)", np.log(0.1) / np.log(0.98), "-> n =", int(np.ceil(np.log(0.1) / np.log(0.98))))
```

```
(a) 0.6358
(b) 113.97408559184939 -> n = 114
```

Corrigé 4. $P(X = 5) = \binom{15}{5} 0,3^5 0,7^{10}$. $E[X] = np = 4,5$; $\sigma = \sqrt{np(1-p)} = \sqrt{3,15} \approx 1,775$; $P(X \geq 8) = 1 - P(X \leq 7)$.

```
X = stats.binom(15, 0.3)
print("P(X=5) =", round(X.pmf(5), 4))
print("E, sigma=", X.mean(), round(X.std(), 3))
print("P(X>=8) =", round(X.sf(7), 4))
```

```
P(X=5) = 0.2061
E, sigma= 4.5 1.775
P(X>=8) = 0.05
```

Corrigé 5. (a) Sur une demi-heure, $N \sim \text{Poisson}(4 \times 0,5 = 2)$: $P(N = 0) = e^{-2} \approx 0,135$. (b) Le temps d'attente est $\text{Exp}(4/h)$, de moyenne $1/4 \text{ h} = 15 \text{ min}$. $20 \text{ min} = 1/3 \text{ h}$: $P(T > 1/3) = e^{-4/3} \approx 0,264$. (c) Par absence de mémoire, c'est la même probabilité : **0,264**.

```
print("(a)", round(np.exp(-2), 4), round(stats.poisson(2).pmf(0), 4))
T = stats.expon(scale=1 / 4) # unité : heure
print("(b)", round(T.sf(1 / 3), 4))
print("(c)", round(T.sf(10 / 60 + 1 / 3) / T.sf(10 / 60), 4))
```

(a) 0.1353 0.1353
 (b) 0.2636
 (c) 0.2636

Corrigé 6. (a) $z = (450 - 500)/40 = -1,25$, $P = \Phi(-1,25) \approx 0,106$. (b) z de -1 à $+1$: $\approx 0,683$. (c) $z_{0,99} \approx 2,326$, donc $500 + 2,326 \times 40 \approx 593$ g.

```
W = stats.norm(500, 40)
print("(a)", round(W.cdf(450), 4))
print("(b)", round(W.cdf(540) - W.cdf(460), 4))
print("(c)", round(W.ppf(0.99), 1))
```

(a) 0.1056
 (b) 0.6827
 (c) 593.1

Corrigé 7. $E = 0,15 \times 10 + 0,05 \times 50 = 1,5 + 2,5 = 4$ DT. $E[X^2] = 0,15 \times 100 + 0,05 \times 2500 = 15 + 125 = 140$, $\text{Var} = 140 - 16 = 124$, $\sigma \approx 11,1$ DT. À 5 DT la partie, l'espérance du **gain net** est $4 - 5 = -1$ DT : défavorable en moyenne (c'est favorable à la boutique). Sur 100 parties (indépendantes) : espérance $100 \times 4 = 400$ DT, variance $100 \times 124 = 12\,400$, écart-type $\sqrt{12\,400} \approx 111$ DT. Remarquez que l'écart-type relatif diminue : $111/400 = 28\%$ contre $11,1/4 = 278\%$ pour une seule partie.

```
gains = np.array([0, 10, 50]); probas = np.array([0.8, 0.15, 0.05])
E = (gains * probas).sum(); V = (gains**2 * probas).sum() - E**2
print("E =", E, " Var =", round(V, 2), " sigma =", round(np.sqrt(V), 2))
print("100 parties : E =", 100 * E, " sigma =", round(np.sqrt(100 * V), 1))
```

```
E = 4.0  Var = 124.0  sigma = 11.14
100 parties : E = 400.0  sigma = 111.4
```

Corrigé 8. Moyennes $\bar{x} = 3$, $\bar{y} = 4$. Écarts : $x - \bar{x} = (-2, -1, 0, 1, 2)$, $y - \bar{y} = (-2, 0, 1, 0, 1)$. Produits : 4, 0, 0, 0, 2, de somme 6, donc $\text{Cov} = 6/5 = 1,2$. $\text{Var}(x) = (4 + 1 + 0 + 1 + 4)/5 = 2$; $\text{Var}(y) = (4 + 0 + 1 + 0 + 1)/5 = 1,2$. $\rho = \frac{1,2}{\sqrt{2 \times 1,2}} = \frac{1,2}{1,549} \approx 0,775$. Lecture : la corrélation est *positive* : plus la livraison est lente, plus la satisfaction serait élevée ? Ce n'est pas plausible : avec seulement cinq points, c'est probablement un hasard de l'échantillon. Nous verrons au chapitre 3 comment tester si une corrélation observée est significative.

```
x = np.array([1, 2, 3, 4, 5.0]); y = np.array([2, 4, 5, 4, 5.0])
print("cov =", ((x - x.mean()) * (y - y.mean())).mean(), " var x, y =", x.var(), y.var())
print("rho =", round(np.corrcoef(x, y)[0, 1], 4))
```

```
cov = 1.2  var x, y = 2.0 1.2
rho = 0.7746
```

Corrigé 9. (a) Par le TCL, $\bar{X}_{100} \approx \mathcal{N}(45, 30^2/100) = \mathcal{N}(45, 3^2)$: erreur-type 3 DT. (b) $z = (50 - 45)/3 \approx 1,667$, $P(\bar{X} > 50) \approx 0,048$. (c) $\sigma/\sqrt{n} < 1 \iff n > 900$. (d) Pour la simulation, une exponentielle de moyenne 45 a un écart-type de 45 (pas 30) : elle ne convient pas. On utilise une loi Gamma de moyenne 45 et d'écart-type 30 (forme $k = (45/30)^2 = 2,25$, échelle $\theta = 30^2/45 = 20$). La simulation donne 5,1 % contre 4,8 % par le TCL : l'écart vient de l'asymétrie résiduelle à $n = 100$.

```
print("(b) TCL :", round(stats.norm(45, 3).sf(50), 4))
rng = np.random.default_rng(9)
echantillons = rng.gamma(shape=2.25, scale=20, size=(100_000, 100))
print("moyenne, écart-type du panier simulé :", echantillons.mean().round(2),
      "\u2192 echantillons.std().round(2))")
print("(b) simulée :", round((echantillons.mean(axis=1) > 50).mean(), 4))
print("(c) n minimal :", (30 / 1) ** 2)
```

```
(b) TCL : 0.0478
moyenne, écart-type du panier simulé : 45.01 30.0
(b) simulée : 0.0512
(c) n minimal : 900.0
```

Corrigé 10. (a) $\mathbf{P} = \begin{pmatrix} 0,9 & 0,1 \\ 0,4 & 0,6 \end{pmatrix}$ (lignes M, P). (b) $\mathbf{P}_{MM}^2 = 0,9 \times 0,9 + 0,1 \times 0,4 = 0,85$. (c) On cherche $\pi = (\pi_M, \pi_P)$ avec $\pi \mathbf{P} = \pi$: de la seconde colonne, $0,1\pi_M + 0,6\pi_P = \pi_P$, soit $0,1\pi_M = 0,4\pi_P$, donc $\pi_M = 4\pi_P$; avec $\pi_M + \pi_P = 1$: $\pi_P = 0,2$. La machine est en panne **20 % du temps** à long terme.

```
P = np.array([[0.9, 0.1], [0.4, 0.6]])
print("P^2[M,M] =", np.linalg.matrix_power(P, 2)[0, 0])
w, V = np.linalg.eig(P.T)
pi = np.real(V[:, np.argmin(np.abs(w - 1))]); pi /= pi.sum()
print("loi stationnaire :", pi.round(3))
```

```
P^2[M,M] = 0.8500000000000001
loi stationnaire : [0.8 0.2]
```

Bilan du chapitre 2

Vous savez maintenant :

- **calculer** des probabilités (complémentaire, union, conditionnel) et **inverser** un conditionnement avec Bayes, en vous méfiant de l'erreur du taux de base ;
- **modéliser** un phénomène par la bonne loi (Bernoulli, binomiale, Poisson, exponentielle, normale) et calculer des probabilités avec `scipy.stats` ;
- **résumer** une loi par son espérance et sa variance, et mesurer le lien entre deux variables par la covariance et la corrélation ;
- **comprendre** pourquoi une moyenne devient fiable (LGN) et pourquoi son erreur est normale (TCL), avec une erreur-type en $\sigma/\sqrt{n}$;
- (en option) **situer** tout cela dans le cadre de la théorie de la mesure et des processus stochastiques.

Le chapitre 3 retourne le problème : on ne **connaît** plus la loi, on a seulement des **données**, et il faut en déduire la loi. C'est la statistique.

Chapitre 3 : Statistique

« Les probabilités vont de la **cause** vers les **données**. La statistique fait le chemin inverse : des **données** vers la cause. »

Au chapitre 2, nous *connaissions* la loi (par exemple « le taux de conversion est 20,5 % ») et nous calculions la probabilité d'observer certaines données. Dans la vie réelle, c'est l'inverse : on **observe** des données (400 commandes) et on veut deviner la loi (« quel est le vrai panier moyen ? Le canal Instagram est-il vraiment moins rentable que la boutique ? »). C'est le travail de la **statistique**.

Le chemin de ce chapitre

- **3.1 Statistique descriptive** : résumer et regarder les données (moyenne, médiane, quantiles, graphiques) avant toute chose.
- **3.2 Estimation** : déduire un paramètre inconnu d'un échantillon (méthode des moments, maximum de vraisemblance), et juger la qualité d'un estimateur.
- **3.3 Intervalles de confiance** : ne pas donner un seul chiffre, mais une **fourchette** honnête.
- **3.4 Tests d'hypothèses** : décider, avec un risque maîtrisé, si un effet observé est réel ou dû au hasard (tests de moyenne, de proportion, du khi-deux, A/B).
- **3.5 p-valeurs, puissance et tests multiples** : bien interpréter les résultats, dimensionner une expérience, éviter les faux positifs.
- **⊕ Pour aller plus loin** : les sondages (comment échantillonner), et les méthodes non paramétriques (quand on ne veut pas supposer de loi).
- **3.8 Exercices corrigés**.

Le fil conducteur : un jeu de 400 commandes. Tout au long du chapitre nous travaillons sur un même tableau de 400 commandes de Dar Jasmin (canal, montant, délai de livraison, satisfaction). Il est **simulé** (graine fixe) pour que vous puissiez reproduire chaque calcul, et vous verrez qu'on y retrouve des phénomènes tout à fait réalistes : montants asymétriques, différences entre canaux, lien entre délai et satisfaction.

Outils. Nous utilisons pandas pour les tableaux (étudié en détail à la section 4.4 : ici on n'utilise que les gestes de base, expliqués au passage) et scipy.stats pour les calculs statistiques.

3.1 Statistique descriptive

Intuition. Avant de modéliser, de tester ou de prédire quoi que ce soit, on **regarde** les données. La statistique descriptive, c'est l'art de résumer un tableau de centaines de lignes en quelques nombres et quelques graphiques *fidèles*. C'est aussi la meilleure façon de repérer des erreurs de saisie, des valeurs aberrantes et des surprises, **avant** qu'elles ne faussent une analyse.

3.1.1 Population, échantillon, variables

Deux mots que nous utiliserons constamment :

- La **population** est l'ensemble complet qui nous intéresse (*toutes* les commandes passées et à venir de Dar Jasmin).
- L'**échantillon** est la partie que l'on a effectivement observée (nos 400 commandes).

On calcule des **statistiques** sur l'échantillon pour apprendre des choses sur les **paramètres** de la population, qui eux restent inconnus. (On notera $\bar{x}$ la moyenne de l'échantillon et μ celle de la population.)

Chaque colonne d'un tableau est une **variable**. Son type décide des calculs et des graphiques qui ont un sens :

Type	Exemple	Résumés adaptés
Quantitative continue	montant (DT)	moyenne, médiane, écart-type, histogramme
Quantitative discrète	nombre d'articles, délai en jours	idem, ou fréquences de chaque valeur
Qualitative nominale	canal (Instagram / Site / Boutique)	effectifs, proportions, diagramme en barres
Qualitative ordinale	satisfaction (1 à 5)	effectifs, médiane, quantiles (la moyenne est discutable)

La moyenne d'une variable ordinale (satisfaction de 1 à 5) est très répandue mais n'a pas de sens strict : l'écart entre 1 et 2 est-il le même qu'entre 4 et 5 ? On la calcule quand même par convention, mais en gardant ceci en tête.

3.1.2 Charger et regarder le jeu de données

Voici la construction du jeu de données utilisé dans tout le chapitre. Vous n'avez pas besoin de comprendre chaque ligne maintenant : l'important est le **résultat**, un tableau de 400 commandes.

```
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(100)
n = 400
canal = rng.choice(["Instagram", "Site", "Boutique"], size=n, p=[0.4, 0.35, 0.25])
base = {"Instagram": 3.7, "Site": 3.9, "Boutique": 4.1}
montant = np.round(np.exp(rng.normal([base[c] for c in canal], 0.55)), 1)
livraison = np.where(canal == "Boutique", 0, np.round(rng.gamma(4, 0.9, size=n)) + 1).astype(int)
satisfaction = np.clip(np.round(4.6 - 0.18 * livraison + rng.normal(0, 0.7, size=n)), 1,
↳ 5).astype(int)
```

```
df = pd.DataFrame({"canal": canal, "montant": montant,
                  "livraison": livraison, "satisfaction": satisfaction})
print(df.head(8))
print()
print("dimensions :", df.shape)
```

	canal	montant	livraison	satisfaction
0	Boutique	44.8	0	4
1	Site	34.5	2	4
2	Instagram	88.2	5	4
3	Instagram	30.1	4	4
4	Boutique	110.1	0	5
5	Site	39.8	5	3
6	Boutique	74.7	0	5
7	Boutique	108.7	0	4

```
dimensions : (400, 4)
```

Chaque ligne est une commande. livraison est le délai en jours (0 pour un retrait en boutique) et satisfaction une note de 1 à 5. Les trois premiers gestes à faire sur n'importe quel tableau :

```
print(df.dtypes)           # le type de chaque colonne
print()
print(df.isna().sum())     # valeurs manquantes par colonne
print()
print(df.describe().round(2)) # résumé numérique
```

```
canal      str
montant    float64
livraison  int64
satisfaction int64
dtype: object
```

```
canal      0
montant    0
livraison  0
satisfaction 0
dtype: int64
```

	montant	livraison	satisfaction
count	400.00	400.00	400.00
mean	60.25	3.29	3.96
std	38.02	2.56	0.80
min	8.60	0.00	1.00
25%	34.18	0.00	3.00
50%	51.00	3.00	4.00
75%	75.82	5.00	5.00
max	255.70	13.00	5.00

describe() donne d'un coup : effectif (count), moyenne (mean), écart-type (std), minimum, quartiles (25 %, 50 %, 75 %) et maximum. Aucune valeur manquante. Le montant moyen est d'environ 60 DT, mais le **maximum dépasse 250 DT** alors que la **médiane** (50 %) est d'environ 51 DT : la distribution est probablement **asymétrique**. Regardons cela de plus près.

3.1.3 Mesures de position : où est le centre ?

La **moyenne** $\bar{x} = \frac{1}{n} \sum x_i$ est le centre de gravité. La **médiane** est la valeur qui partage l'échantillon en deux moitiés égales : 50 % des commandes sont en dessous. Le **mode** est la valeur la plus fréquente.

Voici un petit exemple à la main pour sentir la différence. Cinq commandes : 20, 25, 30, 35, 400 DT (la dernière est un gros achat professionnel). La moyenne vaut $(20+25+30+35+400)/5 = 102$ DT : aucune commande n'est proche de ce chiffre ! La médiane (la valeur du milieu après tri) vaut 30 DT, bien plus représentative d'une commande « typique ».

Règle de base. La moyenne est sensible aux valeurs extrêmes ; la médiane est **robuste**. Quand la distribution est asymétrique, la moyenne est tirée vers la queue longue : **moyenne > médiane** pour une asymétrie à droite (cas des montants, revenus, durées).

```
m = df["montant"]
print("moyenne :", round(m.mean(), 2))
print("médiane :", round(m.median(), 2))
print("moyenne tronquée (10 % de chaque côté) :", round(stats.trim_mean(m, 0.10), 2))
print("mode approximatif (classes de 10 DT) :", int(m.round(-1).mode()[0]), "DT")
```

```
moyenne : 60.25
médiane : 51.0
moyenne tronquée (10 % de chaque côté) : 54.9
mode approximatif (classes de 10 DT) : 40 DT
```

La moyenne (60 DT) dépasse nettement la médiane (51 DT) : quelques grosses commandes tirent la moyenne vers le haut. La **moyenne tronquée**, qui écarte les 10 % de valeurs les plus extrêmes de chaque côté, se situe entre les deux.

Les quantiles généralisent la médiane : le quantile à q % est la valeur sous laquelle se trouvent q % des observations. Les **quartiles** (25 %, 50 %, 75 %) découpent l'échantillon en quatre parts égales.

```
print(m.quantile([0.05, 0.25, 0.50, 0.75, 0.95]).round(1))
```

```
0.05    19.0
0.25    34.2
0.50    51.0
0.75    75.8
0.95   128.6
Name: montant, dtype: float64
```

On lit par exemple : « 95 % des commandes font moins de ~130 DT », une information très utile pour dimensionner un seuil de livraison gratuite.

3.1.4 Mesures de dispersion : de combien ça varie ?

Deux boutiques dont le panier moyen est de 60 DT peuvent être très différentes si l'une a des paniers tous compris entre 55 et 65 et l'autre entre 5 et 300.

- **L'étendue** : $\max - \min$. Simple, mais entièrement dictée par deux valeurs extrêmes.
- **La variance** $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$ et l'**écart-type** $s = \sqrt{s^2}$.
- **L'écart interquartile** (IQR) : $Q_3 - Q_1$, l'étalement des 50 % du milieu. Robuste.
- **Le coefficient de variation** $s/\bar{x}$: l'écart-type en proportion de la moyenne, sans unité, utile pour comparer des échelles différentes.

Pourquoi $n-1$ et pas n ? Pandas et NumPy ne donnent pas toujours la même chose : pandas divise par $n-1$ par défaut, np.var par n . Nous démontrons au 3.2 que $n-1$ est la bonne version pour **estimer** la variance de la population à partir d'un échantillon. Pour $n = 400$ la différence est minime, mais pour $n = 5$ elle est de 25 %.

```
print("écart-type (pandas, n-1) :", round(m.std(), 2))
print("écart-type (numpy, n)   :", round(np.std(m), 2))
print("étendue                 :", round(m.max() - m.min(), 1))
q1, q3 = m.quantile(0.25), m.quantile(0.75)
print("IQR                     :", round(q3 - q1, 1))
print("coefficient de variation :", round(m.std() / m.mean(), 2))
```

```
écart-type (pandas, n-1) : 38.02
écart-type (numpy, n)   : 37.97
étendue                 : 247.1
IQR                     : 41.6
coefficient de variation : 0.63
```

Un écart-type de 38 DT pour une moyenne de 60 DT : un coefficient de variation de 63 %, ce qui est **très dispersé** (typique des montants).

3.1.5 La forme : histogrammes, asymétrie, boîtes à moustaches

Les nombres ne disent pas tout. L'**histogramme** découpe l'axe en classes et compte les observations dans chacune. La **boîte à moustaches** (*boxplot*) résume la distribution en cinq nombres : la boîte va du premier au troisième quartile, le trait central est la médiane, et les « moustaches » s'étendent jusqu'aux dernières valeurs situées à moins de $1,5 \times \text{IQR}$ de la boîte ; les points au-delà sont signalés comme **valeurs atypiques** (*outliers*).

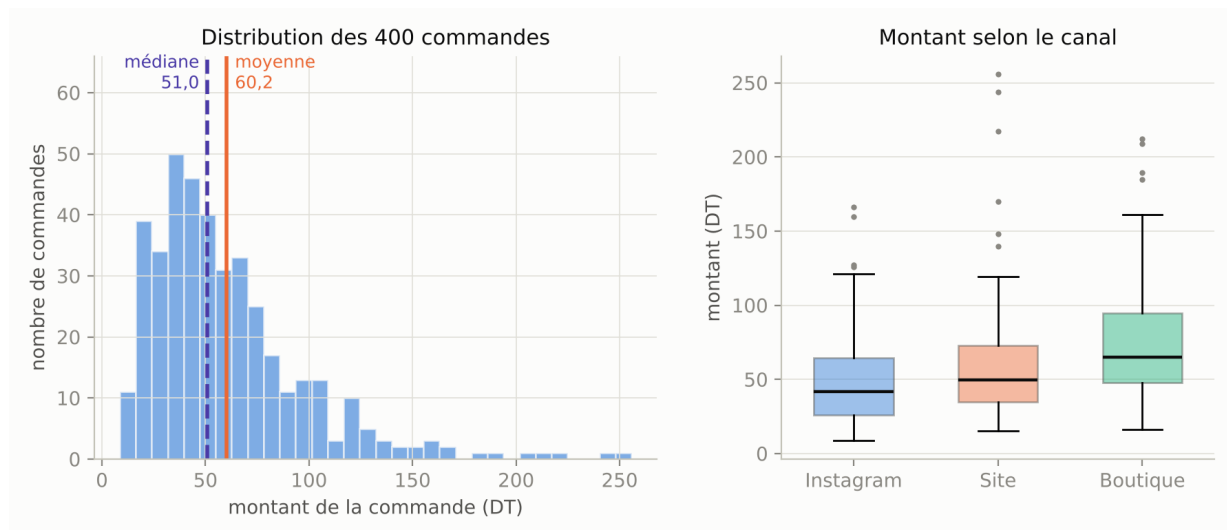


Figure 16: À gauche : histogramme des 400 montants, avec la moyenne (orange) tirée vers la droite de la médiane (violet). À droite : boîtes à moustaches du montant selon le canal.

On lit sur l'histogramme une **asymétrie à droite** : beaucoup de petites commandes, quelques très grosses. Sur le boxplot, la boutique a des commandes plus élevées que le site, lui-même plus élevé qu'Instagram. (Est-ce une vraie différence ou du hasard d'échantillonnage ? C'est la question du 3.4.)

L'**asymétrie** (*skewness*) se mesure par un nombre : nulle pour une courbe symétrique, positive à droite.

```
print("asymétrie :", round(stats.skew(m), 2))
q1, q3 = m.quantile([0.25, 0.75])
borne_haute = q3 + 1.5 * (q3 - q1)
print("seuil haut des valeurs atypiques :", round(borne_haute, 1), "DT")
print("nombre de commandes au-dessus   :", int((m > borne_haute).sum()))
```



```

asymétrie : 1.75
seuil haut des valeurs atypiques : 138.3 DT
nombre de commandes au-dessus : 17

```

Que faire d'une valeur atypique ? Surtout ne pas la supprimer automatiquement. Se demander : est-ce une *erreur* (saisie : 2 500 au lieu de 25,00) ? Alors on corrige. Est-ce une valeur *légitime* mais rare (gros client professionnel) ? Alors on la garde et on utilise des résumés robustes. Les valeurs atypiques sont souvent l'information la plus intéressante : un fraudeur, une panne, une opportunité.

Transformer pour symétriser. Pour des variables positives très asymétriques, le **logarithme** rapproche la forme d'une cloche :

```

lm = np.log(m)
print("asymétrie du montant      :", round(stats.skew(m), 2))
print("asymétrie du logarithme   :", round(stats.skew(lm), 2))
print("moyenne et médiane de log(montant) :", round(lm.mean(), 2), round(lm.median(), 2))

```

```

asymétrie du montant      : 1.75
asymétrie du logarithme   : -0.07
moyenne et médiane de log(montant) : 3.92 3.93

```

Après transformation, l'asymétrie est presque nulle et moyenne $\approx$ médiane : le montant suit approximativement une loi **log-normale** (le logarithme est normal). Beaucoup de grandeurs économiques sont dans ce cas, car elles résultent de **multiplications** d'effets (là où la loi normale vient d'additions, TCL du 2.4).

3.1.6 Variables qualitatives et comparaison de groupes

Pour une variable catégorielle, on compte (**effectifs**) et on calcule des **proportions** (fréquences). Pour comparer des groupes, on utilise groupby.

```

print(df["canal"].value_counts())
print()
print(df["canal"].value_counts(normalize=True).round(3))
print()
print(df.groupby("canal")["montant"].agg(["count", "mean", "median", "std"]).round(1))

```

```

canal
Site      148
Instagram 138
Boutique  114
Name: count, dtype: int64

```

```

canal
Site      0.370
Instagram 0.345
Boutique  0.285
Name: proportion, dtype: float64

```

	count	mean	median	std
canal				
Boutique	114	74.8	64.8	40.6
Instagram	138	49.0	41.5	31.1
Site	148	59.5	49.5	38.3

Les trois canaux apportent respectivement 138, 148 et 114 commandes (Instagram, Site, Boutique). Les paniers moyens sont d'environ 49, 60 et 75 DT : la boutique domine en valeur par commande. Mais attention :

Une différence observée n'est pas forcément une différence réelle. Avec 114 à 148 commandes par canal, le hasard seul peut créer des écarts de quelques dinars. Pour savoir si l'écart entre 49 et 75 DT est « assez grand » pour être crédible, il faut un **test** (3.4) ou un **intervalle de confiance** (3.3). La statistique descriptive décrit ; elle ne *conclut* pas.

3.1.7 Relier deux variables : corrélation, et pourquoi il faut toujours dessiner

La **corrélation de Pearson** (2.3.3), calculée sur l'échantillon, mesure le lien *linéaire* entre deux variables quantitatives.

```
print(df[["montant", "livraison", "satisfaction"]].corr().round(2))
```

	montant	livraison	satisfaction
montant	1.00	-0.20	0.07
livraison	-0.20	1.00	-0.53
satisfaction	0.07	-0.53	1.00

La corrélation entre le délai de livraison et la satisfaction est d'environ $-0,53$: plus la livraison est lente, moins les clients sont satisfaits ; c'est cohérent avec le bon sens. En revanche, le montant n'est que faiblement lié à la satisfaction ($+0,07$).

La **corrélation de Spearman** est la corrélation de Pearson calculée sur les **rangs** (1er, 2e, 3e...). Elle détecte toute relation **monotone** (pas seulement linéaire) et résiste aux valeurs extrêmes.

```
rho_s, _ = stats.spearmanr(df["livraison"], df["satisfaction"])
print("Spearman (livraison, satisfaction) :", round(rho_s, 2))
```

```
Spearman (livraison, satisfaction) : -0.51
```

Le quartet d'Anscombe : pourquoi on dessine toujours. En 1973, le statisticien Francis Anscombe a construit **quatre jeux de données** qui ont exactement les mêmes moyennes, variances, corrélation et droite de régression... mais des formes totalement différentes.

```
x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
ys = [[8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68],
      [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74],
      [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73],
      [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89]]
xs = [x, x, x, [8] * 7 + [19] + [8] * 3]

for nom, a, b in zip(["I", "II", "III", "IV"], xs, ys):
    pente, ordonnee = np.polyfit(a, b, 1)
    print(f"jeu {nom:<3}: moy x = {np.mean(a):.2f}  moy y = {np.mean(b):.2f}  "
          f"corr = {np.corrcoef(a, b)[0, 1]:.3f}  droite : y = {pente:.2f} x + {ordonnee:.2f}")

jeu I   : moy x = 9.00  moy y = 7.50  corr = 0.816  droite : y = 0.50 x + 3.00
jeu II  : moy x = 9.00  moy y = 7.50  corr = 0.816  droite : y = 0.50 x + 3.00
jeu III : moy x = 9.00  moy y = 7.50  corr = 0.816  droite : y = 0.50 x + 3.00
jeu IV  : moy x = 9.00  moy y = 7.50  corr = 0.817  droite : y = 0.50 x + 3.00
```

Les quatre lignes sont quasiment identiques. Voici pourtant les quatre jeux :

Règle d'or : on dessine d'abord, on calcule ensuite. Un résumé numérique est une compression de l'information, avec perte. Seul un graphique révèle ce qui a été perdu.

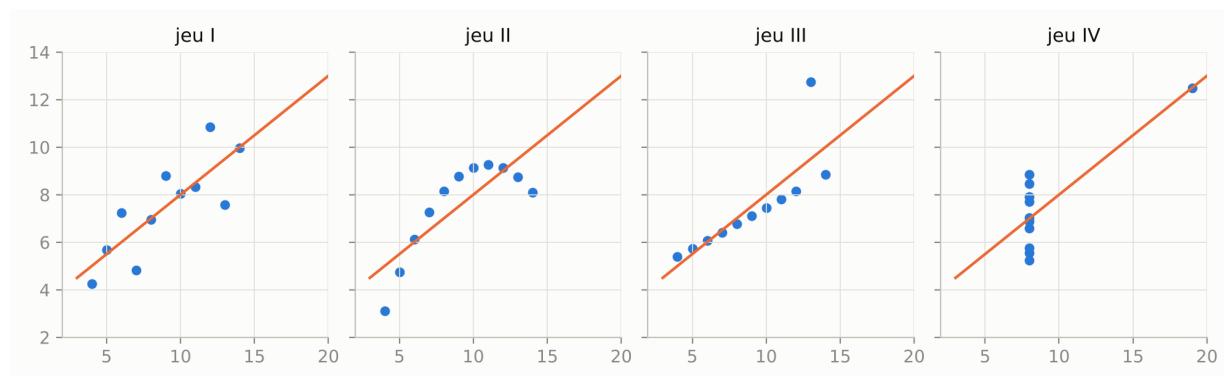


Figure 17: Le quartet d'Anscombe : mêmes statistiques, formes radicalement différentes. I : relation linéaire ordinaire. II : relation courbe. III : une valeur atypique fausse la droite. IV : un seul point (à droite) crée toute la corrélation.

À retenir (statistique descriptive).

- Population (inconnue) vs échantillon (observé) ; paramètres vs statistiques.
- Position : moyenne (sensible aux extrêmes), médiane (robuste), quantiles. Dispersion : écart-type ($n - 1$), IQR, coefficient de variation.
- Montants, revenus, durées : souvent asymétriques à droite ; le logarithme symétrise. Ne supprimez pas les valeurs atypiques sans réfléchir.
- Histogramme et boxplot pour la forme ; groupby pour comparer des groupes ; Pearson (linéaire) et Spearman (monotone) pour deux variables.
- **Toujours tracer** (Anscombe). Une différence observée n'est pas encore une différence prouvée.

3.2 Estimation

Intuition. La population a des paramètres (un panier moyen μ , un taux de conversion p , un taux d'arrivée λ) que personne ne connaît. À partir d'un **échantillon**, on fabrique une **estimation**. La formule qui transforme l'échantillon en estimation s'appelle un **estimateur**. Il y a toujours plusieurs estimateurs possibles ; la question est : **lequel choisir, et à quel point peut-on lui faire confiance ?**

3.2.1 Un estimateur est une variable aléatoire

Notons θ un paramètre inconnu (n'importe lequel) et $\hat{\theta}$ (« thêta chapeau ») son estimateur. Comme l'échantillon est aléatoire, $\hat{\theta}$ **l'est aussi** : un autre échantillon donnerait une autre estimation. C'est exactement l'idée de la moyenne d'échantillon $\bar{X}_n$ au 2.4.1.

On peut le **voir** en jouant à Dieu : traitons nos 400 commandes comme *toute* la population (de moyenne connue) et tirons dedans des échantillons de 30 commandes, en recalculant la moyenne à chaque fois.

```
import numpy as np
import pandas as pd
from scipy import stats
from scipy import optimize

rng = np.random.default_rng(11)
population = df["montant"].to_numpy()
print("moyenne de la 'population' :", round(population.mean(), 2))
```

```
moyennes = np.array([rng.choice(population, size=30, replace=False).mean() for _ in range(10_000)])
print("moyenne des 10 000 moyennes d'échantillons :", round(moyennes.mean(), 2))
print("écart-type de ces moyennes (erreur-type)      :", round(moyennes.std(), 2))
print("théorie sigma/sqrt(n)                          :", round(population.std() / np.sqrt(30), 2))
print("3 échantillons au hasard :", [round(float(rng.choice(population, 30).mean()), 1) for _ in
    range(3)])
```

```
moyenne de la 'population' : 60.25
moyenne des 10 000 moyennes d'échantillons : 60.3
écart-type de ces moyennes (erreur-type)      : 6.59
théorie sigma/sqrt(n)                          : 6.93
3 échantillons au hasard : [54.2, 59.5, 60.3]
```

Chaque échantillon de 30 commandes donne une moyenne différente (ci-dessus : 54,2 ; 59,5 ; 60,3), mais **en moyenne** ces moyennes tombent sur la vraie valeur (60,25), avec une dispersion de l'ordre de 7 DT. Cette dispersion est l'**erreur-type** : elle mesure la précision de l'estimateur. (Le petit écart avec la théorie vient du tirage **sans remise** dans une population finie.)

3.2.2 Qu'est-ce qu'un bon estimateur ?

On juge un estimateur sur trois critères, que l'on comprend très bien avec l'image d'un tir à la cible :

- **Le biais** : $\text{Biais}(\hat{\theta}) = E[\hat{\theta}] - \theta$. Les tirs sont-ils centrés sur la cible, ou systématiquement décalés ? Un estimateur est **sans biais** si ce biais vaut 0.
- **La variance** : $\text{Var}(\hat{\theta})$. Les tirs sont-ils groupés ou dispersés ?
- **La cohérence** (ou *consistance*) : l'estimateur converge-t-il vers θ quand $n \rightarrow \infty$? (La loi des grands nombres l'assure pour la moyenne.)

Un bon estimateur est à la fois **peu biaisé** et **peu variable**. On les combine dans l'**erreur quadratique moyenne** :

Décomposition biais-variance.

$$\text{EQM}(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + \text{Biais}(\hat{\theta})^2.$$

Preuve. Notons $m = E[\hat{\theta}]$. On écrit $\hat{\theta} - \theta = (\hat{\theta} - m) + (m - \theta)$ et on développe le carré :

$$E[(\hat{\theta} - \theta)^2] = E[(\hat{\theta} - m)^2] + 2(m - \theta) E[\hat{\theta} - m] + (m - \theta)^2.$$

Le terme du milieu est nul car $E[\hat{\theta} - m] = 0$. Il reste $\text{Var}(\hat{\theta}) + (m - \theta)^2$. ■

Cette formule a une conséquence profonde, qui reviendra tout au long du volume II : **accepter un petit biais peut réduire l'erreur totale si cela réduit beaucoup la variance**. C'est le principe de la régularisation en apprentissage automatique.

3.2.3 Pourquoi divise-t-on par $n - 1$ pour la variance ?

Nous avons promis cette démonstration (3.1.4 et 2.3.3). Soit $X_1, \dots, X_n$ i.i.d. de moyenne μ et de variance σ^2 . L'estimateur « naturel » de σ^2 est $\hat{\sigma}_n^2 = \frac{1}{n} \sum (X_i - \bar{X})^2$. Est-il sans biais ?

Calcul. On insère $\mu : X_i - \bar{X} = (X_i - \mu) - (\bar{X} - \mu)$. Alors

$$\sum_i (X_i - \bar{X})^2 = \sum_i (X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

(En développant : $\sum (X_i - \mu)^2 - 2(\bar{X} - \mu) \sum (X_i - \mu) + n(\bar{X} - \mu)^2$ et $\sum (X_i - \mu) = n(\bar{X} - \mu)$, d'où le résultat.) Prenons l'espérance : $E[(X_i - \mu)^2] = \sigma^2$ et $E[(\bar{X} - \mu)^2] = \text{Var}(\bar{X}) = \sigma^2/n$. Donc

$$E\left[\sum_i (X_i - \bar{X})^2\right] = n\sigma^2 - n \cdot \frac{\sigma^2}{n} = (n-1)\sigma^2.$$

Par conséquent $E[\hat{\sigma}_n^2] = \frac{n-1}{n}\sigma^2 < \sigma^2$: l'estimateur « divisé par n » **sous-estime** la variance. Pour corriger, on divise par $n-1$:

$$S^2 = \frac{1}{n-1} \sum_i (X_i - \bar{X})^2, \quad E[S^2] = \sigma^2. \blacksquare$$

Pourquoi intuitivement ? Les données sont toujours plus proches de **leur propre** moyenne $\bar{X}$ que de la vraie moyenne μ (car $\bar{X}$ est justement construite pour être au centre des données). Mesurer les écarts à $\bar{X}$ sous-estime donc légèrement les écarts à μ . L'échantillon a aussi « perdu un degré de liberté » : une fois $\bar{X}$ calculée, seules $n-1$ valeurs sont libres, la dernière est imposée.

Voyons-le sur ordinateur pour de **tout petits** échantillons ($n = 5$), où l'effet est maximal (vraie variance = 1) :

```
rng = np.random.default_rng(3)
x = rng.normal(0, 1, size=(100_000, 5))          # 100 000 échantillons de taille 5
v_n = x.var(axis=1, ddof=0).mean()                # divisé par n
v_n1 = x.var(axis=1, ddof=1).mean()               # divisé par n-1
print("moyenne des estimations, division par n :", round(v_n, 3), " (théorie (n-1)/n = 0.8)")
print("moyenne des estimations, division par n-1 :", round(v_n1, 3))
```

```
moyenne des estimations, division par n : 0.801 (théorie (n-1)/n = 0.8)
moyenne des estimations, division par n-1 : 1.001
```

Subtilité. S^2 est sans biais pour la **variance**, mais $S = \sqrt{S^2}$ reste (très légèrement) biaisé pour l'écart-type, car la racine carrée n'est pas linéaire. Ce biais est négligeable en pratique.

3.2.4 La méthode des moments

Idée. Les paramètres d'une loi s'expriment à l'aide de ses moments (espérance, variance...). La **méthode des moments** consiste à **égaler les moments théoriques aux moments observés** et à résoudre. C'est simple et intuitif.

Exemple 1 : le taux d'arrivée. Les temps (en minutes) séparant 25 commandes successives suivent une loi exponentielle de paramètre λ inconnu. On sait que $E[T] = 1/\lambda$. On égale à la moyenne observée $\bar{t}$: $1/\hat{\lambda} = \bar{t}$, donc $\hat{\lambda} = 1/\bar{t}$.

```
rng = np.random.default_rng(7)
attentes = rng.exponential(scale=2.0, size=25)    # vraie valeur : lambda = 0.5 par minute
↳ (moyenne 2 min)
print("moyenne observée :", round(attentes.mean(), 3), "minutes")
print("lambda estimé (1/moyenne) :", round(1 / attentes.mean(), 3), "(vraie valeur : 0.5)")
```

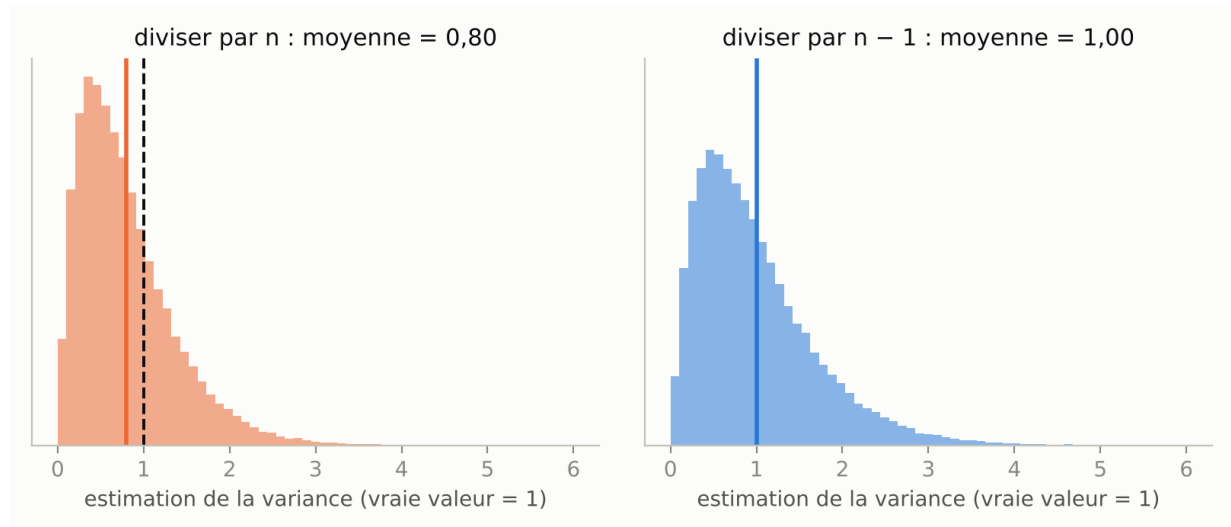


Figure 18: Distribution des estimations de la variance sur 100 000 échantillons de taille 5 : en divisant par n , on sous-estime en moyenne (0,80) ; en divisant par $n-1$, on est centré sur la vraie valeur (1).

```
moyenne observée : 1.988 minutes
lambda estimé (1/moyenne) : 0.503 (vraie valeur : 0.5)
```

Exemple 2 : une loi à deux paramètres. Les montants sont modélisés par une loi **Gamma** de forme k et d'échelle θ , avec $E[X] = k\theta$ et $\text{Var}(X) = k\theta^2$. En égalant à la moyenne $\bar{x}$ et à la variance s^2 observées, on obtient deux équations : $\hat{\theta} = s^2/\bar{x}$ et $\hat{k} = \bar{x}^2/s^2$.

```
xbar, s2 = m.mean(), m.var()
theta_mom = s2 / xbar
k_mom = xbar**2 / s2
print("Gamma par les moments : k =", round(k_mom, 3), " theta =", round(theta_mom, 2))
```

```
Gamma par les moments : k = 2.511 theta = 23.99
```

La méthode des moments est rapide, mais elle n'est pas toujours la plus précise. La suivante est la référence.

3.2.5 Le maximum de vraisemblance

Intuition. Yasmine lance une nouvelle promotion et observe 7 achats sur 20 visiteurs. Quelle valeur du taux de conversion p rend ces données **les plus plausibles** ? Si p valait 0,05, observer 7 acheteurs sur 20 serait très improbable ; si p valait 0,9, tout autant. Il existe une valeur intermédiaire pour laquelle ce résultat est **le moins surprenant possible** : c'est l'estimation du maximum de vraisemblance.

La vraisemblance $L(\theta)$ est la probabilité (ou la densité) d'observer **les données effectivement observées**, vue comme une fonction du paramètre θ . Pour des observations indépendantes $x_1, \dots, x_n$:

$$L(\theta) = \prod_{i=1}^n f(x_i; \theta).$$

L'estimateur du maximum de vraisemblance (EMV, MLE) est la valeur $\hat{\theta}$ qui **maximise** $L(\theta)$.

Comme les produits sont pénibles à dériver et numériquement instables (le produit de nombreuses probabilités minuscules s'écrase vers 0), on maximise plutôt le **logarithme** de la vraisemblance, qui transforme le produit en somme et a le même maximum (le logarithme est croissant) :

$$\ell(\theta) = \ln L(\theta) = \sum_{i=1}^n \ln f(x_i; \theta).$$

C'est de l'optimisation (section 1.3) : on dérive et on annule.

Exemple complet : le taux de conversion. On observe k achats sur n visiteurs. Le modèle est binomial : $L(p) = \binom{n}{k} p^k (1-p)^{n-k}$, donc

$$\ell(p) = \ln \binom{n}{k} + k \ln p + (n-k) \ln(1-p).$$

On dérive : $\ell'(p) = \frac{k}{p} - \frac{n-k}{1-p}$. En annulant : $k(1-p) = (n-k)p \iff k = np$, d'où

$$\hat{p} = \frac{k}{n}.$$

La dérivée seconde $\ell''(p) = -\frac{k}{p^2} - \frac{n-k}{(1-p)^2} < 0$: c'est bien un **maximum**. ■

L'estimateur du maximum de vraisemblance est donc tout simplement la **proportion observée**, ce qui rassure : la méthode retrouve le bon sens.

Pour $k = 7$, $n = 20$: $\hat{p} = 0,35$. La figure montre la vraisemblance et la log-vraisemblance en fonction de p ; le maximum est atteint en 0,35, et la log-vraisemblance est une courbe en cloche inversée, bien plus facile à optimiser.

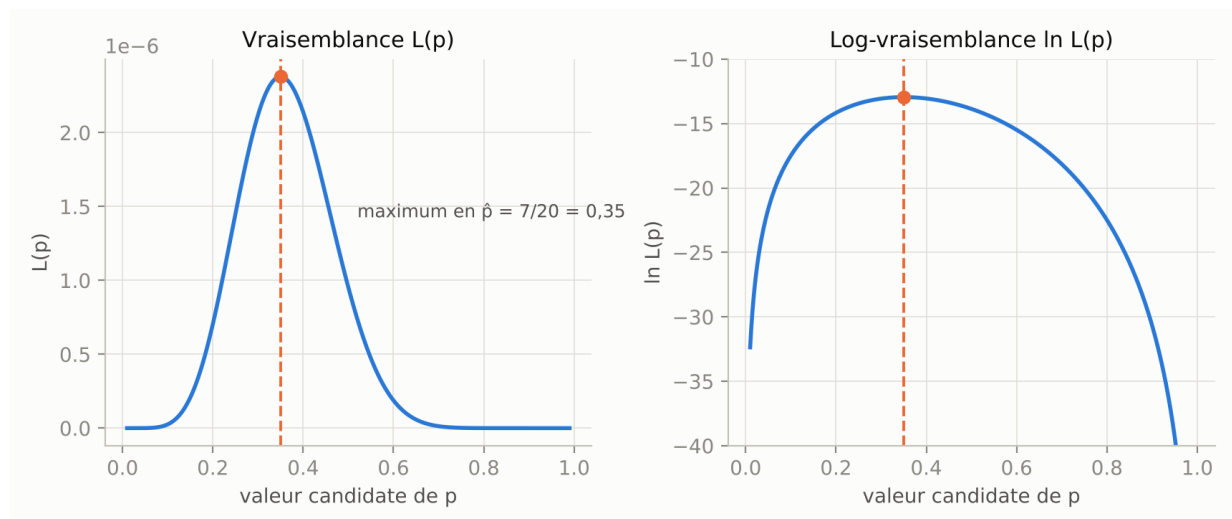


Figure 19: Vraisemblance (à gauche) et log-vraisemblance (à droite) pour 7 achats sur 20 visiteurs. Le maximum est en $p = 0,35$.

Vérifions par calcul numérique : on cherche le maximum sur une grille, puis avec un optimiseur (la méthode générale quand il n'y a pas de formule).

```
k, n_obs = 7, 20
grille = np.linspace(0.001, 0.999, 999)
loglik = stats.binom.logpmf(k, n_obs, grille)
print("maximum sur une grille :", round(grille[np.argmax(loglik)], 3))
```

```
# optimiseur : on minimise la log-vraisemblance NÉGATIVE
res = optimize.minimize_scalar(lambda p: -stats.binom.logpmf(k, n_obs, p), bounds=(0.001, 0.999),
    ↪ method="bounded")
print("optimiseur          :", round(res.x, 4))
```

```
maximum sur une grille : 0.35
optimiseur              : 0.35
```

D'autres exemples classiques (mêmes calculs, à faire en exercice) :

Modèle	Estimateur du maximum de vraisemblance
Bernoulli / binomiale	$\hat{p} = \bar{x}$ (proportion observée)
Poisson(λ)	$\hat{\lambda} = \bar{x}$
Exponentielle(λ)	$\hat{\lambda} = 1/\bar{x}$
Normale(μ, σ^2)	$\hat{\mu} = \bar{x}$; $\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$ ($\triangle$ divisé par n : biaisé !)

On retrouve pour l'exponentielle la même formule qu'avec les moments. Remarquez la dernière ligne : l'EMV de la variance divisé par n , donc est **légèrement biaisé**. Le maximum de vraisemblance n'est pas toujours sans biais ; il a d'autres qualités.

Quand il n'y a pas de formule : l'optimisation numérique. Pour la loi Gamma des montants, on ne peut pas résoudre à la main. On confie la log-vraisemblance à un optimiseur, exactement comme au chapitre 1 (la descente de gradient en est l'ancêtre) :

```
def neg_loglik_gamma(params, x):
    k, theta = params
    if k <= 0 or theta <= 0:
        return np.inf
    return -stats.gamma.logpdf(x, a=k, scale=theta).sum()

res = optimize.minimize(neg_loglik_gamma, x0=[k_mom, theta_mom], args=(m.to_numpy(),),
    ↪ method="Nelder-Mead")
k_mle, theta_mle = res.x
print("Gamma par maximum de vraisemblance : k =", round(k_mle, 3), " theta =", round(theta_mle, 2))
print("Gamma par moments                  : k =", round(k_mom, 3), " theta =", round(theta_mom, 2))
print("log-vraisemblance maximale :", round(-res.fun, 1))
print("log-vraisemblance (moments):", round(-neg_loglik_gamma([k_mom, theta_mom], m.to_numpy()), 1))
```

```
Gamma par maximum de vraisemblance : k = 3.001  theta = 20.07
Gamma par moments                  : k = 2.511  theta = 23.99
log-vraisemblance maximale : -1938.9
log-vraisemblance (moments): -1942.2
```

Le maximum de vraisemblance trouve des paramètres de log-vraisemblance **plus élevée** que ceux des moments (c'est sa définition : il est le meilleur *pour les données observées*). Ici les deux approches donnent des valeurs du même ordre (la forme passe de 2,5 à 3,0). N'oubliez pas qu'une loi Gamma n'est qu'un **modèle** des montants parmi d'autres (on a vu au 3.1.5 qu'une loi log-normale convient aussi bien) : estimer les paramètres ne dit pas si le modèle est juste. Notez aussi que la bibliothèque sait faire tout cela d'un coup :

```
k_sp, loc_sp, theta_sp = stats.gamma.fit(m, floc=0) # floc=0 : on impose une borne inférieure à 0
print("scipy.stats.gamma.fit :", round(k_sp, 3), round(theta_sp, 2))
```

```
scipy.stats.gamma.fit : 3.001 20.07
```


3.2.6 Pourquoi le maximum de vraisemblance est la référence

Sous des conditions de régularité raisonnables, l'EMV a quatre propriétés remarquables, que l'on admettra :

1. **Cohérent** : $\hat{\theta} \rightarrow \theta$ quand $n \rightarrow \infty$.
2. **Asymptotiquement normal** : $\hat{\theta} \approx \mathcal{N}(\theta, 1/(nI(\theta)))$ pour n grand, où $I(\theta)$ est l'**information de Fisher** (la courbure moyenne de la log-vraisemblance autour du maximum : plus le pic est pointu, plus on est précis).
3. **Asymptotiquement efficace** : parmi les estimateurs cohérents, il a (presque) la plus petite variance possible.
4. **Invariant par reparamétrisation** : si $\hat{\theta}$ est l'EMV de θ , alors $g(\hat{\theta})$ est l'EMV de $g(\theta)$.

Le point 2 donne directement l'**erreur-type** : pour la proportion, $I(p) = \frac{1}{p(1-p)}$, donc

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

```
p_hat, n_obs = 7 / 20, 20
se = np.sqrt(p_hat * (1 - p_hat) / n_obs)
print("p chapeau =", p_hat, " erreur-type =", round(se, 3))
print("avec 10 fois plus de données (70/200) :", round(np.sqrt(0.35 * 0.65 / 200), 3))
```

```
p chapeau = 0.35   erreur-type = 0.107
avec 10 fois plus de données (70/200) : 0.034
```

Avec 20 visiteurs, l'erreur-type est de 0,107 (près de 11 points !) : $\hat{p} = 0,35$ est très imprécis. Avec 200 visiteurs, elle tombe à 0,034. Estimer, c'est bien ; **chiffrer l'incertitude de l'estimation** est mieux : c'est l'objet de la section suivante.

À retenir (estimation).

- Un **estimateur** est une formule appliquée à l'échantillon ; c'est une variable aléatoire dont on étudie le **biais**, la **variance**, la **cohérence**. $EQM = Var + Biais^2$.
- $\bar{X}$ est sans biais pour μ , et S^2 (**divisé par $n-1$**) est sans biais pour σ^2 (preuve en 3.2.3).
- **Moments** : on égale moments théoriques et observés. **Maximum de vraisemblance** : on maximise $\ell(\theta) = \sum \ln f(x_i; \theta)$; formule fermée si possible, optimiseur sinon.
- Pour une proportion, l'EMV est la fréquence observée, d'erreur-type $\sqrt{\hat{p}(1-\hat{p})/n}$.
- L'EMV est cohérent, asymptotiquement normal et efficace : c'est l'outil standard.

3.3 Intervalles de confiance

Intuition. Dire « le panier moyen est de 60,25 DT » est trompeur : cela suggère une précision que l'on n'a pas. Un meilleur énoncé est : « le panier moyen se situe, avec une confiance de 95 %, entre 56,5 et 64,0 DT ». L'**intervalle de confiance** (IC) transforme une estimation ponctuelle en une **fourchette honnête**, dont la largeur reflète l'incertitude.

3.3.1 Construire un intervalle pour une moyenne

Rappelons ce que nous savons (2.4) : par le théorème central limite, $\bar{X}_n \approx \mathcal{N}(\mu, \sigma^2/n)$. Donc le score centré réduit $\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$ suit à peu près $\mathcal{N}(0, 1)$, et comme $P(-1,96 \leq Z \leq 1,96) = 0,95$:

Construction.

$$P\left(-1,96 \leq \frac{\bar{X}_n - \mu}{\sigma / \sqrt{n}} \leq 1,96\right) = 0,95.$$

On isole μ au milieu de l'encadrement : multiplier par $\sigma / \sqrt{n}$, puis soustraire $\bar{X}_n$ et multiplier par -1 (ce qui renverse les inégalités) :

$$P\left(\bar{X}_n - 1,96 \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X}_n + 1,96 \frac{\sigma}{\sqrt{n}}\right) = 0,95.$$

L'intervalle de confiance à 95 % est donc $\bar{x} \pm 1,96 \frac{\sigma}{\sqrt{n}}$. ■

Il a une structure à retenir absolument, qui se retrouvera partout :

$$\text{estimation} \pm (\text{valeur critique}) \times (\text{erreur-type}).$$

Exemple à la main. Sur 400 commandes, $\bar{x} = 60,25$ DT et $s = 38,02$. L'erreur-type est $38,02 / \sqrt{400} = 1,90$. L'IC à 95 % est $60,25 \pm 1,96 \times 1,90 = 60,25 \pm 3,73$, soit **[56,5 ; 64,0]** DT.

3.3.2 Que veut dire « 95 % de confiance » ?

C'est la phrase la plus mal comprise de la statistique. Elle ne signifie **pas** « il y a 95 % de chances que la vraie moyenne soit dans cet intervalle-ci ». La vraie moyenne μ est un nombre **fixe** (inconnu) : elle y est ou elle n'y est pas. Ce qui est aléatoire, c'est l'**intervalle** (il change à chaque échantillon).

La bonne lecture : si l'on répétait l'expérience un très grand nombre de fois, avec un nouvel échantillon à chaque fois, 95 % des intervalles ainsi construits contiendraient la vraie valeur. La confiance porte sur la **méthode**, pas sur un intervalle particulier.

Voyons-le. Soixante échantillons de 40 commandes, un intervalle à 95 % pour chacun, et la vraie moyenne (connue ici car on traite nos 400 commandes comme la population) en pointillés :

Mesurons-le précisément, sur 20 000 échantillons :

```
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(21)
population = df["montant"].to_numpy()
mu = population.mean()
n_ech, essais = 40, 20_000
t_crit = stats.t.ppf(0.975, n_ech - 1)

couvre = 0
largeurs = []
for _ in range(essais):
    e = rng.choice(population, size=n_ech, replace=False)
    se = e.std(ddof=1) / np.sqrt(n_ech)
    lo, hi = e.mean() - t_crit * se, e.mean() + t_crit * se
    couvre += (lo <= mu <= hi)
    largeurs.append(hi - lo)
print("part des intervalles contenant la vraie moyenne :", round(couvre / essais, 4))
print("largeur moyenne :", round(np.mean(largeurs), 2), "DT")
```

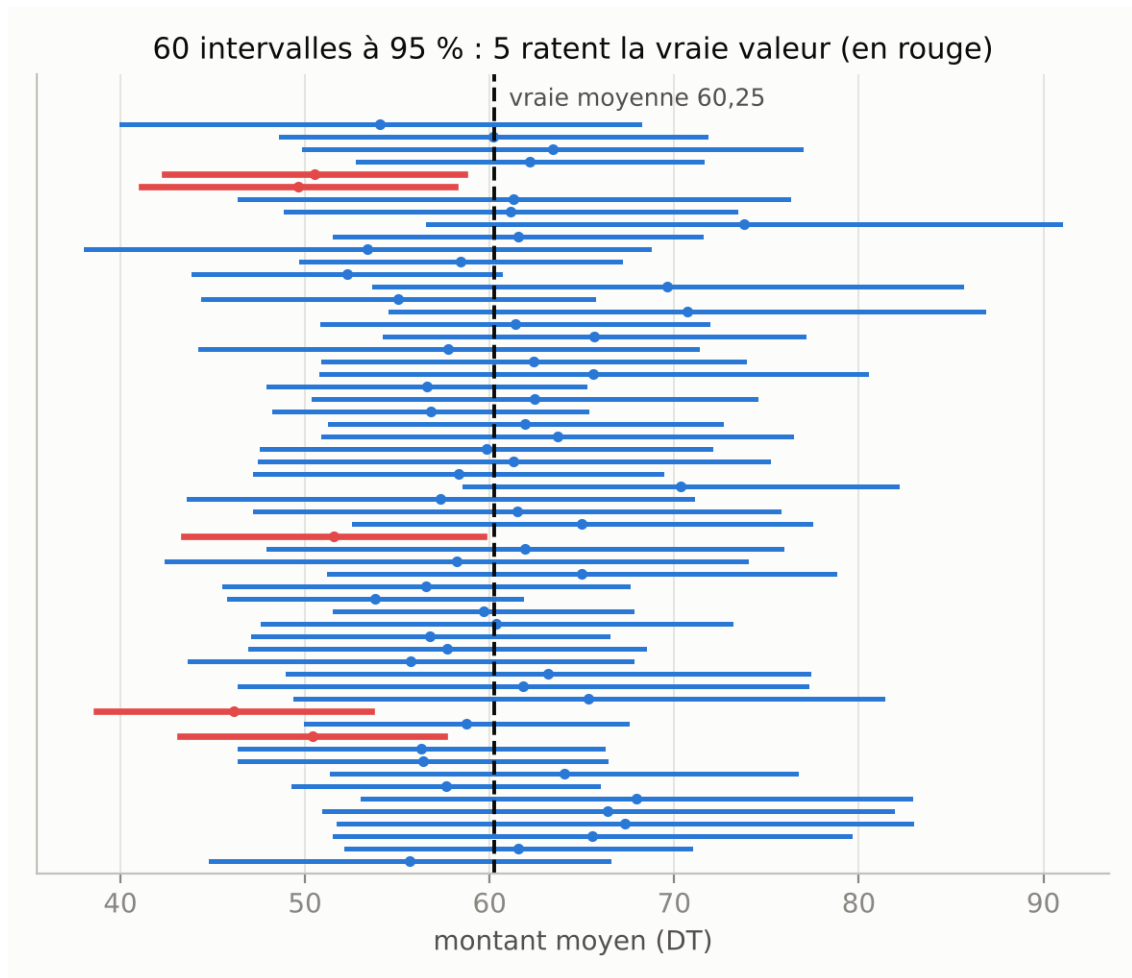


Figure 20: 60 intervalles de confiance à 95 % construits sur 60 échantillons différents de 40 commandes. Les intervalles en rouge n'atteignent pas la vraie moyenne : environ 1 sur 20 en moyenne (5 sur 60 ici, une fluctuation normale).

```
part des intervalles contenant la vraie moyenne : 0.9451
largeur moyenne : 23.88 DT
```

La couverture est proche de 95 % (un peu moins : la loi des montants est asymétrique et $n = 40$ est modeste ; nous reviendrons sur ces limites). L'idée est donc validée.

Deux erreurs d'interprétation à éviter.

1. « La vraie valeur a 95 % de chances d'être dans $[56,5 ; 64,0]$ » : formulation courante, rigoureusement fautive dans l'approche fréquentiste (dans l'approche bayésienne, elle est correcte pour un *intervalle de crédibilité*).
2. « 95 % des **commandes** sont dans cet intervalle » : confusion entre la précision de la **moyenne** et la dispersion des **données**. L'IC de la moyenne est étroit ($[56,5 ; 64,0]$), alors que 95 % des commandes sont entre 19 et 129 DT (3.1.3).

3.3.3 Quand l'écart-type est inconnu : la loi de Student

En pratique, on ne connaît **pas** σ ; on le remplace par son estimation s . Mais s est elle-même aléatoire, ce qui ajoute de l'incertitude : pour de petits échantillons, on tomberait trop souvent à côté avec la valeur 1,96. William Gosset (qui signait « Student » en 1908, alors qu'il travaillait dans une brasserie Guinness) a montré que la bonne loi pour $\frac{\bar{X} - \mu}{S/\sqrt{n}}$ est la **loi de Student** à $n - 1$ **degrés de liberté** (si les données sont à peu près normales).

C'est une cloche comme la normale, mais avec des **queues plus lourdes** (plus de prudence), qui tend vers $\mathcal{N}(0, 1)$ quand n augmente :

```
for ddl in (2, 5, 10, 30, 100, 1000):
    print(f"degrés de liberté = {ddl:>4} : valeur critique à 95 % = {stats.t.ppf(0.975, ddl):.3f}")
print("loi normale", round(stats.norm.ppf(0.975), 3))
```

```
degrés de liberté = 2 : valeur critique à 95 % = 4.303
degrés de liberté = 5 : valeur critique à 95 % = 2.571
degrés de liberté = 10 : valeur critique à 95 % = 2.228
degrés de liberté = 30 : valeur critique à 95 % = 2.042
degrés de liberté = 100 : valeur critique à 95 % = 1.984
degrés de liberté = 1000 : valeur critique à 95 % = 1.962
loi normale : 1.96
```

Avec 2 degrés de liberté (3 observations), la valeur critique est 4,30 : l'intervalle est plus de deux fois plus large qu'avec 1,96. Dès 30 degrés de liberté, on est proche de 2,04 ; avec 400 observations, la différence avec 1,96 est imperceptible.

L'intervalle devient $\bar{x} \pm t_{n-1, 0,975} \frac{s}{\sqrt{n}}$. Voici le calcul pour nos 400 commandes, à la main puis avec `scipy` :

```
m = df["montant"]
n = len(m)
xbar, s = m.mean(), m.std(ddof=1)
se = s / np.sqrt(n)
t_crit = stats.t.ppf(0.975, n - 1)
print(f"moyenne = {xbar:.2f}   erreur-type = {se:.2f}   t critique = {t_crit:.3f}")
print(f"IC à 95 % (à la main) : [{xbar - t_crit * se:.2f} ; {xbar + t_crit * se:.2f}]")
print(f"IC à 95 % (scipy) : ", np.round(stats.t.interval(0.95, n - 1, loc=xbar, scale=se), 2))
```

```
moyenne = 60.25   erreur-type = 1.90   t critique = 1.966
IC à 95 % (à la main) : [56.51 ; 63.98]
IC à 95 % (scipy) : [56.51 63.98]
```

Et pour chaque canal ? On répète le calcul par groupe :

```
def ic_moyenne(x, niveau=0.95):
    x = np.asarray(x)
    se = x.std(ddof=1) / np.sqrt(len(x))
    return stats.t.interval(niveau, len(x) - 1, loc=x.mean(), scale=se)

for canal in ["Instagram", "Site", "Boutique"]:
    x = df.loc[df["canal"] == canal, "montant"]
    lo, hi = ic_moyenne(x)
    print(f"{canal:<10} n = {len(x):>3}    moyenne = {x.mean():5.1f}    IC95 % = [{lo:5.1f} ; {hi:5.1f}]"
```

```
Instagram n = 138    moyenne = 49.0    IC95 % = [ 43.8 ; 54.2]
Site      n = 148    moyenne = 59.5    IC95 % = [ 53.3 ; 65.7]
Boutique  n = 114    moyenne = 74.8    IC95 % = [ 67.3 ; 82.4]
```

Les intervalles d'Instagram ([43,8 ; 54,2]) et de la boutique ([67,3 ; 82,4]) **sont très éloignés** : c'est un indice sérieux que ces deux canaux diffèrent vraiment. L'intervalle du site ([53,3 ; 65,7]) chevauche légèrement celui d'Instagram mais pas celui de la boutique. Attention : « les intervalles se chevauchent » ne prouve **pas** que les moyennes sont égales, et même des intervalles qui se touchent peuvent cacher une différence significative. La bonne méthode est de construire un intervalle (ou un test) pour la **différence** elle-même, ce que nous ferons au 3.4.

Ce qui fait varier la largeur. La demi-largeur est $t \times s / \sqrt{n}$. Elle **diminue** quand n augmente (en $1/\sqrt{n}$), **augmente** quand la dispersion s augmente, et **augmente** quand on exige plus de confiance (99 % donne un intervalle plus large que 95 %). Il n'y a pas de gratuité : plus de certitude coûte en précision.

```
for niveau in (0.80, 0.90, 0.95, 0.99):
    lo, hi = ic_moyenne(m, niveau)
    print(f"confiance {niveau:.0%} : [{lo:.2f} ; {hi:.2f}]    largeur = {hi - lo:.2f}")
```

```
confiance 80% : [57.81 ; 62.69]    largeur = 4.88
confiance 90% : [57.11 ; 63.38]    largeur = 6.27
confiance 95% : [56.51 ; 63.98]    largeur = 7.47
confiance 99% : [55.33 ; 65.17]    largeur = 9.84
```

3.3.4 Intervalle pour une proportion

Pour un taux de conversion $\hat{p} = k/n$, l'erreur-type vue au 3.2.6 est $\sqrt{\hat{p}(1-\hat{p})/n}$, et l'intervalle approché (dit de **Wald**) est

$$\hat{p} \pm 1,96 \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Sur 1 000 visiteurs dont 205 achètent : $0,205 \pm 1,96 \times 0,0128 = 0,205 \pm 0,025$, soit **[18,0 % ; 23,0 %]**.

Mais cet intervalle devient **mauvais** pour de petits échantillons ou des proportions proches de 0 ou 1. Par exemple, avec 0 achat sur 20 visiteurs, $\hat{p} = 0$ et l'intervalle de Wald est $[0 ; 0]$: « on est certain que le taux de conversion est exactement nul » ! Absurde. L'**intervalle de Wilson** corrige cela : il est centré non pas sur $\hat{p}$ mais sur une valeur légèrement « tirée vers 1/2 », et ne sort jamais de $[0, 1]$.

```
def ic_wald(k, n, niveau=0.95):
    p = k / n
    z = stats.norm.ppf(1 - (1 - niveau) / 2)
```

```

d = z * np.sqrt(p * (1 - p) / n)
return max(0, p - d), min(1, p + d)

def ic_wilson(k, n, niveau=0.95):
    return stats.binomtest(k, n).proportion_ci(confidence_level=niveau, method="wilson")[:2]

for k, n_obs in [(205, 1000), (7, 20), (0, 20), (2, 15)]:
    wald, wilson = ic_wald(k, n_obs), ic_wilson(k, n_obs)
    print(f"{k:>3}/{n_obs:<5} p = {k / n_obs:.3f}    Wald = [{wald[0]:.3f} ; {wald[1]:.3f}]    Wilson = "
          ↪ [{wilson[0]:.3f} ; {wilson[1]:.3f}]" )

```

205/1000	p = 0.205	Wald = [0.180 ; 0.230]	Wilson = [0.181 ; 0.231]
7/20	p = 0.350	Wald = [0.141 ; 0.559]	Wilson = [0.181 ; 0.567]
0/20	p = 0.000	Wald = [0.000 ; 0.000]	Wilson = [0.000 ; 0.161]
2/15	p = 0.133	Wald = [0.000 ; 0.305]	Wilson = [0.037 ; 0.379]

Pour 1 000 visiteurs, les deux méthodes coïncident. Pour 7/20, l'intervalle est très large : [0,18 ; 0,57] pour Wilson, c'est-à-dire que 20 visiteurs ne permettent quasiment rien de conclure. Pour 0/20, Wilson dit que le taux réel peut aller jusqu'à environ 16 % (la fameuse « règle de trois » : avec 0 événement sur n , la borne haute à 95 % est environ $3/n = 15\%$).

Conseil pratique. Pour une proportion, utilisez **Wilson** (ou une méthode exacte) plutôt que Wald, sauf si n est très grand et p loin de 0 et 1.

3.3.5 Quand il n'y a pas de formule : le bootstrap

Et si l'on veut un intervalle pour la **médiane**, un quantile, un rapport, ou toute autre statistique pour laquelle on ne connaît pas de formule d'erreur-type ? Le **bootstrap** (Efron, 1979) offre une solution étonnamment simple et générale.

Idée. On ne peut pas retirer de nouveaux échantillons dans la vraie population, mais on peut **rééchantillonner dans l'échantillon lui-même**. On tire n valeurs **avec remise** dans nos n observations (certaines apparaissent plusieurs fois, d'autres pas), on recalcule la statistique, et on recommence des milliers de fois. La dispersion des valeurs obtenues imite la dispersion qu'on aurait observée en échantillonnant la vraie population.

L'algorithme (intervalle « percentile »).

1. Répéter B fois (par exemple 10 000) : tirer un échantillon de taille n avec remise ; calculer la statistique.
2. L'intervalle à 95 % est formé des percentiles 2,5 % et 97,5 % des B valeurs obtenues.

```

rng = np.random.default_rng(42)
x = df["montant"].to_numpy()
B = 10_000

meds = np.array([np.median(rng.choice(x, size=len(x), replace=True)) for _ in range(B)])
moys = np.array([rng.choice(x, size=len(x), replace=True).mean() for _ in range(B)])

print("médiane observée :", np.median(x))
print("IC bootstrap 95 % de la médiane :", np.percentile(meds, [2.5, 97.5]).round(1))
print("IC bootstrap 95 % de la moyenne :", np.percentile(moys, [2.5, 97.5]).round(1))
print("(à comparer à l'IC de Student de la moyenne :", np.round(stats.t.interval(0.95, len(x) - 1,
    ↪ loc=x.mean(), scale=x.std(ddof=1) / np.sqrt(len(x))), 1), ")")

```

```

médiane observée : 51.0
IC bootstrap 95 % de la médiane : [47.3 55.1]
IC bootstrap 95 % de la moyenne : [56.7 64. ]

```

(à comparer à l'IC de Student de la moyenne : [56.5 64.])

Pour la moyenne, le bootstrap donne pratiquement le même résultat que la formule de Student : rassurant. Pour la **médiane**, qui n'a pas de formule simple, il fournit un intervalle ([environ 47 ; 55] DT) que l'on n'aurait pas pu obtenir à la main.

Limites. Le bootstrap suppose que l'échantillon est représentatif de la population ; il marche mal pour des statistiques « extrêmes » (le maximum), avec de très petits échantillons, ou en présence de très fortes dépendances entre observations. Il reste un outil de base du data scientist, car il généralise à **n'importe quelle** statistique sans calcul mathématique.

3.3.6 Dimensionner un échantillon

On peut renverser le raisonnement : *quelle précision veut-on ?* Si Yasmine veut estimer le panier moyen à ± 2 DT près, avec une confiance de 95 % et en supposant $s \approx 38$ DT, il faut $1,96 \times 38 / \sqrt{n} \leq 2$, soit

$$n \geq \left(\frac{1,96 s}{\epsilon} \right)^2 = \left(\frac{1,96 \times 38}{2} \right)^2 \approx 1387 \text{ commandes.}$$

```
s, eps = 38, 2
print("n pour une marge de ±2 DT :", int(np.ceil((1.96 * s / eps) ** 2)))
print("n pour une marge de ±1 DT :", int(np.ceil((1.96 * s / 1) ** 2)))
```

```
n pour une marge de ±2 DT : 1387
n pour une marge de ±1 DT : 5548
```

Diviser la marge par 2 demande **4 fois plus de données** (la loi en $1/\sqrt{n}$ du 2.4). C'est pourquoi gagner de la précision devient vite très coûteux.

À retenir (intervalles de confiance).

- Structure universelle : **estimation $\pm$ valeur critique $\times$ erreur-type**.
- IC de la moyenne : $\bar{x} \pm t_{n-1} s / \sqrt{n}$ (Student). IC d'une proportion : préférez **Wilson** à Wald.
- « Confiance à 95 % » signifie : **la méthode** capture la vraie valeur dans 95 % des échantillons possibles. Ce n'est pas une probabilité sur la vraie valeur.
- Largeur : $\downarrow$ avec n (en $1/\sqrt{n}$) ; $\uparrow$ avec la dispersion et avec le niveau de confiance.
- **Bootstrap** : rééchantillonner avec remise pour obtenir un IC de n'importe quelle statistique.
- $n \geq (zs/\epsilon)^2$ pour viser une marge ϵ .

3.4 Tests d'hypothèses

Intuition : le tribunal. Un test d'hypothèses fonctionne comme un procès. On part de la **présomption d'innocence** (l'hypothèse « il ne se passe rien », appelée H_0). On examine les **preuves** (les données). Si les preuves sont **très improbables** dans un monde où l'accusé est innocent, on le **condamne** (on rejette H_0). Sinon, on **acquitte** : cela ne prouve pas son innocence, cela veut seulement dire que les preuves sont insuffisantes.

3.4.1 Le vocabulaire et la méthode

- **Hypothèse nulle** H_0 : l'état de référence, « pas d'effet, pas de différence ». Par exemple : « le panier moyen vaut 55 DT ».
- **Hypothèse alternative** H_1 : ce que l'on cherche à montrer. « Le panier moyen est différent de 55 DT » (test **bilatéral**), ou « supérieur à 55 DT » (test **unilatéral**).
- **Statistique de test** T : un nombre calculé sur l'échantillon qui mesure l'écart entre les données et ce que prédit H_0 .
- **Niveau de signification** α (souvent 5 %) : le risque que l'on accepte de se tromper en rejetant H_0 alors qu'elle est vraie.
- **p-valeur** : la probabilité, si H_0 est vraie, d'obtenir un résultat **au moins aussi extrême** que celui observé. Petite p-valeur = les données sont surprenantes sous H_0 .

Les deux erreurs possibles.

	H_0 est vraie	H_0 est fausse
On rejette H_0	✗ Erreur de type I (faux positif), probabilité α	✓ bonne décision (probabilité $1 - \beta =$ puissance)
On ne rejette pas H_0	✓ bonne décision	✗ Erreur de type II (faux négatif), probabilité β

On fixe α à l'avance, et on cherche à garder β petit (c'est la question de la puissance, 3.5).

La recette en 5 étapes, valable pour tous les tests de ce chapitre :

1. Poser H_0 et H_1 .
2. Choisir α (avant de regarder les données !).
3. Calculer la statistique de test.
4. Calculer la p-valeur (ou comparer à la valeur critique).
5. Conclure : si $p < \alpha$, **rejeter** H_0 ; sinon, **ne pas rejeter** (et jamais « accepter »).

3.4.2 Test de Student sur une moyenne

Question de Yasmine. Son objectif de panier moyen était de 55 DT. Les 400 commandes confirment-elles que le panier moyen **diffère** de 55 DT ?

1. $H_0 : \mu = 55 ; H_1 : \mu \neq 55$.
2. $\alpha = 0,05$.
3. Statistique de test (la même construction qu'au 3.3.3, centrée sur la valeur de H_0) :

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} = \frac{60,25 - 55}{38,02/\sqrt{400}} = \frac{5,25}{1,90} \approx 2,76.$$

Sous H_0 , t suit une loi de Student à $n - 1 = 399$ degrés de liberté. 4. La p-valeur est la probabilité qu'une telle loi donne une valeur **au moins aussi éloignée de 0** que 2,76, des deux côtés : $p = 2P(T_{399} > 2,76)$.

```
import numpy as np
import pandas as pd
from scipy import stats

m = df["montant"]
```



```

mu0 = 55
n = len(m)
t_obs = (m.mean() - mu0) / (m.std(ddof=1) / np.sqrt(n))
p_val = 2 * stats.t.sf(abs(t_obs), df=n - 1)
print("t observé :", round(t_obs, 3))
print("p-valeur :", round(p_val, 4))
print("valeur critique à 5 % :", round(stats.t.ppf(0.975, n - 1), 3))
res = stats.ttest_1samp(m, popmean=mu0) # la fonction toute faite
print("scipy : t =", round(res.statistic, 3), " p =", round(res.pvalue, 4))

```

```

t observé : 2.76
p-valeur : 0.0061
valeur critique à 5 % : 1.966
scipy : t = 2.76 p = 0.0061

```

5. **Conclusion.** $p = 0,006 < 0,05$: on rejette H_0 . Le panier moyen est significativement supérieur à 55 DT (il est en fait de 60,25 ; l'IC à 95 % du 3.3.3 était [56,5 ; 64,0], qui n'inclut pas 55 : un test bilatéral à 5 % et un IC à 95 % disent la même chose).

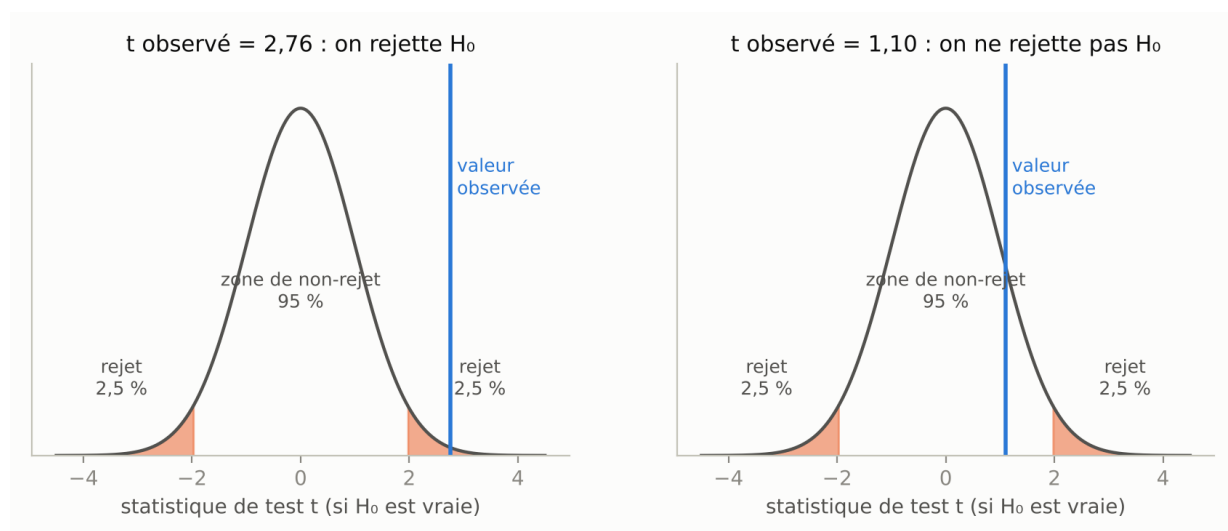


Figure 21: Statistique de test sous H_0 . Gauche : la valeur observée (2,76) tombe dans la zone de rejet (queues orange, 5 % au total). Droite : une valeur de 1,10 tomberait dans la zone de non-rejet.

« Ne pas rejeter » n'est pas « accepter ». Si p avait été de 0,30, on aurait dit « les données ne permettent pas de conclure que $\mu \neq 55$ », pas « $\mu = 55$ ». L'absence de preuve n'est pas la preuve de l'absence. (Un petit échantillon peut échouer à détecter un grand effet : c'est le problème de la puissance.)

3.4.3 Comparer deux groupes : le test de Welch

Question de Yasmine. Les clients de la boutique dépensent-ils plus que ceux d'Instagram ? Les moyennes observées sont 74,8 et 49,0 DT, soit un écart de 25,8 DT. Cet écart est-il crédible ou dû au hasard ?

On teste $H_0 : \mu_B = \mu_I$ contre $H_1 : \mu_B \neq \mu_I$. On compare la différence des moyennes à son erreur-type. Comme les deux échantillons sont indépendants, les variances s'additionnent (2.3.3) :

$$t = \frac{\bar{x}_B - \bar{x}_I}{\sqrt{\frac{s_B^2}{n_B} + \frac{s_I^2}{n_I}}}$$

C'est le **test de Welch**, qui n'exige pas que les deux groupes aient la même variance (c'est la version à utiliser par défaut ; l'ancien test de Student à variances égales est moins sûr).

```
b = df.loc[df["canal"] == "Boutique", "montant"]
i = df.loc[df["canal"] == "Instagram", "montant"]

diff = b.mean() - i.mean()
se_diff = np.sqrt(b.var(ddof=1) / len(b) + i.var(ddof=1) / len(i))
print("différence des moyennes :", round(diff, 2), "DT")
print("erreur-type de la différence :", round(se_diff, 2))
print("t à la main :", round(diff / se_diff, 3))

res = stats.ttest_ind(b, i, equal_var=False)          # equal_var=False -> test de Welch
print("scipy (Welch) : t =", round(res.statistic, 3), " p =", res.pvalue, " ddl =", round(res.df, 1))
```

```
différence des moyennes : 25.8 DT
erreur-type de la différence : 4.64
t à la main : 5.565
scipy (Welch) : t = 5.565    p = 8.016365853327231e-08    ddl = 208.4
```

La statistique est $t \approx 5,56$ et la p-valeur est de l'ordre de 10^{-7} : si les deux canaux avaient la même dépense moyenne, observer un écart aussi grand serait **quasi impossible**. On rejette H_0 .

Un test ne dit pas « de combien ». Une p-valeur minuscule dit que l'effet est *réel*, pas qu'il est *grand*. Il faut toujours accompagner un test d'un **intervalle de confiance de la différence** et d'une **taille d'effet** :

```
t_c = stats.t.ppf(0.975, res.df)
print(f"IC95 % de la différence : [{diff - t_c * se_diff:.1f} ; {diff + t_c * se_diff:.1f}] DT")

s_pooled = np.sqrt(((len(b) - 1) * b.var(ddof=1) + (len(i) - 1) * i.var(ddof=1)) / (len(b) + len(i) - 2))
print("d de Cohen :", round(diff / s_pooled, 2))
```

```
IC95 % de la différence : [16.7 ; 34.9] DT
d de Cohen : 0.72
```

Le client de la boutique dépense en moyenne entre 17 et 35 DT de plus (IC à 95 %). Le **d de Cohen** (la différence en nombre d'écarts-types) vaut environ 0,7 : un effet « moyen à grand » selon les conventions usuelles (0,2 petit, 0,5 moyen, 0,8 grand).

Et la distribution asymétrique ? Le test de Student suppose des moyennes à peu près normales (ce que le TCL assure pour des groupes de plus d'une centaine d'observations) ; il reste correct ici. Pour de petits groupes très asymétriques, on teste plutôt $\log(\text{montant})$, ou on utilise un test non paramétrique (⊕ 3.7). Vérifions que la conclusion tient sur l'échelle logarithmique :

```
res_log = stats.ttest_ind(np.log(b), np.log(i), equal_var=False)
print("test sur log(montant) : t =", round(res_log.statistic, 2), " p =", res_log.pvalue)
res_si = stats.ttest_ind(df.loc[df["canal"] == "Site", "montant"], i, equal_var=False)
print("Site contre Instagram : t =", round(res_si.statistic, 2), " p =", round(res_si.pvalue, 4))
```

```
test sur log(montant) : t = 6.46    p = 5.400582683884821e-10
Site contre Instagram : t = 2.55    p = 0.0113
```

Même conclusion. Pour Site contre Instagram, $p \approx 0,011$: l'écart (10,5 DT) est aussi significatif au seuil de 5 %, mais bien moins fortement.

Le test apparié. Quand les deux séries concernent **les mêmes individus** (avant/après), on ne compare pas deux groupes indépendants : on calcule la **différence pour chaque individu** et on teste que sa moyenne est nulle. Exemple : 8 colis dont on a mesuré le délai avant et après un changement de transporteur.

```
avant = np.array([5, 4, 6, 7, 5, 6, 8, 5])
apres = np.array([4, 4, 5, 6, 5, 5, 6, 4])
d = avant - apres
print("différences :", d, " moyenne :", d.mean())
res = stats.ttest_rel(avant, apres)
print("test apparié : t =", round(res.statistic, 2), " p =", round(res.pvalue, 4))
print("(ttest_lsamp sur les différences donne la même chose :", round(stats.ttest_lsamp(d, 0).pvalue, 4), ")")
```

```
différences : [1 0 1 1 0 1 2 1]  moyenne : 0.875
test apparié : t = 3.86  p = 0.0062
(ttest_lsamp sur les différences donne la même chose : 0.0062 )
```

Le nouveau transporteur fait gagner en moyenne 0,875 jour ($p \approx 0,006$). Ignorer l'appariement (comme si les groupes étaient indépendants) donnerait un test beaucoup moins sensible, car on gaspillerait l'information que chaque colis est comparé à lui-même :

```
print("(à tort, test non apparié : p =", round(stats.ttest_ind(avant, apres).pvalue, 3), ")")
```

```
(à tort, test non apparié : p = 0.128 )
```

3.4.4 Test sur une proportion

Question de Yasmine. Historiquement, le taux de conversion était de 18 %. Sur les 1 000 dernières visites, 205 ont acheté (20,5 %). Y a-t-il une amélioration réelle ?

$H_0 : p = 0,18$ contre $H_1 : p \neq 0,18$. Sous H_0 , l'erreur-type est $\sqrt{p_0(1-p_0)/n}$ (on utilise la valeur de H_0 , pas l'estimation) et la statistique

$$z = \frac{\hat{p} - p_0}{\sqrt{p_0(1-p_0)/n}} = \frac{0,205 - 0,18}{\sqrt{0,18 \times 0,82/1000}} = \frac{0,025}{0,01215} \approx 2,06.$$

On compare à la loi normale (TCL). Il existe aussi un test **exact** basé sur la loi binomiale, sans approximation :

```
k, n_v, p0 = 205, 1000, 0.18
z = (k / n_v - p0) / np.sqrt(p0 * (1 - p0) / n_v)
print("z =", round(z, 3), " p-valeur (approx. normale) =", round(2 * stats.norm.sf(abs(z)), 4))
print("test exact binomial : p-valeur =", round(stats.binomtest(k, n_v, p0).pvalue, 4))
```

```
z = 2.058  p-valeur (approx. normale) = 0.0396
test exact binomial : p-valeur = 0.0436
```

Les deux p-valeurs (0,040 et 0,044) sont **juste en dessous** de 0,05. On rejette H_0 , mais de peu : c'est une preuve **modérée**, pas écrasante. Un intervalle de Wilson pour p , [18,1 % ; 23,1 %], inclut à peine 18 %. Lecture honnête : « il y a des indices d'amélioration, à confirmer avec davantage de données ».

3.4.5 Le test A/B : comparer deux proportions

C'est le test le plus utilisé en pratique dans le web et le marketing. Yasmine essaie deux versions de sa page produit. La version A (1 000 visiteurs) donne 120 achats (12 %), la version B (1 000 visiteurs) donne 150 achats (15 %). B est-elle meilleure ?

$H_0 : p_A = p_B$. Sous H_0 , les deux groupes ont le même taux, estimé en **regroupant** les données :

$\hat{p} = \frac{120+150}{2000} = 0,135$. L'erreur-type de la différence est $\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_A} + \frac{1}{n_B})}$ et

$$z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_A} + \frac{1}{n_B})}} = \frac{0,03}{0,01528} \approx 1,96.$$

```
kA, nA, kB, nB = 120, 1000, 150, 1000
p_pool = (kA + kB) / (nA + nB)
se = np.sqrt(p_pool * (1 - p_pool) * (1 / nA + 1 / nB))
z = (kB / nB - kA / nA) / se
print("z =", round(z, 3), " p-valeur =", round(2 * stats.norm.sf(abs(z)), 4))

# IC de la différence (erreur-type non regroupée)
se_nr = np.sqrt((kA / nA) * (1 - kA / nA) / nA + (kB / nB) * (1 - kB / nB) / nB)
d = kB / nB - kA / nA
print(f"différence = {d:.3f}    IC95 % = [{d - 1.96 * se_nr:.4f} ; {d + 1.96 * se_nr:.4f}]")

z = 1.963    p-valeur = 0.0496
différence = 0.030    IC95 % = [0.0001 ; 0.0599]
```

$p \approx 0,0496$: **tout juste** sous le seuil de 5 %, et l'intervalle de la différence ([0,0 ; 6 points]) frôle zéro. La conclusion « B est meilleure » est **fragile**. Ce cas, très courant, illustre pourquoi le seuil de 0,05 n'est pas une frontière magique : 0,0496 et 0,0504 ne sont pas deux mondes différents (nous y revenons en 3.5).

3.4.6 Le test du khi-deux : deux variables qualitatives sont-elles liées ?

Question de Yasmine. La proportion de clients **satisfaits** (note ≥ 4) dépend-elle du canal de vente ?

On range les données dans un **tableau de contingence** :

```
df["satisfait"] = df["satisfaction"] >= 4
tableau = pd.crosstab(df["canal"], df["satisfait"])
print(tableau)
print()
print(pd.crosstab(df["canal"], df["satisfait"], normalize="index").round(3))
```

satisfait	False	True
canal		
Boutique	5	109
Instagram	51	87
Site	45	103

satisfait	False	True
canal		
Boutique	0.044	0.956
Instagram	0.370	0.630
Site	0.304	0.696

H_0 : le canal et la satisfaction sont **indépendants**. Si c'était vrai, la proportion de satisfaits serait la même dans chaque canal (et égale à la proportion globale). On calcule alors, pour chaque case,

l'effectif attendu sous H_0 :

$$E_{ij} = \frac{(\text{total de la ligne } i) \times (\text{total de la colonne } j)}{\text{total général}}.$$

La statistique du khi-deux mesure l'écart entre effectifs observés (O_{ij}) et attendus :

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

Sous H_0 , elle suit une loi du khi-deux à $(\text{lignes} - 1)(\text{colonnes} - 1)$ degrés de liberté. Plus les écarts sont grands, plus χ^2 est grand.

```
from scipy.stats import chi2_contingency
chi2, p, ddl, attendus = chi2_contingency(tableau)
print("effectifs attendus sous H0 :")
print(pd.DataFrame(attendus, index=tableau.index, columns=tableau.columns).round(1))
print(f"\nkhi-deux = {chi2:.2f}    ddl = {ddl}    p-valeur = {p:.2e}")

n_tot = tableau.values.sum()
cramer_v = np.sqrt(chi2 / (n_tot * (min(tableau.shape) - 1)))
print("V de Cramér :", round(cramer_v, 2))
```

```
effectifs attendus sous H0 :
satisfait False True
canal
Boutique    28.8    85.2
Instagram   34.8   103.2
Site        37.4   110.6
```

```
khi-deux = 38.40    ddl = 2    p-valeur = 4.60e-09
V de Cramér : 0.31
```

Dans la boutique, il y a **109 satisfaits sur 114** (96 %), alors que l'on en attendrait environ 85 si le canal n'avait aucun effet (soit 24 de plus) ; sur Instagram, 63 % seulement (87 sur 138). La statistique est $\chi^2 \approx 38$ pour 2 degrés de liberté : $p \approx 5 \times 10^{-9}$. On rejette l'indépendance. Le **V de Cramér** (0 = indépendance, 1 = lien parfait) vaut 0,31 : un lien d'intensité moyenne. (Attention : la boutique n'a pas de délai de livraison, ce qui explique sans doute en grande partie l'écart ; l'association n'est pas une causalité.)

Condition de validité. L'approximation du khi-deux est fiable si **tous les effectifs attendus sont au moins 5**. Sinon, on utilise le test exact de Fisher (`scipy.stats.fisher_exact` pour un tableau 2x2).

3.4.7 Comment choisir son test ?

Question	Données	Test
La moyenne vaut-elle μ_0 ?	1 variable quantitative	Student à un échantillon
Deux groupes indépendants ont-ils la même moyenne ?	quantitative × 2 groupes	Welch
Avant/après sur les mêmes individus ?	quantitatives appariées	Student apparié
Plus de 2 groupes ?	quantitative × k groupes	ANOVA (<code>f_oneway</code>) ou Kruskal-Wallis (⊗ 3.7)

Question	Données	Test
La proportion vaut-elle p_0 ?	1 variable binaire	z (ou binomial exact)
Deux proportions égales ? (A/B)	binaire $\times$ 2 groupes	z à deux proportions, ou khi-deux
Deux variables qualitatives liées ?	catégorielle $\times$ catégorielle	khi-deux (ou Fisher)
Deux variables quantitatives liées ?	quantitative $\times$ quantitative	test de corrélation (pearsonr, spearmanr)

À retenir (tests d'hypothèses).

- On fixe H_0 (« rien ne se passe »), H_1 , α ; on calcule une statistique de test et sa **p-valeur** ; on rejette si $p < \alpha$.
- Deux erreurs : **type I** (faux positif, probabilité α) et **type II** (faux négatif, probabilité β). Puissance = $1 - \beta$.
- « Ne pas rejeter » $\neq$ « accepter ». Une p-valeur faible dit que l'effet est *réel*, pas qu'il est *grand* : donnez toujours l'**IC** de l'effet et une **taille d'effet**.
- Student (une moyenne), **Welch** (deux moyennes), apparié, z (proportions), **khi-deux** (deux variables qualitatives).
- Un test bilatéral à 5 % équivaut à regarder si 0 (ou la valeur de H_0) est dans l'IC à 95 %.

3.5 p-valeurs, puissance et tests multiples

Au 3.4, nous avons *utilisé* la p-valeur. Voici maintenant comment ne pas s'en servir de travers. Cette section est celle qui vous évitera le plus d'erreurs concrètes : une bonne partie des « découvertes » publiées qui ne se reproduisent pas viennent de ce que nous allons voir.

3.5.1 Ce que la p-valeur est vraiment

Définition. La **p-valeur** est la probabilité, calculée **en supposant H_0 vraie**, d'obtenir une statistique de test **au moins aussi extrême** que celle observée.

$$p = P(\text{résultat aussi extrême ou plus} \mid H_0).$$

Observez la direction du conditionnement : c'est $P(\text{données} \mid H_0)$ et **non** $P(H_0 \mid \text{données})$. Nous avons vu au 2.1 que **inverser un conditionnement est l'erreur classique**.

Pour la sentir, rien de mieux que de fabriquer un monde où H_0 est vraie et de regarder les p-valeurs qu'on obtient. Simulons 10 000 tests de Student de deux groupes de 30 individus **tirés dans la même loi** (donc aucune vraie différence) :

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(1)
pvals_h0 = np.array([stats.ttest_ind(rng.normal(0, 1, 30), rng.normal(0, 1, 30)).pvalue for _ in
    range(10_000)])

print("proportion de p < 0,05 quand H0 est vraie :", round((pvals_h0 < 0.05).mean(), 4))
print("proportion de p < 0,01", round((pvals_h0 < 0.01).mean(), 4))
print("proportion de p < 0,50", round((pvals_h0 < 0.50).mean(), 4))
print("moyenne des p-valeurs", round(pvals_h0.mean(), 3))
```

```

proportion de  $p < 0,05$  quand  $H_0$  est vraie : 0.0486
proportion de  $p < 0,01$  : 0.0103
proportion de  $p < 0,50$  : 0.5013
moyenne des p-valeurs : 0.499

```

Quand H_0 est vraie, la **p-valeur suit une loi uniforme sur $[0,1]$** : 5 % des tests donnent $p < 0,05$, 1 % donnent $p < 0,01$, etc. C'est exactement ce que signifie « niveau $\alpha = 5\%$ » : **un test sur vingt crie au loup à tort quand il n'y a rien**. Ce n'est pas un défaut du test, c'est sa définition.

Et quand H_0 est **faus**se ? Les p-valeurs se tassent vers 0 :

```

pvals_h1 = np.array([stats.ttest_ind(rng.normal(0.8, 1, 30), rng.normal(0, 1, 30)).pvalue for _ in
    range(10_000)])
print("avec un vrai effet (d = 0,8) : proportion de  $p < 0,05$  :", round((pvals_h1 < 0.05).mean(), 3))
print("histogramme (10 classes) sous  $H_0$  :", np.histogram(pvals_h0, bins=10, range=(0, 1))[0])
print("histogramme (10 classes) sous  $H_1$  :", np.histogram(pvals_h1, bins=10, range=(0, 1))[0])

```

```

avec un vrai effet (d = 0,8) : proportion de  $p < 0,05$  : 0.865
histogramme (10 classes) sous  $H_0$  : [ 980 1008 1028 1006 991 985 1036 964 1024 978]
histogramme (10 classes) sous  $H_1$  : [9248 419 160 62 47 25 16 9 7 7]

```

Sous H_0 , l'histogramme est **plat** ; sous H_1 , il est entassé à gauche. C'est un outil de diagnostic précieux : si vous testez des milliers de variables et que l'histogramme de vos p-valeurs est plat, il n'y a probablement **rien** à trouver.

3.5.2 Ce que la p-valeur n'est pas

Cinq contresens fréquents. Une p-valeur de 0,03 ne signifie **pas** :

1. que H_0 a 3 % de chances d'être vraie ;
2. que H_1 a 97 % de chances d'être vraie ;
3. que l'effet est **grand** ou **important** ;
4. que l'on obtiendrait de nouveau $p < 0,05$ en refaisant l'étude (la **reproductibilité** dépend de la puissance) ;
5. qu'on a 3 % de chances de se tromper en rejetant H_0 .

Le point 5 mérite une démonstration, car il est lourd de conséquences. Le risque réel de se tromper quand on rejette dépend de la **proportion d'hypothèses qui sont vraies** au départ : on retrouve la formule de Bayes de la section 2.1 et l'erreur du taux de base !

Exemple. Yasmine teste 1 000 idées d'amélioration (couleur d'un bouton, texte d'une promotion, ordre des produits...). Réalistement, **10 %** seulement ont un vrai effet (100 vraies idées, 900 inutiles). Son test a un niveau $\alpha = 5\%$ et une puissance de 80 %.

- Vraies idées détectées : $100 \times 0,80 = 80$.
- Idées inutiles « détectées » à tort : $900 \times 0,05 = 45$.
- Au total, 125 résultats « significatifs », dont **45 sont des faux positifs**.

$$P(\text{fausse découverte} \mid \text{significatif}) = \frac{45}{125} = 36\%.$$

Plus d'un résultat « significatif » sur trois est faux, alors que α n'est que de 5 % ! Même mécanisme que l'alerte antifraude du 2.1.6. C'est pourquoi on exige des preuves plus fortes pour des hypothèses peu plausibles a priori (« des affirmations extraordinaires exigent des preuves extraordinaires »).

```
alpha, puissance, part_vraies = 0.05, 0.80, 0.10
vrais_positifs = part_vraies * puissance
faux_positifs = (1 - part_vraies) * alpha
print("proportion de faux parmi les significatifs :", round(faux_positifs / (vrais_positifs +
↵ faux_positifs), 3))
```

```
proportion de faux parmi les significatifs : 0.36
```

3.5.3 Signification statistique $\neq$ importance pratique

Avec assez de données, **n'importe quelle** différence, même ridicule, devient « significative ». Un exemple extrême : deux versions d'une page ont des taux de conversion de 20,00 % et 20,10 %, mesurés sur 10 millions de visiteurs chacune.

```
nA = nB = 10_000_000
pA, pB = 0.2000, 0.2010
p_pool = (pA + pB) / 2
z = (pB - pA) / np.sqrt(p_pool * (1 - p_pool) * (2 / nA))
print("z =", round(z, 2), " p-valeur =", f"{2 * stats.norm.sf(z):.1e}")
print("gain absolu :", round((pB - pA) * 100, 2), "point de pourcentage")
```

```
z = 5.58      p-valeur = 2.3e-08
gain absolu : 0.1 point de pourcentage
```

La p-valeur est inférieure à 0,001 (hautement « significatif »), mais le gain est de **0,1 point** de conversion. Est-il utile ? Cela dépend du coût du changement, pas de la p-valeur. **Toujours rapporter la taille de l'effet et son intervalle de confiance**, jamais seulement « $p < 0,05$ ».

3.5.4 La puissance : savoir si l'on peut voir ce qu'on cherche

Intuition. Un test est comme un détecteur de métaux. Un détecteur peu sensible ne signale pas un petit objet enterré profondément : **l'absence de signal ne prouve pas l'absence d'objet**. La **puissance** $1 - \beta$ est la probabilité que le test détecte un effet **s'il existe vraiment** (de taille donnée).

La puissance dépend de quatre choses liées entre elles :

Facteur	Si...	...alors la puissance
Taille de l'effet	grandit	augmente
Taille d'échantillon n	grandit	augmente
Dispersion des données	diminue	augmente
Niveau α	on l'assouplit (0,10 au lieu de 0,05)	augmente (au prix de plus de faux positifs)

Retour sur notre test A/B du 3.4.5 (12 % contre 15 %, 1 000 visiteurs par version). Quelle était la puissance de cette expérience ? Sous H_1 avec $p_A = 0,12$ et $p_B = 0,15$, l'erreur-type de la différence est $\sqrt{\frac{0,12 \times 0,88}{1000} + \frac{0,15 \times 0,85}{1000}} = 0,0154$, et la statistique de test est centrée sur $0,03 / 0,0154 = 1,95$. La puissance est donc la probabilité que cette statistique dépasse 1,96 :

$$\text{puissance} = P(Z > 1,96 - 1,95) \approx 0,50.$$


```
def puissance_ab(p1, p2, n, alpha=0.05):
    se = np.sqrt(p1 * (1 - p1) / n + p2 * (1 - p2) / n)
    return stats.norm.cdf(abs(p2 - p1) / se - stats.norm.ppf(1 - alpha / 2))

print("puissance de l'expérience A/B du 3.4.5 :", round(puissance_ab(0.12, 0.15, 1000), 3))

# vérification par simulation
rng = np.random.default_rng(2)
rejets = 0
for _ in range(10_000):
    a, b = rng.binomial(1000, 0.12), rng.binomial(1000, 0.15)
    pp = (a + b) / 2000
    z = (b / 1000 - a / 1000) / np.sqrt(pp * (1 - pp) * 2 / 1000)
    rejets += abs(z) > 1.96
print("puissance simulée          :", rejets / 10_000)

puissance de l'expérience A/B du 3.4.5 : 0.502
puissance simulée          : 0.4954
```

La puissance n'était que de **50 %** : même si B est réellement meilleure de 3 points, l'expérience n'avait qu'une **chance sur deux** de le détecter. Le résultat « tout juste significatif » obtenu était donc de la chance autant que de l'information. Un test sous-dimensionné est un pari.

Dimensionner l'expérience avant de la lancer. On fixe l'effet minimal intéressant (ici +3 points), $\alpha = 5\%$ et la puissance voulue (80 % est l'usage), puis on calcule n :

$$n \text{ par groupe} = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 [p_1(1-p_1) + p_2(1-p_2)]}{(p_2 - p_1)^2}.$$

```
za, zb = stats.norm.ppf(0.975), stats.norm.ppf(0.80)
def n_par_groupe(p1, p2):
    return (za + zb) ** 2 * (p1 * (1 - p1) + p2 * (1 - p2)) / (p2 - p1) ** 2

for delta in (0.05, 0.03, 0.02, 0.01):
    print(f"détecter 12 % -> {12 + delta * 100:.0f} % : {int(np.ceil(n_par_groupe(0.12, 0.12 + delta)))>6} visiteurs par version")

détecter 12 % -> 17 % :    775 visiteurs par version
détecter 12 % -> 15 % :   2033 visiteurs par version
détecter 12 % -> 14 % :   4435 visiteurs par version
détecter 12 % -> 13 % :  17166 visiteurs par version
```

Pour détecter 3 points avec 80 % de puissance, il faut **environ 2 000 visiteurs par version**, soit le double de ce que Yasmine avait. Pour détecter 1 point, il en faut **près de 18 000**. La loi en $1/\text{effet}^2$ est impitoyable : diviser l'effet par 3 multiplie les besoins par 9.

L'arrêt prématuré (*peeking*). Dans une expérience en ligne, la tentation est forte de regarder les résultats chaque jour et de s'arrêter dès que $p < 0,05$. La figure de droite montre le résultat : en regardant 20 fois, le **taux de faux positifs passe de 5 % à environ 25 %**, alors qu'il n'y a *aucun* effet réel. Règle : **fixer la taille d'échantillon à l'avance et ne conclure qu'à la fin** (ou utiliser des méthodes séquentielles conçues pour cela).

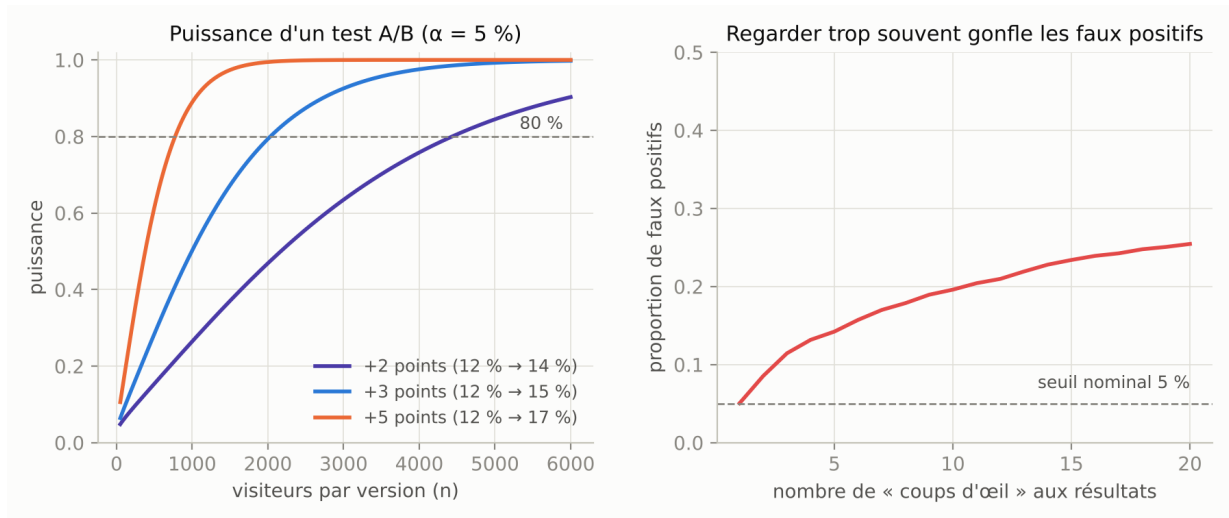


Figure 22: À gauche : puissance d'un test A/B en fonction du nombre de visiteurs, pour trois tailles d'effet. À droite : si l'on « jette un œil » aux résultats de plus en plus souvent et que l'on s'arrête dès que $p < 0,05$, le taux de faux positifs explose (sous H_0).

3.5.5 Les tests multiples : le piège du « fouillis de comparaisons »

Intuition. Si vous lancez un dé 20 fois, vous obtiendrez presque sûrement un 6 quelque part. Si vous effectuez 20 tests à 5 %, il est presque sûr que l'un d'eux sera « significatif » par pur hasard.

Raisonnons comme au 2.1.4 (« au moins un ») : si les 20 tests sont indépendants et que toutes les hypothèses nulles sont vraies, la probabilité d'avoir **au moins un faux positif** est

$$1 - (1 - \alpha)^m = 1 - 0,95^{20} \approx 0,64.$$

```
for m_tests in (1, 5, 10, 20, 50, 100):
    print(f"{m_tests:>3} tests : P(au moins un faux positif) = {1 - 0.95 ** m_tests:.3f}")
```

```
1 tests : P(au moins un faux positif) = 0.050
5 tests : P(au moins un faux positif) = 0.226
10 tests : P(au moins un faux positif) = 0.401
20 tests : P(au moins un faux positif) = 0.642
50 tests : P(au moins un faux positif) = 0.923
100 tests : P(au moins un faux positif) = 0.994
```

Avec 100 tests, c'est quasi certain (99,4 %). Dès que l'on teste plusieurs variables, segments ou métriques, il faut en tenir compte. C'est exactement ce qui arrive quand on « fouille » un jeu de données : on regarde 40 sous-groupes et on rapporte celui qui sort (« les femmes de 25 à 34 ans achètent plus le jeudi »).

Les corrections classiques. On teste m hypothèses nulles, avec les p-valeurs $p_1, \dots, p_m$.

- **Bonferroni** : on rejette H_i si $p_i < \alpha/m$. Simple, très prudent (le **FWER**, probabilité d'au moins un faux positif, reste $\leq \alpha$) mais il perd beaucoup de puissance quand m est grand.
- **Holm** : trie les p-valeurs et applique des seuils $\alpha/m, \alpha/(m-1), \dots$; **toujours** au moins aussi puissant que Bonferroni, avec la même garantie. À préférer.
- **Benjamini-Hochberg (BH)** : contrôle non plus le risque d'un seul faux positif, mais la **proportion de fausses découvertes** parmi les rejets (le **FDR**, *false discovery rate*). Moins strict,

beaucoup plus puissant. Idéal en exploration (criblage de centaines de variables).

Procédure de Benjamini–Hochberg. Trier les p-valeurs : $p_{(1)} \leq \dots \leq p_{(m)}$. Trouver le plus grand k tel que $p_{(k)} \leq \frac{k}{m} \alpha$. Rejeter les hypothèses correspondant à $p_{(1)}, \dots, p_{(k)}$.

Mettons-les à l'épreuve dans une simulation réaliste : Yasmine compare 100 catégories de produits entre deux périodes. Parmi elles, **10** ont vraiment changé (effet $d = 1$) et **90** n'ont pas bougé. Chaque comparaison utilise 40 observations par période.

```
rng = np.random.default_rng(5)
m_tests, n_vrais, n_obs = 100, 10, 40
vraie_diff = np.array([1.0] * n_vrais + [0.0] * (m_tests - n_vrais))
pvals = np.array([stats.ttest_ind(rng.normal(d, 1, n_obs), rng.normal(0, 1, n_obs)).pvalue for d in
    ↪ vraie_diff])

def bonferroni(p, alpha=0.05):
    return p < alpha / len(p)

def holm(p, alpha=0.05):
    ordre = np.argsort(p)
    rejet = np.zeros(len(p), dtype=bool)
    for rang, idx in enumerate(ordre):
        if p[idx] < alpha / (len(p) - rang):
            rejet[idx] = True
        else:
            break
    return rejet

def benjamini_hochberg(p, alpha=0.05):
    m = len(p)
    ordre = np.argsort(p)
    seuils = (np.arange(1, m + 1) / m) * alpha
    ok = p[ordre] <= seuils
    rejet = np.zeros(m, dtype=bool)
    if ok.any():
        k = np.max(np.where(ok)[0])
        rejet[ordre[: k + 1]] = True
    return rejet

vrai = vraie_diff > 0
for nom, rejet in [("aucune correction (p < 0,05)", pvals < 0.05), ("Bonferroni", bonferroni(pvals)),
    ("Holm", holm(pvals)), ("Benjamini-Hochberg", benjamini_hochberg(pvals))]:
    tp, fp = int((rejet & vrai).sum()), int((rejet & ~vrai).sum())
    print(f"{nom:<30} découvertes = {tp + fp:>2}    vraies = {tp:>2}    fausses = {fp:>2}")
```

aucune correction (p < 0,05)	découvertes = 12	vraies = 9	fausses = 3
Bonferroni	découvertes = 7	vraies = 7	fausses = 0
Holm	découvertes = 7	vraies = 7	fausses = 0
Benjamini-Hochberg	découvertes = 9	vraies = 9	fausses = 0

Lecture (pour cette graine) : sans correction, on « découvre » 12 effets, dont **3 sont de fausses alertes** (sur 90 hypothèses nulles, on s'attend à environ 4,5 faux positifs à 5 %). Bonferroni et Holm n'en gardent que 7, **tous vrais**, mais au prix d'avoir **raté** 3 vrais effets. Benjamini-Hochberg en retrouve 9 vrais sans aucun faux ici ; par construction, il garantit seulement qu'en moyenne la part de fausses découvertes reste sous 5 %. Le compromis est net : plus on corrige strictement, moins on se trompe, mais plus on rate de vrais effets.

Quel choix pratique ?

- Décision importante, peu d'hypothèses, un faux positif coûteux (lancer un produit, un traitement) : **Holm** (ou Bonferroni).
- Exploration de nombreuses hypothèses, où l'on vérifiera ensuite les candidats : **Benjamini-Hochberg**.
- Le mieux de tout : **décider à l'avance** de la ou des questions testées. Une analyse exploratoire est une source d'hypothèses, pas une preuve.

3.5.6 Les bonnes pratiques, en dix lignes

1. Écrire H_0 , H_1 , α et le plan d'analyse **avant** de regarder les données.
2. Dimensionner l'échantillon pour une puissance d'au moins 80 %.
3. Ne pas s'arrêter dès que $p < 0,05$ (*peeking*).
4. Compter **tous** les tests effectués, pas seulement ceux qui « marchent », et corriger si nécessaire.
5. Rapporter la **taille d'effet** et un **intervalle de confiance**, pas seulement la p-valeur.
6. Distinguer significativité statistique et importance pratique.
7. Ne pas dire « accepter H_0 » : dire « pas de preuve suffisante ».
8. Se méfier d'un résultat « juste significatif » (0,04) : il est fragile.
9. Marquer clairement ce qui est **exploratoire** et ce qui est **confirmatoire**.
10. Refaire l'expérience si la décision est importante : la **réplication** est la meilleure preuve.

À retenir (p-valeurs, puissance, tests multiples).

- $p = P(\text{données aussi extrêmes} \mid H_0)$. Sous H_0 , elle est **uniforme** sur $[0, 1]$; 5 % des tests donnent $p < 0,05$ par hasard.
- Ce n'est ni $P(H_0 \mid \text{données})$, ni la taille de l'effet. Le taux de fausses découvertes dépend de la proportion d'hypothèses vraies (Bayes).
- **Puissance** = $1 - \beta$: dépend de l'effet, de n , de la dispersion et de α . Dimensionner avant l'expérience : $n \propto 1/\text{effet}^2$.
- **Tests multiples** : $1 - (1 - \alpha)^m$; corriger par **Holm** (FWER) ou **Benjamini-Hochberg** (FDR). Pas de *peeking*, pas de *p-hacking*.

3.6 ☉ Pour aller plus loin : les sondages et l'échantillonnage

Section optionnelle. Tout ce chapitre suppose que l'échantillon est « tiré au hasard dans la population ». Mais **comment** l'obtient-on, et que se passe-t-il quand ce n'est pas le cas ? La théorie des sondages répond à ces questions, essentielles pour une enquête de satisfaction, une étude de marché, ou tout jeu de données dont on ne maîtrise pas la collecte.

3.6.1 Le biais de sélection : le pire ennemi

Une leçon historique. En 1936, le magazine *Literary Digest* prédit la défaite de Roosevelt à l'élection américaine, d'après plus de **2 millions** de réponses reçues à son questionnaire. Roosevelt a été réélu largement. Au même moment, le tout jeune institut Gallup, avec un échantillon de quelques milliers de personnes seulement mais mieux choisi, avait prévu la victoire. Le magazine avait sollicité ses abonnés, des annuaires et des propriétaires de voitures : des personnes **plus aisées que la moyenne** des électeurs. Aucune quantité de données ne corrige un échantillon qui **ne représente pas** la population.

C'est la leçon centrale de cette section : **la taille de l'échantillon réduit la variance, pas le biais**. Un million d'observations mal choisies donne un résultat précis... et faux.

Les formes de biais les plus courantes :

Biais	Mécanisme	Exemple chez Dar Jasmin
Sélection	la méthode de recrutement favorise certains profils	enquête par e-mail : seuls les clients déjà inscrits à la newsletter répondent
Non-réponse	les répondants diffèrent des non-répondants	seuls les clients très contents (ou très fâchés) répondent
Survie	on n'observe que ceux « qui restent »	analyser uniquement les clients encore actifs surestime la satisfaction
Couverture	une partie de la population n'est pas dans la base	un sondage en ligne ignore les clients sans accès à Internet

3.6.2 L'échantillonnage aléatoire simple

Dans un **échantillon aléatoire simple** (EAS), chaque individu de la base de sondage a la **même probabilité** d'être choisi, et tous les groupes de n individus sont également probables. C'est le modèle de tout ce que nous avons fait. L'estimateur de la moyenne est $\bar{x}$; quand on tire **sans remise** dans une population de taille finie N , l'erreur-type est corrigée par le **facteur de population finie** :

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}} \sqrt{1 - \frac{n}{N}}.$$

Si l'on interroge une grande part de la population (n/N non négligeable), l'incertitude diminue plus vite. Si $n \ll N$ (le cas habituel), le facteur vaut presque 1 : **ce qui compte, c'est n , pas la fraction interrogée**. Voilà pourquoi sonder 1 000 personnes suffit autant pour un pays de 10 millions d'habitants que pour une ville de 100 000.

La marge d'erreur d'un sondage. Pour une proportion estimée à $\hat{p}$ avec n personnes, la marge d'erreur à 95 % est $1,96\sqrt{\hat{p}(1 - \hat{p})/n}$. Son maximum est atteint pour $\hat{p} = 0,5$, ce qui donne la **règle à retenir** : marge $\approx \frac{1}{\sqrt{n}}$.

```
import numpy as np
from scipy import stats
```

```
for n_s in (100, 400, 1000, 2500, 10000):
    marge = 1.96 * np.sqrt(0.25 / n_s)
    print(f"n = {n_s:>6} : marge d'erreur maximale = ±{marge * 100:.1f} points    (règle 1/sqrt(n) =
    ↳ ±{100 / np.sqrt(n_s):.1f})")
```

```
n =    100 : marge d'erreur maximale = ±9.8 points    (règle 1/sqrt(n) = ±10.0)
n =    400 : marge d'erreur maximale = ±4.9 points    (règle 1/sqrt(n) = ±5.0)
n =   1000 : marge d'erreur maximale = ±3.1 points    (règle 1/sqrt(n) = ±3.2)
n =   2500 : marge d'erreur maximale = ±2.0 points    (règle 1/sqrt(n) = ±2.0)
n =  10000 : marge d'erreur maximale = ±1.0 points    (règle 1/sqrt(n) = ±1.0)
```

Avec 1 000 personnes : $\pm 3,1$ points. Pour obtenir ± 1 point, il en faut près de 10 000. C'est pourquoi les sondages nationaux s'arrêtent le plus souvent autour de 1 000 à 2 000 personnes. Et cette marge ne couvre que l'**erreur d'échantillonnage** : elle ne dit rien du biais de sélection ou de non-réponse, qui sont souvent plus grands.

3.6.3 L'échantillonnage stratifié

Intuition. Si la population est composée de groupes **homogènes en eux-mêmes mais différents entre eux** (les canaux de vente !), il est dommage de laisser le hasard décider combien de chaque groupe tombera dans l'échantillon. On **découpe** la population en **strates** et on tire un échantillon aléatoire **dans chaque strate**, en proportion de sa taille. On garantit ainsi une représentation fidèle, et l'on gagne en précision.

L'**estimateur stratifié** pondère les moyennes de strates par leur poids dans la population : $\bar{x}_{\text{strat}} = \sum_h W_h \bar{x}_h$ avec $W_h = N_h/N$.

Montrons le gain par simulation. La base clients de Dar Jasmin compte 10 000 personnes réparties en trois canaux, dont les dépenses moyennes diffèrent nettement :

```
rng = np.random.default_rng(50)
tailles = {"Instagram": 4000, "Site": 3500, "Boutique": 2500}
base = {"Instagram": 3.7, "Site": 3.9, "Boutique": 4.1}
canaux_pop = np.concatenate([[c] * n for c, n in tailles.items()])
depenses_pop = np.concatenate([np.exp(rng.normal(base[c], 0.55, size=n)) for c, n in tailles.items()])
N = len(depenses_pop)
vraie_moyenne = depenses_pop.mean()
print("taille de la population :", N, "   vraie dépense moyenne :", round(vraie_moyenne, 2), "DT")

n_ech, essais = 200, 5000
est_eas, est_strat = [], []
masques = {c: canaux_pop == c for c in tailles}
poids = {c: tailles[c] / N for c in tailles}
for _ in range(essais):
    # EAS : 200 clients au hasard dans toute la base
    est_eas.append(rng.choice(depenses_pop, size=n_ech, replace=False).mean())
    # stratifié proportionnel : 80 Instagram, 70 Site, 50 Boutique
    moy = 0
    for c in tailles:
        n_h = int(round(n_ech * poids[c]))
        moy += poids[c] * rng.choice(depenses_pop[masques[c]], size=n_h, replace=False).mean()
    est_strat.append(moy)

print("EAS           : moyenne =", round(np.mean(est_eas), 2), "   erreur-type =", round(np.std(est_eas),
↳ 2))
print("Stratifié     : moyenne =", round(np.mean(est_strat), 2), "   erreur-type =",
↳ round(np.std(est_strat), 2))
print("gain de variance :", round(1 - np.var(est_strat) / np.var(est_eas), 3))
```

```
taille de la population : 10000   vraie dépense moyenne : 56.4 DT
EAS           : moyenne = 56.39   erreur-type = 2.46
Stratifié     : moyenne = 56.39   erreur-type = 2.36
gain de variance : 0.081
```

Les deux estimateurs sont **sans biais** (leur moyenne tombe sur la vraie valeur), mais l'estimateur stratifié est **plus précis** : son erreur-type (2,36 DT) est inférieure d'environ 4 % à celle de l'EAS (2,46 DT), soit 8 % de variance en moins. Le gain est modeste ici car les différences entre canaux, bien que réelles, restent petites comparées à la dispersion à l'intérieur de chaque canal. Il serait bien plus grand si les strates étaient très différentes entre elles.

Pourquoi ça marche : décomposition de la variance. La variance totale se décompose en variance **entre** strates et variance **à l'intérieur** des strates : $\sigma^2 = \sigma_{\text{entre}}^2 + \sigma_{\text{intra}}^2$. Dans un EAS, le hasard de la composition de l'échantillon introduit l'incertitude liée à la variance *entre* strates. La stratification **fixe** cette composition, et seule la variance *intra* demeure : l'erreur-

type diminue exactement de la part « entre ».

L'allocation de Neyman. On peut aller plus loin : au lieu d'allouer proportionnellement à la taille, on interroge **davantage** les strates **plus hétérogènes** (de grand écart-type) : $n_h \propto N_h \sigma_h$. Ici, la boutique est la plus dispersée (écart-type de ses dépenses plus élevé en valeur absolue) : on gagnerait à en sur-échantillonner un peu.

3.6.4 Autres plans de sondage

Plan	Principe	Avantage	Inconvénient
Systématique	un individu tous les k dans la liste	simple	biais si la liste a une périodicité
Par grappes	on tire des groupes entiers (magasins, classes) puis on interroge tout le groupe	peu coûteux (déplacements)	moins précis (individus d'une grappe se ressemblent)
À plusieurs degrés	tirage de grappes puis d'individus dans les grappes	pratique pour de vastes populations	calcul d'erreur plus complexe
Par quotas	on remplit des quotas (âge, sexe...) sans tirage aléatoire	rapide, peu coûteux	pas de théorie d'erreur rigoureuse
De convenance	on prend ceux qui sont disponibles	très facile	biais incontrôlable

Les plans aléatoires (EAS, stratifié, grappes) permettent de **quantifier** l'incertitude ; les plans de quotas ou de convenance non. C'est une raison de plus de se méfier des « sondages » de réseaux sociaux.

3.6.5 Redresser un échantillon biaisé : la pondération

On ne choisit pas toujours son échantillon. Yasmine envoie un questionnaire de satisfaction à tous ses clients ; les **réponses sont inégalement réparties** : les clients de la boutique répondent très peu (ils ne laissent pas d'e-mail), ceux d'Instagram beaucoup. Parmi les 300 réponses : 150 d'Instagram, 120 du site et 30 de la boutique, alors que la clientèle réelle est répartie en 40 % / 35 % / 25 %.

Si l'on moyenne naïvement les 300 réponses, la boutique est **sous-représentée** (10 % au lieu de 25 %). Or les clients de la boutique sont aussi les plus satisfaits : on **sous-estime** donc la satisfaction globale. La solution est de **pondérer** chaque réponse par $w = \frac{\text{part dans la population}}{\text{part dans l'échantillon}}$ (une *post-stratification*).

```
satisf_vraie = {"Instagram": 0.63, "Site": 0.70, "Boutique": 0.96} # taux de satisfaits par canal
↳ (3.4.6)
pop_part = {"Instagram": 0.40, "Site": 0.35, "Boutique": 0.25}
rep = {"Instagram": 150, "Site": 120, "Boutique": 30}
n_rep = sum(rep.values())

naif = sum(rep[c] * satisf_vraie[c] for c in rep) / n_rep
poids_c = {c: pop_part[c] / (rep[c] / n_rep) for c in rep}
pondere = sum(rep[c] * poids_c[c] * satisf_vraie[c] for c in rep) / sum(rep[c] * poids_c[c] for c in
↳ rep)
vrai = sum(pop_part[c] * satisf_vraie[c] for c in pop_part)
```



```
print("poids :", {c: round(w, 2) for c, w in poids_c.items()})
print("satisfaction vraie (population) :", round(vrai, 3))
print("estimation naïve          :", round(naif, 3))
print("estimation pondérée       :", round(pondere, 3))
```

```
poids : {'Instagram': 0.8, 'Site': 0.87, 'Boutique': 2.5}
satisfaction vraie (population) : 0.737
estimation naïve          : 0.691
estimation pondérée       : 0.737
```

La moyenne naïve (environ 69 %) sous-estime la vraie valeur (73,7 %) ; la pondération corrige l'erreur. Chaque réponse de la boutique « compte pour » 2,5 réponses (poids 2,5) et chaque réponse d'Instagram pour 0,8. (Cette correction n'est valable que si, **à l'intérieur de chaque canal**, répondants et non-répondants sont comparables : la pondération redresse les déséquilibres **observables**, pas ceux que l'on ne mesure pas.)

À retenir (sondages).

- **La taille ne corrige pas le biais** (*Literary Digest*) ; ce qui compte, c'est la qualité du tirage.
- **EAS** : marge d'erreur $\approx 1/\sqrt{n}$ (± 3 points pour 1 000 personnes), indépendante de la taille de la population si elle est grande.
- **Stratification** : on tire dans chaque groupe, en proportion ; estimateur $\sum W_h \bar{x}_h$, plus précis que l'EAS quand les strates diffèrent.
- Plans par grappes, de quotas, de convenance : moins précis ou sans théorie d'erreur.
- On redresse un échantillon déséquilibré par **pondération** ($w = \text{part population} / \text{part échantillon}$), sous réserve de comparabilité à l'intérieur des groupes.

3.7 ☉ Pour aller plus loin : les méthodes non paramétriques

Section optionnelle. Les tests du 3.4 (Student, Welch) supposent, au moins approximativement, une loi normale des moyennes. Les méthodes **non paramétriques** (ou *sans loi*) évitent de postuler une forme de distribution. Elles sont précieuses pour de petits échantillons, des variables ordinales (notes de 1 à 5) ou des données très asymétriques et pleines de valeurs extrêmes.

3.7.1 Remplacer les valeurs par leurs rangs

Intuition. Au lieu de travailler sur les valeurs, on les **range** du plus petit au plus grand et l'on travaille sur leurs **rangs** (1er, 2e, 3e...). Une valeur extrême de 1 000 000 n'a que le rang « dernier » : elle ne peut plus fausser le résultat. Les rangs perdent un peu d'information (l'ampleur des écarts), mais gagnent une **robustesse** considérable.

```
import numpy as np
import pandas as pd
from scipy import stats

x = np.array([12, 15, 14, 10, 13, 40]) # un client a dépensé 40 : valeur extrême
print("valeurs :", x)
print("rangs   :", stats.rankdata(x))
```

```
valeurs : [12 15 14 10 13 40]
rangs   : [2. 5. 4. 1. 3. 6.]
```


L'extrême (40) reçoit simplement le rang 6, le même qu'il aurait eu en valant 16.

3.7.2 Le test de Mann-Whitney (deux groupes indépendants)

C'est l'équivalent non paramétrique du test de Welch. On mélange les deux groupes, on range toutes les valeurs, puis on regarde si les rangs d'un groupe sont systématiquement plus élevés que ceux de l'autre.

Interprétation très parlante. La statistique $U/(n_1n_2)$ est la probabilité qu'une observation tirée au hasard dans le groupe A **dépasse** une observation tirée au hasard dans le groupe B. Valeur 0,5 : aucune différence ; proche de 1 : A domine B. (C'est aussi l'AUC du volume II.)

Un cas où le test de Student se trompe. Deux petits groupes de 6 commandes. Dans le premier, un client dépense une somme exceptionnelle :

```
ga = np.array([12, 15, 14, 10, 13, 40])
gb = np.array([9, 8, 11, 10, 7, 12])
print("moyennes :", ga.mean().round(1), gb.mean().round(1))
print("Welch      : p =", round(stats.ttest_ind(ga, gb, equal_var=False).pvalue, 3))
print("Mann-Whitney : p =", round(stats.mannwhitneyu(ga, gb).pvalue, 3))
```

```
moyennes : 17.3 9.5
Welch      : p = 0.15
Mann-Whitney : p = 0.02
```

Presque **tous** les clients du premier groupe dépensent plus que ceux du second ; seul le cas de 40 gonfle la variance et noie l'effet dans le test de Student ($p \approx 0,15$, non significatif). Le test de Mann-Whitney, lui, voit la domination systématique du groupe A ($p \approx 0,02$, significatif). C'est le gain de puissance de la robustesse quand les données sont « sales ».

Sur nos 400 commandes (boutique contre Instagram) :

```
b = df.loc[df["canal"] == "Boutique", "montant"]
i = df.loc[df["canal"] == "Instagram", "montant"]
u = stats.mannwhitneyu(b, i, alternative="two-sided")
print("U =", u.statistic, " p-valeur =", u.pvalue)
print("P(commande boutique > commande Instagram) =", round(u.statistic / (len(b) * len(i)), 3))
```

```
U = 11246.0    p-valeur = 4.410098845865466e-09
P(commande boutique > commande Instagram) = 0.715
```

Une commande de la boutique dépasse une commande d'Instagram dans 71 % des paires comparées ($p \approx 4 \times 10^{-9}$). Conclusion identique à celle du test de Welch, avec une interprétation plus intuitive.

3.7.3 Autres tests de rangs

Situation	Test paramétrique	Équivalent non paramétrique
2 groupes indépendants	Welch	Mann-Whitney (mannwhitneyu)
2 séries appariées	Student apparié	Wilcoxon des rangs signés (wilcoxon)
$k > 2$ groupes	ANOVA (f_oneway)	Kruskal-Wallis (kruskal)
Corrélation	Pearson	Spearman / Kendall (spearmanr, kendalltau)

```
# Wilcoxon : les 8 colis avant/après du 3.4.3
avant = np.array([5, 4, 6, 7, 5, 6, 8, 5])
apres = np.array([4, 4, 5, 6, 5, 5, 6, 4])
print("Wilcoxon apparié :", round(stats.wilcoxon(avant, apres).pvalue, 4))

# Kruskal-Wallis : les trois canaux ensemble
groupes = [df.loc[df["canal"] == c, "montant"] for c in ["Instagram", "Site", "Boutique"]]
print("ANOVA : p =", f"{stats.f_oneway(*groupes).pvalue:.1e}")
print("Kruskal-Wallis : p =", f"{stats.kruskal(*groupes).pvalue:.1e}")

# Corrélation entre le délai et la satisfaction (variable ordinale)
print("Pearson :", np.round(stats.pearsonr(df["livraison"], df["satisfaction"]), 3))
print("Spearman :", np.round(stats.spearmanr(df["livraison"], df["satisfaction"]), 3))
print("Kendall :", np.round(stats.kendalltau(df["livraison"], df["satisfaction"]), 3))

Wilcoxon apparié : 0.0312
ANOVA : p = 3.4e-07
Kruskal-Wallis : p = 9.4e-09
Pearson : [-0.532 0. ]
Spearman : [-0.511 0. ]
Kendall : [-0.437 0. ]
```

Les lignes de corrélation donnent (coefficient, p-valeur) ; la p-valeur arrondie vaut 0. Pour la satisfaction (note de 1 à 5, **ordinale**), Spearman et Kendall sont plus appropriés que Pearson. Dans les trois cas, le lien entre délai et satisfaction est très significatif.

Quand choisir le non paramétrique ? Données ordinales ; petits échantillons ($n < 20$) d'allure non normale ; valeurs extrêmes qu'on ne veut pas supprimer. Si les données sont vraiment normales, le test de Student est un peu plus puissant (de l'ordre de 5 %) : le non paramétrique est une **assurance bon marché**.

3.7.4 Les tests de permutation : l'idée la plus simple de la statistique

Intuition. H_0 dit : « le canal n'a aucun effet sur le montant ». Si c'est vrai, l'étiquette « boutique » ou « Instagram » collée sur une commande est **arbitraire** : on aurait pu l'échanger avec n'importe quelle autre. Alors **mélangeons** les étiquettes au hasard, recalculons la différence de moyennes, et recommençons des milliers de fois. On obtient ainsi la **distribution de la différence quand H_0 est vraie**, sans aucune hypothèse de loi. La p-valeur est la fréquence des mélanges qui donnent une différence au moins aussi grande que celle observée.

```
rng = np.random.default_rng(12)
valeurs = np.concatenate([b.to_numpy(), i.to_numpy()])
n_b = len(b)
diff_obs = b.mean() - i.mean()

n_perm = 20_000
diffs = np.empty(n_perm)
for k in range(n_perm):
    melange = rng.permutation(valeurs)
    diffs[k] = melange[:n_b].mean() - melange[n_b:].mean()

p_perm = (np.sum(np.abs(diffs) >= abs(diff_obs)) + 1) / (n_perm + 1)
print("différence observée :", round(diff_obs, 2), "DT")
print("plus grande différence parmi les 20 000 mélanges :", round(np.abs(diffs).max(), 2), "DT")
print("p-valeur de permutation :", p_perm)
```

```
différence observée : 25.8 DT
plus grande différence parmi les 20 000 mélanges : 20.58 DT
```

```
p-valeur de permutation : 4.999750012499375e-05
```

Aucun des 20 000 mélanges n'atteint la différence observée de 25,8 DT : la différence maximale obtenue par hasard est bien plus petite. On majore donc la p-valeur par $1/20\,001 \approx 5 \times 10^{-5}$ (le « +1 » évite de déclarer $p = 0$). Faisons maintenant la même chose sur la petite expérience à 6 + 6 commandes, où l'on peut même énumérer toutes les permutations possibles :

```
from itertools import combinations
tout = np.concatenate([ga, gb])
obs = tout[:6].mean() - tout[6:].mean()
n_plus_extreme, total = 0, 0
for idx in combinations(range(12), 6):          # les 924 façons de choisir 6 commandes sur 12
    masque = np.zeros(12, dtype=bool)
    masque[list(idx)] = True
    d = tout[masque].mean() - tout[~masque].mean()
    n_plus_extreme += abs(d) >= abs(obs) - 1e-12
    total += 1
print("nombre de permutations :", total)
print("p-valeur exacte de permutation :", round(n_plus_extreme / total, 4))
```

```
nombre de permutations : 924
p-valeur exacte de permutation : 0.0173
```

Il y a $\binom{12}{6} = 924$ manières de répartir les 12 valeurs en deux groupes (clin d'œil au 1.6 !) ; la p-valeur est la proportion de ces 924 répartitions dont l'écart de moyennes est au moins aussi grand que celui observé. Le test est **exact** et n'a besoin d'aucune hypothèse. Il est très souple : on peut l'appliquer à **n'importe quelle statistique** (médiane, rapport, corrélation), comme le bootstrap.

3.7.5 Tester la normalité

Comment savoir si l'hypothèse de normalité du test de Student est raisonnable ? Deux outils.

Le diagramme quantile-quantile (QQ-plot) : on compare les quantiles des données à ceux d'une loi normale ; si les points suivent la droite, c'est normal. **Le test de Shapiro-Wilk** : $H_0 =$ « les données sont normales ».

```
print("Shapiro-Wilk sur le montant      : p =", f"{stats.shapiro(df['montant']).pvalue:.1e}")
print("Shapiro-Wilk sur log(montant)    : p =", round(stats.shapiro(np.log(df["montant"])).pvalue, 3))
z = stats.zscore(np.log(df["montant"]))
print("Kolmogorov-Smirnov (log, normal) : p =", round(stats.kstest(z, "norm").pvalue, 3))
```

```
Shapiro-Wilk sur le montant      : p = 3.0e-18
Shapiro-Wilk sur log(montant)    : p = 0.846
Kolmogorov-Smirnov (log, normal) : p = 0.879
```

Le montant brut est **clairement non normal** ($p \approx 10^{-18}$), alors que son logarithme est tout à fait compatible avec la normalité (pas de rejet : $p \approx 0,85$), ce qui confirme la structure **log-normale** vue au 3.1.5. Le **test de Kolmogorov-Smirnov** compare la fonction de répartition observée à celle d'une loi donnée (ou deux échantillons entre eux).

Piège : tester la normalité n'est pas toujours utile. Avec beaucoup de données, ces tests rejettent la normalité pour des écarts infimes sans conséquence ; avec peu de données, ils ne détectent rien. On s'appuie surtout sur les **graphiques** et sur le **TCL** : la normalité de la **moyenne** (ce qui compte pour Student) est assurée dès que n est grand, même si les données ne sont pas normales.

À retenir (non paramétrique).

- Travailler sur les **rangs** rend robuste aux valeurs extrêmes et applicable aux données ordinales.
- **Mann-Whitney** (2 groupes), **Wilcoxon** (apparié), **Kruskal-Wallis** (k groupes), **Spearman/Kendall** (corrélation).
- **Test de permutation** : on mélange les étiquettes pour fabriquer la loi de la statistique sous H_0 ; valable pour toute statistique.
- **Shapiro-Wilk** et **Kolmogorov-Smirnov** testent une loi, mais préférez les graphiques et le TCL.

3.8 Exercices du chapitre 3

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse.

Énoncés

Exercice 1 ★ (descriptif). Huit commandes en DT : 12, 15, 15, 18, 20, 22, 25, 60. Calculez la moyenne, la médiane, le mode, l'écart-type (avec $n - 1$) et l'écart interquartile. Quelle mesure de position est la plus représentative, et pourquoi ?

Exercice 2 ★ (variance sans biais). Un échantillon de 5 délais de livraison : 4, 8, 6, 5, 7 jours. Calculez la variance en divisant par n puis par $n - 1$. Laquelle utiliser pour estimer la variance de **tous** les délais ?

Exercice 3 ★★ (maximum de vraisemblance). Le nombre de commandes par heure a été relevé 8 fois : 2, 3, 1, 4, 0, 3, 2, 5. On modélise par une loi de Poisson(λ). (a) Écrivez la log-vraisemblance et trouvez $\hat{\lambda}$ en dérivant. (b) Avec cette estimation, quelle est la probabilité de ne recevoir aucune commande pendant une heure ?

Exercice 4 ★ (IC d'une moyenne). Sur $n = 36$ commandes : $\bar{x} = 52$ DT, $s = 12$ DT. Donnez un intervalle de confiance à 95 % de la moyenne, puis à 99 %. Quelle est la signification du « 95 % » ?

Exercice 5 ★★ (IC d'une proportion). Un questionnaire envoyé à 60 clients donne 18 « très satisfaits ». Calculez l'IC à 95 % de la vraie proportion par la méthode de Wald et par celle de Wilson. Pourquoi sont-ils différents ?

Exercice 6 ★★ (test sur une moyenne). Le transporteur promet un délai moyen de 3 jours. Sur 25 colis, on mesure en moyenne 3,6 jours avec un écart-type de 1,5 jour. (a) Testez $H_0 : \mu = 3$ contre $H_1 : \mu \neq 3$ à 5 %. (b) Que change un test unilatéral $H_1 : \mu > 3$? (c) Peut-on dire que le transporteur respecte sa promesse ?

Exercice 7 ★★ (test A/B). Version A : 45 achats sur 500 visiteurs. Version B : 66 achats sur 500 visiteurs. (a) B est-elle meilleure, à 5 % ? (b) Donnez un IC de la différence. (c) Quelle était la puissance de ce test si le vrai écart est celui observé ?

Exercice 8 ★★ (khi-deux). Parmi 100 clients ayant reçu un code promo, 30 ont acheté ; parmi 100 clients sans code, 45 ont acheté. Le code a-t-il un effet ? Construisez le tableau, calculez les effectifs attendus et le test du khi-deux. Comparez avec le test de deux proportions.

Exercice 9 ★★★ (taille d'échantillon). Le taux de conversion actuel est de 20 %. Yasmine veut détecter une hausse à 24 % avec une puissance de 80 % et $\alpha = 5$ %. (a) Combien de visiteurs par version ? (b) Et pour détecter 22 % ? (c) Expliquez le rapport entre les deux résultats.

Exercice 10 ★★★ (tests multiples). Un analyste teste 8 hypothèses et obtient les p-valeurs 0,001; 0,008; 0,012; 0,030; 0,040; 0,200; 0,500; 0,700. Lesquelles sont rejetées à 5 % (a) sans correction, (b) avec Bonferroni, (c) avec Holm, (d) avec Benjamini-Hochberg ?

Corrigés

Corrigé 1. Moyenne : $187/8 = 23,375$. Triées : 12, 15, 15, 18, 20, 22, 25, 60 : la médiane est $(18 + 20)/2 = 19$. Mode : 15. Écart-type ($n - 1$) : voir le code ($\approx 15,4$). Les quartiles (méthode de NumPy) donnent un IQR de 7,75. La **médiane** (19) est la plus représentative : la moyenne (23,4) est tirée vers le haut par la commande de 60 DT (valeur extrême), qui est supérieure de plus du double au reste.

```
import numpy as np
from scipy import stats
x = np.array([12, 15, 15, 18, 20, 22, 25, 60])
print("moyenne :", x.mean(), " médiane :", np.median(x), " mode :", stats.mode(x).mode)
print("écart-type (n-1) :", round(x.std(ddof=1), 2))
print("IQR :", np.percentile(x, 75) - np.percentile(x, 25))
```

```
moyenne : 23.375  médiane : 19.0  mode : 15
écart-type (n-1) : 15.38
IQR : 7.75
```

Corrigé 2. $\bar{x} = 6$; écarts : $-2, 2, 0, -1, 1$; carrés : $4, 4, 0, 1, 1$; somme = 10. Division par n : $10/5 = 2$. Division par $n - 1$: $10/4 = 2,5$. Pour estimer la variance de la **population**, on utilise $n - 1$ (estimateur sans biais, 3.2.3).

```
d = np.array([4, 8, 6, 5, 7])
print("var (n) :", d.var(ddof=0), " var (n-1) :", d.var(ddof=1))
```

```
var (n) : 2.0  var (n-1) : 2.5
```

Corrigé 3. (a) $\ell(\lambda) = \sum_i (-\lambda + x_i \ln \lambda - \ln x_i!) = -n\lambda + \ln \lambda \sum x_i - \sum \ln x_i!$. $\ell'(\lambda) = -n + \frac{\sum x_i}{\lambda} = 0 \Rightarrow \hat{\lambda} = \bar{x} = \frac{20}{8} = 2,5$ (c'est un maximum car $\ell'' = -\sum x_i/\lambda^2 < 0$). (b) $P(N = 0) = e^{-2,5} \approx 0,082$.

```
x = np.array([2, 3, 1, 4, 0, 3, 2, 5])
lam = x.mean()
print("lambda chapeau =", lam, " P(0 commande) =", round(np.exp(-lam), 4))
grille = np.linspace(0.5, 6, 1101)
print("maximum sur grille :", round(grille[np.argmax([stats.poisson.logpmf(x, g).sum() for g in
    ↪ grille])], 2))
```

```
lambda chapeau = 2.5  P(0 commande) = 0.0821
maximum sur grille : 2.5
```

Corrigé 4. Erreur-type : $12/\sqrt{36} = 2$. Valeur critique de Student à 35 ddl : 2,030 (95 %) et 2,724 (99 %). IC 95 % : $52 \pm 2,030 \times 2 = [47,9 ; 56,1]$. IC 99 % : $52 \pm 2,724 \times 2 = [46,6 ; 57,4]$ (plus large : plus de confiance coûte en précision). « 95 % » : si l'on répétait l'enquête de nombreuses fois, environ 95 % des intervalles ainsi construits contiendraient la vraie moyenne.

```
n, xbar, s = 36, 52, 12
se = s / np.sqrt(n)
for niveau in (0.95, 0.99):
    t = stats.t.ppf(1 - (1 - niveau) / 2, n - 1)
    print(f"{niveau:.0%} : t = {t:.3f}  IC = [{xbar - t * se:.1f} ; {xbar + t * se:.1f}]")
```

```
95% : t = 2.030  IC = [47.9 ; 56.1]
99% : t = 2.724  IC = [46.6 ; 57.4]
```

Corrigé 5. $\hat{p} = 18/60 = 0,30$. Wald : erreur-type $\sqrt{0,3 \times 0,7/60} = 0,0592$, IC $0,30 \pm 0,116 = [0,184; 0,416]$. Wilson (voir le code) donne environ $[0,199; 0,425]$: légèrement décalé vers le centre et plus large à droite. Wilson est plus fiable car il tient compte du fait que l'erreur-type dépend de p lui-même, d'où une meilleure couverture pour n modeste.

```
k, n = 18, 60
p = k / n
d = 1.96 * np.sqrt(p * (1 - p) / n)
print("Wald : [", round(p - d, 3), ";", round(p + d, 3), "]")
w = stats.binomtest(k, n).proportion_ci(confidence_level=0.95, method="wilson")
print("Wilson : [", round(w.low, 3), ";", round(w.high, 3), "]")
```

```
Wald : [ 0.184 ; 0.416 ]
Wilson : [ 0.199 ; 0.425 ]
```

Corrigé 6. (a) $t = \frac{3,6 - 3}{1,5/\sqrt{25}} = \frac{0,6}{0,3} = 2,0$ avec 24 ddl. Valeur critique bilatérale à 5 % : 2,064. Comme $2,0 < 2,064$ (et $p \approx 0,057$), on **ne rejette pas** H_0 de justesse. (b) En unilatéral, $p \approx 0,028 < 0,05$: on rejette. Mais on ne peut choisir le sens du test **qu'avant** de voir les données et si l'on n'est vraiment intéressé que par un dépassement ; décider après coup serait tricher. (c) Honnêtement : les données sont **à la limite** : le retard moyen estimé est de 0,6 jour, l'IC à 95 % ($3,6 \pm 2,064 \times 0,3 = [2,98; 4,22]$) inclut 3 de justesse. On ne peut ni affirmer que la promesse est tenue, ni qu'elle ne l'est pas ; il faut davantage de colis.

```
n, xbar, s, mu0 = 25, 3.6, 1.5, 3
t = (xbar - mu0) / (s / np.sqrt(n))
print("t =", t, " p bilatérale =", round(2 * stats.t.sf(t, n - 1), 4), " p unilatérale =",
      round(stats.t.sf(t, n - 1), 4))
tc = stats.t.ppf(0.975, n - 1)
print("IC95 % : [", round(xbar - tc * s / np.sqrt(n), 2), ";", round(xbar + tc * s / np.sqrt(n), 2),
      "]")
```

```
t = 2.0000000000000004    p bilatérale = 0.0569    p unilatérale = 0.0285
IC95 % : [ 2.98 ; 4.22 ]
```

Corrigé 7. (a) $\hat{p}_A = 0,09, \hat{p}_B = 0,132, \hat{p} = \frac{111}{1000} = 0,111$. $z = \frac{0,042}{\sqrt{0,111 \times 0,889 \times (2/500)}} = \frac{0,042}{0,01987} \approx 2,11, p \approx 0,035$: significatif à 5 %. (b) Erreur-type non regroupée $\approx 0,0194$, donc IC = $0,042 \pm 0,039 = [0,003; 0,081]$: l'écart est positif, mais très imprécis (de 0,3 à 8 points !). (c) La puissance, si le vrai écart est celui observé, est d'environ 56 % : l'expérience était sous-dimensionnée (voir le code).

```
kA, kB, n = 45, 66, 500
pA, pB = kA / n, kB / n
pp = (kA + kB) / (2 * n)
z = (pB - pA) / np.sqrt(pp * (1 - pp) * 2 / n)
print("z =", round(z, 3), " p =", round(2 * stats.norm.sf(z), 4))
se = np.sqrt(pA * (1 - pA) / n + pB * (1 - pB) / n)
print("IC95 % de la différence : [", round(pB - pA - 1.96 * se, 4), ";", round(pB - pA + 1.96 * se,
      4), "]")
print("puissance :", round(stats.norm.cdf((pB - pA) / se - 1.96), 3))
```

```
z = 2.114    p = 0.0345
IC95 % de la différence : [ 0.0031 ; 0.0809 ]
puissance : 0.563
```

Corrigé 8. Tableau : code promo : 30 achats, 70 non ; sans code : 45 achats, 55 non. Totaux : colonnes 75 et 125 ; lignes 100 et 100. Effectifs attendus (sous indépendance) : achats $100 \times 75/200 = 37,5$ dans chaque ligne ; non-achats 62,5. Chaque case s'écarte de son attendu de 7,5, donc $\chi^2 =$

$\sum \frac{(O-E)^2}{E} = 2 \times \frac{7,5^2}{37,5} + 2 \times \frac{7,5^2}{62,5} = 3,0 + 1,8 = 4,8$ (sans correction de continuité), 1 ddl, $p \approx 0,029$ (0,041 avec la correction de Yates, plus prudente). **Attention : le code promo est associé à moins d'achats** (30 % contre 45 %) : le test signale une différence, pas son sens. (Et le test à deux proportions donne $z^2 = \chi^2$: même p .)

```
from scipy.stats import chi2_contingency
tab = np.array([[30, 70], [45, 55]])
chi2, p, ddl, att = chi2_contingency(tab, correction=False)
print("attendus :\n", att)
print("khi-deux =", round(chi2, 3), " p =", round(p, 4), " (avec correction de Yates :",
    ↪ round(chi2_contingency(tab)[1], 4), ")")
pp = 75 / 200
z = (0.45 - 0.30) / np.sqrt(pp * (1 - pp) * 2 / 100)
print("z =", round(z, 3), " z^2 =", round(z**2, 3), " p =", round(2 * stats.norm.sf(abs(z)), 4))
```

```
attendus :
[[37.5 62.5]
 [37.5 62.5]]
khi-deux = 4.8  p = 0.0285  (avec correction de Yates : 0.0409 )
z = 2.191  z^2 = 4.8  p = 0.0285
```

Corrigé 9. (a) $n = \frac{(1,96 + 0,8416)^2 [0,2 \times 0,8 + 0,24 \times 0,76]}{0,04^2} \approx 1\,680$ par version. (b) Pour 22 %, l'effet est deux fois plus petit : $n \approx 6\,500$. (c) Diviser l'effet par 2 multiplie n par **environ 4** : $n \propto 1/\text{effet}^2$.

```
za, zb = stats.norm.ppf(0.975), stats.norm.ppf(0.80)
def n_groupe(p1, p2):
    return (za + zb) ** 2 * (p1 * (1 - p1) + p2 * (1 - p2)) / (p2 - p1) ** 2
print("20 -> 24 % :", int(np.ceil(n_groupe(0.20, 0.24))), " 20 -> 22 % :", int(np.ceil(n_groupe(0.20,
    ↪ 0.22))))
print("rapport :", round(n_groupe(0.20, 0.22) / n_groupe(0.20, 0.24), 2))
```

```
20 -> 24 % : 1680  20 -> 22 % : 6507
rapport : 3.87
```

Corrigé 10. (a) Sans correction : $p < 0,05$ pour les 5 premières. (b) Bonferroni : seuil $0,05/8 = 0,00625$: seule 0,001 est rejetée. (c) Holm : seuils $0,05/8 = 0,00625$, $0,05/7 = 0,00714$, $0,05/6 = 0,00833$, ... : $0,001 < 0,00625$ ✓ ; $0,008 > 0,00714$ ✗ : on s'arrête. Un seul rejet, comme Bonferroni. (d) BH : seuils $\frac{k}{8} \times 0,05$: $0,00625$; $0,0125$; $0,01875$; $0,025$; $0,03125$; ... ; $p_{(1)} = 0,001 \leq 0,00625$ ✓, $p_{(2)} = 0,008 \leq 0,0125$ ✓, $p_{(3)} = 0,012 \leq 0,01875$ ✓, $p_{(4)} = 0,030 > 0,025$ ✗, $p_{(5)} = 0,040 > 0,03125$ ✗. Le plus grand k qui convient est 3 : les **trois** premières sont rejetées.

```
p = np.array([0.001, 0.008, 0.012, 0.030, 0.040, 0.200, 0.500, 0.700])
m = len(p)
print("sans correction :", int((p < 0.05).sum()), "rejets")
print("Bonferroni      :", int((p < 0.05 / m).sum()), "rejets")
rang = np.arange(1, m + 1)
holm = np.cumprod(p < 0.05 / (m - rang + 1)) # on s'arrête au premier échec
print("Holm          :", int(holm.sum()), "rejets")
ok = p <= rang / m * 0.05
print("Benjamini-Hochberg :", int(np.max(np.where(ok)[0]) + 1) if ok.any() else 0, "rejets")
```

```
sans correction : 5 rejets
Bonferroni      : 1 rejets
Holm            : 1 rejets
Benjamini-Hochberg : 3 rejets
```

Bilan du chapitre 3

Vous savez maintenant :

- **décrire** un jeu de données (position, dispersion, forme, relations) et **toujours dessiner avant de calculer** ;
- **estimer** un paramètre (moments, maximum de vraisemblance), juger un estimateur par son biais et sa variance, et savoir pourquoi on divise par $n - 1$;
- **quantifier l'incertitude** par des intervalles de confiance (Student, Wilson, bootstrap), sans en faire une mauvaise lecture ;
- **tester** une hypothèse (Student, Welch, proportions, A/B, khi-deux) en distinguant significativité et importance pratique ;
- **dimensionner** une expérience (puissance), et **éviter les faux positifs** (tests multiples, *peeking*) ;
- (en option) **échantillonner** correctement et employer des méthodes **sans hypothèse de loi**.

Le chapitre 4 change de registre : après la théorie, la **pratique du code**. Python, R, algorithmes, NumPy, pandas, visualisations : ce sont les outils qui permettront d'appliquer tout ce que vous venez d'apprendre sur de vraies données.

Chapitre 4 : Programmation

« Les mathématiques vous disent **quoi** calculer. La programmation vous permet de le calculer sur **un million de lignes**. »

Jusqu'ici, nous avons utilisé du code comme un outil de vérification. Ce chapitre prend le code **au sérieux** : c'est lui qui transforme vos idées en résultats reproductibles. Pas besoin d'avoir jamais programmé : on part de zéro. Si vous programmez déjà, parcourez les premières sections en diagonale et attardez-vous sur NumPy, pandas et les visualisations.

Le chemin de ce chapitre

- **4.1 Python** : le langage de ce livre, des variables aux fonctions, avec un petit programme complet (la caisse de la boutique).
- **4.2 R** : l'autre grand langage de la statistique ; on refait les mêmes analyses pour comparer.
- **4.3 Algorithmes et structures de données** : piles, files, dictionnaires, récursion, tris, recherche. Penser comme un informaticien.
- **4.4 NumPy et pandas** : les deux bibliothèques qui font de Python un outil d'analyse de données.
- **4.5 Visualisations** : choisir le bon graphique, le tracer proprement, avec matplotlib, pandas, seaborn et ggplot2.
- **⊕ Pour aller plus loin** : programmation orientée objet, code propre et tests (4.6) ; d'autres langages (4.7) ; complexité algorithmique (4.8).
- **4.9 Exercices corrigés**.

Comment travailler avec ce chapitre. Tapez le code vous-même plutôt que de le copier : c'est la seule façon d'apprendre à programmer. Modifiez-le, cassez-le, observez les messages d'erreur. Ils sont vos amis : un message d'erreur lu attentivement dit presque toujours où est le problème. Pour exécuter du code Python, vous pouvez utiliser un notebook Jupyter (section 6.2) ou simplement un terminal avec la commande `python`.

Le fichier de données. Au chapitre 3, nous avons construit un tableau de 400 commandes. Il a été enregistré dans le fichier `donnees/commandes.csv`, fourni avec le livre (c'est la sortie de `df.to_csv("donnees/commandes.csv", index=False)` appliqué au tableau du 3.1.2). Nous l'utiliserons pour les sections 4.2, 4.4 et 4.5, ainsi qu'au chapitre 5 (SQL).

4.1 Les fondamentaux de Python

Intuition. Un programme est une **recette de cuisine** écrite pour un exécutant très docile mais totalement dépourvu de bon sens : il fait *exactement* ce que vous écrivez, à une vitesse folle, sans jamais se fatiguer, et sans jamais deviner ce que vous vouliez dire. Apprendre à programmer, c'est apprendre à écrire des recettes sans ambiguïté. Python est un excellent choix pour cela : ses recettes se lisent presque comme des phrases.

Dans cette section, nous partons de zéro et nous terminons par un **programme complet** : la caisse de la boutique de Yasmine. Chaque notion suit le même rythme que dans le reste du livre : une image, un exemple fait à la main, puis le code et sa sortie réelle.

Section à lire dans l'ordre. Si vous programmez déjà, lisez seulement les titres et les encadrés $\triangle$, puis passez au programme final (4.1.10) pour vérifier que tout vous semble familier.

4.1.1 Pourquoi Python, et comment l'exécuter

Python est gratuit, lisible, et il dispose de milliers de bibliothèques pour les données (NumPy, pandas, SciPy, matplotlib..., que nous rencontrerons en 4.4 et 4.5). C'est le langage le plus utilisé en data science, et c'est celui des exemples de ce livre.

Il y a trois façons d'exécuter du code Python :

1. **Interactivement**, en tapant python dans un terminal : on écrit une ligne, on voit le résultat tout de suite. Idéal pour essayer.
2. **Dans un fichier** `mon_programme.py`, lancé avec `python mon_programme.py`. Idéal pour garder et rejouer son travail.
3. **Dans un notebook Jupyter** (section 6.2) : des cellules de code mélangées à du texte. Idéal pour explorer et raconter.

Le tout premier programme du monde informatique :

```
print("Bonjour Dar Jasmin !")
print(2 + 3 * 4)
```

```
Bonjour Dar Jasmin !
14
```

La fonction `print` affiche ce qu'on lui donne. La deuxième ligne montre que Python respecte la priorité habituelle des opérations : 3×4 d'abord, puis $+2$, soit 14.

Les commentaires. Tout ce qui suit un `#` sur une ligne est ignoré par Python : c'est une note pour le lecteur humain (vous, dans six mois). Un bon commentaire explique **pourquoi**, pas **quoi**.

4.1.2 Variables et types

Une **variable** est une étiquette collée sur une valeur. L'instruction `prix = 12.5` se lit : « colle l'étiquette `prix` sur la valeur 12,5 ». Ce n'est **pas** une égalité mathématique : à droite on calcule, à gauche on nomme.

Les quatre types de base :

Type	Nom Python	Exemple	À quoi ça sert
Entier	int	3	compter (articles, jours)
Décimal	float	12.5	mesurer (prix, poids) ; séparateur : le point
Texte	str	"bol"	noms, étiquettes
Booléen	bool	True, False	vrai ou faux

Exemple à la main. Yasmine vend 3 bols à 12,5 DT hors taxe. Le total HT est $3 \times 12,5 = 37,5$ DT. Avec 19 % de TVA : $37,5 \times 1,19 = 44,625$ DT, soit 44,63 DT si l'on arrondit « comme à l'école » (la moitié vers le haut). Faisons-le faire à Python :

```
quantite = 3
prix_ht = 12.5
total_ht = quantite * prix_ht
total_ttc = total_ht * 1.19

print("total HT :", total_ht)
print("total TTC :", total_ttc)
print("arrondi :", round(total_ttc, 2))
print(type(quantite), type(prix_ht), type("bol"), type(True))
```

```
total HT : 37.5
total TTC : 44.625
arrondi : 44.62
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

Les totaux correspondent au calcul à la main, **sauf l'arrondi** : Python affiche 44.62 et non 44,63. La raison : 44,625 tombe pile à mi-chemin entre 44,62 et 44,63, et round arrondit ces cas vers le chiffre **pair** (« arrondi du banquier »), d'où 44,62. Dans d'autres cas, c'est l'approximation binaire des décimaux (1.5.1) qui fait pencher l'arrondi d'un côté ou de l'autre. Pour des centimes exacts, on utilise le module decimal ; pour un ticket de caisse, on peut aussi calculer en **millimes** (entiers). Gardez cet écart en tête : il illustre qu'un résultat de programme se **vérifie** toujours contre un calcul indépendant. Notez enfin que `type(...)` révèle le type d'une valeur : une commande que vous utiliserez souvent pour comprendre une erreur.

Les opérateurs arithmétiques sont + - * / et trois autres moins connus :

Opérateur	Sens	Exemple	Résultat
**	puissance	2 ** 10	1024
//	division entière	17 // 5	3
%	reste (modulo)	17 % 5	2

```
print(2 ** 10)
print(17 // 5, 17 % 5)      # 17 = 3*5 + 2
print(17 / 5)               # la division « / » donne toujours un float
print(7 % 2 == 0)           # un nombre est pair si son reste modulo 2 vaut 0
```

```
1024
3 2
3.4
False
```

Les décimaux sont approchés. Python (comme tous les langages) stocke les float en binaire, ce qui explique les petites surprises du type $0,1 + 0,2 \neq 0,3$ expliquées en 1.5.1. Consé-

quence pratique : on **n'écrit jamais** `a == b` pour comparer deux décimaux calculés, on utilise `math.isclose(a, b)`. Et pour des montants d'argent, on arrondit explicitement avec `round(x, 2)` à l'affichage.

4.1.3 Le texte et les f-strings

Un texte (`str`) s'écrit entre guillemets. On peut le découper, le mettre en majuscules, le chercher :

```
nom = " Bol en céramique bleue "
print(nom.strip())           # enlève les espaces au début et à la fin
print(nom.strip().upper())
print(nom.strip().replace("bleue", "verte"))
print(len(nom.strip()))      # nombre de caractères
print("céramique" in nom)    # True si le morceau est présent
print(nom.strip().split(" ")) # découpe en liste de mots
```

```
Bol en céramique bleue
BOL EN CÉRAMIQUE BLEUE
Bol en céramique verte
22
True
['Bol', 'en', 'céramique', 'bleue']
```

Pour **insérer des valeurs dans une phrase**, la méthode moderne est la **f-string** : on fait précéder le guillemet d'un `f` et on met les valeurs entre accolades. Après les deux-points, on peut régler le format.

```
produit, quantite, prix = "bol", 3, 12.5
print(f"{quantite} x {produit} à {prix} DT")
print(f"total : {quantite * prix:.2f} DT")      # .2f : 2 décimales
print(f"part de TVA : {0.19:.0%}")             # .0% : pourcentage
print(f"{'produit':<10}|{'prix':>8}")           # < aligne à gauche, > à droite
print(f"{'produit':<10}|{'prix':>8.2f}")
```

```
3 x bol à 12.5 DT
total : 37.50 DT
part de TVA : 19%
produit | prix
bol      | 12.50
```

Les f-strings sont la base de tous les rapports et tickets de caisse que vous écrirez.

4.1.4 Les collections : listes, tuples, dictionnaires, ensembles

Une variable ne contient pas forcément une seule valeur. Python offre quatre « boîtes » pour en regrouper plusieurs. On les étudie plus en détail à la section 4.3 (algorithmes et structures de données) ; voici l'essentiel.

Collection	Notation	Ordonnée ?	Modifiable ?	Doublons ?	Cas typique
liste	[1, 2, 3]	oui	oui	oui	une suite de montants
tuple	(1, 2, 3)	oui	non	oui	un couple (code, quantité)
dictionnaire	{"bol": 12.5}	par insertion	oui	clés uniques	un catalogue : nom → prix

Collection	Notation	Ordonnée ?	Modifiable ?	Doublons ?	Cas typique
ensemble	{1, 2, 3}	non	oui	non	les clients distincts

Les listes. Les positions (**indices**) commencent à 0. Un indice négatif compte depuis la fin. Le **découpage** `liste[a:b]` prend de l'indice a inclus à b exclu.

```
montants = [44.8, 34.5, 88.2, 30.1, 110.1]
print(montants[0], montants[-1])    # premier et dernier
print(montants[1:4])                # indices 1, 2, 3
montants.append(39.8)                # ajoute à la fin
print(len(montants), sum(montants), max(montants), min(montants))
print(sorted(montants))              # copie triée ; la liste d'origine ne change pas
print(f"moyenne = {sum(montants) / len(montants):.2f}")
```

```
44.8 110.1
[34.5, 88.2, 30.1]
6 347.5 110.1 30.1
[30.1, 34.5, 39.8, 44.8, 88.2, 110.1]
moyenne = 57.92
```

Piège classique du débutant : le décalage de 1. Dans une liste de 6 éléments, les indices vont de 0 à 5 ; `montants[6]` provoque une erreur. Et `montants[1:4]` contient 3 éléments (1, 2, 3), pas 4. Règle : la longueur d'un découpage est `b - a`.

Les tuples sont des listes figées : une fois créés on ne les modifie plus. Parfaits pour des paires qui ne doivent pas bouger, comme (code produit, quantité).

Les dictionnaires associent une **clé** à une **valeur**, comme un annuaire : on cherche par le nom, pas par la position.

```
catalogue = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0}
print(catalogue["tasse"])
catalogue["bougie"] = 15.9           # ajout
catalogue["bol"] = 13.0              # modification
print(catalogue)
print(catalogue.get("lampe", "inconnu")) # .get évite l'erreur si la clé n'existe pas
for nom, prix in catalogue.items():
    print(f" {nom:<8} {prix:>6.2f} DT")
```

```
8.0
{'bol': 13.0, 'tasse': 8.0, 'plateau': 45.0, 'bougie': 15.9}
inconnu
bol      13.00 DT
tasse    8.00 DT
plateau  45.00 DT
bougie   15.90 DT
```

Les ensembles oublient l'ordre et éliminent les doublons : exactement ce qu'il faut pour compter des clients **distincts**.

```
acheteurs = ["Sana", "Mehdi", "Sana", "Ines", "Mehdi", "Sana"]
distincts = set(acheteurs)
print(len(acheteurs), "achats par", len(distincts), "clients distincts")
print(sorted(distincts))             # trié pour un affichage stable
```

```
6 achats par 3 clients distincts
['Ines', 'Mehdi', 'Sana']
```

L'ordre d'affichage d'un ensemble peut changer d'une exécution à l'autre : n'y comptez jamais (c'est pourquoi nous l'avons passé par `sorted` pour l'afficher).

4.1.5 Décider : conditions

Un programme doit pouvoir **choisir**. L'instruction `if` exécute un bloc seulement si la condition est vraie. En Python, l'**indentation** (4 espaces) délimite le bloc : c'est la grammaire du langage, pas un détail de présentation.

Comparaisons : `==` (égal), `!=` (différent), `<`, `<=`, `>`, `>=`. Combinaisons : `and`, `or`, `not`.

Exemple à la main. Règle de livraison de Dar Jasmin : *gratuite à partir de 100 DT, sinon 7 DT ; et pour un retrait en boutique, toujours 0*. Pour un panier de 80 DT livré : 7 DT. Pour 120 DT livré : 0. Pour 80 DT en retrait : 0.

```
def frais_livraison(total, retrait_boutique):
    if retrait_boutique:
        return 0
    elif total >= 100:
        return 0
    else:
        return 7

print(frais_livraison(80, False), frais_livraison(120, False), frais_livraison(80, True))
```

```
7 0 0
```

Les trois cas retombent sur les valeurs trouvées à la main. (Nous n'avons pas encore parlé des **fonctions** : le mot `def` en 4.1.7 va tout expliquer.)

= et ==. Un seul signe égal **colle** une étiquette ; deux signes égaux **testent** l'égalité. Python refuse d'ailleurs `if x = 3`, ce qui vous évite un grand classique des autres langages.

4.1.6 Répéter : boucles

Une **boucle** répète un bloc. Deux formes :

- `for élément in collection` : une fois pour chaque élément (on sait combien).
- `while condition` : tant que la condition est vraie (on ne sait pas combien).

Exemple à la main. Yasmine place 1 000 DT à 5 % par an, intérêts composés. Au bout d'un an : $1000 \times 1,05 = 1050$. Deux ans : $1050 \times 1,05 = 1102,5$. Combien d'années pour **doubler** ? C'est une question « jusqu'à ce que » : une boucle `while`.

```
capital, annees = 1000.0, 0
while capital < 2000:
    capital = capital * 1.05
    annees += 1
    # raccourci pour annees = annees + 1
    print(f"année {annees:2d} : {capital:8.2f} DT")
print("doublé en", annees, "ans")
```

```
année 1 : 1050.00 DT
année 2 : 1102.50 DT
année 3 : 1157.62 DT
année 4 : 1215.51 DT
année 5 : 1276.28 DT
année 6 : 1340.10 DT
année 7 : 1407.10 DT
année 8 : 1477.46 DT
année 9 : 1551.33 DT
```

```

année 10 : 1628.89 DT
année 11 : 1710.34 DT
année 12 : 1795.86 DT
année 13 : 1885.65 DT
année 14 : 1979.93 DT
année 15 : 2078.93 DT
doublé en 15 ans

```

Vérification mathématique : on cherche le plus petit n tel que $1,05^n \geq 2$, soit $n \geq \ln 2 / \ln 1,05$.

```

import math
print(round(math.log(2) / math.log(1.05), 2))

```

14.21

Le résultat réel est entre 14 et 15 : il faut donc 15 années entières, ce que la boucle a trouvé.

La boucle infinie. Si, dans un `while`, la condition ne devient jamais fausse (ici, si on oubliait la ligne `capital = ...`), le programme tourne éternellement. Dans un terminal, `Ctrl+C` l'arrête. Avant de lancer un `while`, demandez-vous : *qu'est-ce qui le fera s'arrêter ?* (La même question sera posée avec rigueur pour les algorithmes en 4.3.)

La boucle `for` parcourt n'importe quelle collection ; `range(n)` produit les entiers de 0 à $n - 1$, et `enumerate` donne en plus la position :

```

for i in range(3):
    print("i =", i)
for rang, nom in enumerate(["Sana", "Mehdi", "Ines"], start=1):
    print(rang, nom)

```

```

i = 0
i = 1
i = 2
1 Sana
2 Mehdi
3 Ines

```

Les compréhensions de liste sont une écriture compacte d'une boucle qui construit une liste : `[expression for x in collection if condition]`. Elles se lisent comme une phrase mathématique « l'ensemble des $f(x)$ pour x dans... tel que... ».

```

montants = [44.8, 34.5, 88.2, 30.1, 110.1, 39.8, 74.7]
ttc = [round(m * 1.19, 2) for m in montants]
gros = [m for m in montants if m > 60]
print(ttc)
print(gros)
carres = {n: n ** 2 for n in range(1, 6)}      # même idée pour un dictionnaire
print(carres)

```

```

[53.31, 41.05, 104.96, 35.82, 131.02, 47.36, 88.89]
[88.2, 110.1, 74.7]
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

```

4.1.7 Fonctions : écrire ses propres outils

Une **fonction** est une recette nommée qu'on peut réutiliser. Elle reçoit des **paramètres**, fait un calcul, et **renvoie** un résultat avec `return`. Sans fonctions, un programme devient vite un long ruban illisible ; avec elles, on le découpe en briques que l'on teste séparément.

Mathématiquement, c'est la même idée que $f(x) = 1,19x$: on la définit une fois, on l'utilise partout.

```
def ttc(prix_ht, taux_tva=0.19):
    """Renvoie le prix TTC arrondi au centime."""
    return round(prix_ht * (1 + taux_tva), 2)

print(ttc(100))          # utilise la TVA par défaut : 19 %
print(ttc(100, 0.07))    # TVA à 7 % : on remplace la valeur par défaut
print(ttc(prix_ht=50))   # appel par nom : plus lisible
```

```
119.0
107.0
59.5
```

Trois choses à retenir : (1) la ligne entre triples guillemets, la **docstring**, documente la fonction ; (2) `taux_tva=0.19` est un paramètre **par défaut** ; (3) une fonction peut renvoyer **plusieurs** valeurs, sous forme de tuple.

```
def resume(valeurs):
    """Renvoie (minimum, moyenne, maximum)."""
    return min(valeurs), sum(valeurs) / len(valeurs), max(valeurs)

mini, moy, maxi = resume([44.8, 34.5, 88.2, 30.1, 110.1])
print(f"min = {mini}, moyenne = {moy:.2f}, max = {maxi}")
```

```
min = 30.1, moyenne = 61.54, max = 110.1
```

La portée. Les variables créées *dans* une fonction n'existent que dans la fonction (on dit qu'elles sont **locales**). Cela évite les mélanges : deux fonctions peuvent utiliser chacune une variable `total` sans se gêner.

Piège : le paramètre par défaut modifiable. N'écrivez jamais `def f(x, liste=[])` : cette liste est créée **une seule fois** et partagée par tous les appels. Écrivez `liste=None` puis `if liste is None: liste = []`.

Les fonctions peuvent aussi être **passées** à d'autres fonctions, ce qui donne des écritures très concises. Exemple : trier des commandes selon un critère choisi avec l'argument `key`.

```
commandes = [("Sana", 44.8), ("Mehdi", 110.1), ("Ines", 30.1)]
print(sorted(commandes, key=lambda c: c[1]))          # du plus petit au plus gros montant
print(sorted(commandes, key=lambda c: c[1], reverse=True)[0]) # la plus grosse
```

```
[('Ines', 30.1), ('Sana', 44.8), ('Mehdi', 110.1)]
('Mehdi', 110.1)
```

`lambda c: c[1]` est une mini-fonction sans nom qui renvoie le second élément du couple.

4.1.8 Les erreurs : vos meilleures amies

Quand quelque chose ne va pas, Python s'arrête et affiche un **message d'erreur** (*traceback*). Il se lit **de bas en haut** : la dernière ligne dit *quel type d'erreur* et *pourquoi* ; les lignes au-dessus disent *où*. Voici les erreurs que vous rencontrerez le plus souvent, provoquées volontairement et rattrapées avec `try / except` pour pouvoir les afficher :

```
def tenter(description, fonction):
    try:
        fonction()
```



```

except Exception as e:
    print(f"{description:<26} -> {type(e).__name__}: {e}")

montants = [44.8, 34.5, 88.2]
tenter("indice hors liste", lambda: montants[5])
tenter("clé absente", lambda: {"bol": 12.5}["lampe"])
tenter("division par zéro", lambda: 10 / 0)
tenter("texte + nombre", lambda: "total : " + 12.5)
tenter("texte vers entier", lambda: int("douze"))
tenter("nom inconnu", lambda: variable_inexistante)

indice hors liste      -> IndexError: list index out of range
clé absente            -> KeyError: 'lampe'
division par zéro      -> ZeroDivisionError: division by zero
texte + nombre         -> TypeError: can only concatenate str (not "float") to str
texte vers entier      -> ValueError: invalid literal for int() with base 10: 'douze'
nom inconnu            -> NameError: name 'variable_inexistante' is not defined

```

Lecture :

- **IndexError** : vous demandez un indice qui n'existe pas (rappelez-vous le décalage de 1).
- **KeyError** : la clé n'est pas dans le dictionnaire (pensez à `.get`).
- **ZeroDivisionError** : on ne divise pas par zéro, ni en Python ni ailleurs.
- **TypeError** : on a mélangé des types incompatibles (texte et nombre).
- **ValueError** : le bon type, mais une valeur impossible à convertir.
- **NameError** : le nom n'existe pas (faute de frappe ? variable pas encore créée ?).

Attraper une erreur à bon escient. Quand on lit des données saisies par des humains, certaines valeurs sont inexploitables. On préfère alors **prévoir** l'erreur plutôt que de planter :

```

saisies = ["12.5", "8", "abc", "", "15,9", "45.0"]
valides, rejetees = [], []
for s in saisies:
    try:
        valides.append(float(s))
    except ValueError:
        rejetees.append(s)
print("valides :", valides)
print("rejetées :", rejetees)

```

```

valides : [12.5, 8.0, 45.0]
rejetées : ['abc', '', '15,9']

```

Remarquez que "15,9" (virgule française) est rejeté : Python attend le **point** décimal. C'est une cause fréquente de données « cassées » quand elles viennent d'un tableur configuré en français.

Méthode pour déboguer (à épingler au-dessus de votre écran). (1) Lisez la **dernière ligne** du message. (2) Repérez la ligne de code citée. (3) Affichez avec `print` les valeurs et les types utilisés à cet endroit. (4) Réduisez le problème au plus petit exemple qui échoue. (5) Seulement ensuite, cherchez le message sur Internet. Neuf fois sur dix, les étapes 1 à 3 suffisent.

4.1.9 Modules, fichiers et données réelles

Python ne contient pas tout, mais il sait **importer** du code déjà écrit. Un **module** est un fichier de fonctions ; la bibliothèque standard en fournit des dizaines (`math`, `random`, `statistics`, `csv`, `datetime`...), et on en installe d'autres avec `pip` (section 6.3).

```
import math
import statistics
from collections import Counter

print(math.sqrt(144), math.pi)
print(statistics.mean([2, 4, 4, 4, 5, 5, 7, 9]), statistics.pstdev([2, 4, 4, 4, 5, 5, 7, 9]))
print(Counter("abracadabra"))      # compte les occurrences

12.0 3.141592653589793
5 2.0
Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
```

Lisons maintenant le fichier de données du livre, `donnees/commandes.csv`, **sans aucune bibliothèque externe**, avec le module `csv`. Un fichier CSV est du texte brut : une ligne par commande, des valeurs séparées par des virgules, la première ligne contenant les noms des colonnes.

```
import csv
from collections import Counter

with open("donnees/commandes.csv", encoding="utf-8") as f:
    lignes = list(csv.DictReader(f))      # chaque ligne devient un dictionnaire

print("nombre de commandes :", len(lignes))
print("première commande   :", lignes[0])
print("type du montant     :", type(lignes[0]["montant"]))

nombre de commandes : 400
première commande   : {'canal': 'Boutique', 'montant': '44.8', 'livraison': '0', 'satisfaction': '4'}
type du montant     : <class 'str'>
```

Le mot-clé `with` ouvre le fichier **et le referme proprement** à la sortie du bloc, même en cas d'erreur. Remarquez que les valeurs sont lues comme du **texte** : "44.8" n'est pas un nombre ! Il faut convertir.

```
montants = [float(l["montant"]) for l in lignes]
canaux = Counter(l["canal"] for l in lignes)
print("montant moyen :", round(sum(montants) / len(montants), 2))
print("répartition   :", dict(canaux))

# montant moyen par canal, avec un dictionnaire de listes
par_canal = {}
for l in lignes:
    par_canal.setdefault(l["canal"], []).append(float(l["montant"]))
for canal, valeurs in par_canal.items():
    print(f" {canal:<10} n = {len(valeurs):3d}   moyenne = {sum(valeurs) / len(valeurs):6.2f} DT")

montant moyen : 60.25
répartition   : {'Boutique': 114, 'Site': 148, 'Instagram': 138}
  Boutique   n = 114   moyenne = 74.81 DT
    Site     n = 148   moyenne = 59.50 DT
  Instagram  n = 138   moyenne = 49.01 DT
```

On retrouve le montant moyen de 60,25 DT calculé au chapitre 3. Nous avons tout fait à la main, avec des boucles et des dictionnaires : c'est précisément le travail que `pandas` fera en **une ligne** à la section 4.4. Savoir le faire « à la main » vous permet de comprendre ce que `pandas` fait pour vous.

4.1.10 Un programme complet : la caisse de la boutique

Rassemblons tout. Yasmine veut un petit programme qui, pour un panier, **édite un ticket de caisse** avec les règles suivantes :

- les prix du catalogue sont **hors taxe** ;
- une **remise fidélité de 10 %** s'applique si le sous-total HT dépasse 100 DT ;
- la **TVA de 19 %** s'applique sur le montant après remise ;
- le ticket affiche chaque ligne, le sous-total, la remise, la TVA et le total TTC.

Calcul à la main pour le panier « 2 bols, 1 plateau, 3 bougies » (prix HT : bol 12,5 ; plateau 45 ; bougie 15,9) :

- bols : $2 \times 12,5 = 25,00$; plateau : 45,00 ; bougies : $3 \times 15,9 = 47,70$;
- sous-total HT : $25 + 45 + 47,7 = 117,70$ DT ;
- le sous-total dépasse 100 DT, donc remise de 10% : 11,77 DT, soit 105,93 DT après remise ;
- TVA : $105,93 \times 0,19 = 20,1267 \approx 20,13$ DT ;
- total TTC : $105,93 + 20,13 = 126,06$ DT.

Le programme, découpé en petites fonctions faciles à tester :

```
CATALOGUE = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0, "bougie": 15.9}
TVA = 0.19
SEUIL_REMISE = 100
TAUX_REMISE = 0.10

def sous_total(panier):
    """panier : liste de couples (nom du produit, quantité)."""
    return sum(CATALOGUE[nom] * qte for nom, qte in panier)

def remise(montant_ht):
    return montant_ht * TAUX_REMISE if montant_ht > SEUIL_REMISE else 0.0

def ticket(panier):
    ht = sous_total(panier)
    r = remise(ht)
    net = ht - r
    tva = net * TVA
    lignes = ["=== DAR JASMIN ==="]
    for nom, qte in panier:
        lignes.append(f"{qte} x {nom:<8} {CATALOGUE[nom]:>6.2f} {qte * CATALOGUE[nom]:>8.2f}")
    lignes.append(f"{'Sous-total HT':<20}{ht:>8.2f}")
    lignes.append(f"{'Remise fidélité':<20}{-r:>8.2f}")
    lignes.append(f"{'TVA 19 %':<20}{tva:>8.2f}")
    lignes.append(f"{'TOTAL TTC':<20}{net + tva:>8.2f}")
    return "\n".join(lignes), round(net + tva, 2)

texte, total = ticket([("bol", 2), ("plateau", 1), ("bougie", 3)])
print(texte)
```

```
=== DAR JASMIN ===
2 x bol      12.50    25.00
1 x plateau  45.00    45.00
3 x bougie   15.90    47.70
Sous-total HT      117.70
Remise fidélité    -11.77
TVA 19 %           20.13
TOTAL TTC          126.06
```

Le ticket affiche 117,70 DT de sous-total, 11,77 de remise, 20,13 de TVA et 126,06 DT au total : **exactement** nos valeurs à la main. Testons aussi les cas limites avec `assert`, une instruction qui ne dit rien quand la condition est vraie et **arrête** le programme avec une erreur quand elle est fausse :

```
# Cas 1 : petit panier, pas de remise. 2 tasses = 16,00 HT ; TVA = 3,04 ; TTC = 19,04
assert ticket([("tasse", 2)])[1] == 19.04
# Cas 2 : panier vide
assert ticket([])[1] == 0.0
# Cas 3 : juste au seuil (100 DT pile) : la remise ne s'applique PAS (condition « > »)
assert remise(100) == 0.0
# Cas 4 : le panier précédent
assert ticket([("bol", 2), ("plateau", 1), ("bougie", 3)])[1] == 126.06
print("tous les tests passent")
```

tous les tests passent

Ce qu'on vient de faire, c'est de la rigueur. Calculer à la main *avant* de coder fournit un « oracle » : si le programme et la main divergent, l'un des deux a tort, et on cherche lequel. Les `assert` transforment cette vérification en filet de sécurité automatique. Nous irons plus loin avec de vrais tests unitaires en 4.6.

Pour aller plus loin, essayez de modifier le programme. Que se passe-t-il si on demande un produit absent du catalogue ? (Réponse : une `KeyError`, comme en 4.1.8.) Comment afficher un message gentil plutôt qu'un plantage ? Comment ajouter un code promo ? Ces trois questions sont des exercices de la section 4.9.

À retenir

- Une **variable** est une étiquette sur une valeur ; les types de base sont `int`, `float`, `str`, `bool`.
- Les décimaux sont approchés : on les arrondit à l'affichage et on ne les compare pas avec `==`.
- **Listes** (ordonnées, modifiables), **tuples** (figés), **dictionnaires** (clé → valeur), **ensembles** (sans doublons). Les indices commencent à 0.
- `if/elif/else` pour décider, `for` et `while` pour répéter ; l'**indentation** délimite les blocs.
- Une **fonction** (`def`, `return`) est une brique réutilisable ; on la documente (`docstring`) et on la teste.
- Un message d'erreur se lit **par la dernière ligne** ; `try/except` permet de prévoir les erreurs attendues.
- Les fichiers CSV sont du texte : il faut convertir les nombres avant de calculer.
- Méthode d'or : **calculer à la main un petit cas, puis vérifier que le programme donne la même chose.**

4.2 Les fondamentaux de R

Intuition. Si Python est un couteau suisse qui sait tout faire, **R est le scalpel du statisticien** : il a été conçu dès l'origine (années 1990, par des statisticiens) pour manipuler des tableaux de données, calculer des statistiques et tracer des graphiques. Les tests, les modèles et les méthodes de recherche récentes y arrivent souvent *en premier*. Dans l'industrie comme à l'université, vous croiserez du R : savoir le lire est un vrai atout.

Cette section ne vous demande pas de choisir un camp. Nous allons **refaire dans R les analyses des sections 3.1, 3.3 et 3.4 et du 4.1**, sur le même fichier `donnees/commmandes.csv`. Retrouver les mêmes nombres avec un autre langage est la meilleure des vérifications : si deux outils indépendants donnent la même réponse, vous pouvez avoir confiance.

Section à lire après 4.1. Nous supposons connues les notions de variable, fonction, boucle et condition vues en 4.1 ; ici on ne s'attarde que sur ce qui **change** en R.

4.2.1 Lancer R, et premières différences

On lance R avec la commande R (session interactive) ou Rscript mon_script.R (pour exécuter un fichier). Le confort d'un éditeur s'obtient avec **RStudio** ou VS Code. Tous les blocs R de ce livre ont été **réellement exécutés** avec R, et leur sortie est affichée juste en dessous, comme pour Python.

Voici un premier contact, à comparer avec Python :

```
version$version.string
x <- 12.5           # l'affectation s'écrit « <- » (le « = » marche aussi, mais on utilise « <- »)
x * 3
print("Bonjour Dar Jasmin !")
```

```
[1] "R version 4.4.3 (2025-02-28)"
[1] 37.5
[1] "Bonjour Dar Jasmin !"
```

Les trois différences à connaître tout de suite :

	Python	R
Affectation	x = 12.5	x <- 12.5
Premier indice d'une liste	0	1
Fin de ligne	indentation	accolades { }

En R, on compte à partir de 1. x[1] est le **premier** élément (en Python : x[0]). Et le découpage x[2:4] donne les éléments 2, 3 et 4 : les deux bornes sont **incluses** (en Python, la borne de droite est exclue). C'est la source n°1 d'erreurs quand on passe d'un langage à l'autre.

4.2.2 Vecteurs : la brique de base

En R, il n'existe pas de « nombre seul » : 12.5 est un **vecteur de longueur 1**. La fonction c() (*combine*) assemble un vecteur, et **toutes les opérations s'appliquent élément par élément**, sans boucle. Cette idée de **vectorisation** est le cœur de R (et, nous le verrons, de NumPy en 4.4).

Exemple à la main. Quatre commandes de 44,8 ; 34,5 ; 88,2 et 30,1 DT. Leur somme est 197,6, leur moyenne 49,4 DT. Avec 19 % de TVA, chaque montant est multiplié par 1,19 : 44,8 → 53,31 (arrondi), etc.

```
montants <- c(44.8, 34.5, 88.2, 30.1)
sum(montants)
mean(montants)
montants * 1.19           # la multiplication s'applique à chaque élément
round(montants * 1.19, 2)
montants[1]               # le PREMIER élément
montants[2:3]             # éléments 2 et 3 (bornes incluses)
montants[-1]              # indice négatif : tout SAUF le premier
montants[montants > 40]   # sélection par condition
```

```
[1] 197.6
[1] 49.4
[1] 53.312 41.055 104.958 35.819
[1] 53.31 41.06 104.96 35.82
```

```
[1] 44.8
[1] 34.5 88.2
[1] 34.5 88.2 30.1
[1] 44.8 88.2
```

Les valeurs 197,6 et 49,4 retombent sur le calcul à la main. Remarquez les deux emplois de `[ ]` :

- `montants[-1]` signifie « **tout sauf** le premier » (en Python, -1 désignait le **dernier** !);
- `montants[montants > 40]` : on donne au crochet un vecteur de vrai/faux et R ne garde que les TRUE. Ce « **filtrage logique** » est l'équivalent exact de la compréhension de liste de 4.1.6 ; on le retrouvera avec pandas.

Les autres types de base ressemblent à ceux de Python : `numeric` (décimaux), `integer`, `character` (texte), `logical` (TRUE/FALSE). Une particularité précieuse : R a une **valeur manquante intégrée**, `NA` (*not available*), qui contamine les calculs tant qu'on ne demande pas explicitement de l'ignorer :

```
ventes <- c(12, 15, NA, 9)
mean(ventes)                # une valeur manquante rend la moyenne inconnue
mean(ventes, na.rm = TRUE)  # on demande d'ignorer les NA
sum(is.na(ventes))          # compter les valeurs manquantes
```

```
[1] NA
[1] 12
[1] 1
```

Pourquoi NA contamine-t-il ? Si une valeur est inconnue, la moyenne l'est aussi : R refuse de faire comme si de rien n'était. C'est un comportement *honnête*, qui évite d'oublier des données manquantes sans s'en rendre compte. Nous verrons au volume suivant comment les traiter avec soin.

4.2.3 Fonctions, conditions et boucles

La syntaxe change, pas les idées :

```
ttc <- function(prix_ht, taux_tva = 0.19) {
  round(prix_ht * (1 + taux_tva), 2)  # la dernière valeur calculée est renvoyée
}
ttc(100)
ttc(100, 0.07)
ttc(c(12.5, 45, 15.9))               # la fonction marche sur un vecteur entier, gratuitement
```

```
[1] 119
[1] 107
[1] 14.88 53.55 18.92
```

La dernière ligne est remarquable : parce que `*` et `round` sont vectorisés, notre fonction `ttc`, écrite pour **un** prix, marche telle quelle sur **une liste** de prix. En Python pur, il aurait fallu une boucle ou une compréhension.

Les conditions et les boucles :

```
frais_livraison <- function(total, retrait_boutique) {
  if (retrait_boutique) {
    0
  } else if (total >= 100) {
    0
  } else {
    7
  }
}
```

```

}
c(frais_livraison(80, FALSE), frais_livraison(120, FALSE), frais_livraison(80, TRUE))

capital <- 1000; annees <- 0
while (capital < 2000) {
  capital <- capital * 1.05
  annees <- annees + 1
}
annees

# version vectorisée de « si... alors... sinon » : ifelse
totaux <- c(80, 120, 100, 35)
ifelse(totaux >= 100, 0, 7)

[1] 7 0 0
[1] 15
[1] 7 0 0 7

```

On retrouve 7, 0, 0 pour les frais, et 15 années pour doubler un capital placé à 5 % : exactement ce que Python a donné en 4.1. La fonction `ifelse` applique le test à **chaque élément** d'un vecteur, ce que `if` ne sait pas faire.

Piège de la vectorisation. `if (totaux >= 100)` appliqué à un vecteur de plusieurs éléments n'a pas de sens : `if` attend **un seul** vrai/faux. Utilisez `ifelse` pour les vecteurs.

La caisse de la boutique, en R. Reprenons le calcul du sous-total du ticket (4.1.10) avec un **vecteur nommé**, qui joue le rôle du dictionnaire :

```

catalogue <- c(bol = 12.5, tasse = 8.0, plateau = 45.0, bougie = 15.9)
panier <- c(bol = 2, plateau = 1, bougie = 3)

catalogue[names(panier)] # les prix des produits du panier
sous_total <- sum(catalogue[names(panier)] * panier)
remise <- if (sous_total > 100) 0.10 * sous_total else 0
net <- sous_total - remise
c(HT = sous_total, remise = remise, TVA = 0.19 * net, TTC = 1.19 * net)

```

```

  bol plateau bougie
12.5   45.0   15.9
   HT  remise   TVA   TTC
117.7000 11.7700 20.1267 126.0567

```

Nous retrouvons un sous-total de 117,70 DT, une remise de 11,77 DT et un total TTC de 126,06 DT (aux arrondis près) : le même ticket qu'en Python, en quatre lignes grâce à la vectorisation.

4.2.4 Le data.frame : le tableau de données

Le tableau est l'**objet central de R**. Un `data.frame` est un tableau dont chaque colonne est un vecteur (de types éventuellement différents). Chargeons les commandes, avec les trois gestes de 3.1.2 : forme, types, résumé.

```

df <- read.csv("donnees/commandes.csv") # lit le CSV directement en tableau
dim(df) # lignes, colonnes
str(df) # structure : type de chaque colonne
head(df, 4)

```

```

[1] 400 4
'data.frame': 400 obs. of 4 variables:
 $ canal : chr "Boutique" "Site" "Instagram" "Instagram" ...
 $ montant : num 44.8 34.5 88.2 30.1 110.1 ...

```

```
$ livraison : int 0 2 5 4 0 5 0 0 5 6 ...
$ satisfaction: int 4 4 4 4 5 3 5 4 4 3 ...
  canal montant livraison satisfaction
1 Boutique 44.8 0 4
2 Site 34.5 2 4
3 Instagram 88.2 5 4
4 Instagram 30.1 4 4
```

`read.csv` a deviné les types : `canal` est du texte (`chr`), les trois autres colonnes sont numériques. Contrairement au module `csv` de Python (4.1.9) qui lisait tout en texte, la conversion est faite pour nous. Le résumé numérique :

```
summary(df)
colSums(is.na(df)) # valeurs manquantes par colonne
```

canal	montant	livraison	satisfaction
Length:400	Min. : 8.60	Min. : 0.000	Min. :1.000
Class :character	1st Qu.: 34.17	1st Qu.: 0.000	1st Qu.:3.000
Mode :character	Median : 51.00	Median : 3.000	Median :4.000
	Mean : 60.25	Mean : 3.288	Mean :3.965
	3rd Qu.: 75.83	3rd Qu.: 5.000	3rd Qu.:5.000
	Max. :255.70	Max. :13.000	Max. :5.000
canal	montant	livraison	satisfaction
0	0	0	0

La ligne `montant` donne un minimum de 8,6, une médiane de 51, une moyenne de 60,25 et un maximum de 255,7 : **les mêmes chiffres** que le `describe()` de pandas au 3.1.2. On accède à une colonne avec `$` :

```
m <- df$montant
c(moyenne = mean(m), mediane = median(m), ecart_type = sd(m), iqr = IQR(m))
quantile(m, c(0.05, 0.25, 0.50, 0.75, 0.95))
```

moyenne	mediane	ecart_type	iqr
60.24575	51.00000	38.01791	41.65000
5%	25%	50%	75%
18.985	34.175	51.000	75.825
			128.555

Les nombres sont ceux du 3.1 : moyenne 60,25 ; médiane 51 ; écart-type 38,02 (R divise par $n - 1$, comme pandas) ; quartiles 34,2 et 75,8. Les deux logiciels utilisent la même définition par défaut du quantile (interpolation linéaire), ce qui explique la concordance exacte. Pour résumer **par groupe** :

```
tapply(df$montant, df$canal, mean) # moyenne par canal
table(df$canal) # effectifs par canal
aggregate(montant ~ canal, data = df, FUN = function(v) c(n = length(v), moyenne = mean(v), sd = sd(v)))
```

Boutique	Instagram	Site	
74.80965	49.01087	59.50338	
Boutique	Instagram	Site	
114	138	148	
canal	montant.n	montant.moyenne	montant.sd
1 Boutique	114.00000	74.80965	40.64683
2 Instagram	138.00000	49.01087	31.08370
3 Site	148.00000	59.50338	38.32863

La formule `montant ~ canal` se lit « le montant **en fonction du** canal » : cette notation en tilde est partout en R (nous la retrouverons pour les modèles de régression au volume suivant). Les moyennes par canal (74,81 ; 59,50 ; 49,01) sont celles que nous avons obtenues avec Python au

4.1.9, à la main.

4.2.5 Les statistiques « sortent de la boîte »

C'est ici que R brille : les procédures de la statistique classique sont **au catalogue de base**, sans rien installer.

Intervalle de confiance et test de Welch. Rappel du 3.3 et du 3.4 : IC à 95 % de la moyenne, [56,51 ; 63,98] ; test boutique contre Instagram, $t = 5,565, 208,4$ degrés de liberté, $p \approx 8 \times 10^{-8}$.

```
test_moyenne <- t.test(df$montant)           # IC de la moyenne (test contre 0 sans intérêt, mais l'IC
  <- est utile)
round(test_moyenne$conf.int, 2)

b <- df$montant[df$canal == "Boutique"]
i <- df$montant[df$canal == "Instagram"]
w <- t.test(b, i)                           # R fait le test de Welch par défaut
c(t = unname(w$statistic), ddl = unname(w$parameter), p = w$p.value)
round(w$conf.int, 1)                       # IC95 % de la différence des moyennes
```

```
[1] 56.51 63.98
attr(,"conf.level")
[1] 0.95
      t      ddl      p
5.564672e+00 2.084304e+02 8.016366e-08
[1] 16.7 34.9
attr(,"conf.level")
[1] 0.95
```

On lit exactement $t = 5,565, 208,4$ degrés de liberté et une p-valeur $p \approx 8 \times 10^{-8}$: les valeurs obtenues avec scipy au 3.4.3. Un seul appel de fonction fait ce que nous avons codé en plusieurs lignes à la main.

R fait le test de Welch par défaut, ce qui est bien la bonne pratique recommandée en 3.4.3 (en Python, il faut penser à écrire `equal_var=False`). Chaque langage a ses défauts : **lisez la documentation** (`?t.test` en R, `help()` en Python) pour savoir ce que fait réellement une fonction.

Corrélation et régression, un avant-goût. Les mêmes outils donnent la corrélation entre délai de livraison et satisfaction, et une droite de régression, que le volume II étudiera en détail :

```
livres <- subset(df, canal != "Boutique")      # on exclut les retraits en boutique (délai = 0)
cor(livres$livraison, livres$satisfaction)
modele <- lm(satisfaction ~ livraison, data = livres)
round(coef(modele), 3)
```

```
[1] -0.4069396
(Intercept)  livraison
      4.581      -0.180
```

La corrélation vaut environ $-0,41$: elle est **négative**, plus le délai est long, moins les clients sont satisfaits. La pente estimée, $-0,18$, est la perte de satisfaction **par jour de retard supplémentaire**. Elle coïncide avec le coefficient $-0,18$ qui a servi à fabriquer les données au 3.1.2 (et l'ordonnée à l'origine, 4,58, est proche du 4,6 utilisé) : un joli moyen de vérifier que la machine retrouve bien ce qu'on y a mis.

4.2.6 Le tidyverse : manipuler les tableaux avec des « phrases »

Les fonctions de base de R sont puissantes mais leur syntaxe est parfois irrégulière. Un ensemble de packages cohérents, le **tidyverse** (dplyr, tidyr, readr, ggplot2...), a standardisé une façon de travailler : des **verbes** simples enchaînés par un **tube** (`|>`, qui se lit « puis »). Chaque verbe prend un tableau et rend un tableau.

Verbe	Rôle	Équivalent SQL (chapitre 5)
<code>filter()</code>	garder des lignes	WHERE
<code>select()</code>	garder des colonnes	SELECT
<code>mutate()</code>	créer ou modifier une colonne	colonne calculée
<code>arrange()</code>	trier	ORDER BY
<code>group_by()</code> + <code>summarise()</code>	résumer par groupe	GROUP BY

```
suppressPackageStartupMessages(library(dplyr))
```

```
df |>
  group_by(canal) |>
  summarise(n = n(),
            panier_moyen = round(mean(montant), 2),
            ecart_type = round(sd(montant), 2),
            satisfaction = round(mean(satisfaction), 2)) |>
  arrange(desc(panier_moyen))
```

```
# A tibble: 3 × 5
  canal      n panier_moyen ecart_type satisfaction
<chr> <int>      <dbl>      <dbl>      <dbl>
1 Boutique  114        74.8        40.6        4.49
2 Site      148        59.5        38.3        3.79
3 Instagram 138        49.0        31.1        3.72
```

Lisez cette « phrase » à voix haute : *prends le tableau des commandes, puis groupe par canal, puis résume (effectif, panier moyen, écart-type, satisfaction), puis trie par panier moyen décroissant*. C'est presque du français, et c'est ce qui rend le tidyverse si agréable à lire. Une seconde phrase, avec filtre et colonne calculée :

```
df |>
  filter(canal != "Boutique", livraison > 8) |>      # livraisons lentes
  mutate(montant_ttc = round(montant * 1.19, 2)) |>
  select(canal, montant, montant_ttc, livraison, satisfaction) |>
  arrange(desc(livraison)) |>
  head(5)
```

```
   canal  montant  montant_ttc  livraison  satisfaction
1   Site    40.1     47.72         13           2
2   Site    65.4     77.83         10           3
3   Site    98.1    116.74         10           1
4 Instagram 61.1     72.71          9           4
5   Site   105.1    125.07          9           3
```

Nous retrouverons exactement cette logique, en Python, avec **pandas** (section 4.4), puis en SQL au chapitre 5. Les trois langages expriment les mêmes idées : *filtrer, sélectionner, créer, trier, regrouper*. Apprendre l'un fait gagner du temps sur les deux autres.

Et les graphiques ? R possède aussi une bibliothèque de graphiques célèbre, **ggplot2**. Nous la présentons à la section 4.5, côte à côte avec matplotlib et seaborn.

4.2.7 Python ou R ? Un tableau de correspondance

Tâche	Python (pandas)	R
Lire un CSV	<code>pd.read_csv("f.csv")</code>	<code>read.csv("f.csv")</code>
Moyenne d'une colonne	<code>df["montant"].mean()</code>	<code>mean(df\$montant)</code>
Filtrer des lignes	<code>df[df["canal"] == "Site"]</code>	<code>df[df\$canal == "Site",]</code> ou <code>filter(df, canal == "Site")</code>
Moyenne par groupe	<code>df.groupby("canal")["montant"].mean()</code>	<code>tapply(df\$montant, df\$canal, mean)</code>
Test de Welch	<code>stats.ttest_ind(a, b, equal_var=False)</code>	<code>t.test(a, b)</code>
Premier élément	<code>x[0]</code>	<code>x[1]</code>
Valeur manquante	NaN / None	NA

Comment choisir ? Python est un langage généraliste : il excelle dès qu'il faut **mettre en production**, automatiser, faire de l'apprentissage automatique (volume III) ou du traitement de texte. R excelle pour **l'analyse statistique exploratoire et les rapports**. Beaucoup de professionnels utilisent les deux. La stratégie de ce livre : **Python comme langage principal**, R en parallèle quand il éclaire un concept ou qu'il est l'outil naturel.

À retenir

- R est un langage pensé pour les statistiques ; les opérations y sont **vectorisées** (élément par élément, sans boucle).
- Attention aux différences de Python : on **compte à partir de 1**, les bornes d'un découpage sont **incluses**, `x[-1]` signifie « tout sauf le premier » et `<-` est l'affectation.
- Le `data.frame` est l'objet central ; `read.csv`, `summary`, `tapply`, `aggregate` couvrent l'essentiel de l'exploration.
- NA représente une valeur manquante et **contamine** les calculs sauf si l'on écrit `na.rm = TRUE`.
- Les tests classiques (`t.test`, `cor`, `lm`) sont fournis d'origine ; `t.test` fait le test de **Welch** par défaut.
- Le tidyverse (`filter`, `mutate`, `group_by`, `summarise`) enchaîne des « verbes » avec `|>`, comme pandas et SQL.
- Retrouver les mêmes nombres dans deux langages indépendants (moyenne 60,25 ; $t = 5,565$) est la meilleure des vérifications.

4.3 Algorithmes et structures de données

Intuition. Un **algorithme** est une méthode précise pour résoudre un problème, comme une recette. Une **structure de données** est la façon dont on range les ingrédients. Deux cuisiniers peuvent faire le même plat, l'un en dix minutes parce que ses ingrédients sont bien rangés, l'autre en deux heures parce qu'il fouille dans tous les tiroirs. En data science, la différence se joue entre un calcul qui prend une seconde et un calcul qui ne finit jamais.

Cette section est plus « informatique » que les autres : on y apprend à **penser** un calcul, à le **prouver** correct et à **compter** son coût. Rassurez-vous, tout part d'exemples faits à la main.

4.3.1 Qu'est-ce qu'un algorithme ?

Un algorithme est une suite d'instructions **non ambiguës** qui, pour toute entrée valide, **se termine** et produit la **bonne** sortie. Trois exigences, donc : être précis, **terminer**, être **correct**.

Exemple fil rouge : trouver la plus grosse commande d'une liste, sans utiliser max. Méthode : on retient « le plus grand vu jusqu'ici » et on parcourt la liste, en le mettant à jour chaque fois qu'on trouve mieux.

À la main, sur [44,8; 34,5; 88,2; 30,1; 110,1] :

Élément lu	Plus grand vu jusqu'ici
44,8 (on commence avec le premier)	44,8
34,5	44,8 (34,5 est plus petit)
88,2	88,2 (nouveau record)
30,1	88,2
110,1	110,1 (nouveau record)

Résultat : 110,1. En code, avec un compteur de comparaisons pour mesurer le coût :

```
def plus_grand(valeurs):
    """Renvoie (le maximum, le nombre de comparaisons effectuées)."""
    record = valeurs[0]
    comparaisons = 0
    for v in valeurs[1:]:
        comparaisons += 1
        if v > record:
            record = v
    return record, comparaisons

print(plus_grand([44.8, 34.5, 88.2, 30.1, 110.1]))
```

```
(110.1, 4)
```

Preuve de correction (par invariant de boucle). Un **invariant** est une propriété vraie *avant* chaque tour de boucle, qu'on prouve comme une récurrence.

Invariant : après avoir lu les k premiers éléments, record est le maximum de ces k éléments. *Initialisation* ($k = 1$) : record est le premier élément, qui est bien le maximum d'une liste d'un élément. *Conservation* : supposons l'invariant vrai pour k . On lit l'élément v n° $k + 1$. Si $v > \text{record}$, le nouveau record est v , qui est supérieur à tous les précédents ; sinon record reste le maximum. Dans les deux cas l'invariant est vrai pour $k + 1$. *Fin* : la boucle for parcourt une liste finie : elle **termine** après $n - 1$ tours. Pour $k = n$, l'invariant dit que record est le maximum de **toute** la liste. ■

Coût : exactement $n - 1$ comparaisons, quelle que soit la liste. On dit que l'algorithme est **linéaire** (section 4.8).

Cette habitude (**invariant, terminaison, coût**) est la boîte à outils de base pour tout algorithme, y compris les plus sophistiqués.

4.3.2 Choisir sa structure de données

Les quatre collections de 4.1.4 ne sont pas interchangeables : chacune est **bonne** pour certaines opérations et **mauvaise** pour d'autres.

Besoin	Meilleure structure	Pourquoi
Garder un ordre, accéder par position	liste	<code>x[i]</code> est immédiat
Retrouver une valeur à partir d'un nom, d'un identifiant	dictionnaire	accès direct par clé
Savoir « est-ce que X est déjà vu ? », éliminer les doublons	ensemble	test d'appartenance immédiat
Traiter dans l'ordre d'arrivée	file (deque)	retrait au début immédiat
Revenir en arrière (annuler)	pile (une liste utilisée par la fin)	dernier entré, premier sorti

Pourquoi un dictionnaire ou un ensemble est-il si rapide ? Ils reposent sur une **table de hachage** : une fonction (le *hachage*) transforme la clé en un numéro de case, et l'on va **directement** à cette case, sans parcourir quoi que ce soit. Imaginez un vestiaire où chaque cintre porte le numéro calculé à partir du nom du client : pas besoin de passer en revue tous les manteaux. Une liste, elle, doit être parcourue de gauche à droite pour savoir si une valeur y figure. Mesurons ce parcours en **nombre de comparaisons** (une mesure qui ne dépend pas de l'ordinateur) sur nos 400 montants :

```
import csv

with open("donnees/commandes.csv", encoding="utf-8") as f:
    lignes = list(csv.DictReader(f))
montants = [float(l["montant"]) for l in lignes]
print("nombre de montants :", len(montants))

def recherche_lineaire(liste, cible):
    """Parcourt la liste de gauche à droite. Renvoie (position ou -1, nombre de comparaisons)."""
    comparaisons = 0
    for i, v in enumerate(liste):
        comparaisons += 1
        if v == cible:
            return i, comparaisons
    return -1, comparaisons

print("cherche 44.8 (en tête) :", recherche_lineaire(montants, 44.8))
print("cherche 255.7 (la plus grosse commande) :", recherche_lineaire(montants, 255.7))
print("cherche 1.0 (absent) :", recherche_lineaire(montants, 1.0))

nombre de montants : 400
cherche 44.8 (en tête) : (0, 1)
cherche 255.7 (la plus grosse commande) : (156, 157)
cherche 1.0 (absent) : (-1, 400)
```

Quand la valeur est **absente**, il faut lire les 400 éléments pour en être sûr : le pire cas est proportionnel à la taille de la liste. Un ensemble ou un dictionnaire, lui, répond en une seule opération, quel que soit le nombre d'éléments. Nous mesurerons l'écart en secondes à la section 4.8 ; reprenez pour l'instant l'ordre de grandeur : pour un million d'éléments, une liste doit parcourir *jusqu'à un million* de cases, un ensemble en regarde *quelques-unes*.

Application directe : **compter les montants différents et les plus fréquents** :

```
from collections import Counter

distincts = set(montants)
print("montants distincts :", len(distincts), "sur", len(montants), "commandes")

frequences = Counter(montants)
print("les 5 montants les plus fréquents :", frequences.most_common(5))
```

```
montants distincts : 344 sur 400 commandes
les 5 montants les plus fréquents : [(37.5, 4), (53.5, 4), (65.8, 3), (35.7, 3), (21.5, 3)]
```

Un Counter est un dictionnaire spécialisé dans le comptage : la clé est la valeur, la valeur est son nombre d'occurrences. C'est l'outil idéal pour une table de fréquences (3.1) construite à la main.

4.3.3 Piles et files

Deux structures très simples, définies par l'ordre dans lequel on y entre et sort.

- La **pile** (*stack*, **LIFO** : *last in, first out*) : le dernier arrivé est le premier servi. Une pile d'assiettes : on pose et on reprend toujours en haut. C'est le mécanisme du bouton « annuler » de votre éditeur de texte, et de l'**historique** de votre navigateur.
- La **file** (*queue*, **FIFO** : *first in, first out*) : le premier arrivé est le premier servi. La file d'attente à la caisse.

En Python, une pile est une simple liste que l'on manipule **par la fin** (append pour empiler, pop pour dépiler). Pour une file, on utilise deque (prononcez « dèque »), car retirer un élément au **début** d'une liste est lent, alors qu'un deque le fait à coût constant.

```
from collections import deque

pile = []
for action in ["ajouter bol", "ajouter tasse", "ajouter plateau"]:
    pile.append(action)           # empiler
print("pile :", pile)
print("annuler :", pile.pop(), "| il reste :", pile)    # dépiler : le dernier entré sort

file = deque()
for client in ["Sana", "Mehdi", "Ines"]:
    file.append(client)          # arriver au bout de la file
print("file :", list(file))
print("servi :", file.popleft(), "| il reste :", list(file)) # le premier arrivé sort

pile : ['ajouter bol', 'ajouter tasse', 'ajouter plateau']
annuler : ajouter plateau | il reste : ['ajouter bol', 'ajouter tasse']
file : ['Sana', 'Mehdi', 'Ines']
servi : Sana | il reste : ['Mehdi', 'Ines']
```

Application 1 : vérifier les parenthèses d'une formule (avec une pile). Un tableur doit rejeter $= (B2+B3) * (1 - (C2/100))$ car il manque une parenthèse fermante. Comment un programme le détecte-t-il ?

Idée : à chaque parenthèse **ouvrante**, on empile ; à chaque parenthèse **fermante**, on dépile et on vérifie qu'elle correspond. La formule est correcte si, **à la fin**, la pile est vide et si nous n'avons jamais dû dépiler une pile vide.

À la main sur $(1 + (2 * 3))$: (→ pile [(; 1, + ignorés ; (→ [(, (; $2 * 3$ ignorés ;) → on dépile : [(;) → on dépile : []. Pile vide à la fin : équilibrée. Sur $(1 + 2)$: après le premier) la pile est vide ; le second) ne trouve rien à dépiler : **erreur**.

```
def equilibre(formule):
    ouvrantes = {"(": "(", ")": "(", "}": "{", "]": "["}
    pile = []
    for caractere in formule:
        if caractere in "({[":
            pile.append(caractere)
        elif caractere in ")}]":
            if not pile or pile.pop() != ouvrantes[caractere]:
                return False
```

```

return not pile          # vrai seulement si tout a été refermé

tests = ["(1+(2*3))", "(1+2)", "(B2+B3)*(1-(C2/100))", "[(1+2)*3]", "[ (1+2)*3)", "" ]
for t in tests:
    print(f"{t!r:<28} -> {equilibre(t)}")

```

```

'(1+(2*3))'          -> True
'(1+2))'            -> False
'=(B2+B3)*(1-(C2/100))' -> False
'[(1+2)*3]'         -> True
'[ (1+2)*3)'        -> False
''                  -> True

```

Le cas $[(1+2)*3]$ est instructif : il y a autant d'ouvrantes que de fermantes, mais elles sont **mal imbriquées** : une simple comptabilité ne suffirait pas, la pile, si. Voilà pourquoi tous les compilateurs et analyseurs de formules utilisent cette structure.

Application 2 : une file d'attente à l'atelier d'emballage. Cinq commandes arrivent aux minutes 0, 1, 2, 10 et 11. L'emballage d'une commande dure 4 minutes, avec **une seule** emballeuse, qui traite les commandes dans l'ordre d'arrivée.

À la main :

Commande	Arrivée	Début d'emballage	Attente
1	0	0	0
2	1	4 (la 1 se termine à 4)	3
3	2	8	6
4	10	12 (la 3 se termine à 12)	2
5	11	16	5

Attente moyenne : $(0 + 3 + 6 + 2 + 5)/5 = 3,2$ minutes. Le programme :

```

def simuler_file(arrivees, duree):
    attente_totale, libre_a = 0, 0
    file = deque(arrivees)          # les commandes en attente, dans l'ordre
    attentes = []
    while file:
        arrivee = file.popleft()     # premier arrivé, premier servi
        debut = max(arrivee, libre_a) # on commence quand la commande est là ET l'emballage libre
        attentes.append(debut - arrivee)
        libre_a = debut + duree
    return attentes

attentes = simuler_file([0, 1, 2, 10, 11], duree=4)
print("attentes :", attentes, "| moyenne :", sum(attentes) / len(attentes))

```

```
attentes : [0, 3, 6, 2, 5] | moyenne : 3.2
```

Cette mini-simulation est le premier pas vers la **théorie des files d'attente**, vue au chapitre 2 (processus de Poisson et loi exponentielle) : on peut maintenant la tester numériquement.

4.3.4 La récursion : une fonction qui s'appelle elle-même

Une fonction est **réursive** quand elle se résout en s'appelant sur un problème **plus petit**. Pour qu'elle marche, il faut toujours deux ingrédients :

1. un **cas de base** (le plus petit problème, résolu directement) ;
2. un **cas général** qui ramène le problème à un problème **strictement plus petit**.

Sans cela, la fonction s'appelle à l'infini (Python finit par s'arrêter avec une erreur `RecursionError`).

Premier exemple : la factorielle. $n! = n \times (n-1)!$ avec $0! = 1$. Elle compte les façons d'ordonner n objets (1.6). **À la main :** $4! = 4 \times 3! = 4 \times 3 \times 2! = 4 \times 3 \times 2 \times 1! = 24$.

```
def factorielle(n):
    if n == 0:
        return 1
    return n * factorielle(n - 1)

print([factorielle(n) for n in range(7)])
```

```
[1, 1, 2, 6, 24, 120, 720]
```

Preuve (par récurrence). *Terminaison :* à chaque appel, n diminue de 1 et reste un entier ≥ 0 : on atteint forcément le cas de base. *Correction :* `factorielle(0)` renvoie $1 = 0!$ (base). Si `factorielle(n-1)` renvoie $(n-1)!$ (hypothèse), alors `factorielle(n)` renvoie $n \times (n-1)! = n!$. ■ C'est la même récurrence qu'en mathématiques : écrire une fonction récursive *est* écrire une preuve par récurrence.

Une application naturelle : parcourir un arbre. Le catalogue de Dar Jasmin est organisé en catégories contenant des sous-catégories contenant des produits (prix, stock). On veut la **valeur totale du stock**. La difficulté : on ne sait pas combien de niveaux il y a. La récursion le fait tout naturellement : *la valeur d'une catégorie est la somme des valeurs de ses éléments, la valeur d'un produit est prix $\times$ stock*.

À la main : bol bleu $12,5 \times 10 = 125$; bol vert $12,5 \times 4 = 50$; tasse $8 \times 20 = 160$; bougie $15,9 \times 6 = 95,4$; plateau $45 \times 2 = 90$. Total : 520,4 DT.

```
catalogue = {
    "Vaisselle": {
        "bols": {"bol bleu": (12.5, 10), "bol vert": (12.5, 4)},
        "tasses": {"tasse": (8.0, 20)},
    },
    "Déco": {"bougie": (15.9, 6), "plateau": (45.0, 2)},
}

def valeur_stock(noeud):
    if isinstance(noeud, tuple):
        prix, stock = noeud
        return prix * stock
    return sum(valeur_stock(enfant) for enfant in noeud.values())

print("valeur totale :", round(valeur_stock(catalogue), 2), "DT")
for categorie, contenu in catalogue.items():
    print(f" {categorie:<10} {valeur_stock(contenu):8.2f} DT")
```

```
valeur totale : 520.4 DT
Vaisselle      335.00 DT
Déco           185.40 DT
```

Le danger : la récursion naïve peut être catastrophique. Les nombres de Fibonacci ($F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$) se codent en deux lignes récursives, mais le programme refait **sans cesse les mêmes calculs** : pour calculer `fib(5)`, on calcule deux fois `fib(3)`, trois fois `fib(2)`, etc. Comptons les appels :

```
from functools import lru_cache

appels = 0
```



```
def fib(n):
    global appels
    appels += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

for n in (10, 20, 25):
    appels = 0
    valeur = fib(n)
    print(f"fib({n}) = {valeur}    nombre d'appels : {appels}")

appels_memo = 0
@lru_cache(maxsize=None)           # « mémoïsation » : on retient les résultats déjà calculés
def fib_memo(n):
    global appels_memo
    appels_memo += 1
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

print(f"fib_memo(25) = {fib_memo(25)}    nombre d'exécutions réelles : {appels_memo}")

fib(10) = 55    nombre d'appels : 177
fib(20) = 6765    nombre d'appels : 21891
fib(25) = 75025    nombre d'appels : 242785
fib_memo(25) = 75025    nombre d'exécutions réelles : 26
```

Le nombre d'appels de la version naïve **explose** (il est lui-même de l'ordre de F_n , donc il croît environ de 62 % à chaque pas), alors que la version qui **mémorise** ne calcule chaque valeur qu'une seule fois. Même problème, même résultat, des ordres de grandeur d'écart : c'est tout l'enjeu de la complexité (section 4.8). La mémoïsation est la première idée de la **programmation dynamique**, très utilisée en optimisation.

Récursion ou boucle ? Tout algorithme récursif peut s'écrire avec une boucle (et inversement). La récursion est souvent plus *lisible* pour les structures **arborescentes** (catalogue, dossiers, expressions) ; la boucle est plus économe en mémoire. Python limite la profondeur de récursion à environ 1 000 appels : au-delà, préférez une boucle.

4.3.5 Rechercher : de la liste à la dichotomie

Reprenons la **recherche linéaire** de 4.3.2 : sur une liste quelconque, elle est inévitable. Mais si la liste est **triée**, on peut faire beaucoup mieux, comme quand vous cherchez un mot dans un dictionnaire papier : vous ouvrez au milieu, vous voyez si le mot est avant ou après, et vous éliminez **la moitié** des pages d'un coup. C'est la **recherche dichotomique** (*binary search*).

À la main. Liste triée de 9 montants : [8; 15; 22; 31; 40; 47; 58; 66; 79] (indices 0 à 8). On cherche 47.

Étape	Zone d'indices [g, d]	Milieu $m = \lfloor (g + d) / 2 \rfloor$	Valeur	Décision
1	[0, 8]	4	40	40 < 47 : on cherche à droite, g = 5
2	[5, 8]	6	58	58 > 47 : on cherche à gauche, d = 5
3	[5, 5]	5	47	trouvé !

Trois étapes au lieu de six pour la recherche linéaire. Le programme :

```
def recherche_dichotomique(triee, cible):
    """Renvoie (position ou -1, nombre de comparaisons). La liste doit être triée."""
    g, d, comparaisons = 0, len(triee) - 1, 0
    while g <= d:
        m = (g + d) // 2
        comparaisons += 1
        if trie[m] == cible:
            return m, comparaisons
        elif trie[m] < cible:
            g = m + 1
        else:
            d = m - 1
    return -1, comparaisons

exemple = [8, 15, 22, 31, 40, 47, 58, 66, 79]
print(recherche_dichotomique(exemple, 47))
print(recherche_dichotomique(exemple, 50))      # absent
```

```
(5, 3)
(-1, 3)
```

Preuve de correction et de terminaison.

Invariant : si la cible est dans la liste, elle se trouve à un indice entre g et d inclus. *Initialisation :* $g = 0, d = n - 1$: toute la liste. *Conservation :* si $triee[m] < cible$, comme la liste est triée, tous les éléments d'indice $\leq m$ sont $< cible$: on peut les éliminer, d'où $g = m + 1$. Symétriquement si $triee[m] > cible$. L'invariant reste vrai. *Conclusion :* si la boucle s'arrête parce que $g > d$, la zone est vide, donc la cible est absente (on renvoie -1) ; si elle s'arrête sur $triee[m] == cible$, c'est gagné.

Terminaison et coût : appelons $s = d - g + 1$ la taille de la zone. À chaque tour, la nouvelle zone a au plus $\lfloor s/2 \rfloor$ éléments (on a éliminé le milieu et une moitié). Après k tours, la taille est au plus $\lfloor n/2^k \rfloor$, qui devient 0 dès que $2^k > n$. L'algorithme s'arrête donc en **au plus** $\lfloor \log_2 n \rfloor + 1$ tours. ■

Vérifions la borne théorique sur nos 400 montants triés, en cherchant **chacun** des 400 montants et en relevant le pire cas :

```
import math

triee = sorted(montants)
pire = max(recherche_dichotomique(triee, v)[1] for v in montants)
moyen = sum(recherche_dichotomique(triee, v)[1] for v in montants) / len(montants)
print("borne théorique (floor(log2 n) + 1) :", math.floor(math.log2(len(montants))) + 1)
print("pire cas observé :", pire)
print("moyenne observée :", round(moyen, 2))
print("recherche absente (linéaire / dichotomique) :",
      recherche_lineaire(triee, 1.0)[1], "/", recherche_dichotomique(triee, 1.0)[1])
```

```
borne théorique (floor(log2 n) + 1) : 9
pire cas observé : 9
moyenne observée : 7.42
recherche absente (linéaire / dichotomique) : 400 / 8
```

Le pire cas observé respecte bien la borne (au plus 9 comparaisons pour 400 éléments, et 8 pour une valeur absente comme 1,0, contre 400 pour la recherche linéaire de cette même valeur). Pour **un million** d'éléments : $\lfloor \log_2 10^6 \rfloor + 1 = 20$ comparaisons seulement. C'est le pouvoir du logarithme : chaque doublement de la taille ne coûte qu'**une** comparaison de plus.

Application : le seuil de livraison gratuite. Yasmine veut offrir la livraison à partir d'un seuil tel qu'environ 30 % des commandes y aient droit. Une liste triée permet de répondre à « quelle part

des commandes dépasse s DT ? » avec le module `bisect`, qui contient la recherche dichotomique toute faite :

```
import bisect
import numpy as np

def part_au_dessus(seuil):
    # bisect_left donne le nombre de montants STRICTEMENT inférieurs au seuil
    return 1 - bisect.bisect_left(triee, seuil) / len(triee)

for seuil in (50, 60, 70, 80, 90, 100):
    print(f"seuil {seuil:3d} DT : {part_au_dessus(seuil):6.1%} des commandes y ont droit")

print("quantile 70 % (numpy) :", round(float(np.quantile(montants, 0.70)), 1), "DT")
```

```
seuil 50 DT : 51.2% des commandes y ont droit
seuil 60 DT : 41.0% des commandes y ont droit
seuil 70 DT : 29.2% des commandes y ont droit
seuil 80 DT : 21.8% des commandes y ont droit
seuil 90 DT : 17.2% des commandes y ont droit
seuil 100 DT : 13.0% des commandes y ont droit
quantile 70 % (numpy) : 68.9 DT
```

Le tableau montre qu'un seuil de **70 DT** concerne 29,2 % des commandes, soit à peu près les 30 % visés ; le quantile à 70 % calculé par NumPy (68,9 DT) pointe au même endroit, puisque par définition 30 % des commandes lui sont supérieures. Nous avons retrouvé par un algorithme de recherche ce que les quantiles du 3.1.3 donnaient directement, un bon moyen de comprendre ce que « quantile » veut dire : *la position dans la liste triée*.

4.3.6 Trier

Trier est l'un des problèmes les plus étudiés. Il sert partout : classer les clients, calculer une médiane, préparer une recherche dichotomique. Voyons deux algorithmes : un simple, un efficace.

Le tri par insertion. On procède comme avec des cartes à jouer : on prend les éléments un à un et on **insère** chacun à sa place dans la partie déjà triée.

À la main sur [30; 12; 25; 8; 19] (la partie triée est à gauche de la double barre ||) :

Étape	Liste	Action
départ	[30 12, 25, 8, 19]	
1	[12, 30 25, 8, 19]	12 passe devant 30
2	[12, 25, 30 8, 19]	25 passe devant 30
3	[8, 12, 25, 30 19]	8 passe devant tous
4	[8, 12, 19, 25, 30]	19 s'insère entre 12 et 25

```
def tri_insertion(liste):
    a = list(liste)
    comparaisons = 0
    for i in range(1, len(a)):
        x = a[i]
        j = i - 1
        while j >= 0:
            comparaisons += 1
            if a[j] > x:
                a[j + 1] = a[j]
                j -= 1
            else:
                a[j + 1] = x
                break
    return a
```

```

        break
    a[j + 1] = x                # on insère x à sa place
    return a, comparaisons

print(tri_insertion([30, 12, 25, 8, 19]))

```

```

([8, 12, 19, 25, 30], 9)

```

Correction. *Invariant* : avant le tour i , les i premiers éléments de a sont triés (et ce sont les i premiers éléments d'origine). Au tour i , on décale vers la droite tous les éléments plus grands que x puis on place x juste avant eux : les $i + 1$ premiers éléments sont triés. À la fin ($i = n$), tout est trié. *Terminaison* : deux boucles bornées. *Coût* : au pire (liste triée à l'envers), le tour i fait i comparaisons, soit $1 + 2 + \dots + (n - 1) = n(n - 1)/2$ comparaisons au total, de l'ordre de n^2 . ■

Le **tri fusion** (*merge sort*) applique la stratégie « **diviser pour régner** » : on coupe la liste en deux, on trie chaque moitié (récursivement), puis on **fusionne** les deux moitiés triées. Fusionner est facile : on compare les deux premiers éléments des moitiés, on prend le plus petit, et on recommence, comme deux files de gens que l'on entrelace.

À la main sur [44; 12; 30; 8; 25; 19] : on coupe en [44, 12, 30] et [8, 25, 19] ; chacun est trié en [12, 30, 44] et [8, 19, 25] ; la fusion donne $8 < 12 \rightarrow 8$; $12 < 19 \rightarrow 12$; $19 < 30 \rightarrow 19$; $25 < 30 \rightarrow 25$; il reste 30, 44 : [8, 12, 19, 25, 30, 44].

```

def tri_fusion(liste):
    """Renvoie (liste triée, nombre de comparaisons)."""
    if len(liste) <= 1:                # cas de base
        return list(liste), 0
    milieu = len(liste) // 2
    gauche, c1 = tri_fusion(liste[:milieu])
    droite, c2 = tri_fusion(liste[milieu:])
    fusion, i, j, c = [], 0, 0, 0
    while i < len(gauche) and j < len(droite):    # on entrelace les deux moitiés triées
        c += 1
        if gauche[i] <= droite[j]:
            fusion.append(gauche[i]); i += 1
        else:
            fusion.append(droite[j]); j += 1
    fusion.extend(gauche[i:])                # il ne reste d'un seul côté
    fusion.extend(droite[j:])
    return fusion, c1 + c2 + c

print(tri_fusion([44, 12, 30, 8, 25, 19]))

```

```

([8, 12, 19, 25, 30, 44], 9)

```

Comparons les deux sur nos 400 montants (ce sont aussi des sorties à contrôler contre le tri de Python) :

```

tri1, c_ins = tri_insertion(montants)
tri2, c_fus = tri_fusion(montants)
print("tri par insertion : ", c_ins, "comparaisons")
print("tri fusion       : ", c_fus, "comparaisons")
print("même résultat que sorted() :", tri1 == sorted(montants) == tri2)
print("n^2 / 4 =", len(montants) ** 2 // 4, " | n log2 n =", round(len(montants) *
    ↪ math.log2(len(montants))))

```

```

tri par insertion : 41010 comparaisons
tri fusion       : 2972 comparaisons
même résultat que sorted() : True

```

```
n^2 / 4 = 40000 | n log2 n = 3458
```

Le tri par insertion fait de l'ordre de $n^2/4$ comparaisons en moyenne (41 010 ici, soit près de quatorze fois plus que les 2 972 du tri fusion), le tri fusion de l'ordre de $n \log_2 n$. Pour 400 éléments, cela fait la différence entre « bien » et « très bien » ; pour 10 millions, entre quelques secondes et plusieurs jours. Le tri intégré de Python, `sorted`, utilise un algorithme hybride très optimisé (*Timsort*) de coût $n \log n$: **en pratique, on utilise toujours `sorted()` ou `.sort()`**, mais comprendre ce qu'il fait permet de raisonner sur son coût.

Un tri est dit stable s'il laisse dans leur ordre d'origine les éléments « égaux » au regard du critère de tri. C'est ce que garantit `sorted` de Python, et cela permet d'enchaîner des tris successifs : trier d'abord par montant, puis par canal, donne les commandes **rangées par canal et, à l'intérieur de chaque canal, par montant croissant**.

```
commandes = [("Site", 40.1), ("Boutique", 65.8), ("Site", 19.6), ("Boutique", 17.4), ("Instagram",
↪ 30.1)]
par_montant = sorted(commandes, key=lambda c: c[1])
par_canal_puis_montant = sorted(par_montant, key=lambda c: c[0]) # stable : conserve l'ordre par
↪ montant
for c in par_canal_puis_montant:
    print(c)

('Boutique', 17.4)
('Boutique', 65.8)
('Instagram', 30.1)
('Site', 19.6)
('Site', 40.1)
```

4.3.7 Une dernière application : les « top k »

Question de Yasmine : « Quelles sont mes **cinq plus grosses commandes** ? » On pourrait trier les 400 montants puis prendre les cinq derniers (coût de l'ordre de $n \log n$). Mais c'est du gaspillage : on n'a pas besoin que *tout* soit trié. Une structure appelée **tas** (*heap*) maintient efficacement les k plus grands vus jusqu'ici, avec un coût de l'ordre de $n \log k$. Le module `heapq` la fournit :

```
import heapq

top5 = heapq.nlargest(5, montants)
print("5 plus grosses commandes (heapq) :", top5)
print("5 plus grosses commandes (tri)   :", sorted(montants, reverse=True)[:5])

# top 3 avec leur canal : on classe les lignes selon une clé
top3_lignes = heapq.nlargest(3, lignes, key=lambda l: float(l["montant"]))
for l in top3_lignes:
    print(f"  {l['canal']}<10} {l['montant']>6} DT   livraison {l['livraison']} j   satisfaction
↪ {l['satisfaction']}/5")

5 plus grosses commandes (heapq) : [255.7, 243.8, 217.1, 212.4, 208.8]
5 plus grosses commandes (tri)   : [255.7, 243.8, 217.1, 212.4, 208.8]
Site      255.7 DT   livraison 4 j   satisfaction 3/5
Site      243.8 DT   livraison 5 j   satisfaction 3/5
Site      217.1 DT   livraison 7 j   satisfaction 4/5
```

Les deux listes sont identiques, la version `heapq` coûtant moins cher quand $k \ll n$ (pensez aux dix meilleurs clients sur dix millions). Ce réflexe, **ne pas faire plus de travail que nécessaire**, est l'une des habitudes les plus rentables de la programmation scientifique.

Pour aller plus loin. Les mêmes idées (hachage, tri, recherche dichotomique, arbres) sont au cœur des **bases de données** : l'**index** d'une table SQL (chapitre 5) est précisément un arbre trié qui permet une recherche dichotomique ; les GROUP BY s'appuient sur des tables de hachage ou des tris. Comprendre cette section, c'est comprendre pourquoi une requête est rapide ou lente.

À retenir

- Un algorithme doit être **précis, terminer, et être correct** ; on le prouve avec un **invariant** (vrai à chaque tour) et un argument de **terminaison** ; on mesure son coût en **nombre d'opérations**.
- **Liste** : ordre et position ; **dictionnaire** / **ensemble** : accès et test d'appartenance immédiats (table de hachage) ; **pile** : dernier entré, premier sorti ; **file** : premier entré, premier sorti.
- La **réursion** exige un cas de base et un problème strictement plus petit ; elle se prouve comme une récurrence. Sans précaution (mémoïsation), elle peut refaire des calculs en nombre exponentiel.
- La **recherche dichotomique** (liste triée) coûte au plus $\lfloor \log_2 n \rfloor + 1$ comparaisons : 9 pour 400 éléments, 20 pour un million.
- Le **tri par insertion** coûte de l'ordre de n^2 , le **tri fusion** de l'ordre de $n \log n$; en pratique on utilise `sorted()`, qui est stable.
- Ne trier que ce qui est nécessaire : pour les « top k », `heapq.nlargest` suffit.

4.4 NumPy et pandas : manipuler des données

Intuition. Python « de base » (section 4.1) sait faire des calculs, mais il est lent et verbeux dès qu'il faut traiter des milliers de nombres. Deux bibliothèques ont changé la donne et sont aujourd'hui le socle de la data science en Python :

- **NumPy** ajoute un nouveau type d'objet, le **tableau** (`ndarray`), qui permet de calculer sur toute une colonne de nombres d'un seul coup, à vitesse quasi native ;
- **pandas** pose par-dessus un **tableau étiqueté** (le `DataFrame`) : des colonnes qui ont un nom et un type, des lignes qui ont une étiquette, des dates, des valeurs manquantes... bref, un tableur programmable, sans souris et sans limite de taille.

Yasmine a l'habitude d'Excel. Tout ce que nous allons faire ici, elle pourrait le faire à la main dans une feuille de calcul, mais pour 400 lignes ce serait pénible, et pour 400 000 lignes ce serait impossible. Surtout, **le code est rejouable** : on corrige une erreur, on relance, on retrouve toutes les analyses mises à jour.

Comment lire cette section. Elle est longue parce que pandas est l'outil que vous utiliserez tous les jours. Les trois premières parties (4.4.1 à 4.4.4) concernent NumPy ; la suite concerne pandas. Chaque notion est introduite par un **petit exemple à la main**, puis appliquée aux 400 commandes de Dar Jasmin. Si vous êtes pressé(e), lisez au moins 4.4.5 à 4.4.9, puis la petite application de 4.4.13.

4.4.1 NumPy : pourquoi un tableau n'est pas une liste

Yasmine veut afficher les prix TTC de cinq articles à partir de leurs prix hors taxe (TVA à 19 %). Avec une **liste** Python classique, on écrit une boucle :

```
import numpy as np

prix_ht = [10, 20, 5, 40, 15]
prix_ttc = [round(p * 1.19, 2) for p in prix_ht]
print(prix_ttc)
```

```
[11.9, 23.8, 5.95, 47.6, 17.85]
```

À la main : $10 \times 1,19 = 11,90$, $20 \times 1,19 = 23,80$, $5 \times 1,19 = 5,95$, $40 \times 1,19 = 47,60$ et $15 \times 1,19 = 17,85$. Le résultat est celui attendu. Avec un **tableau NumPy**, la boucle disparaît :

```
ht = np.array([10, 20, 5, 40, 15])
print(ht * 1.19)
print(type(ht), ht.dtype, ht.shape)
```

```
[11.9 23.8  5.95 47.6 17.85]
<class 'numpy.ndarray'> int64 (5,)
```

Une seule opération `ht * 1.19` s'applique à **chaque élément** : on dit qu'elle est **vectorisée**. Ce n'est pas qu'une commodité d'écriture : les listes Python sont des collections d'objets quelconques (un entier, puis un texte, puis une autre liste...), alors qu'un tableau NumPy est un **bloc de mémoire homogène** (ici des entiers de 64 bits, `dtype=int64`) traité par du code compilé en C. D'où deux différences que tout le monde rencontre un jour :

```
print([10, 20] * 2)           # liste : * 2 répète la liste
print(np.array([10, 20]) * 2) # tableau : * 2 multiplie chaque élément
```

```
[10, 20, 10, 20]
[20 40]
```

Piège classique. Sur une liste, `*` et `+` **répètent et concatènent**. Sur un tableau, ils **calculent**. Si vous voyez une liste de 10 000 nombres se mettre à « doubler de longueur » au lieu de doubler de valeur, c'est qu'un tableau a été oublié.

Mesurons maintenant l'écart de vitesse sur un million de prix :

```
import time

x = np.random.default_rng(0).uniform(10, 100, size=1_000_000)
liste = x.tolist()

t0 = time.perf_counter()
ttc_boucle = [v * 1.19 for v in liste]      # une boucle Python
t1 = time.perf_counter()
ttc_numpy = x * 1.19                       # une opération vectorisée
t2 = time.perf_counter()

print("mêmes résultats :", np.array_equal(ttc_boucle, ttc_numpy))
print("NumPy au moins 5 fois plus rapide :", (t1 - t0) / (t2 - t1) > 5)
```

```
mêmes résultats : True
NumPy au moins 5 fois plus rapide : True
```

Les durées exactes dépendent de votre machine (nous n'affichons donc que des conclusions robustes). Sur du calcul plus lourd que cette simple multiplication, l'écart se compte en **dizaines, voire centaines de fois**. C'est la raison pour laquelle on cherche toujours à **vectoriser** : écrire `x * 1.19` plutôt qu'une boucle `for`.

À retenir. Un tableau NumPy = un bloc de nombres **du même type**, sur lequel les opérations s'appliquent **élément par élément** et vite. Règle d'or : *pas de boucle for sur des données, sauf si l'on n'a vraiment pas le choix.*

4.4.2 Créer, indexer, trancher

Quelques manières courantes de fabriquer des tableaux :

```
print(np.arange(0, 10, 2))      # de 0 à 10 (exclu), pas de 2
print(np.linspace(0, 1, 5))    # 5 points régulièrement espacés entre 0 et 1
print(np.zeros(3), np.ones(3)) # que des 0, que des 1
print(np.full((2, 3), 7))      # un tableau 2x3 rempli de 7
```

```
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1. ]
[0. 0. 0.] [1. 1. 1.]
[[7 7 7]
 [7 7 7]]
```

Un tableau a trois caractéristiques à toujours avoir en tête : sa **forme** (shape), son nombre de dimensions (ndim) et son type (dtype). Prenons les ventes hebdomadaires (en DT) des trois canaux de Dar Jasmin sur quatre semaines : une **matrice** de 3 lignes (canaux) et 4 colonnes (semaines), exactement comme celles du chapitre 1.

	Sem. 1	Sem. 2	Sem. 3	Sem. 4
Instagram	120	150	90	140
Site	90	100	120	70
Boutique	60	50	90	105

```
A = np.array([[120, 150, 90, 140],
              [ 90, 100, 120, 70],
              [ 60, 50, 90, 105]])
print(A.shape, A.ndim, A.dtype, A.size)
```

```
(3, 4) 2 int64 12
```

Indexation. On désigne un élément par `A[ligne, colonne]`, en comptant à **partir de 0** (comme partout en Python). Quelques exemples à vérifier *sur le tableau ci-dessus* avant de regarder la sortie :

- `A[1, 2]` : ligne 1 (le Site), colonne 2 (semaine 3) → **120** ;
- `A[0]` : toute la ligne 0 (Instagram) ;
- `A[:, 3]` : toute la colonne 3 (semaine 4) ; le « : » signifie « tout » ;
- `A[0:2, 1:3]` : lignes 0 et 1, colonnes 1 et 2 (la borne de fin est **exclue**).

```
print(A[1, 2])
print(A[0])
print(A[:, 3])
print(A[0:2, 1:3])
print(A[-1, -1])    # les indices négatifs partent de la fin : dernière ligne, dernière colonne
```

```
120
[120 150 90 140]
[140 70 105]
[[150 90]
 [100 120]]
```


105

Filtrer avec un masque booléen. Comparer un tableau à un nombre produit un tableau de True/False de même forme : le **masque**. Utilisé comme indice, il ne garde que les cases True.

```
masque = A > 100
print(masque)
print(A[masque]) # les ventes strictement supérieures à 100 DT
print("combien ?", masque.sum()) # True compte pour 1 : on compte donc les cases vraies
print(np.where(A > 100, "bien", "-")) # np.where(condition, si_vrai, si_faux)
```

```
[[ True  True False  True]
 [False False  True False]
 [False False False  True]]
[120 150 140 120 105]
combien ? 5
[['bien' 'bien' '-' 'bien']
 ['- ' '-' 'bien' '-']
 ['- ' '-' '-' 'bien']]
```

Dans le masque, on compte 5 cases vraies : 120, 150 et 140 (Instagram), 120 (Site, semaine 3) et 105 (Boutique, semaine 4). Notez que le 100 du Site (semaine 2) n'est **pas** compté : la condition est « strictement supérieur à 100 ». La somme d'un masque est une astuce très utile : elle **compte** les True.

Pour combiner deux conditions, on utilise & (et), | (ou) et ~ (non), **avec des parenthèses** :

```
print(A[(A > 80) & (A < 130)]) # entre 80 et 130 exclus
```

```
[120  90  90 100 120  90 105]
```

Tranche = vue, pas copie. Tailler un morceau d'un tableau avec : ne copie pas les données : la tranche est une **fenêtre** sur le tableau d'origine. Modifier la fenêtre modifie l'original ! Pour obtenir une copie indépendante, écrivez explicitement `.copy()`.

```
C = A.copy()
fenetre = C[0, :] # une vue sur la ligne d'Instagram
fenetre[0] = 999
print(C[0, 0], "<- modifié par la vue ; A[0, 0] =", A[0, 0], "(intact, car C est une copie)")
```

```
999 <- modifié par la vue ; A[0, 0] = 120 (intact, car C est une copie)
```

Enfin, un tableau a **un seul type** : si l'on mélange entiers et décimaux, tout devient décimal. Et un tableau d'entiers ne sait pas représenter une valeur manquante, ce qui sera une raison de plus d'aimer pandas (4.4.11).

```
print(np.array([1, 2, 3.5])) # tout devient décimal
print(np.array([1, 2, 3]).astype(float)) # conversion explicite
```

```
[1.  2.  3.5]
[1.  2.  3.]
```

4.4.3 Le broadcasting : calculer entre tableaux de formes différentes

Que se passe-t-il si l'on fait $A - m$ où A est une matrice 3×4 et m un vecteur de 4 nombres ? Mathématiquement, la soustraction n'est pas définie (les formes diffèrent) ; NumPy, lui, étire le plus petit tableau pour qu'il s'adapte : c'est le **broadcasting** (« diffusion »).

Reprenons l'exemple de Yasmine. Elle veut savoir, pour chaque semaine, **de combien chaque**

canal s'écarte de la moyenne de cette semaine. Moyenne de la semaine 1 : $(120 + 90 + 60)/3 = 90$. Semaine 2 : $(150 + 100 + 50)/3 = 100$. Semaine 3 : $(90 + 120 + 90)/3 = 100$. Semaine 4 : $(140 + 70 + 105)/3 = 105$. Donc $m = (90, 100, 100, 105)$ et, par exemple, Instagram en semaine 1 s'écarte de $120 - 90 = +30$.

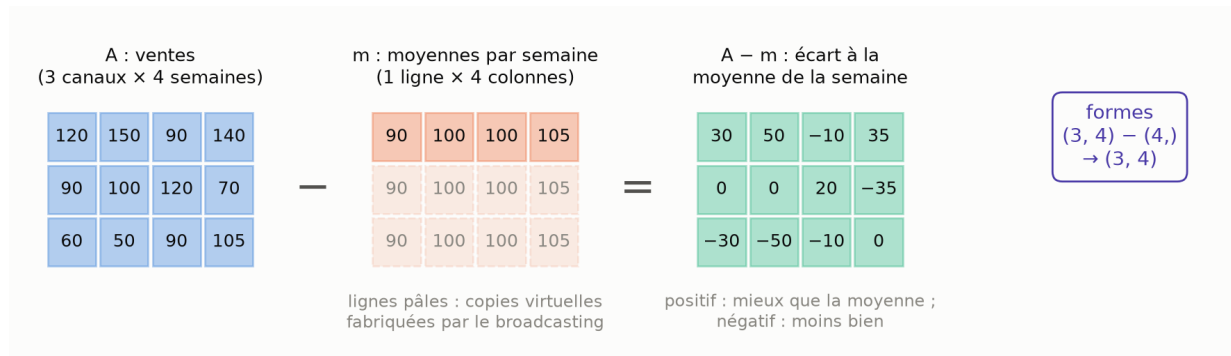


Figure 23: Le broadcasting : le vecteur m (4 nombres) est « étiré » sur les trois lignes de A , puis la soustraction se fait case par case.

```
m = A.mean(axis=0)      # moyenne de chaque colonne (axis=0 : on « écrase » les lignes)
print("moyennes par semaine :", m)
print(A - m)
```

```
moyennes par semaine : [ 90. 100. 100. 105.]
[[ 30.  50. -10.  35.]
 [  0.   0.  20. -35.]
 [-30. -50. -10.   0.]]
```

Le résultat reproduit les valeurs du dessin : 30 pour Instagram en semaine 1, -35 pour le Site en semaine 4, etc.

La règle du broadcasting. NumPy compare les formes **en partant de la droite**. Deux dimensions sont compatibles si elles sont **égales** ou si l'une des deux vaut 1 (elle est alors étirée). Une dimension manquante à gauche est comptée comme 1.

- $(3, 4)$ et $(4,)$: $4 = 4$ ✓, puis le 3 n'a pas de vis-à-vis $\rightarrow$ compatible, résultat $(3, 4)$;
- $(3, 4)$ et $(3, 1)$: 4 vs 1 ✓, $3 = 3$ ✓ $\rightarrow$ compatible, résultat $(3, 4)$;
- $(3, 4)$ et $(3,)$: 4 vs 3 ✗ $\rightarrow$ **erreur**.

Pour soustraire cette fois la **moyenne de chaque ligne** (la vente moyenne de chaque canal), il faut un vecteur *colonne* de forme $(3, 1)$. L'argument `keepdims=True` garde la dimension écrasée avec la taille 1, ce qui rend le broadcasting possible :

```
moy_canal = A.mean(axis=1, keepdims=True)
print(moy_canal.shape, "->", moy_canal.ravel())
print(A - moy_canal)
```

```
(3, 1) -> [125.    95.   76.25]
[[ -5.    25.  -35.   15. ]
 [ -5.     5.   25.  -25. ]
 [-16.25 -26.25  13.75  28.75]]
```

À la main : la moyenne d'Instagram est $(120 + 150 + 90 + 140)/4 = 125$, donc la première ligne devient $(-5, 25, -35, 15)$. Et si on oublie `keepdims` ? Un message d'erreur clair nous rappelle la règle :

```
try:
    A - A.mean(axis=1)          # forme (3,) au lieu de (3, 1)
except ValueError as e:
    print("ValueError :", e)
```

ValueError : operands could not be broadcast together with shapes (3,4) (3,)

Application : standardiser des colonnes. Au chapitre 1, nous avons vu que les algorithmes de modélisation aiment que les variables aient la même échelle. Centrer-réduire chaque colonne (soustraire sa moyenne, diviser par son écart-type) s'écrit sans boucle grâce au broadcasting : $(X - X.mean(axis=0)) / X.std(axis=0, ddof=1)$. Vous le ferez avec pandas à la section 4.4.7.

4.4.4 Agréger, trier, tirer au hasard

Les fonctions de résumé (sum, mean, std, min, max...) prennent un argument **axis** qui choisit la direction du calcul. Une mnémotechnique : *axis est la dimension qui disparaît*.

Appel	Ce qui disparaît	Résultat pour notre A (3 × 4)
A.sum()	tout	un seul nombre (le chiffre d'affaires total)
A.sum(axis=0)	les lignes	un total par semaine (4 nombres)
A.sum(axis=1)	les colonnes	un total par canal (3 nombres)

```
print("total général :", A.sum())
print("par semaine   :", A.sum(axis=0))
print("par canal     :", A.sum(axis=1))
ligne, colonne = divmod(int(A.argmax()), A.shape[1]) # argmax numérote les cases ligne après ligne
print("meilleure case :", A.max(), "en ligne", ligne, ", colonne", colonne)
```

```
total général : 1185
par semaine   : [270 300 300 315]
par canal     : [500 380 305]
meilleure case : 150 en ligne 0 , colonne 1
```

Vérification à la main : Instagram 120 + 150 + 90 + 140 = 500, Site 90 + 100 + 120 + 70 = 380, Boutique 60 + 50 + 90 + 105 = 305, soit 1 185 DT au total, ce qui est aussi la somme des totaux par semaine (270 + 300 + 300 + 315).

Autres opérations fréquentes :

```
ventes_instagram = A[0]
print("cumul       :", np.cumsum(ventes_instagram)) # somme cumulée
print("variation   :", np.diff(ventes_instagram))  # différences successives
print("tri croissant :", np.sort(ventes_instagram))
print("ordre des semaines (de la pire à la meilleure) :", np.argsort(ventes_instagram))
print("écart-type (n-1) :", round(ventes_instagram.std(ddof=1), 2))
```

```
cumul       : [120 270 360 500]
variation   : [ 30 -60  50]
tri croissant : [ 90 120 140 150]
ordre des semaines (de la pire à la meilleure) : [2 0 3 1]
écart-type (n-1) : 26.46
```

ddof encore. Comme pour np.std à la section 3.1.4, NumPy divise par n par défaut. Pour estimer la variance d'une population à partir d'un échantillon, il faut **ddof=1** (division par

$n - 1$). pandas, lui, utilise $n - 1$ par défaut : un écart entre `np.std(x)` et `serie.std()` n'est donc pas un bug.

Nombres aléatoires. On rencontre le générateur de nombres aléatoires depuis le chapitre 2 : on le crée une fois avec une **graine** (*seed*), puis on tire.

```
rng = np.random.default_rng(42)
jours = rng.normal(loc=120, scale=15, size=100_000) # 100 000 journées simulées, N(120 ; 15²)
print("moyenne simulée :", round(jours.mean(), 1))
print("part des jours à plus de 150 ventes :", round((jours > 150).mean(), 4))
```

```
moyenne simulée : 119.9
part des jours à plus de 150 ventes : 0.0233
```

La moyenne d'un masque booléen est la **proportion** de True : c'est la façon la plus économique d'estimer une probabilité par simulation. On retrouve (à très peu près) les 2,3 % calculés à la main au 2.2 pour un jour à plus de deux écarts-types au-dessus de la moyenne.

Enfin, NumPy sait faire l'algèbre linéaire du chapitre 1 : produit matriciel @, transposée .T, inverse, valeurs propres, résolution de système, etc.

```
M = np.array([[2.0, 1.0], [1.0, 3.0]])
b = np.array([5.0, 10.0])
print("solution de M x = b :", np.linalg.solve(M, b))
print("valeurs propres de M :", np.round(np.linalg.eigvalsh(M), 3))
```

```
solution de M x = b : [1. 3.]
valeurs propres de M : [1.382 3.618]
```

À la main : $2x + y = 5$ et $x + 3y = 10$ donnent $x = 1, y = 3$.

À retenir (NumPy). (1) tableau = type unique + forme ; (2) indexer avec [ligne, colonne], tranches a:b (fin exclue) et masques booléens ; (3) le **broadcasting** étire les dimensions de taille 1 ; (4) axis = la dimension qui disparaît ; (5) une tranche est une **vue** : `.copy()` pour travailler sans risque.

4.4.5 pandas : Series et DataFrame

NumPy ne connaît que des nombres rangés dans des cases numérotées. Mais une vraie table de données, ce sont des colonnes **nommées** (« montant », « canal ») de **types différents** (nombres, texte, dates), avec des lignes qu'on veut pouvoir **identifier**. C'est ce que fait pandas.

Deux objets suffisent pour commencer :

- la **Series** : une colonne, c'est-à-dire un tableau NumPy muni d'un **index** (une étiquette par valeur) et d'un nom ;
- le **DataFrame** : un tableau de plusieurs Series partageant le même index.

```
import numpy as np
import pandas as pd

pd.set_option("display.width", 110) # pour que les tableaux larges ne soient pas coupés à
↳ l'affichage
pd.set_option("display.max_columns", 20)

ventes = pd.Series([500, 380, 305], index=["Instagram", "Site", "Boutique"], name="ventes")
print(ventes)
print()
print("accès par étiquette :", ventes["Site"], "| par position :", ventes.iloc[2])
```

```
Instagram    500
Site         380
Boutique     305
Name: ventes, dtype: int64
```

accès par étiquette : 380 | par position : 305

Un DataFrame s'obtient, par exemple, à partir d'un dictionnaire « nom de colonne → valeurs » :

```
mini = pd.DataFrame({
    "canal": ["Instagram", "Site", "Boutique"],
    "ventes": [500, 380, 305],
    "ouvert_le_dimanche": [True, True, False],
})
print(mini)
print()
print(mini.dtypes)
```

```
   canal  ventes  ouvert_le_dimanche
0  Instagram    500                True
1     Site     380                True
2  Boutique    305                False
```

```
canal          str
ventes         int64
ouvert_le_dimanche  bool
dtype: object
```

Chaque colonne a son type : str (texte), int64 (entiers), bool (vrai/faux).

pandas 3.0 : le texte a désormais son propre type. Dans les versions de pandas antérieures à la 3.0, les colonnes de texte avaient le type object (« n'importe quoi »). Depuis la **version 3.0**, elles ont un type dédié, affiché str, plus rapide, plus économe en mémoire, et qui représente les valeurs manquantes de façon uniforme. Si vous lisez un vieux tutoriel qui affiche dtype: object pour une colonne de texte, ne vous inquiétez pas : seul l'affichage a changé. Ce livre a été exécuté avec pandas 3.0.

Charger le fichier de données. Voici le fichier `donnees/commandes.csv` du chapitre 3 :

```
df = pd.read_csv("donnees/commandes.csv")
print(df.shape)
print(df.head())
print()
df.info()
```

```
(400, 4)
   canal  montant  livraison  satisfaction
0  Boutique    44.8         0           4
1     Site    34.5         2           4
2  Instagram    88.2         5           4
3  Instagram    30.1         4           4
4  Boutique   110.1         0           5
```

```
<class 'pandas.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   canal      400 non-null      str
1   montant    400 non-null      float64
```

```

2  livraison      400 non-null    int64
3  satisfaction  400 non-null    int64
dtypes: float64(1), int64(2), str(1)
memory usage: 12.6 KB

```

`info()` est le meilleur premier réflexe : nombre de lignes, nom et type de chaque colonne, nombre de valeurs **non nulles** (ici, 400 partout : aucune valeur manquante) et mémoire utilisée.

Le fichier ne contient pas de **date**, or la plupart des vraies données de vente en ont une. Pour illustrer le travail sur les dates (4.4.12) et sur plusieurs tables (4.4.10), nous allons **ajouter deux colonnes simulées** : la date de chaque commande (réparties sur 20 semaines à partir du lundi 5 janvier 2026) et un numéro de client (120 clients possibles). Rappelons que tout est fictif et reproductible grâce à la graine :

```

rng = np.random.default_rng(7)
jours = rng.integers(0, 140, size=len(df)) # un jour tiré parmi 140 (= 20 semaines)
df["date"] = pd.Timestamp("2026-01-05") + pd.to_timedelta(jours, unit="D")
df["id_client"] = rng.integers(1, 121, size=len(df)) # clients numérotés de 1 à 120
df = df.sort_values("date").reset_index(drop=True) # on trie par date et on renumérote les lignes
df.insert(0, "id_commande", np.arange(1, len(df) + 1)) # une colonne identifiant, placée en premier
print(df.head(6))
print()
print(df.dtypes)

```

	id_commande	canal	montant	livraison	satisfaction	date	id_client
0	1	Site	49.9	8	2	2026-01-05	100
1	2	Instagram	21.5	3	4	2026-01-05	46
2	3	Boutique	34.2	0	4	2026-01-05	26
3	4	Site	103.0	3	5	2026-01-05	97
4	5	Instagram	26.7	4	4	2026-01-06	105
5	6	Boutique	37.4	0	4	2026-01-06	120

```

id_commande      int64
canal             str
montant          float64
livraison         int64
satisfaction      int64
date             datetime64[us]
id_client        int64
dtype: object

```

Notez le type `datetime64` de la colonne `date` : pandas sait que ce ne sont pas du texte, mais de vraies dates, avec lesquelles on peut calculer. Dernier réflexe, le résumé statistique :

```

print(df.describe().round(2))
print()
print(df["canal"].value_counts())

```

	id_commande	montant	livraison	satisfaction	date	id_client
count	400.00	400.00	400.00	400.00	400	400.00
mean	200.50	60.25	3.29	3.96	2026-03-18 06:25:12	61.87
min	1.00	8.60	0.00	1.00	2026-01-05 00:00:00	2.00
25%	100.75	34.18	0.00	3.00	2026-02-09 00:00:00	30.75
50%	200.50	51.00	3.00	4.00	2026-03-19 12:00:00	62.00
75%	300.25	75.82	5.00	5.00	2026-04-25 00:00:00	93.00
max	400.00	255.70	13.00	5.00	2026-05-24 00:00:00	120.00
std	115.61	38.02	2.56	0.80	NaN	34.71

canal	
Site	148
Instagram	138
Boutique	114

```
Name: count, dtype: int64
```

4.4.6 Sélectionner : colonnes, lignes, conditions

Pour sélectionner, on dispose de plusieurs outils, résumés dans le tableau ci-dessous. Prenons une toute petite table pour voir clairement ce qui se passe :

```
petit = df[["id_commande", "canal", "montant"]].head(5)
print(petit)
```

```
   id_commande  canal  montant
0            1    Site    49.9
1            2  Instagram    21.5
2            3   Boutique    34.2
3            4    Site   103.0
4            5  Instagram    26.7
```

Je veux...	J'écris	Remarque
une colonne	<code>petit["montant"]</code>	donne une Series
plusieurs colonnes	<code>petit[["canal", "montant"]]</code>	doublets crochets : une liste de noms
des lignes par position	<code>petit.iloc[1:3]</code>	<code>iloc</code> = <i>integer location</i> ; fin exclue
des lignes par étiquette	<code>petit.loc[1:3]</code>	<code>loc</code> = <i>label</i> ; fin incluse !
des lignes par condition	<code>petit[petit["montant"] > 50]</code>	masque booléen, comme en NumPy

```
print(petit["montant"].tolist())
print(petit[["canal", "montant"]].iloc[1:3])
print(petit.loc[1:3, ["canal", "montant"]])    # lignes d'étiquettes 1 à 3 INCLUSES
```

```
[49.9, 21.5, 34.2, 103.0, 26.7]
   canal  montant
1  Instagram    21.5
2   Boutique    34.2
   canal  montant
1  Instagram    21.5
2   Boutique    34.2
3     Site   103.0
```

iloc exclut la fin, loc l'inclut. `iloc[1:3]` renvoie les lignes 1 et 2 ; `loc[1:3]` renvoie les lignes 1, 2 et 3 (car on désigne des étiquettes, et on veut « de 1 jusqu'à 3 »). C'est l'une des étourderies les plus fréquentes.

Filtrer avec des conditions. Les opérateurs `&`, `|`, `~` demandent **des parenthèses** autour de chaque condition (car `&` s'évalue avant `>`) :

```
gros_insta = df[(df["canal"] == "Instagram") & (df["montant"] > 100)]
print("commandes Instagram de plus de 100 DT :", len(gros_insta))

# trois autres formes pratiques
print(df["canal"].isin(["Site", "Boutique"]).sum())
print(df["montant"].between(50, 100).sum())    # bornes incluses
print(df.query("canal == 'Site' and livraison >= 7").shape[0])
```

```
commandes Instagram de plus de 100 DT : 11
262
```

```
153
21
```

La méthode `query` accepte une condition écrite comme une phrase : elle se lit mieux sur des conditions longues. Pour **trier** et chercher les extrêmes :

```
print(df.sort_values("montant", ascending=False).head(3)[["id_commande", "canal", "montant"]])
print(df.nsmallest(3, "montant")[["id_commande", "canal", "montant"]])
```

```
   id_commande canal  montant
380         381  Site    255.7
102         103  Site    243.8
198         199  Site    217.1
   id_commande canal  montant
289         290 Instagram    8.6
366         367 Instagram   10.3
326         327 Instagram   10.7
```

4.4.7 Créer et transformer des colonnes

Une nouvelle colonne s'obtient en l'assignant, avec des opérations **vectorisées** (comme dans NumPy, sans boucle) :

```
df["livraison_rapide"] = df["livraison"] <= 3
df["gros_panier"] = np.where(df["montant"] >= 100, "oui", "non")
df["canal_court"] = df["canal"].str[:3].str.upper() # méthodes .str : opérations sur le texte
print(df[["canal", "canal_court", "montant", "gros_panier", "livraison_rapide"]].head(4))
```

```
   canal canal_court  montant gros_panier  livraison_rapide
0    Site      SIT     49.9         non             False
1 Instagram      INS     21.5         non             True
2  Boutique      BOU     34.2         non             True
3    Site      SIT    103.0         oui             True
```

Le préfixe `.str` donne accès à toutes les méthodes de texte (`upper`, `lower`, `contains`, `replace`, `split`...) appliquées à **chaque élément** de la colonne.

Découper une variable continue en classes. `pd.cut` fabrique des classes de bornes choisies, `pd.qcut` des classes d'effectifs égaux (par quantiles). Par exemple, quatre tranches de panier : « petit » (moins de 30 DT), « moyen » (30 à 60), « grand » (60 à 100) et « très grand » (plus de 100) :

```
bornes = [0, 30, 60, 100, np.inf]
noms = ["petit", "moyen", "grand", "très grand"]
df["tranche"] = pd.cut(df["montant"], bins=bornes, labels=noms)
print(df["tranche"].value_counts().reindex(noms))
print()
quartiles = pd.qcut(df["montant"], q=4, labels=["Q1", "Q2", "Q3", "Q4"])
print(quartiles.value_counts().sort_index()) # ~100 commandes dans chaque quartile par
↳ construction
```

```
tranche
petit      76
moyen     160
grand     112
très grand  52
Name: count, dtype: int64
```

```
montant
Q1    100
Q2    100
```



```
Q3    100
Q4    100
Name: count, dtype: int64
```

Remarquez la nuance : avec `cut`, les classes sont **définies par vous** et leurs effectifs sont inégaux ; avec `qcut`, ce sont les effectifs qui sont **égaux** ($400 / 4 = 100$) et les bornes qui s'adaptent aux données.

Remplacer des valeurs selon un dictionnaire avec `map` :

```
etiquettes = {1: "très mécontent", 2: "mécontent", 3: "neutre", 4: "content", 5: "très content"}
df["avis"] = df["satisfaction"].map(etiquettes)
print(df["avis"].value_counts().reindex(list(etiquettes.values())))
```

```
avis
très mécontent    1
mécontent         15
neutre            85
content          195
très content     104
Name: count, dtype: int64
```

Standardiser (centrer-réduire), comme annoncé à la fin de 4.4.3, se fait sur une colonne entière :

```
z = (df["montant"] - df["montant"].mean()) / df["montant"].std()
print("moyenne de z nulle :", bool(np.isclose(z.mean(), 0)), "| écart-type de z :", round(z.std(), 6))
print("commandes à plus de 3 écarts-types de la moyenne :", int((z.abs() > 3).sum()))
```

```
moyenne de z nulle : True | écart-type de z : 1.0
commandes à plus de 3 écarts-types de la moyenne : 7
```

Par construction, `z` a une moyenne nulle et un écart-type de 1. Sept commandes dépassent 3 écarts-types. Si les montants suivaient une loi normale, on n'en attendrait qu'une **seule** sur 400 environ (la probabilité d'être à plus de 3 écarts-types est de 0,27 %, d'après les repères du 2.2). En trouver sept confirme ce que nous avons vu au 3.1.5 : la distribution des montants a une **queue lourde à droite**.

apply : à garder en dernier recours. Si une transformation n'existe pas en version vectorisée, `df["col"].apply(ma_fonction)` appelle votre fonction **ligne par ligne** : pratique, mais lent (une boucle Python déguisée). Réflexe : chercher d'abord une opération vectorisée (`.str`, `np.where`, `cut`, `map`, opérateurs arithmétiques...). Sur nos 400 lignes la différence est invisible ; pour la mesurer, on répète les montants 500 fois (200 000 lignes) :

```
import time

gros = pd.concat([df["montant"]] * 500, ignore_index=True)      # 200 000 montants

t0 = time.perf_counter()
a = gros.apply(lambda v: v * 1.19 if v > 50 else v)             # une boucle Python déguisée
t1 = time.perf_counter()
b = np.where(gros > 50, gros * 1.19, gros)                       # vectorisé
t2 = time.perf_counter()
print("même résultat :", np.allclose(a, b))
print("apply plus lent que la version vectorisée :", (t1 - t0) > (t2 - t1))
```

```
même résultat : True
apply plus lent que la version vectorisée : True
```

4.4.8 Copie, vue et pièges de pandas 3.0

Une question revient sans cesse : *si je modifie un morceau de mon tableau, est-ce que je modifie aussi l'original ?* Dans les versions anciennes de pandas, la réponse dépendait de détails obscurs (c'était le fameux `SettingWithCopyWarning`). **Depuis pandas 3.0, la règle est simple : le *copy-on-write* (copie à l'écriture).** Tout objet dérivé d'un autre se comporte comme **une copie indépendante** ; modifier l'un ne modifie jamais l'autre.

```
montants = df["montant"]           # une Series dérivée de df
montants.iloc[0] = -1             # on la modifie...
print("df['montant'] au premier rang :", df["montant"].iloc[0], "(inchangé : montants est indépendant  
↪ de df)")
```

```
df['montant'] au premier rang : 49.9 (inchangé : montants est indépendant de df)
```

La conséquence la plus importante : l'**assignation en chaîne** (`df[condition]["colonne"] = valeur`) **ne fonctionne plus**, car `df[condition]` fabrique une copie temporaire que l'on modifie puis que l'on jette. pandas 3.0 émet même un avertissement. Vérifions-le :

```
import warnings

copie = df.copy()
with warnings.catch_warnings(record=True) as avert:
    warnings.simplefilter("always")
    copie[copie["canal"] == "Site"]["montant"] = 0      # [X] assignation en chaîne
print("avertissement reçu :", avert[0].category.__name__)
print("montants mis à 0 par la mauvaise écriture :", int((copie["montant"] == 0).sum()))

copie.loc[copie["canal"] == "Site", "montant"] = 0     # [OK] une seule opération avec loc
print("montants mis à 0 par la bonne écriture :", int((copie["montant"] == 0).sum()))
```

```
avertissement reçu : ChainedAssignmentError
montants mis à 0 par la mauvaise écriture : 0
montants mis à 0 par la bonne écriture : 148
```

La bonne écriture, toujours : `df.loc[condition, "colonne"] = valeur`. Une seule opération, qui désigne à la fois les lignes et la colonne. Et pour *vraiment* garder une copie intacte avant de modifier : `df2 = df.copy()`.

4.4.9 Regrouper : le « split-apply-combine » (groupby)

C'est l'outil le plus important de pandas pour répondre à des questions du type : « *quel est le montant moyen par canal ?* », « *combien de commandes par semaine ?* » La méthode s'appelle **découper – appliquer – combiner** (*split-apply-combine*) :

1. **découper** le tableau en groupes (une valeur de canal = un groupe) ;
2. **appliquer** un calcul à chaque groupe (moyenne, somme, comptage...) ;
3. **combiner** les résultats dans un nouveau tableau.

Faisons-le à la main sur six commandes :

Commande	Canal	Montant
1	Instagram	20
2	Site	30
3	Instagram	40
4	Site	50
5	Boutique	100

Commande	Canal	Montant
6	Site	70

Groupes : Instagram {20, 40}, Site {30, 50, 70}, Boutique {100}. Moyennes : 30, 50 et 100. Comptes : 2, 3, 1. Voici le même calcul en code (pandas range les groupes par ordre alphabétique) :

```
six = pd.DataFrame({"canal": ["Instagram", "Site", "Instagram", "Site", "Boutique", "Site"],
                    "montant": [20, 30, 40, 50, 100, 70]})
print(six.groupby("canal")["montant"].agg(["mean", "count"]))
```

```
      mean  count
canal
Boutique 100.0     1
Instagram 30.0     2
Site      50.0     3
```

Passons aux 400 commandes. L'**agrégation nommée** donne des colonnes lisibles : `nom=("colonne", "fonction")`.

```
resume = df.groupby("canal").agg(
    commandes=("montant", "count"),
    ca=("montant", "sum"),
    panier_moyen=("montant", "mean"),
    panier_medan=("montant", "median"),
    satisfaction=("satisfaction", "mean"),
).round(2)
print(resume.sort_values("ca", ascending=False))
```

```
      commandes      ca  panier_moyen  panier_medan  satisfaction
canal
Site           148  8806.5           59.50          49.50           3.79
Boutique        114  8528.3           74.81          64.85           4.49
Instagram       138  6763.5           49.01          41.50           3.72
```

On peut regrouper selon **plusieurs** critères, ou demander une répartition en proportions :

```
print(df.groupby(["canal", "gros_panier"]).size().unstack())      # unstack : un niveau d'index
↳ devient colonnes
print()
print(pd.crosstab(df["canal"], df["tranche"], normalize="index").round(2))  # proportions par ligne
```

```
gros_panier  non  oui
canal
Boutique      87   27
Instagram     127   11
Site          134   14

tranche      petit  moyen  grand  très grand
canal
Boutique      0.07   0.37   0.32         0.24
Instagram     0.32   0.38   0.22         0.08
Site          0.16   0.44   0.30         0.09
```

`crosstab` (tableau croisé) est un raccourci pour compter les effectifs de deux variables qualitatives ; avec `normalize="index"` chaque ligne est ramenée à 1, ce qui répond à « *parmi* les commandes Instagram, quelle part de petits paniers ? ». Pour des tableaux de synthèse à deux entrées sur une variable numérique, on a `pivot_table` :

```
print(df.pivot_table(index="canal", columns="gros_panier", values="satisfaction",
↳ aggfunc="mean").round(2))
```

gros_panier	non	oui
canal		
Boutique	4.49	4.48
Instagram	3.72	3.64
Site	3.80	3.71

Enfin transform calcule un résultat **par groupe** mais le renvoie **aligné sur les lignes d'origine**, ce qui permet de comparer chaque commande à son groupe :

```
df["ecart_moy_canal"] = df["montant"] - df.groupby("canal")["montant"].transform("mean")
print(df[["canal", "montant", "ecart_moy_canal"]].head(4).round(1))
print("moyenne des écarts nulle dans chaque canal :",
      bool(np.allclose(df.groupby("canal")["ecart_moy_canal"].mean(), 0)))
```

	canal	montant	ecart_moy_canal
0	Site	49.9	-9.6
1	Instagram	21.5	-27.5
2	Boutique	34.2	-40.6
3	Site	103.0	43.5

moyenne des écarts nulle dans chaque canal : True

À retenir (groupby). `df.groupby(clé)[colonne].fonction()` : découper, appliquer, combiner. `agg` pour plusieurs résumés, `size` pour compter les lignes, `transform` pour ré-aligner un résultat de groupe sur chaque ligne, `pivot_table` et `crosstab` pour des tableaux croisés.

4.4.10 Combiner plusieurs tables : merge et concat

Dans la vraie vie, l'information est **répartie sur plusieurs tables** : la liste des commandes d'un côté, celle des clients de l'autre (nous verrons au chapitre 5 pourquoi, et comment SQL fait la même chose avec JOIN). Le travail s'appelle une **jointure** : on associe les lignes qui partagent la même clé.

Créons la table des clients : 125 clients, chacun avec une ville. (Les clients 121 à 125 n'ont, par construction, jamais commandé ; comme les commandes ont été attribuées au hasard à des clients de 1 à 120, quelques autres clients n'auront rien commandé non plus. Cela nous servira.)

```
rng_c = np.random.default_rng(11)
clients = pd.DataFrame({
    "id_client": np.arange(1, 126),
    "ville": rng_c.choice(["Tunis", "Sfax", "Sousse", "Nabeul", "Bizerte"], size=125, p=[0.4, 0.2,
        0.2, 0.1, 0.1]),
})
print(clients.head(3))
print("clients :", len(clients), "| commandes :", len(df))
```

	id_client	ville
0	1	Tunis
1	2	Sfax
2	3	Sousse

clients : 125 | commandes : 400

Voici d'abord un exemple minuscule pour comprendre les types de jointure. Deux commandes (clients 1 et 9) et deux fiches clients (1 et 2) :

```
cmd = pd.DataFrame({"id_client": [1, 9], "montant": [50, 80]})
fiche = pd.DataFrame({"id_client": [1, 2], "ville": ["Tunis", "Sfax"]})
for how in ["inner", "left", "outer"]:
    print(f"--- how='{how}'")
    print(cmd.merge(fiche, on="id_client", how=how))
```

```

--- how='inner'
  id_client  montant  ville
0          1        50  Tunis
--- how='left'
  id_client  montant  ville
0          1        50  Tunis
1          9        80   NaN
--- how='outer'
  id_client  montant  ville
0          1      50.0  Tunis
1          2       NaN   Sfax
2          9      80.0   NaN

```

- **inner** : seulement les clés présentes **des deux côtés** (client 1) ;
- **left** : toutes les lignes de la table de gauche ; on met NaN (valeur manquante) quand il n'y a pas de correspondance (client 9 sans fiche) ;
- **outer** : tout ce qui existe d'un côté **ou** de l'autre.

Quel how choisir ? Dans 90 % des cas : **left**, avec votre table « principale » (les commandes) à gauche. On garde alors *toutes* les commandes et on y ajoute des informations, sans en perdre en route. Et on **vérifie** toujours le nombre de lignes avant/après.

Sur nos données :

```

cmd_ville = df.merge(clients, on="id_client", how="left")
print("lignes avant/après la jointure :", len(df), len(cmd_ville))
print("commandes sans ville connue   :", int(cmd_ville["ville"].isna().sum()))
print()
print(cmd_ville.groupby("ville")["montant"].agg(["count", "mean"]).round(1).sort_values("count",
↪ ascending=False))

```

```

lignes avant/après la jointure : 400 400
commandes sans ville connue   : 0

```

	count	mean
ville		
Tunis	176	59.6
Sousse	92	59.7
Sfax	71	63.6
Nabeul	42	56.0
Bizerte	19	65.5

Quels sont les clients qui n'ont **jamais** commandé ? `indicator=True` ajoute une colonne `_merge` qui dit d'où vient chaque ligne (`both` : présent des deux côtés ; `left_only` : seulement dans la table de gauche) :

```

test = clients.merge(df[["id_client"]].drop_duplicates(), on="id_client", how="left", indicator=True)
print(test["_merge"].value_counts())
print("sans commande :", test.loc[test["_merge"] == "left_only", "id_client"].tolist())

```

```

_merge
both          114
left_only      11
right_only      0
Name: count, dtype: int64
sans commande : [1, 6, 43, 50, 67, 84, 121, 122, 123, 124, 125]

```

Onze clients n'ont jamais commandé : les cinq prévus (121 à 125) et six clients de la plage 1 à 120 que le hasard n'a pas tirés (1, 6, 43, 50, 67 et 84). Le comptage `both` (114) + `left_only` (11) retombe bien sur nos 125 clients.

Le piège n°1 des jointures : l'explosion du nombre de lignes. Si la clé est **dupliquée** dans la table de droite, chaque ligne de gauche est recopiée autant de fois. Imaginez que la fiche du client 1 ait été saisie deux fois : sa commande apparaîtrait en double, et le chiffre d'affaires serait faux sans qu'aucune erreur ne soit signalée. Le paramètre **validate** déclenche une erreur dans ce cas : "m:1" signifie « plusieurs lignes à gauche pour une seule à droite ».

```
fiche_doublon = pd.DataFrame({"id_client": [1, 1], "ville": ["Tunis", "Tunis"]})
print("sans validate :", len(cmd.merge(fiche_doublon, on="id_client", how="left")), "lignes au lieu
↳ de 2")
try:
    cmd.merge(fiche_doublon, on="id_client", how="left", validate="m:1")
except Exception as e:
    print(type(e).__name__, ":", str(e).split("\n")[0])
```

```
sans validate : 3 lignes au lieu de 2
MergeError : Merge keys are not unique in right dataset; not a many-to-one merge
```

Pour **empiler** des tables de même structure (les commandes de janvier et celles de février, par exemple), on utilise concat :

```
janvier = df[df["date"] < "2026-02-01"]
fevrier = df[(df["date"] >= "2026-02-01") & (df["date"] < "2026-03-01")]
empile = pd.concat([janvier, fevrier])
print(len(janvier), "+", len(fevrier), "=", len(empile))
```

```
75 + 69 = 144
```

4.4.11 Valeurs manquantes

Dans les données réelles, il manque toujours quelque chose : un client n'a pas laissé de note, un capteur est tombé en panne, une case n'a pas été remplie. pandas représente ces trous par **NaN** (*not a number*) pour les nombres, et par **NaT** pour les dates. Créons une copie de nos données où 30 notes de satisfaction sont perdues :

```
rng_m = np.random.default_rng(3)
trous = rng_m.choice(len(df), size=30, replace=False)
dfm = df[["id_commande", "canal", "montant", "satisfaction"]].copy()
dfm["satisfaction"] = dfm["satisfaction"].astype(float)      # float, car un entier NumPy ne peut pas
↳ contenir NaN
dfm.loc[trous, "satisfaction"] = np.nan
print(dfm["satisfaction"].isna().sum(), "notes manquantes sur", len(dfm))
print(dfm["satisfaction"].isna().groupby(dfm["canal"]).sum())
```

```
30 notes manquantes sur 400
canal
Boutique      7
Instagram    10
Site          13
Name: satisfaction, dtype: int64
```

D'abord, **comment les calculs se comportent-ils ?** Sur un exemple à la main : les notes 4, 5, NaN, 3 ont pour moyenne $(4 + 5 + 3)/3 = 4$ (pandas **ignore** le NaN, il ne le compte pas comme 0 ; sinon on trouverait $12/4 = 3$).

```
notes = pd.Series([4, 5, np.nan, 3])
print("moyenne :", notes.mean(), "| effectif :", notes.count(), "| taille :", len(notes))
print("somme avec skipna=False :", notes.sum(skipna=False))      # un seul NaN contamine la somme
```

```
moyenne : 4.0 | effectif : 3 | taille : 4
somme avec skipna=False : nan
```

Piège. Comparer avec `==` ne détecte pas un NaN (`NaN == NaN` est faux, par convention). Utilisez `isna()` / `notna()`.

Que faire des trous ? Trois stratégies, à choisir selon le contexte :

```
a_supprimer = dfm.dropna(subset=["satisfaction"]) # 1. supprimer les lignes
↳ incomplètes
remplie_med = dfm["satisfaction"].fillna(dfm["satisfaction"].median()) # 2. remplacer par une valeur
↳ « raisonnable »
remplie_canal = dfm["satisfaction"].fillna( # 3. remplacer par la médiane
↳ du groupe
    dfm.groupby("canal")["satisfaction"].transform("median"))

print("lignes après dropna :", len(a_supprimer))
print("moyenne exacte (données complètes) :", round(df["satisfaction"].mean(), 3))
print("moyenne avec trous (ignorés) :", round(dfm["satisfaction"].mean(), 3))
print("moyenne après remplissage (médiane):", round(remplie_med.mean(), 3))
print("moyenne après remplissage (par canal):", round(remplie_canal.mean(), 3))
```

```
lignes après dropna : 370
moyenne exacte (données complètes) : 3.965
moyenne avec trous (ignorés) : 3.984
moyenne après remplissage (médiane): 3.985
moyenne après remplissage (par canal): 4.003
```

Lisons les résultats. La vraie moyenne (calculée avant de perdre les 30 notes) est **3,965**. En ignorant simplement les trous, on trouve 3,984 ; avec un remplissage par la médiane globale, 3,985 ; avec la médiane de chaque canal, 4,003. Les écarts sont **petits** parce que nous avons effacé les notes **au hasard** (30 tirées au sort). Si les notes manquantes avaient été celles des clients mécontents, toutes les méthodes auraient surestimé la satisfaction, et d'autant plus que les trous seraient nombreux.

Ce qu'il faut comprendre. Il n'y a pas de bonne réponse universelle. Supprimer des lignes est simple mais perd de l'information, et **biaise** les résultats si les données manquantes ne sont pas dues au hasard (par exemple, si les clients mécontents répondent moins souvent). Remplir par la moyenne ou la médiane garde les effectifs mais **écrase la variabilité** (tous les trous prennent la même valeur). Retenez surtout un réflexe : **toujours compter les manquants** (`isna().sum()`) **avant d'analyser**, et se demander *pourquoi* ils manquent. Les méthodes d'imputation plus fines viendront dans la suite de la série.

Enfin, pandas propose des types à **valeurs manquantes natives** (Int64 avec un grand « I ») qui permettent de garder des entiers malgré les trous :

```
print(pd.Series([1, 2, None]).dtype, "|", pd.Series([1, 2, None], dtype="Int64").dtype)
```

```
float64 | Int64
```

4.4.12 Travailler avec des dates

Avec une colonne de vrais **dates** (type `datetime64`), pandas offre un accès `.dt` aux composantes (année, mois, jour de la semaine...), de l'arithmétique et des regroupements par période.

```
print(df["date"].min(), "->", df["date"].max())
print("durée couverte :", df["date"].max() - df["date"].min())
```

```
df["jour_semaine"] = df["date"].dt.dayofweek      # 0 = lundi ... 6 = dimanche
df["mois"] = df["date"].dt.month
print(df[["date", "jour_semaine", "mois"]].head(4))
```

```
2026-01-05 00:00:00 -> 2026-05-24 00:00:00
durée couverte : 139 days 00:00:00
   date  jour_semaine  mois
0 2026-01-05         0     1
1 2026-01-05         0     1
2 2026-01-05         0     1
3 2026-01-05         0     1
```

Si vos dates sont lues comme du **texte** (cas fréquent à l'import d'un fichier), on les convertit avec `pd.to_datetime`, en précisant le format pour éviter l'ambiguïté jour/mois :

```
textes = pd.Series(["05/01/2026", "12/01/2026", "03/02/2026"])
dates = pd.to_datetime(textes, format="%d/%m/%Y")
print(dates.dt.month.tolist(), "<- 3 février = mois 2, comme attendu")
```

```
[1, 1, 2] <- 3 février = mois 2, comme attendu
```

Pour **regrouper par semaine**, on convertit chaque date en période hebdomadaire (du lundi au dimanche) et on prend son premier jour :

```
df["semaine"] = df["date"].dt.to_period("W").dt.start_time      # le lundi de la semaine de chaque
↳ commande
print(df[["date", "semaine"]].head(3))
print("nombre de semaines distinctes :", df["semaine"].nunique())
```

```
   date  semaine
0 2026-01-05 2026-01-05
1 2026-01-05 2026-01-05
2 2026-01-05 2026-01-05
nombre de semaines distinctes : 20
```

Vérifions sur un exemple. Le 5 janvier 2026 est un lundi : la semaine du 5 au 11 janvier est donc repérée par le **5 janvier**. Une commande du jeudi 8 janvier doit avoir pour semaine le

```
test = pd.Series(pd.to_datetime(["2026-01-05", "2026-01-08", "2026-01-11", "2026-01-12"]))
print(pd.DataFrame({"date": test, "jour": test.dt.day_name(), "semaine":
↳ test.dt.to_period("W").dt.start_time}))
```

```
   date  jour  semaine
0 2026-01-05  Monday 2026-01-05
1 2026-01-08  Thursday 2026-01-05
2 2026-01-11  Sunday 2026-01-05
3 2026-01-12  Monday 2026-01-12
```

Dernier outil, la **moyenne mobile** (*rolling*), qui lisse une série en moyennant les k dernières valeurs. À la main : pour les ventes 10, 20, 30, 40, la moyenne mobile sur 2 périodes vaut NaN, 15, 25, 35 (la première valeur n'a pas assez d'historique).

```
print(pd.Series([10, 20, 30, 40]).rolling(2).mean().tolist())
```

```
[nan, 15.0, 25.0, 35.0]
```


4.4.13 ■ Petite application : le rapport hebdomadaire de Yasmine

Rassemblons tout ce que nous venons d'apprendre dans une vraie **mini-application** : un petit programme qui produit, pour une semaine donnée, le rapport que Yasmine lit chaque lundi. D'abord le **tableau de bord hebdomadaire** : une ligne par semaine.

```
heβδο = df.groupby("semaine").agg(
    commandes=("montant", "count"),
    ca=("montant", "sum"),
    panier_moyen=("montant", "mean"),
    part_instagram=("canal", lambda s: (s == "Instagram").mean()),
).round(2)
heβδο["ca_lisse"] = heβδο["ca"].rolling(4).mean().round(1) # moyenne mobile sur 4 semaines
print(heβδο.head(6))
print("...")
print(heβδο.tail(3))
```

semaine	commandes	ca	panier_moyen	part_instagram	ca_lisse
2026-01-05	20	1082.4	54.12	0.25	NaN
2026-01-12	19	1083.0	57.00	0.53	NaN
2026-01-19	18	1403.0	77.94	0.22	NaN
2026-01-26	20	750.7	37.54	0.40	1079.8
2026-02-02	21	1447.6	68.93	0.33	1171.1
2026-02-09	14	975.8	69.70	0.21	1144.3
...					
semaine	commandes	ca	panier_moyen	part_instagram	ca_lisse
2026-05-04	20	1482.8	74.14	0.35	1162.0
2026-05-11	23	1306.3	56.80	0.39	1225.5
2026-05-18	27	1585.4	58.72	0.37	1433.7

(La fonction anonyme `lambda s: (s == "Instagram").mean()` calcule, dans chaque semaine, la proportion de commandes Instagram, grâce à l'astuce « moyenne d'un masque » vue en 4.4.4.) Puis une **fonction** qui rédige le rapport d'une semaine :

```
def rapport_heβδο(df, debut):
    """Rapport de la semaine commençant le lundi `debut` (texte 'AAAA-MM-JJ')."""
    debut = pd.Timestamp(debut)
    sem = df[df["semaine"] == debut]
    if sem.empty:
        return f"Aucune commande la semaine du {debut:%d/%m/%Y}."
    precedente = df[df["semaine"] == debut - pd.Timedelta(weeks=1)]
    ca, ca_prec = sem["montant"].sum(), precedente["montant"].sum()
    par_canal = sem.groupby("canal")["montant"].sum().sort_values(ascending=False)
    top = sem.groupby("id_client")["montant"].sum().nlargest(3)

    lignes = [f"Semaine du {debut:%d/%m/%Y}",
              f" commandes : {len(sem)} | panier moyen : {sem['montant'].mean():.2f} DT",
              f" chiffre d'affaires : {ca:.2f} DT"]
    if ca_prec > 0:
        lignes[-1] += f" ({(ca / ca_prec - 1) * 100:+.1f} % vs semaine précédente)"
    lignes.append(" par canal : " + ", ".join(f"{c} {v:.0f} DT" for c, v in par_canal.items()))
    lignes.append(" meilleurs clients : " + ", ".join(f"n°{i} ({v:.0f} DT)" for i, v in top.items()))
    note = sem["satisfaction"].mean()
    lignes.append(f" satisfaction moyenne : {note:.2f}/5 + (" (!) à surveiller" if note < 3.5 else ""))
    return "\n".join(lignes)

print(rapport_heβδο(df, "2026-03-02"))
print()
print(rapport_heβδο(df, "2026-05-18"))
print()
```

```
print(rapport_hebdo(df, "2025-01-06"))      # une semaine sans données

# contrôle croisé : le rapport et le tableau hebdomadaire doivent donner les mêmes nombres
print(hebdo.loc["2026-03-02", ["commandes", "ca"]].tolist())
```

```
Semaine du 02/03/2026
commandes : 23 | panier moyen : 72.96 DT
chiffre d'affaires : 1678.00 DT (+59.4 % vs semaine précédente)
par canal : Site 707 DT, Boutique 502 DT, Instagram 469 DT
meilleurs clients : n°42 (189 DT), n°7 (170 DT), n°83 (160 DT)
satisfaction moyenne : 3.78/5
```

```
Semaine du 18/05/2026
commandes : 27 | panier moyen : 58.72 DT
chiffre d'affaires : 1585.40 DT (+21.4 % vs semaine précédente)
par canal : Site 682 DT, Instagram 465 DT, Boutique 438 DT
meilleurs clients : n°9 (256 DT), n°22 (237 DT), n°93 (146 DT)
satisfaction moyenne : 3.85/5
```

```
Aucune commande la semaine du 06/01/2025.
[23.0, 1678.0]
```

Le contrôle croisé final confirme que le rapport et le tableau hebdomadaire disent la même chose (23 commandes et 1 678 DT pour la semaine du 2 mars). Ce petit programme utilise presque tout : filtrage, groupby, tri, dates, formatage. Il est surtout **réutilisable** : la semaine prochaine, Yasmine changera la date et obtiendra le nouveau rapport sans rien refaire. C'est ce qui sépare une analyse « à la souris » d'une analyse reproductible.

À retenir (pandas).

1. `read_csv` → `head()`, `info()`, `describe()` : toujours **regarder** avant de calculer ;
2. sélection : `df["col"]`, `df[["a", "b"]]`, `loc` (étiquettes, fin incluse), `iloc` (positions, fin exclue), masques avec `&`, `|`, `~` **et parenthèses** ;
3. écrire `df.loc[condition, "col"] = valeur`, jamais d'assignation en chaîne (copy-on-write de pandas 3.0) ;
4. `groupby(...).agg(...)` : découper, appliquer, combiner ;
5. `merge(..., how="left", validate="m:1")` : toujours vérifier le nombre de lignes avant/après ;
6. compter les manquants avec `isna().sum()` *avant* de choisir quoi en faire ;
7. des dates en type `datetime64` (`to_datetime`), puis `.dt`, `to_period`, `rolling`.

Pour s'entraîner : les exercices de la section 4.9 reprennent ces notions.

4.5 Premières visualisations

Intuition. Un tableau de 400 lignes, personne ne le « lit » vraiment ; un graphique bien choisi, on le comprend en trois secondes. Au 3.1 nous avons vu avec le quartet d'Anscombe que des données très différentes peuvent avoir **les mêmes statistiques** : seul le dessin révèle la différence. Visualiser n'est donc pas de la décoration, c'est un **outil de réflexion** (on dessine pour *comprendre*) et un **outil de communication** (on dessine pour *convaincre* sans tromper). Cette section a trois objectifs : savoir **quel graphique choisir** selon la question (4.5.1), savoir **le tracer** avec les trois bibliothèques les plus utilisées (matplotlib et pandas, seaborn, puis ggplot2 en R : 4.5.2 à 4.5.5), et savoir **éviter les graphiques trompeurs** (4.5.6).

Pour la suite du livre. Nous ne ferons ici que les graphiques fondamentaux. Les graphiques interactifs, les cartes ou les tableaux de bord complets sont abordés dans la série Data Analyst.

Pour être autonome, cette section recharge les données de 4.4 (même graine, mêmes résultats) et règle l'apparence des figures du livre : traits fins, grille discrète, pas de cadre inutile.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

BLEU, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"    # la palette du livre
COULEURS = {"Instagram": BLEU, "Site": ORANGE, "Boutique": AQUA}

plt.rcParams.update({
    "figure.facecolor": "white", "axes.facecolor": "white",
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "axes.titlesize": 11, "axes.labelcolor": "#52514e", "legend.frameon": False,
})

# le jeu de données de 4.4 (identique : mêmes graines)
df = pd.read_csv("donnees/commandes.csv")
rng = np.random.default_rng(7)
jours = rng.integers(0, 140, size=len(df))
df["date"] = pd.Timestamp("2026-01-05") + pd.to_timedelta(jours, unit="D")
df["id_client"] = rng.integers(1, 121, size=len(df))
df = df.sort_values("date").reset_index(drop=True)
df["semaine"] = df["date"].dt.to_period("W").dt.start_time
hebdo = df.groupby("semaine").agg(commandes=("montant", "count"), ca=("montant", "sum"))
print(df.shape, "|", len(hebdo), "semaines | CA total :", round(df["montant"].sum(), 1), "DT")

(400, 7) | 20 semaines | CA total : 24098.3 DT
```

4.5.1 Quel graphique pour quelle question ?

La première erreur du débutant est de commencer par se demander « quel joli graphique ? ». La bonne démarche est inverse : on part de la **question**, puis du **type des variables** (revoyez le tableau du 3.1.1), et le graphique s'impose presque tout seul.

La question	Les variables	Le graphique	Pourquoi
Comment se répartissent les montants ?	1 quantitative	histogramme (ou boîte à moustaches)	montre la forme : symétrie, queue, valeurs atypiques
Quelle part de chaque canal ?	1 qualitative	diagramme en barres (trié)	on compare des longueurs alignées, ce que l'œil fait très bien
Quel canal vend le plus ?	1 qualitative + 1 quantitative	barres (une valeur par groupe) ou boîtes à moustaches (toute la distribution)	comparer des groupes
Comment évolue le chiffre d'affaires ?	1 quantitative + le temps	courbe	la ligne relie les points : on voit la tendance
Les retards de livraison font-ils baisser la satisfaction ?	2 quantitatives	nuage de points	montre la forme de la relation, pas seulement un nombre

La question	Les variables	Le graphique	Pourquoi
Comment se croisent deux variables qualitatives ?	2 qualitatives	carte de chaleur (<i>heatmap</i>) d'un tableau croisé	couleur = fréquence

Trois questions avant de tracer. (1) *Quel message* veux-je faire passer en une phrase ? (2) *Quelles variables* sont en jeu, et de quel type ? (3) *Qui va lire* ce graphique, en combien de temps ? Un graphique pour votre propre exploration peut être brouillon ; un graphique pour un patron pressé doit contenir **un message et un seul**.

Et le camembert ? Il est partout, mais il est un mauvais choix dès qu'il y a plus de trois parts : l'œil compare mal des angles et des aires, bien mieux des **longueurs**. Pour montrer 3 canaux, un camembert est acceptable ; pour 8 produits, des barres triées sont toujours plus lisibles. (Les camemberts « en 3D » sont à proscrire absolument, voir 4.5.6.)

4.5.2 Anatomie d'un graphique matplotlib

matplotlib est la bibliothèque de base de la visualisation en Python : presque toutes les autres (pandas, seaborn...) s'appuient dessus. Elle est très complète, donc un peu intimidante ; comprendre sa structure suffit à s'y retrouver.

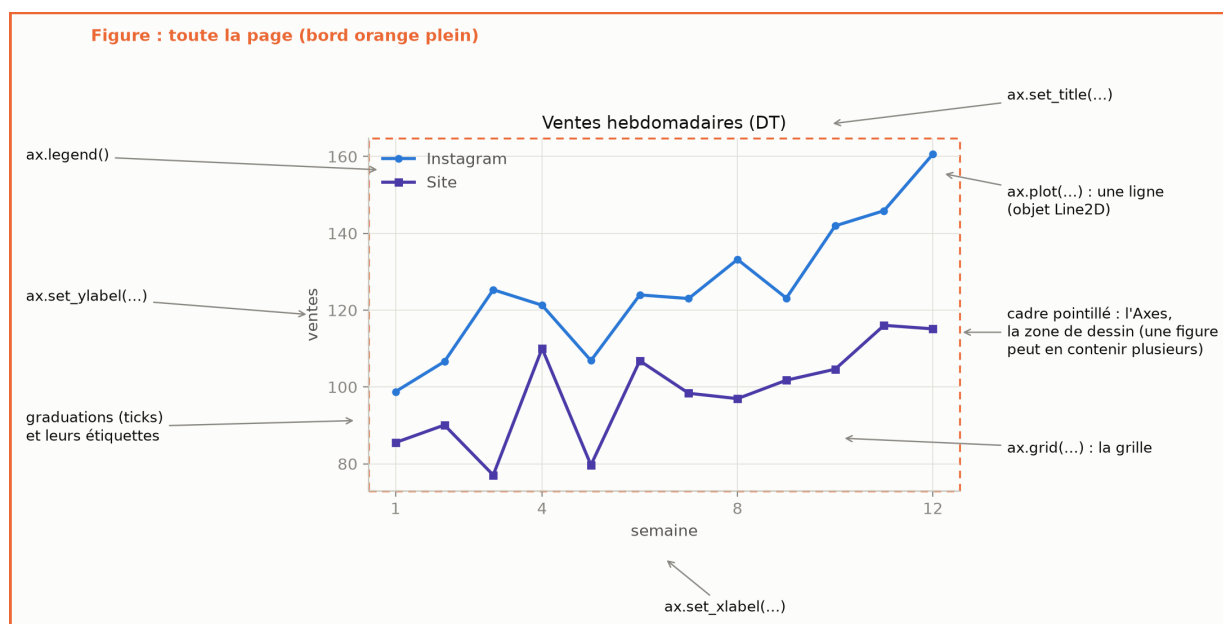


Figure 24: Les pièces d'un graphique matplotlib : la Figure (la page entière), l'Axes (le repère, cadre pointillé), puis les éléments dessinés dedans.

- La **Figure** est la page entière (sa taille se règle avec `figsize=(largeur, hauteur)` en pouces).
- L'**Axes** (au pluriel malgré les apparences : c'est *un* repère) est la zone où l'on dessine. Une figure peut en contenir un seul ou plusieurs (une grille 2×2, par exemple).
- Dans un Axes vivent des **objets** : lignes, barres, points, textes, légende, graduations, grille.

Il existe deux manières de s'en servir. L'interface « état » (`plt.plot(...)`) agit sur « le graphique courant » : très courte, mais confuse dès qu'il y a plusieurs panneaux. L'interface **orientée objet**

crée explicitement la figure et l'axes, puis envoie des commandes à *l'axes* : c'est celle que nous utiliserons toujours, car elle ne laisse aucune ambiguïté sur « où » l'on dessine.

```
fig, ax = plt.subplots(figsize=(7, 3.6))          # une figure contenant un seul Axes
ax.plot(heβδο.index, hebdo["ca"], marker="o", ms=4, color=BLEU)
ax.set_title("Chiffre d'affaires hebdomadaire de Dar Jasmin")
ax.set_xlabel("semaine (lundi)")
ax.set_ylabel("chiffre d'affaires (DT)")
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d/%m")) # dates au format jour/mois
fig.autofmt_xdate()                                     # incline les dates si nécessaire
fig.savefig("figures/ch04-premier-graphique.png", dpi=150, bbox_inches="tight")
plt.close(fig)                                         # libère la mémoire
print("figure enregistrée :", "semaine la plus forte =", hebdo["ca"].idxmax().date(),
      f"({hebdo['ca'].max():.0f} DT)")
```

figure enregistrée : semaine la plus forte = 2026-03-23 (1737 DT)

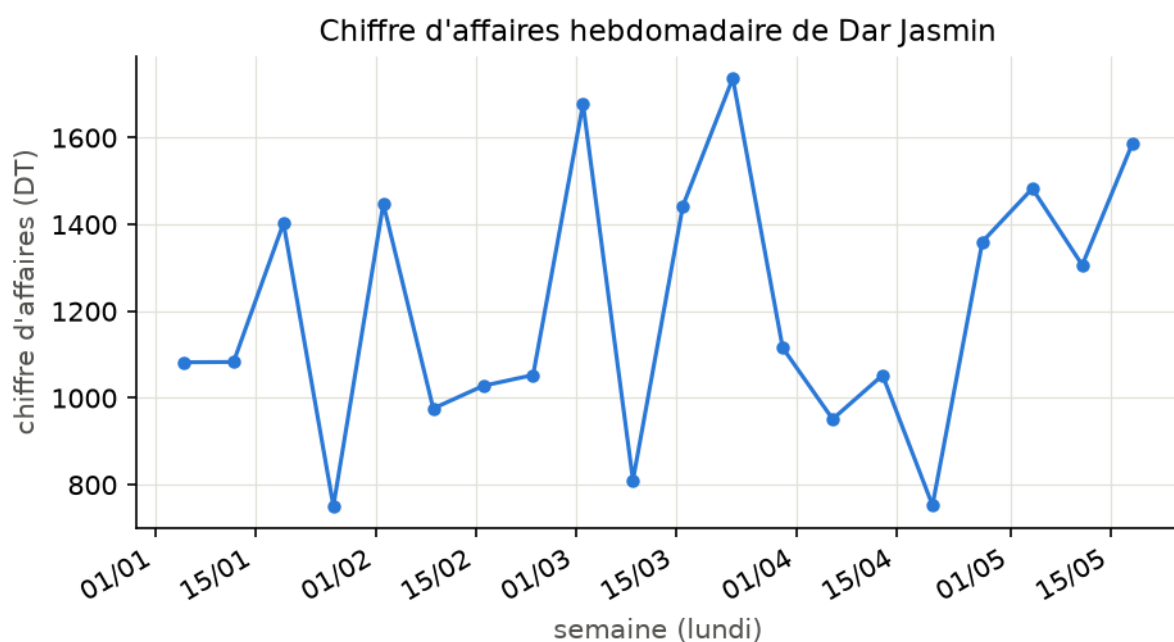


Figure 25: Notre premier graphique : une courbe du chiffre d'affaires par semaine.

Tout y est : un **titre**, des **axes nommés avec leur unité**, une ligne avec des **marqueurs** (un point par semaine, pour ne pas laisser croire que l'on connaît les valeurs entre les semaines). La courbe est irrégulière d'une semaine à l'autre, ce qui est normal : chaque semaine ne compte qu'entre 14 et 27 commandes, donc quelques gros paniers suffisent à faire bondir le total. Pour voir la **tendance**, on lisse avec une moyenne mobile, que nous ajouterons en 4.5.3.

Le patron de tous les graphiques matplotlib. (1) `fig, ax = plt.subplots(...)` ; (2) `ax.plot / ax.bar / ax.hist / ax.scatter(...)` ; (3) `ax.set_title / set_xlabel / set_ylabel` ; (4) `fig.savefig(...)`. Pour *plusieurs* panneaux : `fig, axes = plt.subplots(2, 2)` renvoie un tableau d'Axes que l'on indexe `axes[0, 0]`, `axes[0, 1]`, etc. (c'est un tableau NumPy, voir 4.4.2).

4.5.3 Les quatre graphiques essentiels, avec pandas

pandas offre une méthode `.plot` sur ses Series et DataFrames, qui appelle matplotlib en coulisses et **renvoie l'Axes** : on peut donc la combiner avec tout ce qui précède, en lui passant l'argument

ax=. Voici les quatre graphiques que vous tracerez le plus souvent, dans une seule figure à quatre panneaux :

1. **histogramme** des montants ;
2. **barres horizontales** du chiffre d'affaires par canal (triées) ;
3. **courbe** du chiffre d'affaires hebdomadaire avec sa moyenne mobile ;
4. **nuage de points** livraison/satisfaction.

Pour le quatrième, un détail d'importance. La livraison est un nombre entier de jours et la satisfaction une note entière de 1 à 5 : beaucoup de commandes tombent *exactement au même point*, et un nuage de points normal en cacherait la plupart. On les **décale aléatoirement d'un tout petit peu** (« jitter », en français *jitter* ou *bruitage*) et on rend les points translucides : les zones denses apparaissent plus foncées.

```
fig, axes = plt.subplots(2, 2, figsize=(10.5, 7.2))
fig.subplots_adjust(hspace=0.38, wspace=0.28)

# 1. histogramme (pandas)
ax = axes[0, 0]
df["montant"].plot.hist(bins=30, ax=ax, color=BLEU, alpha=0.7, edgecolor="white")
ax.axvline(df["montant"].median(), color=ORANGE, ls="--", lw=2)
ax.text(df["montant"].median() + 5, ax.get_ylim()[1] * 0.9, "médiane", color=ORANGE)
ax.set(title="Histogramme : la distribution des montants", xlabel="montant d'une commande (DT)",
      ↪ ylabel="nombre de commandes")

# 2. barres horizontales, triées (pandas)
ax = axes[0, 1]
ca_canal = df.groupby("canal")["montant"].sum().sort_values()
ca_canal.plot.barh(ax=ax, color=[COULEURS[c] for c in ca_canal.index], alpha=0.8)
for i, v in enumerate(ca_canal):
    ax.text(v + 80, i, f"{v:,.0f} DT".replace(", ", " "), va="center")
ax.set(title="Barres : le chiffre d'affaires par canal", xlabel="chiffre d'affaires (DT)", ylabel="",
      ↪ xlim=(0, ca_canal.max() * 1.2))
ax.grid(axis="y", visible=False)

# 3. courbe + moyenne mobile (pandas)
ax = axes[1, 0]
hebd["ca"].plot(ax=ax, color=BLEU, alpha=0.45, marker="o", ms=3, label="une semaine")
hebd["ca"].rolling(4).mean().plot(ax=ax, color=ORANGE, lw=2.5, label="moyenne mobile sur 4 semaines")
ax.set_ylim(top=2100) # on laisse de la place en haut pour la légende
ax.legend(loc="upper left")
ax.set(title="Courbe : l'évolution dans le temps", xlabel="semaine", ylabel="chiffre d'affaires (DT)")

# 4. nuage de points avec jitter (matplotlib)
ax = axes[1, 1]
bruit = np.random.default_rng(5)
x = df["livraison"] + bruit.uniform(-0.25, 0.25, len(df))
y = df["satisfaction"] + bruit.uniform(-0.25, 0.25, len(df))
ax.scatter(x, y, s=14, alpha=0.35, color=VIOLET, edgecolors="none")
ax.set_yticks(range(1, 6))
ax.set(title="Nuage de points : livraison et satisfaction", xlabel="délai de livraison (jours)",
      ↪ ylabel="note de satisfaction")

fig.savefig("figures/ch04-essentiels.png", dpi=150, bbox_inches="tight")
plt.close(fig)
r = df[["livraison", "satisfaction"]].corr().iloc[0, 1]
print("corrélacion livraison / satisfaction :", round(r, 2))
print("part du CA du canal en tête :", f"{ca_canal.iloc[-1] / ca_canal.sum():.0%}")
```

```
corrélacion livraison / satisfaction : -0.53
part du CA du canal en tête : 37%
```

Comment lire chaque panneau (c'est aussi comme cela qu'on doit *légènder* un graphique dans

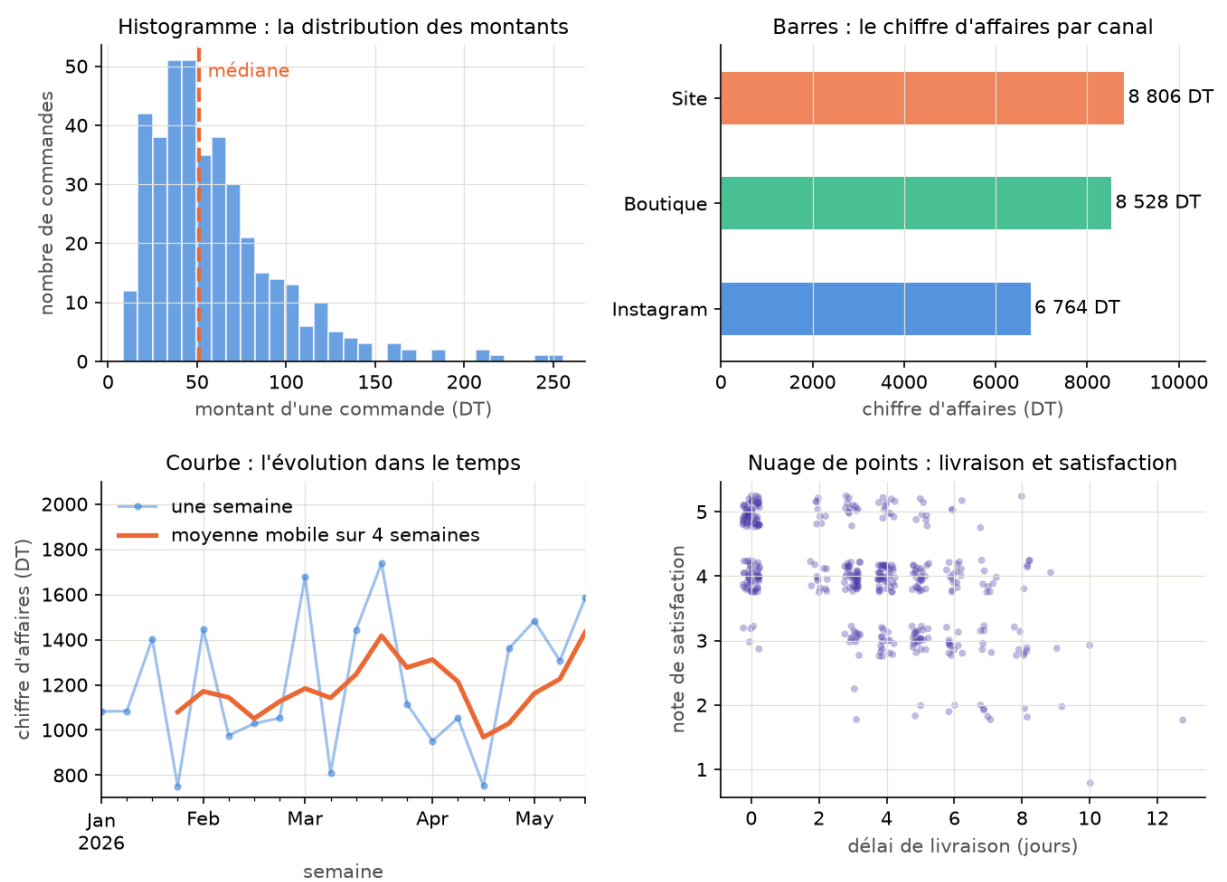


Figure 26: Les quatre graphiques essentiels : histogramme, barres triées, courbe avec moyenne mobile, nuage de points « bruité ».

un rapport) :

- **Histogramme** : la forme est asymétrique à droite, avec une longue queue de grosses commandes ; la médiane (trait orange, 51 DT) est nettement sous la moyenne (60 DT), tirée vers le haut par les grosses commandes (revoir 3.1.5).
- **Barres** : on lit au premier coup d'œil l'ordre des canaux, et les valeurs sont écrites au bout des barres : plus besoin de deviner sur l'axe. Les barres sont **triées** : un classement doit être lisible.
- **Courbe** : le trait pâle est le chiffre d'affaires de chaque semaine, très irrégulier ; le trait orange, plus lisse, montre la **tendance**.
- **Nuage de points** : plus le délai augmente, plus les notes basses apparaissent ; la corrélation affichée ($-0,53$) est négative et d'intensité moyenne : un retard fait *tendanciellement* baisser la note, sans que ce soit une règle absolue (beaucoup de commandes tardives ont quand même 4).

Pourquoi .plot renvoie-t-il l'Axe ? Parce que ainsi tout est modifiable après coup : titre, limites, annotations. Si vous écrivez `ax = df["montant"].plot.hist()`, vous pouvez ensuite faire `ax.set_title(...)`. pandas propose aussi `.plot.bar()`, `.plot.line()`, `.plot.box()`, `.plot.scatter(x=, y=)`, `.plot.pie()`, `.plot.area()`... Pour l'exploration rapide, c'est imbattable (une ligne par graphique).

4.5.4 seaborn : des graphiques statistiques en une ligne

seaborn est une couche au-dessus de matplotlib spécialisée dans les graphiques **statistiques**. Son atout : on lui donne le **tableau complet** (`data=df`) et on dit quelle colonne va où (`x=`, `y=`, `hue=` pour la couleur), et seaborn s'occupe des regroupements, des légendes et des couleurs.

Deux exemples. D'abord la distribution des montants **selon le canal**, en histogramme et en boîte à moustaches :

```
import seaborn as sns

sns.set_theme(style="whitegrid", rc={"axes.spines.top": False, "axes.spines.right": False})
ordre = ["Instagram", "Site", "Boutique"]

fig, axes = plt.subplots(1, 2, figsize=(11, 4), gridspec_kw={"width_ratios": [1.3, 1], "wspace":
    ↪ 0.25})
sns.histplot(data=df, x="montant", hue="canal", hue_order=ordre, palette=COULEURS, bins=30,
    element="step", alpha=0.3, ax=axes[0])
axes[0].set(title="Histogrammes superposés selon le canal", xlabel="montant (DT)", ylabel="nombre de
    ↪ commandes")

sns.boxplot(data=df, x="canal", y="montant", order=ordre, hue="canal", palette=COULEURS,
    ↪ legend=False,
    width=0.55, fliersize=3, ax=axes[1])
axes[1].set(title="Boîtes à moustaches selon le canal", xlabel="", ylabel="montant (DT)")

fig.savefig("figures/ch04-seaborn-distributions.png", dpi=150, bbox_inches="tight")
plt.close(fig)
print(df.groupby("canal")["montant"].median().reindex(ordre).round(1).to_string())
```

canal	
Instagram	41.5
Site	49.5
Boutique	64.8

Les médianes imprimées confirment la lecture : 64,8 DT pour la boutique, 49,5 DT pour le site, 41,5 DT pour Instagram. (Cette différence est-elle réelle ou due au hasard ? C'est la question d'un

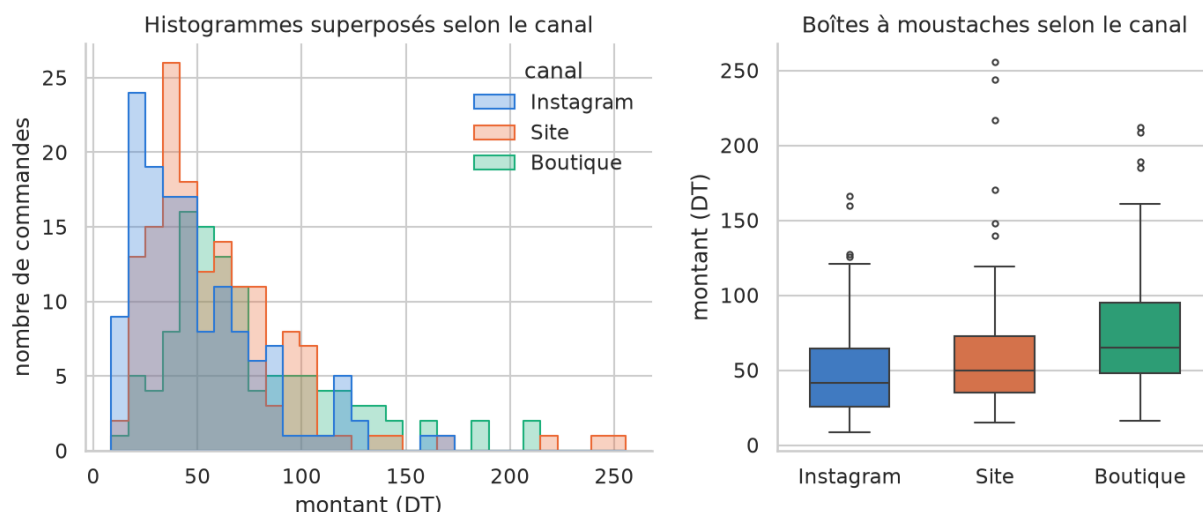


Figure 27: Avec seaborn, une ligne suffit pour séparer les données par canal : histogrammes superposés et boîtes à moustaches.

test statistique, 3.4.)

Deuxième exemple : une **carte de chaleur** pour le croisement de deux variables qualitatives, le canal et la note de satisfaction. On calcule d'abord le tableau croisé (en proportions par canal), puis on le colore :

```
tab = pd.crosstab(df["canal"], df["satisfaction"], normalize="index").loc[ordre]
print(tab.round(2))

fig, ax = plt.subplots(figsize=(6.5, 3))
sns.heatmap(tab, annot=True, fmt=".0%", cmap="Blues", cbar=False, linewidths=2, linecolor="white",
            ax=ax)
ax.set(title="Part de chaque note, selon le canal", xlabel="note de satisfaction", ylabel="")
fig.savefig("figures/ch04-seaborn-heatmap.png", dpi=150, bbox_inches="tight")
plt.close(fig)
```

satisfaction	1	2	3	4	5
canal					
Instagram	0.00	0.06	0.31	0.49	0.14
Site	0.01	0.05	0.25	0.54	0.16
Boutique	0.00	0.00	0.04	0.42	0.54

Chaque ligne du tableau somme à 1 (donc 100 %) : on lit « *parmi* les commandes de la boutique, 54 % ont donné la note 5 » (contre 14 % pour Instagram et 16 % pour le Site). La case la plus foncée de la ligne Boutique est à droite (la note 5), celles d'Instagram et du Site sont sur la note 4, avec une part importante de 3 : la boutique, où la livraison est immédiate, a les clients les plus satisfaits.

matplotlib, pandas ou seaborn ? Ce n'est pas un choix exclusif : seaborn et pandas dessinent dans des Axes matplotlib, que l'on peut retoucher avec les méthodes de 4.5.2. Règle pratique : pandas .plot pour regarder vite, seaborn pour les graphiques statistiques avec groupes, matplotlib pour tout ce qui doit être personnalisé au pixel près.

4.5.5 ggplot2 : la grammaire des graphiques en R

En R, la bibliothèque de référence est **ggplot2**. Son idée, la « **grammaire des graphiques** » (*grammar of graphics*), est de **décrire** un graphique par couches plutôt que de le dessiner pas à pas

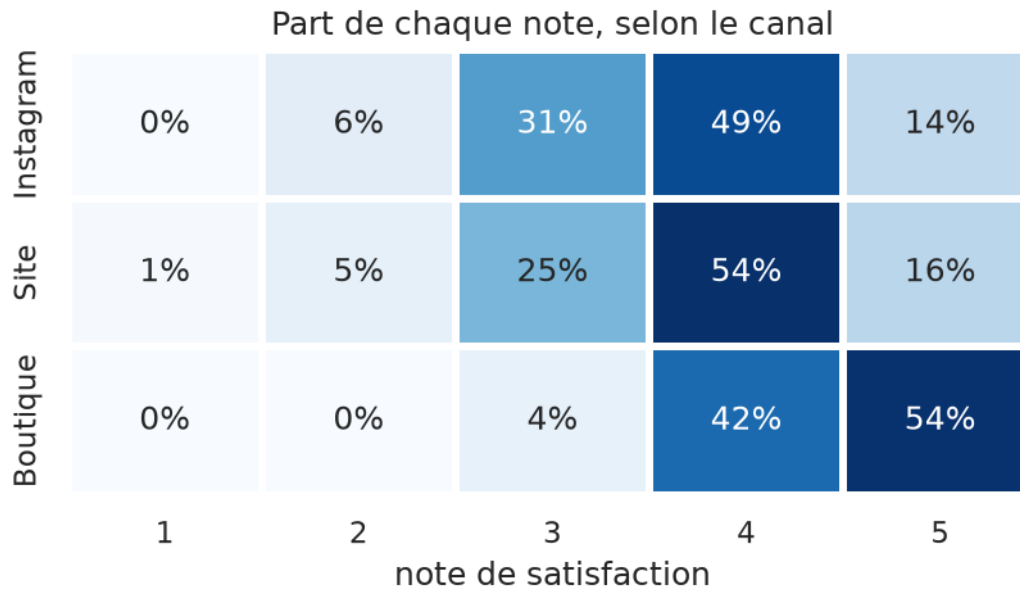


Figure 28: Carte de chaleur : chaque ligne (canal) somme à 100 %. Plus la case est foncée, plus la note est fréquente dans ce canal.

:

- les **données** (`ggplot(data, ...)`) ;
- une **correspondance esthétique** `aes(x=, y=, fill=)` : quelle colonne va sur quel axe ou dans quelle couleur ;
- une ou plusieurs **géométries** `geom_...()` : histogramme, barres, points, lignes ;
- éventuellement des **facettes** (`facet_wrap`) pour faire un panneau par groupe, des **échelles** et un **thème**.

On additionne ces couches avec le signe `+`. Reproduisons l'histogramme par canal ; R lit le même fichier CSV (les bases du langage R sont présentées en 4.2) :

```
library(ggplot2)
commandes <- read.csv("donnees/commandes.csv")
commandes$canal <- factor(commandes$canal, levels = c("Instagram", "Site", "Boutique"))
print(aggregate(montant ~ canal, data = commandes, FUN = function(x) round(median(x), 1)))

p <- ggplot(commandes, aes(x = montant, fill = canal)) +
  geom_histogram(bins = 30, alpha = 0.8, colour = "white") +
  facet_wrap(~ canal, ncol = 1) +
  scale_fill_manual(values = c(Instagram = "#2a78d6", Site = "#eb6834", Boutique = "#1baf7a")) +
  labs(title = "Distribution des montants selon le canal", x = "montant (DT)", y = "nombre de
  ↪ commandes") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "none")
ggsave("figures/ch04-ggplot-montants.png", plot = p, width = 7, height = 5, dpi = 150)
cat("figure enregistrée\n")
```

```
      canal montant
1 Instagram   41.5
2      Site   49.5
3  Boutique   64.8
figure enregistrée
```

On retrouve les mêmes médianes qu'en Python (confirmant que les deux outils lisent bien les mêmes données). Le tableau suivant résume la différence de philosophie :

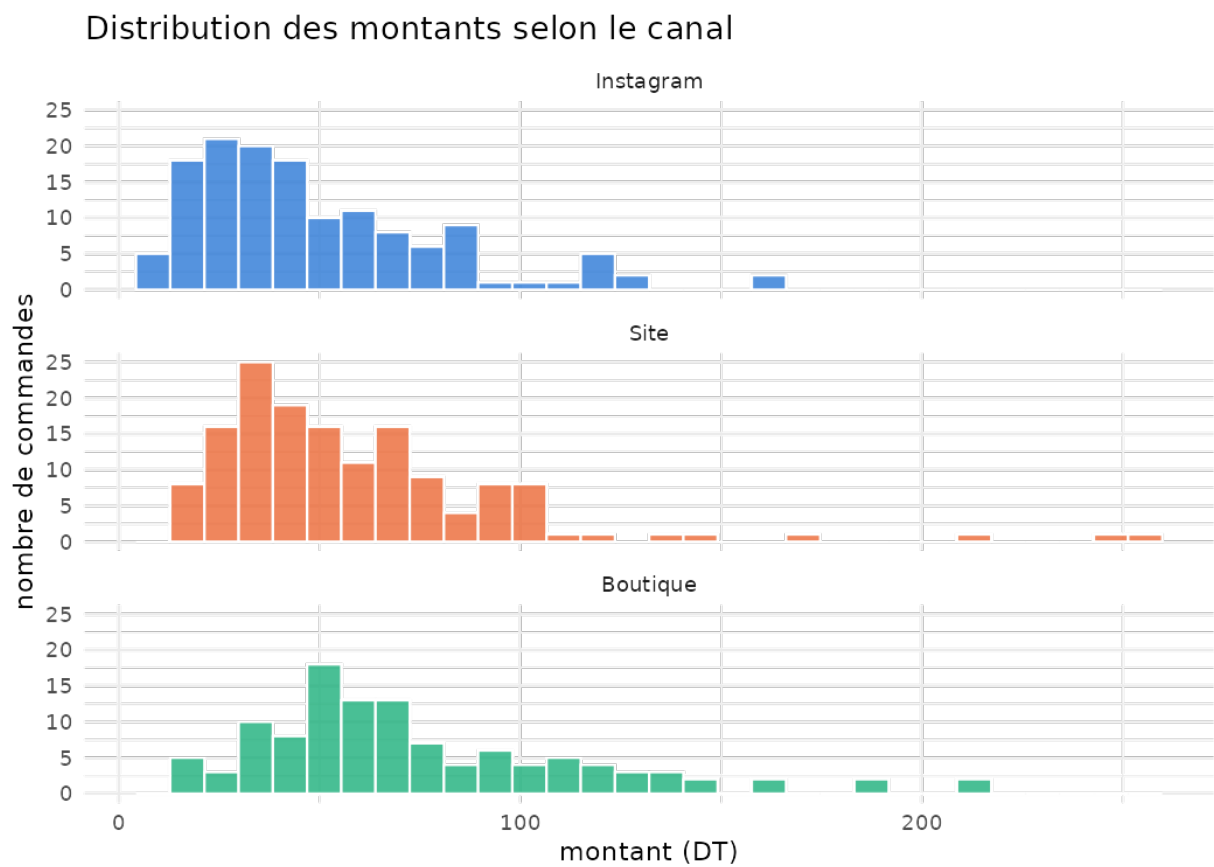


Figure 29: Le même type de graphique avec ggplot2 : un panneau par canal (facettes), même échelle horizontale.

	matplotlib / seaborn (Python)	ggplot2 (R)
Style	impératif : on construit pas à pas (figure → axes → éléments)	déclaratif : on décrit le résultat par couches
Séparer par groupe	hue= (seaborn) ou une boucle sur les groupes	fill=, colour=, ou facet_wrap()
Personnalisation fine	très grande, mais verbeuse	grande, via theme() et scale_*()
Combiner plusieurs couches	appels successifs sur le même ax	+ geom_...()

Honnêteté d'exécution. Les graphiques de cette section ont tous été produits par le code affiché (sauf trois dessins explicatifs, produits par build/fig_ch04.py : l'anatomie d'une figure, le broadcasting et l'axe tronqué) ; le bloc R ci-dessus a bien été exécuté (R avec ggplot2). Les versions de bibliothèques peuvent légèrement modifier l'aspect (polices, marges) sans changer l'information.

4.5.6 Bien faire, mal faire : les pièges du graphique trompeur

Un graphique peut mentir sans qu'une seule donnée soit fausse. Voici les six pièges les plus répandus. Le premier mérite une démonstration.

Piège n°1 : l'axe tronqué. Comparons la satisfaction moyenne d'Instagram et du Site. Les deux graphiques ci-dessous montrent **exactement les mêmes deux nombres**.

```
moy = df[df["canal"].isin(["Instagram", "Site"])].groupby("canal")["satisfaction"].mean()
insta, site = moy["Instagram"], moy["Site"]
print("moyennes :", round(insta, 2), "et", round(site, 2))
print("écart réel : +", round((site / insta - 1) * 100, 1), "%")
print("barre du Site / barre d'Instagram si l'axe commence à 3,70 :", round((site - 3.70) / (insta - 3.70), 1), "fois plus haute")
```

moyennes : 3.72 et 3.79

écart réel : + 2.0 %

barre du Site / barre d'Instagram si l'axe commence à 3,70 : 5.2 fois plus haute

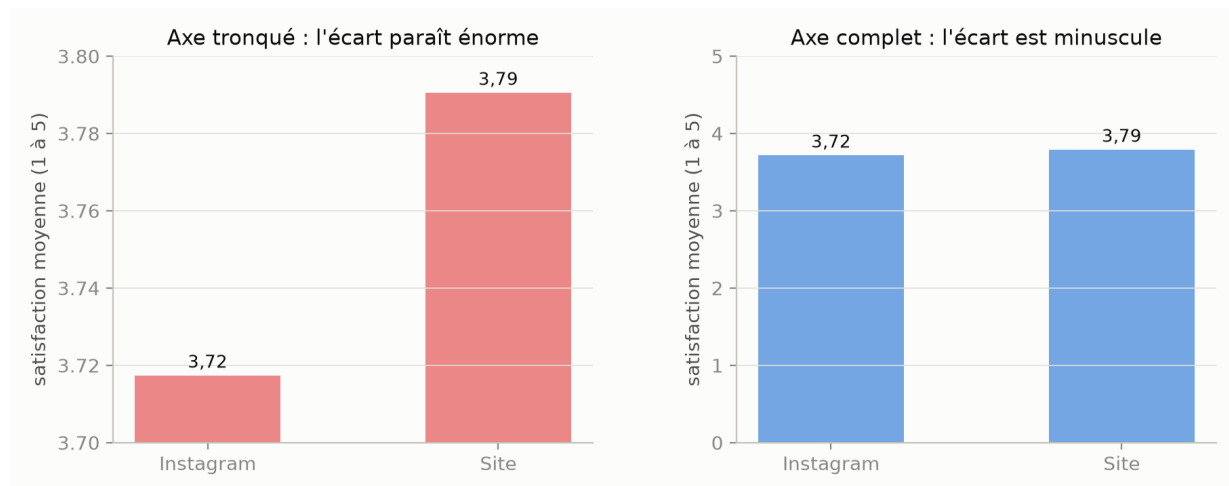


Figure 30: Mêmes données, deux impressions opposées : à gauche l'axe commence à 3,70, à droite à 0.

À gauche, avec un axe qui commence à 3,70, la barre du Site paraît **plus de 5 fois plus haute** que celle d'Instagram (c'est le « 5,2 fois » imprimé ci-dessus) ; à droite, sur un axe complet, les deux barres sont quasiment identiques, ce qui correspond bien à l'écart réel de 2 %. **Règle : pour**

un diagramme en barres, l'axe doit commencer à 0, car c'est la *longueur* de la barre qui porte l'information. (Pour une courbe ou un nuage de points, c'est la position qui compte : on peut zoomer, à condition de le signaler.)

Les autres pièges, et leurs remèdes :

Piège	Pourquoi c'est un problème	Remède
Camembert en 3D, ou à beaucoup de parts	la perspective déforme les aires ; l'œil compare mal les angles	barres triées, en 2D
Double axe vertical	on peut rendre n'importe quelle corrélation « visible » en choisissant les échelles	deux graphiques superposés, ou un seul axe
Trop de couleurs	10 couleurs sans ordre : personne ne retient la légende	3 à 5 couleurs, avec un sens (une couleur = un canal, partout dans le document)
Points superposés	400 observations qui se cachent les unes les autres (voir le jitter, 4.5.3)	transparence, bruitage, ou histogramme 2D
Titre vague (« Graphique 3 »)	le lecteur doit deviner la conclusion	un titre qui dit le message : « La boutique a les clients les plus satisfaits »
Axes sans nom ni unité	« 60 », mais de quoi ?	toujours nommer et donner l'unité (DT, jours, %)

La liste de contrôle d'un bon graphique. (1) Un message, un titre qui le dit. (2) Le bon type de graphique pour la question (tableau 4.5.1). (3) Des axes nommés avec leurs unités ; **zéro pour les barres**. (4) Des barres **triées** quand elles représentent un classement. (5) Peu de couleurs, avec un sens constant. (6) Lisible en noir et blanc et pour un daltonien : ne pas reposer sur l'opposition rouge/vert seule. (7) La source des données et la date, si l'on communique à d'autres.

4.5.7 Enregistrer, réutiliser : de la figure au rapport

Un graphique n'est utile que s'il sort de votre ordinateur. `fig.savefig(chemin)` choisit le format d'après l'extension, avec trois réglages à connaître :

Format	Quand l'utiliser
PNG (.png)	pages web, diapositives, e-mails : image « pixels », léger
SVG / PDF (.svg, .pdf)	impression, articles, LaTeX : image vectorielle , nette à toute taille
<code>dpi=</code>	résolution en pixels par pouce : 150 pour l'écran, 300 pour l'impression
<code>bbox_inches="tight"</code>	rogne les marges blanches inutiles

```
import os
import tempfile

dossier = tempfile.mkdtemp()
fig, ax = plt.subplots(figsize=(4, 2.5))
ax.bar(ordre, [df[df["canal"] == c]["montant"].sum() for c in ordre], color=[COULEURS[c] for c in
↳ ordre])
```

```
for ext in ["png", "svg", "pdf"]:
    chemin = os.path.join(dossier, f"exemple.{ext}")
    fig.savefig(chemin, dpi=150, bbox_inches="tight")
    print(f".{ext} enregistré, taille non nulle :", os.path.getsize(chemin) > 0)
plt.close(fig)
```

```
.png enregistré, taille non nulle : True
.svg enregistré, taille non nulle : True
.pdf enregistré, taille non nulle : True
```

Application : le tableau de bord de Yasmine. Mettons tout en commun dans une **fonction** qui produit en une fois la figure que Yasmine joint à son rapport du lundi (celui de 4.4.13). Remarquez que chaque panneau a un **titre qui énonce sa conclusion**, calculée à partir des données (et non écrite à la main) : si les chiffres changent, le titre reste vrai.

```
def tableau_de_bord(df, chemin):
    """Dessine le tableau de bord de Dar Jasmin à partir du DataFrame `df` et l'enregistre dans
    ↪ `chemin`."""
    ca = df.groupby("semaine")["montant"].sum()
    lisse = ca.rolling(4).mean().dropna()
    par_canal = df.groupby("canal")["montant"].sum().sort_values()
    notes = df["satisfaction"].value_counts().sort_index()
    sens = "monte" if lisse.iloc[-1] > lisse.iloc[0] else "baisse"

    fig = plt.figure(figsize=(10.5, 6.6))
    grille = fig.add_gridspec(2, 2, height_ratios=[1.15, 1], hspace=0.5, wspace=0.3)

    ax = fig.add_subplot(grille[0, :]) # panneau du haut : toute la largeur
    ax.plot(ca.index, ca.values, color=BLEU, alpha=0.35, marker="o", ms=3)
    ax.plot(lisse.index, lisse.values, color=BLEU, lw=2.5)
    ax.set_title(f"Le chiffre d'affaires hebdomadaire {sens} : de {lisse.iloc[0]:.0f} à
    ↪ {lisse.iloc[-1]:.0f} DT (moyenne mobile)")
    ax.set_ylabel("DT par semaine")
    ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=mdates.MO, interval=4))
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%d/%m"))

    ax = fig.add_subplot(grille[1, 0])
    ax.barh(par_canal.index, par_canal.values, color=[COULEURS[c] for c in par_canal.index],
    ↪ alpha=0.85)
    for i, v in enumerate(par_canal):
        ax.text(v + 80, i, f"{v / par_canal.sum():.0%}".replace("%", " %"), va="center")
    ax.set_title(f"{par_canal.index[-1]} : {par_canal.iloc[-1] / par_canal.sum():.0%} du chiffre
    ↪ d'affaires".replace("%", " %"))
    ax.set_xlabel("chiffre d'affaires (DT)")
    ax.set_xlim(0, par_canal.max() * 1.2)
    ax.grid(axis="y", visible=False)

    ax = fig.add_subplot(grille[1, 1])
    ax.bar(notes.index, notes.values, color=[ORANGE if n <= 2 else BLEU for n in notes.index],
    ↪ alpha=0.85)
    ax.set_title(f"{(df['satisfaction'] >= 4).mean():.0%} des clients notent 4 ou 5".replace("%", "
    ↪ %"))
    ax.set_xlabel("note de satisfaction")
    ax.set_ylabel("commandes")
    ax.grid(axis="x", visible=False)

    fig.savefig(chemin, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return par_canal.index[-1], round(float(lisse.iloc[-1]), 1)

print(tableau_de_bord(df, "figures/ch04-tableau-de-bord.png"))
```

```
('Site', 1433.7)
```

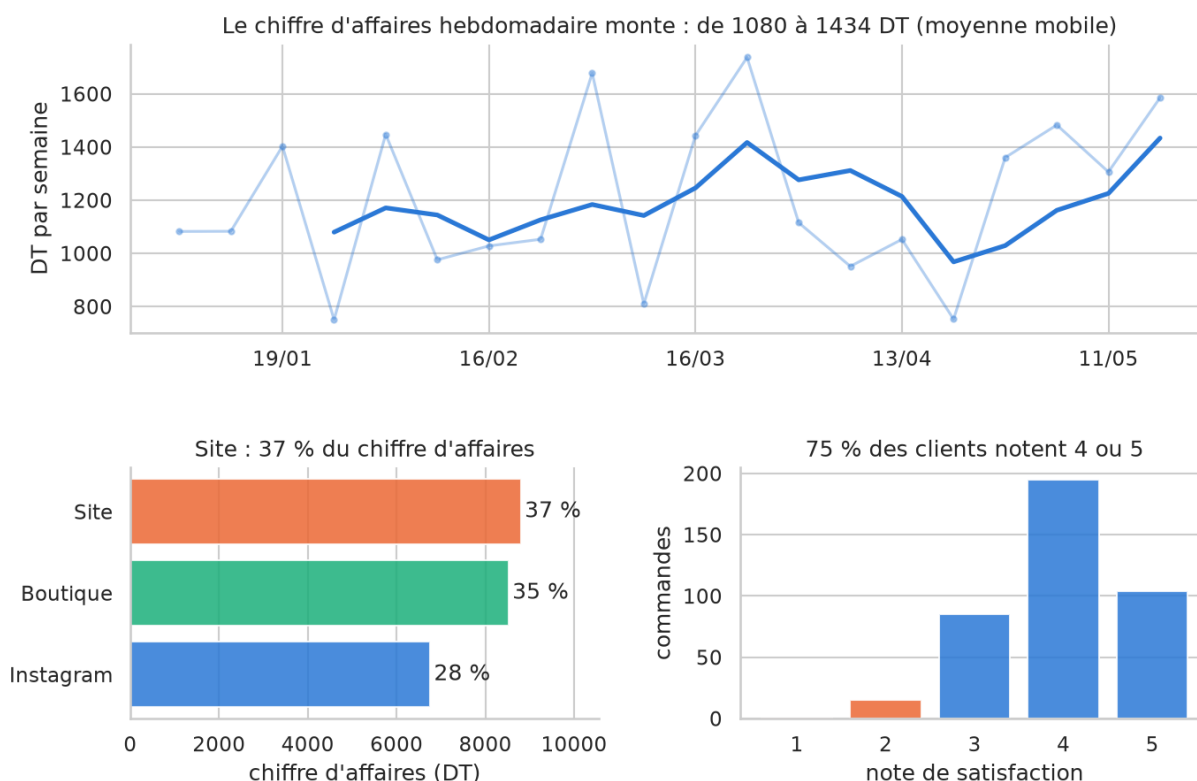


Figure 31: Le tableau de bord produit par la fonction : trois panneaux, chacun avec un titre qui énonce sa conclusion.

Lisons-le comme le ferait Yasmine : la tendance du chiffre d'affaires est à la hausse (1 080 → 1 434 DT par semaine en moyenne mobile), le Site et la Boutique pèsent chacun plus d'un tiers des ventes, et trois clients sur quatre sont satisfaits (4 ou 5). Voilà le chemin complet : des **données brutes** (fichier CSV) aux **tableaux** (pandas, 4.4) puis aux **figures** prêtes à insérer dans un rapport, le tout dans un script que l'on peut relancer chaque semaine. C'est l'esprit de la **recherche reproductible** que nous retrouverons au chapitre 6.

À retenir (visualisation).

1. On part de la **question** et du **type des variables**, pas du joli graphique ;
2. matplotlib : `fig, ax = plt.subplots()`, on dessine dans l'ax, on nomme, on enregistre ; pandas .plot et seaborn dessinent dans le même cadre ;
3. quatre fondamentaux : histogramme (forme), barres triées (comparaison), courbe (temps), nuage de points (relation) ;
4. **barres** : l'axe commence à zéro ; pas de 3D ; peu de couleurs, avec un sens ; titre = message ;
5. ggplot2 (R) décrit un graphique par **couches** : données + esthétique + géométrie ;
6. **PNG** pour l'écran, **PDF/SVG** pour l'impression ; une fonction de tracé rend l'analyse reproductible.

4.6 ☉ Pour aller plus loin : programmation orientée objet, code propre et tests

Section optionnelle. Vous pouvez faire toute une carrière d'analyste avec des fonctions, des listes et des tableaux pandas. Mais dès que votre code dépasse quelques dizaines de lignes, ou qu'un collègue (ou vous-même, dans six mois) doit le relire, trois outils changent la vie : **regrouper** les données et les opérations qui vont ensemble (les *objets*), **écrire clairement** (le *code propre*), et **vérifier automatiquement** que le code fait ce qu'on croit (les *tests*). Cette section les présente sur un exemple concret : le panier d'une cliente de Dar Jasmin.

Pour cette section, nous écrivons de vrais **fichiers** Python, puis nous les exécutons depuis un terminal, exactement comme vous le feriez sur votre machine. Les blocs ci-dessous sont donc des commandes du terminal (bash) : la commande `cat > fichier <<'FIN' ... FIN` crée un fichier avec le texte qui suit, et `python fichier.py` l'exécute. (Le chapitre 6.3 détaille le terminal.)

4.6.1 Pourquoi des objets ? Le problème des dictionnaires

Intuition. Jusqu'ici, un panier pouvait être une simple liste de prix. Mais un panier, ce n'est pas qu'une liste : c'est aussi « *savoir calculer son total, ajouter un article, refuser une quantité négative* ». Un **objet** est un petit paquet qui contient à la fois des **données** (les *attributs*) et les **opérations** qui vont avec (les *méthodes*). Sa **classe** est le moule qui fabrique ces objets, comme un patron de couture fabrique des robes.

Voyons pourquoi cela sert à quelque chose. Voici un panier représenté « à la main » par un dictionnaire, et une fonction qui calcule le total :

```
panier = {"savon": (20.0, 2), "plateau": (30.0, 1)}      # nom -> (prix HT, quantité)

def total_ht(p):
    return sum(prix * qte for prix, qte in p.values())

print("total HT :", total_ht(panier))
panier["plateau"] = (30.0, -3)                        # une erreur de saisie...
print("total HT :", total_ht(panier), " <- personne ne nous a prévenus !")
```

```
total HT : 70.0
total HT : -50.0   <- personne ne nous a prévenus !
```

Rien ne protège le dictionnaire : une quantité négative passe sans bruit, et le total devient faux **sans aucune erreur**. C'est le pire cas possible pour un analyste (un résultat faux qui a l'air normal). Avec une classe, on décide à **un seul endroit** de ce qui est permis.

4.6.2 Votre première classe

Commençons par la version « longue », avec une classe ordinaire, pour comprendre les mécanismes :

```
cat > article_simple.py <<'FIN'
class Article:
    def __init__(self, nom, prix_ht):          # appelée à la création de l'objet
        self.nom = nom                        # self = l'objet en cours de création
        self.prix_ht = prix_ht

    def prix_ttc(self):                        # une méthode : une fonction de l'objet
        return round(self.prix_ht * 1.19, 2)
```



```
savon = Article("savon", 20.0)
print(savon.nom, savon.prix_ht, savon.prix_ttc())
print(savon)                                # affichage par défaut : peu lisible
FIN
python article_simple.py | sed -E 's/0x[0-9a-f]+/0x.../'    # on masque l'adresse mémoire, qui change à
↳ chaque exécution
```

```
savon 20.0 23.8
<__main__.Article object at 0x...>
```

Lisez ligne à ligne :

- `class Article`: définit le moule.
- `__init__` est le **constructeur** : Python l'appelle quand on écrit `Article("savon", 20.0)`. Le premier paramètre, `self`, désigne l'objet qu'on est en train de fabriquer ; on y accroche les attributs (`self.nom`, `self.prix_ht`).
- `prix_ttc` est une **méthode** : on l'appelle avec un point, `savon.prix_ttc()`, et `self` est passé automatiquement.
- Le dernier `print` montre le défaut de cette version : l'affichage `<__main__.Article object at 0x...>` ne dit rien d'utile (et l'adresse change à chaque exécution).

Écrire `__init__` et un affichage lisible pour chaque classe devient vite répétitif. C'est le rôle des **dataclasses** (`@dataclass`) : Python écrit pour vous le constructeur, un affichage lisible et la comparaison `==`.

```
cat > article_dc.py <<'FIN'
from dataclasses import dataclass

@dataclass
class Article:
    nom: str
    prix_ht: float

a = Article("savon", 20.0)
b = Article("savon", 20.0)
print(a)                # affichage lisible, fabriqué automatiquement
print("a == b ?", a == b)
FIN
python article_dc.py
```

```
Article(nom='savon', prix_ht=20.0)
a == b ? True
```

Deux articles ayant les mêmes attributs sont égaux : c'est ce qu'on attend d'une « valeur ». Deux lignes de déclaration ont remplacé une quinzaine de lignes de code.

4.6.3 Le cahier des charges du module

Nous construirons le module de la boutique en 4.6.5, en deux temps : d'abord une fonction de remise, volontairement écrite **trop vite** (c'est l'occasion de découvrir les tests), puis les classes. Mais avant d'écrire du code, posons les règles.

Voici les règles métier, que nous vérifierons à **la main** avant de coder :

- la TVA est de 19 % ;
- un panier de 2 savons à 20 DT et 1 plateau à 30 DT vaut $2 \times 20 + 30 = 70$ DT hors taxe, soit $70 \times 1,19 = 83,30$ DT TTC ;
- avec une remise de 10 % sur le hors-taxe : $70 \times 0,90 = 63$ DT HT, soit $63 \times 1,19 = 74,97$ DT TTC ;

- sur le site, la livraison coûte 7 DT, **offerte** si le panier TTC atteint 100 DT ; en boutique, le retrait est gratuit.

4.6.4 Code propre : lisible avant tout

Intuition. Le code est lu bien plus souvent qu'il n'est écrit. L'objectif n'est pas de « faire marcher » un programme, mais de le rendre **compréhensible** par quelqu'un qui n'était pas là quand vous l'avez écrit (y compris vous, dans six mois). Cinq habitudes suffisent pour 90 % du résultat :

1. **Des noms qui parlent.** `total_ttc` plutôt que `t`, `remise` plutôt que `r`. Un nom long et clair vaut mieux qu'un commentaire.
2. **Des fonctions courtes qui font une seule chose.** Si vous devez écrire « et » pour décrire ce que fait une fonction, coupez-la en deux.
3. **Pas de nombres magiques.** Écrivez `TVA = 0.19` une fois en haut du fichier, pas `1.19` à quinze endroits (le jour où le taux change, vous n'en oublierez aucun).
4. **Une docstring :** une phrase entre triples guillemets sous la ligne `def` ou `class`, qui dit *ce que* fait la fonction. Elle s'affiche avec `help(...)`.
5. **Des annotations de type** (`nom: str, -> float`) : elles documentent ce que la fonction attend et renvoie.

Les annotations de type ne sont pas vérifiées à l'exécution. Python les lit, mais ne les impose pas. Ce sont des indications pour les humains et pour les outils de vérification (comme `mypy`). Regardez :

```
cat > typage.py <<'FIN'
def double(x: int) -> int:
    return x * 2

print(double(21))
print(double("ab"))      # une chaîne n'est pas un entier... et pourtant, aucune erreur
FIN
python typage.py
```

```
42
abab
```

Piège classique : l'argument par défaut modifiable. Une valeur par défaut comme `[]` est créée **une seule fois**, à la définition de la fonction, puis partagée entre tous les appels. C'est une des erreurs les plus fréquentes en Python :

```
cat > piege.py <<'FIN'
def ajouter_mauvais(article, panier=[]):          # MAUVAIS : la liste est partagée
    panier.append(article)
    return panier

def ajouter_bon(article, panier=None):            # BON : on crée la liste à chaque appel
    if panier is None:
        panier = []
    panier.append(article)
    return panier

print("mauvais :", ajouter_mauvais("savon"), ajouter_mauvais("plateau"))
print("bon      :", ajouter_bon("savon"), ajouter_bon("plateau"))
FIN
python piege.py
```

```
mauvais : ['savon', 'plateau'] ['savon', 'plateau']
bon      : ['savon'] ['plateau']
```

Avec la version fautive, le deuxième panier *contient aussi le savon* du premier : deux clientes se retrouvent avec le même panier. Vous retrouverez ce motif (None puis création) dans les dataclasses sous la forme `field(default_factory=list)`.

4.6.5 Tester son code : le filet de sécurité

Intuition. Un **test unitaire** est un petit programme qui appelle une fonction avec des entrées dont **vous connaissez la bonne réponse**, et vérifie qu'elle renvoie bien cette réponse. Vous les écrivez une fois ; ils se rejouent en une seconde après chaque modification. Si un test devient rouge, vous savez *quoi* vous venez de casser, *tout de suite*, et pas un mois plus tard en lisant un rapport faux.

Le schéma universel d'un test s'appelle **Arrange – Act – Assert** : on **prépare** les données (*arrange*), on **exécute** la fonction (*act*), on **vérifie** le résultat (*assert*). Nous utilisons **pytest**, l'outil standard : il suffit d'écrire des fonctions dont le nom commence par `test_` et d'y mettre des `assert`.

Étape 1 : une fonction de remise, écrite trop vite.

```
cat > remises.py <<'FIN'
def prix_apres_remise(prix, taux):
    """Prix après une remise de `taux` (0,25 pour 25 %)."""
    return prix - taux
FIN

cat > test_remises.py <<'FIN'
from remises import prix_apres_remise

def test_remise_de_25_pour_cent():
    # 80 DT avec 25 % de remise : 80 * (1 - 0,25) = 60 DT
    assert prix_apres_remise(80.0, 0.25) == 60.0

def test_sans_remise():
    assert prix_apres_remise(80.0, 0.0) == 80.0
FIN

python -m pytest -q --color=no --tb=short -p no:cacheprovider test_remises.py 2>&1 | sed -E 's/ in ↵ [0-9.]+s//'
```

```
F. [100%]
===== FAILURES =====
_____ test_remise_de_25_pour_cent _____
test_remises.py:5: in test_remise_de_25_pour_cent
    assert prix_apres_remise(80.0, 0.25) == 60.0
E     assert 79.75 == 60.0
E     + where 79.75 = prix_apres_remise(80.0, 0.25)
===== short test summary info =====
FAILED test_remises.py::test_remise_de_25_pour_cent - assert 79.75 == 60.0
1 failed, 1 passed
```

Le test a fait son travail : la fonction soustrait le *taux* (0,25 DT !) au lieu d'appliquer le pourcentage. Notez que `test_sans_remise` passe, ce qui montre pourquoi **un seul test ne suffit pas** : un code faux peut réussir un cas particulier. Le message affiche la ligne en cause, la valeur obtenue (79,75) et la valeur attendue (60).

Étape 2 : on corrige, on relance.

```
cat > remises.py <<'FIN'
def prix_apres_remise(prix, taux):
    """Prix après une remise de `taux` (0,25 pour 25 %)."""
    if not 0 <= taux <= 1:
        raise ValueError(f"le taux doit être entre 0 et 1, reçu {taux}")
    return prix * (1 - taux)
FIN

python -m pytest -q --color=no --tb=short -p no:cacheprovider test_remises.py 2>&1 | sed -E 's/ in
↳ [0-9.]+s//'

.. [100%]
2 passed
```

Deux tests verts. Remarquez que nous avons aussi ajouté une **validation** : un taux de 25 (au lieu de 0,25) leverait une erreur claire plutôt que de produire un prix négatif.

Un réflexe d'expert : écrire le test *avant* le correctif. Quand vous découvrez un bogue, écrivez d'abord un test qui l'attrape (il est rouge), puis corrigez le code (il devient vert). Ce bogue ne reviendra jamais sans que quelqu'un le remarque. Cette discipline s'appelle le **développement piloté par les tests (TDD)**.

Étape 3 : le module de la boutique. Les quatre classes, avec docstrings, annotations de types, validation et une constante pour la TVA :

```
cat > boutique.py <<'FIN'
"""Modèle objet minimal de la boutique Dar Jasmin."""
from dataclasses import dataclass, field

from remises import prix_apres_remise

TVA = 0.19
SEUIL_LIVRAISON_OFFERTE = 100.0      # DT, panier TTC
FRAIS_LIVRAISON_SITE = 7.0          # DT

@dataclass(frozen=True)              # frozen : on ne peut plus modifier un article créé
class Article:
    """Un article du catalogue : un nom et un prix hors taxe (DT)."""
    nom: str
    prix_ht: float

    def __post_init__(self) -> None:
        if self.prix_ht < 0:
            raise ValueError(f"prix négatif pour {self.nom!r} : {self.prix_ht}")

@dataclass
class Panier:
    """Un panier : des lignes (article, quantité)."""
    lignes: list = field(default_factory=list)

    def ajouter(self, article: Article, quantite: int = 1) -> None:
        if quantite <= 0:
            raise ValueError("la quantité doit être strictement positive")
        self.lignes.append((article, quantite))

    def __len__(self) -> int:          # len(panier) = nombre total d'unités
        return sum(quantite for _, quantite in self.lignes)

    def total_ht(self) -> float:
        return sum(article.prix_ht * quantite for article, quantite in self.lignes)
```

```

def total_ttc(self, remise: float = 0.0) -> float:
    """Total TTC arrondi au centime, après une remise sur le hors-taxe."""
    return round(prix_apres_remise(self.total_ht(), remise) * (1 + TVA), 2)

class Commande:
    """Une commande en boutique : retrait gratuit."""

    def __init__(self, panier: Panier) -> None:
        self.panier = panier

    def frais_livraison(self) -> float:
        return 0.0

    def total_a_payer(self) -> float:
        return round(self.panier.total_ttc() + self.frais_livraison(), 2)

class CommandeSite(Commande):
    """Une commande sur le site : 7 DT de livraison, offerts dès 100 DT TTC."""

    def frais_livraison(self) -> float:
        if self.panier.total_ttc() >= SEUIL_LIVRAISON_OFFERTE:
            return 0.0
        return FRAIS_LIVRAISON_SITE
FIN
echo "module écrit : $(wc -l < boutique.py) lignes"

```

```
module écrit : 62 lignes
```

Quelques points de lecture :

- `frozen=True` rend l'article **immuable** : impossible d'écrire `article.prix_ht = -5` après coup. Moins de bogues possibles.
- `__post_init__` est appelé juste après le constructeur généré par la dataclass : c'est l'endroit idéal pour **valider** les données.
- `__len__` est une **méthode spéciale** (on les reconnaît à leurs doubles tirets bas) : elle fait fonctionner `len(panier)`.
- `CommandeSite(Commande)` est un exemple d'**héritage** : la classe fille reprend tout de la classe mère et ne **redéfinit** que ce qui change, ici `frais_livraison`. La méthode `total_a_payer`, écrite une seule fois dans `Commande`, appelle `self.frais_livraison()` et obtient automatiquement le bon comportement selon le type d'objet : c'est le **polymorphisme**.

Quand utiliser l'héritage ? Avec parcimonie. Deux classes dont l'une « *est une sorte de* » l'autre (une commande du site *est une* commande) : oui. Pour simplement réutiliser du code, préférez la **composition** (un objet qui *contient* un autre, comme `Commande` contient un `Panier`). Beaucoup de projets de data science n'ont besoin que de fonctions et de dataclasses.

Étape 4 : les tests du module. Chaque règle métier vérifiée à la main plus haut devient un test. Le décorateur `@pytest.fixture` prépare le panier de l'exemple (le *arrange*) ; `pytest.approx` compare des nombres décimaux avec une petite tolérance (rappelez-vous, section 1.5 : `0.1 + 0.2 != 0.3` en binaire !) ; `@pytest.mark.parametrize` rejoue le même test avec plusieurs valeurs.

```

cat > test_boutique.py <<'FIN'
import pytest

from boutique import Article, Commande, CommandeSite, Panier

```

```

@pytest.fixture
def panier():
    """Le panier de l'exemple : 2 savons à 20 DT + 1 plateau à 30 DT."""
    p = Panier()
    p.ajouter(Article("savon", 20.0), 2)
    p.ajouter(Article("plateau", 30.0))
    return p

def test_nombre_d_unites(panier):
    assert len(panier) == 3

def test_total_ht(panier):
    assert panier.total_ht() == 70.0

def test_total_ttc(panier):
    assert panier.total_ttc() == pytest.approx(83.30)

def test_total_ttc_avec_remise(panier):
    assert panier.total_ttc(remise=0.10) == pytest.approx(74.97)

@pytest.mark.parametrize("quantite", [0, -2])
def test_quantite_invalide(quantite):
    with pytest.raises(ValueError):
        Panier().ajouter(Article("savon", 20.0), quantite)

def test_prix_negatif_refuse():
    with pytest.raises(ValueError):
        Article("cadeau", -1.0)

def test_retrait_boutique_gratuit(panier):
    assert Commande(panier).total_a_payer() == pytest.approx(83.30)

def test_livraison_payante_sous_le_seuil(panier):
    # 83,30 DT < 100 DT : 7 DT de frais -> 90,30 DT
    assert CommandeSite(panier).total_a_payer() == pytest.approx(90.30)

def test_livraison_offerte_au-dessus_du_seuil(panier):
    panier.ajouter(Article("coffret", 20.0))    # 90 DT HT -> 107,10 DT TTC
    assert CommandeSite(panier).total_a_payer() == pytest.approx(107.10)
FIN

python -m pytest -v --color=no --tb=short -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s// ' | grep
↪ -v -E "^(platform|rootdir|cachedir|plugins|configfile)"

```

```

===== test session starts =====
collecting ... collected 12 items

test_boutique.py::test_nombre_d_unites PASSED [ 8%]
test_boutique.py::test_total_ht PASSED [ 16%]
test_boutique.py::test_total_ttc PASSED [ 25%]
test_boutique.py::test_total_ttc_avec_remise PASSED [ 33%]
test_boutique.py::test_quantite_invalide[0] PASSED [ 41%]
test_boutique.py::test_quantite_invalide[-2] PASSED [ 50%]
test_boutique.py::test_prix_negatif_refuse PASSED [ 58%]

```

```
test_boutique.py::test_retrait_boutique_gratuit PASSED [ 66%]
test_boutique.py::test_livraison_payante_sous_le_seuil PASSED [ 75%]
test_boutique.py::test_livraison_offerte_au-dessus_du_seuil PASSED [ 83%]
test_remises.py::test_remise_de_25_pour_cent PASSED [ 91%]
test_remises.py::test_sans_remise PASSED [100%]
```

```
===== 12 passed =====
```

Chaque ligne verte est une promesse tenue. Vérifiez qu'elles correspondent bien aux calculs faits à la main : 83,30, 74,97, $83,30 + 7 = 90,30$, et le panier de 90 DT HT qui vaut $90 \times 1,19 = 107,10$ DT TTC, donc livraison offerte.

Que se passe-t-il si on casse le code ? Modifions le seuil de livraison offerte à 1 000 DT dans le module (une faute de frappe plausible : un zéro en trop), et relançons les tests. Un seul test devrait passer au rouge, celui qui protège précisément cette règle :

```
sed -i 's/SEUIL_LIVRAISON_OFFERTE = 100.0/SEUIL_LIVRAISON_OFFERTE = 1000.0/' boutique.py
python -m pytest -q --color=no --tb=no -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s// ' | tail -2
sed -i 's/SEUIL_LIVRAISON_OFFERTE = 1000.0/SEUIL_LIVRAISON_OFFERTE = 100.0/' boutique.py # on remet
↪ la bonne valeur
python -m pytest -q --color=no -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s// ' | tail -1
```

```
FAILED test_boutique.py::test_livraison_offerte_au-dessus_du_seuil - assert 1...
1 failed, 11 passed
12 passed
```

Voilà la valeur d'une suite de tests : une modification « innocente » est détectée **immédiatement**, avec le nom du test et la règle violée.

4.6.6 Une application : tester une fonction d'analyse

Les tests ne servent pas qu'aux classes : une fonction d'analyse de données mérite les mêmes soins, surtout si elle sera réutilisée dans un rapport. Voici le panier moyen par canal (le même calcul que celui du chapitre 3), testé sur un **petit tableau dont on connaît la réponse à la main**.

```
cat > analyse.py <<'FIN'
import pandas as pd

def panier_moyen_par_canal(commandes: pd.DataFrame) -> pd.Series:
    """Montant moyen des commandes pour chaque canal de vente."""
    return commandes.groupby("canal")["montant"].mean()
FIN

cat > test_analyse.py <<'FIN'
import pandas as pd
import pytest

from analyse import panier_moyen_par_canal

def test_panier_moyen_par_canal():
    petit = pd.DataFrame({
        "canal": ["Site", "Site", "Boutique"],
        "montant": [10.0, 30.0, 50.0],
    })
    resultat = panier_moyen_par_canal(petit)
    assert resultat["Site"] == pytest.approx(20.0) # (10 + 30) / 2
    assert resultat["Boutique"] == pytest.approx(50.0) # une seule commande
FIN
```

```
python -m pytest -v --color=no --tb=short -p no:cacheprovider test_analyse.py 2>&1 | sed -E 's/ in ↵ [0-9.]+s// ' | grep -E "PASSED|FAILED|passed|failed"
```

```
test_analyse.py::test_panier_moyen_par_canal PASSED [100%]
===== 1 passed =====
```

Cette fonction renvoie la même chose que `df.groupby("canal")["montant"].mean()` appliqué aux 400 commandes (section 4.4), mais elle est maintenant **nommée, documentée et protégée**. Dans un vrai projet, on range le code dans un dossier `src/` et les tests dans un dossier `tests/` ; la commande `pytest` trouve alors tout seule les fichiers `test_*.py` (chapitre 6.1 pour les ranger sous Git).

À retenir (objets, code propre, tests).

- Une **classe** regroupe des **données** (attributs) et des **opérations** (méthodes) ; `self` désigne l'objet courant. Une **dataclass** génère le constructeur, l'affichage et l'égalité à votre place.
- On **valide** les données à l'entrée (`__post_init__`, `raise ValueError`) : mieux vaut une erreur claire qu'un résultat faux silencieux.
- **Héritage** : la classe fille redéfinit seulement ce qui change ; **composition** : un objet en contient un autre. Dans le doute, préférez la composition.
- **Code propre** : noms parlants, fonctions courtes, constantes nommées (TVA), docstrings, annotations de type (non vérifiées à l'exécution), pas de `[]` comme valeur par défaut.
- **Test unitaire** = Arrange, Act, Assert. Avec **pytest** : fonctions `test_*`, `assert`, `pytest.approx` pour les décimaux, `pytest.raises` pour les erreurs attendues, `@pytest.mark.parametrize` pour plusieurs cas.
- Un bogue trouvé ? Écrivez d'abord le test qui l'attrape, puis corrigez.

4.7 ☉ Pour aller plus loin : d'autres langages (SAS, MATLAB, Julia)

Section optionnelle. Python et R vous couvriront dans la grande majorité des cas. Mais vous croiserez peut-être, dans une offre d'emploi, un stage ou le code d'un collègue, **SAS**, **MATLAB** ou **Julia**. Cette section ne vous apprend pas ces langages : elle vous donne le **vocabulaire** pour les lire sans panique, en refaisant *la même petite analyse* dans chacun.

Honnêteté sur ce qui a été exécuté. Python et R sont installés sur la machine qui a produit ce livre : leurs sorties ci-dessous sont **réelles**. SAS, MATLAB et Julia ne le sont pas : leurs blocs de code sont marqués « **non exécuté** » et ne sont accompagnés d'aucune sortie. Ils sont écrits à partir de leur documentation, avec les conventions habituelles de chaque langage ; **vérifiez-les chez vous** avant de les utiliser dans un travail important. Le résultat attendu est celui de Python et R (section 3.1 : il est le même partout, car le calcul est le même).

4.7.1 La tâche : un résumé par canal

Yasmine demande : « *Pour chaque canal de vente, combien de commandes, quel montant moyen, et quel écart-type ?* » C'est le calcul du 3.1, appliqué au fichier `donnees/commandes.csv` : lire, regrouper, résumer. Cinq lignes dans chaque langage.

En Python (pandas, section 4.4) :


```
import pandas as pd

commandes = pd.read_csv("donnees/commandes.csv")
resume = (commandes.groupby("canal")["montant"]
          .agg(n="count", moyenne="mean", ecart_type="std")
          .round(1))
print(resume)
```

	n	moyenne	ecart_type
canal			
Boutique	114	74.8	40.6
Instagram	138	49.0	31.1
Site	148	59.5	38.3

En R (le même calcul avec le paquet dplyr, section 4.2) :

```
suppressPackageStartupMessages(library(dplyr))

commandes <- read.csv("donnees/commandes.csv")
commandes |>
  group_by(canal) |>
  summarise(n = n(), moyenne = round(mean(montant), 1), ecart_type = round(sd(montant), 1))
```

```
# A tibble: 3 × 4
  canal      n moyenne ecart_type
  <chr> <int> <dbl> <dbl>
1 Boutique  114  74.8  40.6
2 Instagram 138  49    31.1
3 Site     148  59.5  38.3
```

Les deux langages donnent exactement les mêmes nombres : 114 commandes en boutique pour un montant moyen d'environ 75 DT, comme au 3.1. (Le tri alphabétique des canaux est le même ; seule la présentation du tableau diffère.)

4.7.2 SAS : le langage des grandes organisations

Intuition. SAS (*Statistical Analysis System*) est un logiciel commercial né dans les années 1970. Il est resté très présent dans les **banques, les assurances, l'industrie pharmaceutique et les administrations**, où l'on valorise la stabilité, la traçabilité et le fait que les résultats d'un programme écrit il y a vingt ans soient toujours identiques. C'est souvent le cas dans les métiers de l'actuariat et du risque (le thème du volume V de cette série). Pour vous former gratuitement, SAS propose une version en ligne destinée à l'enseignement (*SAS OnDemand for Academics*).

Un programme SAS est une suite d'**étapes** : les étapes DATA fabriquent ou transforment des tables, les étapes PROC (procédures) appliquent un traitement statistique prêt à l'emploi. Chaque instruction se termine par un point-virgule, et un bloc par run ;.

```
“sas noexec /* SAS — non exécuté dans ce livre */ proc import datafile="donnees/commandes.csv"
out=commandes dbms=csv replace; guessingrows=max; run;
```

```
proc means data=commandes n mean std maxdec=1; class canal; /* un résumé par canal / var
montant; / la variable à résumer */ run;
```

On lit : « importer le CSV dans une table `commandes` » puis « *procédure MEANS* : pour chaque `canal`

4.7.3 MATLAB : le calcul numérique des ingénieurs

> (i) **Intuition.** **MATLAB** (**Matrix Laboratory**) est un environnement commercial centré sur les

Deux particularités à connaître : les indices **commencent à 1**, et les fichiers de données se lisen

```
```matlab noexec
% MATLAB — non exécuté dans ce livre
T = readtable("donnees/commandes.csv");

G = groupsummary(T, "canal", ["mean" "std"], "montant");
disp(G)
```

groupsummary regroupe les lignes par canal et calcule la moyenne et l'écart-type de montant. Le résultat est une nouvelle table avec les colonnes mean\_montant et std\_montant.

#### 4.7.4 Julia : la promesse « rapide comme C, simple comme Python »

**Intuition.** **Julia** (libre, créé en 2012) vise un compromis : une syntaxe lisible proche de Python et de MATLAB, mais un code **compilé à la volée** presque aussi rapide qu'un programme en C. Il est apprécié pour le calcul scientifique, les simulations lourdes et l'optimisation. Son écosystème de science des données est plus jeune que celui de Python ; une particularité est le temps d'attente à la première exécution (la compilation).

```
```julia noexec # Julia — non exécuté dans ce livre using CSV, DataFrames, Statistics
```

```
commandes = CSV.read("donnees/commandes.csv", DataFrame)
```

```
resume = combine(groupby(commandes, :canal), nrow => :n, :montant => mean => :moyenne,
:montant => std => :ecart_type) println(resume)
```

On retrouve la logique de pandas : `groupby` puis `combine` (on lit `colonne => fonction => nom_du_ré`

4.7.5 Le même calcul, vu de près : un dictionnaire de traduction

Voici de quoi passer d'un langage à l'autre. (Les cases SAS, MATLAB et Julia sont **non exécutées**,)

	Idée	Python	R	MATLAB	Julia	SAS
Affecter	<code>x = 5</code>	<code>x = 5</code>	<code>x <- 5</code>	<code>x = 5;</code>	<code>x = 5</code>	<code>x = 5;</code> (étape DATA)
Premier élément d'une liste	<code>x[0]</code>	<code>x[0]</code>	<code>x[1]</code>	<code>x(1)</code>	<code>x[1]</code>	<code>-</code>
Moyenne	<code>np.mean(x)</code>	<code>mean(x)</code>	<code>mean(x)</code>	<code>mean(x)</code>	<code>mean(x)</code>	<code>proc means</code>
Écart-type	<code>np.std(x, ddof=1)</code>	<code>sd(x)</code>	<code>std(x)</code>	<code>std(x)</code>	<code>std(x)</code>	<code>proc means std</code>
Lire un CSV	<code>pd.read_csv(f)</code>	<code>read.csv(f)</code>	<code>readtable(f)</code>	<code>CSV.read(f, DataFrame)</code>	<code>proc</code>	
Résumé par groupe	<code>groupby().agg()</code>	<code>group_by()</code>	<code>summarise()</code>	<code>groupsummary</code>	<code>proc m</code>	
Blocs	indentation	<code>{ }</code>	<code>end</code>	<code>end</code>	<code>run;</code>	

> (!) **Le piège des indices : 0 ou 1 ?** Python (comme C, Java) compte à partir de **0** ; R, MATLAB e

```
```python
x = [10, 20, 30, 40, 50]
print("x[0] =", x[0])
print("x[0:2] =", x[0:2], " (la borne de droite est exclue)")
print("x[-1] =", x[-1], " (le dernier élément)")
```

```
x[0] = 10
x[0:2] = [10, 20] (la borne de droite est exclue)
x[-1] = 50 (le dernier élément)
```

```
x <- c(10, 20, 30, 40, 50)
cat("x[1] =", x[1], "\n")
cat("x[1:2] =", x[1:2], " (la borne de droite est incluse)\n")
cat("x[-1] =", x[-1], " (tout sauf le premier élément !)\n")
```

```
x[1] = 10
x[1:2] = 10 20 (la borne de droite est incluse)
x[-1] = 20 30 40 50 (tout sauf le premier élément !)
```

Même symbole `x[-1]`, **deux sens opposés** : le dernier élément en Python, « tout sauf le premier » en R. Si vous traduisez du code d'un langage à l'autre, c'est la première chose à vérifier.

**Un point commun rassurant** :  $n - 1$ . Les écarts-types par défaut de pandas, R (`sd`), MATLAB (`std`) et Julia (`std`) divisent par  $n - 1$  (l'estimateur sans biais démontré au 3.2) ; seul `np.std` de NumPy divise par  $n$  si on ne précise pas `ddof=1`. Même formule, mêmes nombres : le fait que vos résultats concordent d'un langage à l'autre est d'ailleurs une excellente manière de **vérifier** un calcul.

#### 4.7.6 Alors, lequel choisir ?

Langage	Atouts	Limites	À privilégier si...
<b>Python</b>	généraliste, immense écosystème, apprentissage automatique, mise en production	graphiques statistiques moins « clés en main » que R	vous voulez <b>un seul langage</b> pour tout (ce livre)
<b>R</b>	statistique de référence, <code>ggplot2</code> , rapports reproductibles	moins à l'aise pour le développement logiciel	votre travail est surtout statistique ou académique
<b>SAS</b>	stabilité, validation réglementaire, très robuste sur de gros fichiers	licence payante, communauté plus fermée	votre employeur (banque, assurance, pharma) l'impose
<b>MATLAB</b>	calcul matriciel, boîtes à outils d'ingénierie	licence payante, moins utilisé en science des données	vous venez de l'ingénierie ou du signal
<b>Julia</b>	très rapide, syntaxe claire, calcul scientifique	écosystème plus jeune, compilation au premier appel	vous faites des <b>simulations lourdes</b> ou de l'optimisation

**Le conseil pratique.** On n'apprend pas un langage par collection, on l'apprend par **projet**. Maîtrisez Python (et un peu de R pour lire du code de statisticien). Le jour où un poste exige SAS, MATLAB ou Julia, la bonne nouvelle est que *les idées sont les mêmes* : tableau, regroupement, résumé, graphique. Il ne reste qu'à apprendre la syntaxe, avec ce dictionnaire sous la main. Regardez les offres d'emploi de votre domaine et de votre pays pour savoir quels langages y sont demandés avant d'investir du temps.

**À retenir (autres langages).**

- **SAS** : étapes DATA et PROC, point-virgule partout, très répandu en banque, assurance et pharma ; version d'apprentissage gratuite en ligne.
- **MATLAB** : tout est matrice, indices à partir de 1 ; **Octave** en est une alternative libre.
- **Julia** : rapide et lisible, compilé à la volée ; écosystème plus jeune.
- Python et R ont été exécutés ici ; **SAS, MATLAB et Julia ne l'ont pas été** (blocs « non exécuté »).
- Pièges de traduction : indices à partir de 0 ou de 1, signification de  $x[-1]$ , bornes des tranches ; l'écart-type par défaut divise par  $n - 1$  presque partout.
- Quel que soit le langage, le raisonnement est le même : **lire** → **regrouper** → **résumer** → **vérifier**.

## 4.8 ☉ Pour aller plus loin : complexité algorithmique et optimisation du code

**Section optionnelle.** Un code correct n'est pas toujours un code **utilisable** : le même résultat peut prendre une seconde ou trois jours selon la manière dont on s'y prend. La **complexité algorithmique** donne un langage pour comparer des méthodes *avant* de les écrire, et la mesure du temps (le *profilage*) dit où agir *après*. Cette section tient la promesse faite à la section 1.6, où nous avons vu que Dar Jasmin ne peut pas « énumérer tous les paniers possibles ».

### 4.8.1 L'idée : compter les opérations, pas les secondes

**Intuition.** Yasmine cherche un client dans son carnet d'adresses. Si le carnet n'est **pas trié**, elle doit lire les noms un par un : pour 1 000 clients, il lui faut en moyenne 500 lectures, et 1 000 dans le pire cas. Si le carnet est **trié par ordre alphabétique**, elle l'ouvre au milieu, regarde si le nom cherché est avant ou après, et élimine la moitié du carnet à chaque étape : 1 000 clients se règlent en **10 étapes** (car  $2^{10} = 1\,024$ ). Avec un million de clients, la première méthode demande un million de lectures, la seconde **20**.

Ce qui compte n'est pas la vitesse de l'ordinateur ou du langage, mais la **manière dont le nombre d'opérations grandit quand la taille  $n$  des données grandit**. C'est la **complexité** de l'algorithme. Vérifions-la en comptant effectivement les comparaisons :

```
def recherche_lineaire(liste, cible):
 """Lit les éléments un par un. Renvoie le nombre de comparaisons effectuées."""
 comparaisons = 0
 for x in liste:
 comparaisons += 1
 if x == cible:
 break
 return comparaisons

def recherche_binaire(triee, cible):
 """Coupe l'intervalle en deux à chaque étape (liste TRIÉE). Renvoie le nombre de comparaisons."""
 bas, haut, comparaisons = 0, len(triee) - 1, 0
 while bas <= haut:
 milieu = (bas + haut) // 2
 comparaisons += 1
 if trie[milieu] == cible:
 break
 if trie[milieu] < cible:
 bas = milieu + 1
 else:
```

```

 haut = milieu - 1
 return comparaisons

for n in [1_000, 1_000_000]:
 clients = list(range(n)) # identifiants triés 0, 1, ..., n-1
 cible = n - 1 # le pire cas pour la lecture linéaire
 print(f"n = {n:>9,} : linéaire = {recherche_lineaire(clients, cible):>9,} comparaisons, "
 f"binaire = {recherche_binaire(clients, cible)} comparaisons")

```

```

n = 1,000 : linéaire = 1,000 comparaisons, binaire = 10 comparaisons
n = 1,000,000 : linéaire = 1,000,000 comparaisons, binaire = 20 comparaisons

```

Ces nombres ne dépendent pas de la machine : c'est ce qui rend la complexité si utile. La première méthode est **linéaire**, la seconde **logarithmique** : multiplier  $n$  par 1 000 multiplie le travail de la première par 1 000, mais ajoute seulement une dizaine d'étapes à la seconde.

#### 4.8.2 La notation $O(\cdot)$ : une définition rigoureuse

On ne se soucie pas des détails (« 3 opérations par tour de boucle » ou « 5 »). On garde seulement la **forme de la croissance**, d'où une notation qui « oublie » les constantes.

**Définition (grand O).** On écrit  $f(n) = O(g(n))$  s'il existe une constante  $c > 0$  et un rang  $n_0$  tels que

$$f(n) \leq c g(n) \quad \text{pour tout } n \geq n_0.$$

En mots : à partir d'un certain rang,  $f$  ne dépasse pas un multiple fixe de  $g$ .

**Exemple fait à la main.** Un algorithme effectue  $f(n) = 3n^2 + 5n + 2$  opérations. Montrons que  $f(n) = O(n^2)$  avec  $c = 4$ . Il faut  $3n^2 + 5n + 2 \leq 4n^2$ , c'est-à-dire  $n^2 - 5n - 2 \geq 0$ . Pour  $n = 5$  :  $25 - 25 - 2 = -2 < 0$  (l'inégalité est fausse). Pour  $n = 6$  :  $36 - 30 - 2 = 4 \geq 0$  (vraie), et le trinôme est croissant ensuite :  $n_0 = 6$  convient. Vérifions par le calcul :

```

f = lambda n: 3 * n**2 + 5 * n + 2
faux = [n for n in range(1, 10_000) if f(n) > 4 * n**2]
print("valeurs de n pour lesquelles f(n) > 4 n^2 :", faux)
print("donc l'inégalité f(n) <= 4 n^2 est vraie dès n =", faux[-1] + 1)

```

```

valeurs de n pour lesquelles f(n) > 4 n^2 : [1, 2, 3, 4, 5]
donc l'inégalité f(n) <= 4 n^2 est vraie dès n = 6

```

Le terme dominant ( $n^2$ ) décide de tout, les termes d'ordre inférieur ( $5n, 2$ ) et le facteur 3 disparaissent. Voici les classes que vous rencontrerez le plus souvent, de la plus rapide à la plus lente :

Classe	Nom	Exemple typique
$O(1)$	constante	lire l'élément d'indice 5 d'une liste ; chercher une clé dans un dictionnaire
$O(\log n)$	logarithmique	recherche binaire dans une liste triée
$O(n)$	linéaire	parcourir une liste ; calculer une moyenne
$O(n \log n)$	quasi-linéaire	trier une liste (tri efficace)
$O(n^2)$	quadratique	comparer <b>toutes les paires</b> d'éléments

Classe	Nom	Exemple typique
$O(2^n)$	exponentielle	énumérer <b>tous les sous-ensembles</b> (tous les paniers possibles)

Pour sentir ce que ces lettres veulent dire en pratique, supposons un ordinateur capable de **un milliard d'opérations par seconde** (c'est du bon matériel) et calculons le temps que chaque classe demande :

```
import math

def duree(operations, vitesse=1e9):
 """Traduit un nombre d'opérations en durée lisible."""
 s = operations / vitesse
 for seuil, nom in [(3.15e7, "ans"), (86400, "jours"), (3600, "heures"), (60, "minutes"), (1,
 "secondes")]:
 if s >= seuil:
 valeur = s / seuil
 return f"{valeur:,.1f} {nom}" if valeur < 1e6 else f"{valeur:.2e} {nom}"
 if s < 1e-6:
 return f"{s * 1e9:.3g} ns"
 if s < 1e-3:
 return f"{s * 1e6:.3g} µs"
 return f"{s * 1e3:.3g} ms"

print(f"{'n':>10} | {'n':>12} | {'n log2 n':>12} | {'n^2':>14} | 2^n")
for n in [10, 40, 1_000, 1_000_000]:
 ligne = [duree(n), duree(n * math.log2(n)), duree(n**2)]
 expo = duree(2**n) if n <= 100 else "(astronomique)"
 print(f"{'n':>10,} | {'ligne[0]:>12} | {'ligne[1]:>12} | {'ligne[2]:>14} | {expo}")
print()
print("Pour n = 100 : 2^100 opérations demandent", duree(2**100))
print("(l'âge de l'Univers est d'environ 13,8 milliards d'années)")
```

n	n	n log2 n	n²	2^n
10	10 ns	33.2 ns	100 ns	1.02 µs
40	40 ns	213 ns	1.6 µs	18.3 minutes
1,000	1 µs	9.97 µs	1 ms	(astronomique)
1,000,000	1 ms	19.9 ms	16.7 minutes	(astronomique)

Pour n = 100 : 2<sup>100</sup> opérations demandent 4.02e+13 ans  
(l'âge de l'Univers est d'environ 13,8 milliards d'années)

Lisez la dernière colonne : pour  $n = 40$  éléments seulement, énumérer tous les sous-ensembles prendrait déjà plus de **18 minutes**... et pour  $n = 100$ , des milliers de fois l'âge de l'Univers (voir les deux lignes sous le tableau). À l'inverse, pour un million de clients, la méthode linéaire ou quasi-linéaire reste à la portée d'un ordinateur portable (de l'ordre de la milliseconde à la vingtaine de millisecondes), alors que la méthode quadratique demande environ **17 minutes**. Même matériel, mêmes données : seule la **complexité** change.

**Le grand O décrit la croissance, pas la vitesse.** Un algorithme  $O(n)$  avec une énorme constante peut être plus lent qu'un  $O(n^2)$  pour  $n$  petit. Mais, quand les données grossissent, la forme de la courbe finit toujours par gagner : c'est ce que montre la figure ci-dessous.

### 4.8.3 Mesurer : le temps, avec méthode

La théorie dit ce qui *devrait* se passer, la **mesure** vérifie ce qui se passe *vraiment*. Python fournit le module `timeit`. Deux précautions : répéter la mesure et garder le **minimum** (les autres pro-

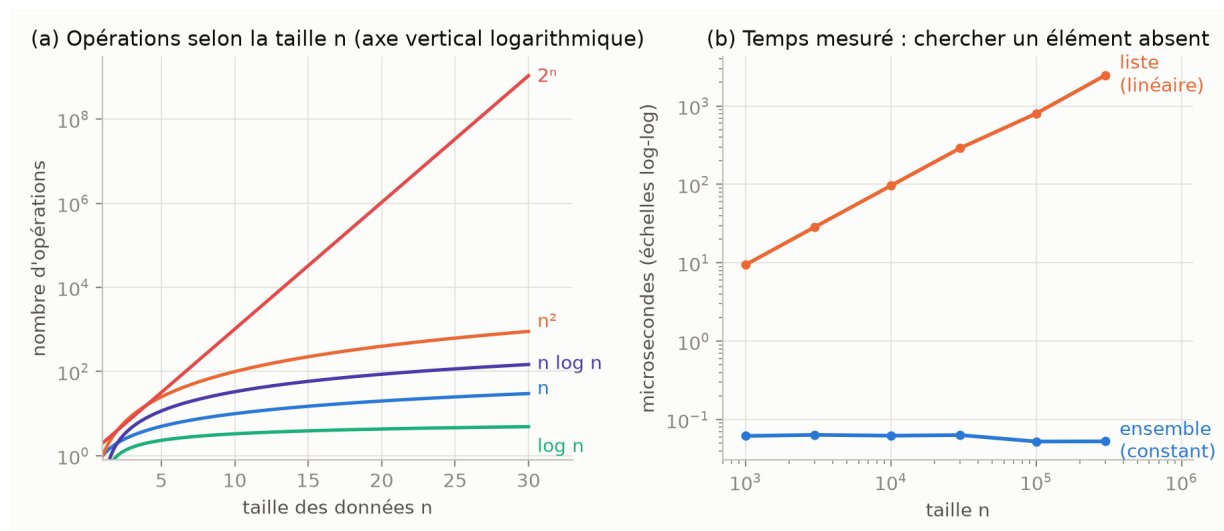


Figure 32: À gauche : nombre d'opérations selon la classe de complexité (échelle logarithmique en ordonnée). À droite : temps mesuré pour chercher un élément absent dans une liste (linéaire) et dans un ensemble (constant).

grammes de la machine ne peuvent que ralentir la mesure, jamais l'accélérer), et comparer des **rapports** plutôt que des durées brutes, car celles-ci dépendent de la machine.

**Les temps de cette section varient d'une machine à l'autre, et d'une exécution à l'autre.** Nous n'affichons donc que des **rapports arrondis**. Chez vous, les valeurs exactes différeront, mais l'**ordre de grandeur** doit être le même. Si ce n'est pas le cas, c'est intéressant : cherchez pourquoi !

**Test 1 : la liste contre l'ensemble.** Chercher si un identifiant est présent parmi  $n$  clients. Dans une liste (`x in liste`), Python lit les éléments un par un :  $O(n)$ . Dans un **ensemble** (set), qui range ses éléments à la manière d'un dictionnaire, la recherche est en  $O(1)$  en moyenne. Nous cherchons un élément **absent** (pire cas pour la liste) et nous multiplions  $n$  par 100 :

```
from timeit import repeat

def temps(f, nombre):
 """Temps minimal (en secondes) d'UN appel de f, sur 5 répétitions de `nombre` appels."""
 return min(repeat(f, number=nombre, repeat=5)) / nombre

resultats = {}
for n in [1_000, 100_000]:
 liste = list(range(n))
 ensemble = set(liste)
 resultats[n] = (temps(lambda: -1 in liste, 200), temps(lambda: -1 in ensemble, 200_000))

for nom, i in [("liste", 0), ("ensemble", 1)]:
 rapport = resultats[100_000][i] / resultats[1_000][i]
 print(f"{nom} : n est multiplié par 100 -> le temps est multiplié par {rapport:.1f}")

liste : n est multiplié par 100 -> le temps est multiplié par 82.7
ensemble : n est multiplié par 100 -> le temps est multiplié par 0.8
```

La liste a bien un temps proportionnel à  $n$  (multiplier  $n$  par 100 multiplie le temps par un nombre de l'ordre de 100), alors que l'ensemble est **insensible** à la taille (rapport proche de 1). Le gain n'est pas de 20 % : à  $n = 100\,000$ , la figure (b) ci-dessous montre un écart de **plusieurs ordres de grandeur** entre les deux courbes.

**Application directe.** Vous avez une liste de 100 000 clients à vérifier contre une liste de 50 000 clients « actifs ». Écrire `[c for c in clients if c in actifs]` avec `actifs` en liste fait jusqu'à  $100\,000 \times 50\,000 = 5 \cdot 10^9$  comparaisons. Une seule ligne, `actifs = set(actifs)`, ramène cela à  $\approx 100\,000$  opérations. C'est probablement l'optimisation la plus rentable de toute la data science pratique.

**Test 2 : le test du « doublement ».** Une technique simple pour deviner la complexité d'un code : doubler  $n$  et regarder de combien le temps est multiplié. Environ  $\times 2$  : linéaire. Environ  $\times 4$  : quadratique. Environ  $\times 8$  : cubique. Comparons deux manières de détecter des commandes en double :

```
def doublons_naif(ids):
 """Compare chaque paire : $O(n^2)$."""
 n = len(ids)
 return [ids[i] for i in range(n) for j in range(i + 1, n) if ids[i] == ids[j]]

def doublons_ensemble(ids):
 """Un seul passage avec un ensemble de valeurs déjà vues : $O(n)$."""
 vus, doubles = set(), []
 for x in ids:
 if x in vus:
 doubles.append(x)
 vus.add(x)
 return doubles

print("même résultat sur un petit exemple :", doublons_naif([4, 8, 4, 1, 8]), doublons_ensemble([4, 8,
↪ 4, 1, 8]))

for nom, f, tailles in [("naïf (paires)", doublons_naif, [1_000, 2_000, 4_000]),
 ("ensemble", doublons_ensemble, [50_000, 100_000, 200_000])]:
 jeux = [list(range(n)) for n in tailles] # les données sont préparées AVANT la
↪ mesure
 t = [temps(lambda ids=ids: f(ids), 1) for ids in jeux]
 print(f"{nom} : n x2 -> temps x{t[1]/t[0]:.1f} ; n x2 -> temps x{t[2]/t[1]:.1f}")
```

```
même résultat sur un petit exemple : [4, 8] [4, 8]
naïf (paires) : n x2 -> temps x4.1 ; n x2 -> temps x4.1
ensemble : n x2 -> temps x2.2 ; n x2 -> temps x2.5
```

La version par paires est **quadratique** : doubler  $n$  multiplie le temps par un nombre proche de 4. La version à un passage est **linéaire** : doubler  $n$  multiplie le temps par un nombre proche de 2 (un peu plus, parfois, à cause de la mémoire cache de l'ordinateur). Les mesures sont bruitées : relancez le bloc plusieurs fois et vous verrez ces rapports fluctuer légèrement, mais pas changer d'ordre de grandeur.

#### 4.8.4 Boucles Python contre calcul vectorisé

Au 4.4, nous avons dit que NumPy et pandas sont « rapides ». Voici de quoi : calculer la somme des carrés de 1 million de montants. Les deux versions sont en  $O(n)$ , mais la **constante** n'est pas du tout la même : une boucle Python interprète les tours un par un, alors que NumPy exécute une boucle en code compilé.

```
import numpy as np

rng = np.random.default_rng(7)
montants = np.exp(rng.normal(3.9, 0.55, size=1_000_000)) # 1 million de montants simulés
liste = montants.tolist()

def somme_carres_boucle(valeurs):
```



```

total = 0.0
for v in valeurs:
 total += v * v
return total

def somme_carres_numpy(valeurs):
 return float(np.sum(valeurs ** 2))

print("mêmes résultats ?", np.isclose(somme_carres_boucle(liste), somme_carres_numpy(montants)))
t_boucle = temps(lambda: somme_carres_boucle(liste), 3)
t_numpy = temps(lambda: somme_carres_numpy(montants), 3)
print(f"la version NumPy est environ {t_boucle / t_numpy:.0f} fois plus rapide")

```

```

mêmes résultats ? True
la version NumPy est environ 20 fois plus rapide

```

Même complexité théorique, mais un facteur de **dizaines** (voire de centaines selon la machine) en faveur de NumPy : la complexité ne dit donc pas tout, les constantes comptent aussi. La règle pratique du data scientist : **dès que vous écrivez une boucle for sur les lignes d'un tableau, demandez-vous s'il existe une opération vectorisée**. Même chose avec pandas : une opération sur une colonne entière est presque toujours préférable à apply ligne par ligne.

```

import pandas as pd

df = pd.DataFrame({"montant_ht": montants[:200_000]})
ttc_apply = df["montant_ht"].apply(lambda x: x * 1.19) # une fonction Python par ligne
ttc_colonne = df["montant_ht"] * 1.19 # une opération sur toute la colonne

print("mêmes résultats ?", np.allclose(ttc_apply, ttc_colonne))
t_apply = temps(lambda: df["montant_ht"].apply(lambda x: x * 1.19), 3)
t_colonne = temps(lambda: df["montant_ht"] * 1.19, 3)
print(f"l'opération sur la colonne est environ {t_apply / t_colonne:.0f} fois plus rapide")

```

```

mêmes résultats ? True
l'opération sur la colonne est environ 117 fois plus rapide

```

Même verdict avec pandas : apply appelle une fonction Python **pour chaque ligne**, alors que la multiplication de la colonne entière se fait d'un coup, dans du code compilé. Le facteur dépend de la machine (de l'ordre de plusieurs dizaines à plusieurs centaines de fois), mais le message est toujours le même.

#### 4.8.5 La mémoïsation : ne jamais calculer deux fois la même chose

**Intuition.** Quand on vous demande « combien font  $17 \times 23$  ? », vous calculez. Si on vous le redemande dix fois, vous n'allez pas refaire le calcul : vous vous souvenez de la réponse. La **mémoïsation** (*memoization*) fait de même : on **mémorise** le résultat d'une fonction pour chaque argument déjà rencontré.

L'exemple classique est la suite de Fibonacci :  $F(0) = 0$ ,  $F(1) = 1$ ,  $F(n) = F(n - 1) + F(n - 2)$ . La définition récursive est très élégante, mais elle recalcule les mêmes valeurs un nombre énorme de fois : pour calculer  $F(5)$ , on calcule  $F(3)$  deux fois,  $F(2)$  trois fois, etc. Comptons les appels :

```

from functools import lru_cache

appels = 0
def fib(n):
 global appels
 appels += 1
 return n if n < 2 else fib(n - 1) + fib(n - 2)

```

```
@lru_cache(maxsize=None) # une seule ligne ajoutée : « mémorise les résultats »
def fib_memo(n):
 return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

for n in [5, 15, 25]:
 appels = 0
 valeur = fib(n)
 print(f"F({n}) = {valeur:>6} : version naïve = {appels:>7,} appels", end="")
 fib_memo.cache_clear()
 fib_memo(n)
 print(f", avec mémoire = {fib_memo.cache_info().misses} calculs")

print("F(80) avec mémoire :", fib_memo(80), "(la version naïve ne terminerait jamais)")
```

```
F(5) = 5 : version naïve = 15 appels, avec mémoire = 6 calculs
F(15) = 610 : version naïve = 1,973 appels, avec mémoire = 16 calculs
F(25) = 75025 : version naïve = 242,785 appels, avec mémoire = 26 calculs
F(80) avec mémoire : 23416728348467685 (la version naïve ne terminerait jamais)
```

Pour  $F(25)$  : 242 785 appels contre 26 (un par valeur de 0 à 25). La version naïve est **exponentielle** (le nombre d'appels croît comme  $1,6^n$ ), la version mémorisée est **linéaire**. On a transformé un calcul impossible en un calcul instantané, en échange d'un peu de **mémoire** : c'est le compromis classique entre le temps et l'espace.

Ce compromis existe partout. Voici un exemple de l'espace utilisé par un million de montants :

```
import sys

liste_python = montants.tolist()
octets_liste = sys.getsizeof(liste_python) + sum(sys.getsizeof(v) for v in liste_python)
print(f"liste Python de 1 000 000 de flottants : {octets_liste / 1e6:.0f} Mo")
print(f"tableau NumPy du même contenu : {montants.nbytes / 1e6:.0f} Mo")

liste Python de 1 000 000 de flottants : 32 Mo
tableau NumPy du même contenu : 8 Mo
```

Un tableau NumPy range ses nombres côte à côte, sans « emballage » : environ quatre fois moins de mémoire ici, ce qui est l'une des raisons de sa vitesse.

#### 4.8.6 Profiler : mesurer avant d'optimiser

**Règle d'or.** « *L'optimisation prématurée est la racine de tous les maux.* » (Donald Knuth). Ne devinez pas où le programme est lent : **mesurez**. Dans un programme de 50 lignes, 95 % du temps se passe typiquement dans 1 ou 2 lignes. Les optimiser donne tout ; optimiser le reste ne change rien.

L'outil s'appelle un **profileur** : cProfile enregistre, pour chaque fonction, le nombre d'appels et le temps passé. Voici un petit rapport qui nettoie des identifiants de commandes, cherche les doublons et calcule une moyenne. Il est écrit sans malice, mais l'une de ses étapes est un piège caché. Laquelle ?

```
import cProfile
import pstats

def nettoyer(ids):
 return [int(x) for x in ids]

def moyenne(ids):
```

```

 return sum(ids) / len(ids)

def rapport(ids):
 propres = nettoyer(ids)
 n_doublons = len(doublons_naif(propres))
 return n_doublons, moyenne(propres)

donnees = [str(i % 3000) for i in range(6000)] # 6 000 identifiants dont 3 000 en double

profil = cProfile.Profile()
profil.runcall(rapport, donnees)
stats = pstats.Stats(profil)
total = sum(tt for (_, _, tt, _, _) in stats.stats.values())
classement = sorted(stats.stats.items(), key=lambda kv: kv[1][2], reverse=True)

print(f"{'fonction':<18}{'appels':>10}{'part du temps':>16}")
for (fichier, ligne, nom), (cc, nc, tt, ct, _) in [c for c in classement if c[0][0] != "~"][:4]: #
 ↪ on écarte les fonctions internes
 print(f"{nom:<18}{nc:>10,}{100 * tt / total:>14.0f} %")

```

fonction	appels	part du temps
doublons_naif	1	100 %
nettoyer	1	0 %
rapport	1	0 %
moyenne	1	0 %

Le coupable est immédiat : `doublons_naif` (notre détecteur par paires, quadratique) occupe l'immense majorité du temps, loin devant le nettoyage et la moyenne, qui ne comptent presque pour rien. Inutile d'accélérer nettoyer : on remplace plutôt `doublons_naif` par `doublons_ensemble`, qui est linéaire (4.8.3). C'est la démarche en trois temps de tout travail d'optimisation : **mesurer, trouver le goulot d'étranglement, changer d'algorithme ou de structure de données**, puis **mesurer encore** pour confirmer le gain.

**Ordre des priorités.** (1) D'abord un code **correct** et lisible (4.6, avec ses tests). (2) Ensuite, si c'est trop lent, **mesurer**. (3) Optimiser d'abord l'**algorithme** (complexité), ensuite la **vectorisation**, et seulement en dernier recours les micro-détails. Un test qui reste vert après l'optimisation prouve que vous n'avez rien cassé.

#### 4.8.7 Application : les paires de produits achetés ensemble

Retour à Dar Jasmin. Yasmine voudrait savoir quels produits sont **souvent achetés ensemble** pour proposer des offres groupées. Son catalogue contient 40 produits. Première idée : examiner **tous les paniers possibles** de 10 produits... Souvenez-vous du 1.6 : il y en a  $\binom{40}{10} = 847\,660\,528$ . Même à un million de paniers examinés par seconde, cela fait plus de 14 minutes *pour une seule taille de panier*, et le nombre de sous-ensembles totaux est  $2^{40} \approx 10^{12}$ . Mais Yasmine n'a pas besoin de tous les paniers **possibles** : seulement des paniers **réellement achetés**. Il suffit de compter, panier par panier, les paires qu'il contient : la complexité dépend alors de la taille des données, pas de la taille de l'univers des possibles.

```

from collections import Counter
from itertools import combinations

rng = np.random.default_rng(21)
n_paniers, n_produits = 20_000, 40
paniers = []
for _ in range(n_paniers):
 taille = int(rng.integers(2, 6)) # entre 2 et 5 produits
 panier = set(rng.choice(n_produits, size=taille, replace=False).tolist())

```

```

if rng.random() < 0.15: # 15 % des clientes achètent le couple 7 + 12
 panier |= {7, 12}
 paniers.append(sorted(panier))

compteur = Counter()
operations = 0
for panier in paniers:
 for paire in combinations(panier, 2): # toutes les paires DU panier
 compteur[paire] += 1
 operations += 1

print(f"{n_paniers:,} paniers lus, {operations:,} paires comptées au total")
print(f"(contre {math.comb(40, 10):,} sous-ensembles à 10 produits en énumérant tout)")
print("les 3 paires les plus fréquentes :")
for paire, effectif in compteur.most_common(3):
 print(f" produits {paire} : {effectif:,} paniers ({100 * effectif / n_paniers:.1f} %)")

```

```

20,000 paniers lus, 120,761 paires comptées au total
(contre 847,660,528 sous-ensembles à 10 produits en énumérant tout)
les 3 paires les plus fréquentes :
 produits (7, 12) : 3,017 paniers (15.1 %)
 produits (7, 8) : 395 paniers (2.0 %)
 produits (5, 12) : 392 paniers (2.0 %)

```

En à peine plus de 120 000 opérations (au lieu de plusieurs centaines de millions), le couple (7, 12) ressort nettement : c'est l'association que nous avons planifiée dans la simulation, et les autres paires, simplement dues au hasard, ont des effectifs bien plus faibles. La méthode est **linéaire** en nombre de paniers (chaque panier de  $k$  produits fournit  $\binom{k}{2}$  paires, et  $k \leq 5$  ici). C'est exactement l'idée derrière les algorithmes de recommandation (« règles d'association ») : on compte ce qui s'est vraiment passé, on n'explore pas ce qui aurait pu se passer.

#### À retenir (complexité et optimisation).

- La **complexité** mesure comment le nombre d'opérations grandit avec  $n$ , indépendamment de la machine.  $f(n) = O(g(n))$  : à partir d'un rang  $n_0$ ,  $f \leq c g$ .
- De la plus lente à la plus rapide :  $O(2^n)$  (impraticable dès  $n \approx 50$ ),  $O(n^2)$ ,  $O(n \log n)$ ,  $O(n)$ ,  $O(\log n)$ ,  $O(1)$ .
- **Structure de données = complexité**. Chercher dans une liste est  $O(n)$ , dans un set ou un dictionnaire  $O(1)$  ; une liste triée permet la recherche binaire en  $O(\log n)$ .
- **Test du doublement** :  $n$  double, temps  $\times 2 \rightarrow$  linéaire ;  $\times 4 \rightarrow$  quadratique.
- **Vectorisez** : NumPy et pandas battent les boucles Python de plusieurs ordres de grandeur, à complexité égale.
- **Mémoïsation** (`@lru_cache`) : échanger de la mémoire contre du temps, en ne calculant jamais deux fois la même chose.
- **Mesurez, ne devinez pas** : profilez (cProfile), trouvez le goulot, changez d'algorithme, remesurez. Les durées varient d'une machine à l'autre ; les ordres de grandeur et les rapports, non.

## 4.9 Exercices corrigés

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 2 et 3 sont ceux que la section 4.1.10 vous avait promis. Les exercices 13 et 14 reprennent les sections optionnelles 4.6 et 4.8.

## Énoncés

**Exercice 1 ★ (4.1 : dictionnaires et boucles).** Cinq commandes (`canal`, `montant`) : ("Site", 40), ("Instagram", 25), ("Site", 60), ("Boutique", 100), ("Instagram", 35). Calculez à la main le chiffre d'affaires de chaque canal et sa part du total. Écrivez ensuite le programme avec un dictionnaire et une boucle, sans bibliothèque.

**Exercice 2 ★★ (4.1.10 : un produit absent du catalogue).** Reprenez le programme de la caisse. (a) Que se passe-t-il pour le panier [ ("bol", 2), ("vase", 1) ] ? (b) Faites afficher un message **aimable** (nom du produit fautif et liste des produits disponibles) au lieu d'un plantage. (c) Variante : ignorez les produits inconnus, calculez le total TTC du reste du panier et listez ce qui a été ignoré. Quel total TTC attendez-vous à la main ?

**Exercice 3 ★★ (4.1.10 : un code promo).** Yasmine crée le code JASMIN15 (15 % de remise sur le hors-tax) et le code RENTREE5 (5 %). Règle : la remise promo **ne se cumule pas** avec la remise fidélité (10 % au-delà de 100 DT) ; on applique la **meilleure des deux**. Un code inconnu est ignoré avec un message. Calculez à la main le TTC du panier « 2 bols, 1 plateau, 3 bougies » avec JASMIN15, puis celui de « 2 tasses » avec le même code, et vérifiez par le code.

**Exercice 4 ★ (4.2 : R).** Quelle part des commandes est « satisfaite » ( $\text{note} \geq 4$ ) dans chaque canal ? (a) Sur les notes 5, 4, 2, 3, 5 de l'exemple, calculez la part à la main. (b) Répondez sur `donnees/commandes.csv` en R de base (tapply sur un test logique), puis (c) avec dplyr, et (d) contrôlez le résultat avec pandas.

**Exercice 5 ★ (4.3.3 : une pile).** La notation polonaise inversée (NPI) écrit l'opérateur **après** ses opérands :  $3\ 4\ +$  signifie  $3 + 4$ . Elle se calcule avec une pile et n'a besoin d'aucune parenthèse. (a) Évaluez à la main  $3\ 4\ +\ 2\ *$ , puis  $5\ 1\ 2\ +\ 4\ * + 3\ -$ . (b) Écrivez la fonction d'évaluation. (c) Le prix TTC d'un article à 80 DT HT avec 10 % de remise s'écrit  $80\ 1\ 0.1\ -\ * 1.19\ *$  : calculez-le. (d) Que doit faire la fonction devant  $3\ +$  ?

**Exercice 6 ★★ (4.3.3 : une file à deux emballeuses).** Six commandes arrivent aux minutes 0, 1, 2, 3, 10 et 11 ; emballer une commande dure 4 minutes. Calculez à la main l'attente de chaque commande (a) avec **une** emballeuse, (b) avec **deux**, puis écrivez la simulation. Indice : retenez, pour chaque emballeuse, l'instant où elle sera libre ; la prochaine commande est confiée à celle qui est libre **le plus tôt** (le module heapq sait trouver le plus petit élément).

**Exercice 7 ★★ (4.3.5 : une dichotomie à variante).** `bisect_left(liste, x)` renvoie la **première** position où  $x$  pourrait être inséré dans une liste triée. (a) Évaluez à la main cette position pour  $x = 2$  dans `[1, 2, 2, 2, 5]`. (b) Écrivez votre propre version par dichotomie, en énonçant son invariant. (c) Déduisez-en le nombre de commandes ayant la note 5 dans la colonne `satisfaction` triée, et comparez avec Counter. (d) Vérifiez votre fonction contre `bisect_left` sur 1 000 listes aléatoires.

**Exercice 8 ★ (4.3.6 : tri stable).** Rangez les commandes ("Site", 40.1), ("Boutique", 65.8), ("Site", 19.6), ("Boutique", 17.4), ("Instagram", 30.1), ("Site", 75.0) **par canal croissant puis, dans chaque canal, par montant décroissant**, avec uniquement des appels à `sorted`. Dans quel ordre faut-il enchaîner les deux tris ? Que se passe-t-il si on les inverse ?

**Exercice 9 ★ (4.4 : NumPy et broadcasting).** Les quantités vendues sur trois jours pour (bol, tasse, plateau, bougie) sont les lignes de  $Q = \begin{pmatrix} 2 & 0 & 1 & 3 \\ 1 & 1 & 0 & 0 \\ 0 & 4 & 2 & 1 \end{pmatrix}$ , les prix HT sont `[12.5, 8, 45, 15.9]`. (a) Calculez à la main le chiffre d'affaires de chaque jour. (b) Une promotion retire `[0, 10 %, 0, 20 %]` aux prix ; recalculez le chiffre d'affaires quotidien **en une seule ligne** grâce au broadcasting. (c) Quel produit s'est le mieux vendu en nombre d'unités ?

**Exercice 10 ★★ (4.4 : groupby et dates).** Six ventes : (5 janv., Site, 30), (20 janv., Instagram, 50), (2 févr., Site, 70), (10 févr., Site, 20), (11 févr., Boutique, 100), (1er mars, Instagram, 40). (a) Dressez à la main le tableau **mois × canal** des montants et les totaux mensuels. (b) Obtenez-le avec `pivot_table`. (c) Sur les 400 commandes (avec les colonnes `date` et `id_client` simulées en 4.4.5), quel mois rapporte le plus, et quel jour de la semaine compte le plus de commandes ?

**Exercice 11 ★★ (4.4.10 : jointure et clients dormants).** (a) Quatre commandes portent les numéros de client 1, 1, 3, 5 ; la table des clients contient les numéros 1 à 5. Qui n'a jamais commandé ? (b) Sur les données du livre (table `clients` de 4.4.10), combien de clients n'ont **aucune** commande ? Répondez de deux façons : avec `isin`, puis avec une jointure `left` et le comptage des valeurs manquantes. (c) Quel est le chiffre d'affaires par ville ?

**Exercice 12 ★★ (4.5 : un graphique honnête).** Tracez le montant moyen par canal en barres **triées par ordre décroissant**, avec un axe qui commence à zéro, un titre qui énonce le message, des axes nommés avec leur unité. Vérifiez par le code que les hauteurs des barres sont bien les moyennes, que l'axe commence à 0 et que le fichier est enregistré. Citez deux défauts qu'aurait une version avec axe tronqué et camembert 3D.

**Exercice 13 ★★ (4.6 : un test unitaire).** Règle de livraison : retrait en boutique gratuit ; sur le Site ou Instagram, 7 DT, **offerts à partir de 100 DT TTC** (100 compris) ; un canal inconnu est une erreur. (a) Dressez la liste des cas à tester, en pensant aux **bornes**. (b) Écrivez la fonction **avec le défaut classique** (`>` au lieu de `>=`) et les tests avec `pytest` ; constatez l'échec. (c) Corrigez et relancez.

**Exercice 14 ★★★ (4.8 : coût d'un algorithme).** On veut compter les **paires de commandes passées par le même client**. (a) Combien de paires de commandes y a-t-il en tout parmi  $n = 400$  ? Et parmi  $n = 800$  ? Quel est le facteur ? (b) Écrivez la version « double boucle » et comptez ses tours. (c) Écrivez une version en  $O(n)$  avec un dictionnaire de compteurs (un client avec  $c$  commandes produit  $c(c-1)/2$  paires). (d) Vérifiez qu'elles donnent le même résultat. (e) Estimez, avec la formule, le nombre de tours de la double boucle pour un million de commandes.

## Corrigés

**Corrigé 1.** Site :  $40 + 60 = 100$  ; Instagram :  $25 + 35 = 60$  ; Boutique : 100. Total : 260. Parts :  $100/260 \approx 38,5\%$  pour le Site,  $60/260 \approx 23,1\%$  pour Instagram,  $38,5\%$  pour la Boutique (la somme des parts doit faire 100 %).

```
commandes = [("Site", 40), ("Instagram", 25), ("Site", 60), ("Boutique", 100), ("Instagram", 35)]
ca = {}
for canal, montant in commandes:
 ca[canal] = ca.get(canal, 0) + montant # .get évite la KeyError au premier passage
total = sum(ca.values())
for canal, valeur in sorted(ca.items()):
 print(f"{canal:<10} {valeur:>4} DT {valeur / total:6.1%}")
print("total :", total, "| somme des parts :", round(sum(v / total for v in ca.values()), 6))
```

```
Boutique 100 DT 38.5%
Instagram 60 DT 23.1%
Site 100 DT 38.5%
total : 260 | somme des parts : 1.0
```

**Corrigé 2.** (a) Le programme cherche `CATALOGUE["vase"]`, qui n'existe pas : Python s'arrête avec `KeyError: 'vase'` (voir 4.1.8). (b) On rattrape l'erreur avec `try / except KeyError`, et on utilise la clé fautive que l'exception transporte. (c) On sépare les produits connus des inconnus **avant** de calculer. À la main : 2 bols = 25 DT HT, sous le seuil de remise ; TVA  $25 \times 0,19 = 4,75$  ; TTC = 29,75 DT.



```

CATALOGUE = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0, "bougie": 15.9} # identique à 4.1.10
TVA, SEUIL_REMISE, TAUX_REMISE = 0.19, 100, 0.10

def sous_total(panier):
 return sum(CATALOGUE[nom] * qte for nom, qte in panier)

def remise(montant_ht):
 return montant_ht * TAUX_REMISE if montant_ht > SEUIL_REMISE else 0.0

panier = [("bol", 2), ("vase", 1)]
try:
 sous_total(panier)
except KeyError as e:
 print("(a) KeyError, clé fautive :", e)
 print(f"(b) Désolé, {e.args[0]!r} n'est pas au catalogue. Disponibles : {'',
 ↪ '.join(sorted(CATALOGUE))}.".")

def total_prudent(panier):
 """Renvoie (total TTC, liste des produits ignorés)."""
 connus = [(nom, qte) for nom, qte in panier if nom in CATALOGUE]
 ignores = [nom for nom, _ in panier if nom not in CATALOGUE]
 ht = sous_total(connus)
 net = ht - remise(ht)
 return round(net * (1 + TVA), 2), ignores

print("(c)", total_prudent(panier))

```

(a) KeyError, clé fautive : 'vase'  
 (b) Désolé, 'vase' n'est pas au catalogue. Disponibles : bol, bougie, plateau, tasse.  
 (c) (29.75, ['vase'])

On retrouve le TTC de 29,75 DT calculé à la main, et 'vase' est signalé comme ignoré. Le choix entre « refuser le panier » (b) et « ignorer et signaler » (c) est une **décision métier**, pas technique : ce qui compte, c'est de ne jamais avaler une erreur en silence.

**Piège.** Un except: nu (sans type d'erreur) rattrape *tout*, y compris vos propres fautes de frappe : on ne rattrape que l'erreur **attendue** (KeyError, ValueError...).

**Corrigé 3.** Panier de 4.1.10 : sous-total HT = 117,70. Remise fidélité : 10 % soit 11,77 DT ; remise promo : 15 % soit 17,655 DT. On garde la meilleure, la promo : net =  $117,70 \times 0,85 = 100,045$  ; TTC =  $100,045 \times 1,19 = 119,05355 \approx 119,05$  DT (contre 126,06 sans code). Panier de 2 tasses : HT = 16, pas de remise fidélité (sous 100 DT), promo 15 % : net = 13,60, TTC =  $13,60 \times 1,19 = 16,184 \approx 16,18$  DT.

```

CODES = {"JASMIN15": 0.15, "RENTREE5": 0.05}

def total_ttc(panier, code=None):
 ht = sous_total(panier)
 taux = TAUX_REMISE if ht > SEUIL_REMISE else 0.0 # remise fidélité
 if code is not None:
 if code in CODES:
 taux = max(taux, CODES[code]) # la meilleure des deux, sans cumul
 else:
 print(f" code {code!r} inconnu : ignoré")
 return round(ht * (1 - taux) * (1 + TVA), 2)

```

```

gros = [("bol", 2), ("plateau", 1), ("bougie", 3)]
print("gros panier sans code :", total_ttc(gros))
print("gros panier JASMIN15 :", total_ttc(gros, "JASMIN15"))
print("gros panier RENTREE5 :", total_ttc(gros, "RENTREE5"), "(la fidélité de 10 % est meilleure)")
print("2 tasses JASMIN15 :", total_ttc([("tasse", 2)], "JASMIN15"))
print("2 tasses code bidon :", total_ttc([("tasse", 2)], "PROM099"))
assert total_ttc(gros, "JASMIN15") == 119.05 and total_ttc([("tasse", 2)], "JASMIN15") == 16.18

```

```

gros panier sans code : 126.06
gros panier JASMIN15 : 119.05
gros panier RENTREE5 : 126.06 (la fidélité de 10 % est meilleure)
2 tasses JASMIN15 : 16.18
code 'PROM099' inconnu : ignoré
2 tasses code bidon : 19.04

```

Les valeurs coïncident avec la main. Remarquez le troisième cas : RENTREE5 (5 %) est **moins bon** que la fidélité (10 %), donc le total reste celui sans code.

**Corrigé 4.** (a) Notes 5, 4, 2, 3, 5 : trois notes sur cinq sont  $\geq 4$ , soit 60 %. (b) En R, un test logique vaut TRUE (1) ou FALSE (0) : la **moyenne** d'un test logique est donc la **proportion** de TRUE. (c) La version dplyr. (d) Le contrôle avec pandas utilise exactement la même idée.

```

notes <- c(5, 4, 2, 3, 5)
mean(notes >= 4) # (a) : 0.6

df <- read.csv("donnees/commandes.csv")
round(tapply(df$satisfaction >= 4, df$canal, mean), 3) # (b) R de base

suppressPackageStartupMessages(library(dplyr))
df |> # (c) dplyr
 group_by(canal) |>
 summarise(n = n(), part_satisfaits = round(mean(satisfaction >= 4), 3))

```

```

[1] 0.6
Boutique Instagram Site
0.956 0.630 0.696
A tibble: 3 × 3
 canal n part_satisfaits
 <chr> <int> <dbl>
1 Boutique 114 0.956
2 Instagram 138 0.63
3 Site 148 0.696

```

```

import pandas as pd

df = pd.read_csv("donnees/commandes.csv")
print((df["satisfaction"] >= 4).groupby(df["canal"]).mean().round(3)) # (d) pandas

```

```

canal
Boutique 0.956
Instagram 0.630
Site 0.696
Name: satisfaction, dtype: float64

```

Les trois méthodes donnent les mêmes proportions : 95,6 % de commandes satisfaites en **Boutique**, 69,6 % sur le **Site** et 63,0 % sur **Instagram**. La Boutique est loin devant : le retrait est immédiat, or la satisfaction baisse d'environ 0,18 point par jour de délai (4.2.5). C'est une description de l'échantillon : savoir si un écart est *significatif* demanderait un test, comme au 3.4.

**Corrigé 5.** (a) 3 4 + 2 \* : on empile 3 puis 4 ; + dépile 4 et 3 et empile 7 ; on empile 2 ; \* dépile 2 et 7



et empile 14. Résultat :  $(3+4) \times 2 = 14$ . Pour  $5 \ 1 \ 2 \ + \ 4 \ * \ + \ 3 \ -$  : pile  $[5], [5, 1], [5, 1, 2]; + \rightarrow [5, 3]; 4 \rightarrow [5, 3, 4]; * \rightarrow [5, 12]; + \rightarrow [17]; 3 \rightarrow [17, 3]; - \rightarrow [14]$ . Soit  $5 + (1 + 2) \times 4 - 3 = 14$ . **Attention à l'ordre** : pour  $-$  et  $/$ , le **premier** dépilé est l'opérande de **droite**. (c)  $80 \times (1 - 0,1) \times 1,19 = 85,68$  DT.

```
def evaluer_npi(expression):
 operations = {"+": lambda a, b: a + b, "-": lambda a, b: a - b,
 "*": lambda a, b: a * b, "/": lambda a, b: a / b}

 pile = []
 for jeton in expression.split():
 if jeton in operations:
 if len(pile) < 2:
 raise ValueError(f"expression invalide : il manque un opérande pour {jeton!r}")
 droite = pile.pop() # le dernier empilé est l'opérande de droite
 gauche = pile.pop()
 pile.append(operations[jeton](gauche, droite))
 else:
 pile.append(float(jeton))
 if len(pile) != 1:
 raise ValueError("expression invalide : il reste plusieurs valeurs sur la pile")
 return pile[0]

print(evaluer_npi("3 4 + 2 *"))
print(evaluer_npi("5 1 2 + 4 * + 3 -"))
print(round(evaluer_npi("80 1 0.1 - * 1.19 *"), 2))
try:
 evaluer_npi("3 +")
except ValueError as e:
 print("(d) ValueError :", e)
```

```
14.0
14.0
85.68
(d) ValueError : expression invalide : il manque un opérande pour '+'
```

(d) Devant  $3 \ +$ , la pile ne contient qu'une valeur quand arrive  $+$  : la fonction doit **signaler l'erreur** plutôt que de planter obscurément sur un pop de pile vide.

**Corrigé 6.** (a) Une emballeuse (libre en 0, 4, 8, 12, ...) : débuts 0, 4, 8, 12, 16, 20 ; attentes 0, 3, 6, 9, 6, 9 ; moyenne  $33/6 = 5,5$  min. (b) Deux emballeuses : la commande 0 démarre en 0 (emballeuse A, libre à 4) ; la 1 démarre en 1 (B, libre à 5), attente 0 ; la 2 attend A jusqu'à 4 : attente 2 (A libre à 8) ; la 3 attend B jusqu'à 5 : attente 2 (B libre à 9) ; la 4 (arrivée 10) trouve tout libre : attente 0 ; la 5 (arrivée 11) trouve B libre depuis 9 : attente 0. Attentes 0, 0, 2, 2, 0, 0 ; moyenne  $4/6 \approx 0,67$  min.

```
import heapq
from collections import deque

def simuler(arrivees, duree, nb_emballeuses):
 libres = [0] * nb_emballeuses # instant où chaque emballeuse sera libre (un « tas »)
 heapq.heapify(libres)
 file = deque(arrivees)
 attentes = []
 while file:
 arrivee = file.popleft() # premier arrivé, premier servi
 libre_a = heapq.heappop(libres) # l'emballeuse libre le plus tôt
 debut = max(arrivee, libre_a)
 attentes.append(debut - arrivee)
 heapq.heappush(libres, debut + duree)
 return attentes
```

```
arrivees = [0, 1, 2, 3, 10, 11]
for k in (1, 2):
 a = simuler(arrivees, 4, k)
 print(f"{k} emballeuse(s) : attentes {a} | moyenne {sum(a) / len(a):.2f} min")
```

```
1 emballeuse(s) : attentes [0, 3, 6, 9, 6, 9] | moyenne 5.50 min
2 emballeuse(s) : attentes [0, 0, 2, 2, 0, 0] | moyenne 0.67 min
```

Doubler l'effectif divise l'attente moyenne **par plus de huit** (de 5,5 à 0,67 minute) : les files d'attente sont non linéaires, et c'est ce qu'étudie la théorie évoquée au chapitre 2.

**Corrigé 7.** (a) Dans [1, 2, 2, 2, 5], la première position où l'on peut insérer 2 sans casser l'ordre est l'indice 1 (juste devant le premier 2). (b) On cherche le **plus petit indice  $p$  tel que  $\text{liste}[p] \geq x$**  (ou  $n$  s'il n'existe pas). *Invariant* : tous les éléments d'indice  $< g$  sont  $< x$  et tous ceux d'indice  $\geq d$  sont  $\geq x$ . *Initialisation* :  $g = 0, d = n$  (zones vides). *Conservation* : si  $\text{liste}[m] < x$ , tous ceux d'indice  $\leq m$  sont  $< x$  (liste triée) donc  $g = m + 1$  ; sinon tous ceux d'indice  $\geq m$  sont  $\geq x$  donc  $d = m$ . *Terminaison* : la zone  $[g, d[$  perd au moins la moitié de sa taille à chaque tour. À la sortie  $g = d$ , et l'invariant dit que  $g$  est la réponse. À la main sur  $x = 2$  :  $[g, d[ = [0, 5[, m = 2, \text{liste}[2] = 2$  n'est pas  $< 2$  donc  $d = 2$  ;  $m = 1$ , même chose,  $d = 1$  ;  $m = 0, \text{liste}[0] = 1 < 2$  donc  $g = 1$  ;  $g = d = 1$  : réponse 1. (c) Les notes sont entières : le nombre de 5 vaut  $\text{premiere\_position}(\text{liste}, 6) - \text{premiere\_position}(\text{liste}, 5)$  ; le code trouve 104 notes de 5 sur 400, comme Counter.

```
import bisect
import random
from collections import Counter

def premiere_position(triee, x):
 g, d = 0, len(triee)
 while g < d:
 m = (g + d) // 2
 if trie[m] < x:
 g = m + 1
 else:
 d = m
 return g

print("(a)", premiere_position([1, 2, 2, 2, 5], 2), "| occurrences de 2 :",
 premiere_position([1, 2, 2, 2, 5], 3) - premiere_position([1, 2, 2, 2, 5], 2))

notes = sorted(pd.read_csv("donnees/commandes.csv")["satisfaction"])
nb5 = premiere_position(notes, 6) - premiere_position(notes, 5)
print("(c) notes 5 :", nb5, "| Counter :", Counter(notes)[5])

random.seed(1)
for _ in range(1000):
 liste = sorted(random.choices(range(20), k=random.randint(0, 30)))
 x = random.randint(-1, 21)
 assert premiere_position(liste, x) == bisect.bisect_left(liste, x)
print("(d) 1000 listes aléatoires : même réponse que bisect_left")
```

```
(a) 1 | occurrences de 2 : 3
(c) notes 5 : 104 | Counter : 104
(d) 1000 listes aléatoires : même réponse que bisect_left
```

**Corrigé 8.** Les deux tris sont **stables** : le second préserve l'ordre produit par le premier pour les éléments à égalité. Il faut donc trier d'abord selon le critère **secondaire** (montant décroissant), puis selon le critère **principal** (canal).

```
cmds = [("Site", 40.1), ("Boutique", 65.8), ("Site", 19.6), ("Boutique", 17.4), ("Instagram", 30.1),
↪ ("Site", 75.0)]
bon = sorted(sorted(cmds, key=lambda c: c[1], reverse=True), key=lambda c: c[0])
print("secondaire puis principal :")
for c in bon:
 print(" ", c)
mauvais = sorted(sorted(cmds, key=lambda c: c[0]), key=lambda c: c[1], reverse=True)
print("ordre inversé (faux) :")
for c in mauvais:
 print(" ", c)
```

```
secondaire puis principal :
('Boutique', 65.8)
('Boutique', 17.4)
('Instagram', 30.1)
('Site', 75.0)
('Site', 40.1)
('Site', 19.6)
ordre inversé (faux) :
('Site', 75.0)
('Boutique', 65.8)
('Site', 40.1)
('Instagram', 30.1)
('Site', 19.6)
('Boutique', 17.4)
```

Le bon résultat est Boutique (65,8 puis 17,4), Instagram (30,1), Site (75,0 ; 40,1 ; 19,6). Dans la version inversée, le dernier tri est celui des montants : les canaux sont mélangés, et leur ordre n'est conservé que pour les montants égaux, ce qui n'arrive pas ici. Raccourci équivalent : `sorted(cmds, key=lambda c: (c[0], -c[1]))`.

**Corrigé 9.** (a) Jour 1 :  $2 \times 12,5 + 0 \times 8 + 1 \times 45 + 3 \times 15,9 = 25 + 45 + 47,7 = 117,7$ . Jour 2 :  $12,5 + 8 = 20,5$ . Jour 3 :  $4 \times 8 + 2 \times 45 + 15,9 = 32 + 90 + 15,9 = 137,9$ . (b) Prix remisés : 12,5 ; 7,2 ; 45 ; 12,72. Jour 1 :  $25 + 45 + 3 \times 12,72 = 108,16$  ; jour 2 :  $12,5 + 7,2 = 19,7$  ; jour 3 :  $4 \times 7,2 + 90 + 12,72 = 131,52$ . (c) Unités par produit (somme des colonnes) : 3, 5, 3, 4 : la tasse.

```
import numpy as np

Q = np.array([[2, 0, 1, 3], [1, 1, 0, 0], [0, 4, 2, 1]])
prix = np.array([12.5, 8, 45, 15.9])
promo = np.array([0, 0.10, 0, 0.20])

print("(a) CA par jour :", Q @ prix)
print("(b) CA avec promotion :", (Q * (prix * (1 - promo))).sum(axis=1))
print(" formes :", Q.shape, "*", prix.shape, "->", (Q * prix).shape, "(le vecteur est répété sur
↪ chaque ligne)")
produits = ["bol", "tasse", "plateau", "bougie"]
print("(c) unités par produit :", {p: int(q) for p, q in zip(produits, Q.sum(axis=0))}, "-> meilleur
↪ :", produits[Q.sum(axis=0).argmax()])
```

```
(a) CA par jour : [117.7 20.5 137.9]
(b) CA avec promotion : [108.16 19.7 131.52]
 formes : (3, 4) * (4,) -> (3, 4) (le vecteur est répété sur chaque ligne)
(c) unités par produit : {'bol': 3, 'tasse': 5, 'plateau': 3, 'bougie': 4} -> meilleur : tasse
```

$Q * \text{prix}$  marche parce que les formes (3,4) et (4,) sont **compatibles** : NumPy « étire » le vecteur sur les trois lignes (broadcasting, 4.4.3). `axis=1` somme **le long des colonnes**, c'est-à-dire produit un total par ligne (par jour).

**Corrigé 10.** (a) Janvier : Instagram 50, Site 30, Boutique 0 (total 80). Février : Site 70 + 20 = 90, Boutique 100 (total 190). Mars : Instagram 40 (total 40). (b) et (c) :

```

mini = pd.DataFrame({
 "date": pd.to_datetime(["2026-01-05", "2026-01-20", "2026-02-02", "2026-02-10", "2026-02-11",
 ↪ "2026-03-01"]),
 "canal": ["Site", "Instagram", "Site", "Site", "Boutique", "Instagram"],
 "montant": [30, 50, 70, 20, 100, 40],
})
mini["mois"] = mini["date"].dt.month
tab = mini.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum", fill_value=0)
tab["total"] = tab.sum(axis=1)
print(tab)

--- les 400 commandes, avec les colonnes simulées de 4.4.5 (même recette, même graine)
df = pd.read_csv("donnees/commandes.csv")
rng = np.random.default_rng(7)
jours = rng.integers(0, 140, size=len(df))
df["date"] = pd.Timestamp("2026-01-05") + pd.to_timedelta(jours, unit="D")
df["id_client"] = rng.integers(1, 121, size=len(df))
df = df.sort_values("date").reset_index(drop=True)
df["mois"] = df["date"].dt.month

par_mois = df.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum",
 ↪ fill_value=0).round(0)
par_mois["total"] = par_mois.sum(axis=1)
print()
print(par_mois)
print("mois le plus rentable :", par_mois["total"].idxmax(), "(" , par_mois["total"].max(), "DT)")

noms = ["lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "dimanche"]
par_jour = df["date"].dt.dayofweek.value_counts().sort_index()
print()
print({noms[j]: int(n) for j, n in par_jour.items()})
print("jour le plus chargé :", noms[par_jour.idxmax()])

```

canal	Boutique	Instagram	Site	total
mois				
1	0	50	30	80
2	100	0	90	190
3	0	40	0	40

canal	Boutique	Instagram	Site	total
mois				
1	1861.0	853.0	1483.0	4197.0
2	1316.0	1366.0	1863.0	4545.0
3	1948.0	2073.0	2037.0	6058.0
4	1648.0	998.0	1752.0	4398.0
5	1755.0	1473.0	1672.0	4900.0

mois le plus rentable : 3 ( 6058.0 DT )

```

{'lundi': 53, 'mardi': 63, 'mercredi': 65, 'jeudi': 53, 'vendredi': 51, 'samedi': 52, 'dimanche': 63}
jour le plus chargé : mercredi

```

Le tableau de la main est reproduit exactement par `pivot_table` (le `fill_value=0` remplace les cases vides par 0 au lieu de NaN). Sur les 400 commandes, **mars** est le mois le plus rentable (6 058 DT). Attention toutefois à comparer des mois **inégalement couverts** : nos données vont du 5 janvier au 24 mai, donc janvier (27 jours) et mai (24 jours) sont incomplets, alors que mars l'est : un mois incomplet est un piège classique de lecture. Quant au jour de la semaine, les 140 jours de la période contiennent exactement 20 lundis, 20 mardis, etc. ; les dates ayant été tirées **au hasard et uniformément**, les écarts (de 51 commandes le vendredi à 65 le mercredi) ne sont que du hasard d'échantillonnage. Sur de vraies ventes, de telles différences guideraient les horaires d'ouverture ; ici, il ne faut surtout pas les « interpréter ».

**Corrigé 11.** (a) Les numéros 2 et 4 n'apparaissent dans aucune commande : ce sont les clients

**dormants.** (b) et (c) :

```

petit_clients = pd.DataFrame({"id_client": [1, 2, 3, 4, 5]})
petites_cmds = pd.DataFrame({"id_client": [1, 1, 3, 5]})
print("(a) sans commande :",
 ↪ petit_clients[~petit_clients["id_client"].isin(petites_cmds["id_client"])]["id_client"].tolist())

rng_c = np.random.default_rng(11) # même recette qu'en 4.4.10
clients = pd.DataFrame({
 "id_client": np.arange(1, 126),
 "ville": rng_c.choice(["Tunis", "Sfax", "Sousse", "Nabeul", "Bizerte"], size=125, p=[0.4, 0.2,
 ↪ 0.2, 0.1, 0.1]),
})

méthode 1 : isin
dormants1 = clients[~clients["id_client"].isin(df["id_client"])]
méthode 2 : jointure « left » depuis les clients, vers le nombre de commandes par client
nb_cmd = df.groupby("id_client").size().rename("nb_commandes").reset_index()
jointure = clients.merge(nb_cmd, on="id_client", how="left")
dormants2 = jointure[jointure["nb_commandes"].isna()]
print("(b) clients dormants :", len(dormants1), "=", len(dormants2), "| identiques :",
 ↪ dormants1["id_client"].tolist() == dormants2["id_client"].tolist())
print(" dont les clients 121 à 125 :", sorted(set(range(121, 126)) & set(dormants1["id_client"])))

ca_ville = df.merge(clients, on="id_client",
 ↪ how="left").groupby("ville")["montant"].sum().round(0).sort_values(ascending=False)
print("(c) CA par ville :")
print(ca_ville)
print("somme :", ca_ville.sum(), "| total des commandes :", round(df["montant"].sum()))

```

```

(a) sans commande : [2, 4]
(b) clients dormants : 11 = 11 | identiques : True
 dont les clients 121 à 125 : [121, 122, 123, 124, 125]
(c) CA par ville :
ville
Tunis 10491.0
Sousse 5492.0
Sfax 4518.0
Nabeul 2352.0
Bizerte 1245.0
Name: montant, dtype: float64
somme : 24098.0 | total des commandes : 24098

```

Le contrôle de cohérence final (la **somme par ville égale le total**) est un réflexe : une jointure qui dupliquerait ou perdrait des lignes (clés en double, clés absentes avec inner) fausserait silencieusement les totaux. C'est pourquoi on précise `how="left"` quand on veut conserver toutes les commandes. Onze clients sur 125 n'ont jamais commandé : les cinq clients 121 à 125 (dormants **par construction**, ce que le résultat confirme) et six autres que le hasard a laissés de côté. Tunis réalise à elle seule près de 44 % du chiffre d'affaires.

**Corrigé 12.** Le message est visible d'un coup d'œil si les barres sont triées et si l'axe part de zéro (la longueur de la barre porte l'information, voir 4.5.6).

```

import os
import tempfile
import matplotlib.pyplot as plt

moy = df.groupby("canal")["montant"].mean().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(6, 3.4))
barres = ax.bar(moy.index, moy.values, color="#2a78d6")
ax.bar_label(barres, fmt="%0.1f") # valeur écrite au-dessus de chaque
 ↪ barre
ax.set_ylim(bottom=0)

```

```

ax.set_title("La boutique a le panier moyen le plus élevé")
ax.set_xlabel("canal de vente")
ax.set_ylabel("montant moyen par commande (DT)")

chemin = os.path.join(tempfile.mkdtemp(), "panier-moyen.png")
fig.savefig(chemin, dpi=150, bbox_inches="tight")

print("ordre des barres :", [t.get_text() for t in ax.get_xticklabels()])
print("hauteurs = moyennes :", np.allclose([b.get_height() for b in barres], moy.values))
print("axe vertical démarre à :", ax.get_ylim()[0])
print("fichier enregistré :", os.path.getsize(chemin) > 0)
plt.close(fig)

```

```

ordre des barres : ['Boutique', 'Site', 'Instagram']
hauteurs = moyennes : True
axe vertical démarre à : 0.0
fichier enregistré : True

```

Les moyennes par canal sont celles obtenues au 4.1.9 (Boutique devant le Site, puis Instagram).

**Défauts d'une version truquée :** (1) un axe tronqué (par exemple de 45 à 75 DT) ferait paraître la barre de la Boutique plusieurs fois plus haute que celle d'Instagram alors que l'écart réel est de l'ordre de 50 % ; (2) un camembert en 3D déforme les aires par la perspective et force l'œil à comparer des angles, ce qu'il fait très mal ; pour trois catégories, des barres triées sont toujours plus lisibles.

**Corrigé 13.** (a) Cas à tester : boutique à n'importe quel montant (0) ; Site à 99,99 (7) ; Site à 100,00 exactement (0, la borne) ; Site à 150 (0) ; Instagram à 50 (7) ; canal inconnu (erreur). Les bornes sont l'endroit où se cachent les bogues. (b) La version fautive utilise > : un panier à 100,00 DT pile paierait 7 DT.

```

cat > livraison_frais.py <<'FIN'
def frais_livraison(canal, total_ttc):
 """Frais de livraison en DT : 0 en boutique ; 7 DT sinon, offerts dès 100 DT TTC."""
 if canal == "Boutique":
 return 0.0
 if canal in ("Site", "Instagram"):
 return 0.0 if total_ttc > 100 else 7.0 # <-- défaut volontaire : devrait être >=
 raise ValueError(f"canal inconnu : {canal!r}")
FIN

cat > test_livraison_frais.py <<'FIN'
import pytest
from livraison_frais import frais_livraison

@pytest.mark.parametrize("canal, total, attendu", [
 ("Boutique", 20.0, 0.0),
 ("Site", 99.99, 7.0),
 ("Site", 100.0, 0.0), # la borne : 100 compris
 ("Site", 150.0, 0.0),
 ("Instagram", 50.0, 7.0),
])
def test_frais(canal, total, attendu):
 assert frais_livraison(canal, total) == attendu

def test_canal_inconnu():
 with pytest.raises(ValueError):
 frais_livraison("Telephone", 10.0)
FIN

python -m pytest -q --color=no --tb=short -p no:cacheprovider test_livraison_frais.py 2>&1 | sed -E
↵ 's/ in [0-9.]+s//'

```

```

..F... [100%]
===== FAILURES =====
_____ test_frais[Site-100.0-0.0] _____
test_livraison_frais.py:12: in test_frais
 assert frais_livraison(canal, total) == attendu
E AssertionError: assert 7.0 == 0.0
E + where 7.0 = frais_livraison('Site', 100.0)
===== short test summary info =====
FAILED test_livraison_frais.py::test_frais[Site-100.0-0.0] - AssertionError: ...
1 failed, 5 passed

```

Le test de la borne a trouvé le défaut : pour 100,00 DT, la fonction renvoie 7 au lieu de 0 ; les cinq autres tests passent, preuve qu'un test « du milieu » n'aurait rien vu. (c) On corrige ( $\geq$ ) et on relance :

```

sed -i 's/total_ttc > 100 else/total_ttc >= 100 else/; s/ *# <-- défaut volontaire.*//'
↪ livraison_frais.py
python -m pytest -q --color=no --tb=short -p no:cacheprovider test_livraison_frais.py 2>&1 | sed -E
↪ 's/ in [0-9.]+s/'

```

```

..... [100%]
6 passed

```

**À retenir.** On teste les **bornes** (juste en dessous, pile, juste au-dessus) et les **cas d'erreur**. Ces tests, écrits une fois, protégeront la règle des 100 DT de toute régression future.

**Corrigé 14.** (a) Le nombre de paires parmi  $n$  éléments est  $\binom{n}{2} = n(n-1)/2$  : pour  $n = 400$ ,  $400 \times 399/2 = 79\,800$  ; pour  $n = 800$ ,  $800 \times 799/2 = 319\,600$ . Le facteur est  $319\,600/79\,800 \approx 4,005$  : **doubler  $n$  quadruple** le travail, signature d'un coût en  $O(n^2)$ . (b)–(d) :

```

from math import comb
import time

ids = df["id_client"].tolist() # les 400 numéros de client (4.4.5)

def paires_double_boucle(ids):
 tours, paires = 0, 0
 for i in range(len(ids)):
 for j in range(i + 1, len(ids)):
 tours += 1
 if ids[i] == ids[j]:
 paires += 1
 return paires, tours

def paires_dictionnaire(ids):
 compteurs = {}
 for c in ids: # un seul passage : O(n)
 compteurs[c] = compteurs.get(c, 0) + 1
 return sum(k * (k - 1) // 2 for k in compteurs.values()), len(ids)

for n in (200, 400):
 p1, tours = paires_double_boucle(ids[:n])
 p2, passages = paires_dictionnaire(ids[:n])
 print(f"n = {n} : double boucle {tours:>6} tours | dictionnaire {passages:>3} passages | "
 f"paires de même client : {p1} = {p2}")

t0 = time.perf_counter(); paires_double_boucle(ids); t1 = time.perf_counter()
paires_dictionnaire(ids); t2 = time.perf_counter()

```

```
print("la version dictionnaire est plus rapide :", (t2 - t1) < (t1 - t0))
print("rapport des tours quand n double (200 -> 400) :", round(79800 / 19900, 2), "pour la double
↳ boucle, 2.0 pour le dictionnaire")
print("(e) pour n = 1 000 000 :", f"{comb(1_000_000, 2):,}".replace(",", " "), "tours de double
↳ boucle")
```

```
n = 200 : double boucle 19900 tours | dictionnaire 200 passages | paires de même client : 212 = 212
n = 400 : double boucle 79800 tours | dictionnaire 400 passages | paires de même client : 702 = 702
la version dictionnaire est plus rapide : True
rapport des tours quand n double (200 -> 400) : 4.01 pour la double boucle, 2.0 pour le dictionnaire
(e) pour n = 1 000 000 : 499 999 500 000 tours de double boucle
```

Les deux versions comptent **le même nombre de paires** (702 parmi les 400 commandes, 212 parmi les 200 premières), mais l'une fait 79 800 tours et l'autre 400 passages. Quand  $n$  passe de 200 à 400, la double boucle est  $\approx 4$  fois plus longue ( $19\,900 \rightarrow 79\,800$ ), le dictionnaire seulement 2 fois. À un million de commandes la double boucle ferait environ  $5 \times 10^{11}$  tours, soit des heures voire des jours de calcul contre une fraction de seconde pour le dictionnaire : **même résultat, deux mondes de coûts**. Seule la mesure du temps dépend de la machine ; le booléen « le dictionnaire est plus rapide » vaut True partout, et les comptes de tours sont exacts.

## Bilan du chapitre 4

Vous savez maintenant :

- **programmer en Python** (4.1) : variables et types, collections (liste, tuple, dictionnaire, ensemble), conditions, boucles, fonctions, erreurs et try / except, lecture d'un fichier CSV, et un programme complet testé par assert ;
- **refaire les mêmes analyses en R** (4.2) : vecteurs, data.frame, tests statistiques « de la boîte », dplyr ; et savoir que Python et R donnent les mêmes nombres quand on leur pose la même question ;
- **raisonner comme un informaticien** (4.3) : choisir une structure (liste, dictionnaire, ensemble, pile, file), écrire une récursion avec son cas de base, prouver une dichotomie ou un tri par un invariant, et ne pas trier plus que nécessaire ;
- **manipuler des données** (4.4) : tableaux NumPy et broadcasting ; DataFrames pandas, sélection, groupby, jointures, valeurs manquantes, dates ;
- **dessiner honnêtement** (4.5) : choisir le bon graphique pour la question, l'anatomie d'un graphique matplotlib, seaborn, ggplot2, et les pièges du graphique trompeur (axe tronqué en tête) ;
- (en option, 4.6) **structurer et tester** : classes et dataclasses, code lisible, tests pytest qui visent les bornes ;
- (en option, 4.7) **situer SAS, MATLAB et Julia** par rapport à Python et R ;
- (en option, 4.8) **compter le coût** d'un algorithme (notation  $O(\cdot)$ ), préférer la vectorisation, mémoïser, et mesurer avant d'optimiser.

**Trois réflexes à emporter.** (1) *Calculer à la main un petit cas avant de coder* : c'est votre oracle. (2) *Lire le message d'erreur par la dernière ligne*. (3) *Vérifier un résultat par une seconde voie* (Python contre R, deux méthodes, une somme de contrôle).

Le chapitre 5 apprend à **aller chercher** les données là où elles vivent réellement : dans des bases de données relationnelles, avec le langage SQL. Vous y retrouverez commandes.csv, des jointures (comme en 4.4.10), des agrégations (comme le groupby) et l'idée de l'**index**, cet arbre trié qui permet une recherche dichotomique.



# Chapitre 5 : Bases de données et SQL

« Les données ne vivent presque jamais dans un fichier CSV. Elles vivent dans une **base de données**, et pour leur parler il faut connaître **SQL**. »

Jusqu'ici, nous avons travaillé sur un tableau déjà prêt, chargé en mémoire dans un notebook. Dans une vraie entreprise, ce n'est presque jamais le cas : les commandes, les clients, les stocks sont enregistrés dans une **base de données**, souvent plusieurs millions de lignes réparties dans des dizaines de tables liées entre elles. Avant de calculer la moindre moyenne, il faut donc **aller chercher** l'information, la **croiser**, la **résumer**. Le langage de cette étape s'appelle **SQL** (*Structured Query Language*). Il a plus de cinquante ans, il est partout, et c'est l'une des compétences les plus demandées dans les offres d'emploi de data scientist et de data analyst.

La bonne nouvelle : SQL se lit presque comme de l'anglais (ou, ici, du français traduit), et un petit nombre d'idées suffisent pour répondre à 90 % des questions réelles.

## Le chemin de ce chapitre

- **5.1 Modèle relationnel et conception de bases de données** : pourquoi une base plutôt qu'un fichier ? Tables, clés, relations. Nous construisons la base de Dar Jasmin (clients, produits, commandes, lignes de commande).
- **5.2 Requêtes SQL** : sélectionner, filtrer, trier, agréger, **joindre** plusieurs tables, imbriquer des requêtes, et éviter les pièges du NULL et des dates.
- **5.3 Fonctions fenêtres et CTE** : les outils des analystes expérimentés : classements, cumuls, moyennes mobiles, comparaison avec la ligne précédente, requêtes lisibles par étapes, requêtes récursives.
- **5.4 Normalisation et conception de schémas** : pourquoi on découpe les données en plusieurs tables, comment le faire proprement (formes normales), puis les index et les transactions.
- **⊕ 5.5 Pour aller plus loin : bases NoSQL** (MongoDB, Redis) : quand et pourquoi sortir du modèle relationnel.
- **5.6 Exercices corrigés.**

**Le fil conducteur : la base de données de Dar Jasmin.** Au chapitre 3, nous avons un seul tableau de 400 commandes. Ici, Yasmine passe à la vitesse supérieure : ses **400 commandes** (le fichier `donnees/commandes.csv`, inchangé) sont rangées dans une vraie base, à côté de la liste de ses **80 clients**, de ses **16 produits** et du **détail de chaque commande**. Tout est simulé avec une graine fixe : vous retrouverez exactement les mêmes résultats que dans le livre.

**Rien à installer.** Nous utilisons **SQLite**, une base de données complète qui tient dans un simple fichier et qui est déjà fournie avec Python (module `sqlite3`). Pas de serveur à configurer, pas de mot de passe. Le SQL que vous apprendrez ici fonctionne, à de petites diffé-

rences près (nous les signalons), sur PostgreSQL, MySQL, SQL Server, Oracle ou BigQuery. Le fichier `donnees/dar_jasmin.db`, fourni avec le livre, contient la base toute faite : vous pouvez aussi l'ouvrir avec n'importe quel outil graphique (DB Browser for SQLite, DBeaver...) ou avec la commande `sqlite3 donnees/dar_jasmin.db`.

**Comment lire les blocs sql.** Chaque requête est suivie de **sa sortie réelle**, exactement comme les blocs Python. Pour exécuter une requête depuis Python (et récupérer le résultat dans un tableau pandas, section 4.4), on écrit `pd.read_sql_query("SELECT ...", con)`, où `con` est la connexion à la base. C'est ce que fait l'outil de fabrication de ce livre en coulisses.

## 5.1 Modèle relationnel et conception de bases de données

**Intuition.** Une base de données relationnelle, c'est **un classeur de tableaux bien rangés** (les *tables*) **qui se parlent entre eux** grâce à des numéros d'identification (les *clés*). Au lieu de tout recopier dans un seul gros tableau, on range chaque chose à **un seul endroit** : les clients dans un tableau, les produits dans un autre, les commandes dans un troisième, et on les relie par des numéros.

### 5.1.1 Pourquoi pas simplement un fichier CSV ?

Yasmine a commencé avec un fichier Excel. Un jour, elle s'est retrouvée avec `commandes_v3_FINAL.xlsx`, `commandes_v3_FINAL_corrige.xlsx` et `commandes_v3_VRAIMENT_FINAL.xlsx`. Dans l'un, la cliente « Amel Ben Salah » habite Sfax, dans l'autre Sousse : laquelle est la bonne ? Personne ne sait. Les problèmes d'un fichier à plat sont toujours les mêmes :

Problème	Exemple chez Dar Jasmin	Ce que fait une base de données
<b>Redondance</b>	l'adresse d'un client est recopiée sur chacune de ses 28 commandes	elle est stockée <b>une seule fois</b>
<b>Incohérence</b>	deux lignes du même client donnent deux villes différentes	impossible : une seule ligne client, référencée par un numéro
<b>Données invalides</b>	une note de satisfaction de 7, un canal « TikTok » qui n'existe pas	des <b>contraintes</b> refusent la saisie
<b>Volume</b>	10 millions de lignes : le fichier ne s'ouvre plus	la base lit seulement ce dont elle a besoin (grâce aux <i>index</i> , 5.4)
<b>Accès simultané</b>	deux employés modifient le fichier en même temps : l'un écrase l'autre	des <b>transactions</b> gèrent les accès concurrents (5.4)
<b>Questions complexes</b>	« les clients de Sfax qui ont acheté des bijoux deux mois de suite »	un langage fait pour ça : <b>SQL</b>

Un **SGBD** (*système de gestion de bases de données*) est le logiciel qui garantit tout cela. Les plus courants : **PostgreSQL** et **MySQL** (libres, très répandus sur le web), **SQL Server** et **Oracle** (grandes entreprises), **SQLite** (une base dans un simple fichier, présente dans votre téléphone, votre navigateur et Python), **BigQuery**, **Snowflake**, **Redshift** (entrepôts de données dans le *cloud*). Tous parlent SQL, avec de petits accents régionaux (*dialectes*).

### 5.1.2 Le modèle relationnel

Le modèle relationnel date de 1970 (Edgar F. Codd, chez IBM). Il tient en quelques mots.

- Une **table** (ou *relation*) représente un type de chose : les clients, les produits...
- Une **colonne** (ou *attribut*) est une propriété de cette chose : prenom, ville...
- Une **ligne** (ou *enregistrement*, ou *n-uplet*) est une chose précise : la cliente n° 1.
- Chaque colonne a un **type** (entier, texte, date...).

Voici trois clients de Dar Jasmin sous forme de table :

id_client	prenom	nom	ville
1	Yassine	Lahmar	La Marsa
2	Sami	Dridi	Ariana
3	Aymen	Hamdi	Sfax

**Définition rigoureuse.** Soient  $D_1, \dots, D_p$  des ensembles appelés **domaines** (par exemple  $D_1 = \mathbb{N}$  pour les numéros,  $D_2 =$  les chaînes de caractères). Une **relation**  $R$  de **schéma**  $R(A_1 : D_1, \dots, A_p : D_p)$  est un **sous-ensemble fini** du produit cartésien

$$R \subseteq D_1 \times D_2 \times \dots \times D_p.$$

Deux conséquences importantes, souvent oubliées : (1)  $R$  étant un **ensemble**, il n'y a **ni doublons ni ordre** entre les lignes : demander « la première ligne » n'a pas de sens sans préciser un critère de tri ; (2) chaque case contient **une seule valeur** de son domaine (nous y reviendrons avec la première forme normale, 5.4).

**Les clés.** C'est là que la magie opère.

- Une **clé candidate** est un ensemble minimal de colonnes qui identifie **sans ambiguïté** une ligne. Dans `clients`, `id_client` en est une. Le couple (`prenom`, `nom`) n'en est pas une : nous verrons au 5.2.4 que notre propre base contient trois « Ines Dridi » dans trois villes différentes, et même deux « Emna Sassi » qui habitent toutes deux Bizerte. Un nom n'est **jamais** un bon identifiant.
- La **clé primaire** (*primary key*, PK) est la clé candidate choisie comme identifiant officiel. Elle ne peut être ni vide (NULL) ni répétée. Par convention, on utilise un **numéro** sans signification (une *clé de substitution*, ou *surrogate key*) : il ne change jamais, même si la personne déménage ou change de nom.
- Une **clé étrangère** (*foreign key*, FK) est une colonne qui **réfère la clé primaire d'une autre table**. Dans `commandes`, la colonne `id_client` désigne le client qui a passé la commande. C'est ainsi que les tables « se parlent ».
- L'**intégrité référentielle** est la règle qui en découle : une clé étrangère ne peut contenir que des valeurs qui **existent** dans la table référencée. Impossible d'enregistrer une commande du client n° 9999 s'il n'existe pas.

**L'analogie du carnet d'adresses.** Dans votre téléphone, vous ne recopiez pas toutes les informations d'Amel dans chaque SMS ; vous gardez une fiche « Amel » et vous écrivez à « Amel ». La fiche est la ligne de `clients`, son numéro est la clé primaire, et chaque SMS porte une clé étrangère vers elle.

**Les relations entre tables** se décrivent en trois types, selon le nombre de lignes de chaque côté :

- Un **à plusieurs** (1-N) : un client passe **plusieurs** commandes, mais chaque com-

mande est passée par **un seul** client. La clé étrangère se place du côté « plusieurs » (commandes.id\_client).

- **Plusieurs à plusieurs (N–N)** : une commande contient **plusieurs** produits, et un produit figure dans **plusieurs** commandes. Une clé étrangère ne suffit plus : on crée une **table d'association** (ou *de jonction*) qui contient une ligne par couple (commande, produit). C'est la table lignes\_commande, qui porte aussi les attributs du lien : quantité et prix\_unitaire.
- **Un à un (1–1)** : rare ; en pratique, on fusionne souvent les deux tables.

Un cas particulier amusant : la relation **réflexive**. Dar Jasmin a un programme de parrainage : un client peut avoir été **parrainé par un autre client**. La table clients se référence donc elle-même (id\_parrain pointe vers id\_client). Nous nous en servons pour les requêtes récursives (5.3).

Voici le **schéma** complet de notre base. Chaque boîte est une table ; l'étiquette **PK** désigne une clé primaire, **FK** une clé étrangère. Un trait relie chaque clé étrangère (côté « N », plusieurs) à la clé primaire qu'elle référence (côté « 1 »).

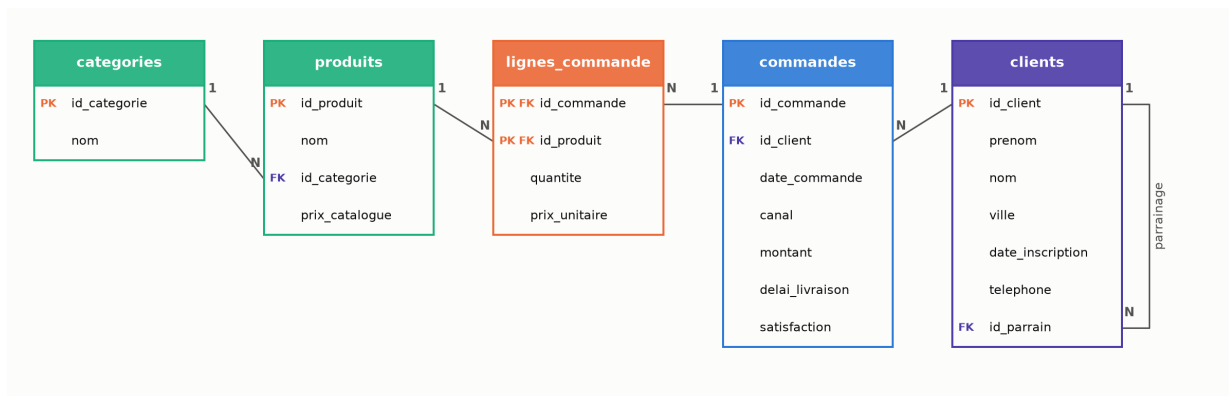


Figure 33: Schéma de la base de Dar Jasmin : cinq tables. Un trait relie chaque clé étrangère (N) à la clé primaire qu'elle référence (1). La table lignes\_commande est la table d'association entre commandes et produits.

**L'algèbre relationnelle : les maths derrière SQL.** Les requêtes SQL sont la traduction de cinq opérations sur les ensembles, que Codd a décrites dès 1970. Les connaître donne une compréhension durable, qui dépasse les différences de syntaxe entre les SGBD :

Opération	Notation	Sens	Équivalent SQL
<b>Sélection</b>	$\sigma_{\text{condition}}(R)$	garder certaines <b>lignes</b>	WHERE
<b>Projection</b>	$\pi_{A_1, \dots, A_k}(R)$	garder certaines <b>colonnes</b>	SELECT col1, col2
<b>Produit cartésien</b>	$R \times S$	<b>toutes les paires</b> (une ligne de R, une ligne de S)	CROSS JOIN
<b>Jointure</b>	$R \bowtie_{\text{cond}} S = \sigma_{\text{cond}}(R \times S)$	les paires qui vérifient une condition	JOIN ... ON
<b>Union / différence</b>	$R \cup S, R \setminus S$	ensembles de lignes	UNION, EXCEPT

La ligne la plus importante est celle de la **jointure** : c'est un produit cartésien **filtré**. Nous le verrons « en vrai » au 5.2.5 : 80 clients  $\times$  400 commandes = 32 000 paires, dont seules 400 sont les « bonnes ».

### 5.1.3 Construire la base de Dar Jasmin

Voici maintenant le code qui fabrique la base. Vous n'avez **pas besoin** de le comprendre ligne à ligne : il joue le rôle de l'« import de données » que, dans la vraie vie, quelqu'un d'autre aurait fait pour vous. Trois choses à remarquer.

1. Il **lit** le fichier `donnees/commandes.csv` du chapitre 3 : les 400 commandes, avec leur canal, leur montant, leur délai de livraison et leur satisfaction, sont **exactement** les mêmes qu'avant.
2. Il **invente** autour de ces commandes tout ce qu'un CSV ne contient pas : une date et un client pour chaque commande, la liste des clients (ceux qui commandent souvent, ceux qui ne commandent jamais), et le **détail** de chaque commande, produit par produit. La graine est fixée (`default_rng(2025)`), donc tout est reproductible.
3. Les lignes de commande sont construites pour que la **somme** (quantité × prix) de chaque commande retombe **exactement** sur son montant. Le prix payé peut différer de quelques pour cent du prix du catalogue : promotions, variation des prix dans l'année. Nous verrons au 5.4 pourquoi il est essentiel de **recopier** le prix dans la ligne de commande au lieu de le relire dans le catalogue.

Le cœur du code est la création des tables, en SQL (`CREATE TABLE`). Lisez-la attentivement : c'est la traduction exacte du schéma ci-dessus.

```
import sqlite3

import numpy as np
import pandas as pd

def construire(chemin_csv="donnees/commandes.csv"):
 rng = np.random.default_rng(2025)
 cmd = pd.read_csv(chemin_csv) # Les 400 commandes du chapitre 3
 n = len(cmd)

 # ---- catalogue : 4 catégories, 16 produits (prix en DT) -----
 categories = ["Poterie", "Textile", "Bijoux", "Cosmétiques"]
 produits = [
 ("Tajine décoratif", 1, 45), ("Bol en céramique de Nabeul", 1, 18),
 ("Vase peint à la main", 1, 65), ("Plat à couscous", 1, 38),
 ("Foutah en coton", 2, 28), ("Margoum (petit tapis)", 2, 120),
 ("Écharpe en soie", 2, 55), ("Pochette brodée", 2, 22),
 ("Bague en argent", 3, 48), ("Pendentif khamsa", 3, 35),
 ("Bracelet de perles", 3, 15), ("Boucles d'oreilles filigrane", 3, 42),
 ("Savon à l'huile d'olive", 4, 6), ("Eau de jasmin", 4, 14),
 ("Huile de nigelle", 4, 24), ("Bougie parfumée au jasmin", 4, 20),
]
 prix = np.array([p[2] for p in produits], dtype=float)

 # ---- dates : 400 commandes sur 2025, plus nombreuses en été et en fin d'année -----
 jours = pd.date_range("2025-01-01", "2025-12-31")
 poids_mois = np.array([0.8, 0.7, 0.9, 1.0, 1.0, 1.2, 1.4, 1.4, 1.0, 0.9, 1.3, 1.9])
 p_jour = poids_mois[jours.month - 1]
 dates = np.sort(rng.choice(jours, size=n, p=p_jour / p_jour.sum())) # id croissant =
 ↪ chronologique

 # ---- clients : 80 inscrits (dont 8 qui n'ont jamais commandé) -----
 prenom = ["Amel", "Sami", "Ines", "Walid", "Rim", "Hatem", "Salma", "Karim", "Nour", "Fares",
 "Mariem", "Oussama", "Lina", "Aymen", "Sarrah", "Yassine", "Dorra", "Bilel", "Emna",
 ↪ "Zied"]
 noms = ["Ben Salah", "Trabelsi", "Gharbi", "Jlassi", "Mansour", "Chaabane", "Hamdi", "Ayari",
 "Bouazizi", "Khelifi", "Dridi", "Mejri", "Sassi", "Zouari", "Ben Ammar", "Lahmar"]
 villes = ["Tunis", "La Marsa", "Ariana", "Sfax", "Sousse", "Nabeul", "Bizerte", "Monastir"]
 ville = rng.choice(villes, size=80, p=[0.22, 0.12, 0.10, 0.14, 0.12, 0.10, 0.10, 0.10])
```

```

local = np.isin(ville, ["Tunis", "La Marsa", "Ariana"]) # le magasin est à Tunis
poids = rng.gamma(1.5, 1.0, size=80) # quelques clients plus fidèles
poids[rng.choice(80, size=8, replace=False)] = 0 # inscrits dormants à vie
client = np.empty(n, dtype=int)
for i, c in enumerate(cmd["canal"]):
 w = poids * (local if c == "Boutique" else 1)
 client[i] = rng.choice(80, p=w / w.sum())
premiere = pd.Series(dates).groupby(client).min() # première commande de chaque
↪ client
inscr = np.array([premiere[k] - pd.Timedelta(days=int(rng.integers(0, 25))) if k in premiere.index
 else rng.choice(jours) for k in range(80)], dtype="datetime64[ns]")
ordre = np.argsort(inscr, kind="stable") # on renumérote : id croissant =
↪ inscription
nouveau = np.empty(80, dtype=int); nouveau[ordre] = np.arange(80)
client = nouveau[client]
clients = []
for rang, k in enumerate(ordre):
 pren, nom = rng.choice(prenoms), rng.choice(noms)
 tel = None if rng.random() < 0.2 else f"{rng.choice([20, 22, 24, 50, 52, 55, 98,
↪ 99])}{rng.integers(100000, 999999)}"
 parrain = int(rng.integers(1, rang + 1)) if rang >= 3 and rng.random() < 0.35 else None
 clients.append((rang + 1, pren, nom, ville[k], str(inscr[k])[:10], tel, parrain))

---- lignes de commande : le total de chaque commande doit retomber sur son montant -
lignes = []
for i in range(n):
 cible, meilleur = cmd["montant"][i], None
 for _ in range(150): # on cherche une composition
 ↪ plausible
 k = int(rng.choice([1, 2, 3], p=[0.55, 0.3, 0.15]))
 prod = rng.choice(16, size=k, replace=False)
 qte = rng.choice([1, 2, 3], size=k, p=[0.7, 0.22, 0.08])
 qte[-1] = 1 # la dernière ligne absorbera les
↪ arrondis
 f = cible / (prix[prod] * qte).sum() # facteur prix payé / prix
↪ catalogue
 if meilleur is None or abs(np.log(f)) < abs(np.log(meilleur[0])):
 meilleur = (f, prod, qte)
 f, prod, qte = meilleur
 pu = np.round(prix[prod] * f, 2) # prix payé : promos, évolution
↪ des prix
 pu[-1] = np.round((cible - (pu[:-1] * qte[:-1]).sum()) / qte[-1], 2)
 for p_, q_, u_ in zip(prod, qte, pu):
 lignes.append((i + 1, int(p_) + 1, int(q_), float(u_)))

---- création de la base (en mémoire) -----
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE categories (id_categorie INTEGER PRIMARY KEY, nom TEXT NOT NULL UNIQUE);
CREATE TABLE produits (
 id_produit INTEGER PRIMARY KEY, nom TEXT NOT NULL,
 id_categorie INTEGER NOT NULL REFERENCES categories(id_categorie),
 prix_catalogue REAL NOT NULL CHECK (prix_catalogue > 0));
CREATE TABLE clients (
 id_client INTEGER PRIMARY KEY, prenom TEXT NOT NULL, nom TEXT NOT NULL, ville TEXT NOT NULL,
 date_inscription TEXT NOT NULL, telephone TEXT,
 id_parrain INTEGER REFERENCES clients(id_client));
CREATE TABLE commandes (
 id_commande INTEGER PRIMARY KEY,
 id_client INTEGER NOT NULL REFERENCES clients(id_client),
 date_commande TEXT NOT NULL,
 canal TEXT NOT NULL CHECK (canal IN ('Instagram', 'Site', 'Boutique')),
 montant REAL NOT NULL, delai_livraison INTEGER NOT NULL,
 satisfaction INTEGER CHECK (satisfaction BETWEEN 1 AND 5));

```

```

CREATE TABLE lignes_commande (
 id_commande INTEGER NOT NULL REFERENCES commandes(id_commande),
 id_produit INTEGER NOT NULL REFERENCES produits(id_produit),
 quantite INTEGER NOT NULL CHECK (quantite > 0), prix_unitaire REAL NOT NULL,
 PRIMARY KEY (id_commande, id_produit));
"""
con.executemany("INSERT INTO categories VALUES (?, ?)", list(enumerate(categories, 1)))
con.executemany("INSERT INTO produits VALUES (?, ?, ?, ?)", [(i + 1, *p) for i, p in
↪ enumerate(produits)])
con.executemany("INSERT INTO clients VALUES (?, ?, ?, ?, ?, ?, ?)", clients)
con.executemany("INSERT INTO commandes VALUES (?, ?, ?, ?, ?, ?, ?)",
↪ [(i + 1, int(client[i]) + 1, str(dates[i])[10], cmd.canal[i],
float(cmd.montant[i]),
↪ int(cmd.livraison[i]), int(cmd.satisfaction[i])) for i in range(n)])
con.executemany("INSERT INTO lignes_commande VALUES (?, ?, ?, ?)", lignes)
con.commit()
return con

```

On crée maintenant la base et on la **sauvegarde dans un fichier** (c'est ce fichier `donnees/dar_jasmin.db` qui est fourni avec le livre) :

```

con = construire() # base en mémoire
fichier = sqlite3.connect("donnees/dar_jasmin.db")
con.backup(fichier) # copie vers le fichier
fichier.close()
print("base construite et enregistrée dans donnees/dar_jasmin.db")

```

base construite et enregistrée dans `donnees/dar_jasmin.db`

Vérifions ce qu'elle contient. Chaque base SQLite possède une table spéciale, `sqlite_master`, qui décrit toutes les autres :

```

SELECT name AS "table", type
FROM sqlite_master
WHERE type = 'table'
ORDER BY name;

```

table	type
categories	table
clients	table
commandes	table
lignes_commande	table
produits	table

Combien de lignes dans chaque table ? (Le mot-clé `UNION ALL` empile les résultats de plusieurs requêtes ; nous y reviendrons.)

```

SELECT 'categories' AS "table", COUNT(*) AS lignes FROM categories
UNION ALL SELECT 'produits', COUNT(*) FROM produits
UNION ALL SELECT 'clients', COUNT(*) FROM clients
UNION ALL SELECT 'commandes', COUNT(*) FROM commandes
UNION ALL SELECT 'lignes_commande', COUNT(*) FROM lignes_commande;

```

table	lignes
categories	4
produits	16
clients	80
commandes	400
lignes_commande	693

Un aperçu de chaque table. `SELECT *` signifie « toutes les colonnes » et `LIMIT 5` « seulement 5 lignes » :



```
SELECT * FROM clients LIMIT 5;
```

id_client	prenom	nom	ville	date_inscription	telephone	id_parrain
1	Yassine	Lahmar	La Marsa	2024-12-14	98725588	None
2	Sami	Dridi	Ariana	2024-12-16	NaN	None
3	Aymen	Hamdi	Sfax	2024-12-16	NaN	None
4	Emna	Zouari	Ariana	2024-12-18	NaN	None
5	Emna	Hamdi	Ariana	2024-12-30	98402250	None

Remarquez la colonne `id_parrain` : vide pour la plupart des clients (pandas affiche `None` ou `NaN` selon la colonne : c'est sa façon de montrer un `NULL`), mais quand elle est remplie, elle contient le numéro d'un autre client. Et le `telephone` vide de certains clients ? Ce « vide » a un nom en SQL : `NULL`. Il nous réservera quelques surprises au 5.2.7.

```
SELECT * FROM commandes LIMIT 5;
```

id_commande	id_client	date_commande	canal	montant	delai_livraison	satisfaction
1	2	2025-01-03	Boutique	44.8	0	4
2	6	2025-01-04	Site	34.5	2	4
3	3	2025-01-04	Instagram	88.2	5	4
4	1	2025-01-05	Instagram	30.1	4	4
5	5	2025-01-08	Boutique	110.1	0	5

```
SELECT * FROM lignes_commande WHERE id_commande <= 4;
```

id_commande	id_produit	quantite	prix_unitaire
1	1	1	44.80
2	10	1	34.50
3	2	1	17.84
3	3	1	64.42
3	13	1	5.94
4	11	1	15.57
4	14	1	14.53

Lisons ensemble cette dernière table, car elle est le cœur de la base. La commande n° 1 (dans la table `commandes`) a un montant de 44,80 DT : une seule ligne ici, le produit n° 1, en un exemplaire à 44,80 DT. La commande n° 3, elle, a un montant de 88,20 DT : trois lignes, trois produits différents, chacun en un exemplaire ( $17,84 + 64,42 + 5,94 = 88,20$ ). La table `commandes` ne dit **pas** ce qu'il y a dans le panier ; c'est `lignes_commande` qui le dit. Ce découpage est le propre d'une bonne base de données.

**Pourquoi une clé primaire à deux colonnes ?** Dans `lignes_commande`, la clé primaire est le **couple** (`id_commande`, `id_produit`) : une même commande ne peut pas contenir deux fois le même produit sur deux lignes séparées ; si le client en prend trois, on écrit `quantite = 3`. C'est une **clé composite**.

### 5.1.4 Types, contraintes : la base se défend

Relisez les `CREATE TABLE`. Outre les noms et les types, on y trouve des **contraintes** : des règles que la base fait respecter **toute seule**.

Contrainte	Exemple dans notre schéma	Effet
PRIMARY KEY	<code>id_client INTEGER PRIMARY KEY</code>	identifiant unique et non vide
NOT NULL	<code>prenom TEXT NOT NULL</code>	la valeur est obligatoire
UNIQUE	<code>nom TEXT NOT NULL UNIQUE</code> (catégories)	pas de doublon



Contrainte	Exemple dans notre schéma	Effet
CHECK	satisfaction BETWEEN 1 AND 5	la valeur doit vérifier une condition
REFERENCES	id_client ... REFERENCES clients(id_client)	clé étrangère (intégrité référentielle)

Voyons-les à l'œuvre en essayant d'enregistrer des données **absurdes** :

```
essais = {
 "client inexistant": "INSERT INTO commandes VALUES (9001, 9999, '2025-06-01', 'Site', 50, 2, 4)",
 "canal inconnu": "INSERT INTO commandes VALUES (9002, 1, '2025-06-01', 'TikTok', 50, 2, 4)",
 "note de 7 sur 5": "INSERT INTO commandes VALUES (9003, 1, '2025-06-01', 'Site', 50, 2, 7)",
 "numéro déjà pris": "INSERT INTO categories VALUES (1, 'Autre')",
 "prénom manquant": "INSERT INTO clients VALUES (500, NULL, 'Test', 'Tunis', '2025-06-01', NULL,
 NULL)",
}
for nom, requete in essais.items():
 try:
 con.execute(requete)
 print(f"{nom:18s} -> accepté (!)")
 except sqlite3.IntegrityError as erreur:
 print(f"{nom:18s} -> refusé : {erreur}")
print("commandes dans la base :", con.execute("SELECT COUNT(*) FROM commandes").fetchone()[0])
```

```
client inexistant -> refusé : FOREIGN KEY constraint failed
canal inconnu -> refusé : CHECK constraint failed: canal IN ('Instagram', 'Site', 'Boutique')
note de 7 sur 5 -> refusé : CHECK constraint failed: satisfaction BETWEEN 1 AND 5
numéro déjà pris -> refusé : UNIQUE constraint failed: categories.id_categorie
prénom manquant -> refusé : NOT NULL constraint failed: clients.prenom
commandes dans la base : 400
```

Chaque donnée absurde est **refusée avec un message clair**, et la table reste intacte. C'est la grande différence avec une feuille Excel, où rien n'empêche de taper « sept » dans la colonne des notes. Une base bien conçue rend les erreurs de saisie **impossibles** au lieu de les laisser se glisser puis fausser silencieusement une analyse.

**Particularités de SQLite (à savoir).** (1) SQLite n'applique les clés étrangères que si on le demande par `PRAGMA foreign_keys = ON` (c'est la deuxième ligne de notre code). Les autres SGBD les appliquent toujours. (2) SQLite est « souple » avec les types : il accepterait du texte dans une colonne d'entiers. PostgreSQL, lui, est strict. (3) SQLite n'a **pas de type date** : on stocke les dates en texte au format **ISO 8601** ('2025-06-01', année-mois-jour). Ce format a l'avantage que **l'ordre alphabétique est l'ordre chronologique**, ce qui permet de comparer et de trier correctement les dates comme des textes. N'utilisez jamais '01/06/2025' : le tri alphabétique donnerait n'importe quoi.

**Et l'argent ?** Nous stockons les montants en REAL (nombres à virgule flottante) par simplicité. En production, c'est une **mauvaise idée** : au chapitre 1 (analyse numérique, 1.5), nous avons vu que  $0,1 + 0,2 \neq 0,3$  en binaire. Les bons choix sont un type décimal exact (NUMERIC / DECIMAL), ou bien des **entiers** exprimés dans la plus petite unité : en Tunisie, le dinar se divise en **1 000 millimes**, donc 44,800 DT s'enregistre 44800. Les comptables vous remercieront.

### 5.1.5 SQL sur un fichier CSV : l'import en cinq lignes

Une dernière remarque pratique. Vous n'aurez pas toujours une base toute faite : souvent, vous recevrez un CSV. La bibliothèque pandas sait le charger dans SQLite d'un trait, et vous pouvez

alors utiliser SQL dessus : très pratique pour des agrégations compliquées ou pour s'entraîner.

```
con_csv = sqlite3.connect(":memory:") # une base temporaire, vide
pd.read_csv("donnees/commandes.csv").to_sql("commandes_csv", con_csv, index=False)

SQLite a deviné les types des colonnes :
print(pd.read_sql_query("SELECT sql FROM sqlite_master", con_csv).iloc[0, 0])
print()
requete = """SELECT canal, COUNT(*) AS commandes, ROUND(AVG(montant), 2) AS panier_moyen
FROM commandes_csv GROUP BY canal ORDER BY panier_moyen DESC"""
print(pd.read_sql_query(requete, con_csv))
```

```
CREATE TABLE "commandes_csv" (
 "canal" TEXT,
 "montant" REAL,
 "livraison" INTEGER,
 "satisfaction" INTEGER
)
```

	canal	commandes	panier_moyen
0	Boutique	114	74.81
1	Site	148	59.50
2	Instagram	138	49.01

On retrouve les effectifs par canal du chapitre 3 (114 commandes en boutique, 148 sur le site, 138 sur Instagram) et, en prime, les paniers moyens : la boutique est la plus généreuse. Remarquez la différence avec la vraie base : ici la table a été **devinée** (aucune clé, aucune contrainte), alors que notre base de Dar Jasmin a été **conçue**. Un import rapide convient pour explorer ; une base conçue est indispensable pour travailler à plusieurs et dans la durée.

### À retenir

- Une base relationnelle range chaque information à **un seul endroit** dans des **tables**, reliées par des **clés** (primaires, étrangères).
- Les **contraintes** (NOT NULL, CHECK, REFERENCES...) font respecter les règles métier **automatiquement**.
- Une relation **plusieurs à plusieurs** passe par une **table d'association** (lignes\_commande).
- Les requêtes SQL traduisent l'**algèbre relationnelle** : sélection, projection, produit cartésien, jointure (= produit filtré), union.
- Dates : format ISO 8601 en texte. Argent : décimaux exacts ou entiers (millimes), jamais de flottants en production.

## 5.2 Requêtes SQL : sélection, jointures, agrégation

**Intuition.** SQL est un langage **déclaratif** : on ne dit pas *comment* trouver le résultat (« parcours la table, compare, recopie... »), on décrit *ce qu'on veut* (« les commandes de plus de 100 DT passées sur Instagram »), et la base choisit seule la meilleure méthode. C'est comme commander au restaurant : on dit « un tajine, sans piment », pas la recette.

### 5.2.1 Anatomie d'une requête : SELECT ... FROM ...

La requête de base a deux morceaux obligatoires : **quoi** (SELECT, les colonnes) et **où** (FROM, la table).

```
SELECT id_commande, canal, montant
FROM commandes
LIMIT 4;
```

id_commande	canal	montant
1	Boutique	44.8
2	Site	34.5
3	Instagram	88.2
4	Instagram	30.1

On peut **calculer** de nouvelles colonnes et les **renommer** avec AS (un *alias*). Les montants de Dar Jasmin sont TTC, avec une TVA de 19 % ; calculons le hors taxe et la TVA :

```
SELECT id_commande,
 montant
 AS ttc,
 ROUND(montant / 1.19, 2)
 AS ht,
 ROUND(montant - montant / 1.19, 2)
 AS tva
FROM commandes
LIMIT 4;
```

id_commande	ttc	ht	tva
1	44.8	37.65	7.15
2	34.5	28.99	5.51
3	88.2	74.12	14.08
4	30.1	25.29	4.81

Voici la requête complète « tout-en-un », avec toutes les clauses possibles, **dans l'ordre où on les écrit** :

```
SELECT colonnes ou calculs -- quoi afficher
FROM table -- d'où ça vient
JOIN autre_table ON ... -- relier d'autres tables
WHERE condition sur les lignes -- quelles lignes garder
GROUP BY colonnes -- regrouper
HAVING condition sur les groupes -- quels groupes garder
ORDER BY colonnes -- trier
LIMIT n -- n lignes au maximum
```

**L'ordre d'écriture n'est pas l'ordre d'exécution.** Logiquement, la base procède ainsi : (1) FROM / JOIN (assembler les lignes), (2) WHERE (filtrer les lignes), (3) GROUP BY (regrouper), (4) HAVING (filtrer les groupes), (5) SELECT (calculer les colonnes demandées), (6) ORDER BY (trier), (7) LIMIT (couper). Conséquence classique : un alias défini dans le SELECT n'existe **pas encore** quand le WHERE s'exécute (certains SGBD tolèrent l'alias dans ORDER BY ou GROUP BY, pas dans WHERE). Si une requête refuse votre alias dans un WHERE, c'est pour cette raison.

**Lien avec l'algèbre relationnelle (5.1.2).** Une requête SELECT a, b FROM T WHERE c se lit :  $\pi_{a,b}(\sigma_c(T))$  : d'abord la **sélection** (les lignes), puis la **projection** (les colonnes). Le vocabulaire est trompeur : le SELECT de SQL correspond à la *projection* de l'algèbre, pas à sa *sélection* ! Le WHERE est la vraie sélection.

### 5.2.2 Filtrer avec WHERE

WHERE garde les lignes pour lesquelles la condition est **vraie**. Les comparaisons sont = (un seul signe égal, contrairement à Python), <> (différent), <, <=, >, >=. On combine avec AND, OR, NOT.

```
SELECT COUNT(*) AS nb
FROM commandes
WHERE canal = 'Instagram' AND montant > 100;
```

```
nb
11
```

(Les textes se mettent entre **apostrophes simples** : 'Instagram'. Les guillemets doubles servent aux noms de colonnes.) Il y a donc peu de grosses commandes sur Instagram. Pour tester une liste de valeurs, IN ; pour un intervalle (**bornes incluses**), BETWEEN ; pour une recherche de motif dans un texte, LIKE (% remplace n'importe quelle suite de caractères, \_ un seul caractère) :

```
SELECT COUNT(*) AS commandes_de_decembre
FROM commandes
WHERE date_commande BETWEEN '2025-12-01' AND '2025-12-31';
```

```
commandes_de_decembre
57
```

```
SELECT id_produit, nom, prix_catalogue
FROM produits
WHERE nom LIKE '%jasmin%';
```

```
id_produit nom prix_catalogue
14 Eau de jasmin 14.0
16 Bougie parfumée au jasmin 20.0
```

**Attention aux priorités.** AND passe avant OR, comme la multiplication passe avant l'addition. L'expression A OR B AND C se lit donc A OR (B AND C), ce qui n'est pas du tout (A OR B) AND C. Mesurons l'écart sur un exemple. On veut compter « les commandes de plus de 100 DT passées sur Instagram ou sur le site » :

```
SELECT 'sans parenthèses' AS version, COUNT(*) AS nb
FROM commandes
WHERE canal = 'Instagram' OR canal = 'Site' AND montant > 100
UNION ALL
SELECT 'avec parenthèses', COUNT(*)
FROM commandes
WHERE (canal = 'Instagram' OR canal = 'Site') AND montant > 100;
```

```
version nb
sans parenthèses 152
avec parenthèses 25
```

Sans parenthèses, la requête compte **toutes** les commandes Instagram (quel que soit le montant) *plus* les commandes du site de plus de 100 DT : 152 lignes. Avec parenthèses, elle compte les grosses commandes (plus de 100 DT) d'Instagram *ou* du site : 25 lignes. Un écart de 1 à 6 dans le résultat, à cause de deux caractères.

**Règle d'or.** Dès qu'on mélange AND et OR, **mettez des parenthèses**, même quand elles sont techniquement inutiles. Votre lecteur (et vous dans six mois) vous en sera reconnaissant.

### 5.2.3 Trier, limiter, dédoublonner

ORDER BY trie (ASC croissant par défaut, DESC décroissant). Rappelez-vous (5.1.2) que sans ORDER BY, **l'ordre des lignes n'est pas garanti**. LIMIT n ne garde que les n premières : indispensable pour des « top 5 ».

```
SELECT id_commande, date_commande, canal, montant
FROM commandes
ORDER BY montant DESC
LIMIT 5;
```

id_commande	date_commande	canal	montant
157	2025-06-23	Site	255.7
61	2025-03-28	Site	243.8
208	2025-07-31	Site	217.1
243	2025-08-26	Boutique	212.4
362	2025-12-16	Boutique	208.8

On peut trier sur plusieurs colonnes (ORDER BY canal, montant DESC : d'abord par canal, puis, à canal égal, du plus gros au plus petit montant). DISTINCT supprime les doublons :

```
SELECT DISTINCT ville
FROM clients
ORDER BY ville;
```

```
ville
Ariana
Bizerte
La Marsa
Monastir
Nabeul
Sfax
Sousse
Tunis
```

#### 5.2.4 Agréger : COUNT, SUM, AVG, GROUP BY

Jusqu'ici, chaque ligne du résultat correspondait à une ligne de la table. Le cœur de l'analyse de données, c'est l'inverse : **résumer** beaucoup de lignes en peu de chiffres. Les **fonctions d'agrégation** sont COUNT (nombre de lignes), SUM (somme), AVG (moyenne), MIN, MAX.

```
SELECT COUNT(*) AS nb_commandes,
 COUNT(DISTINCT id_client) AS nb_clients_distincts,
 ROUND(SUM(montant), 2) AS chiffre_affaires,
 ROUND(AVG(montant), 2) AS panier_moyen,
 MIN(montant) AS plus_petite,
 MAX(montant) AS plus_grande
FROM commandes;
```

nb_commandes	nb_clients_distincts	chiffre_affaires	panier_moyen	plus_petite	plus_grande
400	66	24098.3	60.25	8.6	255.7

Les 400 commandes totalisent environ 24 098 DT, soit un panier moyen de **60,25 DT** : c'est exactement la moyenne du chapitre 3. SQL et pandas calculent la même chose ; seule la syntaxe change. Autre information intéressante : sur les 80 clients inscrits, **66 seulement** ont passé au moins une commande.

**Une table de correspondance pandas ↔ SQL** (que vous retrouverez en 4.4) : `df[df.canal == "Site"] ↔ WHERE canal = 'Site' ; df.groupby("canal")["montant"].mean() ↔ SELECT canal, AVG(montant) ... GROUP BY canal ; df.sort_values("montant") ↔ ORDER BY montant.`

**GROUP BY** découpe la table en **groupes** (un par valeur de la colonne) et applique les agrégations **dans chaque groupe**. Voici, par canal, le nombre de commandes, le panier moyen, le chiffre d'affaires, la satisfaction moyenne et le délai de livraison moyen :

```
SELECT canal,
 COUNT(*) AS commandes,
 ROUND(AVG(montant), 2) AS panier_moyen,
 ROUND(SUM(montant)) AS chiffre_affaires,
```

```

ROUND(AVG(satisfaction), 2) AS satisfaction,
ROUND(AVG(delai_livraison), 2) AS delai_moyen
FROM commandes
GROUP BY canal
ORDER BY chiffre_affaires DESC;

```

canal	commandes	panier_moyen	chiffre_affaires	satisfaction	delai_moyen
Site	148	59.50	8807.0	3.79	4.70
Boutique	114	74.81	8528.0	4.49	0.00
Instagram	138	49.01	6764.0	3.72	4.49

En une requête de six lignes, nous avons la photographie de l'activité. Le **site** fait le plus de chiffre d'affaires parce qu'il a le plus de commandes ; la **boutique**, avec moins de commandes, a le panier moyen et la satisfaction les plus élevés (le délai de livraison y est nul : les clients repartent avec leur achat). **Instagram** a les paniers les plus modestes.

**La règle du GROUP BY.** Dans un SELECT avec GROUP BY, chaque colonne affichée doit être **soit dans le GROUP BY, soit à l'intérieur d'une fonction d'agrégation**. Demander SELECT canal, montant ... GROUP BY canal n'a pas de sens : quelle valeur de montant choisir parmi les 148 lignes du groupe « Site » ? (SQLite en choisit une, arbitrairement, sans protester ; PostgreSQL refuse avec une erreur : c'est plus prudent.)

**WHERE contre HAVING.** WHERE filtre les **lignes avant** le regroupement ; HAVING filtre les **groupes après** l'agrégation. On ne peut pas écrire WHERE COUNT(\*) > 10, car le décompte n'existe pas encore à ce moment-là.

```

SELECT ville, COUNT(*) AS clients
FROM clients
GROUP BY ville
HAVING COUNT(*) >= 11
ORDER BY clients DESC, ville;

```

ville	clients
Ariana	13
Bizerte	11
Sousse	11
Tunis	11

Un usage très pratique de GROUP BY ... HAVING : **détecter les doublons**. Reprenons notre remarque du 5.1.2 : peut-on identifier un client par son prénom et son nom ? Cherchons les couples qui apparaissent plus d'une fois (GROUP\_CONCAT recolle les villes en un seul texte) :

```

SELECT prenom, nom, COUNT(*) AS homonymes, GROUP_CONCAT(ville, ' / ') AS villes
FROM clients
GROUP BY prenom, nom
HAVING COUNT(*) > 1
ORDER BY homonymes DESC, nom;

```

prenom	nom	homonymes	villes
Oussama	Chaabane	3	Ariana / La Marsa / Sfax
Ines	Dridi	3	Ariana / Tunis / Nabeul
Salma	Bouazizi	2	Sousse / Sfax
Nour	Hamdi	2	Tunis / Sousse
Salma	Jlassi	2	Nabeul / Bizerte
Yassine	Lahmar	2	La Marsa / Ariana
Dorra	Mejri	2	Monastir / Tunis
Emna	Sassi	2	Bizerte / Bizerte

Huit couples de prénom et nom apparaissent plusieurs fois, dont trois « Ines Dridi » ! Et deux

« Emna Sassi » habitent la même ville : sans le numéro `id_client`, il serait **impossible** de les distinguer. La clé primaire n'est pas un luxe.

**COUNT(\*) ou COUNT(colonne) ?** COUNT(\*) compte les **lignes**. COUNT(colonne) compte les lignes où la colonne **n'est pas vide** (NULL) : nous y reviendrons au 5.2.7, c'est un grand classique des erreurs de comptage.

### 5.2.5 Les jointures : relier les tables

Le problème : la table commandes dit *quel numéro de client* a passé chaque commande, mais pas son nom ni sa ville. Pour les afficher ensemble, il faut **joindre** les deux tables.

**Une jointure est un produit cartésien filtré.** Reprenons l'algèbre relationnelle du 5.1.2 :  $R \bowtie_{\text{cond}} S = \sigma_{\text{cond}}(R \times S)$ . Voyons-le concrètement. Le produit cartésien « clients × commandes » forme **toutes les paires possibles** (une cliente, une commande), même absurdes :

```
SELECT COUNT(*) AS paires_possibles
FROM clients CROSS JOIN commandes;
```

```
paires_possibles
32000
```

Il y a bien  $80 \times 400 = 32\,000$  paires. Mais seules **400** sont « les bonnes » : celles où le numéro de client de la commande est celui de la cliente. En filtrant, on obtient la jointure :

```
SELECT COUNT(*) AS paires_valides
FROM clients CROSS JOIN commandes
WHERE clients.id_client = commandes.id_client;
```

```
paires_valides
400
```

C'est exactement ce que fait JOIN ... ON (qu'on écrit ainsi pour la lisibilité, et parce que le moteur l'exécute beaucoup plus finement que de fabriquer les 32 000 paires). L'équivalent :

```
SELECT cl.prenom, cl.nom, cl.ville, c.date_commande, c.montant
FROM commandes AS c
JOIN clients AS cl ON cl.id_client = c.id_client
ORDER BY c.montant DESC
LIMIT 5;
```

prenom	nom	ville	date_commande	montant
Fares	Bouazizi	Sousse	2025-06-23	255.7
Nour	Chaabane	Sfax	2025-03-28	243.8
Mariem	Sassi	Tunis	2025-07-31	217.1
Sami	Dridi	Ariana	2025-08-26	212.4
Sami	Dridi	Ariana	2025-12-16	208.8

Deux détails de lisibilité : on donne des **alias courts aux tables** (c, cl), et on **préfixe** chaque colonne par la table d'où elle vient (cl.nom), ce qui évite les ambiguïtés quand deux tables ont une colonne du même nom (comme `id_client`).

**Lire une jointure.** JOIN clients AS cl ON cl.id\_client = c.id\_client se lit : « pour chaque commande c, va chercher la ligne de clients dont le numéro est c.id\_client et accole ses colonnes ». Toujours : **clé étrangère = clé primaire**.

**Joindre plus de deux tables** se fait en enchaînant les JOIN. Quel est le chiffre d'affaires par **catégorie de produit** ? Il faut aller de `lignes_commande` à `produits`, puis à `categories` :



```

SELECT ca.nom AS categorie,
 SUM(l.quantite) AS unites_vendues,
 ROUND(SUM(l.quantite * l.prix_unitaire)) AS chiffre_affaires
FROM lignes_commande AS l
JOIN produits AS p ON p.id_produit = l.id_produit
JOIN categories AS ca ON ca.id_categorie = p.id_categorie
GROUP BY ca.nom
ORDER BY chiffre_affaires DESC;

```

categorie	unites_vendues	chiffre_affaires
Bijoux	200	7006.0
Textile	184	6994.0
Poterie	180	6950.0
Cosmétiques	196	3148.0

Les trois premières catégories sont presque à égalité (les **bijoux** devançant de justesse le **textile** et la **poterie**, autour de 7 000 DT chacun) ; les **cosmétiques** se vendent en grand nombre (196 unités, presque autant que les bijoux avec 200) mais à petits prix, donc leur total est bien plus faible. Ce genre d'écart entre « ce qui se vend le plus » et « ce qui rapporte le plus » est exactement le type d'information qu'on cherche.

Souvenez-vous aussi de la promesse faite en 5.1.3 : les lignes de commande devaient avoir été construites pour que chaque commande « retombe » sur son montant. Vérifions-le, avec une requête qui compte les commandes **incohérentes** (dont le montant diffère de la somme de leurs lignes) :

```

SELECT COUNT(*) AS commandes_incoherentes
FROM (
 SELECT c.id_commande
 FROM commandes AS c
 JOIN lignes_commande AS l ON l.id_commande = c.id_commande
 GROUP BY c.id_commande
 HAVING ABS(c.montant - SUM(l.quantite * l.prix_unitaire)) > 0.005
);

```

commandes_incoherentes
0

Zéro : tout est cohérent. Ce type de **contrôle de cohérence** (on appelle cela un test de qualité de données) est un réflexe à avoir à chaque fois qu'on reçoit une base. Vous avez aussi vu une **sous-requête** (la requête entre parenthèses dans le FROM) ; nous y revenons au 5.2.6.

**INNER JOIN contre LEFT JOIN.** Le JOIN simple (en réalité INNER JOIN) ne garde que les lignes qui **ont une correspondance des deux côtés**. Un client qui n'a jamais commandé **disparaît** du résultat ! Si l'on veut garder *tous* les clients, même sans commande, on utilise LEFT JOIN : toutes les lignes de la table de **gauche** sont conservées, et les colonnes de la table de droite sont remplies de NULL quand il n'y a pas de correspondance.

```

SELECT cl.id_client, cl.prenom, cl.nom, cl.ville
FROM clients AS cl
LEFT JOIN commandes AS c ON c.id_client = cl.id_client
WHERE c.id_commande IS NULL
ORDER BY cl.id_client;

```

id_client	prenom	nom	ville
10	Nour	Hamdi	Tunis
23	Hatem	Mejri	Sousse
31	Yassine	Chaabane	Nabeul
38	Oussama	Sassi	Monastir
42	Hatem	Bouazizi	Bizerte



56	Lina	Dridi	Bizerte
58	Salma	Jlassi	Nabeul
67	Zied	Gharbi	Sousse
69	Salma	Jlassi	Bizerte
73	Salma	Bouazizi	Sfax
75	Lina	Mejri	La Marsa
77	Nour	Hamdi	Sousse
78	Ines	Ayari	Tunis
79	Hatem	Trabelsi	Sousse

Ces quatorze clients se sont inscrits mais n'ont jamais acheté : une liste précieuse pour une campagne de relance (« votre premier achat à -10 % »). Le motif **LEFT JOIN ... WHERE droite IS NULL** (« les lignes de gauche sans correspondance à droite ») est l'un des plus utiles de tout SQL. Avec un **INNER JOIN**, ces quatorze clients n'auraient jamais été trouvés, puisqu'ils n'ont aucune commande à joindre.

**Les quatre jointures, en une phrase.** **INNER** : l'intersection (couples valides seulement). **LEFT** : tous les couples valides, plus les lignes de gauche sans partenaire (complétées par NULL). **RIGHT** : l'inverse (rarement utilisé : on échange simplement les deux tables). **FULL** : les deux. SQLite gère **INNER**, **LEFT**, **CROSS** et, depuis la version 3.39, **RIGHT** et **FULL**.

**L'auto-jointure.** Rien n'interdit de joindre une table **avec elle-même** (en lui donnant deux alias différents). C'est ce qu'il faut pour afficher, pour chaque client parrainé, le nom de son parrain :

```
SELECT f.prenom || ' ' || f.nom AS filleul,
 p.prenom || ' ' || p.nom AS parrain
FROM clients AS f
JOIN clients AS p ON p.id_client = f.id_parrain
ORDER BY f.id_client
LIMIT 5;
```

filleul	parrain
Bilel Bouazizi	Zied Ben Salah
Salma Mansour	Hatem Ayari
Oussama Bouazizi	Zied Ben Salah
Fares Bouazizi	Nour Hamdi
Amel Ayari	Emna Hamdi

(L'opérateur || recolle deux textes.) Imaginez la même table lue deux fois : une fois dans le rôle des filleuls (f), une fois dans celui des parrains (p).

**Les opérations ensemblistes** **UNION** (réunion sans doublon), **UNION ALL** (réunion en gardant les doublons), **INTERSECT** (intersection) et **EXCEPT** (différence) combinent **deux requêtes de même forme**. Combien de clients ont acheté à la fois en boutique et sur le site ?

```
SELECT COUNT(*) AS clients_boutique_et_site
FROM (
 SELECT id_client FROM commandes WHERE canal = 'Boutique'
 INTERSECT
 SELECT id_client FROM commandes WHERE canal = 'Site'
);
```

```
clients_boutique_et_site
21
```

### 5.2.6 Les sous-requêtes : une requête dans une requête

Une **sous-requête** est une requête placée entre parenthèses à l'intérieur d'une autre. Trois usages courants.

(a) **Une valeur unique**, utilisable comme un nombre : « les commandes dont le montant dépasse la moyenne ».

```
SELECT COUNT(*) AS commandes_au-dessus_de_la_moyenne,
 ROUND(AVG(montant), 2) AS leur_montant_moyen
FROM commandes
WHERE montant > (SELECT AVG(montant) FROM commandes);
```

commandes_au-dessus_de_la_moyenne	leur_montant_moyen
160	95.12

La sous-requête (SELECT AVG(montant) FROM commandes) vaut 60,25 ; la requête externe garde donc les commandes au-dessus de ce seuil. Il y en a **160 sur 400** : 40 %, bien moins que la moitié, signe d'une distribution asymétrique à droite (la moyenne dépasse la médiane, 3.1.3).

(b) **Une liste de valeurs**, avec IN : « les clients qui ont passé au moins une commande de plus de 200 DT ».

```
SELECT cl.id_client, cl.prenom, cl.nom
FROM clients AS cl
WHERE cl.id_client IN (SELECT id_client FROM commandes WHERE montant > 200);
```

id_client	prenom	nom
2	Sami	Dridi
14	Fares	Bouazizi
39	Nour	Chaabane
71	Mariam	Sassi

(c) **Une condition d'existence**, avec EXISTS : « les clients ayant déjà acheté en boutique ». La sous-requête est ici **corrélée** : elle fait référence à la ligne courante de la requête externe (clients.id\_client).

```
SELECT COUNT(*) AS clients_deja_venus_en_boutique
FROM clients
WHERE EXISTS (SELECT 1 FROM commandes AS c
 WHERE c.id_client = clients.id_client AND c.canal = 'Boutique');
```

clients_deja_venus_en_boutique
26

**Quand préférer une jointure à une sous-requête ?** Les deux marchent souvent. Une règle simple : si vous avez besoin des colonnes de l'autre table dans le résultat, **jointure**. Si l'autre table ne sert que de **filtre** (existence, appartenance), **EXISTS** / **IN**. Les CTE (5.3) rendront les requêtes imbriquées beaucoup plus lisibles.

### 5.2.7 Les pièges : NULL et dates

**Le NULL : ni zéro, ni texte vide, mais « inconnu »**

NULL signifie « **valeur absente ou inconnue** ». Ce n'est pas zéro, ce n'est pas le texte vide. Et il suit une logique à **trois valeurs** : vrai, faux et *inconnu*. Toute comparaison avec NULL donne *inconnu* : NULL = NULL n'est pas vrai (deux inconnues sont-elles égales ? On ne sait pas !), NULL <> 5 n'est pas vrai non plus. Et WHERE ne garde que les lignes **vraies**.

Dans notre base, la colonne `telephone` contient des `NULL` (le client n'a pas donné son numéro). Dénombrons :

```
SELECT COUNT(*) AS clients,
 COUNT(telephone) AS avec_telephone,
 SUM(telephone IS NULL) AS sans_telephone
FROM clients;
```

clients	avec_telephone	sans_telephone
80	65	15

`COUNT(*)` compte les 80 lignes ; `COUNT(telephone)` ignore les `NULL` et n'en compte que 65. (Dans SQLite, `telephone IS NULL` vaut 1 ou 0, d'où la somme.) La **bonne** façon de tester l'absence de valeur est `IS NULL` (ou `IS NOT NULL`). Voici ce qui arrive si l'on écrit naïvement `= NULL` :

```
SELECT COUNT(*) AS avec_egal_null
FROM clients
WHERE telephone = NULL;
```

avec_egal_null
0

Résultat : **zéro**, alors qu'il y a des clients sans téléphone, et aucun message d'erreur. Un deuxième piège, plus sournois : le **filtre « différent de »** qui oublie les inconnus.

```
SELECT (SELECT COUNT(*) FROM clients WHERE telephone <> '98725588') AS differents,
 (SELECT COUNT(*) FROM clients WHERE telephone <> '98725588' OR telephone IS NULL) AS
↪ differents_ou_inconnus;
```

differents	differents_ou_inconnus
64	79

Parmi les 80 clients, un seul a le numéro 98725588 ; on s'attendrait donc à 79 « autres » clients. Le filtre `<>` n'en trouve que **64** : les quinze clients sans téléphone ont disparu, car pour eux la condition n'est pas « vraie » mais « inconnue ». Il faut ajouter explicitement `OR telephone IS NULL`.

Les fonctions d'agrégation, elles, **ignorent** les `NULL` : `AVG(id_parrain)` ne moyennise que les clients parrainés (ce qui d'ailleurs n'a aucun sens, un numéro n'est pas une quantité). Pour remplacer un `NULL` par une valeur par défaut, on utilise **COALESCE**, qui renvoie son premier argument non vide :

```
SELECT id_client, prenom, COALESCE(telephone, '(non renseigné)') AS telephone
FROM clients
LIMIT 4;
```

id_client	prenom	telephone
1	Yassine	98725588
2	Sami (non renseigné)	
3	Aymen (non renseigné)	
4	Emna (non renseigné)	

Enfin, le piège **le plus dangereux** du chapitre : `NOT IN` avec une sous-requête qui contient un `NULL`. Cherchons les clients qui **n'ont parrainé personne**, c'est-à-dire dont le numéro n'apparaît jamais dans la colonne `id_parrain` :

```
SELECT (SELECT COUNT(*) FROM clients
 WHERE id_client NOT IN (SELECT id_parrain FROM clients)) AS naif,
 (SELECT COUNT(*) FROM clients
 WHERE id_client NOT IN (SELECT id_parrain FROM clients WHERE id_parrain IS NOT NULL)) AS
↪ corrige,
```

```
(SELECT COUNT(*) FROM clients AS c
WHERE NOT EXISTS (SELECT 1 FROM clients AS f WHERE f.id_parrain = c.id_client)) AS
↪ avec_not_exists;
```

naïf	corrige	avec_not_exists
0	55	55

La version naïve donne **zéro** (aucun client n'est « sans filleul » ? C'est absurde) ; les deux autres donnent 55. Pourquoi ? La liste (SELECT id\_parrain FROM clients) contient des NULL (tous les clients non parrainés). Or  $x \text{ NOT IN } (a, b, \text{NULL})$  se lit  $x \neq a \text{ AND } x \neq b \text{ AND } x \neq \text{NULL}$  ; le dernier terme est *inconnu*, donc le tout n'est jamais vrai, quelle que soit la valeur de  $x$ . Le filtre élimine toutes les lignes sans le moindre avertissement.

**Règle pratique.** Pour exprimer « n'est pas dans l'autre table », utilisez **NOT EXISTS** (insensible aux NULL) ou **LEFT JOIN ... IS NULL**, ou à défaut ajoutez **WHERE col IS NOT NULL** dans la sous-requête. Évitez **NOT IN** (sous-requête) sur une colonne qui peut contenir des NULL.

## Les dates

Comme dit en 5.1.4, SQLite stocke les dates en texte ISO 8601 ; on les manipule avec les fonctions `strftime`, `date` et `julianday`. Le premier réflexe de l'analyste : **regrouper par mois**.

```
SELECT strftime('%Y-%m', date_commande) AS mois,
COUNT(*) AS commandes,
ROUND(SUM(montant)) AS chiffre_affaires
FROM commandes
GROUP BY mois
ORDER BY mois;
```

mois	commandes	chiffre_affaires
2025-01	16	996.0
2025-02	20	1168.0
2025-03	25	1687.0
2025-04	36	1843.0
2025-05	36	2314.0
2025-06	33	2491.0
2025-07	42	2571.0
2025-08	44	2435.0
2025-09	31	1713.0
2025-10	20	1272.0
2025-11	40	2284.0
2025-12	57	3325.0

`strftime('%Y-%m', ...)` extrait « année-mois » (%Y année, %m mois, %d jour, %w jour de la semaine, 0 = dimanche). On voit la saisonnalité : un creux en janvier-février, une belle période estivale, un petit creux en octobre, et le pic des fêtes en **décembre** (57 commandes, 3 325 DT).

Autres opérations utiles : `date('2025-12-31', '-90 days')` (soustraire une durée), `julianday(d2) - julianday(d1)` (nombre de jours entre deux dates).

```
SELECT date('2025-12-31', '-90 days') AS il_y_a_90_jours,
date('2025-03-15', 'start of month') AS debut_du_mois,
CAST(julianday('2025-12-31') - julianday('2025-01-01') AS INTEGER) AS jours_ecoules;
```

il_y_a_90_jours	debut_du_mois	jours_ecoules
2025-10-02	2025-03-01	364

**N'utilisez jamais `date('now')` dans une analyse censée être reproductible.** 'now' change tous les jours : votre requête donnera un résultat différent demain, et nul ne pourra la rejouer. Dans ce livre, la « date du jour » est **fixée** au **31 décembre 2025** (date d'arrêt de la base). En entreprise, on passe la date d'arrêt en paramètre.

### 5.2.8 Mises en pratique

Réunissons tous ces outils pour répondre à trois vraies questions de Yasmine.

**Question 1 : « Qui sont mes dix meilleurs clients ? »** On joint clients et commandes, on regroupe par client, on trie par chiffre d'affaires décroissant.

```
SELECT cl.id_client,
 cl.prenom || ' ' || cl.nom AS client,
 cl.ville,
 COUNT(*) AS commandes,
 ROUND(SUM(c.montant), 2) AS chiffre_affaires,
 ROUND(AVG(c.montant), 2) AS panier_moyen,
 MAX(c.date_commande) AS derniere_commande
FROM clients AS cl
JOIN commandes AS c ON c.id_client = cl.id_client
GROUP BY cl.id_client
ORDER BY chiffre_affaires DESC
LIMIT 10;
```

id_client	client	ville	commandes	chiffre_affaires	panier_moyen	derniere_commande
2	Sami Dridi	Ariana	23	1874.3	81.49	2025-12-24
1	Yassine Lahmar	La Marsa	28	1701.7	60.77	2025-12-20
47	Sarra Hamdi	La Marsa	22	1317.3	59.88	2025-12-17
11	Bilel Bouazizi	Tunis	18	1218.9	67.72	2025-12-29
45	Emna Trabelsi	Ariana	15	1009.3	67.29	2025-12-25
17	Ines Dridi	Ariana	15	937.6	62.51	2025-12-22
39	Nour Chaabane	Sfax	10	833.1	83.31	2025-12-10
27	Salma Bouazizi	Sousse	13	808.9	62.22	2025-12-28
14	Fares Bouazizi	Sousse	5	566.0	113.20	2025-11-09
51	Dorra Mejri	Monastir	12	552.0	46.00	2025-12-10

On trouve un client très fidèle (Yassine Lahmar : 28 commandes) et un client qui dépense le plus (Sami Dridi : 1 874 DT, panier moyen de 81 DT). Notez que le dixième client (Dorra Mejri) a un chiffre d'affaires de 552 DT, **plus de trois fois moins** que le premier : la clientèle est très inégale. Remarquez aussi le `GROUP BY cl.id_client` et non `GROUP BY cl.nom` : regrouper par nom aurait fusionné les homonymes !

**Question 2 : « Quel est le chiffre d'affaires de chaque mois, canal par canal ? »** On veut un tableau avec un canal par colonne : c'est un **tableau croisé** (*pivot*). SQL n'a pas de commande universelle pour cela, mais l'astuce `SUM(CASE WHEN ... THEN ... ELSE 0 END)` fait très bien l'affaire. `CASE WHEN` est le « si... alors... sinon » de SQL.

```
SELECT strftime('%Y-%m', date_commande) AS mois,
 ROUND(SUM(CASE WHEN canal = 'Instagram' THEN montant ELSE 0 END)) AS instagram,
 ROUND(SUM(CASE WHEN canal = 'Site' THEN montant ELSE 0 END)) AS site,
 ROUND(SUM(CASE WHEN canal = 'Boutique' THEN montant ELSE 0 END)) AS boutique,
 ROUND(SUM(montant)) AS total
FROM commandes
GROUP BY mois
ORDER BY mois;
```

mois	instagram	site	boutique	total
2025-01	262.0	379.0	356.0	996.0
2025-02	382.0	203.0	582.0	1168.0

2025-03	737.0	824.0	126.0	1687.0
2025-04	367.0	919.0	557.0	1843.0
2025-05	693.0	679.0	942.0	2314.0
2025-06	816.0	723.0	951.0	2491.0
2025-07	647.0	992.0	932.0	2571.0
2025-08	601.0	1161.0	672.0	2435.0
2025-09	454.0	675.0	584.0	1713.0
2025-10	324.0	531.0	416.0	1272.0
2025-11	551.0	759.0	974.0	2284.0
2025-12	928.0	960.0	1437.0	3325.0

**Question 3 : « Les clients en retard de livraison sont-ils moins satisfaits ? »** CASE WHEN sert aussi à créer des **classes** à partir d'une variable numérique : ici, le délai de livraison.

```
SELECT CASE WHEN delai_livraison = 0 THEN '0 j (retrait)'
 WHEN delai_livraison <= 3 THEN '1 à 3 j'
 WHEN delai_livraison <= 6 THEN '4 à 6 j'
 ELSE '7 j et plus' END AS delai,
 COUNT(*) AS commandes,
 ROUND(AVG(satisfaction), 2) AS satisfaction_moyenne
FROM commandes
GROUP BY delai
ORDER BY MIN(delai_livraison);
```

delai	commandes	satisfaction_moyenne
0 j (retrait)	114	4.49
1 à 3 j	88	4.03
4 à 6 j	156	3.76
7 j et plus	42	3.17

La satisfaction **baisse régulièrement** avec le délai : de 4,49 pour un retrait en boutique à 3,17 au-delà d'une semaine. C'est exactement la relation que nous avons mise en évidence par la corrélation au chapitre 3, retrouvée ici avec de simples agrégations SQL. (Attention, comme toujours, à ne pas conclure trop vite à la causalité : le retrait en boutique diffère des commandes en ligne à bien d'autres égards.)

**Application : repérer les clients « dormants ».** Une cliente qui achetait régulièrement et qui ne revient plus est un signe d'alerte (on parle de *churn* quand elle part pour de bon). Définissons : un client **dormant** a passé **au moins 3 commandes** mais **aucune depuis plus de 90 jours** avant la date d'arrêt (le 31 décembre 2025). Le seuil se calcule : `date('2025-12-31', '-90 days')` donne le 2 octobre 2025.

```
SELECT cl.id_client,
 cl.prenom || ' ' || cl.nom AS client,
 COUNT(*) AS commandes,
 MAX(c.date_commande) AS derniere_commande,
 CAST(julianday('2025-12-31') - julianday(MAX(c.date_commande)) AS INTEGER) AS jours_sans_achat
FROM clients AS cl
JOIN commandes AS c ON c.id_client = cl.id_client
GROUP BY cl.id_client
HAVING COUNT(*) >= 3 AND MAX(c.date_commande) < '2025-10-02'
ORDER BY jours_sans_achat DESC;
```

id_client	client	commandes	derniere_commande	jours_sans_achat
20	Aymen Bouazizi	3	2025-05-07	238
62	Emna Sassi	3	2025-06-16	198
37	Dorra Lahmar	3	2025-07-21	163
16	Amel Ayari	5	2025-07-26	158
40	Sarra Sassi	3	2025-08-14	139
50	Lina Sassi	4	2025-08-17	136

8	Oussama Hamdi	6	2025-08-30	123
6	Hatem Ayari	6	2025-09-24	98
29	Walid Ayari	7	2025-09-26	96

Neuf clients répondent à ces critères : voilà la liste de relance de Yasmine. Le client le plus ancien (Aymen Bouazizi, dernière commande le 7 mai) n'est pas revenu depuis 238 jours. Notez l'emploi de HAVING avec **deux conditions sur le groupe** : le nombre de commandes **et** la date de la dernière.

#### À retenir

- Ordre d'écriture : SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT. Ordre d'exécution : FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- WHERE filtre les **lignes** (avant regroupement) ; HAVING filtre les **groupes** (après).
- Une **jointure** est un produit cartésien filtré par clé étrangère = clé primaire. LEFT JOIN conserve les lignes sans correspondance : c'est l'outil pour trouver « ceux qui n'ont jamais... ».
- NULL = « inconnu » : on le teste avec IS NULL, jamais avec =. COUNT(col) ignore les NULL. Méfiez-vous de NOT IN quand la liste peut contenir un NULL.
- Dates en ISO 8601, strftime pour regrouper, jamais de date( 'now' ) dans une analyse reproductible.
- Mettez des **parenthèses** dès que AND et OR se mélangent.

## 5.3 Fonctions fenêtres et CTE

**Intuition.** Avec GROUP BY (5.2.4), on **écrase** les lignes : cent commandes deviennent une seule ligne « total ». Mais souvent on veut garder chaque commande **et**, à côté, la comparer au reste : « cette commande est-elle au-dessus du panier moyen de son canal ? », « quel est son rang ? », « combien de jours depuis la commande précédente du même client ? ». Une **fonction fenêtre** calcule un résultat sur un *groupe de lignes voisines*, **sans réduire le nombre de lignes**. C'est l'outil qui distingue l'analyste débutant de l'analyste expérimenté.

### 5.3.1 Le principe : fonction(...) OVER (...)

Reprenons les commandes. On aimerait voir, pour chaque commande, **le panier moyen de son canal** et l'écart entre les deux. Avec GROUP BY, on obtiendrait 3 lignes (une par canal) et on perdrait les commandes. Avec une fenêtre :

```
SELECT id_commande, canal, montant,
 ROUND(AVG(montant) OVER (PARTITION BY canal), 2) AS moyenne_du_canal,
 ROUND(montant - AVG(montant) OVER (PARTITION BY canal), 2) AS ecart
FROM commandes
ORDER BY id_commande
LIMIT 6;
```

id_commande	canal	montant	moyenne_du_canal	ecart
1	Boutique	44.8	74.81	-30.01
2	Site	34.5	59.50	-25.00
3	Instagram	88.2	49.01	39.19
4	Instagram	30.1	49.01	-18.91
5	Boutique	110.1	74.81	35.29
6	Site	39.8	59.50	-19.70

Chaque commande est toujours là, et trois colonnes se sont ajoutées. Par exemple, la commande n° 1 (boutique, 44,80 DT) est 30,01 DT **en dessous** du panier moyen de la boutique (74,81 DT),

alors que la commande n° 3 (Instagram, 88,20 DT) est 39,19 DT **au-dessus** de celui d'Instagram (49,01 DT). Une commande de 88 DT est « grosse » sur Instagram mais « moyenne » en boutique : l'écart à son propre canal est plus parlant que le montant brut.

La syntaxe est toujours : **fonction(...) OVER ( PARTITION BY ... ORDER BY ... cadre )**.

Morceau	Rôle	Analogie
fonction(...)	le calcul (AVG, SUM, RANK, LAG...)	ce qu'on mesure
PARTITION BY	découpe en <b>groupes indépendants</b> (facultatif ; sans lui, une seule partition = toute la table)	« à l'intérieur de chaque canal »
ORDER BY	<b>ordonne</b> les lignes dans chaque partition	« du plus ancien au plus récent »
cadre (ROWS BETWEEN ...)	quelles lignes voisines sont prises en compte pour la ligne courante	« les 3 dernières lignes »

**Définition précise.** Pour **chaque ligne**  $r$ , la base détermine sa **partition**  $P(r)$  (les lignes qui ont la même valeur de PARTITION BY), les trie selon ORDER BY, puis retient un **cadre**  $F(r) \subseteq P(r)$  (par exemple « de la première ligne de la partition jusqu'à  $r$  »). Le résultat de la ligne  $r$  est  $f(F(r))$  : la fonction appliquée à ce sous-ensemble. GROUP BY est le cas dégénéré où toutes les lignes d'une partition reçoivent **le même** résultat, qu'on réduit alors à une ligne. Le nombre de lignes du résultat d'une fenêtre, lui, est **le même** qu'en entrée.

### 5.3.2 Les classements : ROW\_NUMBER, RANK, DENSE\_RANK

Trois fonctions numérotent les lignes d'une partition, selon l'ordre demandé. Elles diffèrent **seulement** par leur façon de traiter les **ex æquo**. Un exemple minimal, avec un classement de cinq clients selon leur note de satisfaction (deux notes à 5, deux à 3), construit directement avec VALUES :

```
WITH notes(client, note) AS (
 VALUES ('Amel', 5), ('Sami', 5), ('Ines', 4), ('Walid', 3), ('Rim', 3)
)
SELECT client, note,
 ROW_NUMBER() OVER (ORDER BY note DESC) AS row_number,
 RANK() OVER (ORDER BY note DESC) AS rank,
 DENSE_RANK() OVER (ORDER BY note DESC) AS dense_rank
FROM notes;
```

client	note	row_number	rank	dense_rank
Amel	5	1	1	1
Sami	5	2	1	1
Ines	4	3	3	2
Walid	3	4	4	3
Rim	3	5	4	3

Lisez-le colonne par colonne :

- ROW\_NUMBER : 1, 2, 3, 4, 5. Aucun ex æquo n'est reconnu : le départage entre Amel et Sami est **arbitraire** (si vous voulez un résultat reproductible, ajoutez un second critère d'ordre).
- RANK : 1, 1, 3, 4, 4. Les ex æquo partagent le même rang, et **le rang suivant saute** (comme aux Jeux olympiques : deux médailles d'or, pas d'argent, puis le bronze).
- DENSE\_RANK : 1, 1, 2, 3, 3. Les rangs sont **consécutifs**, sans trou.



(Au passage : `WITH notes(...) AS (...)` est une CTE, nous l'étudions au 5.3.5.) Voici un cas réel : le classement des **trois plus grosses commandes de chaque canal**. Le plus naturel serait d'écrire `WHERE ROW_NUMBER() OVER (...) <= 3`, mais c'est **interdit** : le `WHERE` s'exécute *avant* le calcul des fenêtres (voir l'ordre d'exécution au 5.2.1). On calcule donc le rang dans une requête intérieure, et on filtre dans la requête extérieure :

```
SELECT canal, rang, id_commande, date_commande, montant
FROM (
 SELECT canal, id_commande, date_commande, montant,
 ROW_NUMBER() OVER (PARTITION BY canal ORDER BY montant DESC) AS rang
 FROM commandes
)
WHERE rang <= 3
ORDER BY canal, rang;
```

canal	rang	id_commande	date_commande	montant
Boutique	1	243	2025-08-26	212.4
Boutique	2	362	2025-12-16	208.8
Boutique	3	115	2025-05-15	189.2
Instagram	1	140	2025-06-10	166.1
Instagram	2	59	2025-03-27	159.9
Instagram	3	125	2025-05-23	127.3
Site	1	157	2025-06-23	255.7
Site	2	61	2025-03-28	243.8
Site	3	208	2025-07-31	217.1

Ce motif (« **top N par groupe** ») est l'un des plus fréquents en entretien d'embauche comme en entreprise : « les 3 meilleurs vendeurs par région », « le dernier achat de chaque client » (`ROW_NUMBER() ... ORDER BY date DESC, puis rang = 1`).

### 5.3.3 Cumuls et moyennes mobiles : le cadre de la fenêtre

Ajoutons `ORDER BY` dans la fenêtre d'une somme : on obtient une **somme cumulée**. Voici l'évolution du chiffre d'affaires mensuel : le total mois par mois, le **cumul depuis janvier**, et une **moyenne mobile sur trois mois** (le mois courant et les deux précédents), qui lisse les fluctuations pour mieux voir la tendance.

```
WITH mensuel AS (
 SELECT strftime('%Y-%m', date_commande) AS mois,
 ROUND(SUM(montant)) AS ca
 FROM commandes
 GROUP BY mois
)
SELECT mois, ca,
 SUM(ca) OVER (ORDER BY mois) AS cumul,
 ROUND(AVG(ca) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS
 ↳ moyenne_mobile_3_mois
FROM mensuel
ORDER BY mois;
```

mois	ca	cumul	moyenne_mobile_3_mois
2025-01	996.0	996.0	996.0
2025-02	1168.0	2164.0	1082.0
2025-03	1687.0	3851.0	1284.0
2025-04	1843.0	5694.0	1566.0
2025-05	2314.0	8008.0	1948.0
2025-06	2491.0	10499.0	2216.0
2025-07	2571.0	13070.0	2459.0
2025-08	2435.0	15505.0	2499.0
2025-09	1713.0	17218.0	2240.0
2025-10	1272.0	18490.0	1807.0
2025-11	2284.0	20774.0	1756.0

2025-12 3325.0 24099.0

2294.0

La clause **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW** définit le cadre : « les deux lignes précédentes plus la ligne courante ». En janvier, il n'y a pas de ligne précédente : la « moyenne sur trois mois » n'utilise qu'une valeur (996), puis deux en février. Dans un rapport sérieux, on masquerait ces deux premières valeurs.

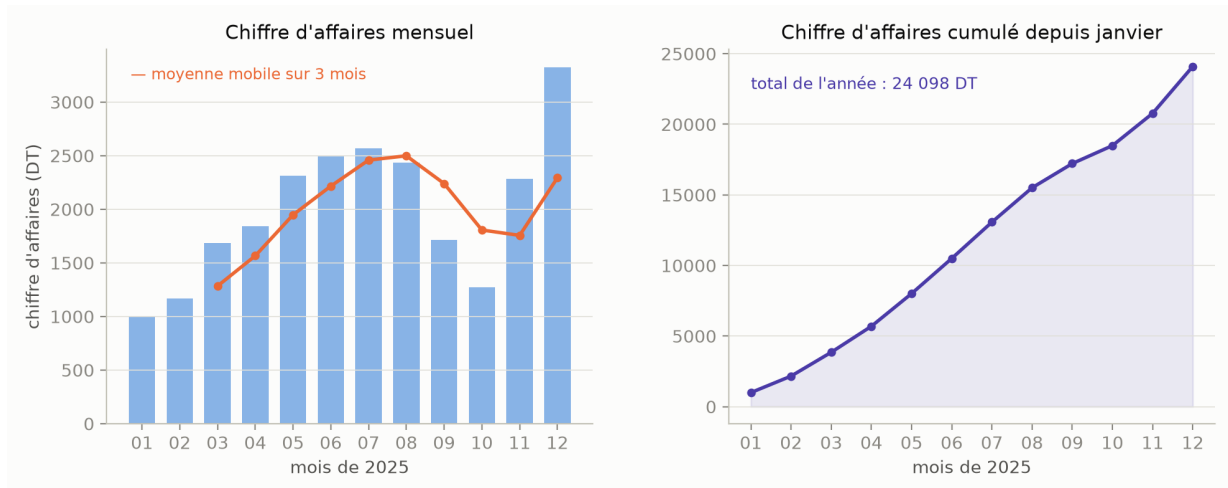


Figure 34: À gauche : chiffre d'affaires mensuel de Dar Jasmin en 2025 (barres) et sa moyenne mobile sur trois mois (courbe orange). À droite : chiffre d'affaires cumulé depuis janvier. Les données sont celles de la requête précédente.

Le graphique fait apparaître ce que les chiffres cachent : la moyenne mobile (courbe orange) gomme le creux d'octobre et la remontée de décembre, mais montre bien la **tendance** : une montée jusqu'à l'été, un repli à l'automne, puis un rebond de fin d'année. Le cumul atteint 24 099 DT en décembre (à l'arrondi près : chaque mois a été arrondi au dinar avant d'être additionné ; le vrai total est 24 098,30 DT, comme le montre le graphique de droite).

**Piège des ex æquo avec ORDER BY.** Quand on écrit `SUM(...)` `OVER (ORDER BY ...)` sans préciser de cadre, le cadre par défaut est `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, et `RANGE` regroupe toutes les lignes **ex æquo** sur la colonne de tri : elles reçoivent le **même** cumul. Illustration sur les six premières commandes, dont deux (n° 2 et n° 3) sont du même jour :

```
SELECT id_commande, date_commande, montant,
 SUM(montant) OVER (ORDER BY date_commande) AS cumul_range,
 SUM(montant) OVER (ORDER BY date_commande, id_commande ROWS UNBOUNDED PRECEDING) AS
 ↳ cumul_ligne_a_ligne
FROM commandes
WHERE id_commande <= 6
ORDER BY date_commande, id_commande;
```

id_commande	date_commande	montant	cumul_range	cumul_ligne_a_ligne
1	2025-01-03	44.8	44.8	44.8
2	2025-01-04	34.5	167.5	79.3
3	2025-01-04	88.2	167.5	167.5
4	2025-01-05	30.1	197.6	197.6
5	2025-01-08	110.1	307.7	307.7
6	2025-01-09	39.8	347.5	347.5

Les commandes 2 et 3 (même jour) affichent le **même** `cumul_range` : la somme des deux d'un

coup, ce qui n'est pas un « cumul ligne à ligne ». Pour un cumul ligne à ligne, précisez toujours **ROWS** et un ordre complet (qui départage les ex æquo, ici par `id_commande`). C'est une des rares occasions où une fenêtre sans cadre explicite donne un résultat inattendu.

### 5.3.4 Comparer avec la ligne précédente : LAG et LEAD

`LAG(colonne)` renvoie la valeur de la ligne **précédente** (dans l'ordre de la fenêtre) ; `LEAD(colonne)`, de la ligne **suivante**. C'est l'outil des évolutions : « par rapport au mois dernier », « depuis la dernière commande ». Commençons par la **croissance mensuelle du chiffre d'affaires**, en pourcentage :

```
WITH mensuel AS (
 SELECT strftime('%Y-%m', date_commande) AS mois,
 ROUND(SUM(montant)) AS ca
 FROM commandes
 GROUP BY mois
)
SELECT mois, ca,
 LAG(ca) OVER (ORDER BY mois) AS mois_precedent,
 ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mois)) / LAG(ca) OVER (ORDER BY mois), 1) AS
 ↪ evolution_pct
FROM mensuel
ORDER BY mois;
```

mois	ca	mois_precedent	evolution_pct
2025-01	996.0	NaN	NaN
2025-02	1168.0	996.0	17.3
2025-03	1687.0	1168.0	44.4
2025-04	1843.0	1687.0	9.2
2025-05	2314.0	1843.0	25.6
2025-06	2491.0	2314.0	7.6
2025-07	2571.0	2491.0	3.2
2025-08	2435.0	2571.0	-5.3
2025-09	1713.0	2435.0	-29.7
2025-10	1272.0	1713.0	-25.7
2025-11	2284.0	1272.0	79.6
2025-12	3325.0	2284.0	45.6

La première ligne n'a pas de précédent : `LAG` renvoie `NULL` (que pandas affiche `NaN`). Les plus fortes hausses sont en novembre (+79,6 %), en décembre (+45,6 %) et en mars (+44,4 %), la plus forte baisse en septembre (-29,7 %).

Une deuxième application, plus riche : **le délai entre deux commandes successives du même client**. Pour chaque commande, `LAG(date_commande)` dans la partition du client donne la date de sa commande précédente. Voici d'abord ce que cela donne pour la cliente n° 1 (les dates sont en texte ISO, `julianday` les convertit en nombres de jours) :

```
SELECT id_client, date_commande,
 LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
 ↪ commande_precedente,
 CAST(julianday(date_commande)
 - julianday(LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande,
 ↪ id_commande))
 AS INTEGER)
 AS jours_ecoules
FROM commandes
WHERE id_client = 1
ORDER BY date_commande, id_commande
LIMIT 6;
```

id_client	date_commande	commande_precedente	jours_ecoules
1	2025-01-05	NaN	NaN

1	2025-02-01	2025-01-05	27.0
1	2025-02-11	2025-02-01	10.0
1	2025-02-14	2025-02-11	3.0
1	2025-03-08	2025-02-14	22.0
1	2025-03-09	2025-03-08	1.0

Et pour l'ensemble de la clientèle : combien de jours séparent en moyenne deux achats successifs d'un même client ? Nous calculons d'abord les intervalles, puis nous les résumons :

```
WITH achats AS (
 SELECT id_client, date_commande,
 LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
 ↳ precedente
 FROM commandes
)
SELECT COUNT(*) AS intervalles,
 ROUND(AVG(julianday(date_commande) - julianday(precedente)), 1) AS delai_moyen_jours,
 MIN(julianday(date_commande) - julianday(precedente)) AS minimum,
 MAX(julianday(date_commande) - julianday(precedente)) AS maximum
FROM achats
WHERE precedente IS NOT NULL;
```

intervalles	delai_moyen_jours	minimum	maximum
334	40.2	0.0	320.0

334 intervalles : 400 commandes moins 66 « premières commandes » (une par client actif, qui n'ont pas de précédente). En moyenne, **40 jours** séparent deux achats d'un même client, avec des extrêmes allant de 0 (deux commandes le même jour) à 320 jours. C'est exactement l'ingrédient d'un indicateur classique du commerce : le **cycle de réachat**. Il guide, par exemple, la date idéale d'un courriel de relance : passé 40 jours sans achat, un client est déjà « en retard » par rapport à la normale.

### 5.3.5 Les CTE : donner un nom à une étape

Vous les avez déjà croisées dans ce chapitre. Une **CTE** (*Common Table Expression*) est une requête **nommée**, déclarée en tête avec `WITH nom AS ( ... )`, que l'on utilise ensuite comme si c'était une table. Elle ne change **rien** au résultat par rapport à une sous-requête imbriquée (5.2.6) : elle change la **lisibilité**. Comparez, pour la même question, l'imbrication :

```
SELECT ... FROM (SELECT ... FROM (SELECT ... FROM commandes GROUP BY ...) GROUP BY ...) WHERE ...
```

et le découpage en étapes nommées :

```
WITH etape1 AS (SELECT ... FROM commandes GROUP BY ...),
 etape2 AS (SELECT ... FROM etape1 ...)
SELECT ... FROM etape2 WHERE ...
```

La deuxième forme se lit **de haut en bas**, comme un script : on peut vérifier chaque étape isolément, ce qui est précieux pour le débogage. On peut enchaîner plusieurs CTE (séparées par des virgules) ; chacune peut utiliser celles qui précèdent.

**Application : segmenter la clientèle par quartiles de dépenses.** Les marketeurs aiment les segmentations de type **RFM** : **R**écence (depuis combien de temps le client n'a-t-il pas acheté ?), **F**réquence (combien de fois a-t-il acheté ?), **M**ontant (combien a-t-il dépensé ?). Construisons-la en deux étapes. La première CTE calcule les trois indicateurs par client ; la seconde utilise `NTILE(4)`, qui découpe les clients, triés par montant décroissant, en **4 groupes d'effectifs égaux** (quartiles) :

```

WITH rfm AS (
 SELECT id_client,
 CAST(julianday('2025-12-31') - julianday(MAX(date_commande)) AS INTEGER) AS recence_jours,
 COUNT(*) AS frequence,
 ROUND(SUM(montant), 2) AS montant
 FROM commandes
 GROUP BY id_client
),
segments AS (
 SELECT *, NTILE(4) OVER (ORDER BY montant DESC) AS quartile
 FROM rfm
)
SELECT quartile,
 COUNT(*) AS clients,
 ROUND(MIN(montant)) AS depense_min,
 ROUND(MAX(montant)) AS depense_max,
 ROUND(SUM(montant)) AS depense_totale,
 ROUND(AVG(frequence), 1) AS achats_moyens,
 ROUND(AVG(recence_jours)) AS jours_depuis_dernier_achat
FROM segments
GROUP BY quartile
ORDER BY quartile;

```

quartile	clients	depense_min	depense_max	depense_totale	achats_moyens	jours_depuis_dernier_achat
1	17	427.0	1874.0	14063.0	12.6	20.0
2	17	270.0	427.0	5678.0	5.8	49.0
3	16	122.0	270.0	3086.0	3.6	66.0
4	16	33.0	122.0	1270.0	1.8	153.0

Le quartile 1 (les 17 plus gros clients) dépense **14 063 DT sur 24 098**, soit **58 %** du chiffre d'affaires, et ils sont revenus il y a 20 jours en moyenne. Le quartile 4 (16 clients) ne pèse que 5 % et n'est pas revenu depuis 153 jours en moyenne. On retrouve la fameuse loi de **Pareto** (« 80-20 », ici plutôt « 25-58 ») : une minorité de clients fait une majorité du chiffre d'affaires. Cette information change la stratégie : chouchouter le quartile 1, relancer le quartile 3, ne pas s'acharner sur le quartile 4.

### 5.3.6 Les CTE récursives : des requêtes qui se rappellent elles-mêmes

Une CTE peut **se référencer elle-même** : c'est une CTE **récursive** (WITH RECURSIVE). Elle comporte toujours deux parties reliées par UNION ALL : un **cas de base** (le point de départ) et un **pas récursif** (comment passer de la ligne précédente à la suivante), avec une condition d'arrêt. Comme en Python, il faut un cas de base *et* une condition d'arrêt, faute de quoi la requête tourne sans fin.

L'exemple le plus simple : générer les carrés des entiers de 1 à 5.

```

WITH RECURSIVE n(i) AS (
 SELECT 1 -- cas de base : on part de 1
 UNION ALL
 SELECT i + 1 FROM n WHERE i < 5 -- pas : on ajoute 1, tant que i < 5
)
SELECT i, i * i AS carre FROM n;

```

i	carre
1	1
2	4
3	9
4	16
5	25

La base procède ainsi : elle écrit la ligne 1 ; pour chaque ligne nouvellement produite, elle applique le pas (1 donne 2, 2 donne 3...) jusqu'à ce que WHERE i < 5 bloque la production.

**Usage 1 : fabriquer un calendrier.** Une table de ventes ne contient que les jours où il y a eu des ventes : les jours **sans** vente n'apparaissent pas, ce qui fausse les moyennes quotidiennes. La parade classique : générer **tous les jours** de l'année avec une CTE récursive, puis les joindre aux commandes par un LEFT JOIN (5.2.5). Combien de jours Dar Jasmin n'a-t-il rien vendu, mois par mois ?

```
WITH RECURSIVE jours(d) AS (
 SELECT '2025-01-01'
 UNION ALL
 SELECT date(d, '+1 day') FROM jours WHERE d < '2025-12-31'
),
ventes_par_jour AS (
 SELECT j.d, COUNT(c.id_commande) AS commandes
 FROM jours AS j
 LEFT JOIN commandes AS c ON c.date_commande = j.d
 GROUP BY j.d
)
SELECT strftime('%Y-%m', d) AS mois,
 COUNT(*) AS jours,
 SUM(commandes = 0) AS jours_sans_commande,
 ROUND(AVG(commandes), 2) AS commandes_par_jour
FROM ventes_par_jour
GROUP BY mois
ORDER BY mois;
```

mois	jours	jours_sans_commande	commandes_par_jour
2025-01	31	19	0.52
2025-02	28	15	0.71
2025-03	31	15	0.81
2025-04	30	7	1.20
2025-05	31	9	1.16
2025-06	30	10	1.10
2025-07	31	7	1.35
2025-08	31	7	1.42
2025-09	30	8	1.03
2025-10	31	18	0.65
2025-11	30	7	1.33
2025-12	31	10	1.84

Cette requête enchaîne **une CTE récursive** (le calendrier) et **une CTE ordinaire** (le comptage). En janvier, par exemple, **19 jours sur 31** se sont passés sans la moindre commande, et ils n'apparaissent dans aucune table de ventes. Sans le calendrier, la « moyenne par jour » de décembre serait calculée seulement sur les jours où l'on a vendu (donc **surestimée**) ; avec lui, elle l'est sur les 31 jours. C'est un exemple concret de **biais de sélection** dans une requête : les jours sans vente sont absents du tableau, donc invisibles.

**Usage 2 : parcourir une hiérarchie.** La table clients contient le parrainage : un client peut avoir été parrainé par un autre, lui-même parrainé... C'est un **arbre**. À quelle profondeur ? Combien de filleuls (directs ou non) a chaque ambassadeur ? Une jointure ne peut parcourir qu'un nombre fixe de niveaux ; une CTE récursive les parcourt tous. Cas de base : les clients **sans parrain** (les « racines »). Pas : on ajoute leurs filleuls, puis les filleuls de leurs filleuls...

```
WITH RECURSIVE reseau(id_client, racine, profondeur) AS (
 SELECT id_client, id_client, 0
 FROM clients
 WHERE id_parrain IS NULL
 UNION ALL
 SELECT c.id_client, r.racine, r.profondeur + 1
 FROM clients AS c
 JOIN reseau AS r ON c.id_parrain = r.id_client
)
SELECT r.racine,
```

```

 cl.prenom || ' ' || cl.nom AS ambassadeur,
 COUNT(*) - 1 AS filleuls,
 MAX(r.profondeur) AS generations
FROM reseau AS r
JOIN clients AS cl ON cl.id_client = r.racine
GROUP BY r.racine
ORDER BY filleuls DESC
LIMIT 4;

```

racine	ambassadeur	filleuls	generations
10	Nour Hamdi	5	4
7	Zied Ben Salah	4	3
1	Yassine Lahmar	4	3
5	Emna Hamdi	3	3

Le premier réseau est celui du client n° 10 : **5 filleuls répartis sur 4 générations**. (COUNT(\*) - 1 : on ne compte pas la racine elle-même.) Voyons-le de plus près en affichant, pour chaque membre du réseau, sa profondeur et son chemin. Le texte chemin recolle les prénoms le long de la branche (cela permet aussi de trier dans l'ordre de parcours d'un arbre) :

```

WITH RECURSIVE arbre(id_client, profondeur, chemin) AS (
 SELECT id_client, 0, prenom || ' ' || nom
 FROM clients
 WHERE id_client = 10
 UNION ALL
 SELECT c.id_client, a.profondeur + 1, a.chemin || ' > ' || c.prenom || ' ' || c.nom
 FROM clients AS c
 JOIN arbre AS a ON c.id_parrain = a.id_client
)
SELECT id_client, profondeur, chemin
FROM arbre
ORDER BY chemin;

```

id_client	profondeur	chemin
10	0	Nour Hamdi
14	1	Nour Hamdi > Fares Bouazizi
23	2	Nour Hamdi > Fares Bouazizi > Hatem Mejri
28	3	Nour Hamdi > Fares Bouazizi > Hatem Mejri > Emna Ben Salah
55	4	Nour Hamdi > Fares Bouazizi > Hatem Mejri > Emna Ben Salah > Oussama Mejri
18	1	Nour Hamdi > Sami Hamdi

Le chemin de chaque client donne **toute sa chaîne de parrainage** depuis l'ambassadeur : Oussama Mejri (profondeur 4) est arrivé par Emna Ben Salah, venue par Hatem Mejri, venu par Fares Bouazizi, venu par Nour Hamdi. Remarquez que Nour Hamdi (client n° 10) **n'a jamais passé de commande** : elle figure dans la liste des quatorze clients du 5.2.5. Pourtant elle a amené cinq clients : une requête qui ne regarderait que les achats la jugerait sans valeur ! On peut ainsi calculer le chiffre d'affaires **généré** par un réseau, ce qui est la base d'une prime au parrainage.

**Récursion et sécurité.** Si le parrainage contenait un **cycle** (A parraine B qui parraine A), la récursion tournerait indéfiniment. Ici c'est impossible, puisqu'un parrain a toujours un numéro plus petit, donc une inscription plus ancienne, que son filleul (nous l'avons garanti à la construction de la base, 5.1.3), mais, sur des données réelles, ajoutez toujours une **condition d'arrêt** (profondeur maximale, par exemple WHERE profondeur < 10). C'est l'équivalent du while qui ne se termine jamais.

### À retenir

- Une fonction **fenêtre** (... OVER (PARTITION BY ... ORDER BY ...)) calcule sur un

groupe de lignes **sans réduire** leur nombre ; GROUP BY les réduit.

- **Classements** : ROW\_NUMBER (jamais d'ex æquo), RANK (ex æquo et trous), DENSE\_RANK (ex æquo sans trous). Le **top N par groupe** passe par une requête intérieure ou une CTE.
- **Cumuls, moyennes mobiles** : SUM/AVG(...) OVER (ORDER BY ... ROWS BETWEEN ...). Précisez ROWS pour éviter le piège des ex æquo.
- **LAG / LEAD** donnent la valeur de la ligne précédente / suivante : évolutions et délais entre événements.
- Une CTE (WITH nom AS (...)) nomme une étape et rend la requête lisible de haut en bas. Une CTE **récursive** (WITH RECURSIVE) génère des suites (calendriers) et parcourt des hiérarchies (parrainage, organigrammes).

## 5.4 Normalisation et conception de schémas

**Intuition.** Pourquoi avoir découpé les données de Dar Jasmin en cinq tables, au lieu d'un seul grand tableau, comme dans Excel ? Parce qu'un tableau unique **répète** les mêmes informations (la ville d'une cliente apparaît sur chacune de ses commandes), et que **tout ce qui est répété finit par se contredire**. La **normalisation** est la méthode qui consiste à ranger chaque fait **une seule fois**, à sa place. Cette section explique *pourquoi* le schéma du 5.1 est bon, et vous donne la méthode pour en concevoir un vous-même. Nous terminerons par deux sujets de **performance et de fiabilité** : les index et les transactions.

### 5.4.1 Le problème : la grande feuille unique

Imaginons que Yasmine ait gardé son habitude du tableur : **une seule feuille** avec une ligne par article vendu, contenant tout ce qu'on sait sur la commande, la cliente et le produit. Fabriquons cette feuille pour les **huit premières commandes** : nous créons dans la base des tables de travail préfixées ex\_ (nous les supprimerons à la fin de la section).

```
CREATE TABLE ex_feuille (
 id_commande INTEGER NOT NULL, id_produit INTEGER NOT NULL,
 date_commande TEXT, id_client INTEGER, prenom TEXT, nom TEXT, ville TEXT,
 nom_produit TEXT, id_categorie INTEGER, nom_categorie TEXT, prix_catalogue REAL,
 quantite INTEGER, prix_unitaire REAL,
 PRIMARY KEY (id_commande, id_produit)
);
INSERT INTO ex_feuille
SELECT c.id_commande, p.id_produit, c.date_commande, cl.id_client, cl.prenom, cl.nom, cl.ville,
 p.nom, ca.id_categorie, ca.nom, p.prix_catalogue, l.quantite, l.prix_unitaire
FROM commandes AS c
JOIN clients AS cl ON cl.id_client = c.id_client
JOIN lignes_commande AS l ON l.id_commande = c.id_commande
JOIN produits AS p ON p.id_produit = l.id_produit
JOIN categories AS ca ON ca.id_categorie = p.id_categorie
WHERE c.id_commande <= 8;
```

Voici un extrait de la feuille (quelques colonnes seulement pour tenir en largeur) :

```
SELECT id_commande, id_produit, id_client, prenom, ville, nom_produit, nom_categorie, quantite
FROM ex_feuille
ORDER BY id_commande, id_produit;
```

id_commande	id_produit	id_client	prenom	ville	nom_produit	nom_categorie	quantite
1	1	2	Sami	Ariana	Tajine décoratif	Poterie	1
2	10	6	Hatem	La Marsa	Pendentif khamsa	Bijoux	1



3	2	3	Aymen	Sfax	Bol en céramique de Nabeul	Poterie	1
3	3	3	Aymen	Sfax	Vase peint à la main	Poterie	1
3	13	3	Aymen	Sfax	Savon à l'huile d'olive	Cosmétiques	1
4	11	1	Yassine	La Marsa	Bracelet de perles	Bijoux	1
4	14	1	Yassine	La Marsa	Eau de jasmin	Cosmétiques	1
5	8	5	Emna	Ariana	Pochette brodée	Textile	2
5	9	5	Emna	Ariana	Bague en argent	Bijoux	1
5	15	5	Emna	Ariana	Huile de nigelle	Cosmétiques	1
6	2	9	Salma	Tunis	Bol en céramique de Nabeul	Poterie	1
6	8	9	Salma	Tunis	Pochette brodée	Textile	1
7	5	4	Emna	Ariana	Foutah en coton	Textile	1
7	9	4	Emna	Ariana	Bague en argent	Bijoux	1
8	4	5	Emna	Ariana	Plat à couscous	Poterie	2
8	5	5	Emna	Ariana	Foutah en coton	Textile	1

Regardez la cliente n° 5, Emna Hamdi : elle apparaît sur **cinq lignes**, et sa ville « Ariana » est écrite cinq fois. Même chose pour le produit n° 9 (Bague en argent) et sa catégorie « Bijoux ». Cette redondance cause trois catégories de problèmes, appelées **anomalies**.

**1. Anomalie de mise à jour.** Emna déménage à Sousse. L'employé de Yasmine ne corrige qu'une ligne sur cinq (il a oublié les autres) :

```
UPDATE ex_feuille SET ville = 'Sousse'
WHERE id_client = 5 AND id_commande = 8 AND id_produit = 5;
```

```
SELECT id_client, prenom, nom, ville, COUNT(*) AS lignes
FROM ex_feuille
WHERE id_client = 5
GROUP BY id_client, prenom, nom, ville;
```

id_client	prenom	nom	ville	lignes
5	Emna	Hamdi	Ariana	4
5	Emna	Hamdi	Sousse	1

Deux villes pour la même personne : on ne sait plus laquelle est vraie. C'est exactement l'histoire de « Amel Ben Salah » du 5.1.1. Dans notre vraie base, cette erreur est **impossible** : la ville d'une cliente n'est écrite qu'à un seul endroit.

**Une incohérence ne se résorbe pas toute seule.** Même un `SELECT DISTINCT id_client, ville` ne sait pas « choisir » la bonne ville : il renvoie les deux. Les doublons contradictoires sont de la **vraie** information fausse, que seul un humain peut trancher. Remettons la ville d'origine pour la suite :

```
UPDATE ex_feuille SET ville = 'Ariana'
WHERE id_client = 5 AND id_commande = 8 AND id_produit = 5;
```

**2. Anomalie d'insertion.** Yasmine veut ajouter au catalogue un nouveau produit, pas encore vendu. Impossible : la feuille n'a de place que pour des *lignes de commande*, et la clé primaire exige un numéro de commande.

```
try:
 con.execute("INSERT INTO ex_feuille (id_produit, nom_produit, prix_catalogue) VALUES (17,
↳ 'Narguilé', 90)")
except sqlite3.IntegrityError as erreur:
 print("refusé :", erreur)
```

```
refusé : NOT NULL constraint failed: ex_feuille.id_commande
```

(Et en supprimant la contrainte, on aurait une ligne avec des trous partout.)

**3. Anomalie de suppression.** Quels produits n'apparaissent que sur **une seule** ligne de la feuille ? Pour ceux-là, supprimer cette ligne (par exemple parce que la commande est annulée) effacerait **toute trace du produit** : son nom, sa catégorie, son prix catalogue.

```
SELECT id_produit, nom_produit, prix_catalogue, MIN(id_commande) AS seule_commande
FROM ex_feuille
GROUP BY id_produit
HAVING COUNT(*) = 1
ORDER BY id_produit;
```

id_produit	nom_produit	prix_catalogue	seule_commande
1	Tajine décoratif	45.0	1
3	Vase peint à la main	65.0	3
4	Plat à couscous	38.0	8
10	Pendentif khamsa	35.0	2
11	Bracelet de perles	15.0	4
13	Savon à l'huile d'olive	6.0	3
14	Eau de jasmin	14.0	4
15	Huile de nigelle	24.0	5

Annuler la commande n° 5 ferait disparaître l'huile de nigelle du catalogue : on a voulu supprimer *une vente* et on a perdu *un produit*.

Résumé : dans une table, **tout fait doit être associé à un seul sujet**. Un client, un produit, une commande et une vente sont quatre sujets différents ; les mélanger dans une même table provoque ces trois anomalies.

#### 5.4.2 Dépendances fonctionnelles : la théorie derrière la normalisation

**Définition.** Dans une table  $R$  d'attributs  $A$ , on dit que  $X$  **détermine fonctionnellement**  $Y$ , noté  $X \rightarrow Y$  (avec  $X, Y \subseteq A$ ), si deux lignes qui ont la **même valeur de  $X$**  ont **forcément la même valeur de  $Y$**  :

$$\forall t_1, t_2 \in R, \quad t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y].$$

Exemples dans notre feuille :  $\text{id\_client} \rightarrow \text{ville}$  (un client n'a qu'une ville) ;  $\text{id\_produit} \rightarrow \text{prix\_catalogue}$  ; mais **pas**  $\text{ville} \rightarrow \text{id\_client}$  (plusieurs clientes habitent Ariana).

Une clé primaire est un cas particulier :  $K$  est une **clé** de  $R$  si  $K \rightarrow A$  (elle détermine *toutes* les colonnes) et si aucun sous-ensemble strict de  $K$  n'en fait autant (minimalité).

Les dépendances fonctionnelles obéissent à trois règles, les **axiomes d'Armstrong** (1974), qui permettent d'en déduire d'autres :

1. **Réflexivité** : si  $Y \subseteq X$ , alors  $X \rightarrow Y$  (trivial).
2. **Augmentation** : si  $X \rightarrow Y$ , alors  $XZ \rightarrow YZ$  pour tout  $Z$ .
3. **Transitivité** : si  $X \rightarrow Y$  et  $Y \rightarrow Z$ , alors  $X \rightarrow Z$ .

La **fermeture**  $X^+$  d'un ensemble d'attributs est l'ensemble de tout ce qu'il détermine, directement ou par transitivité. On la calcule en partant de  $X$  et en ajoutant les attributs déterminés tant que c'est possible. Appliquons-le à notre feuille. Les dépendances que nous croyons vraies (règles de gestion de Dar Jasmin) sont :

```
dependances = [
 (["id_commande"], ["date_commande", "id_client"]),
 (["id_client"], ["prenom", "nom", "ville"]),
 (["id_produit"], ["nom_produit", "id_categorie", "prix_catalogue"]),
 (["id_categorie"], ["nom_categorie"]),
 (["id_commande", "id_produit"], ["quantite", "prix_unitaire"]),
```

```

]

def fermeture(X, deps):
 """Ensemble des attributs déterminés par X (algorithme de point fixe)."""
 res, change = set(X), True
 while change:
 change = False
 for gauche, droite in deps:
 if set(gauche) <= res and not set(droite) <= res:
 res |= set(droite)
 change = True
 return res

```

Avant de s'en servir, vérifions que ces dépendances sont **respectées par les données** de la feuille. On charge celle-ci dans pandas, et l'on teste, pour chaque dépendance  $X \rightarrow Y$ , qu'aucun groupe de lignes de même  $X$  ne contient deux valeurs différentes de  $Y$  :

```

feuille = pd.read_sql_query("SELECT * FROM ex_feuille", con)

def verifie(df, X, Y):
 """Vrai si X -> Y n'est contredit par aucune paire de lignes."""
 return bool((df.groupby(X)[Y].nunique() <= 1).all().all())

for gauche, droite in dependances:
 print(f'{"' + '.join(gauche):25s}' -> {'', '.join(droite):45s' {verifie(feuille, gauche, droite)}}")

print()
print("ville -> id_client ? ", verifie(feuille, ["ville"], ["id_client"]))
print("id_commande -> prix_unitaire ? ", verifie(feuille, ["id_commande"], ["prix_unitaire"]))

```

id_commande	-> date_commande, id_client	True
id_client	-> prenom, nom, ville	True
id_produit	-> nom_produit, id_categorie, prix_catalogue	True
id_categorie	-> nom_categorie	True
id_commande + id_produit	-> quantite, prix_unitaire	True
ville -> id_client ?		False
id_commande -> prix_unitaire ?		False

Les cinq dépendances sont respectées ; les deux « fausses » dépendances ( $\text{ville} \rightarrow \text{id\_client}$ ,  $\text{id\_commande} \rightarrow \text{prix\_unitaire}$ ) sont **contredites** par les données, comme prévu. Attention à la nuance logique : un jeu de données peut seulement **réfuter** une dépendance (une paire de lignes suffit), jamais la **démontrer** ; seule la connaissance du métier (« un client n'a qu'une ville de livraison par défaut ») l'affirme.

Calculons maintenant des fermetures et cherchons la clé de la feuille :

```

from itertools import combinations

attributs = set(feuille.columns)
print("fermeture de {id_commande} :", sorted(fermeture(["id_commande"], dependances)))
print("fermeture de {id_client} :", sorted(fermeture(["id_client"], dependances)))
print()

clés candidates : sous-ensembles minimaux dont la fermeture est tous les attributs
cles = []
for k in range(1, 4):
 for X in combinations(sorted(attributs), k):
 if fermeture(X, dependances) == attributs and not any(set(c) <= set(X) for c in cles):
 cles.append(X)
print("clé(s) candidate(s) :", cles)

```

```

fermeture de {id_commande} : ['date_commande', 'id_client', 'id_commande', 'nom', 'prenom',
↳ 'ville']
fermeture de {id_client} : ['id_client', 'nom', 'prenom', 'ville']

clé(s) candidate(s) : [('id_commande', 'id_produit')]

```

La seule clé est le couple (id\_commande, id\_produit) : c'est la clé primaire que nous avons déclarée. Le calcul explique aussi l'origine des anomalies : beaucoup d'attributs dépendent seulement d'**une partie** de la clé (prenom, ville... dépendent de id\_commande seul ; nom\_produit, prix\_catalogue... de id\_produit seul), ou d'un attribut qui n'est pas la clé (ville dépend de id\_client, qui dépend de id\_commande). On a trouvé la source du mal. Il reste à la soigner.

### 5.4.3 Les trois premières formes normales

Les **formes normales** sont des niveaux de « propreté » d'un schéma, chaque niveau supprimant un type de redondance. Retenez la formule qui résume les trois premières, dans l'esprit du serment d'un témoin au tribunal : *chaque attribut dépend de la clé, de toute la clé, et rien que de la clé* (« so help me Codd »).

Forme	Exigence	Anomalie évitée
1FN	chaque case contient <b>une seule valeur</b> (atomique) ; pas de listes ni de colonnes répétées	cases du type « Tajine ; Foutah »
2FN	1FN <b>et</b> aucun attribut ne dépend d'une <b>partie</b> de la clé (utile quand la clé est composite)	informations sur le produit répétées sur chaque vente
3FN	2FN <b>et</b> aucun attribut non-clé ne dépend d'un <b>autre attribut non-clé</b> (pas de dépendance transitive)	ville du client recopiée parce que id_client détermine ville

**1FN.** Si Yasmine avait noté le panier dans une seule case (articles = "Foutah en coton ; Pochette brodée"), elle n'aurait pu ni compter les ventes par produit, ni les joindre au catalogue : on ne sait pas joindre un morceau de texte. La solution est **une ligne par article** : c'est ce que fait notre feuille, qui est donc déjà en 1FN (et nous aurions eu le même problème avec des colonnes produit1, produit2, produit3).

**2FN.** La clé de la feuille est composite (id\_commande, id\_produit). Or les infos du produit ne dépendent que de id\_produit, et celles de la commande que de id\_commande : ce sont des **dépendances partielles**. On découpe : chaque groupe d'attributs va dans une table dont la clé est l'attribut qui le détermine.

```

CREATE TABLE ex_lignes AS
SELECT id_commande, id_produit, quantite, prix_unitaire FROM ex_feuille;

CREATE TABLE ex_commandes AS
SELECT DISTINCT id_commande, date_commande, id_client, prenom, nom, ville FROM ex_feuille;

CREATE TABLE ex_produits AS
SELECT DISTINCT id_produit, nom_produit, id_categorie, nom_categorie, prix_catalogue FROM ex_feuille;

```

(DISTINCT supprime les doublons : chaque commande et chaque produit n'est plus écrit qu'une fois.) La redondance a déjà fortement baissé, mais elle n'a pas disparu : dans ex\_commandes, la ville d'Emna est encore écrite pour **chacune** de ses commandes (n° 5 et n° 8). Cause : id\_commande → id\_client → ville : c'est une **dépendance transitive**.

**3FN.** On extrait ce qui dépend d'un attribut non-clé dans sa propre table : les clients (clé id\_client) d'un côté, les catégories (clé id\_categorie) de l'autre.

```
CREATE TABLE ex_clients AS
SELECT DISTINCT id_client, prenom, nom, ville FROM ex_commandes;

CREATE TABLE ex_categories AS
SELECT DISTINCT id_categorie, nom_categorie FROM ex_produits;

CREATE TABLE ex_commandes3 AS
SELECT DISTINCT id_commande, date_commande, id_client FROM ex_commandes;

CREATE TABLE ex_produits3 AS
SELECT DISTINCT id_produit, nom_produit, id_categorie, prix_catalogue FROM ex_produits;
```

Comptons les lignes de chaque table pour voir la différence :

```
SELECT 'feuille unique' AS table_, COUNT(*) AS lignes FROM ex_feuille
UNION ALL SELECT 'lignes', COUNT(*) FROM ex_lignes
UNION ALL SELECT 'commandes', COUNT(*) FROM ex_commandes3
UNION ALL SELECT 'clients', COUNT(*) FROM ex_clients
UNION ALL SELECT 'produits', COUNT(*) FROM ex_produits3
UNION ALL SELECT 'categories', COUNT(*) FROM ex_categories;
```

table_	lignes
feuille unique	16
lignes	16
commandes	8
clients	7
produits	12
categories	4

La grande feuille (16 lignes) est devenue cinq petites tables : exactement la structure du 5.1 ! La normalisation est ce qui a conduit à notre schéma, et un dessin préalable des entités (5.1.2) aurait donné le même résultat plus vite. Les anomalies ont disparu : changer la ville d'Emna se fait par **un seul** UPDATE sur **une seule** ligne ; ajouter un produit au catalogue est un simple INSERT dans produits ; supprimer une vente laisse produit et client intacts.

**Le test de sécurité : « décomposer sans rien perdre ».** Découper une table en plusieurs ne doit pas **perdre d'information**. Vérifions que, si l'on recolle les morceaux par des jointures, on retrouve **exactement** la feuille d'origine, ni plus ni moins. On compare dans les deux sens avec EXCEPT (5.2.5) : les lignes reconstituées absentes de la feuille, puis l'inverse.

```
CREATE TEMP VIEW ex_recollee AS
SELECT l.id_commande, l.id_produit, c.date_commande, c.id_client, cl.prenom, cl.nom, cl.ville,
 p.nom_produit, p.id_categorie, ca.nom_categorie, p.prix_catalogue, l.quantite, l.prix_unitaire
FROM ex_lignes AS l
JOIN ex_commandes3 AS c ON c.id_commande = l.id_commande
JOIN ex_clients AS cl ON cl.id_client = c.id_client
JOIN ex_produits3 AS p ON p.id_produit = l.id_produit
JOIN ex_categories AS ca ON ca.id_categorie = p.id_categorie;
```

```
SELECT (SELECT COUNT(*) FROM (SELECT * FROM ex_recollee EXCEPT SELECT * FROM ex_feuille)) AS en_trop,
 (SELECT COUNT(*) FROM (SELECT * FROM ex_feuille EXCEPT SELECT * FROM ex_recollee)) AS
↪ manquantes;
```

en_trop	manquantes
0	0

Zéro et zéro : la décomposition est **sans perte**. (Et grâce à la remarque du 5.4.1 sur la ville d'Emna, nous avons pris soin de remettre la feuille en état avant de la découper : normaliser des don-

nées **déjà contradictoires** aurait recopié fidèlement la contradiction dans la table `ex_clients`, avec deux lignes pour la même cliente. Un schéma normalisé rend les *nouvelles* incohérences impossibles, par exemple en déclarant `id_client` clé primaire de `clients`, mais ne répare pas les anciennes.)

**Pourquoi la décomposition sans perte fonctionne (théorème de Heath).** Soit  $R(X, Y, Z)$  une table avec la dépendance  $X \rightarrow Y$ . Alors  $R = \pi_{X,Y}(R) \bowtie \pi_{X,Z}(R)$ . *Preuve.* L'inclusion  $R \subseteq \pi_{X,Y}(R) \bowtie \pi_{X,Z}(R)$  est évidente (toute ligne se retrouve en recollant ses propres morceaux). Réciproquement, soit  $(x, y, z)$  dans la jointure :  $(x, y)$  vient d'une ligne  $(x, y, z') \in R$  et  $(x, z)$  d'une ligne  $(x, y', z) \in R$ . Ces deux lignes ont la **même valeur**  $x$ , donc, comme  $X \rightarrow Y$ , la **même valeur**  $y = y'$ . La ligne  $(x, y', z) = (x, y, z)$  est donc dans  $R$ .  $\square$  Sans la dépendance, la jointure pourrait fabriquer de **fausses lignes** : c'est le piège de la décomposition faite « au feeling ».

Pour finir proprement, supprimons nos tables de travail (il faut le faire pour que la base rede-vienne celle du 5.1) :

```
con.execute("DROP VIEW IF EXISTS ex_recollee")
for t in ["ex_lignes", "ex_commandes3", "ex_clients", "ex_produits3", "ex_categories",
 "ex_commandes", "ex_produits", "ex_feuille"]:
 con.execute(f"DROP TABLE IF EXISTS {t}")
con.commit()
print([r[0] for r in con.execute("SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY
 ↪ name")])
```

```
['categories', 'clients', 'commandes', 'lignes_commande', 'produits']
```

#### 5.4.4 Aller plus loin, ou s'arrêter plus tôt ?

**La forme normale de Boyce-Codd (BCNF)** renforce la 3FN : pour *toute* dépendance  $X \rightarrow Y$  non triviale,  $X$  doit être une clé. Dans la pratique, la 3FN suffit presque toujours, et un schéma en 3FN est souvent déjà en BCNF. Il existe des formes supérieures (4FN, 5FN) pour des cas plus rares.

**Mais attention :** « **plus normalisé** » n'est pas toujours « **mieux** ». Un schéma normalisé est idéal pour **enregistrer** des données (les systèmes *transactionnels*, dits **OLTP** : caisse, site de vente, réservations) : peu de redondance, mises à jour sûres. Pour **analyser** (les systèmes **OLAP** : entrepôts de données, tableaux de bord), l'analyste doit en revanche joindre cinq tables pour la moindre question, ce qui est lent et fastidieux. On **dénormalise** donc volontairement dans les entrepôts, par exemple avec un **schéma en étoile** : au centre, une grosse table de **faits** (les ventes : une ligne par article vendu, avec des montants et des clés), autour d'elle de petites tables de **dimensions** (client, produit, date) qui décrivent le contexte. Notre `lignes_commande` entourée de `commandes`, `produits`, `clients` ressemble déjà à une étoile.

Règle pratique : **normalisez pour écrire, dénormalisez pour lire**. En attendant de construire un entrepôt, une **vue** donne à l'analyste une « grande table » toute prête, sans dupliquer les données : une vue est une **requête enregistrée sous un nom**, qu'on interroge comme une table.

```
CREATE VIEW v_ventes AS
SELECT l.id_commande, c.date_commande, c.canal, c.satisfaction,
 cl.id_client, cl.ville,
 ca.nom AS categorie, p.nom AS produit,
 l.quantite, l.prix_unitaire, l.quantite * l.prix_unitaire AS montant_ligne
FROM lignes_commande AS l
JOIN commandes AS c ON c.id_commande = l.id_commande
JOIN clients AS cl ON cl.id_client = c.id_client
JOIN produits AS p ON p.id_produit = l.id_produit
JOIN categories AS ca ON ca.id_categorie = p.id_categorie;
```

```
SELECT categorie, canal, ROUND(SUM(montant_ligne)) AS chiffre_affaires
FROM v_ventes
GROUP BY categorie, canal
ORDER BY categorie, canal;
```

categorie	canal	chiffre_affaires
Bijoux	Boutique	2256.0
Bijoux	Instagram	1914.0
Bijoux	Site	2836.0
Cosmétiques	Boutique	878.0
Cosmétiques	Instagram	1099.0
Cosmétiques	Site	1170.0
Poterie	Boutique	2397.0
Poterie	Instagram	2143.0
Poterie	Site	2410.0
Textile	Boutique	2997.0
Textile	Instagram	1607.0
Textile	Site	2390.0

Une requête qui exigeait quatre jointures tient maintenant en trois lignes, et la vue peut évoluer (changer de définition) sans que les analystes changent leurs requêtes.

### 5.4.5 Les index : retrouver une ligne sans tout lire

Quand on écrit `WHERE id_client = 2`, comment la base trouve-t-elle les commandes du client ? Sans aide, elle doit **lire toutes les lignes** de la table, une par une (un *balayage complet*, ou *full scan*). Avec 400 lignes, c'est instantané ; avec 100 millions, c'est insupportable. Un **index** est une structure annexe, triée, comparable à l'**index alphabétique à la fin d'un livre** : au lieu de feuilleter tout l'ouvrage pour trouver « Khi-deux », on consulte l'index qui renvoie à la bonne page. Techniquement c'est le plus souvent un **arbre B** (*B-tree*) : on trouve une valeur parmi  $N$  en environ  $\log_2 N$  comparaisons au lieu de  $N$ .

Pour  $N = 500\,000$ , c'est  $\log_2 N \approx 19$  comparaisons contre 500 000 : un facteur **plus de 25 000**. On peut demander à la base **comment elle compte exécuter** une requête, avec `EXPLAIN QUERY PLAN` (c'est le premier outil de l'analyste qui s'occupe de performance) :

```
def plan(requete):
 for ligne in con.execute("EXPLAIN QUERY PLAN " + requete):
 print(" ", ligne[3])

print("Avant l'index :")
plan("SELECT * FROM commandes WHERE id_client = 2")
con.execute("CREATE INDEX idx_commandes_client ON commandes(id_client)")
print("Après CREATE INDEX :")
plan("SELECT * FROM commandes WHERE id_client = 2")
print("Recherche par la clé primaire (index automatique) :")
plan("SELECT * FROM commandes WHERE id_commande = 2")
print("Filtre sur une colonne non indexée :")
plan("SELECT * FROM commandes WHERE montant > 100")
```

```
Avant l'index :
 SCAN commandes
Après CREATE INDEX :
 SEARCH commandes USING INDEX idx_commandes_client (id_client=?)
Recherche par la clé primaire (index automatique) :
 SEARCH commandes USING INTEGER PRIMARY KEY (rowid=?)
Filtre sur une colonne non indexée :
 SCAN commandes
```

SCAN signifie « lire toute la table » ; `SEARCH ... USING INDEX` signifie « chercher via l'arbre ». La clé primaire est **toujours** indexée automatiquement ; c'est aussi pourquoi les clés étrangères mal



indexées sont une cause classique de lenteur. Le filtre sur montant reste un SCAN : aucun index ne l'aide.

Pour mesurer le gain « pour de vrai », construisons une table de **500 000 lignes** (par une CTE récursive, 5.3.6) et chronométrons la même recherche avant et après un index. Les durées exactes dépendent de votre machine ; nous n'affichons donc que le **facteur de gain**, arrondi à la puissance de dix inférieure, pour que le résultat reste le même d'une exécution à l'autre.

```
import time, math

con.executescript("""
CREATE TABLE ex_gros (id INTEGER PRIMARY KEY, id_client INTEGER, montant REAL);
WITH RECURSIVE s(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM s WHERE i < 500000)
INSERT INTO ex_gros SELECT i, (i * 7919) % 50000, ROUND(10 + (i * 31) % 190, 2) FROM s;
""")

def duree(repetitions=20):
 debut = time.perf_counter()
 for k in range(repetitions):
 con.execute("SELECT COUNT(*), SUM(montant) FROM ex_gros WHERE id_client = ?", (123 +
↵ k,)).fetchone()
 return (time.perf_counter() - debut) / repetitions

sans_index = duree()
con.execute("CREATE INDEX idx_gros_client ON ex_gros(id_client)")
avec_index = duree()

gain = sans_index / avec_index
print("lignes dans la table :", con.execute("SELECT COUNT(*) FROM ex_gros").fetchone()[0])
print("gain au moins égal à :", 10 ** int(math.log10(gain)), "fois")
con.execute("DROP TABLE ex_gros")

lignes dans la table : 500000
gain au moins égal à : 100 fois
```

Chez nous, le gain réel est de l'ordre de plusieurs centaines de fois (le programme n'affiche que la puissance de dix inférieure, pour rester reproductible) : voilà pourquoi les index sont **la première optimisation**. Mais ils ne sont pas gratuits : un index occupe de la place, et il doit être **mis à jour à chaque insertion ou modification**, ce qui ralentit l'écriture. Règle d'usage : indexer les colonnes **souvent utilisées dans un WHERE ou un JOIN** (surtout les clés étrangères) et ne pas indexer à tout va. Notez enfin que les index peuvent changer le **temps** d'une requête, **jamais son résultat**.

#### 5.4.6 Les transactions : tout ou rien

Enregistrer une commande, c'est plusieurs écritures : une ligne dans commandes, puis une ligne par article dans lignes\_commande (et, dans un vrai système, la mise à jour du stock). Que se passe-t-il si le courant saute **entre** deux de ces écritures ? On aurait une commande **sans** articles : une base incohérente. Une **transaction** regroupe plusieurs opérations en un bloc **indivisible** : soit toutes réussissent (COMMIT), soit aucune n'a d'effet (ROLLBACK). On résume les garanties d'une transaction par l'acronyme **ACID** :

Lettre	Garantie	Signification
Atomicité	tout ou rien	si une étape échoue, tout est annulé
Cohérence	les règles tiennent toujours	les contraintes (clés, CHECK) sont respectées avant et après



Lettre	Garantie	Signification
Isolation	pas d'interférence	deux transactions simultanées ne voient pas les états intermédiaires l'une de l'autre
Durabilité	c'est définitif	une fois validée, la modification survit à une panne

Voyons l'atomicité en action. On tente d'enregistrer une commande dont la **deuxième** étape échoue (un produit n° 999 qui n'existe pas) :

```
def nb_commandes():
 return con.execute("SELECT COUNT(*) FROM commandes").fetchone()[0]

print("avant :", nb_commandes(), "commandes")
try:
 with con:
 # ouvre une transaction ; ROLLBACK automatique si
 ↪ exception
 con.execute("INSERT INTO commandes VALUES (9001, 1, '2025-12-31', 'Site', 80, 3, 5)")
 print("pendant la transaction :", nb_commandes(), "commandes (la nouvelle est visible pour
 ↪ nous)")
 con.execute("INSERT INTO lignes_commande VALUES (9001, 999, 1, 80)") # produit inexistant :
 ↪ échec
except sqlite3.IntegrityError as erreur:
 print("échec :", erreur)
print("après :", nb_commandes(), "commandes")
```

```
avant : 400 commandes
pendant la transaction : 401 commandes (la nouvelle est visible pour nous)
échec : FOREIGN KEY constraint failed
après : 400 commandes
```

La commande n° 9001, pourtant bien insérée à l'étape 1, a **disparu** : l'échec de l'étape 2 a annulé toute la transaction. Sans transaction, on aurait gardé une commande fantôme sans article. Dans le bloc `with con:`, Python déclenche un `COMMIT` si tout se passe bien, un `ROLLBACK` à la première exception.

**Pourquoi l'isolation compte.** Deux caissiers enregistrent en même temps la vente du dernier exemplaire d'un tajine. Sans isolation, chacun lit « stock = 1 », chacun vend, et le stock tombe à -1. Avec une transaction isolée, la seconde attend (ou échoue) et lit « stock = 0 ». Les SGBD offrent plusieurs **niveaux d'isolation**, du plus laxiste au plus strict, avec un compromis entre sécurité et vitesse ; le détail dépasse ce chapitre, mais retenez que les bases relationnelles gèrent pour vous un problème que les fichiers CSV ignorent complètement.

### À retenir

- Une table qui mélange plusieurs sujets provoque des **anomalies** de mise à jour, d'insertion et de suppression.
- Une **dépendance fonctionnelle**  $X \rightarrow Y$  dit que  $X$  détermine  $Y$ . La fermeture  $X^+$  permet de trouver les **clés**. Les données peuvent **réfuter** une dépendance, pas la prouver.
- **1FN** : cases atomiques. **2FN** : rien ne dépend d'une partie de la clé. **3FN** : rien ne dépend d'un non-clé. Formule : « la clé, toute la clé, et rien que la clé ».
- Une décomposition doit être **sans perte** (théorème de Heath). Normaliser ne répare pas des données déjà contradictoires.
- **Normalisez pour écrire, dénormalisez pour lire** (OLTP contre OLAP, schéma en étoile, vues).

- Un **index** accélère les recherches (de  $N$  à  $\log N$  comparaisons) au prix de l'espace et de l'écriture ; `EXPLAIN QUERY PLAN` montre si la base lit tout (SCAN) ou utilise l'index (SEARCH).
- Une **transaction** est un bloc « tout ou rien » (ACID).

## 5.5 ☉ Pour aller plus loin : les bases NoSQL (MongoDB, Redis)

**Section optionnelle.** Le reste du livre ne dépend pas de cette section. Elle vous donne la carte du territoire « au-delà du relationnel », utile si vous lisez des offres d'emploi (MongoDB, Redis, Cassandra, Neo4j y sont souvent cités) ou si un projet vous met un jour devant l'une de ces bases.

**Honnêteté sur l'exécution.** MongoDB et Redis sont des **serveurs** qui ne sont pas installés dans l'environnement qui a servi à fabriquer ce livre. Les commandes qui leur sont destinées sont donc marquées **non exécutées** : elles n'ont pas de sortie, et nous ne prétendons pas en avoir vu une. À la place, nous reproduisons leur **logique** avec du Python et du JSON, exécutés, de sorte que vous puissiez voir à *quoi ressemblent* les données et les requêtes.

**Intuition.** « NoSQL » signifie « *Not only SQL* » (« pas seulement SQL »). Ce n'est pas *un* modèle, mais une famille de bases nées dans les années 2000 chez des géants du web (Google, Amazon, Facebook), qui avaient des besoins pour lesquels les bases relationnelles étaient mal adaptées : des **milliards** d'enregistrements répartis sur des **centaines de machines**, des données de **formes variées** qui changent sans cesse, des temps de réponse de **quelques millisecondes**.

### 5.5.1 Pourquoi sortir du relationnel ?

Le modèle relationnel est excellent, mais il a des prix : le **schéma est rigide** (ajouter un champ à une table de 10 milliards de lignes est lourd), les **jointures coûtent cher** quand les données sont réparties sur plusieurs machines, et les **garanties ACID** (5.4.6) ralentissent. Les bases NoSQL font des compromis différents. Il en existe quatre familles principales :

Famille	Idée	Exemples	Cas d'usage typique
<b>Clé-valeur</b>	un énorme dictionnaire : on retrouve une valeur par sa clé, très vite	<b>Redis</b> , DynamoDB	cache, paniers, sessions, compteurs, classements
<b>Documents</b>	des documents JSON imbriqués, de forme libre	<b>MongoDB</b> , CouchDB	catalogues, profils, contenus, données semi-structurées
<b>Colonnes larges</b>	tables géantes réparties, écritures très rapides	Cassandra, HBase	journaux, mesures de capteurs, historiques
<b>Graphes</b>	nœuds et relations en première classe	Neo4j	réseaux sociaux, recommandations, détection de fraude

**Le théorème CAP (Brewer, 2000 ; démontré par Gilbert et Lynch, 2002).** Dans un système de données **réparti** sur plusieurs machines, on ne peut pas garantir en même temps les trois propriétés suivantes : Cohérence (tous les lecteurs voient la dernière écriture), Availability,

la **disponibilité** (chaque requête reçoit une réponse), et la **Partition tolérance** (le système continue de fonctionner quand le réseau entre machines est coupé). Comme les coupures réseau **arrivent** (on ne peut pas les exclure), il faut choisir en cas de coupure : **cohérence** (on refuse de répondre pour ne pas donner une donnée périmée) ou **disponibilité** (on répond, quitte à donner une donnée un peu ancienne : on parle de *cohérence à terme*). Les bases relationnelles classiques penchent vers la cohérence ; beaucoup de bases NoSQL, vers la disponibilité. Ce n'est pas « meilleur » ou « moins bon » : c'est un **choix de conception** selon l'application (un virement bancaire exige la cohérence ; le nombre de « j'aime » d'une publication peut être approximatif quelques secondes).

### 5.5.2 Les bases de documents : l'exemple de MongoDB

Dans une base de documents, une commande n'est pas répartie sur plusieurs tables : c'est **un seul document JSON** qui contient tout (le client, les lignes) **imbriqué**. Construisons ces documents à partir de notre base relationnelle, et voyons celui de la commande n° 3 (celle du 5.1.3 aux trois lignes) :

```
import json

lignes_par_commande = {}
requete_lignes = """
 SELECT l.id_commande, p.nom, ca.nom, l.quantite, l.prix_unitaire
 FROM lignes_commande AS l
 JOIN produits AS p ON p.id_produit = l.id_produit
 JOIN categories AS ca ON ca.id_categorie = p.id_categorie
 ORDER BY l.id_commande, l.id_produit"""
for id_commande, produit, categorie, quantite, prix in con.execute(requete_lignes):
 lignes_par_commande.setdefault(id_commande, []).append(
 {"produit": produit, "categorie": categorie, "quantite": quantite, "prix": prix})

requete_commandes = """
 SELECT c.id_commande, c.date_commande, c.canal, c.montant, cl.id_client, cl.prenom, cl.nom,
 ↪ cl.ville
 FROM commandes AS c JOIN clients AS cl ON cl.id_client = c.id_client
 ORDER BY c.id_commande"""
documents = [
 {"_id": i, "date": date, "canal": canal, "montant": montant,
 "client": {"id": id_client, "nom": f"{prenom} {nom}", "ville": ville},
 "lignes": lignes_par_commande[i]}
 for i, date, canal, montant, id_client, prenom, nom, ville in con.execute(requete_commandes)
]
print(len(documents), "documents")
print(json.dumps(documents[2], ensure_ascii=False, indent=2))
```

```
400 documents
{
 "_id": 3,
 "date": "2025-01-04",
 "canal": "Instagram",
 "montant": 88.2,
 "client": {
 "id": 3,
 "nom": "Aymen Hamdi",
 "ville": "Sfax"
 },
 "lignes": [
 {
 "produit": "Bol en céramique de Nabeul",
 "categorie": "Poterie",
 "quantite": 1,
```

```

 "prix": 17.84
 },
 {
 "produit": "Vase peint à la main",
 "categorie": "Poterie",
 "quantite": 1,
 "prix": 64.42
 },
 {
 "produit": "Savon à l'huile d'olive",
 "categorie": "Cosmétiques",
 "quantite": 1,
 "prix": 5.94
 }
]
}

```

Ce document **contient tout ce qu'il faut** pour afficher la commande : pas de jointure à faire, une seule lecture suffit. C'est l'argument central des bases de documents : ce qu'on lit ensemble est rangé ensemble. On les interroge avec des filtres sur les champs, y compris dans les listes imbriquées. Voici « les commandes d'au moins 100 DT qui contiennent un bijou », d'abord à la manière de MongoDB (les filtres se décrivent avec des documents) :

```
javascript noexec // Non exécuté : nécessite un serveur MongoDB. db.commandes.countDocuments({
montant: { $gte: 100 }, "lignes.categorie": "Bijoux" })
```

Reproduisons cette logique en Python sur nos documents (une *compréhension de liste* fait le même travail que le filtre), et comparons avec la réponse **relationnelle** (jointure SQL du 5.2) pour vérifier qu'on obtient bien la même chose :

```

avec_bijou = [d["_id"] for d in documents
 if d["montant"] >= 100 and any(l["categorie"] == "Bijoux" for l in d["lignes"])]

sql = """SELECT COUNT(DISTINCT c.id_commande)
FROM commandes AS c
JOIN lignes_commande AS l ON l.id_commande = c.id_commande
JOIN produits AS p ON p.id_produit = l.id_produit
WHERE c.montant >= 100 AND p.id_categorie = 3"""
print("version documents (Python) :", len(avec_bijou))
print("version relationnelle (SQL):", con.execute(sql).fetchone()[0])

```

```

version documents (Python) : 27
version relationnelle (SQL): 27

```

Même résultat, deux philosophies : dans l'une, on **reconstruit** les liens à la lecture (jointure) ; dans l'autre, on les a **pré-assemblés** à l'écriture (imbrication).

Notez qu'il n'est même pas nécessaire de quitter SQLite pour jouer avec des documents : il sait stocker du JSON dans une colonne de texte et l'interroger avec les fonctions `json_extract` et `json_each`. C'est aussi le cas de PostgreSQL (type `jsonb`), ce qui permet un mélange des deux mondes.

```

con.execute("CREATE TABLE ex_docs (doc TEXT)")
con.executemany("INSERT INTO ex_docs VALUES (?)", [(json.dumps(d, ensure_ascii=False),) for d in
↳ documents])
con.commit()

```

```

SELECT json_extract(doc, '$.canal') AS canal,
COUNT(*) AS commandes,
ROUND(AVG(json_extract(doc, '$.montant')), 2) AS panier_moyen
FROM ex_docs

```

```
GROUP BY canal
ORDER BY canal;
```

canal	commandes	panier_moyen
Boutique	114	74.81
Instagram	138	49.01
Site	148	59.50

On retrouve les paniers moyens par canal du 5.2.4. ('\$canal' est un *chemin* dans le document ; \$.client.ville atteindrait un champ imbriqué.) La même question du bijou, avec json\_each qui « déplie » la liste des lignes :

```
SELECT COUNT(*) AS commandes_avec_bijou_100dt
FROM ex_docs
WHERE json_extract(doc, '$.montant') >= 100
AND EXISTS (SELECT 1 FROM json_each(ex_docs.doc, '$.lignes') AS l
 WHERE json_extract(l.value, '$.categorie') = 'Bijoux');
```

```
commandes_avec_bijou_100dt
27
```

Pour un agrégat complet, MongoDB utilise un **pipeline** d'étapes qui s'enchaînent (le même esprit que les CTE du 5.3) :

```
javascript noexec // Non exécuté : chiffre d'affaires et nombre de commandes par
canal, pour les commandes de 100 DT et plus. db.commandes.aggregate([{ $match: {
montant: { $gte: 100 } } }, { $group: { _id: "$canal", ca: { $sum: "$montant" },
commandes: { $sum: 1 } } }, { $sort: { ca: -1 } }])
```

**Le revers de la médaille : la redondance.** Chaque document contient la ville du client. Combien de documents faudrait-il modifier si Sami Dridi (client n° 2) déménageait ?

```
a_modifier = sum(1 for d in documents if d["client"]["id"] == 2)
print("documents à modifier pour un seul déménagement :", a_modifier)
con.execute("DROP TABLE ex_docs")
con.commit()
```

```
documents à modifier pour un seul déménagement : 23
```

Vingt-trois ! C'est exactement l'anomalie de mise à jour du 5.4.1 : en choisissant l'**imbrication**, on assume la **redondance**, et c'est à l'application de la gérer. Dans le monde des documents, on décide au cas par cas ce qu'on imbrique (ce qui est lu ensemble, qui change rarement) et ce qu'on référence (ce qui change souvent). Aucun modèle n'est gratuit.

### 5.5.3 Les bases clé-valeur : l'exemple de Redis

**Redis** est le plus simple des modèles : un dictionnaire géant **en mémoire** (donc extrêmement rapide : des centaines de milliers d'opérations par seconde). On y range des valeurs sous une clé : SET cle valeur, GET cle. Il propose aussi des types pratiques : compteurs, listes, ensembles, **ensembles triés** (classements). Voici quelques commandes typiques pour une boutique en ligne :

```
bash noexec # Non exécuté : nécessite un serveur Redis. redis-cli SET panier:2
'{"articles": 3, "total": 88.2}' EX 3600 # panier du client 2, expire dans 1 heure
redis-cli GET panier:2 redis-cli INCR visites:page_accueil #
compteur atomique redis-cli ZADD classement_clients 1874.3 "Sami Dridi" 1701.7
"Yassine Lahmar" # ensemble trié par score redis-cli ZREVRANGE classement_clients
0 2 WITHSCORES # les 3 meilleurs
```

Le cas d'usage numéro un est le **cache** : stocker le résultat d'une requête lente (par exemple le tableau de bord du chiffre d'affaires) pour ne pas la recalculer à chaque visite. Voici le mécanisme avec un simple dictionnaire Python : la première fois, la requête SQL est exécutée ; les fois suivantes, la réponse vient directement du « cache » :

```
cache = {}
nb_requetes_sql = 0

def chiffre_affaires_du_canal(canal):
 global nb_requetes_sql
 cle = f"ca:{canal}"
 if cle in cache:
 return cache[cle]
 nb_requetes_sql += 1
 valeur = con.execute("SELECT ROUND(SUM(montant)) FROM commandes WHERE canal = ?",
 (canal,)).fetchone()[0]
 cache[cle] = valeur
 return valeur

for canal in ["Site", "Site", "Boutique", "Site", "Boutique", "Instagram", "Site"]:
 print(f"{canal:10s}", chiffre_affaires_du_canal(canal))
print("7 demandes, requêtes SQL réellement exécutées :", nb_requetes_sql)
```

```
Site 8807.0
Site 8807.0
Boutique 8528.0
Site 8807.0
Boutique 8528.0
Instagram 6764.0
Site 8807.0
7 demandes, requêtes SQL réellement exécutées : 3
```

Sept demandes, trois requêtes seulement. Il reste le problème classique : **quand périmé le cache** ? (si une nouvelle commande arrive, la valeur en cache devient fausse). Redis résout cela avec une **durée de vie** (EX 3600, comme ci-dessus) : la clé s'efface toute seule. Un mot célèbre résume la difficulté : « *il n'y a que deux choses difficiles en informatique : invalider un cache et nommer les choses.* »

### 5.5.4 Alors, que choisir ?

Critère	Relationnel (SQL)	NoSQL
Structure des données	stable, bien définie	variable, évolutive
Relations entre données	nombreuses (jointures)	peu, ou imbriquées
Cohérence	forte (ACID)	souvent « à terme »
Requêtes	très riches (SQL)	plus limitées, spécialisées
Volume	de petit à très grand	pensé pour le très grand, réparti
Analyse de données	<b>excellent</b>	souvent à exporter d'abord

Pour un data scientist, la réponse pratique est la suivante. **Commencez par le relationnel** (PostgreSQL en particulier : fiable, gratuit, et capable de stocker du JSON) : il répond à l'immense majorité des besoins, et l'analyse y est la plus confortable. N'allez vers le NoSQL que lorsqu'un besoin précis l'impose (cache ultra-rapide, volume réparti, données de forme libre, graphes). Dans les grandes entreprises, on trouve d'ailleurs souvent **plusieurs bases à la fois** (on parle de *persistance polyglotte*) : un SGBD relationnel pour les commandes, Redis pour le cache, un moteur de documents pour le catalogue, un entrepôt de données pour l'analyse. En tant qu'analyste, vous serez souvent celui qui les **réunit**, d'où l'intérêt de connaître chacune de ces familles.

**À retenir**

- **NoSQL** = « pas seulement SQL » : quatre familles (clé-valeur, documents, colonnes, graphes), nées pour le **très gros volume**, la **souplesse** du schéma et la **répartition**.
- Le **théorème CAP** impose un choix, en cas de coupure réseau, entre cohérence et disponibilité.
- Une base de **documents** imbrique ce qui est lu ensemble : lecture sans jointure, mais **redondance** à gérer (anomalie de mise à jour du 5.4.1).
- Une base **clé-valeur** (Redis) est un dictionnaire ultra-rapide, idéal comme **cache** ; tout l'art est de savoir quand le périmer.
- En pratique : **commencez par le relationnel**, choisissez le NoSQL quand un besoin précis l'exige.
- Les commandes MongoDB et Redis de cette section n'ont **pas** été exécutées ; leur logique a été reproduite et vérifiée en Python/SQLite.

## 5.6 Exercices du chapitre 5

Cherchez d'abord seul(e) (sur papier, ou en écrivant la requête dans votre éditeur), vérifiez ensuite en exécutant, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Tous les exercices s'appuient sur la base de Dar Jasmin du 5.1 (donnees/dar\_jasmin.db). La date « du jour » est le 31 décembre 2025.

**Énoncés**

**Exercice 1 ★ (clés et contraintes).** Yasmine veut enregistrer les **avis** des clients sur les produits. Un avis a un numéro, est écrit par **un** client sur **un** produit, à une date, avec une note de 1 à 5 et un commentaire facultatif. Un client ne peut laisser **qu'un seul avis par produit**. (a) Quelle est la clé primaire, quelles sont les clés étrangères ? (b) Écrivez le CREATE TABLE avec toutes les contraintes. (c) Vérifiez qu'un avis valide est accepté, et que trois avis invalides (note 6, doublon client/produit, produit inexistant) sont refusés.

**Exercice 2 ★ (filtrer).** Combien de commandes de **plus de 100 DT** ont été passées **en boutique** en **décembre 2025** ? Affichez les trois plus grosses.

**Exercice 3 ★ (agréger et joindre).** Pour chaque **ville de client**, donnez le nombre de clients ayant commandé, le nombre de commandes, le chiffre d'affaires et le panier moyen, classées par chiffre d'affaires décroissant. Quelle ville a le meilleur panier moyen ? Est-ce aussi celle qui a le plus gros chiffre d'affaires ?

**Exercice 4 ★★ (HAVING, jointure externe).** Quels produits se sont vendus à **moins de 40 unités** sur l'année ? Pour chacun, donnez les unités vendues et le chiffre d'affaires. Faut-il arrêter de vendre tous ces produits ?

**Exercice 5 ★★ (anti-jointure).** Les clients qui habitent près de la boutique (**Tunis, La Marsa, Ariana**) mais n'y ont **jamaïs acheté** sont une cible de choix pour une invitation. Listez-les, de deux façons différentes (NOT EXISTS et LEFT JOIN ... IS NULL), et vérifiez que les deux donnent le même nombre.

**Exercice 6 ★★ (NULL).** (a) Pour chaque ville, quel est le **pourcentage de clients dont le téléphone est renseigné** ? (b) Un stagiaire écrit WHERE id\_parrain != 3 pour compter les clients **qui n'ont pas été parrainés par le client n° 3**. Combien de lignes obtient-il ? Combien devrait-il en obtenir ? Corrigez.



**Exercice 7 ★★ (dates, et regard statistique).** Quel est le **jour de la semaine** le plus chargé (en nombre de commandes et en chiffre d'affaires) ? Cette différence entre jours est-elle significative, ou du bruit ? (Utilisez un test du khi-deux d'adéquation, chapitre 3.)

**Exercice 8 ★★★ (fenêtre).** Pour **chaque catégorie**, quel est le produit au **plus gros chiffre d'affaires** ?

**Exercice 9 ★★★ (CTE et fenêtres).** (a) Quelle proportion des clients actifs a commandé **au moins deux fois** (taux de réachat) ? (b) Pour ces clients, comparez le montant de leur **première** commande à celui de leur **dernière** : combien dépensent plus à la fin qu'au début ?

**Exercice 10 ★★★ (récursivité).** Pour les trois clients ayant le plus de filleuls (directs ou non), calculez le **nombre de membres** de leur réseau (sans eux-mêmes) et le **chiffre d'affaires cumulé de ces membres**. Qui est l'ambassadeur le plus rentable ?

**Exercice 11 ★★★ (dépendances fonctionnelles).** Dar Jasmin enregistre ses livraisons dans une seule table : livraisons(id\_livraison, id\_commande, transporteur, tel\_transporteur, ville\_livraison, frais). Règles de gestion : une livraison concerne une commande, est assurée par un transporteur et part vers une ville ; un transporteur n'a qu'un seul numéro de téléphone ; les **frais ne dépendent que de la ville** de livraison (barème par ville). (a) Écrivez les dépendances fonctionnelles. (b) Déterminez la clé. (c) La table est-elle en 2FN ? en 3FN ? (d) Proposez une décomposition en 3FN, et vérifiez-la avec la fonction fermeture du 5.4.2.

## Corrigés

**Corrigé 1.** (a) La clé primaire est id\_avis. Les clés étrangères sont id\_client (vers clients) et id\_produit (vers produits). La règle « un avis par client et par produit » se traduit par une contrainte UNIQUE (id\_client, id\_produit) : le couple est une **clé candidate** en plus de id\_avis. (b) et (c) :

```
CREATE TABLE ex_avis (
 id_avis INTEGER PRIMARY KEY,
 id_client INTEGER NOT NULL REFERENCES clients(id_client),
 id_produit INTEGER NOT NULL REFERENCES produits(id_produit),
 date_avis TEXT NOT NULL,
 note INTEGER NOT NULL CHECK (note BETWEEN 1 AND 5),
 commentaire TEXT,
 UNIQUE (id_client, id_produit)
);
INSERT INTO ex_avis VALUES (1, 2, 8, '2025-12-02', 5, 'Magnifique broderie');

essais = {
 "note de 6": "INSERT INTO ex_avis VALUES (2, 3, 8, '2025-12-03', 6, NULL)",
 "doublon client/produit": "INSERT INTO ex_avis VALUES (3, 2, 8, '2025-12-04', 4, NULL)",
 "produit inexistant": "INSERT INTO ex_avis VALUES (4, 2, 99, '2025-12-05', 4, NULL)",
}
for nom, requete in essais.items():
 try:
 con.execute(requete)
 print(f"{nom:24s} -> accepté (!)")
 except sqlite3.IntegrityError as erreur:
 print(f"{nom:24s} -> refusé : {erreur}")
print("avis enregistrés :", con.execute("SELECT COUNT(*) FROM ex_avis").fetchone()[0])
con.execute("DROP TABLE ex_avis")
```

```
note de 6 -> refusé : CHECK constraint failed: note BETWEEN 1 AND 5
doublon client/produit -> refusé : UNIQUE constraint failed: ex_avis.id_client, ex_avis.id_produit
produit inexistant -> refusé : FOREIGN KEY constraint failed
avis enregistrés : 1
```



L'avis valide est enregistré, les trois autres sont refusés chacun par une contrainte différente (CHECK, UNIQUE, clé étrangère). Le commentaire est facultatif : c'est la seule colonne sans NOT NULL.

**Corrigé 2.** On filtre sur trois conditions (AND) ; les dates ISO se comparent comme du texte (5.1.4) : « à partir du 1er décembre » suffit, puisque la base s'arrête au 31.

```
SELECT COUNT(*) AS nb_commandes
FROM commandes
WHERE canal = 'Boutique' AND date_commande >= '2025-12-01' AND montant > 100;
```

```
nb_commandes
 6
```

```
SELECT id_commande, date_commande, montant
FROM commandes
WHERE canal = 'Boutique' AND date_commande >= '2025-12-01' AND montant > 100
ORDER BY montant DESC
LIMIT 3;
```

```
id_commande date_commande montant
 362 2025-12-16 208.8
 376 2025-12-22 147.6
 391 2025-12-29 128.5
```

**Corrigé 3.** Il faut joindre clients (la ville) et commandes (les montants). Pour compter les *clients distincts*, COUNT(DISTINCT ...).

```
SELECT cl.ville,
 COUNT(DISTINCT cl.id_client) AS clients_actifs,
 COUNT(*) AS commandes,
 ROUND(SUM(c.montant)) AS chiffre_affaires,
 ROUND(AVG(c.montant), 2) AS panier_moyen
FROM clients AS cl
JOIN commandes AS c ON c.id_client = cl.id_client
GROUP BY cl.ville
ORDER BY chiffre_affaires DESC;
```

ville	clients_actifs	commandes	chiffre_affaires	panier_moyen
Ariana	13	100	6456.0	64.56
La Marsa	9	80	4619.0	57.73
Tunis	9	49	3407.0	69.53
Sousse	7	38	2200.0	57.89
Monastir	6	41	2101.0	51.25
Sfax	6	34	2043.0	60.09
Bizerte	8	31	1735.0	55.95
Nabeul	8	27	1538.0	56.96

Ariana est en tête pour le chiffre d'affaires (6 456 DT) mais pas pour le panier : c'est **Tunis** qui a le meilleur panier moyen (69,53 DT), avec un nombre de commandes beaucoup plus faible (49 contre 100). Le chiffre d'affaires est le produit *nombre de commandes* × *panier moyen* : un fort volume de petits paniers peut battre un faible volume de gros paniers.

**Corrigé 4.** LEFT JOIN pour ne pas perdre un éventuel produit **jamais vendu** (il aurait 0 unité, ou NULL : voir COALESCE). Le filtre sur une valeur agrégée s'écrit avec HAVING.

```
SELECT p.nom AS produit,
 COALESCE(SUM(l.quantite), 0) AS unites,
 ROUND(COALESCE(SUM(l.quantite * l.prix_unitaire), 0)) AS chiffre_affaires
FROM produits AS p
LEFT JOIN lignes_commande AS l ON l.id_produit = p.id_produit
GROUP BY p.id_produit
HAVING COALESCE(SUM(l.quantite), 0) < 40
```

```
ORDER BY unites;
```

	produit	unites	chiffre_affaires
Margoum	(petit tapis)	13	1593.0
Vase peint à la main		37	2411.0
Écharpe en soie		37	2031.0

Trois produits. Mais **faible volume ne veut pas dire faible intérêt** : le margoum (petit tapis), à 120 DT l'unité, ne s'est vendu qu'à 13 exemplaires et rapporte pourtant 1 593 DT, plus que bien des produits très vendus. Le vase peint et l'écharpe en soie (37 unités chacun) sont même parmi les produits au plus gros chiffre d'affaires du magasin. Arrêter de les vendre serait une erreur : le bon indicateur dépend de la question (rotation, chiffre d'affaires, marge). Aucun de nos produits n'est resté invendu.

**Corrigé 5.** Version NOT EXISTS : on garde les clients de ces villes pour lesquels il n'existe **aucune** commande en boutique. Version LEFT JOIN : on joint **seulement** les commandes de boutique (la condition sur le canal va dans le ON, pas dans le WHERE !), puis on garde ceux qui n'ont aucun partenaire.

```
SELECT cl.id_client, cl.prenom, cl.nom, cl.ville
FROM clients AS cl
WHERE cl.ville IN ('Tunis', 'La Marsa', 'Ariana')
AND NOT EXISTS (SELECT 1 FROM commandes AS c
 WHERE c.id_client = cl.id_client AND c.canal = 'Boutique')
ORDER BY cl.id_client;
```

id_client	prenom	nom	ville
10	Nour	Hamdi	Tunis
20	Aymen	Bouazizi	Tunis
28	Emna	Ben Salah	Ariana
32	Oussama	Khelifi	La Marsa
55	Oussama	Mejri	La Marsa
65	Dorra	Mejri	Tunis
75	Lina	Mejri	La Marsa
78	Ines	Ayari	Tunis

```
SELECT COUNT(*) AS avec_left_join
FROM clients AS cl
LEFT JOIN commandes AS c ON c.id_client = cl.id_client AND c.canal = 'Boutique'
WHERE cl.ville IN ('Tunis', 'La Marsa', 'Ariana')
AND c.id_commande IS NULL;
```

```
avec_left_join
8
```

Huit clients, quelle que soit la méthode. Si l'on avait placé `c.canal = 'Boutique'` dans le WHERE, la requête aurait éliminé justement les lignes « sans partenaire » (dont `c.canal` est NULL), et le résultat aurait été vide : un cas d'école du 5.2.7. Parmi ces huit clients, certains n'ont **jamais** acheté du tout (comme Nour Hamdi, la grande ambassadrice du 5.3.6) : l'invitation à la boutique serait pour eux un premier achat.

**Corrigé 6.** (a) `telephone IS NOT NULL` vaut 1 ou 0 : sa moyenne est la proportion de numéros renseignés.

```
SELECT ville,
 COUNT(*) AS clients,
 SUM(telephone IS NOT NULL) AS avec_telephone,
 ROUND(100.0 * AVG(telephone IS NOT NULL), 1) AS pourcentage
FROM clients
GROUP BY ville
```

**ORDER BY** pourcentage **DESC**;

ville	clients	avec_telephone	pourcentage
Monastir	7	7	100.0
Bizerte	11	11	100.0
Sousse	11	10	90.9
Sfax	7	6	85.7
Nabeul	10	8	80.0
Tunis	11	8	72.7
Ariana	13	9	69.2
La Marsa	10	6	60.0

Les numéros sont toujours renseignés à Monastir et à Bizerte, mais seulement à 60 % à La Marsa.

(b) Le comparatif != renvoie *inconnu* quand id\_parrain est NULL : les 49 clients **sans parrain** sont éliminés, alors qu'ils ne sont évidemment pas parrainés par le client n° 3.

```
SELECT (SELECT COUNT(*) FROM clients WHERE id_parrain != 3) AS naif,
 (SELECT COUNT(*) FROM clients WHERE id_parrain != 3 OR id_parrain IS NULL) AS corrige,
 (SELECT COUNT(*) FROM clients WHERE id_parrain = 3) AS parraines_par_3,
 (SELECT COUNT(*) FROM clients WHERE id_parrain IS NULL) AS sans_parrain;
```

naif	corrige	parraines_par_3	sans_parrain
30	79	1	49

Le stagiaire obtient 30 lignes, alors qu'il en faut 79 (80 clients moins l'unique filleul du client n° 3). Les 49 sans parrain manquent à l'appel ;  $30 + 49 = 79$ . On peut aussi écrire `WHERE id_parrain IS NOT 3` (opérateur de SQLite qui traite proprement NULL) ou `COALESCE(id_parrain, 0) != 3`.

**Corrigé 7.** `strftime('%w', ...)` donne 0 pour dimanche... 6 pour samedi ; un CASE donne des noms lisibles.

```
SELECT CASE strftime('%w', date_commande)
 WHEN '0' THEN 'dimanche' WHEN '1' THEN 'lundi' WHEN '2' THEN 'mardi'
 WHEN '3' THEN 'mercredi' WHEN '4' THEN 'jeudi' WHEN '5' THEN 'vendredi'
 ELSE 'samedi' END
 AS jour,
 COUNT(*)
 AS commandes,
 ROUND(SUM(montant))
 AS chiffre_affaires
FROM commandes
GROUP BY strftime('%w', date_commande)
ORDER BY commandes DESC, chiffre_affaires DESC;
```

jour	commandes	chiffre_affaires
mercredi	66	3722.0
lundi	63	4071.0
vendredi	61	3794.0
dimanche	56	3350.0
mardi	54	3203.0
jeudi	50	3319.0
samedi	50	2641.0

Le mercredi est en tête pour le nombre de commandes (66), le lundi pour le chiffre d'affaires. Mais l'écart est-il autre chose que du bruit ? Test du khi-deux d'adéquation (3.4) de l'hypothèse « les commandes se répartissent **uniformément** sur les sept jours » :

```
from scipy import stats
effectifs = [r[0] for r in con.execute(
 "SELECT COUNT(*) FROM commandes GROUP BY strftime('%w', date_commande) ORDER BY strftime('%w',
 ↳ date_commande)")]
khi2, p = stats.chisquare(effectifs)
print("effectifs (dimanche ... samedi) :", effectifs)
print(f"khi-deux = {khi2:.2f}, p-valeur = {p:.3f}")
```

```
effectifs (dimanche ... samedi) : [56, 63, 54, 66, 50, 61, 50]
khi-deux = 4.21, p-valeur = 0.648
```

Avec une p-valeur largement supérieure à 5 %, on **ne peut pas rejeter** l'uniformité : les différences entre jours sont compatibles avec le simple hasard. (C'est d'ailleurs normal : ces dates ont été simulées sans aucun effet de jour de semaine.) Leçon : un classement (« le mercredi est le meilleur jour ») n'est pas une découverte tant qu'on ne l'a pas confronté au hasard.

**Corrigé 8.** On calcule d'abord le chiffre d'affaires par produit (CTE `par_produit`), puis un RANK par catégorie, et l'on ne garde que le rang 1 (5.3.2).

```
WITH par_produit AS (
 SELECT cat.nom AS categorie, p.nom AS produit,
 ROUND(SUM(l.quantite * l.prix_unitaire)) AS chiffre_affaires
 FROM lignes_commande AS l
 JOIN produits AS p ON p.id_produit = l.id_produit
 JOIN categories AS cat ON cat.id_categorie = p.id_categorie
 GROUP BY p.id_produit
)
SELECT categorie, produit, chiffre_affaires
FROM (
 SELECT *, RANK() OVER (PARTITION BY categorie ORDER BY chiffre_affaires DESC) AS rang
 FROM par_produit
)
WHERE rang = 1
ORDER BY categorie;
```

categorie	produit	chiffre_affaires
Bijoux	Bague en argent	2638.0
Cosmétiques	Huile de nigelle	1128.0
Poterie	Vase peint à la main	2411.0
Textile	Écharpe en soie	2031.0

La bague en argent domine les bijoux, le vase peint la poterie, l'écharpe en soie le textile et l'huile de nigelle les cosmétiques. (RANK laisserait apparaître deux lignes en cas d'égalité parfaite ; ici il n'y en a pas.)

**Corrigé 9.** (a) Un client « actif » est un client qui a au moins une commande. (b) On numérote les commandes de chaque client dans les deux sens (ROW\_NUMBER croissant et décroissant) : la première a le rang 1 dans l'ordre croissant, la dernière a le rang 1 dans l'ordre décroissant.

```
SELECT COUNT(*) AS clients_actifs,
 SUM(n >= 2) AS rechetent,
 ROUND(100.0 * SUM(n >= 2) / COUNT(*), 1) AS taux_de_reachat_pct
FROM (SELECT id_client, COUNT(*) AS n FROM commandes GROUP BY id_client);
```

clients_actifs	rechetent	taux_de_reachat_pct
66	58	87.9

```
WITH numerotees AS (
 SELECT id_client, montant,
 ROW_NUMBER() OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
 ↳ depuis_le_debut,
 ROW_NUMBER() OVER (PARTITION BY id_client ORDER BY date_commande DESC, id_commande DESC) AS
 ↳ depuis_la_fin,
 COUNT(*) OVER (PARTITION BY id_client) AS
 ↳ nb_commandes
 FROM commandes
),
premiere_et_derniere AS (
 SELECT id_client,
 MAX(CASE WHEN depuis_le_debut = 1 THEN montant END) AS premiere,
```

```

 MAX(CASE WHEN depuis_la_fin = 1 THEN montant END) AS derniere
FROM numerotees
WHERE nb_commandes >= 2
GROUP BY id_client
)
SELECT COUNT(*) AS clients,
 SUM(derniere > premiere) AS dependant_plus_a_la_fin,
 ROUND(AVG(premiere), 2) AS premiere_moyenne,
 ROUND(AVG(derniere), 2) AS derniere_moyenne
FROM premiere_et_derniere;

```

clients	dependant_plus_a_la_fin	premiere_moyenne	derniere_moyenne
58	29	61.91	57.4

Sur 66 clients actifs, 58 ont recommandé au moins une fois : un **taux de réachat de 87,9 %**, excellent. Parmi eux, la moitié exactement (29 sur 58) dépense davantage lors de la dernière commande que lors de la première, et les montants moyens sont proches (61,91 DT contre 57,40 DT) : **pas de tendance** à dépenser plus avec le temps dans ces données (là encore, un test de comparaison de moyennes au sens du chapitre 3 serait à faire avant de conclure à autre chose que du hasard).

**Corrigé 10.** Une CTE récursive parcourt les réseaux, en gardant la **racine** de chacun comme au 5.3.6. On retire la racine elle-même (profondeur 0) du décompte et du chiffre d'affaires.

```

WITH RECURSIVE reseau(id_client, racine, profondeur) AS (
 SELECT id_client, id_client, 0 FROM clients WHERE id_parrain IS NULL
 UNION ALL
 SELECT c.id_client, r.racine, r.profondueur + 1
 FROM clients AS c JOIN reseau AS r ON c.id_parrain = r.id_client
),
ca_client AS (
 SELECT id_client, SUM(montant) AS ca FROM commandes GROUP BY id_client
)
SELECT r.racine,
 cl.prenom || ' ' || cl.nom AS ambassadeur,
 COUNT(*) AS membres,
 ROUND(COALESCE(SUM(ca.ca), 0), 2) AS ca_des_membres
FROM reseau AS r
JOIN clients AS cl ON cl.id_client = r.racine
LEFT JOIN ca_client AS ca ON ca.id_client = r.id_client
WHERE r.profondueur > 0
GROUP BY r.racine
ORDER BY membres DESC, ca_des_membres DESC
LIMIT 3;

```

racine	ambassadeur	membres	ca_des_membres
10	Nour Hamdi	5	808.2
7	Zied Ben Salah	4	2048.7
1	Yassine Lahmar	4	1117.0

Le LEFT JOIN est indispensable : un membre qui n'a jamais commandé n'a pas de ligne dans ca\_client ; avec un JOIN ordinaire, il disparaîtrait du décompte des membres. Les trois plus gros réseaux sont ceux de Nour Hamdi (n° 10, 5 membres), de Zied Ben Salah (n° 7, 4 membres) et de Yassine Lahmar (n° 1, 4 membres ; il est classé après Zied car on départage les ex æquo par le chiffre d'affaires). L'ambassadeur le plus **rentable** est **Zied Ben Salah** : ses quatre filleuls ont dépensé 2 048,70 DT, contre 1 117,00 DT pour ceux de Yassine Lahmar et 808,20 DT seulement pour les cinq membres du plus grand réseau, celui de Nour Hamdi. Le réseau le plus **grand** n'est donc pas le plus **rentable** : il faut mesurer ce qu'on veut optimiser.

**Corrigé 11.** (a) Dépendances :

- $\text{id\_livraison} \rightarrow \text{id\_commande}, \text{transporteur}, \text{ville\_livraison}$  (une livraison fixe sa commande, son transporteur et sa destination) ;
- $\text{transporteur} \rightarrow \text{tel\_transporteur}$  ;
- $\text{ville\_livraison} \rightarrow \text{frais}$ .

- (b) La fermeture de  $\text{id\_livraison}$  contient tous les attributs (elle atteint  $\text{tel\_transporteur}$  par  $\text{transporteur}$  et  $\text{frais}$  par  $\text{ville\_livraison}$ ), et c'est un attribut seul : c'est donc **la clé**.  
 (c) **2FN** : oui, trivialement, car la clé est formée d'**un seul** attribut (il ne peut pas y avoir de dépendance partielle). **3FN** : **non**, car deux dépendances sont **transitives** :  $\text{id\_livraison} \rightarrow \text{transporteur} \rightarrow \text{tel\_transporteur}$ , et  $\text{id\_livraison} \rightarrow \text{ville\_livraison} \rightarrow \text{frais}$ . Conséquence concrète : le téléphone d'un transporteur est répété sur toutes ses livraisons, et le barème d'une ville sur chaque livraison vers elle (anomalies du 5.4.1). (d) Décomposition :  $\text{livraisons}(\text{id\_livraison}, \text{id\_commande}, \text{transporteur}, \text{ville\_livraison})$ ,  $\text{transporteurs}(\text{transporteur}, \text{tel\_transporteur})$ ,  $\text{tarifs}(\text{ville\_livraison}, \text{frais})$ . Vérification par le calcul :

```

deps = [
 (["id_livraison"], ["id_commande", "transporteur", "ville_livraison"]),
 (["transporteur"], ["tel_transporteur"]),
 (["ville_livraison"], ["frais"]),
]
tous = {"id_livraison", "id_commande", "transporteur", "tel_transporteur", "ville_livraison", "frais"}
print("fermeture de {id_livraison} :", sorted(fermeture(["id_livraison"], deps)))
print("c'est une clé :", fermeture(["id_livraison"], deps) == tous)

Dans chaque table de la décomposition, la clé détermine bien toutes les colonnes de la table.
decomposition = {
 "livraisons": ({"id_livraison", "id_commande", "transporteur", "ville_livraison"},
 ↳ {"id_livraison"}),
 "transporteurs": ({"transporteur", "tel_transporteur"}, {"transporteur"}),
 "tarifs": ({"ville_livraison", "frais"}, {"ville_livraison"}),
}
for nom, (attributs, cle) in decomposition.items():
 print(f"{nom:14s} clé {sorted(cle)} détermine toute la table : {fermeture(cle, deps) ==
 ↳ attributs}")

fermeture de {id_livraison} : ['frais', 'id_commande', 'id_livraison', 'tel_transporteur',
↳ 'transporteur', 'ville_livraison']
c'est une clé : True
livraisons clé ['id_livraison'] détermine toute la table : True
transporteurs clé ['transporteur'] détermine toute la table : True
tarifs clé ['ville_livraison'] détermine toute la table : True

```

Dans chaque table, la clé détermine toutes les autres colonnes, et les deux dépendances transitives ont été isolées chacune dans sa propre table (aucune dépendance entre colonnes non-clés ne subsiste) : le schéma est en 3FN, et même en BCNF puisque le seul déterminant de chaque table est sa clé. La décomposition est **sans perte** (théorème de Heath, 5.4.3) : chaque découpage sépare un attribut déterminant ( $\text{transporteur}, \text{ville\_livraison}$ ) de ce qu'il détermine, et le garde dans la table de gauche comme clé étrangère.

## Bilan du chapitre 5

Vous savez maintenant :

- **expliquer pourquoi** une base relationnelle vaut mieux qu'un fichier (redondance, incohérence, contraintes, volume, accès simultanés), et lire un schéma : tables, **clés primaires et étrangères**, relations 1-N et N-N ;

- **écrire des requêtes SQL** complètes : SELECT, WHERE, ORDER BY, GROUP BY/HAVING, **jointures** (INNER, LEFT, auto-jointure), sous-requêtes, opérations ensemblistes ;
- **éviter les pièges classiques** : priorité de AND/OR, NULL (trois valeurs de vérité, NOT IN), dates et date( 'now' ), WHERE contre HAVING ;
- **calculer sans écraser les lignes** grâce aux **fonctions fenêtres** (classements, cumuls, moyennes mobiles, LAG/LEAD), structurer une requête en **CTE**, et parcourir des hiérarchies par **réursion** ;
- **concevoir un schéma** : dépendances fonctionnelles, 1FN/2FN/3FN, décomposition sans perte, et savoir quand dénormaliser (OLTP contre OLAP, vues, schéma en étoile) ;
- comprendre ce que font les **index** (EXPLAIN QUERY PLAN) et les **transactions** (ACID) ;
- (en option) situer les bases **NoSQL** (documents, clé-valeur) et le théorème **CAP**.

Le chapitre 6 clôt la partie « boîte à outils » avec les habitudes de travail du professionnel : **Git** pour garder l'historique de vos analyses (y compris vos requêtes SQL, qui sont du code comme les autres), les **notebooks Jupyter** pour mélanger code, résultats et explications, et la **ligne de commande** pour tout automatiser. Ensuite, le projet de clôture du volume réunira tout ce que vous avez appris, des mathématiques au SQL, dans une seule étude de bout en bout.

---

# Chapitre 6 : Outils de travail

« Un bon artisan ne se reconnaît pas seulement à ses gestes, mais à l'ordre de son atelier. »

Vous savez maintenant calculer (chapitres 1 à 3), programmer (chapitre 4) et interroger une base de données (chapitre 5). Reste une question que personne n'ose poser en cours, mais qui décide de la vie quotidienne d'un data scientist : **comment ne pas se perdre dans son propre travail ?**

Voici la scène, que tout le monde a vécue au moins une fois. Yasmine ouvre le dossier de son analyse des ventes de Dar Jasmin :

```
analyse.py
analyse_v2.py
analyse_v2_corrige.py
analyse_FINAL.py
analyse_FINAL_vrai.py
analyse_FINAL_vrai_ne_pas_toucher.py
rapport_mars.docx
rapport_mars (copie).docx
```

Quelle version a produit le graphique envoyé à la banque la semaine dernière ? Impossible de le savoir. Que s'est-il passé quand elle a « juste changé un petit truc » mardi soir, et que tous les chiffres se sont mis à bouger ? Mystère. Et si son frère Amine veut l'aider, comment fusionnent-ils leurs deux versions sans écraser le travail de l'autre ?

Ce chapitre vous donne **quatre outils** qui répondent à ces questions, et qui sont utilisés dans toutes les équipes de données du monde.

## Le chemin de ce chapitre

- **6.1 Git et gestion de versions** : une machine à remonter le temps pour vos fichiers. On y apprend à photographier son travail (*commit*), à comparer, à défaire, à travailler à plusieurs (*branches*, *fusions*, *conflits*) et à partager (*dépôts distants*).
- **6.2 Notebooks Jupyter** : le carnet de laboratoire interactif où code, résultats et explications cohabitent. On y apprend aussi à **s'en méfier** : un notebook mal utilisé produit des résultats que personne ne peut reproduire.
- **6.3 Ligne de commande et environnements** : parler directement à l'ordinateur avec du texte, enchaîner de petits outils, et isoler les bibliothèques de chaque projet dans un **environnement virtuel**.
- **⊕ Pour aller plus loin** : scripts shell et bases de Docker (6.4) ; recherche reproductible, de Pandoc à Quarto en passant par LaTeX et Make (6.5).
- **6.6 Exercices corrigés** et bilan du chapitre.



**Le fil conducteur : la reproductibilité.** Un résultat n'a de valeur que si **quelqu'un d'autre (ou vous-même dans six mois) peut le refaire**. Chaque outil de ce chapitre attaque une cause différente de non-reproductibilité : Git (« quelle version du code ? »), les notebooks bien tenus (« dans quel ordre a-t-on exécuté les cellules ? »), les environnements (« avec quelles versions des bibliothèques ? »), les graines aléatoires et les `Makefile` (« avec quelles données et quelles étapes ? »). Le terme technique est la *reproductibilité computationnelle*.

**Comment travailler avec ce chapitre.** Contrairement aux précédents, les exemples de ce chapitre se tapent dans un **terminal** (aussi appelé *console* ou *shell*), pas dans Python. Sous Linux et macOS, ouvrez l'application « Terminal ». Sous Windows, installez **Git for Windows** (il fournit *Git Bash*, un terminal compatible avec tous nos exemples) ou, mieux, **WSL** (le sous-système Linux de Windows). Les lignes que vous devez taper sont celles des blocs de code ; les blocs gris qui suivent montrent ce que l'ordinateur répond.

## L'atelier de Yasmine

Pour que les exemples soient concrets, nous préparons un petit **atelier** : un dossier de travail dans lequel nous copions le fichier de données du livre.

**À propos de \$DONNEES.** Le fichier `commandes.csv` est fourni avec le livre dans le dossier `donnees/`. Dans les exemples qui suivent, la variable `$DONNEES` désigne le **chemin de ce dossier** sur la machine qui exécute le code. Chez vous, remplacez "`$DONNEES`" par l'endroit où vous avez rangé le fichier (par exemple `~/livre/donnees`). Le caractère `~` est une abréviation

```
mkdir -p ~/atelier/dar-jasmin
cd ~/atelier/dar-jasmin
cp "$DONNEES/commandes.csv" .
ls
wc -l commandes.csv
```

```
commandes.csv
401 commandes.csv
```

Chaque ligne tapée est une **commande** : un nom (`mkdir`, `cd`, `cp`, `ls`) suivi d'**arguments**. Ici : créer un dossier (`mkdir`, pour *make directory* ; l'option `-p` crée aussi les dossiers intermédiaires et ne se plaint pas s'il existe déjà), s'y placer (`cd`, *change directory*), y copier le fichier (`cp`, *copy* ; le point `.` désigne « le dossier où je suis »), puis lister le contenu (`ls`, *list*) et compter les lignes du fichier (`wc -l`, *word count*, option *lines*). Le résultat est celui qu'on attend : 400 commandes, une ligne chacune, plus la ligne d'en-tête, soit 401 lignes.

**Honnêteté sur l'exécution.** Tous les exemples de ce chapitre qui peuvent s'exécuter sans réseau ni logiciel spécial ont été **réellement exécutés** dans un atelier jetable, et la sortie affichée est la vraie. Les rares exceptions (Docker, Quarto, services en ligne comme GitHub) sont clairement signalées « **non exécuté** » : nous n'y affichons que la commande, jamais une sortie inventée.

## 6.1 Git et gestion de versions

**Intuition.** Imaginez que vous puissiez, à tout moment, prendre une **photographie complète** de votre dossier de travail, lui donner un titre (« ajout du calcul par canal de vente »), et la ranger dans un album. Plus tard, vous pouvez feuilleter l'album, comparer deux photos, revenir à celle d'il y a un mois, ou même ouvrir **deux albums parallèles** pour tester une idée folle sans toucher à la version qui marche. **Git** est exactement cet album, avec trois cadeaux en plus : il est gratuit, il fonctionne hors ligne, et il permet à plusieurs personnes de travailler sur le même dossier sans se marcher dessus.

### 6.1.1 Pourquoi pas simplement « analyse\_FINAL\_vrai.py » ?

Copier ses fichiers avec un suffixe (`_v2`, `_FINAL`) semble naturel, mais les défauts apparaissent vite :

Problème	Avec des copies de fichiers	Avec Git
« Qu'est-ce qui a changé entre hier et aujourd'hui ? »	Ouvrir deux fichiers et comparer à l'œil	<code>git diff</code> liste chaque ligne modifiée
« Pourquoi ai-je changé ça ? »	Personne ne s'en souvient	Chaque version porte un <b>message</b> daté et signé
« Je veux revenir à la version de mars »	Si elle existe encore et si son nom est clair	Une commande
« Je veux tester une idée sans risque »	Dupliquer tout le dossier	Une <b>branche</b> , en une seconde
« Amine et moi avons modifié le même fichier »	L'un des deux écrase l'autre, ou on recopie à la main	Git <b>fusionne</b> et signale seulement les vrais conflits
« Quel code a produit ce graphique ? »	Aucune idée	On retrouve la version exacte

Git a été créé en 2005 par Linus Torvalds pour développer le noyau Linux ; il est aujourd'hui le standard de fait, bien au-delà de la programmation : vous l'utiliserez pour du code, des notebooks, des rapports en texte, des fichiers de configuration. (Ce livre lui-même est écrit dans des fichiers texte suivis avec Git.)

**Git et GitHub, ce n'est pas pareil.** *Git* est le logiciel qui tourne **sur votre machine** et garde l'historique. *GitHub*, *GitLab* ou *Bitbucket* sont des **sites web** qui hébergent des copies de dépôts Git pour les partager. On peut utiliser Git des années sans jamais toucher à GitHub.

### 6.1.2 Premiers réglages

Git vérifie que vous êtes bien installé, puis il faut lui dire **qui vous êtes** : chaque version enregistrée est signée.

```
git --version
git config --global user.name "Yasmine de Dar Jasmin"
git config --global user.email "yasmine@dar-jasmin.example"
git config --global init.defaultBranch main
git config --global --list
```

```
git version 2.48.1
user.name=Yasmine de Dar Jasmin
user.email=yasmine@dar-jasmin.example
init.defaultbranch=main
```

Les trois réglages sont enregistrés une fois pour toutes dans le fichier `.gitconfig` de votre dossier personnel. La branche par défaut s'appellera `main` (l'ancien nom était `master` ; beaucoup de dépôts anciens l'utilisent encore).

**Une astuce de reproductibilité (à ne pas reproduire chez vous).** Chaque version Git est identifiée par une empreinte qui dépend, entre autres, **de la date**. Pour que le livre affiche toujours les mêmes empreintes à chaque exécution, les exemples de cette section fixent artificiellement la date de toutes les versions (variables `GIT_AUTHOR_DATE` et `GIT_COMMITTER_DATE`). Chez vous, ne fixez rien : Git utilise l'heure de l'horloge, et vos empreintes seront donc différentes des nôtres. Pas de souci : aucun raisonnement ne dépend de leur valeur.

```
export GIT_AUTHOR_DATE="2026-03-02T10:00:00+01:00"
export GIT_COMMITTER_DATE="2026-03-02T10:00:00+01:00"
echo "dates fixées pour la démonstration"
```

```
dates fixées pour la démonstration
```

### 6.1.3 Le premier dépôt : `init`, `add`, `commit`

Un **dépôt** (*repository*, ou *repo*) est un dossier que Git surveille. On l'initialise avec `git init`. Écrivons d'abord le tout premier script d'analyse de Yasmine : il calcule le montant moyen des commandes.

```
cd ~/atelier/dar-jasmin
git init -q
cat > analyse.py <<'FIN'
import pandas as pd

df = pd.read_csv("commandes.csv")
print("Nombre de commandes :", len(df))
print("Montant moyen :", round(df["montant"].mean(), 2), "DT")
FIN
python analyse.py
```

```
Nombre de commandes : 400
Montant moyen : 60.25 DT
```

(`cat > analyse.py <<'FIN' ...` `FIN` est une façon de créer un fichier directement depuis le terminal : tout ce qui se trouve entre les deux `FIN` est écrit dans `analyse.py`. En pratique, vous utiliserez plutôt votre éditeur de code.) Le script fonctionne, avec le fil rouge habituel : 60,25 DT de montant moyen. Que pense Git de l'état du dossier ?

```
git status
```

```
On branch main
```

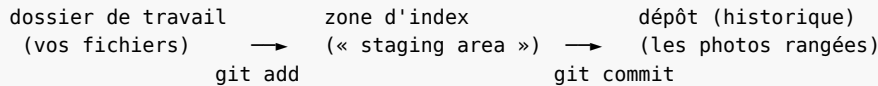
```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
analyse.py
commandes.csv
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Git voit deux fichiers **non suivis** (*untracked*) : il les a remarqués mais ne les surveille pas encore. Pour comprendre la suite, il faut connaître les **trois zones** de Git :



- Le **dossier de travail** contient vos fichiers, tels que vous les éditez.
- La **zone d'index** (*staging area*) est la table où l'on **prépare la prochaine photo** : on y dépose seulement ce qu'on veut y voir.
- Le **dépôt** contient les photos prises, à jamais (les *commits*).

Pourquoi cette zone intermédiaire ? Parce qu'on a souvent modifié trois fichiers pour trois raisons différentes ; on peut ainsi faire **trois commits distincts et propres** plutôt qu'un fourre-tout. Ajoutons le script, puis prenons la photo.

```
git add analyse.py
git status --short
git commit -q -m "Premier script : montant moyen des commandes"
git log --oneline
```

```
A analyse.py
?? commandes.csv
aa98f06 Premier script : montant moyen des commandes
```

`git status --short` affiche une lettre par fichier : A (ajouté à l'index) pour `analyse.py`, ?? (non suivi) pour `commandes.csv`. Après le commit, `git log --oneline` montre une ligne : l'empreinte courte de la version, suivie de son message.

**Écrire de bons messages.** Un message de commit répond à la question « **que fait** ce changement, et pourquoi ? », dans un langage que vous comprendrez dans six mois. « maj » ou « modifs » sont inutiles ; « Ajout du montant moyen par canal de vente » est excellent. Convention courante : une première ligne courte ( $\approx 50$  caractères), à l'impératif ou au passé de façon cohérente.

On a laissé `commandes.csv` de côté volontairement. Faut-il suivre les données dans Git ? Pour un petit fichier fixe comme celui-ci, oui, cela garantit que l'analyse reste rejouable avec **exactement** les mêmes données. Pour de gros fichiers, non (voir 6.1.11). Ici, on l'ajoute :

```
git add commandes.csv
git commit -q -m "Ajout du fichier de données (400 commandes)"
git log --oneline
```

```
09e8168 Ajout du fichier de données (400 commandes)
aa98f06 Premier script : montant moyen des commandes
```

### 6.1.4 Ce qu'il y a vraiment dans un commit

**Rigueur : le modèle d'objets de Git.** Un *commit* n'est pas une liste de différences : c'est une **photographie complète** du dossier, accompagnée de métadonnées et d'un **pointeur vers le commit précédent** (son *parent*). Chaque objet est stocké sous un nom qui est l'**empreinte de son contenu** (une fonction de hachage SHA-1, qui transforme n'importe quel contenu en un nombre de 40 chiffres hexadécimaux). Il y a trois sortes d'objets :

- un **blob** : le contenu d'un fichier, sans son nom ;
- un **arbre** (*tree*) : la liste des fichiers d'un dossier, avec pour chacun le nom et l'empreinte de son blob ;
- un **commit** : l'empreinte d'un arbre, l'empreinte du ou des parents, l'auteur, la date, le

message.

Conséquence cruciale : si on change **un seul caractère** dans un fichier, l'empreinte du blob change, donc celle de l'arbre, donc celle du commit, et de tous ses descendants. Un historique Git ne peut donc pas être modifié en douce : c'est ce qui le rend digne de confiance. Autre conséquence : deux fichiers de contenu identique partagent le même blob, et Git ne les stocke qu'une fois.

C'est vérifiable à la main ! L'empreinte d'un blob est simplement le SHA-1 de la chaîne blob, suivie de la taille du contenu, d'un caractère nul, puis du contenu. Calculons-la de deux façons : avec Git, et avec l'outil `shasum`, sans Git :

```
echo "Dar Jasmin" > bonjour.txt
git hash-object bonjour.txt
printf 'blob 11\0Dar Jasmin\n' | shasum
```

```
6f7ea4d27e6759345d14bb395b92f846451619de
6f7ea4d27e6759345d14bb395b92f846451619de -
```

Les deux empreintes sont **identiques** : Git ne fait rien de magique, il hache le contenu (11 octets : les 10 caractères de « Dar Jasmin » plus le saut de ligne). Voyons maintenant le dernier commit « de l'intérieur » :

```
git cat-file -p HEAD
echo "----"
git cat-file -p 'HEAD^{tree}'
rm bonjour.txt
```

```
tree 9def47f7022f460faed35f54531145c9e6c3373d
parent aa98f06b76c0b3fd78b10dbb79a071afd2c878b4
author Yasmine de Dar Jasmin <yasmine@dar-jasmin.example> 1772442000 +0100
committer Yasmine de Dar Jasmin <yasmine@dar-jasmin.example> 1772442000 +0100
```

Ajout du fichier de données (400 commandes)

```

100644 blob 7141581ddb6f60852e4f5e00be8e4b8aa412c6a73 analyse.py
100644 blob 39d5bed793efe2ff7b963d7ff459bf3a8023daa1 commandes.csv
```

On lit d'abord le commit : l'empreinte de l'arbre (`tree`), celle du parent (`parent`), l'auteur, et le message. Puis l'arbre : deux lignes, une par fichier, chacune avec le mode, le type blob, l'empreinte et le nom. Le mot `HEAD` désigne « le commit où je suis en ce moment » ; `HEAD^` est son parent.

### 6.1.5 Modifier, comparer, enregistrer : le cycle de travail

Le quotidien avec Git est une boucle de quatre gestes : **modifier** des fichiers, **regarder** ce qui a changé (`git status`, `git diff`), **choisir** ce qui part dans la prochaine photo (`git add`), **enregistrer** (`git commit`). Yasmine veut maintenant le montant moyen **par canal de vente**.

```
cat >> analyse.py <<'FIN'
print()
print("Montant moyen par canal :")
print(df.groupby("canal")["montant"].mean().round(2))
FIN
git status --short
git diff
```

```
M analyse.py
diff --git a/analyse.py b/analyse.py
index 7141581..feccec9 100644
--- a/analyse.py
```

```
+++ b/analyse.py
@@ -3,3 +3,6 @@ import pandas as pd
df = pd.read_csv("commandes.csv")
print("Nombre de commandes :", len(df))
print("Montant moyen :", round(df["montant"].mean(), 2), "DT")
+print()
+print("Montant moyen par canal :")
+print(df.groupby("canal")["montant"].mean().round(2))
```

`git diff` montre les différences entre le dossier de travail et la dernière photo : les lignes précédées de + sont ajoutées, celles précédées de - seraient supprimées. Le bloc @@ ... @@ situe la modification dans le fichier. Lançons le script, puis enregistrons :

```
python analyse.py
git add analyse.py
git commit -q -m "Ajout du montant moyen par canal"
git log --format='%h %s'
```

```
Nombre de commandes : 400
Montant moyen : 60.25 DT

Montant moyen par canal :
canal
Boutique 74.81
Instagram 49.01
Site 59.50
Name: montant, dtype: float64
11985b5 Ajout du montant moyen par canal
09e8168 Ajout du fichier de données (400 commandes)
aa98f06 Premier script : montant moyen des commandes
```

**Trois variantes utiles de `git diff`.** `git diff` compare le dossier de travail à l'index (« ce que je n'ai pas encore préparé ») ; `git diff --staged` compare l'index au dernier commit (« ce qui partira au prochain commit ») ; `git diff HEAD~1 HEAD` compare deux commits (ici, le précédent et le dernier).

Pour relire l'historique avec le détail des changements :

```
git log --stat
git show --stat HEAD~1
```

```
commit 11985b58f7a7c55501f7f918b708d0baa4f29019
Author: Yasmine de Dar Jasmin <yasmine@dar-jasmin.example>
Date: Mon Mar 2 10:00:00 2026 +0100

 Ajout du montant moyen par canal

analyse.py | 3 +++
1 file changed, 3 insertions(+)

commit 09e8168a199194eccfc0277b890b4b17948587ff
Author: Yasmine de Dar Jasmin <yasmine@dar-jasmin.example>
Date: Mon Mar 2 10:00:00 2026 +0100

 Ajout du fichier de données (400 commandes)

commandes.csv | 401 +++
1 file changed, 401 insertions(+)

commit aa98f06b76c0b3fd78b10dbb79a071afd2c878b4
Author: Yasmine de Dar Jasmin <yasmine@dar-jasmin.example>
```

```
Date: Mon Mar 2 10:00:00 2026 +0100

Premier script : montant moyen des commandes

analyse.py | 5 +++++
1 file changed, 5 insertions(+)
commit 09e8168a199194eccfc0277b890b4b17948587ff
Author: Yasmine de Dar Jasmin <yasmine@dar-jasmin.example>
Date: Mon Mar 2 10:00:00 2026 +0100

Ajout du fichier de données (400 commandes)

commandes.csv | 401 +++
1 file changed, 401 insertions(+)
```

--stat résume chaque commit par fichiers touchés et nombre de lignes ajoutées (+) ou supprimées (-). HEAD~1 désigne « un commit avant HEAD », HEAD~2 deux avant, etc.

### 6.1.6 Défaire une erreur

C'est la plus grande qualité de Git : **presque tout se rattrape**, à condition de choisir la bonne commande. Trois situations classiques, de la plus bénigne à la plus lourde.

**Situation 1 : « J'ai cassé un fichier, je veux retrouver la version du dernier commit. »** Yasmine, fatiguée, efface par erreur tout le contenu de son script :

```
wc -c analyse.py
> analyse.py
wc -c analyse.py
python analyse.py
git restore analyse.py
wc -c analyse.py
python analyse.py | head -n 2
```

```
255 analyse.py
0 analyse.py
255 analyse.py
Nombre de commandes : 400
Montant moyen : 60.25 DT
```

wc -c compte les octets du fichier. La ligne > analyse.py (une redirection vers un fichier, sans commande devant : voir 6.3) **vide** le fichier : 0 octet, et le script n'affiche plus rien du tout. Puis git restore analyse.py ramène le fichier à l'état du dernier commit : la taille d'origine revient, et le script affiche de nouveau ses résultats.

**git restore est irréversible pour les modifications non enregistrées.** Ce que Git n'a jamais photographié, il ne peut pas le retrouver. D'où la règle d'or : **faites des commits petits et fréquents.**

**Situation 2 : « J'ai enregistré une erreur dans un commit, mais je l'ai déjà partagé ou je veux garder une trace. »** On annule par un nouveau commit qui fait l'inverse : c'est git revert. L'historique reste intact, il s'enrichit d'une ligne « annulation ». Yasmine ajoute une ligne erronée, la committe, puis la retire proprement :

```
echo 'print("TODO supprimer cette ligne")' >> analyse.py
git commit -q -am "Ligne de debug oubliée"
git revert --no-edit HEAD
git log --format='%h %s'
python analyse.py | tail -n 1
```

```
[main 23f717b] Revert "Ligne de debug oubliée"
Date: Mon Mar 2 10:00:00 2026 +0100
1 file changed, 1 deletion(-)
23f717b Revert "Ligne de debug oubliée"
57b0242 Ligne de debug oubliée
11985b5 Ajout du montant moyen par canal
09e8168 Ajout du fichier de données (400 commandes)
aa98f06 Premier script : montant moyen des commandes
Name: montant, dtype: float64
```

(L'option `-a` de `commit` ajoute automatiquement tous les fichiers **déjà suivis** et modifiés ; pratique, mais elle ne règle pas la question « qu'ai-je vraiment modifié ? » : on vérifie avec `git status` avant.)

L'historique contient maintenant les deux lignes (l'erreur *et* son annulation), et le script est redevenu propre : sa dernière ligne de sortie est de nouveau la fin du tableau par canal, sans notre message de debug.

**Situation 3 : « Je veux revoir l'état du dossier à une date passée. »** On se déplace temporairement dans l'historique avec `git switch --detach` (le dossier prend l'apparence du commit demandé, sans rien détruire), puis on revient :

```
git switch --detach HEAD~3 2>&1 | head -n 1
cat analyse.py
git switch main
```

```
HEAD is now at 09e8168 Ajout du fichier de données (400 commandes)
import pandas as pd

df = pd.read_csv("commandes.csv")
print("Nombre de commandes :", len(df))
print("Montant moyen :", round(df["montant"].mean(), 2), "DT")
Previous HEAD position was 09e8168 Ajout du fichier de données (400 commandes)
Switched to branch 'main'
```

On y voit le script tel qu'il était à ce moment-là, **sans** le calcul par canal. `git switch main` nous ramène au présent. Pendant qu'on est dans le passé, on peut regarder, lancer le code, mais on ne doit pas y travailler : Git met d'ailleurs en garde (état *detached HEAD*).

**Les commandes dangereuses.** `git reset --hard` et `git clean -fd` détruisent les modifications non enregistrées sans demander confirmation. Et réécrire un historique déjà partagé (`git push --force`) peut effacer le travail des autres. En cas de doute, créez une branche de secours (`git branch sauvegarde`) avant d'expérimenter : cela ne coûte rien.

### 6.1.7 Ce qu'il ne faut pas suivre : `.gitignore`

Certains fichiers n'ont rien à faire dans l'historique : les fichiers temporaires de Python (`__pycache__`), l'environnement virtuel (`.venv`, section 6.3), les points de contrôle des notebooks, les mots de passe, les données volumineuses ou confidentielles. On les déclare dans un fichier texte nommé `.gitignore`.

```
mkdir -p .venv __pycache__
touch .venv/pyvenv.cfg __pycache__/analyse.cpython-313.pyc secrets.env
git status --short
```

```
?? .venv/
?? __pycache__/
?? secrets.env
```



Git voit trois choses nouvelles qui polluent la vue : `.venv/`, `__pycache__` et `secrets.env`. (Git ne s'intéresse qu'aux fichiers, jamais aux dossiers vides : c'est pourquoi nous avons mis un fichier dans `.venv/`.) Ajoutons un fichier `.gitignore` :

```
cat > .gitignore <<'FIN'
environnement et fichiers temporaires
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/
secrets : JAMAIS dans Git
*.env
FIN
git status --short
git add .gitignore
git commit -q -m "Ajout du .gitignore"
git check-ignore -v secrets.env
```

```
?? .gitignore
.gitignore:7:*.env secrets.env
```

Il ne reste que `.gitignore` lui-même (qu'on **veut** suivre, pour que toute l'équipe ignore les mêmes choses). La dernière commande, `git check-ignore -v`, explique pourquoi un fichier est ignoré : elle cite la règle et la ligne responsable.

**Un secret commité est un secret perdu.** Si vous enregistrez par erreur un mot de passe ou une clé d'accès, le retirer dans un commit suivant ne suffit pas : il reste lisible dans l'historique. Considérez-le comme compromis et **changez-le**. Prévenez l'erreur en mettant `.gitignore` en place **avant** de créer le fichier secret.

### 6.1.8 Les branches : tester une idée sans risque

Une **branche** est une ligne de travail parallèle. Techniquement, c'est trivial : un simple **pointeur** (une étiquette) vers un commit, qui avance à chaque nouveau commit. La branche par défaut s'appelle `main`. Créer une branche ne copie rien : cela prend une milliseconde, quelle que soit la taille du dépôt.

Yasmine se demande à partir de quel montant offrir la livraison. Elle ouvre une branche dédiée à cette expérience, plutôt que de modifier le script principal :

```
git branch
git switch -c seuil-livraison
git branch
```

```
* main
Switched to a new branch 'seuil-livraison'
main
* seuil-livraison
```

L'étoile `*` marque la branche sur laquelle on se trouve. Travaillons sur la branche : on ajoute un seuil de livraison gratuite et on calcule la part de commandes concernées.

```
cat >> analyse.py <<'FIN'

SEUIL = 80 # seuil de livraison gratuite (DT)
part = (df["montant"] >= SEUIL).mean()
print(f"Part des commandes dès {SEUIL} DT : {part:.1%}")
FIN
python analyse.py | tail -n 2
git commit -q -am "Seuil de livraison gratuite à 80 DT"
```

```
git switch main
tail -n 3 analyse.py
```

```
Name: montant, dtype: float64
Part des commandes dès 80 DT : 21.8%
Switched to branch 'main'
print()
print("Montant moyen par canal :")
print(df.groupby("canal")["montant"].mean().round(2))
```

Retour sur main : les lignes du seuil **ont disparu** du fichier, car elles n'existent que sur la branche seuil-livraison. Rien n'est perdu : Git remplace simplement le contenu du dossier par celui du commit pointé par main. En repassant sur la branche, tout revient.

### Fusionner : le cas facile (avance rapide)

Yasmine est contente de l'expérience. Elle veut la reporter dans main : c'est une **fusion** (*merge*). Comme main n'a pas bougé depuis la création de la branche, Git n'a rien à réconcilier : il se contente d'**avancer le pointeur** de main (*fast-forward*).

```
git merge seuil-livraison
git log --oneline --graph
git branch -d seuil-livraison
```

```
Updating ef0e5ce..164d45e
Fast-forward
 analyse.py | 4 ++++
 1 file changed, 4 insertions(+)
* 164d45e Seuil de livraison gratuite à 80 DT
* ef0e5ce Ajout du .gitignore
* 23f717b Revert "Ligne de debug oubliée"
* 57b0242 Ligne de debug oubliée
* 11985b5 Ajout du montant moyen par canal
* 09e8168 Ajout du fichier de données (400 commandes)
* aa98f06 Premier script : montant moyen des commandes
Deleted branch seuil-livraison (was 164d45e).
```

### Fusionner : le cas où les deux ont bougé

Le cas intéressant arrive quand **deux personnes** ont modifié le dépôt en parallèle. Amine, de son côté, veut tester un seuil plus bas (60 DT) ; Yasmine, en parallèle, décide d'ajouter une moyenne des notes de satisfaction. On simule les deux lignes de travail :

```
Amine : une branche pour un autre seuil
git switch -c seuil-amine
sed -i 's/^SEUIL = 80.*/SEUIL = 60 # seuil proposé par Amine (DT)/' analyse.py
git commit -q -am "Seuil à 60 DT (proposition d'Amine)"

Yasmine revient sur main et ajoute la satisfaction
git switch main
cat >> analyse.py <<'FIN'

print("Satisfaction moyenne :", round(df["satisfaction"].mean(), 2))
FIN
git commit -q -am "Ajout de la satisfaction moyenne"

git log --oneline --graph --all
```

```
Switched to a new branch 'seuil-amine'
Switched to branch 'main'
```

```
* 0f93f5f Ajout de la satisfaction moyenne
| * 5e60494 Seuil à 60 DT (proposition d'Amine)
|/
* 164d45e Seuil de livraison gratuite à 80 DT
* ef0e5ce Ajout du .gitignore
* 23f717b Revert "Ligne de debug oubliée"
* 57b0242 Ligne de debug oubliée
* 11985b5 Ajout du montant moyen par canal
* 09e8168 Ajout du fichier de données (400 commandes)
* aa98f06 Premier script : montant moyen des commandes
```

Le graphe a maintenant la forme d'un « Y » : les deux branches sont parties du même commit et ont chacune avancé de leur côté. Fusionnons celle d'Amine dans main :

```
git merge --no-edit seuil-amine
git log --oneline --graph
```

```
Auto-merging analyse.py
Merge made by the 'ort' strategy.
analyse.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
* 7fbd1eb Merge branch 'seuil-amine'
|\
| * 5e60494 Seuil à 60 DT (proposition d'Amine)
* | 0f93f5f Ajout de la satisfaction moyenne
|/
* 164d45e Seuil de livraison gratuite à 80 DT
* ef0e5ce Ajout du .gitignore
* 23f717b Revert "Ligne de debug oubliée"
* 57b0242 Ligne de debug oubliée
* 11985b5 Ajout du montant moyen par canal
* 09e8168 Ajout du fichier de données (400 commandes)
* aa98f06 Premier script : montant moyen des commandes
```

Git a réussi **automatiquement** : les deux changements portaient sur des zones différentes du fichier (une ligne du seuil d'un côté, des lignes ajoutées à la fin de l'autre). Il a créé un **commit de fusion** à deux parents, ce qui apparaît sur le graphe. Python confirme que le script fonctionne et applique bien les deux modifications :

```
python analyse.py | tail -n 3
```

```
Name: montant, dtype: float64
Part des commandes dès 60 DT : 41.0%
Satisfaction moyenne : 3.96
```

### Quand Git ne peut pas décider : un conflit

Un **conflit** survient quand deux branches modifient **la même ligne** de façons différentes : Git ne sait pas laquelle choisir, et vous demande de trancher. Ce n'est pas une catastrophe, c'est le fonctionnement normal du travail à plusieurs. Provoquons-en un : Yasmine veut un seuil de 70 DT, Amine veut 65 DT.

```
git switch -c seuil-yasmine
sed -i 's/^SEUIL = .*/SEUIL = 70 # seuil retenu par Yasmine (DT)/' analyse.py
git commit -q -am "Seuil à 70 DT"

git switch main
sed -i 's/^SEUIL = .*/SEUIL = 65 # compromis d'""Amine (DT)/' analyse.py
git commit -q -am "Seuil à 65 DT"

git merge --no-edit seuil-yasmine
```

```
Switched to a new branch 'seuil-yasmine'
Switched to branch 'main'
Auto-merging analyse.py
CONFLICT (content): Merge conflict in analyse.py
Automatic merge failed; fix conflicts and then commit the result.
```

Git s'arrête et prévient : `CONFLICT (content)`. Le dépôt est en cours de fusion ; consultons l'état, puis le fichier :

```
git status --short
grep -n -A6 '<<<<<<' analyse.py
```

```
UU analyse.py
10:<<<<<< HEAD
11-SEUIL = 65 # compromis d'Amine (DT)
12-=====
13-SEUIL = 70 # seuil retenu par Yasmine (DT)
14->>>>>> seuil-yasmine
15-part = (df["montant"] >= SEUIL).mean()
16-print(f"Part des commandes dès {SEUIL} DT : {part:.1%}")
```

Git a écrit dans le fichier, autour de la ligne litigieuse, trois **marqueurs** :

```
<<<<<< HEAD
...version de la branche où l'on se trouve (main : 65 DT)...
=====
...version de la branche qu'on fusionne (seuil-yasmine : 70 DT)...
>>>>>> seuil-yasmine
```

Résoudre le conflit, c'est **éditer le fichier à la main** pour ne garder que ce qu'on veut (ici, après discussion, 70 DT), supprimer les marqueurs, puis déclarer le conflit réglé avec `git add` et conclure par un `commit`.

```
sed -i '/^<<<<<< /d; /^=====$/d; /^>>>>>> /d; /^SEUIL = 65/d' analyse.py
grep -n '^SEUIL' analyse.py
python analyse.py | tail -n 3
git add analyse.py
git commit -q --no-edit
git log --oneline --graph | head -n 8
```

```
10:SEUIL = 70 # seuil retenu par Yasmine (DT)
Name: montant, dtype: float64
Part des commandes dès 70 DT : 29.2%
Satisfaction moyenne : 3.96
* 5b8831d Merge branch 'seuil-yasmine'
|\
| * dd1b0a7 Seuil à 70 DT
* | 8bcaa34 Seuil à 65 DT
|/
* 7fbd1eb Merge branch 'seuil-amine'
|\
| * 5e60494 Seuil à 60 DT (proposition d'Amine)
```

(Le long `sed` enlève les trois lignes de marqueurs et la ligne de la version d'Amine. Avec un éditeur comme VS Code, vous cliquez simplement sur « Accepter la modification actuelle / entrante » ; le principe est identique.) Avant de valider, **toujours relancer le code** : une fusion peut réussir sans conflit et pourtant produire un programme qui ne marche plus.

**Réduire les conflits.** Les conflits sont d'autant plus rares que (1) les branches vivent peu de temps, (2) on rapatrie souvent `main` dans sa branche, (3) on évite de reformater un fichier

entier « pour faire joli » pendant que d'autres y travaillent, (4) on se répartit les fichiers. Un conflit est surtout un signal de **communication** à rattraper.

Nettoyons les branches devenues inutiles :

```
git branch -d seuil-amine seuil-yasmine
git branch
```

```
Deleted branch seuil-amine (was 5e60494).
Deleted branch seuil-yasmine (was dd1b0a7).
* main
```

6.1.9 Étiqueter une version : les tags

Quand une version a une signification particulière (« le rapport remis à la banque »), on lui pose une **étiquette** (*tag*) qui ne bouge plus, contrairement à une branche. On pourra toujours retrouver exactement le code de ce jour-là.

```
git tag -a v1.0-rapport-banque -m "Version remise à la banque (mars 2026)"
git tag
git show --stat v1.0-rapport-banque | head -n 6
```

```
v1.0-rapport-banque
tag v1.0-rapport-banque
Tagger: Yasmine de Dar Jasmin <yasmine@dar-jasmin.example>
Date: Mon Mar 2 10:00:00 2026 +0100

Version remise à la banque (mars 2026)
```

C'est la réponse à la question qui ouvrirait ce chapitre : « quel code a produit le graphique envoyé à la banque ? » Il suffit de noter le *tag* dans le rapport.

6.1.10 Dépôts distants : partager et sauvegarder

Jusqu'ici tout se passe sur une seule machine. Pour collaborer, ou simplement se **sauvegarder**, on utilise un **dépôt distant** (*remote*) : un autre dépôt Git, hébergé sur un serveur ou un site comme GitHub. Quatre commandes suffisent à comprendre l'essentiel :

Commande	Effet
git clone <adresse>	copie complète d'un dépôt distant (historique compris) sur votre machine
git push	envoie vos nouveaux commits vers le dépôt distant
git fetch	récupère les nouveaux commits du distant <b>sans</b> toucher à vos fichiers
git pull	fetch puis fusion dans votre branche courante

Un dépôt distant n'a rien de mystérieux : c'est un dépôt Git comme les autres, souvent « nu » (*bare*, sans dossier de travail, car personne n'y édite directement). Nous pouvons donc le **simuler sans Internet** avec un second dossier de notre machine ! Voici le scénario complet : Yasmine envoie son travail ; Amine le clone, ajoute une ligne et l'envoie à son tour ; Yasmine récupère.

```
cd ~/atelier
git init -q --bare depot-partage.git
cd dar-jasmin
git remote add origin ~/atelier/depot-partage.git
```

```
git remote -v | sed "s|$HOME|~|"
git push -q -u origin main --tags 2>&1 | tail -n 3
cd ..
git clone -q depot-partage.git copie-amine
cd copie-amine
git config user.name "Amine"
git config user.email "amine@dar-jasmin.example"
git log --oneline | wc -l
```

```
origin ~/atelier/depot-partage.git (fetch)
origin ~/atelier/depot-partage.git (push)
13
```

(Le `sed` ne sert qu'à abrégier le chemin de l'atelier en `~` pour l'affichage.) Après le `push`, le dépôt partagé contient tout l'historique de Yasmine ; le `clone` d'Amine en reçoit une copie intégrale (la dernière commande compte les commits reçus : les 13 de l'historique, fusions comprises ; les deux lignes `git config` donnent à ce clone sa propre identité, Amine travaillant sur une autre machine). Amine travaille :

```
echo 'print("Commandes Instagram :", (df["canal"] == "Instagram").sum())' >> analyse.py
git commit -q -am "Ajout du nombre de commandes Instagram"
git push -q origin main 2>&1 | tail -n 3
cd ../dar-jasmin
git pull -q origin main 2>&1 | tail -n 3
git log --format='%an : %s' | head -n 3
python analyse.py | tail -n 1
```

```
Amine : Ajout du nombre de commandes Instagram
Yasmine de Dar Jasmin : Merge branch 'seuil-yasmine'
Yasmine de Dar Jasmin : Seuil à 65 DT
Commandes Instagram : 138
```

Yasmine a récupéré la modification d'Amine : chaque commit porte le nom de son auteur, et le script affiche bien la nouvelle ligne.

**Et avec GitHub, concrètement ?** Non exécuté ici (cela demande un compte et une connexion). Le principe est le même qu'au-dessus, avec une adresse de la forme `https://github.com/nom/depot` ou `git@github.com:nom/depot.git`. Le flux de travail standard en équipe est : (1) créer une **branche** pour chaque tâche ; (2) la pousser sur le site ; (3) ouvrir une **pull request** (*demande de fusion*) : une page où les collègues relisent les changements, commentent, puis acceptent la fusion dans `main` ; (4) supprimer la branche. Le *fork* est une copie personnelle d'un dépôt qui ne vous appartient pas, pour y proposer des modifications.

### 6.1.11 Bonnes pratiques pour la data science

1. **Un commit = une idée.** Corriger un bug et ajouter une analyse, ce sont deux commits.
2. **Ne versionnez pas les gros fichiers de données ni les secrets.** Pour des jeux de données volumineux, on les place ailleurs et on versionne un petit fichier décrivant leur origine, ou l'on utilise un outil dédié (*Git LFS*, *DVC*).
3. **Versionnez le code qui produit les résultats, pas les résultats.** Un graphique se régénère ; un script ne se reconstitue pas.
4. **Commitez souvent, poussez régulièrement.** Un disque dur peut mourir ; un dépôt distant est une sauvegarde.
5. **Ajoutez un fichier README :** à quoi sert le projet, comment l'exécuter (6.3 et 6.5).
6. **Posez des tags** sur les versions livrées.

Le mini-aide-mémoire des commandes de la section :

Je veux...	Commande
démarrer un dépôt	<code>git init</code>
voir l'état du dossier	<code>git status</code>
préparer / enregistrer	<code>git add fichier puis git commit -m "message"</code>
voir ce qui a changé	<code>git diff, git diff --staged</code>
lire l'historique	<code>git log --oneline --graph</code>
jeter mes modifications d'un fichier	<code>git restore fichier</code>
annuler un commit proprement	<code>git revert &lt;commit&gt;</code>
créer / changer de branche	<code>git switch -c nom, git switch nom</code>
fusionner	<code>git merge nom</code>
poser une étiquette	<code>git tag -a nom -m "message"</code>
copier / envoyer / récupérer	<code>git clone, git push, git pull</code>

### À retenir

- Git garde un **album de photographies complètes** de votre dossier ; chaque photo (*commit*) porte un message, un auteur, une date et un pointeur vers sa parente.
- Trois zones : dossier de travail → `git add` → index → `git commit` → dépôt.
- Presque tout se rattrape (`restore`, `revert`, `switch --detach`) **si l'on a enregistré**. Faites des commits petits et fréquents.
- Une **branche** est un pointeur : en créer coûte rien ; on y teste une idée avant de **fusionner**.
- Un **conflit** n'apparaît que lorsque deux changements touchent la même ligne ; on édite, on `add`, on `commit`, et on **relance le code**.
- `.gitignore` pour les fichiers temporaires et secrets ; les secrets commités sont perdus.
- Un **dépôt distant** (GitHub...) est un dépôt Git ordinaire qui sert à partager et à sauvegarder.

## 6.2 Notebooks Jupyter

**Intuition.** Un **notebook** (« carnet ») est un document où l'on mélange, dans l'ordre où l'on réfléchit : du **texte** (« voici la question »), du **code** (« voici le calcul ») et le **résultat** de ce code (tableau, graphique), juste en dessous. C'est le cahier de laboratoire du data scientist : on explore, on essaie, on commente, et le tout se lit comme un récit. Le projet s'appelle **Jupyter** (de *Julia*, *Python*, *R*, les trois langages d'origine) ; ses fichiers portent l'extension `.ipynb`.

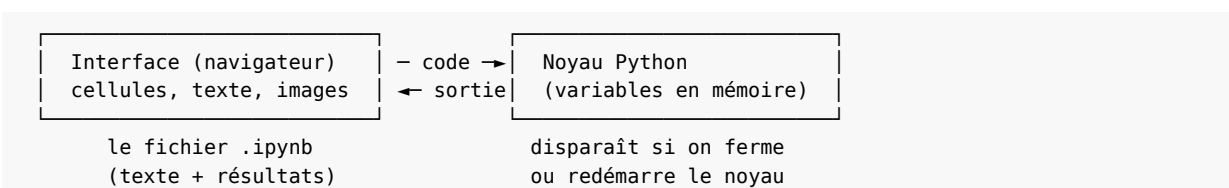
Dans un notebook, vous tapez un calcul, appuyez sur Maj + Entrée, et le résultat s'affiche à l'instant. Pas de cycle « écrire le fichier, l'enregistrer, lancer le programme, regarder l'écran » : c'est idéal pour **explorer** des données, **enseigner**, ou **raconter** une analyse. Mais cet outil a un défaut sournois, et toute la seconde moitié de cette section lui est consacrée : **un notebook mal tenu produit des résultats qu'on ne peut pas reproduire**.

### 6.2.1 Les ingrédients : cellules et noyau

Un notebook est une suite de **cellules** de deux sortes principales :

Type de cellule	Contenu	Résultat de l'exécution
<b>Code</b>	du Python (ou un autre langage)	la sortie du code : texte, tableau, graphique, erreur
<b>Markdown</b>	du texte mis en forme, titres, listes, formules $\dots$	du texte formaté (rien n'est « calculé »)

Derrière l'interface se cache un second ingrédient, essentiel pour tout comprendre : le **noyau** (*kernel*). C'est un **programme Python qui tourne en arrière-plan**, avec sa **mémoire**. Quand vous exécutez une cellule, son code est envoyé au noyau, qui l'exécute et renvoie le résultat. Les variables créées par une cellule **restent en mémoire dans le noyau** et sont donc visibles par toutes les cellules exécutées ensuite.



Retenez bien cette image : **le fichier .ipynb** et **la mémoire** du noyau sont deux choses distinctes. Le fichier garde le texte, le code et les derniers résultats affichés ; la mémoire, elle, contient l'état *actuel* des variables, qui peut très bien ne plus correspondre à ce que le code écrit dans le fichier produirait.

Pour installer et lancer l'interface (*non exécuté ici : l'interface s'ouvre dans un navigateur et ne peut pas être reproduite dans un livre*) :

```
bash noexec pip install jupyterlab # installe JupyterLab (l'interface moderne)
jupyter lab # lance le serveur et ouvre le navigateur
```

On peut aussi ouvrir des notebooks directement dans **VS Code** (extension Jupyter), ou en ligne, sans rien installer, dans **Google Colab** (service gratuit de Google ; non testé ici). Le format des fichiers est le même partout.

## 6.2.2 Un notebook est un fichier texte (JSON)

Il n'y a rien de magique dans un fichier .ipynb : c'est du **JSON**, un format texte qui décrit des données imbriquées (dictionnaires et listes), lisible par n'importe quel langage. Pour s'en convaincre, nous allons **fabriquer un notebook en Python** avec la bibliothèque nbformat, puis l'ouvrir comme un simple texte. Yasmine veut un petit carnet « Ventes de Dar Jasmin » en trois cellules : un titre, le chargement des données, un calcul par canal.

```
import json
import nbformat
from nbformat import v4 as nbf

nb = nbf.new_notebook()
nb.metadata["kernelspec"] = {"name": "python3", "display_name": "Python 3", "language": "python"}
nb.cells = [
 nbf.new_markdown_cell("# Ventes de Dar Jasmin\nMontant moyen des commandes, par canal.",
 id="titre"),
 nbf.new_code_cell("import pandas as pd\nndf = pd.read_csv('donnees/commandes.csv')\nndf.shape",
 id="chargement"),
 nbf.new_code_cell("df.groupby('canal')['montant'].mean().round(2)", id="par-canal"),
]
texte = nbformat.writes(nb)
print(texte[:1100])
```



```
{
 "cells": [
 {
 "cell_type": "markdown",
 "id": "titre",
 "metadata": {},
 "source": [
 "# Ventes de Dar Jasmin\n",
 "Montant moyen des commandes, par canal."
]
 },
 {
 "cell_type": "code",
 "execution_count": null,
 "id": "chargement",
 "metadata": {},
 "outputs": [],
 "source": [
 "import pandas as pd\n",
 "df = pd.read_csv('donnees/commandes.csv')\n",
 "df.shape"
]
 },
 {
 "cell_type": "code",
 "execution_count": null,
 "id": "par-canal",
 "metadata": {},
 "outputs": [],
 "source": [
 "df.groupby('canal')['montant'].mean().round(2)"
]
 }
],
 "metadata": {
 "kernel_spec": {
 "display_name": "Python 3",
 "language": "python",
 "name": "python3"
 }
 },
 "nbformat": 4,
 "nbformat_minor": 5
}
```

Voilà le notebook, vu comme du texte. On y reconnaît : une liste de `cells` ; pour chaque cellule, son `cell_type` (markdown ou code), sa source (le texte tapé), et, pour les cellules de code, deux champs : `execution_count` (le numéro d'exécution, vide tant que la cellule n'a pas tourné) et `outputs` (la liste des résultats affichés, vide pour l'instant). Les `metadata` décrivent le noyau à utiliser. Pas encore de résultats : le carnet n'a jamais été exécuté. Exécutons-le avec `nbclient`, la bibliothèque qui pilote un noyau depuis un programme (c'est ce que fait le bouton « Exécuter tout ») :

```
from nbclient import NotebookClient

NotebookClient(nb, timeout=120, kernel_name="python3", record_timing=False,
 resources={"metadata": {"path": "."}}).execute()

for cellule in nb.cells:
 if cellule.cell_type == "code":
 print(f"[{cellule.execution_count}] {cellule.source.splitlines()[-1]}")
 for sortie in cellule.outputs:
 print(" ->", sortie.output_type, ":", sortie.data["text/plain"].replace("\n", "\n↵"))
```

```
[1] df.shape
-> execute_result : (400, 4)
[2] df.groupby('canal')['montant'].mean().round(2)
-> execute_result : canal
 Boutique 74.81
 Instagram 49.01
 Site 59.50
 Name: montant, dtype: float64
```

Chaque cellule de code porte désormais son **numéro d'exécution** ([1], [2]) et ses **sorties**, enregistrées dans le fichier lui-même. Le texte de la sortie est stocké sous le type text/plain ; un graphique serait stocké sous image/png, encodé en texte (base64) : un notebook avec beaucoup de graphiques devient donc volumineux.

**Ce que cela implique, concrètement.** Comme les résultats sont *écrits dans le fichier*, vous pouvez ouvrir un notebook et voir des résultats **sans rien exécuter**. C'est pratique pour partager... et dangereux, car **rien ne garantit que ces résultats correspondent au code affiché** (voir 6.2.3).

### 6.2.3 Le piège : l'état caché

Dans un script Python ordinaire, l'ordre d'exécution est celui du fichier, de haut en bas, toujours. Dans un notebook, **vous choisissez l'ordre** : vous pouvez relancer la cellule 5 trois fois, sauter la cellule 3, retourner modifier la cellule 1 sans relancer les suivantes, ou même supprimer une cellule dont la variable continue d'exister en mémoire. Le noyau, lui, se souvient de tout.

Pour voir ce phénomène sans notebook, **simulons** un noyau avec Python pur : un dictionnaire `memoire` joue la mémoire du noyau, et `executer` joue le rôle de Maj + Entrée. Trois cellules calculent le prix total d'une commande de Yasmine : trois paniers à 50 DT, avec TVA à 19 %.

```
memoire = {}

def executer(code):
 exec(code, memoire)

cellules = {
 "A": "prix_unitaire = 50",
 "B": "total = prix_unitaire * 3 * 1.19",
 "C": "print('Total TTC :', round(total, 2), 'DT')",
}

Exécution normale : A, puis B, puis C
for nom in "ABC":
 executer(cellules[nom])
```

Total TTC : 178.5 DT

Le total attendu est  $50 \times 3 \times 1,19 = 178,5$  DT. Maintenant, Yasmine se rend compte que le prix unitaire est en fait de 80 DT. Elle retourne à la cellule A, corrige, la relance... puis relance la cellule C pour voir le résultat, **en oubliant de relancer B** :

```
cellules["A"] = "prix_unitaire = 80" # on corrige la cellule A
executer(cellules["A"]) # on la relance
executer(cellules["C"]) # on relance C (mais pas B !)
```

Total TTC : 178.5 DT

Le total affiché est **toujours 178,5 DT** alors que le prix a changé : la variable `total` en mémoire date de l'ancienne exécution de B. Sur l'écran, la cellule A affiche 80, la cellule C affiche un total faux, et rien ne signale l'incohérence. Pire : si Yasmine enregistre et envoie ce notebook, son collègue qui l'exécutera de haut en bas obtiendra un résultat **différent** du sien. Voici ce que donnerait une exécution propre, sur un noyau neuf :

```
memoire = {} # noyau tout neuf
for nom in "ABC":
 executer(cellules[nom])
print("Vérification à la main :", 80 * 3 * 1.19)
```

```
Total TTC : 285.6 DT
Vérification à la main : 285.59999999999997
```

Le vrai total est **285,6 DT**, et non 178,5 DT : l'écart est considérable. (La ligne de vérification affiche 285.59999999999997 au lieu de 285.6 : c'est l'artefact de calcul en virgule flottante rencontré en 1.5, sans importance ici.)

Un second exemple du même piège, encore plus traître : la **cellule supprimée**. Yasmine définit une remise dans une cellule, l'utilise plus bas, puis supprime la cellule de la remise « pour faire propre ». Tout continue de marcher (la variable vit toujours en mémoire)... jusqu'à ce que quelqu'un d'autre ouvre le notebook :

```
memoire = {}
executer("remise = 0.10") # cellule D, qui sera supprimée plus tard
executer("prix_remise = 80 * (1 - remise)") # cellule E : utilise la variable de D
print("prix remisé (noyau de Yasmine) :", memoire["prix_remise"])
```

```
memoire = {} # noyau neuf chez un collègue, sans la cellule D
try:
 executer("prix_remise = 80 * (1 - remise)")
except NameError as erreur:
 print("collègue : NameError :", erreur)
```

```
prix remisé (noyau de Yasmine) : 72.0
collègue : NameError : name 'remise' is not defined
```

Ces deux scénarios ont un point commun : **le résultat dépendait d'un état invisible**, la mémoire du noyau, qui n'est écrit nulle part dans le fichier. C'est la première cause de notebooks non reproductibles. L'antidote est simple :

**La règle du « Restart & Run All ».** Avant de partager un notebook, de le commiter ou d'en tirer un chiffre pour un rapport, faites **Noyau → Redémarrer et tout exécuter** (*Restart Kernel and Run All Cells*). Cela efface la mémoire et rejoue toutes les cellules **dans l'ordre du fichier**. Si tout passe et que les résultats ne changent pas, votre notebook est sain. S'il casse, vous venez de découvrir un état caché, et mieux vaut le découvrir maintenant que devant votre client.

**Pourquoi cela revient à un problème d'ordre.** Un programme est reproductible si son résultat est une **fonction** de ses entrées (code + données + graine aléatoire). Dans un notebook, le résultat dépend en plus de la **suite des cellules exécutées** : une *séquence*, pas seulement un ensemble. Il existe  $n!$  façons d'ordonner  $n$  cellules, et le fichier n'en garde qu'une trace partielle (les numéros [1], [2]...). Imposer l'ordre du fichier (« Run All ») ramène la dépendance à **une seule** séquence canonique : celle qu'on lit.

### 6.2.4 Un détecteur d'ordre suspect

Les numéros d'exécution enregistrés dans le fichier permettent de repérer les notebooks douteux **sans les exécuter**. Un notebook exécuté d'un seul trait, sur un noyau neuf, affiche [1], [2], [3]... sans trou ni saut. Si les numéros sont désordonnés (3, 1, 2), ou si une cellule n'a jamais tourné, ou s'il y a une erreur enregistrée, l'alarme sonne. Écrivons-en un petit détecteur : c'est un bon exemple de **petite application** de ce que nous avons appris au chapitre 4 (fonctions, listes) à un fichier de notebook.

```
import copy

def audit(carnet):
 """Retourne la liste des problèmes détectés dans un notebook déjà exécuté."""
 problemes = []
 code = [c for c in carnet.cells if c.cell_type == "code"]
 comptes = [c.execution_count for c in code]
 if any(n is None for n in comptes):
 problemes.append("au moins une cellule n'a jamais été exécutée")
 vus = [n for n in comptes if n is not None]
 if vus != list(range(1, len(vus) + 1)):
 problemes.append(f"numéros d'exécution {comptes} : pas 1, 2, 3... (ordre ou noyau douteux)")
 if any(s.output_type == "error" for c in code for s in c.outputs):
 problemes.append("une erreur est enregistrée dans les sorties")
 return problemes or ["OK : notebook exécuté dans l'ordre, sans erreur"]

print("notebook de Yasmine :", audit(nb))

Un notebook « bidouillé » : les cellules ont été exécutées dans le désordre
douteux = copy.deepcopy(nb)
douteux.cells[1].execution_count = 3
douteux.cells[2].execution_count = 1
print("notebook bidouillé :", audit(douteux))
```

```
notebook de Yasmine : ["OK : notebook exécuté dans l'ordre, sans erreur"]
notebook bidouillé : ["numéros d'exécution [3, 1] : pas 1, 2, 3... (ordre ou noyau douteux)"]
```

Le premier notebook passe l'audit. Le second, dont nous avons truqué les numéros pour imiter une session désordonnée, est signalé. Le détecteur ne *prouve* pas qu'un notebook est reproductible (la seule preuve est de le réexécuter), mais il repère les cas flagrants en une milliseconde, par exemple dans un script de vérification avant chaque commit.

### 6.2.5 Bonnes pratiques

1. **Restart & Run All avant tout partage.** Règle numéro un.
2. **Les imports et les réglages en première cellule :** bibliothèques, graine aléatoire (`rng = np.random.default_rng(42)`), chemins des fichiers. On voit d'emblée ce dont le notebook dépend.
3. **Un notebook, une question.** Un notebook de 200 cellules est ingérable ; découpez : 01-nettoyage.ipynb, 02-exploration.ipynb, 03-modele.ipynb.
4. **Sortez le code réutilisable dans des fichiers .py** (fonctions de nettoyage, de calcul) et importez-les : `from outils import nettoyer`. Ce code se teste (4.6), se versionne proprement avec Git, et sert à d'autres notebooks.
5. **Écrivez du texte entre les cellules :** titres, hypothèses, interprétation. Le notebook est un récit, pas un brouillon.
6. **Chemins relatifs** (`donnees/commandes.csv`), jamais `C:\Users\Yasmine\Bureau\...` : le notebook doit marcher sur une autre machine.
7. **N'utilisez pas le notebook pour la production.** Une fois l'analyse stabilisée, un script .py lancé depuis la ligne de commande (6.3) est plus fiable qu'un carnet qu'on clique à la main.

### 6.2.6 Notebooks et Git : le problème des sorties

Un notebook est un fichier texte : Git peut donc le suivre. Mais, comme les **sorties** et les **numéros d'exécution** sont stockés dans le fichier, la moindre ré-exécution modifie des dizaines de lignes (graphiques encodés en base64, numéros qui changent) : les comparaisons git diff deviennent illisibles et les conflits de fusion cauchemardesques. La pratique courante est de **ne garder dans Git que les sources**, en effaçant les sorties avant de commiter. Voici l'opération, que nous écrivons nous-mêmes avec nbformat :

```
def nettoyer(carnet):
 """Copie du notebook sans sorties ni numéros d'exécution."""
 propre = copy.deepcopy(carnet)
 for c in propre.cells:
 if c.cell_type == "code":
 c.outputs = []
 c.execution_count = None
 return propre

complet, propre = nbformat.writes(nb), nbformat.writes(nettoyer(nb))
print("lignes du JSON avec sorties :", complet.count("\n"))
print("lignes du JSON sans sorties :", propre.count("\n"))
cellule = json.loads(propre)["cells"][2]
print("cellule 3 nettoyée :", cellule["execution_count"], cellule["outputs"])
```

```
lignes du JSON avec sorties : 81
lignes du JSON sans sorties : 55
cellule 3 nettoyée : None []
```

Dans la pratique, on n'écrit pas ce nettoyage à la main : l'outil `nbstripout` (non utilisé ici) s'installe en « crochet » Git et efface les sorties automatiquement à chaque commit. Le revers de la médaille : le fichier commité n'affiche plus de résultats ; on publie alors, à côté, une version **exportée** (voir ci-dessous).

### 6.2.7 Exporter : du notebook au rapport, au script

La bibliothèque `nbconvert` transforme un notebook en d'autres formats : **HTML** (à envoyer par e-mail), **PDF**, **Markdown**, ou **script Python** (le code seul). Avec Python :

```
from nbconvert import PythonExporter, MarkdownExporter

script, _ = PythonExporter().from_notebook_node(nb)
print(script)
```

```
#!/usr/bin/env python
coding: utf-8

Ventes de Dar Jasmin
Montant moyen des commandes, par canal.

In[1]:

import pandas as pd
df = pd.read_csv('donnees/commandes.csv')
df.shape

In[2]:

df.groupby('canal')['montant'].mean().round(2)
```

Le script contient le code de chaque cellule, précédé d'un commentaire # In[1]: marquant les cellules ; le texte Markdown est transformé en commentaires. Version rapport :

```
rapport, _ = MarkdownExporter().from_notebook_node(nb)
print(rapport.replace("`", "~~~")) # ~~~ à la place des accents graves, pour l'affichage dans le
↳ livre

Ventes de Dar Jasmin
Montant moyen des commandes, par canal.

~~~python
import pandas as pd
df = pd.read_csv('donnees/commandes.csv')
df.shape
~~~

(400, 4)

~~~python
df.groupby('canal')['montant'].mean().round(2)
~~~

canal
Boutique 74.81
Instagram 49.01
Site 59.50
Name: montant, dtype: float64
```

Le résultat est un document Markdown contenant le titre, le code de chaque cellule (dans un bloc de code) et ses sorties. (Les blocs de code Markdown s'écrivent normalement avec trois accents graves ; nous les avons remplacés par ~~~ uniquement pour pouvoir afficher le résultat à l'intérieur de ce livre.) C'est ainsi que l'on peut générer un rapport propre à partir d'un notebook.

Même chose en ligne de commande, avec l'option --execute qui rejoue d'abord tout le notebook dans un noyau neuf (c'est exactement la règle « Restart & Run All », automatisée). Dans l'atelier, nous créons un petit notebook depuis le terminal, puis le convertissons :

```
mkdir -p ~/atelier/notebook
cd ~/atelier/notebook
cp ~/atelier/dar-jasmin/commandes.csv .
python - <<'FIN'
import nbformat
from nbformat import v4 as nbformat
nb = nbformat.new_notebook(cells=[
 nbformat.new_markdown_cell("# Satisfaction par canal", id="a"),
 nbformat.new_code_cell("import pandas as pd\n"
↳ "df = pd.read_csv('commandes.csv')\n"
"df.groupby('canal')['satisfaction'].mean().round(2)", id="b"),
])
nb.metadata["kernelspec"] = {"name": "python3", "display_name": "Python 3", "language": "python"}
nbformat.write(nb, "satisfaction.ipynb")
FIN
jupyter nbconvert --to markdown --execute satisfaction.ipynb 2>&1 | grep -v -i warning
sed 's/`/~~~/g' satisfaction.md
```

```
[NbConvertApp] Converting notebook satisfaction.ipynb to markdown
[NbConvertApp] Writing 268 bytes to satisfaction.md
Satisfaction par canal

~~~python
import pandas as pd
df = pd.read_csv('commandes.csv')
df.groupby('canal')['satisfaction'].mean().round(2)
~~~

canal
Boutique 4.49
Instagram 3.72
Site 3.79
Name: satisfaction, dtype: float64
```

La commande de conversion affiche ce qu'elle fait (lecture du notebook, écriture du fichier `satisfaction.md`) ; `sed` affiche ensuite le document obtenu (avec la même substitution `~~~` que ci-dessus). Pour obtenir un fichier HTML, il suffit de remplacer `markdown` par `html` ; pour un PDF, `--to pdf` demande en plus une installation de LaTeX (6.5).

### 6.2.8 Pour finir : choisir son outil

Je veux...	J'utilise...
explorer des données, essayer des idées	un <b>notebook</b>
un traitement répétable, automatisé (tous les soirs)	un <b>script</b> <code>.py</code>
une fonction que je vais réutiliser	un <b>module</b> <code>.py</code> , importé dans le notebook
un rapport reproductible de bout en bout	un document qui mélange texte et code, exécuté d'un trait (6.5)

#### À retenir

- Un notebook = des **cellules** (code / Markdown) + un **noyau** (la mémoire). Le fichier `.ipynb` est du JSON qui stocke le code **et** les derniers résultats.
- Piège principal : l'**état caché**. La mémoire du noyau peut contenir des variables qui ne correspondent plus au code écrit (cellule modifiée mais pas relancée, cellule supprimée...).
- Avant de partager : **Restart & Run All**. Des numéros d'exécution `[1]`, `[2]`, `[3]`... sans trou sont bon signe.
- Rangez le code réutilisable dans des `.py`, ne versionnez pas les sorties, exportez avec `nbconvert`.
- Un notebook sert à **explorer et raconter**, un script à **produire**.

## 6.3 Ligne de commande et environnements

**Intuition.** Une interface graphique, c'est un **menu de restaurant** : on montre du doigt ce qu'on veut parmi ce qui est proposé. La **ligne de commande** (*command line*, *terminal*), c'est parler directement au cuisinier : on peut demander des choses que le menu n'a jamais prévues, et surtout **écrire sa commande sur un papier** pour la rejouer à l'identique demain, ou

la donner à un collègue. Pour un data scientist, ce second point est capital : une suite de commandes est une **recette reproductible**, alors qu'une série de clics ne laisse aucune trace.

Le terminal fait peur au début : un écran noir, un curseur clignotant, aucune indication. Il suffit pourtant d'une douzaine de commandes pour devenir efficace, et elles servent pour toute une carrière : sur votre ordinateur, sur un serveur distant, dans un conteneur, dans un outil d'automatisation. Dans ce livre, nous utilisons **Bash**, le terminal de Linux, de macOS et de Git Bash/WSL sous Windows.

6.3.1 S'orienter dans les dossiers

Les fichiers sont rangés dans une **arborescence** : un dossier contient des fichiers et d'autres dossiers. Le terminal est toujours « placé » dans un dossier, le **dossier courant**. Cinq commandes pour s'y repérer :

Commande	Signification	Mnémonique
pwd	afficher le dossier courant	<i>print working directory</i>
ls	lister le contenu du dossier	<i>list</i>
cd dossier	se déplacer dans un dossier	<i>change directory</i>
mkdir dossier	créer un dossier	<i>make directory</i>
touch fichier	créer un fichier vide (ou mettre à jour sa date)	

Quelques **chemins spéciaux** à connaître : `.` est le dossier courant, `..` le dossier **parent** (celui qui contient le dossier courant), `~` votre dossier personnel, `/` la racine de toute l'arborescence. Un chemin est **absolu** s'il part de la racine (`/home/yasmine/atelier`) et **relatif** s'il part du dossier courant (`donnees/commandes.csv`).

Yasmine organise enfin son travail. Voici l'arborescence d'un projet d'analyse bien rangé, créée d'un coup :

```
cd ~/atelier
mkdir -p etude-ventes/{donnees,notebooks,src,rapports}
cd etude-ventes
cp ../dar-jasmin/commandes.csv donnees/
touch README.md src/outils.py
pwd | sed "s|^$HOME|~|"
ls
find . -not -name '.' | sort
```

```
~/atelier/etude-ventes
README.md
donnees
notebooks
rapports
src
./README.md
./donnees
./donnees/commandes.csv
./notebooks
./rapports
./src
./src/outils.py
```

La syntaxe `{a,b,c}` est une **expansion d'accolades** : `mkdir -p etude-ventes/{donnees,notebooks,src,rapport` équivaut à quatre créations. `pwd` donne le dossier courant (nous avons remplacé le début du chemin par `~`, comme le fait votre terminal dans son invite de commande), `ls` en liste le contenu,



et find en affiche l'arborescence complète, triée. Chaque dossier a un rôle : les données brutes ne se modifient jamais, les notebooks explorent, src contient le code réutilisable, rapports reçoit les résultats.

Pour **déplacer, renommer, supprimer** :

```
mv README.md LISEZMOI.md # renommer
mv LISEZMOI.md README.md # et on revient en arrière
cp -r rapports rapports-copie # copier un dossier entier (-r : récursif)
rm -r rapports-copie # supprimer un dossier et son contenu
ls
```

```
README.md
donnees
notebooks
rapports
src
```

**rm n'a pas de corbeille.** Ce qui est supprimé dans le terminal est **définitivement perdu**. Ne tapez jamais `rm -r` sans relire la ligne entière, et n'écrivez jamais de commande de suppression avec un chemin que vous n'avez pas vérifié avec `ls` juste avant. Avec Git (6.1), au moins, vous pouvez toujours récupérer une version enregistrée.

**Quatre astuces qui changent la vie.** (1) La touche **Tab** complète les noms de fichiers : tapez `cd` et puis Tab. (2) La flèche **↑** rappelle les commandes précédentes ; `Ctrl + R` recherche dans l'historique. (3) **Ctrl + C** interrompt une commande qui tourne. (4) commande `--help` (ou `man commande`) affiche la documentation de n'importe quelle commande.

### 6.3.2 Regarder le contenu d'un fichier

Un fichier CSV est un fichier **texte** : on peut le lire dans le terminal sans l'ouvrir dans un tableur. C'est très pratique pour vérifier rapidement un fichier de plusieurs millions de lignes que Excel ne saurait pas ouvrir.

```
cd donnees
head -n 5 commandes.csv
echo "... "
tail -n 3 commandes.csv
wc -l commandes.csv
```

```
canal,montant,livraison,satisfaction
Boutique,44.8,0,4
Site,34.5,2,4
Instagram,88.2,5,4
Instagram,30.1,4,4
...
Site,62.6,4,3
Instagram,37.5,3,4
Site,31.4,4,4
401 commandes.csv
```

`head -n 5` affiche les 5 premières lignes, `tail -n 3` les 3 dernières, `wc -l` compte les lignes. Le CSV a une ligne d'en-tête (les noms de colonnes) puis une commande par ligne : on retrouve bien 401 lignes. Pour parcourir un long fichier page par page, il existe `less` (on avance avec la barre d'espace, on quitte avec q) ; il est interactif, donc nous ne pouvons pas l'illustrer ici. `cat` fichier affiche tout le fichier d'un coup.

### 6.3.3 Interroger des données avec des outils de texte

Il existe quelques outils très anciens, très rapides, et parfaitement adaptés aux fichiers de données en colonnes. Les voici, appliqués au fichier de Dar Jasmin. Chaque outil fait **une chose** :

Outil	Rôle
grep motif	garder les lignes qui contiennent le motif
cut -d, -f2	extraire la colonne n°2 (séparateur ,)
sort	trier (-n : numérique, -r : décroissant, -t, : séparateur, -k2 : colonne)
uniq -c	compter les lignes identiques <b>consécutives</b> (d'où le sort avant)
awk	mini-langage pour calculer sur les colonnes

Première question : combien de commandes viennent d'Instagram ?

```
grep -c Instagram commandes.csv
```

```
138
```

grep -c compte les lignes contenant le mot. (Pandas nous donnerait la même chose avec `(df["canal"] == "Instagram").sum())`. Combien de commandes par canal ? Il faut extraire la colonne des canaux (sans l'en-tête), la trier pour regrouper les valeurs identiques, puis compter :

```
tail -n +2 commandes.csv | cut -d, -f1 | sort | uniq -c | sort -rn
```

```
148 Site
138 Instagram
114 Boutique
```

Cette ligne est un **pipeline** (« tuyau ») : `tail -n +2` supprime l'en-tête (« commence à la ligne 2 »), `cut -d, -f1` garde la première colonne, `sort` regroupe les canaux, `uniq -c` compte chaque groupe, et `sort -rn` classe du plus fréquent au plus rare. Le site est donc le premier canal en nombre de commandes (148), devant Instagram (138) et la boutique (114) : 400 commandes au total, comme prévu. Le symbole `|` (« pipe ») envoie la sortie d'une commande à l'entrée de la suivante. Quelles sont les trois plus grosses commandes ?

```
tail -n +2 commandes.csv | sort -t, -k2 -n -r | head -n 3
```

```
Site,255.7,4,3
Site,243.8,5,3
Site,217.1,7,4
```

On trie sur la 2<sup>e</sup> colonne (`-k2`), numériquement (`-n`), de la plus grande à la plus petite (`-r`), et on garde les trois premières lignes. La plus grosse commande atteint 255,7 DT : c'est le maximum déjà vu avec `describe()` au chapitre 3. Et pour le montant moyen ? Il faut calculer, ce que fait `awk` : il lit le fichier ligne par ligne, `$2` désigne la 2<sup>e</sup> colonne, `NR` le numéro de ligne.

```
awk -F, 'NR > 1 { somme += $2; n++ } END { printf "montant moyen : %.2f DT sur %d commandes\n",
↵ somme/n, n }' commandes.csv
```

```
montant moyen : 60.25 DT sur 400 commandes
```

L'option `-F`, fixe le séparateur. Pour chaque ligne sauf l'en-tête (`NR > 1`), on ajoute le montant à une somme et on compte ; à la fin (`END`) on affiche la moyenne. Retrouve-t-on bien le 60,25 DT du

chapitre 3 ? On peut même faire une moyenne **par canal**, avec un tableau associatif (l'équivalent d'un dictionnaire Python) :

```
awk -F, 'NR > 1 { somme[$1] += $2; n[$1]++ }
 END { for (canal in n) printf "%-10s %.2f\n", canal, somme[canal]/n[canal] }' commandes.csv |
↪ sort
```

```
Boutique 74.81
Instagram 49.01
Site 59.50
```

Ces trois moyennes sont celles que `df.groupby("canal")["montant"].mean()` nous avait données en 6.1 : les deux outils sont d'accord, ce qui est rassurant.

**Quand utiliser quoi ?** Ces commandes brillent pour un **coup d'œil rapide**, pour traiter des fichiers trop gros pour la mémoire, ou pour assembler une chaîne de traitement dans un script. Dès que l'analyse devient subtile (jointures, valeurs manquantes, statistiques), on passe à pandas (4.4). Les deux approches sont complémentaires.

### 6.3.4 Pipes et redirections : assembler de petits outils

Toutes ces commandes obéissent à la même philosophie, dite « **philosophie Unix** » : *chaque programme fait une seule chose, la fait bien, et lit/écrit du texte*. Ainsi tous les programmes se branchent les uns sur les autres, comme des briques de LEGO. Il y a trois flux : l'**entrée standard** (clavier ou tuyau), la **sortie standard** (écran) et la **sortie d'erreur** (écran aussi, mais distincte). Les quatre opérateurs de redirection vers ou depuis des fichiers :

Opérateur	Effet	Exemple
>	écrire la sortie dans un fichier ( <b>écrase</b> l'existant)	<code>ls &gt; liste.txt</code>
>>	<b>ajouter</b> la sortie à la fin du fichier	<code>echo "fin" &gt;&gt; liste.txt</code>
<	lire l'entrée depuis un fichier	<code>sort &lt; liste.txt</code>
2>	rediriger les <b>messages d'erreur</b>	<code>ls absent 2&gt; erreurs.txt</code>

À ces quatre opérateurs s'ajoute le **tuyau** (la barre verticale `|`, *pipe* en anglais), déjà utilisé plus haut : il branche la sortie d'une commande directement sur l'entrée de la suivante, sans passer par un fichier. Sauvegardons un résumé des ventes dans un fichier, puis voyons comment les erreurs se redirigent :

```
cd ..
tail -n +2 donnees/commandes.csv | cut -d, -f1 | sort | uniq -c > rapports/commandes-par-canal.txt
echo "# généré par la ligne de commande" >> rapports/commandes-par-canal.txt
cat rapports/commandes-par-canal.txt

ls donnees/inexistant.csv 2> rapports/erreurs.txt
echo "code de sortie : $?"
cat rapports/erreurs.txt
```

```
114 Boutique
138 Instagram
148 Site
généré par la ligne de commande
code de sortie : 2
ls: cannot access 'donnees/inexistant.csv': No such file or directory
```

Le premier bloc écrit le décompte dans un fichier (>), puis y ajoute une ligne de commentaire (>>). Le second tente de lister un fichier qui n'existe pas : le message d'erreur va dans `erreurs.txt` au lieu de s'afficher. (Ici, la commande `ls` a échoué et le code vaut 2.) La variable spéciale `$?` contient le **code de sortie** de la dernière commande : **0** signifie « **tout s'est bien passé** », toute autre valeur signale une erreur. Les scripts s'appuient sur ces codes pour enchaîner des étapes : `commande1 && commande2` lance la seconde **seulement si** la première a réussi ; `commande1 || commande2` la lance seulement si la première a échoué.

### 6.3.5 Lancer des programmes Python depuis le terminal

Jusqu'ici nous avons utilisé des commandes « toutes faites ». Le plus utile est de pouvoir **lancer ses propres scripts Python**, avec des paramètres. Écrivons un petit outil : `src/resume.py` prend en argument le nom d'un fichier CSV et affiche un résumé de ses colonnes numériques. Les arguments tapés après le nom du script sont disponibles en Python dans la liste `sys.argv` (le premier élément, `sys.argv[0]`, est le nom du script lui-même).

```
cat > src/resume.py <<'FIN'
"""Résumé rapide d'un fichier CSV : python src/resume.py fichier.csv"""
import sys
import pandas as pd

if len(sys.argv) != 2:
 print("usage : python src/resume.py fichier.csv")
 sys.exit(2)

try:
 df = pd.read_csv(sys.argv[1])
except FileNotFoundError:
 print(f"fichier introuvable : {sys.argv[1]}")
 sys.exit(1)

print(f"{len(df)} lignes, {df.shape[1]} colonnes")
print(df.describe().loc[["mean", "min", "max"]].round(2))
FIN
python src/resume.py donnees/commandes.csv
echo "code de sortie : $?"
python src/resume.py donnees/absent.csv
echo "code de sortie : $?"
python src/resume.py
echo "code de sortie : $?"
```

```
400 lignes, 4 colonnes
 montant livraison satisfaction
mean 60.25 3.29 3.96
min 8.60 0.00 1.00
max 255.70 13.00 5.00
code de sortie : 0
fichier introuvable : donnees/absent.csv
code de sortie : 1
usage : python src/resume.py fichier.csv
code de sortie : 2
```

Notre script respecte la convention : code **0** quand tout va bien, **1** pour un fichier absent, **2** pour un mauvais usage (appel sans argument). Ces codes permettent à d'autres programmes d'enchaîner les étapes automatiquement, et de s'arrêter proprement en cas de problème. Deux variantes utiles de python : `python -c "print(2+3)"` exécute une ligne, `python -m module` exécute un module de la bibliothèque (par exemple `python -m venv`, juste après).

### 6.3.6 Le problème des bibliothèques : les environnements virtuels

Voici un scénario qui arrive à tout le monde. En janvier, Yasmine installe pandas pour son analyse. En juin, elle commence un autre projet qui nécessite une **ancienne** version de la même bibliothèque. Si tout est installé au même endroit, mettre à jour casse l'ancien projet, et rétrograder casse le nouveau. Pire : quand elle envoie son code à Amine, comment sait-il quelles versions utiliser ? D'où l'**environnement virtuel** :

**Définition.** Un environnement virtuel est un **dossier isolé** contenant une copie de l'interpréteur Python et ses propres bibliothèques. Chaque projet a le sien : les bibliothèques de l'un n'affectent jamais celles de l'autre. L'environnement se **crée**, s'**active**, puis on y installe ce dont le projet a besoin avec **pip**, le gestionnaire de paquets de Python.

Voici le cycle complet, avec exactement les commandes de l'avant-propos. Pour ne pas télécharger des centaines de mégaoctets dans cet exemple, nous installerons une toute petite bibliothèque, *tabulate* (qui formate joliment des tableaux en texte), à la place de pandas.

```
cd ~/atelier/etude-ventes
python -m venv .venv
source .venv/bin/activate
python -c "import sys; print('environnement virtuel actif :', sys.prefix != sys.base_prefix)"
pip list 2>/dev/null
```

```
environnement virtuel actif : True
Package Version

pip 25.0
```

`python -m venv .venv` crée l'environnement dans un dossier caché nommé `.venv` (le point initial le cache de `ls`). `source .venv/bin/activate` l'**active** : dès cet instant, `python` et `pip` désignent ceux du dossier `.venv`, et l'invite du terminal affiche généralement `(.venv)`. Sous Windows, la commande d'activation est `.venv\Scripts\activate`. L'environnement est **vierge** : la liste des bibliothèques ci-dessus ne contient que `pip` lui-même (ou à peine plus). Installons-en une, et vérifions qu'elle fonctionne :

```
pip install --quiet tabulate 2>&1 | grep -v -i -E "notice|warning"
python -c "from tabulate import tabulate; print(tabulate(['Boutique', 74.81], ['Instagram', 49.01],
↳ ['Site', 59.5]), headers=['canal', 'montant moyen'], floatfmt='.2f'))"
pip freeze
```

```
canal montant moyen

Boutique 74.81
Instagram 49.01
Site 59.50
tabulate==0.10.0
```

`pip install` télécharge et installe la bibliothèque (et ses dépendances) **dans .venv uniquement**. `pip freeze` liste tout ce qui est installé, avec les **numéros de version exacts**. Ce dernier point est la clé de la reproductibilité : on enregistre cette liste dans un fichier, traditionnellement nommé `requirements.txt`, que l'on versionne avec Git (6.1) :

```
pip freeze > requirements.txt
cat requirements.txt
deactivate
```

```
tabulate==0.10.0
```

`deactivate` quitte l'environnement et rend le terminal à son état ordinaire. Quelqu'un qui reçoit

le projet (Amine, ou Yasmine dans un an) reconstruit **le même environnement** en deux commandes :

```
python -m venv .venv-amine
source .venv-amine/bin/activate
pip install --quiet -r requirements.txt 2>&1 | grep -v -i -E "notice|warning"
pip freeze
deactivate
```

```
tabulate==0.10.0
```

La liste obtenue est **identique** à celle de Yasmine : mêmes bibliothèques, mêmes versions. Le dossier `.venv` lui-même ne se partage pas (il est propre à la machine et volumineux) : il figure dans le `.gitignore` de 6.1, et seul `requirements.txt` voyage avec le projet.

**requirements.txt fige les bibliothèques, pas Python lui-même.** Mentionnez aussi la version de Python dans le README (« testé avec Python 3.13 »). Pour des besoins plus avancés, il existe des outils qui gèrent aussi la version de Python : `conda` (très répandu en data science, notamment sous Windows), `uv` (récent et très rapide), ou `Poetry`. Ils répondent au même besoin : isoler, figer, reproduire.

Pour le livre complet, l'installation recommandée dans l'avant-propos serait donc (*non exécutée ici : elle télécharge environ 200 Mo de bibliothèques*) :

```
bash noexec python -m venv .venv source .venv/bin/activate # sous Windows :
.venv\Scripts\activate pip install numpy pandas scipy matplotlib seaborn pip freeze
> requirements.txt
```

### 6.3.7 Variables d'environnement et PATH

Le terminal garde en mémoire des **variables** (texte nommé), que l'on affiche avec `$` : c'est ainsi que nous avons utilisé `$?` et `$HOME`. On en crée avec `NOM=valeur` (sans espaces autour du `=`) ; `export` les rend visibles aux programmes lancés ensuite.

```
BOUTIQUE="Dar Jasmin"
echo "Bienvenue chez $BOUTIQUE"
export TAUX_TVA=0.19
python -c "import os; print('TVA lue depuis Python : ', float(os.environ['TAUX_TVA']))"
```

```
Bienvenue chez Dar Jasmin
TVA lue depuis Python : 0.19
```

Ce mécanisme sert à **transmettre des réglages** à un programme sans les écrire dans son code : un chemin de fichier, un taux, ou un **mot de passe de base de données** (qu'on ne met jamais dans le code ni dans Git !). Une variable est particulièrement importante : **PATH**, la liste des dossiers où le terminal cherche les programmes. Quand vous tapez `python`, le terminal parcourt les dossiers de `PATH` dans l'ordre et lance le premier `python` trouvé. **Activer un environnement virtuel, c'est simplement placer son dossier `bin` en tête du `PATH`** : voilà pourquoi `python` désigne alors celui de `.venv`.

### 6.3.8 Et sous Windows ?

Trois options : **WSL** (un vrai Linux intégré à Windows, recommandé), **Git Bash** (fourni avec Git for Windows), ou **PowerShell** qui a ses propres noms de commandes :

Bash	PowerShell	Rôle
pwd	pwd ou Get-Location	dossier courant
ls	ls ou dir	lister
cp, mv, rm	cp, mv, rm	copier, déplacer, supprimer
cat	cat ou type	afficher un fichier
source .venv/bin/activate	.venv\Scripts\Activate.ps1	activer l'environnement

### À retenir

- Une suite de commandes est une **recette reproductible** ; un clic n'en est pas une.
- Se repérer : pwd, ls, cd, mkdir, cp, mv, rm (sans corbeille !). . = ici, .. = parent, ~ = maison.
- Lire et interroger du texte : head, tail, wc, grep, cut, sort, uniq, awk. On les assemble avec |, on redirige avec >, >>, 2>.
- Un programme renvoie un **code de sortie** (0 = succès) ; && et || s'appuient dessus.
- **Un environnement virtuel par projet** : python -m venv .venv, source .venv/bin/activate, pip install, pip freeze > requirements.txt. On partage requirements.txt, pas .venv.

## 6.4 ⊕ Pour aller plus loin : scripts shell, environnements avancés et Docker

**Section optionnelle.** Elle prolonge la section 6.3 pour celles et ceux qui veulent **automatiser** leurs analyses (scripts shell), **isoler** plus finement leurs projets, et découvrir **Docker**, l'outil qui emballe un projet avec tout son système. Vous pouvez passer directement à 6.5 sans rien perdre du fil.

### 6.4.1 Écrire un script shell

Quand on retape trois fois la même suite de commandes, on la range dans un **script shell** : un fichier texte contenant des commandes, exécuté d'un coup. Notre premier script, scripts/rapport.sh, affiche pour un fichier de commandes le nombre de commandes par canal et leur part. Il introduit les ingrédients de base : **arguments** (\$1 est le premier argument, \$# leur nombre), **variables**, **test** (if), **boucle** (for) et **substitution de commande** (\$( ... )) insère le résultat d'une commande dans une autre).

```
cd ~/atelier/etude-ventes
mkdir -p scripts
cat > scripts/rapport.sh <<'FIN'
#!/usr/bin/env bash
rapport.sh : nombre de commandes par canal, avec leur part
usage : bash scripts/rapport.sh donnees/commandes.csv
set -euo pipefail

if ["$#" -ne 1]; then
 echo "usage : $0 fichier.csv" >&2
 exit 2
fi
fichier="$1"
if [! -f "$fichier"]; then
 echo "fichier introuvable : $fichier" >&2
 exit 1
fi
```

```
total=$(($(wc -l < "$fichier") - 1))
echo "== Rapport sur $fichier ($total commandes) =="
for canal in Boutique Instagram Site; do
 n=$(grep -c "^$canal," "$fichier")
 part=$(awk -v n="$n" -v t="$total" 'BEGIN { printf "%.1f", 100 * n / t }')
 echo " $canal : $n commandes ($part %)"
done
FIN
bash scripts/rapport.sh donnees/commandes.csv
```

```
== Rapport sur donnees/commandes.csv (400 commandes) ==
Boutique : 114 commandes (28.5 %)
Instagram : 138 commandes (34.5 %)
Site : 148 commandes (37.0 %)
```

Passons en revue ce fichier, ligne par ligne :

- La première ligne, `#!/usr/bin/env bash`, s'appelle le **shebang** : elle indique avec quel programme exécuter le fichier. Les lignes qui commencent par `#` sont des **commentaires**.
- `set -euo pipefail` est une **ceinture de sécurité** que l'on met en tête de tout script sérieux : `-e` arrête le script à la première commande qui échoue, `-u` refuse d'utiliser une variable jamais définie (souvent une faute de frappe), `-o pipefail` fait échouer un pipeline si l'une de ses commandes échoue (et pas seulement la dernière). Nous verrons plus bas ce qu'elle apporte.
- Le premier `if` vérifie qu'on a reçu un argument (`$#` vaut 1) ; sinon il affiche un mode d'emploi **sur la sortie d'erreur** (`>&2`) et quitte avec le code 2. Le second vérifie que le fichier existe (`-f`).
- `total=$(( $(wc -l < "$fichier") - 1 ))` calcule le nombre de lignes moins l'en-tête (la double parenthèse `$(( ... ))` fait de l'arithmétique entière).
- La boucle `for` passe en revue chaque canal ; `grep -c "^$canal,"` compte les lignes **commençant** par le nom du canal suivi d'une virgule (`^` marque le début de ligne) ; `awk` calcule le pourcentage, car le shell ne sait pas faire de décimales.

Les trois parts s'ajoutent bien à 100 %. Le script est utilisable depuis n'importe où ; on peut aussi le rendre directement exécutable :

```
chmod +x scripts/rapport.sh
./scripts/rapport.sh donnees/commandes.csv | head -n 2
bash scripts/rapport.sh donnees/absent.csv
echo "code de sortie : $?"
bash scripts/rapport.sh
echo "code de sortie : $?"
```

```
== Rapport sur donnees/commandes.csv (400 commandes) ==
Boutique : 114 commandes (28.5 %)
fichier introuvable : donnees/absent.csv
code de sortie : 1
usage : scripts/rapport.sh fichier.csv
code de sortie : 2
```

`chmod +x` ajoute le droit d'**exécution** au fichier. Les deux appels fautifs affichent leur message et renvoient des codes différents (1 puis 2), comme au 6.3.5. Un autre usage typique : appliquer un traitement à **chaque fichier** d'un groupe, avec une boucle. Séparons le fichier de données en un fichier par canal (en recopiant l'en-tête dans chacun) :

```
mkdir -p donnees/par-canal
for canal in Boutique Instagram Site; do
 (head -n 1 donnees/commandes.csv; grep "^$canal," donnees/commandes.csv) >
 "donnees/par-canal/$canal.csv"
```



```
done
wc -l donnees/par-canal/*.csv
```

```
115 donnees/par-canal/Boutique.csv
139 donnees/par-canal/Instagram.csv
149 donnees/par-canal/Site.csv
403 total
```

Le **\*** est un **joker** : `donnees/par-canal/*.csv` désigne tous les fichiers `.csv` du dossier. Chaque fichier compte une ligne de plus que son nombre de commandes (l'en-tête) ; le total est donc de 400 commandes + 3 en-têtes.

### 6.4.2 Enchaîner des étapes : un petit pipeline

Un vrai projet d'analyse est une **chaîne d'étapes** : vérifier les données, calculer, produire le rapport. Écrivons-la sous forme d'un script qui appelle notre script shell et notre programme Python de 6.3.5 :

```
cat > scripts/pipeline.sh <<'FIN'
#!/usr/bin/env bash
set -euo pipefail
echo "[1/3] comptage par canal"
bash scripts/rapport.sh "$1" > rapports/rapport.txt
echo "[2/3] résumé des colonnes numériques"
python src/resume.py "$1" >> rapports/rapport.txt
echo "[3/3] terminé : rapport dans rapports/rapport.txt"
FIN
bash scripts/pipeline.sh donnees/commandes.csv
cat rapports/rapport.txt
```

```
[1/3] comptage par canal
[2/3] résumé des colonnes numériques
[3/3] terminé : rapport dans rapports/rapport.txt
== Rapport sur donnees/commandes.csv (400 commandes) ==
 Boutique : 114 commandes (28.5 %)
 Instagram : 138 commandes (34.5 %)
 Site : 148 commandes (37.0 %)
400 lignes, 4 colonnes
 montant livraison satisfaction
mean 60.25 3.29 3.96
min 8.60 0.00 1.00
max 255.70 13.00 5.00
```

Tout se passe bien. Voyons maintenant ce qui se passe quand les données sont introuvables, **avec** puis **sans** la ceinture de sécurité `set -e` (nous fabriquons une copie du script d'où cette ligne est retirée) :

```
echo "--- avec set -euo pipefail ---"
bash scripts/pipeline.sh donnees/absent.csv
echo "code de sortie : $?"

echo
echo "--- sans set -e (script imprudent) ---"
sed '/^set -euo/d' scripts/pipeline.sh > scripts/pipeline-imprudent.sh
bash scripts/pipeline-imprudent.sh donnees/absent.csv
echo "code de sortie : $?"
```

```
--- avec set -euo pipefail ---
[1/3] comptage par canal
fichier introuvable : donnees/absent.csv
code de sortie : 1
```

```

--- sans set -e (script imprudent) ---
[1/3] comptage par canal
fichier introuvable : donnees/absent.csv
[2/3] résumé des colonnes numériques
[3/3] terminé : rapport dans rapports/rapport.txt
code de sortie : 0

```

Dans le premier cas, le script s'**arrête net** à la première étape défaillante et renvoie un code d'erreur : on sait immédiatement que quelque chose ne va pas. Dans le second, il continue comme si de rien n'était, annonce « terminé » et renvoie le code 0 (succès !) alors que **rien n'a été calculé** : le pire des scénarios, car personne ne se doute que le rapport est vide ou partiel. D'où la règle : **un script qui échoue doit échouer bruyamment**.

**Les limites du shell.** Le shell est parfait pour **enchaîner des programmes**, manipuler des fichiers et des dossiers. Mais dès qu'il faut des calculs, des structures de données ou de la logique non triviale, passez à Python : ses 300 lignes seront plus lisibles et plus testables que 300 lignes de shell. Règle pratique : si le script dépasse une cinquantaine de lignes ou nécessite des tableaux, c'est un script Python.

### 6.4.3 Faire tourner un script tout seul : cron

Sous Linux et macOS, le programme **cron** exécute des commandes à heure fixe. On le configure en ajoutant des lignes à la **table cron** (crontab -e). Une ligne comporte cinq champs de temps (minute, heure, jour du mois, mois, jour de la semaine) suivis de la commande. Par exemple, pour lancer notre pipeline tous les jours à 7 h du matin :

```
bash noexec # minute heure jour mois jour-semaine commande 0 7 * * * cd ~/etude-ventes && bash scripts/pipeline.sh donnees/commandes.csv
```

(Non exécuté : la table cron appartient à l'utilisateur de la machine, et le résultat ne se verrait qu'à 7 h.) Le \* signifie « toutes les valeurs ». Sous Windows, l'équivalent est le *Planificateur de tâches*. Dans le cloud, on utilise des services d'orchestration plus riches (comme Airflow), que vous rencontrerez dans les volumes suivants de la série.

### 6.4.4 Isoler plus finement : plusieurs versions d'une même bibliothèque

Au 6.3.6, nous avons dit que des projets distincts peuvent avoir besoin de **versions différentes** d'une même bibliothèque. Démonstrons-le : en plus de l'environnement .venv (qui contient la dernière version de tabulate), créons .venv-ancien avec une version plus ancienne, **exigée avec ==** dans la commande d'installation. Les deux coexistent sans se gêner.

```

cd ~/atelier/etude-ventes
python -m venv .venv-ancien
.venv-ancien/bin/pip install --quiet "tabulate==0.8.10" 2>&1 | grep -v -i -E "notice|warning"
echo "projet ancien :"
.venv-ancien/bin/pip freeze
echo "projet récent :"
.venv/bin/pip freeze

```

```

projet ancien :
tabulate==0.8.10
projet récent :
tabulate==0.10.0

```

(Ici nous avons appelé directement le pip de chaque environnement par son chemin, sans l'activer : c'est strictement équivalent, et très pratique dans un script.) Les deux projets travaillent chacun

avec **leur** version : on a deux « mondes » indépendants sur la même machine. Dans un fichier `requirements.txt`, les **numéros de version** peuvent s'écrire avec plus ou moins de rigueur :

Écriture	Signification
<code>pandas==3.0.0</code>	exactement cette version (reproductibilité maximale)
<code>pandas&gt;=3.0,&lt;4</code>	n'importe quelle version 3.x, à partir de la 3.0
<code>pandas</code>	n'importe quelle version (déconseillé : le résultat change avec le temps)

La bonne pratique à retenir : **== pour reproduire un résultat** (rapport remis à un client, article), **fourchette** pour une bibliothèque en développement qui doit rester compatible avec d'autres.

Deux outils alternatifs que vous croiserez (*non exécutés ici : non installés*) :

“`bash noexec # conda : un gestionnaire qui installe aussi Python lui-même et des bibliothèques non Python`  
`conda create --name etude-ventes python=3.13 pandas matplotlib conda activate etude-ventes`  
`conda env export > environment.yml` # l'équivalent de `requirements.txt`

# uv : un installateur très rapide, compatible avec pip

```
uv venv uv pip install -r requirements.txt
```

```
6.4.5 Docker : emballer le projet avec son système
```

Un environnement virtuel isole les **bibliothèques Python**. Mais votre analyse peut dépendre d'autres

> (i) **Une analogie.** Un environnement virtuel, c'est une **étagère personnelle** dans une cuisine

Trois mots à retenir :

- Une **image** est le **modèle** figé : le système de base, Python, vos bibliothèques, votre code. On
- Un **conteneur** est une **instance en marche** d'une image. On peut en lancer dix à partir de la même
- Un **registre** (comme *Docker Hub*) est un entrepôt d'images, un peu comme GitHub pour le code : on

Voici le `Dockerfile` d'un projet d'analyse comme le nôtre (dans un vrai projet, `requirements.txt`) :

```
``dockerfile
1. on part d'une image Python toute prête
FROM python:3.13-slim
2. le dossier de travail dans le conteneur
WORKDIR /app
3. on copie la liste des bibliothèques...
COPY requirements.txt .
4. ...et on les installe
RUN pip install --no-cache-dir -r requirements.txt
5. on copie ensuite le reste du projet
COPY . .
6. commande lancée au démarrage du conteneur
CMD ["python", "src/resume.py", "donnees/commandes.csv"]
```

Chaque ligne non commentée est une **instruction** (les lignes en # sont des commentaires, qui doivent être sur leur propre ligne dans un Dockerfile) : FROM choisit la base, WORKDIR fixe le dossier, COPY copie des fichiers de votre machine vers l'image, RUN exécute une commande **pendant la construction**, CMD définit ce que fera le conteneur **au démarrage**. L'ordre des lignes n'est pas anodin : Docker garde en cache chaque étape, et la copie de requirements.txt **avant** celle du code permet de ne pas réinstaller les bibliothèques à chaque modification d'un script.

On construit puis on lance :

```
bash noexec docker build -t etude-ventes . # construit l'image à partir du
Dockerfile docker run --rm etude-ventes # lance un conteneur, le supprime
```

```
à la fin (--rm) docker run --rm -v "$PWD/rapports:/app/rapports" etude-ventes #
partage le dossier rapports/
```

La troisième commande montre un point essentiel : un conteneur est **éphémère**, ce qu'il écrit disparaît avec lui. Pour récupérer des résultats, on **monte** un dossier de votre machine dans le conteneur (option -v).

**Non exécuté, honnêtement.** Docker n'est pas disponible dans l'environnement où ce livre a été rédigé. Vérifions-le, plutôt que de le supposer :

```
command -v docker || echo "docker : absent de cet environnement"
```

```
docker : absent de cet environnement
```

Le Dockerfile et les commandes ci-dessus sont donc donnés **sans sortie**, et n'ont pas été testés ici. Ils suivent le modèle standard de la documentation de Docker ; si vous les essayez, une erreur de syntaxe ou de version est possible, et le message d'erreur vous guidera.

**Docker en data science, à quoi ça sert ?** (1) **Reproduire** une analyse des années plus tard, à l'identique. (2) **Déployer** un modèle sur un serveur (volume III). (3) **Travailler en équipe** sans que chacun installe la même pile logicielle. (4) Lancer facilement des logiciels complexes (une base de données, Jupyter) sans les installer : `docker run postgres`. Le coût : une courbe d'apprentissage et des images volumineuses (quelques centaines de Mo à plusieurs Go).

#### À retenir

- Un **script shell** range des commandes dans un fichier : shebang, arguments (\$1, \$#), if, for, \$(...). Commencez-le par `set -euo pipefail` pour qu'il échoue **bruyamment**.
- Pour tout ce qui est calcul ou logique complexe, passez à Python ; gardez le shell pour **enchaîner**.
- **cron** lance des commandes à heure fixe (5 champs de temps + commande).
- Plusieurs environnements virtuels peuvent contenir **des versions différentes** de la même bibliothèque ; `==` fige une version, `>=`, `<` la borne.
- **Docker** emballe un projet avec son système (image → conteneur). Ici : concepts et recette donnés, mais **non exécutés**.

## 6.5 ⊕ Pour aller plus loin : recherche reproductible

**Section optionnelle.** Elle rassemble les outils du chapitre pour répondre à une exigence centrale de la science des données : **n'importe qui doit pouvoir refaire votre analyse, et obtenir les mêmes résultats**. Vous y verrez comment produire un rapport **qui se régénère tout seul** à partir des données, des graines aléatoires, un fichier Makefile, R Markdown, Quarto et LaTeX.

### 6.5.1 Reproductible, répliquable : de quoi parle-t-on ?

Deux mots proches, deux idées différentes :

Terme	Question posée	Ce qu'on garde fixe	Ce qu'on change
<b>Reproductibilité</b>	« Si je relance <b>exactement</b> la même analyse, ai-je les mêmes chiffres ? »	données, code, environnement	rien (c'est une exigence <b>technique</b> )
<b>Répliquabilité</b>	« Si quelqu'un refait l'étude avec <b>de nouvelles données</b> , arrive-t-il à la même conclusion ? »	la méthode	les données (c'est une exigence <b>scientifique</b> )

On ne peut pas espérer la seconde sans la première : si je ne sais même pas refaire mes propres calculs, personne ne pourra juger si ma conclusion est solide. Une partie des « crises de réplication » observées dans plusieurs disciplines vient de simples **défauts de reproductibilité** : données modifiées à la main dans un tableur, numéros de version oubliés, code introuvable, étapes faites « à la souris ». Les outils des sections précédentes en sont le remède. Voici la **liste de contrôle** que nous allons dérouler :

Élément	Question à se poser	Outil (section)
<b>Données</b>	Ai-je gardé les données brutes intactes ? Sont-elles identifiables ?	dossier donnees/ en lecture seule, empreinte sha256 (6.5.6)
<b>Code</b>	Est-il versionné ? Retrouve-t-on la version qui a produit le résultat ?	Git (6.1)
<b>Aléa</b>	Mes simulations donnent-elles les mêmes nombres à chaque fois ?	graines aléatoires (6.5.2)
<b>Environnement</b>	Quelles versions de Python et des bibliothèques ?	requirements.txt (6.3), relevé de versions (6.5.3)
<b>Exécution</b>	Les étapes s'enchaînent-elles sans intervention manuelle ?	scripts, Makefile (6.4, 6.5.5)
<b>Rapport</b>	Les chiffres du texte viennent-ils <i>directement</i> du code ?	documents dynamiques (6.5.4)

### 6.5.2 L'aléa maîtrisé : les graines

Beaucoup d'analyses font appel au hasard : échantillonnage, simulation, validation croisée, bootstrap (3.7), initialisation de modèles. Or un ordinateur ne tire pas vraiment au hasard : il calcule une suite de nombres qui a l'air aléatoire à partir d'un point de départ, la **graine** (*seed*). Même graine, même suite. Sans graine explicite, Python en choisit une différente à chaque exécution (à partir de l'horloge, par exemple), et les résultats changent.

Voici ce que cela donne pour la moyenne d'un petit échantillon tiré au hasard dans les montants des commandes :

```
import numpy as np
import pandas as pd

montants = pd.read_csv("donnees/commandes.csv")["montant"].to_numpy()

def moyenne_echantillon(rng, taille=10):
 return montants[rng.choice(len(montants), size=taille, replace=False)].mean()
```

```
Sans graine : chaque exécution tire des nombres différents
a = moyenne_echantillon(np.random.default_rng())
b = moyenne_echantillon(np.random.default_rng())
print("sans graine, deux exécutions identiques ?", a == b)

Avec graine : la même suite à chaque exécution
c = moyenne_echantillon(np.random.default_rng(42))
d = moyenne_echantillon(np.random.default_rng(42))
print("graine 42 deux fois :", round(c, 2), "et", round(d, 2), "-> identiques ?", c == d)

sans graine, deux exécutions identiques ? False
graine 42 deux fois : 46.55 et 46.55 -> identiques ? True
```

Sans graine, les deux tirages diffèrent (à une coïncidence près, de probabilité quasi nulle) ; avec la même graine 42, on obtient **exactement** le même nombre à chaque fois, sur n'importe quel ordinateur (pour la même version de NumPy). C'est ce qui rend les nombres du livre reproductibles.

Reste à savoir **quelle graine choisir**. La réponse est contre-intuitive : *n'importe laquelle*, mais **sans la choisir en regardant le résultat**. Changer de graine jusqu'à obtenir le résultat qui nous arrange est une forme de triche appelée *p-hacking* (voir 3.5). La graine sert à **figer** une exécution, pas à l'améliorer. Pour vérifier qu'un résultat n'est pas un accident de graine, on l'observe pour **plusieurs graines** :

```
moyennes = [moyenne_echantillon(np.random.default_rng(graine)) for graine in range(1, 9)]
print("moyennes pour les graines 1 à 8 :", [round(float(m), 1) for m in moyennes])
print("vraie moyenne de la population :", round(montants.mean(), 2))

moyennes pour les graines 1 à 8 : [54.1, 60.5, 45.2, 63.4, 71.8, 73.3, 59.5, 43.3]
vraie moyenne de la population : 60.25
```

Les moyennes d'échantillons de 10 commandes varient fortement d'une graine à l'autre (voir l'erreur-type, 3.2) : la graine 42 n'a donc rien de spécial, et une conclusion qui ne tiendrait que pour elle serait suspecte. Deux règles pratiques : (1) créer **un seul** générateur `rng = np.random.default_rng(graine)` en début de programme, et le passer aux fonctions qui en ont besoin (comme ci-dessus), plutôt que de multiplier les graines cachées ; (2) se souvenir que chaque bibliothèque a sa propre source d'aléa (random de Python, NumPy, et plus tard PyTorch ou scikit-learn avec leur paramètre `random_state`) : il faut fixer **chacune** de celles qu'on utilise.

**Une graine ne garantit pas l'identité entre versions.** Les mêmes graine et code peuvent donner des tirages différents si la **version** de NumPy change (les algorithmes évoluent). D'où la nécessité de noter aussi les versions, juste après.

### 6.5.3 Relever l'environnement

Le fichier `requirements.txt` (6.3) fige les bibliothèques à *installer*. On peut aussi, en fin de rapport, **imprimer ce qui a réellement servi**. Cela prend quelques lignes et sauve bien des enquêtes :

```
import sys
import importlib.metadata as meta

def empreinte_environnement(paquets=("numpy", "pandas", "scipy", "matplotlib")):
 lignes = [f"Python {sys.version.split()[0]}"]
 lignes += [f"{p} {meta.version(p)}" for p in paquets]
 return lignes

print("\n".join(empreinte_environnement()))
```

```
Python 3.13.3
numpy 2.5.3
pandas 3.0.6
scipy 1.18.1
matplotlib 3.11.2
```

La sortie est le **relevé des versions** de cette exécution. Vous pouvez l'ajouter au bas de chaque rapport (nous le ferons juste après) ; en R, la fonction `sessionInfo()` joue ce rôle.

### 6.5.4 Un rapport qui se fabrique tout seul

Le défaut le plus courant d'un rapport : on calcule un chiffre dans un notebook, on le **recopie à la main** dans un document Word, et trois semaines plus tard on met à jour les données sans mettre à jour le texte. Le remède s'appelle le **document dynamique** (*literate programming*, « programmation lettrée ») : le texte et les chiffres sont produits **par le même programme**, donc ils ne peuvent pas diverger.

Nous allons construire cette chaîne, étape par étape, avec ce que nous avons déjà :

```
donnees/commandes.csv → src/faire_rapport.py → rapport.md → pandoc → HTML, PDF
 (données) (code : calcule et (texte et chiffres (mise en forme)
 écrit le texte) cohérents)
```

Première étape : un programme Python qui **écrit** un document Markdown en y insérant les chiffres calculés. Le format de sortie est du Markdown, le même que celui qui sert à écrire ce livre.

```
cd ~/atelier/etude-ventes
cat > src/faire_rapport.py <<'FIN'
"""Écrit sur la sortie standard un rapport Markdown : python src/faire_rapport.py commandes.csv"""
import sys
import numpy as np
import pandas as pd

def fr(x, decimales=2):
 """Nombre au format français : virgule décimale."""
 return f"{x:.{decimales}f}".replace(".", ",")

df = pd.read_csv(sys.argv[1])
n = len(df)
m = df["montant"]
marge = 1.96 * m.std() / np.sqrt(n) # demi-largeur de l'intervalle de confiance à 95 %
par_canal = df.groupby("canal")["montant"].agg(["count", "mean"]).sort_index()

print("---")
print('title: "Les ventes de Dar Jasmin"')
print("lang: fr")
print("---")
print()
print("## Résumé")
print()
print(f"Le fichier contient **{n} commandes**. Le montant moyen est de **{fr(m.mean())} DT** "
 f"(intervalle de confiance à 95 % : de {fr(m.mean() - marge)} à {fr(m.mean() + marge)} DT), "
 f"et la médiane de **{fr(m.median())} DT**.")
print()
print(f"L'intervalle est calculé par $\frac{1.96}{\sqrt{n}}$ ")
print()
print("## Par canal de vente")
print()
print("| Canal | Commandes | Montant moyen (DT) |")
print("|:-----|:-----:|:-----:|")
for canal, ligne in par_canal.iterrows():
```



```

 print(f"| {canal} | {int(ligne['count'])} | {fr(ligne['mean'])} |")
print()
meilleur = par_canal["mean"].idxmax()
print(f"Le canal au panier moyen le plus élevé est **{meilleur}**.")
FIN
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
cat rapports/rapport-ventes.md

```

```

title: "Les ventes de Dar Jasmin"
lang: fr

Résumé

Le fichier contient **400 commandes**. Le montant moyen est de **60,25 DT** (intervalle de confiance à
↳ 95 % : de 56,52 à 63,97 DT), et la médiane de **51,00 DT**.

L'intervalle est calculé par $\bar{x} \pm 1{,}96 \cdot \frac{s}{\sqrt{n}}$.

Par canal de vente

| Canal | Commandes | Montant moyen (DT) |
|:-----|:-----|:-----|
| Boutique | 114 | 74,81 |
| Instagram | 138 | 49,01 |
| Site | 148 | 59,50 |

Le canal au panier moyen le plus élevé est **Boutique**.

```

Le programme n'écrit **aucun chiffre en dur** : le nombre de commandes, la moyenne, l'intervalle, le tableau et même la phrase sur le « meilleur canal » sont calculés. Si les données changent demain, il suffit de relancer. (Les doubles barres `\` dans le texte du programme sont des échappements Python pour obtenir une seule barre `\` dans le LaTeX de la formule.) Le résultat est un fichier Markdown propre. **Pandoc**, le « couteau suisse » de la conversion de documents (installé avec ce livre), le transforme en d'autres formats. Convertissons-le en page web HTML, vérifions son contenu en texte brut, et fabriquons un PDF :

```

pandoc --standalone --mathml rapports/rapport-ventes.md -o rapports/rapport-ventes.html
pandoc rapports/rapport-ventes.md -t plain | head -n 14
pandoc rapports/rapport-ventes.md -o rapports/rapport-ventes.pdf --pdf-engine=xelatex
head -c 5 rapports/rapport-ventes.pdf; echo

```

```

[WARNING] Could not convert TeX math \bar{x} \pm 1{,}96 \cdot \frac{s}{\sqrt{n}}, rendering as TeX
Résumé

```

```

Le fichier contient 400 commandes. Le montant moyen est de 60,25 DT
(intervalle de confiance à 95 % : de 56,52 à 63,97 DT), et la médiane de
51,00 DT.

```

```

L'intervalle est calculé par $\bar{x} \pm 1{,}96 \cdot \frac{s}{\sqrt{n}}$.

```

```

Par canal de vente

```

Canal	Commandes	Montant moyen (DT)
Boutique	114	74,81
Instagram	138	49,01

```
%PDF-
```

L'option `--mathml` demande à Pandoc de convertir la formule en MathML, une écriture que les navigateurs savent afficher. La commande `-t plain` montre le texte tel qu'un lecteur le verrait en

texte brut : le tableau est mis en forme, et Pandoc **avertit** ([WARNING]) qu'il ne sait pas écrire une fraction avec de simples caractères et laisse donc la formule en LaTeX. C'est normal : un format purement textuel n'a pas de fraction. Les cinq premiers octets du fichier PDF sont %PDF- : c'est la signature de ce format, preuve que le PDF a bien été produit (ouvrez-le avec votre lecteur habituel pour voir la mise en page). Le moteur xelatex fabrique le PDF en passant par **LaTeX**, que nous verrons plus bas.

Pour ajouter le relevé des versions en bas du rapport, il suffirait d'une petite fonction (celle du 6.5.3) dans `faire_rapport.py`. Retenez le principe : **pas de date du jour ni de valeur aléatoire non figée dans un rapport reproductible**, sinon deux exécutions ne produisent plus le même fichier.

### 6.5.5 make : ne refaire que ce qui a changé

Tant que la chaîne tient en deux commandes, on les retape. Mais un vrai projet compte des dizaines d'étapes : nettoyage, tableaux, graphiques, rapport. Relancer **tout** à chaque modification est lent ; relancer **à la main** les bonnes étapes est source d'oublis. L'outil **make** (né en 1976, toujours en pleine forme) résout les deux problèmes. On lui décrit dans un fichier nommé `Makefile` des **règles** de la forme :

```
cible: dépendances
<TAB>commande qui fabrique la cible à partir des dépendances
```

`make` compare les **dates de modification** : si une dépendance est plus récente que la cible, il **refait** la cible ; sinon il ne fait rien. Voici le `Makefile` de notre projet. (Attention : l'indentation des commandes **doit être une vraie tabulation**, pas des espaces.)

```
cat > Makefile <<'FIN'
.PHONY: tout propre

tout: rapports/rapport-ventes.html

rapports/rapport-ventes.md: donnees/commandes.csv src/faire_rapport.py
<TAB>python src/faire_rapport.py donnees/commandes.csv > $@

rapports/rapport-ventes.html: rapports/rapport-ventes.md
<TAB>pandoc --standalone --mathml $< -o $@

propre:
<TAB>rm -f rapports/rapport-ventes.md rapports/rapport-ventes.html
FIN
sed -i 's/^(<TAB>)/t/' Makefile
make propre
make
echo "--- deuxième appel, rien n'a changé ---"
make
echo "--- on touche le fichier de données ---"
touch donnees/commandes.csv
make

rm -f rapports/rapport-ventes.md rapports/rapport-ventes.html
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
pandoc --standalone --mathml rapports/rapport-ventes.md -o rapports/rapport-ventes.html
--- deuxième appel, rien n'a changé ---
make: Nothing to be done for 'tout'.
--- on touche le fichier de données ---
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
pandoc --standalone --mathml rapports/rapport-ventes.md -o rapports/rapport-ventes.html
```

(Dans le fichier, nous avons écrit `<TAB>` puis la commande `sed` le remplace par une vraie tabula-

tion, pour que l'exemple soit sans ambiguïté.) Lisons les règles. La première cible, tout, est celle qu'on construit par défaut ; elle dépend du rapport HTML. Le rapport HTML dépend du fichier Markdown ; le fichier Markdown dépend des **données** et du **programme** qui le fabrique. Dans les commandes, \$@ désigne la cible, \$< la première dépendance. Cette fois, **make affiche chaque commande qu'il lance** : on voit qu'après make propre (qui supprime les fichiers produits), le premier make refait les **deux** étapes, le deuxième make répond qu'il n'y a **rien à faire**, et que, quand on « touche » le fichier de données (ce qui met à jour sa date), les **deux** étapes sont de nouveau déclenchées.

**Le grand intérêt.** Il suffit de modifier le texte du rapport, et seule la conversion se refait. Dans un grand projet, c'est un gain de temps considérable, et, surtout, **la documentation de la chaîne de traitement** est le Makefile lui-même : un lecteur voit d'un coup d'œil de quoi dépend quoi. Le README du projet peut alors se résumer à : « Pour tout reproduire : make ».

### 6.5.6 Les outils « tout-en-un » : R Markdown et Quarto

Notre chaîne maison (Python qui écrit du Markdown, puis Pandoc) fonctionne, mais le monde de la data science a standardisé la même idée dans des outils dédiés : un **seul fichier** mélange texte Markdown et **blocs de code** ; à la compilation, le code est exécuté et **ses résultats sont insérés** dans le document final (HTML, PDF, Word, diaporama).

**R Markdown** (pour R, mais aussi Python) fonctionne ainsi : le fichier .Rmd contient un en-tête, du texte, des blocs de code {r} et du code R **en ligne** dans le texte (``r ...``). Comme le format utilise les trois accents graves que ce livre emploie lui-même pour afficher du code, nous écrivons ici les blocs avec des tildes `~~~`, puis une commande sed les remplace par les vrais accents graves avant la compilation :

```
mkdir -p ~/atelier/etude-rmd
cd ~/atelier/etude-rmd
cp ~/atelier/etude-ventes/donnees/commandes.csv .
cat > rapport.Rmd <<'FIN'

title: "Satisfaction des clients de Dar Jasmin"
output: html_document

Voici le lien entre le délai de livraison et la note de satisfaction.

~~~{r}
d <- read.csv("commandes.csv")
d <- d[d$canal != "Boutique", ]      # on ne garde que les commandes livrées
tapply(d$satisfaction, d$livraison <= 3, mean)
~~~

La satisfaction moyenne des commandes livrées est de round(mean(d$satisfaction), 2)
sur nrow(d) commandes.
FIN
sed -i 's/^~~~{r}$/{r}/; s/^~~~$/`r`/' rapport.Rmd
Rscript -e 'rmarkdown::render("rapport.Rmd", quiet = TRUE)'
ls
pandoc rapport.html -t plain
```

```
commandes.csv
rapport.Rmd
rapport.html
Satisfaction des clients de Dar Jasmin
```

Voici le lien entre le délai de livraison et la note de satisfaction.

```
d <- read.csv("commandes.csv")
d <- d[d$canal != "Boutique",] # on ne garde que les commandes livrées
tapply(d$satisfaction, d$livraison <= 3, mean)

FALSE TRUE
3.631313 4.034091
```

La satisfaction moyenne des commandes livrées est de 3.76 sur 286 commandes.

On y retrouve le même principe : le chiffre du texte (le code `r round(...)` écrit entre accents graves, au milieu de la phrase) est **calculé** à la compilation, et le bloc de code est exécuté, avec son résultat inséré. Le bloc de code a été exécuté lors de la compilation, et sa sortie est insérée sous lui (les lignes qui commencent par `##`). `tapply` calcule la satisfaction moyenne séparément pour `FALSE` (livraison de plus de 3 jours : environ 3,63) et `TRUE` (3 jours ou moins : environ 4,03) : les clients livrés rapidement sont plus satisfaits, ce qui est conforme à l'intuition. Les chiffres du paragraphe suivant (la moyenne générale et le nombre de commandes) sont eux aussi calculés par le code R en ligne. Notez le détail : R affiche un point décimal (3.76), alors que notre rapport Python écrivait une virgule grâce à sa petite fonction `fr()`.

**Quarto** est le successeur de R Markdown : même idée, mais indépendant du langage (Python, R, Julia), avec davantage de formats (sites, livres, présentations). Un fichier `.qmd` ressemble à ceci :

```

title: "Les ventes de Dar Jasmin"
format: html

Le montant moyen est de `{python} round(moyenne, 2)` DT.

``{python}
import pandas as pd
df = pd.read_csv("donnees/commandes.csv")
moyenne = df["montant"].mean()
df.groupby("canal")["montant"].mean().round(2)
``
```

```
bash noexec quarto render rapport.qmd # produit rapport.html quarto
render rapport.qmd --to pdf # produit un PDF (via LaTeX)
```

**Non exécuté.** Quarto n'est pas installé dans l'environnement de rédaction : l'exemple `.qmd` et les deux commandes ci-dessus sont donnés **à titre d'illustration, sans avoir été testés ici**. (R Markdown, lui, a bien été exécuté ci-dessus.) La syntaxe exacte, notamment celle du code en ligne `{python}`, peut dépendre de la version de Quarto : consultez sa documentation si vous l'installez.

### 6.5.7 LaTeX : composer proprement les formules et les PDF

**LaTeX** (on prononce « latèk ») est le langage de composition de documents scientifiques : c'est lui qui met en forme les formules des chapitres 1 à 3 de ce livre. Vous l'avez déjà écrit sans le savoir : `\bar{x}`, `\sum`, `\frac{a}{b}` sont des commandes LaTeX. Un document complet se compose d'un **préambule** (les réglages) et d'un **corps**. Compilons un minuscule document, qui contient l'intervalle de confiance du chapitre 3 :

```
mkdir -p ~/atelier/mini-latex
cd ~/atelier/mini-latex
cat > mini.tex <<'FIN'
\documentclass{article}
```

```

\usepackage{fontspec}
\usepackage{amsmath}
\usepackage[french]{babel}
\begin{document}
\section*{Intervalle de confiance}
Pour un échantillon de taille n , de moyenne $\bar{x}$ et d'écart-type s ,
l'intervalle de confiance à 95%, de la moyenne est
\[
\bar{x} \pm 1.96 \frac{s}{\sqrt{n}} .
\]
\end{document}

```

**FIN**

```

xelatex -interaction=nonstopmode -halt-on-error mini.tex > compilation.log 2>&1
echo "code de sortie : $?"
head -c 5 mini.pdf; echo

```

```

code de sortie : 0
%PDF-

```

Le code de sortie 0 indique que la compilation a réussi, et le fichier `mini.pdf` commence bien par la signature `%PDF-`. Si une erreur survient (une accolade oubliée), `xelatex` s'arrête et la raison se trouve dans `compilation.log`. En pratique, vous n'écrirez presque jamais du LaTeX *complet* : vous le laisserez Pandoc ou Quarto générer à partir du Markdown, et vous n'écrirez à la main que les formules. Pour écrire à plusieurs un document LaTeX sans rien installer, le service en ligne **Overleaf** est très utilisé (non testé ici).

### 6.5.8 Garder une empreinte des données

Dernier maillon de la chaîne : s'assurer que les **données** n'ont pas changé en cachette. Une **empreinte cryptographique** (*hash*) est un court texte, calculé à partir du contenu d'un fichier : le moindre changement du fichier (même un seul chiffre) change complètement l'empreinte. C'est le principe que nous avons vu au 6.1.4 pour les objets de Git. On calcule celle de `commandes.csv`, on la **note dans le README**, et n'importe qui peut vérifier :

```

cd ~/atelier/etude-ventes
sha256sum donnees/commandes.csv > donnees/EMPREINTES.sha256
cat donnees/EMPREINTES.sha256
sha256sum -c donnees/EMPREINTES.sha256
echo "--- une faute de frappe glisse dans le fichier ---"
sed -i '2s/44.8/448.0/' donnees/commandes.csv
sha256sum -c donnees/EMPREINTES.sha256 || echo "ALERTE : les données ont été modifiées"

```

```

824d99115defe345920b4c8d4425fb4a1ab6b7a01e20984831cf564827e8d1b3 donnees/commandes.csv
donnees/commandes.csv: OK
--- une faute de frappe glisse dans le fichier ---
donnees/commandes.csv: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
ALERTE : les données ont été modifiées

```

La première vérification répond OK. Après avoir **modifié un seul nombre** (le premier montant, 44,8, devenu 448,0 par une faute de frappe), la vérification échoue : l'alerte sonne avant même qu'on lance l'analyse. Remettons le fichier en état :

```

sed -i '2s/448.0/44.8/' donnees/commandes.csv
sha256sum -c donnees/EMPREINTES.sha256

```

```

donnees/commandes.csv: OK

```

Le « **kit de reproductibilité** » d'un projet, en une page. Un dossier de projet bien tenu contient : un README (but, comment reproduire, version de Python), un fichier `requirements.txt`, les données brutes **non modifiées** avec leur empreinte, le code versionné avec Git, un Makefile (ou un script) qui enchaîne les étapes, et le rapport généré. Pour archiver une version définitive de manière pérenne, on dépose le tout sur un service qui attribue un identifiant stable (DOI), comme **Zenodo** (non testé ici).

### À retenir

- **Reproductible** = mêmes données + même code + même environnement  $\Rightarrow$  mêmes résultats. **Répliquable** = mêmes conclusions sur de nouvelles données.
- **Graine aléatoire** : fixer *une* graine par exécution, la choisir *avant* de voir les résultats, et vérifier sur plusieurs graines que la conclusion tient. Noter aussi les versions des bibliothèques.
- **Document dynamique** : le texte et les chiffres sortent du **même** programme (Python + Pandoc, R Markdown, Quarto) ; on ne recopie jamais un chiffre à la main.
- **make** ne refait que ce qui est périmé ; le Makefile documente la chaîne de traitement.
- **LaTeX** compose les formules et les PDF ; on le fait généralement générer par Pandoc ou Quarto.
- Une **empreinte** (sha256sum) détecte toute modification des données.

## 6.6 Exercices du chapitre 6

Cherchez d'abord par vous-même (en tapant les commandes dans un vrai terminal !), vérifiez ensuite, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les corrigés ont été **réellement exécutés** dans l'atelier ; leurs sorties sont celles du livre. Les exercices utilisent un dossier de travail neuf, `~/atelier/exercices`, et le fichier `commandes.csv`.

### Énoncés

**Exercice 1** ★ (**se repérer**). Créez le dossier `projet-yasmine` contenant deux sous-dossiers, `donnees` et `src`, copiez-y `commandes.csv` (dans `donnees`), puis (a) affichez l'arborescence créée, (b) affichez les **trois dernières** lignes du fichier, (c) comptez ses lignes.

**Exercice 2** ★ (**tuyaux**). Combien de commandes ont reçu chacune des notes de satisfaction (1 à 5) ? Répondez avec **une seule ligne** de commandes enchaînées par des tuyaux, puis vérifiez avec `pandas`.

**Exercice 3** ★ (**premiers pas avec Git**). Dans un nouveau dépôt `journal`, créez un fichier `notes.txt` contenant la ligne « Hypothèse : les commandes Instagram sont plus petites. » et enregistrez-le (`commit 1`). Ajoutez une seconde ligne « Test à faire : comparer les moyennes par canal. » et enregistrez (`commit 2`). Par maladresse, vous supprimez ensuite `notes.txt` avec `rm`. Retrouvez-le **sans refaire à la main**, puis affichez l'historique en une ligne par `commit`.

**Exercice 4** ★★ (**awk**). Calculez, pour chaque canal, le **pourcentage de commandes notées 4 ou 5** (colonne satisfaction), avec `awk`. Vérifiez avec `pandas`.

**Exercice 5** ★★ (**conflit**). Un fichier `params.txt` contient la ligne `seuil=50`. Sur une branche `prudent`, vous passez la valeur à `seuil=40` ; sur `main`, votre collègue la passe à `seuil=60`. Fusionnez `prudent` dans `main`. Que se passe-t-il ? Résolvez le conflit en gardant la valeur la **plus élevée**, et terminez la fusion.

**Exercice 6 ★★ (état caché).** Un notebook contient, de haut en bas, les cinq cellules suivantes :

(A) `n = 100` ; (B) `taux = 0.19` ; (C) `tva = n * taux` ; (D) `n = 200` ; (E) `print(tva)`. Yasmine les exécute dans l'ordre **A, B, D, C, E** (elle a lancé D avant C). (a) Qu'affiche E ? (b) Qu'afficherait E après « Restart & Run All » ? (c) Quelle conclusion en tirez-vous ?

**Exercice 7 ★★ (environnement).** Créez un environnement virtuel `env-a`, installez-y `tabulate==0.8.9` et enregistrez les versions dans `requirements.txt`. Reconstituez ensuite, dans un second environnement `env-b`, **exactement les mêmes** bibliothèques à partir de ce seul fichier, et prouvez qu'elles sont identiques.

**Exercice 8 ★★ (script shell).** Écrivez un script `compte.sh` qui prend **deux arguments**, un fichier CSV et un numéro de colonne, et affiche le nombre d'occurrences de chaque valeur de cette colonne (sans l'en-tête). Il doit s'arrêter avec un message et un code d'erreur non nul si on lui donne un mauvais nombre d'arguments ou un fichier inexistant. Testez-le sur la colonne 1 puis sur la colonne 4.

**Exercice 9 ★★★ (Makefile).** Écrivez un Makefile dont la cible `resume.txt` dépend de `commandes.csv` et du script `resume.py`, et qui la fabrique par `python resume.py commandes.csv > resume.txt`. Montrez que : (a) un premier `make` construit la cible, (b) un deuxième ne fait rien, (c) modifier le script (date de modification) relance la construction, (d) modifier un fichier **sans rapport** (`LISEZMOI.txt`) ne la relance pas.

**Exercice 10 ★★★ (synthèse : un projet reproductible).** Montez un mini-projet « rapport sur les ventes » qui réunit tout le chapitre : dépôt Git avec `.gitignore`, données, script de rapport, Makefile, empreinte sha256 des données, un commit étiqueté `v1.0`. Démontrez sa reproductibilité : **clonez** le dépôt dans un autre dossier, vérifiez l'empreinte des données, lancez `make`, et prouvez que le rapport obtenu est **identique octet pour octet** à l'original.

## Corrigés

**Corrigé 1.** `mkdir -p` crée d'un coup les dossiers imbriqués (et l'accolade en crée deux) ; `find` affiche l'arborescence ; `tail -n 3` et `wc -l` répondent à (b) et (c).

```
mkdir -p ~/atelier/exercices
cd ~/atelier/exercices
mkdir -p projet-yasmine/{donnees,src}
cp ~/atelier/dar-jasmin/commandes.csv projet-yasmine/donnees/
find projet-yasmine | sort
tail -n 3 projet-yasmine/donnees/commandes.csv
wc -l projet-yasmine/donnees/commandes.csv
```

```
projet-yasmine
projet-yasmine/donnees
projet-yasmine/donnees/commandes.csv
projet-yasmine/src
Site,62.6,4,3
Instagram,37.5,3,4
Site,31.4,4,4
401 projet-yasmine/donnees/commandes.csv
```

Le fichier compte 401 lignes : 400 commandes plus l'en-tête.

**Corrigé 2.** La note est dans la 4<sup>e</sup> colonne. On enlève l'en-tête (`tail -n +2`), on extrait la colonne (`cut`), on trie (indispensable avant `uniq`), puis on compte les groupes (`uniq -c`) :

```
cd ~/atelier/exercices/projet-yasmine/donnees
tail -n +2 commandes.csv | cut -d, -f4 | sort -n | uniq -c
```



```

1 1
15 2
85 3
195 4
104 5

```

Vérification avec pandas (même résultat attendu) :

```

import pandas as pd
df = pd.read_csv("donnees/commandes.csv")
print(df["satisfaction"].value_counts().sort_index())

```

```

satisfaction
1 1
2 15
3 85
4 195
5 104
Name: count, dtype: int64

```

Les deux méthodes donnent les mêmes effectifs : la note 4 est la plus fréquente (195 commandes sur 400), et les notes très basses sont rares (une seule note 1), ce qui est cohérent avec une satisfaction moyenne proche de 4 (3.1). Le total fait bien 400.

**Corrigé 3.** `git restore` ramène un fichier supprimé ou modifié à son état du dernier commit. Comme notre `rm` a supprimé le fichier **sans** l'enregistrer, c'est la bonne commande :

```

cd ~/atelier/exercices
mkdir journal
cd journal
git init -q
echo "Hypothèse : les commandes Instagram sont plus petites." > notes.txt
git add notes.txt
git commit -q -m "Première hypothèse"
echo "Test à faire : comparer les moyennes par canal." >> notes.txt
git commit -q -am "Ajout du test à faire"
rm notes.txt
ls
git restore notes.txt
cat notes.txt
git log --oneline

```

```

Hypothèse : les commandes Instagram sont plus petites.
Test à faire : comparer les moyennes par canal.
07f3ebd Ajout du test à faire
7c4ddf0 Première hypothèse

```

Après le `rm`, `ls` n'affiche rien (le fichier a disparu) ; après `git restore`, le fichier est revenu avec **ses deux lignes**, car elles figuraient dans le dernier commit. Le tout a été possible parce qu'on avait enregistré : un fichier jamais commité aurait été perdu.

**Corrigé 4.** On compte, pour chaque canal, le nombre total de commandes (`n`) et le nombre de commandes notées 4 ou 5 (bons) ; à la fin, on affiche le pourcentage. (Dans `awk`, une variable ou une case de tableau jamais utilisée vaut 0, donc `bons[c]` fonctionne même si un canal n'a aucune bonne note.)

```

cd ~/atelier/exercices/projet-yasmine/donnees
awk -F, 'NR > 1 { n[$1]++; if ($4 >= 4) bons[$1]++ }
 END { for (c in n) printf "%-10s %.1f %%\n", c, 100 * bons[c] / n[c] }' commandes.csv | sort

```



Boutique	95.6 %
Instagram	63.0 %
Site	69.6 %

Et la vérification avec pandas :

```
satisfaits = (df["satisfaction"] >= 4).groupby(df["canal"]).mean() * 100
print(satisfaits.round(1))
```

```
canal
Boutique 95.6
Instagram 63.0
Site 69.6
Name: satisfaction, dtype: float64
```

Les deux approches coïncident. Les clients de la **boutique** sont les plus satisfaits : pas de délai de livraison, donc pas de pénalité (voir la construction du jeu de données au 3.1.2, où la note diminue avec le délai).

**Corrigé 5.** Les deux branches ont modifié **la même ligne** : Git ne peut pas choisir, il signale un conflit. On édite le fichier pour garder `seuil=60`, on déclare le conflit résolu avec `git add`, puis on termine par un `commit`.

```
cd ~/atelier/exercices
mkdir conflit
cd conflit
git init -q
echo "seuil=50" > params.txt
git add params.txt
git commit -q -m "Paramètre initial"
git switch -q -c prudent
echo "seuil=40" > params.txt
git commit -q -am "Seuil prudent"
git switch -q main
echo "seuil=60" > params.txt
git commit -q -am "Seuil ambitieux"
git merge --no-edit prudent
echo "----- fichier en conflit -----"
cat params.txt
```

```
Auto-merging params.txt
CONFLICT (content): Merge conflict in params.txt
Automatic merge failed; fix conflicts and then commit the result.
----- fichier en conflit -----
<<<<<< HEAD
seuil=60
=====
seuil=40
>>>>>> prudent
```

Le fichier contient maintenant les marqueurs `<<<<<<`, `=====` et `>>>>>>` autour des deux versions. On résout en écrivant la valeur voulue (ici nous la réécrivons directement, ce qui revient à supprimer les marqueurs et la ligne `seuil=40`) :

```
echo "seuil=60" > params.txt
git add params.txt
git commit -q --no-edit
git log --oneline --graph
cat params.txt
```

```
* a1b9309 Merge branch 'prudent'
|\
```

```
| * fd20715 Seuil prudent
* | 6669eef Seuil ambitieux
|/
* ded7106 Paramètre initial
seuil=60
```

Le graphe montre bien la **fusion** de deux lignes de travail en un commit à deux parents. N'oubliez pas, dans un vrai projet, de **relancer le code** avant de valider.

**Corrigé 6.** On simule le noyau avec un dictionnaire, comme au 6.2.3. Le point clé est l'**ordre d'exécution**, pas l'ordre d'écriture :

```
def jouer(ordre):
 memoire = {}
 cellules = {"A": "n = 100", "B": "taux = 0.19", "C": "tva = n * taux",
 "D": "n = 200", "E": "print(round(tva, 2))"}
 for nom in ordre:
 exec(cellules[nom], memoire)

print("(a) ordre A, B, D, C, E :", end=" ")
jouer("ABDCE")
print("(b) Restart & Run All (A, B, C, D, E) :", end=" ")
jouer("ABCDE")
```

```
(a) ordre A, B, D, C, E : 38.0
(b) Restart & Run All (A, B, C, D, E) : 19.0
```

- (a) Dans l'ordre réellement joué,  $n$  vaut déjà 200 quand C calcule la TVA : E affiche  $200 \times 0,19 = 38$ . (b) Dans l'ordre du fichier, C est exécutée alors que  $n$  vaut encore 100 : E affiche  $100 \times 0,19 = 19$ . (c) Le **même notebook** donne deux résultats différents selon l'ordre d'exécution : le chiffre que Yasmine voyait à l'écran (38) n'est pas celui qu'obtiendra quiconque ouvrira le fichier et exécutera tout (19). C'est exactement l'état caché du 6.2.3 : **toujours** redémarrer et tout exécuter avant de se fier à un résultat. (Une analyse correcte de la logique du carnet dirait aussi que la cellule D, placée *après* C mais qui change  $n$ , est probablement mal placée.)

**Corrigé 7.** On crée le premier environnement, on y installe la version exacte demandée, puis on fige ; le second environnement est reconstruit uniquement depuis `requirements.txt`.

```
cd ~/atelier/exercices
mkdir env-demo
cd env-demo
python -m venv env-a
env-a/bin/pip install --quiet "tabulate==0.8.9" 2>&1 | grep -v -i -E "notice|warning"
env-a/bin/pip freeze > requirements.txt
cat requirements.txt
python -m venv env-b
env-b/bin/pip install --quiet -r requirements.txt 2>&1 | grep -v -i -E "notice|warning"
env-b/bin/pip freeze > freeze-b.txt
cmp requirements.txt freeze-b.txt && echo "les deux environnements sont identiques"
```

```
tabulate==0.8.9
les deux environnements sont identiques
```

La commande `cmp` compare deux fichiers octet par octet et ne dit rien s'ils sont identiques ; notre `&& echo` confirme alors le succès. (On appelle ici directement `env-a/bin/pip` au lieu d'activer l'environnement : c'est strictement équivalent, cf. 6.4.4.)

**Corrigé 8.** Le script vérifie ses arguments (`$#`), l'existence du fichier (`-f`), puis enchaîne les outils du 6.3 : `tail` (supprimer l'en-tête), `cut` (extraire la colonne), `sort`, `uniq` -c.

```

cd ~/atelier/exercices/projet-yasmine
cat > src/compte.sh <<'FIN'
#!/usr/bin/env bash
compte.sh : effectifs de chaque valeur d'une colonne d'un CSV
usage : bash src/compte.sh fichier.csv numero_de_colonne
set -euo pipefail
if ["$#" -ne 2]; then
 echo "usage : $0 fichier.csv numero_de_colonne" >&2
 exit 2
fi
if [! -f "$1"]; then
 echo "fichier introuvable : $1" >&2
 exit 1
fi
tail -n +2 "$1" | cut -d, -f"$2" | sort | uniq -c
FIN
echo "--- colonne 1 (canal) ---"
bash src/compte.sh donnees/commandes.csv 1
echo "--- colonne 4 (satisfaction) ---"
bash src/compte.sh donnees/commandes.csv 4
echo "--- erreurs ---"
bash src/compte.sh donnees/commandes.csv; echo "code : $?"
bash src/compte.sh absent.csv 1; echo "code : $?"

```

```

--- colonne 1 (canal) ---
114 Boutique
138 Instagram
148 Site
--- colonne 4 (satisfaction) ---
1 1
15 2
85 3
195 4
104 5
--- erreurs ---
usage : src/compte.sh fichier.csv numero_de_colonne
code : 2
fichier introuvable : absent.csv
code : 1

```

Les effectifs de la colonne 4 sont ceux de l'exercice 2, et ceux de la colonne 1 retrouvent les 148, 138 et 114 commandes du 6.3. Les deux appels fautifs affichent leur message et renvoient des codes 2 (mauvais usage) et 1 (fichier absent), comme convenu.

**Corrigé 9.** Le Makefile décrit la dépendance de `resume.txt` à ses deux sources. Pour l'écrire sans ambiguïté sur les tabulations, nous employons encore la substitution `<TAB>` du 6.5.5.

```

cd ~/atelier/exercices
mkdir make-demo
cd make-demo
cp ~/atelier/etude-ventes/src/resume.py .
cp ~/atelier/dar-jasmin/commandes.csv .
echo "Notes de lecture" > LISEZMOI.txt
cat > Makefile <<'FIN'
resume.txt: commandes.csv resume.py
<TAB>python resume.py commandes.csv > resume.txt
FIN
sed -i 's/^\<TAB>/\t/' Makefile
echo "--- (a) premier make ---"
make
echo "--- (b) deuxième make ---"
make
echo "--- (c) on touche resume.py ---"
touch resume.py

```

```
make
echo "--- (d) on touche LISEZMOI.txt ---"
touch LISEZMOI.txt
make
cat resume.txt
```

```
--- (a) premier make ---
python resume.py commandes.csv > resume.txt
--- (b) deuxième make ---
make: 'resume.txt' is up to date.
--- (c) on touche resume.py ---
python resume.py commandes.csv > resume.txt
--- (d) on touche LISEZMOI.txt ---
make: 'resume.txt' is up to date.
400 lignes, 4 colonnes
 montant livraison satisfaction
mean 60.25 3.29 3.96
min 8.60 0.00 1.00
max 255.70 13.00 5.00
```

- (a) Le premier make exécute la commande ; (b) le deuxième répond que la cible est déjà à jour ; (c) en modifiant la date du script (ce qui simule une modification), make détecte que la dépendance est plus récente que la cible et **refabrique** ; (d) LISEZMOI.txt n'est pas une dépendance : make n'y réagit pas. Seul ce qui est **déclaré** comme dépendance déclenche une reconstruction, d'où l'importance de **lister toutes** les sources dans la règle.

**Corrigé 10.** C'est le projet de synthèse du chapitre. On le construit pas à pas, puis on le **clone** pour le rejouer ailleurs. D'abord le projet, avec son script de rapport (très simple, il n'écrit ni date ni valeur aléatoire), son Makefile et l'empreinte des données :

```
cd ~/atelier/exercices
mkdir ventes-reproductibles
cd ventes-reproductibles
git init -q
mkdir donnees src rapports
cp ~/atelier/dar-jasmin/commandes.csv donnees/
sha256sum donnees/commandes.csv > donnees/EMPREINTES.sha256
cat > src/rapport.py <<'FIN'
import sys
import pandas as pd

df = pd.read_csv(sys.argv[1])
print("# Rapport sur les ventes")
print()
print(f"- Commandes : {len(df)}")
print(f"- Montant moyen : {df['montant'].mean():.2f} DT")
print(f"- Satisfaction moyenne : {df['satisfaction'].mean():.2f} / 5")
FIN
cat > Makefile <<'FIN'
rapports/rapport.md: donnees/commandes.csv src/rapport.py
<TAB>python src/rapport.py donnees/commandes.csv > $@
FIN
sed -i 's/^<TAB>/\t/' Makefile
printf '.venv/\n__pycache__/\n*.env\n' > .gitignore
printf 'Pour tout reproduire : sha256sum -c donnees/EMPREINTES.sha256 && make\n' > README.md
make
cat rapports/rapport.md
```

```
python src/rapport.py donnees/commandes.csv > rapports/rapport.md
Rapport sur les ventes

- Commandes : 400
```

```
- Montant moyen : 60.25 DT
- Satisfaction moyenne : 3.96 / 5
```

Puis on enregistre dans Git : le **rapport produit** est lui aussi versionné ici pour pouvoir le comparer, mais en pratique on ne versionne que les sources (6.1.11). On pose l'étiquette v1.0 :

```
git add .
git commit -q -m "Projet de rapport reproductible"
git tag -a v1.0 -m "Version livrée"
git log --oneline
git status --short
```

```
295a1b8 Projet de rapport reproductible
```

Il ne reste rien d'« en attente » (`git status` est vide). Passons à la **preuve** : un collègue clone le dépôt dans un dossier tout neuf, vérifie l'intégrité des données, reconstruit le rapport (après avoir supprimé le rapport cloné, pour être sûr que `make` le refabrique), et compare :

```
cd ~/atelier/exercices
git clone -q ventes-reproductibles colleague
cd colleague
git checkout -q v1.0
sha256sum -c donnees/EMPREINTES.sha256
rm rapports/rapport.md
make
cmp rapports/rapport.md ../ventes-reproductibles/rapports/rapport.md && echo "rapports identiques,
↪ octet pour octet"
```

```
donnees/commandes.csv: OK
python src/rapport.py donnees/commandes.csv > rapports/rapport.md
rapports identiques, octet pour octet
```

Les données sont intègres (OK), le rapport a été refabriqué par `make`, et la comparaison `cmp` confirme qu'il est **identique** à l'original. C'est la définition opérationnelle de la reproductibilité : *un tiers, sur un dossier neuf, obtient exactement le même résultat avec une commande*. Chaque élément du chapitre y a joué son rôle : Git (le code et ses versions), le `.gitignore`, le `Makefile`, l'empreinte des données, et, pour l'environnement, le `requirements.txt` que l'on ajouterait dans un vrai projet (ici l'environnement est celui du livre).

## Bilan du chapitre 6

Vous savez maintenant :

- **versionner** votre travail avec Git : photographier (`add`, `commit`), comparer (`diff`), défaire (`restore`, `revert`), travailler en parallèle (branches, fusions, conflits) et partager (dépôts distants) ;
- **utiliser un notebook** pour explorer et raconter, **sans tomber dans le piège de l'état caché** (Restart & Run All, détecteur d'ordre, ne pas versionner les sorties) ;
- **vous servir du terminal** : vous repérer, lire, filtrer et résumer des fichiers avec `grep`, `cut`, `sort`, `uniq` et `awk`, assembler des commandes avec les tuyaux et les redirections, comprendre les codes de sortie ;
- **isoler chaque projet** dans un environnement virtuel et **figer** ses dépendances dans `requirements.txt` ;
- (en option) **automatiser** avec des scripts shell et `make`, comprendre Docker, fabriquer un **rapport dynamique** (Python + Pandoc, R Markdown, Quarto, LaTeX), maîtriser les graines

aléatoires et surveiller l'intégrité des données avec une empreinte.

Le fil rouge de tout le chapitre tient en une phrase : **un résultat n'existe que s'il peut être refait**. Vous disposez désormais de l'ensemble des fondations du volume : les mathématiques (chapitre 1), les probabilités (2), la statistique (3), la programmation (4), les bases de données (5) et les outils de travail (6). Il est temps de tout assembler dans le **projet du volume** : une étude complète des ventes de Dar Jasmin, des données brutes jusqu'au rapport.

# Projet du volume : l'étude Dar Jasmin 2025

« Un projet de data science, ce n'est pas un modèle. C'est une **question**, des données, une réponse honnête... et un message que quelqu'un pourra lire. »

Six chapitres : des maths, des probabilités, de la statistique, du Python, du SQL, des outils. Chaque brique a été vue séparément, sur de petits exemples. Ce projet les **assemble** dans une seule étude de bout en bout, comme on le ferait dans un vrai travail : on reçoit une demande floue, on va chercher les données, on les contrôle, on les explore, on répond avec rigueur, et on rédige un rapport qu'une personne non spécialiste peut lire.

**Comment lire ce projet.** Il n'introduit **aucune notion nouvelle** : chaque étape renvoie à la section du livre qui l'explique. Si vous bloquez sur une étape, c'est le signal d'aller relire cette section, pas de tout recommencer. Le meilleur usage : lire d'abord le cahier des charges (P.1), **fermer le livre**, essayer de répondre à Yasmine avec vos propres moyens, puis comparer.

## P.1 Le cahier des charges

Fin décembre 2025. Yasmine, la propriétaire de Dar Jasmin, vous envoie ce message :

*« Bonjour ! L'année est finie et j'ai un peu le vertige : des commandes partout, trois canaux de vente, et aucune idée de ce qui marche vraiment. Quatre questions me trottent dans la tête pendant les fêtes :*

- 1. Comment s'est passée mon année 2025 ? (chiffre d'affaires, rythme, canaux)*
- 2. Instagram me prend beaucoup de temps. Est-ce que ces clients dépensent vraiment moins que ceux de la boutique, ou est-ce une impression ?*
- 3. Je soupçonne que les retards de livraison font baisser la satisfaction. Est-ce vrai, et de combien ?*
- 4. Qui dois-je relancer en janvier ?*

*Je n'ai besoin ni de formules ni de jargon : juste des chiffres fiables et ce que vous me conseillez. Merci ! »*

Transformer ce message en travail demande une **méthode**. La nôtre tient en six étapes, que nous suivrons dans l'ordre :

Étape	Question que l'on se pose	Outils du livre
1. Charger et contrôler	« Les données sont-elles fiables ? »	SQL (ch. 5), assertions (4.1.10, 4.6)

Étape	Question que l'on se pose	Outils du livre
<b>2. Photographier</b>	« Que s'est-il passé, en gros ? »	SQL agrégé et fenêtres (5.2, 5.3), figures (4.5)
<b>3. Comparer</b>	« Cette différence est-elle réelle ou due au hasard ? »	tests, intervalles, tests multiples (3.3 à 3.5)
<b>4. Relier</b>	« Deux variables évoluent-elles ensemble ? »	corrélacion, moindres carrés (1.3, 2.3, 3.1)
<b>5. Segmenter</b>	« Qui sont les clients à surveiller ? »	CTE, NTILE (5.3)
<b>6. Rapporter</b>	« Que dire, et avec quelles limites ? »	pandas, reproductibilité (4.4, ch. 6)

**Intuition.** Un bon analyste passe **plus de temps sur les étapes 1 et 6** que sur l'étape « sophistiquée ». Des données mal contrôlées ruinent n'importe quelle analyse ; une analyse juste mal expliquée ne change aucune décision.

## P.2 Étape 1 : charger et contrôler les données

La base de Dar Jasmin a été construite au chapitre 5 (section 5.1.3) ; elle est fournie avec le livre dans `donnees/dar_jasmin.db`. Commençons par ouvrir la connexion et regarder ce qu'elle contient.

```
import sqlite3
import numpy as np
import pandas as pd

con = sqlite3.connect("donnees/dar_jasmin.db")
for table in ["categories", "produits", "clients", "commandes", "lignes_commande"]:
 n = con.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
 print(f"{table:16s} {n:4d} lignes")
```

```
categories 4 lignes
produits 16 lignes
clients 80 lignes
commandes 400 lignes
lignes_commande 693 lignes
```

Avant la moindre analyse, on **teste** les données. Ces contrôles sont des questions dont on connaît la réponse attendue : si elle diffère, il y a un problème à comprendre avant d'aller plus loin.

```
def un_seul_chiffre(sql):
 return con.execute(sql).fetchone()[0]

controles = {
 "aucune commande sans client connu":
 un_seul_chiffre("SELECT COUNT(*) FROM commandes c LEFT JOIN clients k USING(id_client) WHERE
↪ k.id_client IS NULL") == 0,
 "identifiants de commande uniques":
 un_seul_chiffre("SELECT COUNT(*) - COUNT(DISTINCT id_commande) FROM commandes") == 0,
 "dates comprises dans 2025":
 un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE date_commande NOT BETWEEN '2025-01-01'
↪ AND '2025-12-31'") == 0,
 "satisfaction toujours entre 1 et 5":
 un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE satisfaction NOT BETWEEN 1 AND 5") == 0,
 "montants strictement positifs":
 un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE montant <= 0") == 0,
 "retrait en boutique = délai nul":
```



```

 un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE (canal = 'Boutique') <> (delai_livraison
↳ = 0)") == 0,
 "les lignes d'une commande somment à son montant":
 un_seul_chiffre("""SELECT COUNT(*) FROM commandes c
 JOIN (SELECT id_commande, ROUND(SUM(quantite * prix_unitaire), 2) AS total
 FROM lignes_commande GROUP BY id_commande) l USING(id_commande)
 WHERE ABS(c.montant - l.total) > 0.005""") == 0,
}
for nom, ok in controles.items():
 print("OK " if ok else "ÉCHEC", nom)
assert all(controles.values()), "au moins un contrôle a échoué : on s'arrête là"

```

```

OK aucune commande sans client connu
OK identifiants de commande uniques
OK dates comprises dans 2025
OK satisfaction toujours entre 1 et 5
OK montants strictement positifs
OK retrait en boutique = délai nul
OK les lignes d'une commande somment à son montant

```

**Pourquoi assert ?** Si un contrôle échoue, le programme **s'arrête** avec un message : mieux vaut un plantage bruyant qu'un rapport faux et élégant. C'est l'application directe de l'idée du 4.6 : un test automatique vaut mieux qu'un « j'ai regardé, ça avait l'air bon ».

Le contrôle du retrait en boutique mérite un mot : il vérifie un fait que **nous** savons sur l'activité (rien n'est livré quand on vient chercher sa commande). Les données se contrôlent avec la connaissance du métier, pas seulement avec des règles techniques.

Dernière préparation : charger en une seule requête un tableau pandas « une ligne par commande », avec la ville du client, sur lequel nous travaillerons.

```

df = pd.read_sql_query("""
 SELECT c.id_commande, c.id_client, c.date_commande, c.canal, c.montant,
 c.delai_livraison, c.satisfaction, k.ville
 FROM commandes c JOIN clients k USING(id_client)
 ORDER BY c.date_commande, c.id_commande
""", con, parse_dates=["date_commande"])
df["mois"] = df["date_commande"].dt.month
print(df.shape)
print(df.dtypes.to_string())

```

```

(400, 9)
id_commande int64
id_client int64
date_commande datetime64[us]
canal str
montant float64
delai_livraison int64
satisfaction int64
ville str
mois int32

```

## P.3 Étape 2 : photographier l'année (question 1)

On commence par les chiffres de base, **calculés par SQL** (c'est la base qui sait agréger efficacement) :

```

total = pd.read_sql_query("""
 SELECT COUNT(*) AS commandes, COUNT(DISTINCT id_client) AS clients_actifs,
 ROUND(SUM(montant), 2) AS chiffre_affaires, ROUND(AVG(montant), 2) AS panier_moyen

```

```
FROM commandes""", con)
print(total.to_string(index=False))
```

commandes	clients_actifs	chiffre_affaires	panier_moyen
400	66	24098.3	60.25

Puis le rythme de l'année, avec la variation d'un mois à l'autre calculée par une **fonction fenêtre** (LAG, section 5.3) :

```
mensuel = pd.read_sql_query("""
 WITH m AS (
 SELECT CAST(strftime('%m', date_commande) AS INTEGER) AS mois,
 COUNT(*) AS commandes, ROUND(SUM(montant), 2) AS ca
 FROM commandes GROUP BY mois
)
 SELECT mois, commandes, ca,
 ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mois)) / LAG(ca) OVER (ORDER BY mois), 1) AS
↪ variation_pct
 FROM m ORDER BY mois
""", con)
print(mensuel.to_string(index=False))
```

mois	commandes	ca	variation_pct
1	16	996.4	NaN
2	20	1167.6	17.2
3	25	1687.1	44.5
4	36	1843.0	9.2
5	36	2314.1	25.6
6	33	2490.7	7.6
7	42	2570.9	3.2
8	44	2434.9	-5.3
9	31	1713.0	-29.6
10	20	1271.7	-25.8
11	40	2284.0	79.6
12	57	3324.9	45.6

Et la répartition par canal et par catégorie de produit (une jointure à quatre tables : commandes → lignes → produits → catégories) :

```
canaux = pd.read_sql_query("""
 SELECT canal, COUNT(*) AS commandes, ROUND(SUM(montant), 2) AS ca,
 ROUND(100.0 * SUM(montant) / (SELECT SUM(montant) FROM commandes), 1) AS part_ca_pct,
 ROUND(AVG(montant), 2) AS panier_moyen
 FROM commandes GROUP BY canal ORDER BY ca DESC""", con)
print(canaux.to_string(index=False))
print()
categories = pd.read_sql_query("""
 SELECT g.nom AS categorie, SUM(l.quantite) AS articles,
 ROUND(SUM(l.quantite * l.prix_unitaire), 2) AS ca
 FROM lignes_commande l
 JOIN produits p USING(id_produit) JOIN categories g USING(id_categorie)
 GROUP BY g.nom ORDER BY ca DESC""", con)
print(categories.to_string(index=False))
```

canal	commandes	ca	part_ca_pct	panier_moyen
Site	148	8806.5	36.5	59.50
Boutique	114	8528.3	35.4	74.81
Instagram	138	6763.5	28.1	49.01

categorie	articles	ca
Bijoux	200	7006.20
Textile	184	6994.13
Poterie	180	6950.23

Cosmétiques 196 3147.74

**Application : un seul regard.** Un tableau de douze lignes se lit mal ; un graphique se lit en deux secondes. Voici le chiffre d'affaires mensuel **empilé par canal** (à gauche) et, pour préparer la question 3, la satisfaction moyenne selon le délai de livraison (à droite, avec son intervalle de confiance à 95 %).

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
couleur = {"Boutique": AQUA, "Site": BLEU, "Instagram": ORANGE}
noms_mois = ["jan", "fév", "mar", "avr", "mai", "juin", "juil", "août", "sept", "oct", "nov", "déc"]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2), gridspec_kw={"width_ratios": [1.25, 1]})

Panneau de gauche : CA mensuel empilé par canal
ca = df.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum").reindex(range(1, 13)).fillna(0)
bas = np.zeros(12)
for canal in ["Boutique", "Site", "Instagram"]:
 ax1.bar(range(1, 13), ca[canal], bottom=bas, color=couleur[canal], width=0.75, label=canal)
 bas += ca[canal].to_numpy()
ax1.set_xticks(range(1, 13))
ax1.set_xticklabels(noms_mois, fontsize=8)
ax1.set_ylabel("chiffre d'affaires (DT)")
ax1.set_title("Chiffre d'affaires mensuel, par canal")
ax1.legend(frameon=False, ncol=3, loc="upper left", fontsize=8)
ax1.grid(axis="x", visible=False)

Panneau de droite : satisfaction moyenne selon le délai (commandes livrées)
livre = df[df["canal"] != "Boutique"]
paliers = [(1, 2), (3, 4), (5, 6), (7, 8), (9, 20)]
etiquettes = ["1-2", "3-4", "5-6", "7-8", "9+"]
moy, demi_ic = [], []
for a, b in paliers:
 s = livre.loc[livre["delai_livraison"].between(a, b), "satisfaction"]
 h = stats.t.ppf(0.975, len(s) - 1) * s.std(ddof=1) / np.sqrt(len(s))
 moy.append(s.mean()); demi_ic.append(h)
ax2.errorbar(range(len(paliers)), moy, yerr=demi_ic, fmt="o-", color=ORANGE, capsize=4, lw=1.8)
ax2.set_xticks(range(len(paliers)))
ax2.set_xticklabels(etiquettes)
ax2.set_xlabel("délai de livraison (jours)")
ax2.set_ylabel("satisfaction moyenne (1 à 5)")
ax2.set_title("Plus c'est long, moins c'est apprécié")
haut_y = max(m + h for m, h in zip(moy, demi_ic)); bas_y = min(m - h for m, h in zip(moy, demi_ic))
ax2.set_ylim(bas_y - 0.15, haut_y + 0.15) # marge pour ne couper aucune barre d'erreur
plt.tight_layout()
plt.savefig("figures/ch07-projet-vue-d-ensemble.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

**Ce que montrent ces chiffres pour la question 1.** (Les nombres ci-dessous sont lus dans les sorties précédentes.)

- Le chiffre d'affaires de l'année est de **24 098,30 DT** pour 400 commandes (panier moyen **60,25 DT**), que reconnaîtront les lecteurs du chapitre 3.
- L'activité est **très saisonnière** : le creux est en janvier (996 DT) et le pic en décembre (3 325

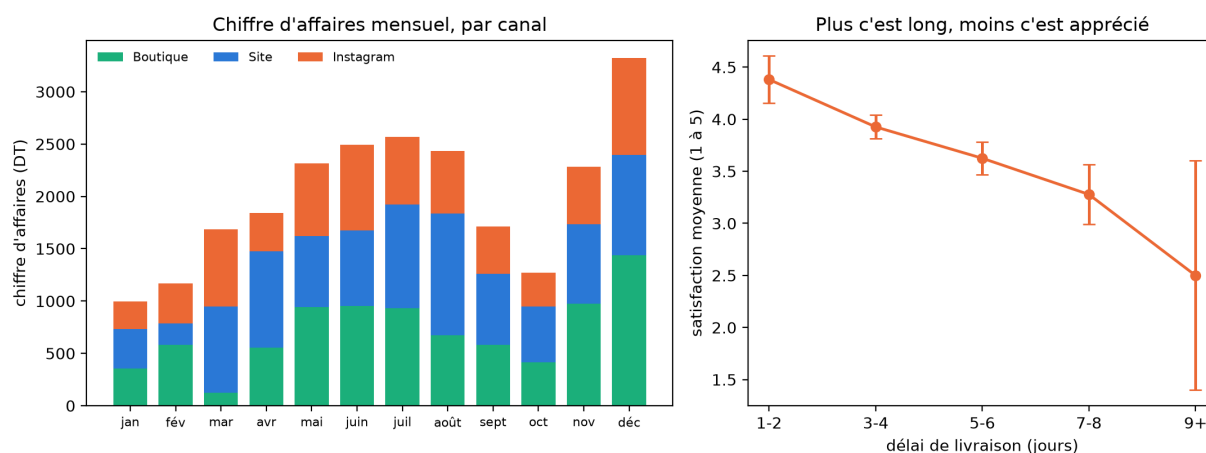


Figure 35: À gauche : chiffre d'affaires mensuel de Dar Jasmin en 2025, empilé par canal. À droite : satisfaction moyenne des commandes livrées selon le délai, avec intervalle de confiance à 95 %.

DT), soit **un facteur 3,3** entre les deux. L'été est un plateau élevé (mai à août), suivi d'une chute en septembre-octobre, puis d'un rebond à l'approche des fêtes.

- **Aucun canal ne domine.** La boutique et le site pèsent chacun environ un tiers du chiffre d'affaires ; Instagram, avec un nombre de commandes comparable (138, contre 148 pour le site et 114 en boutique), pèse moins (28 %) : ses paniers sont plus petits. C'est précisément le point de la question 2.

Une remarque sur le graphique de droite : le dernier palier (9 jours et plus) a un intervalle de confiance **très large**, parce qu'il ne contient que quelques commandes. Un point isolé ne prouve rien ; c'est la **tendance d'ensemble** des cinq paliers qui est convaincante. Nous la chiffrons à l'étape 4.

## P.4 Étape 3 : Instagram dépense-t-il vraiment moins ? (question 2)

Reformulons la question de façon testable. Yasmine voit que les paniers Instagram *semblent* plus petits. Ce qu'elle veut savoir : **cette différence observée dans nos 400 commandes reflète-t-elle une différence réelle entre les clientèles, ou pourrait-elle être due au hasard de l'échantillonnage ?** C'est exactement le cadre du chapitre 3 : un test de comparaison de deux moyennes (Welch, section 3.4), accompagné d'un **intervalle de confiance** (3.3) et d'une mesure de **taille d'effet**, car « significatif » ne veut pas dire « important » (3.5).

Nous avons **trois comparaisons** à faire (Boutique–Instagram, Boutique–Site, Site–Instagram). Selon la section 3.5, tester trois fois augmente le risque d'une fausse alerte : nous corrigerons les p-valeurs avec la méthode de **Holm**.

```
from scipy import stats

paniers = {c: g["montant"].to_numpy() for c, g in df.groupby("canal")}
paires = [("Boutique", "Instagram"), ("Boutique", "Site"), ("Site", "Instagram")]

lignes = []
for a, b in paires:
 x, y = paniers[a], paniers[b]
 res = stats.ttest_ind(x, y, equal_var=False)
 ic = res.confidence_interval(0.95)
 s_commun = np.sqrt(((len(x) - 1) * x.var(ddof=1) + (len(y) - 1) * y.var(ddof=1)) / (len(x) + len(y) - 2))
 lignes.append({"comparaison": f"{a} - {b}", "ecart_DT": x.mean() - y.mean(),
```

```

 "ic95_bas": ic.low, "ic95_haut": ic.high, "p_brute": res.pvalue,
 "d_de_Cohen": (x.mean() - y.mean()) / s_commun})
tab = pd.DataFrame(lignes)

Correction de Holm : on trie les p-valeurs, on multiplie la k-ième plus petite par (m - k + 1),
on impose que la suite soit croissante, et on plafonne à 1.
m = len(tab)
ordre = tab["p_brute"].argsort().to_numpy()
corrige = np.empty(m)
courant = 0.0
for rang, i in enumerate(ordre):
 courant = max(courant, min(1.0, (m - rang) * tab.loc[i, "p_brute"]))
 corrige[i] = courant
tab["p_holm"] = corrige

with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 140):
 print(tab.to_string(index=False))

```

	comparaison	ecart_DT	ic95_bas	ic95_haut	p_brute	d_de_Cohen	p_holm
	Boutique - Instagram	25.8	16.66	34.94	8.016e-08	0.7222	2.405e-07
	Boutique - Site	15.31	5.571	25.04	0.002188	0.3889	0.004377
	Site - Instagram	10.49	2.393	18.59	0.0113	0.2996	0.0113

**Lire le tableau.** `ecart_DT` est la différence de paniers moyens ; `ic95_bas` et `ic95_haut` encadrent la différence **réelle** avec 95 % de confiance (au sens du 3.3 : la *méthode* encadre la vérité dans 95 % des cas). Le **d de Cohen** exprime l'écart en nombre d'écarts-types : environ 0,2 est « petit », 0,5 « moyen », 0,8 « grand ». Enfin `p_holm` est la p-valeur **après** correction des comparaisons multiples : c'est elle que l'on compare à 0,05.

Les trois écarts sont tous significatifs, même après correction. Mais les p-valeurs ne disent pas à quel point ces écarts comptent : le tableau montre qu'ils sont **de tailles très différentes**. L'écart entre la boutique et Instagram est d'environ 26 DT par commande (un d de Cohen de plus de 0,7 : une différence franchement visible), alors que l'écart Site-Instagram est de l'ordre de 10 DT (un d autour de 0,3, plutôt petit).

Les montants sont **asymétriques** (3.1.3 : la moyenne est tirée par quelques gros paniers). Vérifions que la conclusion ne dépend pas de ce choix en comparant aussi les **médianes**, avec un **bootstrap** (3.3.5) :

```

rng = np.random.default_rng(42)
B = 5000
x, y = paniers["Boutique"], paniers["Instagram"]
diffs = np.empty(B)
for k in range(B):
 diffs[k] = np.median(rng.choice(x, len(x))) - np.median(rng.choice(y, len(y)))
bas, haut = np.percentile(diffs, [2.5, 97.5])
print(f"médiane Boutique : {np.median(x):.2f} DT médiane Instagram : {np.median(y):.2f} DT")
print(f"écart de médianes : {np.median(x) - np.median(y):.2f} DT IC95 bootstrap : [{bas:.1f} ;
↪ {haut:.1f}])")

```

```

médiane Boutique : 64.85 DT médiane Instagram : 41.50 DT
écart de médianes : 23.35 DT IC95 bootstrap : [14.6 ; 31.6]

```

L'écart de médianes est du même ordre que l'écart de moyennes, et son intervalle exclut nettement zéro : la conclusion est **robuste**.

**Ce que ces tests ne disent pas.** Ils établissent que les paniers Instagram sont plus petits ; ils n'expliquent **pas pourquoi** (produits moins chers ? clientèle plus jeune ? achats d'impulsion

?). Et surtout, un panier plus petit ne signifie pas un canal moins rentable : nous n'avons ni les coûts, ni le temps passé, ni la marge. Dire « Instagram dépense moins par commande » est un fait ; dire « Instagram ne vaut pas le coup » serait une conclusion que ces données **ne permettent pas** de tirer.

## P.5 Étape 4 : les retards font-ils baisser la satisfaction ? (question 3)

Ici, il ne s'agit plus de comparer des groupes mais de **relier deux variables** : le délai de livraison (en jours) et la satisfaction (de 1 à 5). Un détail essentiel : la boutique a un délai nul par construction (le client repart avec sa commande). Si nous l'incluons, nous mélangerions deux effets : « le client a-t-il attendu ? » et « le client est-il venu en boutique ? ». Pour isoler l'effet du **délai**, nous nous limitons aux commandes **livrées** (Site et Instagram).

```
livre = df[df["canal"] != "Boutique"].copy()
print(len(livre), "commandes livrées")
print(livre.groupby("canal")["delai_livraison"].agg(["count", "mean", "median", "max"]).round(2))
print()
r_pearson = livre["delai_livraison"].corr(livre["satisfaction"])
r_spearman = stats.spearmanr(livre["delai_livraison"], livre["satisfaction"])
print(f"corrélation de Pearson : {r_pearson:.3f}")
print(f"corrélation de Spearman : {r_spearman.statistic:.3f} (p = {r_spearman.pvalue:.1e})")
```

```
286 commandes livrées
 count mean median max
canal
Instagram 138 4.49 4.0 9
Site 148 4.70 4.0 13

corrélation de Pearson : -0.407
corrélation de Spearman : -0.365 (p = 1.9e-10)
```

Nous avons calculé **deux** corrélations. Pearson mesure la liaison *linéaire* ; Spearman travaille sur les rangs et ne suppose pas de linéarité (3.1.7), ce qui convient mieux à une note de 1 à 5 (variable **ordinaire**, 3.1.1). Les deux disent la même chose : une corrélation négative modérée. (Au 3.1.7, nous avons trouvé environ  $-0,53$  sur **toutes** les commandes ; la valeur est ici plus faible parce que nous avons retiré la boutique, dont les délais nuls et les notes élevées renforçaient artificiellement le lien. C'est un bon exemple de la façon dont une décision de **périmètre** change un chiffre.)

Quantifions maintenant l'**effet moyen d'un jour de retard**. C'est la pente de la droite des moindres carrés (1.3 : on choisit la droite qui minimise la somme des carrés des erreurs ; la formule ci-dessous en est la solution) :

$$\hat{b} = \frac{\sum_i (d_i - \bar{d})(s_i - \bar{s})}{\sum_i (d_i - \bar{d})^2}$$

où  $d_i$  est le délai de la commande  $i$  et  $s_i$  sa note de satisfaction.

```
d = livre["delai_livraison"].to_numpy(dtype=float)
s = livre["satisfaction"].to_numpy(dtype=float)
pente = np.sum((d - d.mean()) * (s - s.mean())) / np.sum((d - d.mean()) ** 2)
ordonnee = s.mean() - pente * d.mean()
print(f"droite des moindres carrés : satisfaction = {ordonnee:.2f} + ({pente:.3f}) x délai")

Intervalle de confiance par bootstrap (on rééchantillonne des COMMANDES entières)
rng = np.random.default_rng(7)
pentes = np.empty(5000)
for k in range(5000):
```

```

idx = rng.integers(0, len(d), len(d))
dk, sk = d[idx], s[idx]
pentes[k] = np.sum((dk - dk.mean()) * (sk - sk.mean())) / np.sum((dk - dk.mean()) ** 2)
pente_bas, pente_haut = np.percentile(pentes, [2.5, 97.5])
print(f"IC95 bootstrap de la pente : [{pente_bas:.3f} ; {pente_haut:.3f}]")

```

```

droite des moindres carrés : satisfaction = 4.58 + (-0.180) x délai
IC95 bootstrap de la pente : [-0.228 ; -0.129]

```

Chaque jour de livraison supplémentaire est associé à une baisse de satisfaction d'environ **0,18 point** (sur une échelle de 5), et l'intervalle de confiance, qui ne contient pas zéro, exclut un effet nul. Pour parler en termes concrets, comparons des **groupes de délais** (puis la même pente, canal par canal, pour vérifier que l'effet n'est pas un simple artefact du mélange Site/Instagram) :

```

livre["groupe"] = pd.cut(livre["delai_livraison"], bins=[0, 3, 6, 100], labels=["1-3 jours", "4-6
↪ jours", "7 jours et +"])
print(livre.groupby("groupe", observed=True)["satisfaction"].agg(commandes="count",
↪ moyenne="mean").round(2))
print()
for canal, g in livre.groupby("canal"):
 res = stats.linregress(g["delai_livraison"], g["satisfaction"])
 print(f"{canal:10s} pente = {res.slope:.3f} point/jour (r = {res.rvalue:.2f}, {len(g)}
↪ commandes)")

```

	commandes	moyenne
groupe		
1-3 jours	88	4.03
4-6 jours	156	3.76
7 jours et +	42	3.17

Instagram	pente = -0.182 point/jour	(r = -0.38, 138 commandes)
Site	pente = -0.181 point/jour	(r = -0.44, 148 commandes)

Les trois groupes sont nettement ordonnés : plus le délai est long, plus la satisfaction moyenne est basse, avec un écart d'environ **0,9 point** entre les livraisons rapides (1 à 3 jours) et les livraisons lentes (7 jours et plus). Et la pente est du même ordre dans les deux canaux : le phénomène n'est pas un effet de mélange.

Une dernière question de Yasmine, naturelle : « si je ramenait tous mes délais de plus de 5 jours à 5 jours, que gagnerais-je ? » Le calcul est facile avec la droite, et il faut l'énoncer avec **prudence** :

```

longs = livre[livre["delai_livraison"] > 5]
gain_par_commande = (pente * (5 - longs["delai_livraison"])).mean() # pente < 0 et (5 - délai) < 0 :
↪ gain > 0
part = len(longs) / len(livre)
print(f"{len(longs)} commandes livrées sur {len(livre)} ({100 * part:.0f} %) dépassent 5 jours")
print(f"gain de satisfaction attendu sur ces commandes : +{gain_par_commande:.2f} point")
print(f"gain sur l'ensemble des livraisons : +{part * gain_par_commande:.3f} point")

```

```

74 commandes livrées sur 286 (26 %) dépassent 5 jours
gain de satisfaction attendu sur ces commandes : +0.36 point
gain sur l'ensemble des livraisons : +0.094 point

```

**Association n'est pas causalité.** La pente décrit comment les deux variables **varient ensemble** dans nos données. Elle ne prouve pas que réduire les délais *fera* monter les notes : un facteur caché pourrait jouer sur les deux à la fois (les commandes volumineuses sont peut-être à la fois plus lentes à préparer et plus exigeantes). Ici les données sont simulées et la relation a été construite pour être réelle, mais dans une vraie étude, la phrase honnête serait : « les commandes livrées plus lentement sont associées à des notes plus basses, d'environ 0,18



*point par jour* ». Distinguer corrélation et causalité est l'un des grands thèmes du volume II (chapitre 7, facultatif).

## P.6 Étape 5 : qui relancer en janvier ? (question 4)

Cette question n'est pas un test : c'est de la **segmentation**. On veut une liste claire de clients à contacter, et une raison de les contacter. Nous la construisons en SQL avec des CTE et NTILE (5.3). Deux critères :

- **La valeur** : le chiffre d'affaires cumulé du client en 2025 ;
- **La récence** : le nombre de jours depuis sa dernière commande, à la date de référence du 31 décembre 2025.

D'abord, le principe de **concentration** : quelle part du chiffre d'affaires vient des 25 % de clients qui achètent le plus ? (C'est la règle de Pareto, souvent « 80/20 ».)

```
concentration = pd.read_sql_query("""
 WITH par_client AS (
 SELECT id_client, SUM(montant) AS ca FROM commandes GROUP BY id_client
), classes AS (
 SELECT id_client, ca, NTILE(4) OVER (ORDER BY ca DESC) AS quartile FROM par_client
)
 SELECT quartile, COUNT(*) AS clients, ROUND(SUM(ca), 2) AS ca,
 ROUND(100.0 * SUM(ca) / (SELECT SUM(ca) FROM par_client), 1) AS part_ca_pct
 FROM classes GROUP BY quartile ORDER BY quartile
""", con)
print(concentration.to_string(index=False))
```

quartile	clients	ca	part_ca_pct
1	17	14063.4	58.4
2	17	5678.4	23.6
3	16	3086.4	12.8
4	16	1270.1	5.3

Le premier quartile (les meilleurs clients) pèse un peu plus de la moitié du chiffre d'affaires : une clientèle **concentrée**. Perdre l'un de ces clients coûte beaucoup plus que d'en perdre un petit.

Cherchons maintenant les **clients précieux devenus silencieux** : ceux dont la valeur est dans les deux meilleurs quartiles mais qui n'ont plus rien commandé depuis plus de 90 jours.

```
relance = pd.read_sql_query("""
 WITH par_client AS (
 SELECT k.id_client, k.prenom, k.nom, k.ville,
 COUNT(c.id_commande) AS commandes,
 COALESCE(SUM(c.montant), 0) AS ca,
 MAX(c.date_commande) AS derniere
 FROM clients k LEFT JOIN commandes c USING(id_client)
 GROUP BY k.id_client
), classes AS (
 SELECT *, julianday('2025-12-31') - julianday(derniere) AS jours_depuis,
 NTILE(4) OVER (ORDER BY ca DESC) AS quartile_valeur
 FROM par_client WHERE commandes > 0
)
 SELECT prenom || ' ' || nom AS client, ville, commandes, ROUND(ca, 2) AS ca_2025,
 derniere AS derniere_commande, CAST(jours_depuis AS INTEGER) AS jours_depuis
 FROM classes
 WHERE jours_depuis > 90 AND quartile_valeur <= 2
 ORDER BY ca DESC
""", con)
print(len(relance), "clients précieux à relancer :")
print(relance.to_string(index=False))
```



```

jamais = un_seul_chiffre("SELECT COUNT(*) FROM clients k WHERE NOT EXISTS (SELECT 1 FROM commandes c
↳ WHERE c.id_client = k.id_client)")
print(f"\n(par ailleurs, {jamais} clients inscrits n'ont jamais commandé : une campagne différente,
↳ de première commande)")

```

5 clients précieux à relancer :

	client	ville	commandes	ca_2025	derniere_commande	jours_depuis
	Amel Ayari	Nabeul	5	393.0	2025-07-26	158
	Lina Sassi	Bizerte	4	297.8	2025-08-17	136
	Hatem Ayari	La Marsa	6	294.9	2025-09-24	98
	Oussama Hamdi	La Marsa	6	284.0	2025-08-30	123
	Walid Ayari	Sousse	7	270.3	2025-09-26	96

(par ailleurs, 14 clients inscrits n'ont jamais commandé : une campagne différente, de première  
↳ commande)

**Deux listes, deux messages.** Les clients qui ont **beaucoup acheté puis disparu** se contactent avec un message personnel (« cela fait un moment, voici les nouveautés »). Les clients **inscrits qui n'ont jamais commandé** appellent plutôt une offre de première commande. Les mélanger, c'est brouiller le message.

## P.7 Étape 6 : rédiger le rapport

Tout le travail précédent n'a de valeur que s'il se transforme en **message clair**. Un rapport pour Yasmine tient sur une page, ne contient aucun jargon, donne les chiffres clés, formule des recommandations et **annonce ses limites**.

Une bonne pratique, héritée du chapitre 6 (recherche reproductible) : **ne jamais recopier un chiffre à la main** dans un rapport. On génère le texte à partir des résultats déjà calculés. Si les données changent demain, le rapport se met à jour tout seul.

```

mois_fr = ["janvier", "février", "mars", "avril", "mai", "juin", "juillet", "août",
 "septembre", "octobre", "novembre", "décembre"]

def fr(x, decimales=2):
 """Format français : 24098.3 -> '24 098,30'."""
 return f"{x:.{decimales}f}".replace(".", ", ").replace(".", ", ")

ca_total = total.loc[0, "chiffre_affaires"]
mois_pic = mensuel.loc[mensuel["ca"].idxmax()]
mois_creux = mensuel.loc[mensuel["ca"].idxmin()]
t_bi = tab.iloc[0] # Boutique - Instagram
groupes = livre.groupby("groupe", observed=True)["satisfaction"].mean()
top_quartile_part = concentration.loc[0, "part_ca_pct"]

rapport = f"""RAPPORT 2025 : DAR JASMIN
{"=" * 60}

1. L'année en bref
- Chiffre d'affaires : {fr(ca_total)} DT pour {int(total.loc[0, 'commandes'])} commandes
 (panier moyen : {fr(total.loc[0, 'panier_moyen'])} DT).
- Un rythme très saisonnier : creux en {mois_fr[int(mois_creux['mois']) - 1]}
↳ ({fr(mois_creux['ca'], 0)} DT),
 pic en {mois_fr[int(mois_pic['mois']) - 1]} ({fr(mois_pic['ca'], 0)} DT).

2. Instagram dépense-t-il moins ?
- Oui : un panier Instagram est inférieur de {fr(t_bi['ecart_DT'], 1)} DT à un panier en boutique
 (intervalle de confiance à 95 % : de {fr(t_bi['ic95_bas'], 1)} à {fr(t_bi['ic95_haut'], 1)} DT).
- Cet écart n'est pas dû au hasard (p corrigée < 0,001) et il est franc (d de Cohen =
↳ {fr(t_bi['d_de_Cohen'])}).

```

```

- Attention : nous ne connaissons ni les marges, ni le temps passé. Un panier plus petit
 ne veut pas dire un canal moins rentable.

3. Les retards pèsent sur la satisfaction
- Chaque jour de livraison en plus est associé à {fr(abs(pente), 2)} point de satisfaction en moins
 (intervalle à 95 % : de {fr(abs(pente_haut), 2)} à {fr(abs(pente_bas), 2)}).
- Livraison en 1 à 3 jours : {fr(groupe.iloc[0])} / 5 ; en 7 jours et plus : {fr(groupe.iloc[2])}
 / 5.
- {len(longs)} livraisons sur {len(livre)} dépassent 5 jours : c'est là que l'on peut gagner.

4. Clients à relancer en janvier
- {len(relance)} clients précieux sont silencieux depuis plus de 90 jours ; les
 {int(concentration.loc[0, 'clients'])} meilleurs clients
 font {fr(top_quartile_part, 1)} % du chiffre d'affaires.
- {jamais} clients inscrits n'ont jamais commandé : prévoir une offre de première commande.

Limites : une seule année de données ; pas de coûts ; association n'est pas causalité.
"""
print(rapport)

```

#### RAPPORT 2025 : DAR JASMIN

```

=====

1. L'année en bref
- Chiffre d'affaires : 24 098,30 DT pour 400 commandes
 (panier moyen : 60,25 DT).
- Un rythme très saisonnier : creux en janvier (996 DT),
 pic en décembre (3 325 DT).

2. Instagram dépense-t-il moins ?
- Oui : un panier Instagram est inférieur de 25,8 DT à un panier en boutique
 (intervalle de confiance à 95 % : de 16,7 à 34,9 DT).
- Cet écart n'est pas dû au hasard (p corrigée < 0,001) et il est franc (d de Cohen = 0,72).
- Attention : nous ne connaissons ni les marges, ni le temps passé. Un panier plus petit
 ne veut pas dire un canal moins rentable.

3. Les retards pèsent sur la satisfaction
- Chaque jour de livraison en plus est associé à 0,18 point de satisfaction en moins
 (intervalle à 95 % : de 0,13 à 0,23).
- Livraison en 1 à 3 jours : 4,03 / 5 ; en 7 jours et plus : 3,17 / 5.
- 74 livraisons sur 286 dépassent 5 jours : c'est là que l'on peut gagner.

4. Clients à relancer en janvier
- 5 clients précieux sont silencieux depuis plus de 90 jours ; les 17 meilleurs clients
 font 58,4 % du chiffre d'affaires.
- 14 clients inscrits n'ont jamais commandé : prévoir une offre de première commande.

Limites : une seule année de données ; pas de coûts ; association n'est pas causalité.

```

**Relisez ce rapport comme Yasmine.** Y a-t-il un mot qu'elle ne comprendrait pas (« d de Cohen », « bootstrap ») ? Dans le rapport final, on garde le **chiffre** et on cache la **méthode** : l'annexe technique, c'est le reste de ce projet, rangé dans le dépôt. Dans la version ci-dessus, le « d de Cohen » est volontairement conservé pour que vous voyiez d'où vient chaque chiffre ; dans le document remis à Yasmine, on écrirait simplement « un écart net ».

## P.8 Rendre le projet reproductible

Un projet n'est terminé que lorsqu'une autre personne peut le refaire. Voici la check-list du chapitre 6, appliquée à ce projet :

Geste	Pourquoi	Où l'apprendre
Un dossier propre : <code>donnees/</code> , <code>sql/</code> , <code>analyse/</code> , <code>figures/</code> , <code>rapport/</code>	on retrouve tout	6.3
Versionner avec <b>Git</b> , un commit par étape logique	on peut revenir en arrière et expliquer les changements	6.1
Fixer les <b>graines</b> ( <code>default_rng(42)</code> , <code>default_rng(7)</code> )	mêmes résultats à chaque exécution	3.7, 6.5
Lister les versions des bibliothèques ( <code>requirements.txt</code> )	l'environnement se recrée	6.3
Garder les contrôles de l'étape 1 dans le script	les données douteuses sont détectées	4.6
Un <b>notebook</b> ou un <b>script</b> qui va des données brutes au rapport	un seul geste pour tout refaire	6.2, 6.5

Voici une structure de dossier possible. Elle est donnée à titre d'exemple (non exécutée) : à vous de l'adapter.

```

etude-dar-jasmin-2025/
├── README.md <- la question, comment relancer, les versions
├── requirements.txt
├── donnees/
│ └── dar_jasmin.db
├── sql/ <- chaque requête dans son fichier : 01_ca_mensuel.sql, ...
├── analyse/
│ ├── 01_controles.py
│ ├── 02_comparaisons.py
│ └── 03_rapport.py
├── figures/
├── rapport/
│ └── rapport-2025.md

```

## P.9 Ce que cette étude ne dit pas, et la suite

Une étude honnête se termine par ses **limites** :

- **Une seule année** : impossible de distinguer une vraie saisonnalité d'un phénomène propre à 2025.
- **Pas de coûts** : nous avons parlé de chiffre d'affaires, jamais de **bénéfice**. Instagram pourrait être très rentable si l'on dépense peu pour y vendre.
- **Pas de causalité** : les relations constatées sont des **associations** ; pour savoir si réduire les délais *améliorerait* les notes, il faudrait une expérience (volume II, chapitre 7 facultatif sur l'inférence causale, et chapitre 8 sur les plans d'expériences).
- **Variables simples** : nous n'avons pas pris en compte, par exemple, la ville de livraison, le type de produit ou le client lui-même (un client très exigeant note toujours bas). Les modèles du volume II (régression multiple, modèles mixtes) permettent de **tenir compte de plusieurs facteurs à la fois**.

**À retenir.** Ce que vous venez de faire, de la question de Yasmine au rapport, est le **cycle de base de la data science** : *poser la question* → *contrôler les données* → *décrire* → *comparer ou relier avec rigueur* → *conclure avec prudence* → *rendre reproductible*. Les volumes suivants ajouteront

des outils (régression, apprentissage automatique, séries temporelles...), mais ce cycle ne changera pas.

# Points clés et auto-évaluation

« On ne sait vraiment une chose que lorsqu'on peut l'expliquer à quelqu'un d'autre sans regarder ses notes. »

Ce dernier chapitre court a deux rôles. Le premier : **fixer l'essentiel** de chaque chapitre en quelques lignes, pour pouvoir y revenir. Le second : vous donner un moyen **honnête** de savoir si le volume a fait son travail, avec trente questions, leurs réponses, et une grille d'auto-évaluation.

## Les points clés, chapitre par chapitre

Chapitre	Ce que vous devez emporter
1. <b>Mathématiques</b>	Un vecteur est une liste de nombres <i>et</i> une flèche ; une matrice <b>transforme</b> l'espace. Les valeurs propres disent de combien une direction est étirée, la SVD en donne la meilleure approximation. La dérivée mesure un taux de variation, le <b>gradient</b> pointe vers la plus forte montée : on <b>descend</b> dans l'opposé pour optimiser. Les ordinateurs calculent avec des approximations ( $0,1 + 0,2 \neq 0,3$ ) : on compare avec une tolérance.
2. <b>Probabilités</b>	Une probabilité mesure l'incertitude ; la <b>formule de Bayes</b> met à jour une croyance quand on observe un fait (et une alerte rare est souvent une fausse alerte). Les <b>lois</b> (Bernoulli, binomiale, Poisson, normale...) sont des modèles ; l'espérance et la variance les résument. La <b>loi des grands nombres</b> dit que la moyenne converge ; le <b>théorème central limite</b> dit comment elle fluctue (en $\sigma/\sqrt{n}$ ).
3. <b>Statistique</b>	<b>Dessiner avant de calculer</b> . Un estimateur se juge à son biais et à sa variance. Un <b>intervalle de confiance</b> quantifie l'incertitude ; une <b>p-valeur</b> n'est <i>pas</i> la probabilité que l'hypothèse soit vraie ; « significatif » n'est pas « important ». Tester plusieurs fois impose de <b>corriger</b> (Bonferroni, Holm, Benjamini-Hochberg). Le bootstrap et les tests de permutation fonctionnent sans hypothèse de loi.
4. <b>Programmation</b>	Python pour tout faire, R pour comparer. Une <b>bonne structure de données</b> (dictionnaire, ensemble) vaut mieux qu'un calcul plus rapide. <b>Vectorisez</b> avec NumPy et pandas au lieu de boucler. Un graphique répond à <b>une question</b> et ne ment pas (axes honnêtes). Un test automatique est un filet de sécurité.
5. <b>SQL</b>	Une base <b>relationnelle</b> sépare l'information en tables reliées par des clés. WHERE filtre les lignes, HAVING filtre les groupes. LEFT JOIN garde les lignes sans correspondance, NULL se teste avec IS NULL. Les <b>fonctions fenêtres</b> calculent sans écraser les lignes ; les <b>CTE</b> donnent un nom à chaque étape. Normaliser évite la redondance et les incohérences.
6. <b>Outils</b>	<b>Git</b> garde l'historique et permet d'essayer sans risque (branches). Un <b>notebook</b> mélange code et texte, mais cache un état : <i>Restart &amp; Run All</i> avant de partager. La <b>ligne de commande</b> assemble de petits outils ; un <b>environnement virtuel</b> et un requirements.txt rendent l'analyse reproductible.
<b>Projet</b>	Le cycle complet : <b>question</b> → <b>contrôle des données</b> → <b>description</b> → <b>comparaison ou liaison rigoureuse</b> → <b>conclusion prudente</b> → <b>reproductibilité</b> .

**Trois idées qui traversent tout le volume.**

1. **Les données varient** : un chiffre calculé sur un échantillon est une estimation, jamais la vérité (chapitres 2 et 3).
2. **Un calcul juste n'est pas une conclusion juste** : il faut vérifier les hypothèses, dessiner, contrôler les données, distinguer association et causalité (chapitres 3 à 5 et projet).
3. **Ce qui n'est pas reproductible n'existe pas** : graines fixées, code versionné, environnement décrit (chapitres 4 à 6).

**Trente questions pour s'auto-évaluer**

**Mode d'emploi.** Répondez à voix haute ou par écrit avant de regarder le corrigé, en une ou deux phrases. Si vous ne savez pas, notez la section indiquée et allez la relire : ce n'est pas un échec, c'est le but de l'exercice. Vingt-cinq bonnes réponses sur trente signalent un volume bien assimilé.

**Mathématiques (chapitre 1)**

1. Que signifie l'égalité  $A\mathbf{v} = \lambda\mathbf{v}$  ?
2. Dans quelle direction faut-il se déplacer pour **faire diminuer** le plus vite possible une fonction ?
3. Pourquoi  $0.1 + 0.2 == 0.3$  vaut-il False en Python, et comment comparer proprement deux nombres décimaux ?
4. Yasmine veut présenter 3 produits choisis parmi 8 dans une vitrine, sans tenir compte de l'ordre. Combien de vitrines possibles ?

**Probabilités (chapitre 2)**

5. Une maladie touche 1 % de la population. Un test la détecte dans 90 % des cas, mais donne un faux positif chez 5 % des personnes saines. Vous êtes positif : quelle est la probabilité d'être malade ?
6. Quelle différence entre la loi des grands nombres et le théorème central limite ?
7. Les montants de commandes ont un écart-type de 38 DT. Quel est l'écart-type de la **moyenne** de 400 commandes ?
8. Quelle loi pour (a) le nombre de commandes reçues en une heure ; (b) le fait qu'une commande soit retournée ou non ?

**Statistique (chapitre 3)**

9. Pourquoi divise-t-on par  $n - 1$  et non par  $n$  pour estimer une variance ?
10. Que signifie « intervalle de confiance à 95 % » ? Que ne signifie-t-il **pas** ?
11. Qu'est-ce qu'une p-valeur ? Citez une mauvaise interprétation fréquente.
12. On réalise 20 tests indépendants au seuil de 5 %, alors qu'**aucun** effet n'existe. Combien de faux positifs attend-on, et quelle est la probabilité d'en obtenir **au moins un** ?
13. Quand préférer la médiane à la moyenne ?
14. Un test donne  $p = 10^{-9}$  pour une différence de 0,3 DT entre deux paniers moyens. Doit-on s'en réjouir ?

**Programmation (chapitre 4)**

15. Pourquoi tester l'appartenance d'un élément est-il bien plus rapide dans un set ou un dict que dans une list de grande taille ?
16. Pourquoi `df["montant"].sum()` est-il préférable à une boucle for sur les lignes ?
17. Que renvoie `df.groupby("canal")["montant"].mean()` : quel type, et quel index ?

18. Citez deux manières de rendre un graphique en barres trompeur.
19. Combien de comparaisons, au maximum, la recherche dichotomique effectue-t-elle sur une liste triée d'un million d'éléments ?
20. À quoi sert un test unitaire, et pourquoi vaut-il mieux que « j'ai regardé, ça avait l'air bon » ?

## SQL (chapitre 5)

21. Quelle est la différence entre une clé primaire et une clé étrangère ?
22. Quelle différence entre WHERE et HAVING ?
23. Vous voulez la liste de **tous** les clients avec leur nombre de commandes, y compris ceux qui n'ont jamais commandé. Quelle jointure ?
24. Pourquoi WHERE telephone = NULL ne renvoie-t-il jamais rien, et que faut-il écrire ?
25. En quoi une fonction fenêtre (OVER) diffère-t-elle d'un GROUP BY ?
26. Pourquoi normaliser une base (jusqu'à la 3FN) ?

## Outils (chapitre 6)

27. Quelle différence entre git add et git commit ?
28. Pourquoi un notebook peut-il donner des résultats différents selon qui l'exécute, et comment s'en protéger ?
29. À quoi sert un fichier requirements.txt ?
30. Quelle commande compte rapidement le nombre de lignes d'un fichier CSV, et pourquoi faut-il en retrancher une ?

## Vérifier les réponses chiffrées

Pour les questions numériques, plutôt que de se fier à sa mémoire, **calculons**. Le code ci-dessous vérifie les réponses des questions 3, 4, 5, 7, 12 et 19.

```
import math

Q3 : arithmétique des flottants
print("Q3 0.1 + 0.2 == 0.3 :", 0.1 + 0.2 == 0.3, "| avec tolérance :", math.isclose(0.1 + 0.2, 0.3))

Q4 : combinaisons
print("Q4 C(8,3) =", math.comb(8, 3))

Q5 : formule de Bayes
prevalence, sensibilite, faux_positifs = 0.01, 0.90, 0.05
p_positif = sensibilite * prevalence + faux_positifs * (1 - prevalence)
print(f"Q5 P(malade | test positif) = {sensibilite * prevalence / p_positif:.4f}")

Q7 : écart-type d'une moyenne
print(f"Q7 38 / sqrt(400) = {38 / math.sqrt(400):.2f} DT")

Q12 : tests multiples
print(f"Q12 faux positifs attendus : {20 * 0.05:.0f} ; P(au moins un) = {1 - 0.95 ** 20:.4f}")

Q19 : recherche dichotomique
print("Q19 comparaisons max pour 1 000 000 éléments :", math.ceil(math.log2(1_000_000 + 1)))
```

```
Q3 0.1 + 0.2 == 0.3 : False | avec tolérance : True
Q4 C(8,3) = 56
Q5 P(malade | test positif) = 0.1538
Q7 38 / sqrt(400) = 1.90 DT
Q12 faux positifs attendus : 1 ; P(au moins un) = 0.6415
Q19 comparaisons max pour 1 000 000 éléments : 20
```

## Corrigé

1.  $\mathbf{v}$  est un **vecteur propre** de  $A$  : la matrice ne change pas sa direction, elle l'étire (ou le comprime) d'un facteur  $\lambda$ , la **valeur propre**. (1.1.3)
2. Dans la direction **opposée au gradient**, qui pointe vers la plus forte montée. C'est le principe de la descente de gradient. (1.2 et 1.3)
3. Les nombres décimaux sont stockés en binaire, et 0,1 n'a pas d'écriture binaire finie : on ne stocke qu'une **approximation**. On compare avec une tolérance (`math.isclose`, `np.isclose`), jamais avec `==`. (1.5)
4.  $\binom{8}{3} = \frac{8!}{3!5!} = 56$  vitrines. L'ordre ne comptant pas, on divise les  $8 \times 7 \times 6 = 336$  arrangements par les  $3! = 6$  façons de les ranger. (1.6)
5. Environ **15,4 %**, loin des 90 % que l'on devine. Sur 1 000 personnes, 10 sont malades (9 détectées) et 990 sont saines (environ 49,5 faux positifs) : seulement 9 positifs sur 58,5 environ sont vraiment malades. C'est ce qui arrive quand la maladie est rare. (2.1)
6. La loi des grands nombres dit que la **moyenne d'échantillon converge** vers l'espérance quand  $n$  grandit ; le théorème central limite décrit **comment elle fluctue** autour de celle-ci : approximativement selon une loi normale d'écart-type  $\sigma/\sqrt{n}$ , quelle que soit la loi d'origine. (2.4)
7.  $38/\sqrt{400} = 38/20 = 1,9$  DT : la moyenne est bien plus stable que chaque commande. C'est la raison pour laquelle on moyenne. (2.4)
8. (a) Une loi de **Poisson** (événements rares et indépendants dans un intervalle de temps) ; (b) une loi de **Bernoulli** (deux issues), ou binomiale si l'on compte le nombre de retours sur  $n$  commandes. (2.2)
9. Parce que l'on mesure les écarts à la moyenne **de l'échantillon**, qui est elle-même ajustée aux données : les écarts sont un peu trop petits. Diviser par  $n - 1$  corrige ce biais et rend l'estimateur **sans biais**. (3.2)
10. La **méthode** produit un intervalle qui contient la vraie valeur dans 95 % des échantillons possibles. Ce n'est **pas** « 95 % de chances que la vraie valeur soit dans cet intervalle-ci » : une fois calculé, l'intervalle contient la vraie valeur ou ne la contient pas. (3.3.2)
11. La probabilité d'observer un résultat **au moins aussi extrême** que le nôtre, *si l'hypothèse nulle était vraie*. Mauvaise interprétation fréquente : « c'est la probabilité que l'hypothèse nulle soit vraie ». (3.5.1 et 3.5.2)
12. On attend  $20 \times 0,05 = 1$  faux positif, et la probabilité d'au moins un est  $1 - 0,95^{20} \approx 64\%$  : d'où la nécessité de corriger les tests multiples. (3.5.5)
13. Quand la distribution est **asymétrique** ou contient des **valeurs extrêmes** (montants, revenus, durées) : la médiane est robuste, la moyenne est tirée par la queue. (3.1.3)
14. Pas vraiment : avec assez de données, même une différence minuscule devient « significative ». 0,3 DT sur un panier de 60 DT n'a **aucune importance pratique**. Il faut toujours regarder la **taille de l'effet** et l'intervalle de confiance, pas seulement la p-valeur. (3.5.3)
15. Un set ou un dict utilise une **table de hachage** : il calcule directement où se trouve l'élément (coût quasi constant). Une liste doit être **parcourue** élément par élément (coût proportionnel à sa taille). (4.3.2 et 4.8)



16. La somme vectorisée s'exécute en **code compilé** sur un tableau contigu, sans le surcoût de l'interpréteur Python à chaque ligne : elle est en général beaucoup plus rapide (au moins plusieurs fois, souvent bien davantage selon la taille du tableau), et plus courte à écrire. (4.4 et 4.8)
17. Une **Series** pandas dont l'index est le canal (Boutique, Instagram, Site) et dont les valeurs sont les montants moyens. (4.4)
18. Par exemple : **tronquer l'axe vertical** (un écart réel de 2 % peut sembler un rapport de 5 à 1), utiliser un **camembert en 3D** (la perspective déforme les aires) ou un **double axe vertical** (on rend « visible » n'importe quelle corrélation en choisissant les échelles). Une barre doit toujours partir de zéro. (4.5.6)
19. 20 comparaisons au plus ( $2^{20} = 1\,048\,576 > 10^6$ ) : chaque comparaison divise l'intervalle de recherche par deux. Chercher dans une liste non triée en demanderait jusqu'à un million. (4.3.5)
20. Un test unitaire **vérifie automatiquement** qu'une fonction renvoie le résultat attendu sur des cas connus. Rejoué à chaque modification, il détecte immédiatement une régression ; un coup d'œil, lui, oublie les cas limites et ne se rejoue pas. (4.6)
21. La **clé primaire** identifie de façon unique chaque ligne d'une table. Une **clé étrangère** est une colonne qui référence la clé primaire d'une autre table : c'est elle qui crée le lien entre les tables. (5.1)
22. WHERE filtre les **lignes** avant le regroupement ; HAVING filtre les **groupes** après l'agrégation (par exemple « les clients avec plus de 5 commandes »). (5.2.4)
23. Un **LEFT JOIN** de clients vers commandes : il garde tous les clients, avec NULL (ou 0 après COUNT sur la colonne de droite) pour ceux qui n'ont pas de commande. Un **INNER JOIN** les ferait disparaître. (5.2.5)
24. Parce que NULL signifie « inconnu » : comparer quoi que ce soit à NULL donne « inconnu », jamais « vrai ». Il faut écrire WHERE telephone IS NULL. (5.2.7)
25. GROUP BY **réduit** plusieurs lignes à une seule par groupe ; une fonction fenêtre **conserve toutes les lignes** et ajoute une colonne calculée sur une « fenêtre » de lignes voisines (classement, cumul, ligne précédente). (5.3.1)
26. Pour **éviter la redondance** (la même information écrite à plusieurs endroits) et donc les **anomalies** de mise à jour, d'insertion et de suppression : chaque fait est stocké **une seule fois**. (5.4)
27. git add **prépare** les modifications (zone d'index) ; git commit **enregistre** ce qui a été préparé dans l'historique, avec un message. Cela permet de composer des commits cohérents. (6.1.3)
28. Parce que l'on peut exécuter les cellules **dans le désordre** et que le noyau garde en mémoire des variables qui n'existent plus dans le fichier : c'est l'**état caché**. Protection : *Restart & Run All* avant de partager ou d'en tirer un résultat. (6.2.3)
29. Il **liste les bibliothèques et leurs versions** nécessaires, pour que n'importe qui puisse recréer le même environnement (pip install -r requirements.txt). (6.3.6)
30. wc -l fichier.csv. Il faut retrancher 1 : la première ligne est l'en-tête (les noms de colonnes), pas une observation. (6.3)

## Votre grille d'auto-évaluation

Pour chaque ligne, cochez mentalement : **je sais l'expliquer** / **je sais le faire** / **à revoir**. Les sections à relire sont indiquées.

Compétence	Où la retravailler
Manipuler des vecteurs et des matrices, interpréter valeurs propres et SVD	1.1
Dériver, calculer un gradient, faire une descente de gradient	1.2, 1.3
Calculer avec des probabilités conditionnelles et appliquer Bayes	2.1
Choisir une loi et en calculer espérance et variance	2.2, 2.3
Expliquer la loi des grands nombres et le théorème central limite	2.4
Décrire un jeu de données (position, dispersion, forme) et le tracer	3.1
Estimer un paramètre, construire et interpréter un intervalle de confiance	3.2, 3.3
Mener un test, lire une p-valeur, corriger les tests multiples	3.4, 3.5
Écrire un programme Python avec fonctions, boucles, dictionnaires	4.1, 4.3
Manipuler un tableau avec pandas (filtrer, regrouper, joindre)	4.4
Produire un graphique honnête et lisible	4.5
Écrire des requêtes SQL avec jointures et agrégations	5.2
Utiliser fonctions fenêtres et CTE	5.3
Concevoir un schéma normalisé	5.1, 5.4
Versionner un projet avec Git	6.1
Utiliser notebooks, ligne de commande et environnements virtuels	6.2, 6.3
Mener un petit projet de bout en bout	Projet du volume

## Et maintenant ?

Le volume I vous a donné le **socle**. Il ne contient volontairement aucun modèle « à la mode » : ceux-ci demandent justement les bases que vous venez d'acquérir. Dans le **volume II : Modélisation statistique**, vous apprendrez à **modéliser** : la régression linéaire (la droite des moindres carrés de ce projet, généralisée à plusieurs variables), les modèles linéaires généralisés, les séries temporelles (la saisonnalité de Dar Jasmin, enfin traitée proprement), l'analyse de survie et la statistique bayésienne.

**Un conseil pour la suite.** Ne passez pas au volume II en vous reprochant de ne pas tout retenir du volume I : personne ne retient tout. Retenez **où chercher**. Gardez ce livre à portée de main, et revenez-y chaque fois qu'une notion (une p-valeur, une jointure, un gradient) revient dans un contexte nouveau. C'est ainsi, en revenant, que les fondations deviennent solides.

**À retenir, tout simplement.** La data science est un artisanat : des mathématiques simples, du code propre, beaucoup de bon sens, et la méthode. Vous avez désormais les gestes. Il ne reste qu'à les répéter.